

Principles of Soft Computing

(2nd Edition)

Dr. S. N. Sivanandam

Formerly Professor and Head

Department of Electrical and Electronics Engineering and

Department of Computer Science and Engineering,

PSG College of Technology,

Coimbatore

Dr. S. N. Deepa

Assistant Professor

Department of Electrical and Electronics Engineering,

Anna University of Technology, Coimbatore

Coimbatore

WILEY

About the Authors

Dr. S. N. Sivanandam completed his BE (Electrical and Electronics Engineering) in 1964 from Government College of Technology, Coimbatore, and MSc (Engineering) in Power System in 1966 from PSG College of Technology, Coimbatore. He acquired PhD in Control Systems in 1982 from Madras University. He received Best Teacher Award in the year 2001 and Dhakshina Murthy Award for Teaching Excellence from PSG College of Technology. He received The CITATION for best teaching and technical contribution in the Year 2002, Government College of Technology, Coimbatore. He has a total teaching experience (UG and PG) of 41 years. The total number of undergraduate and postgraduate projects guided by him for both Computer Science and Engineering and Electrical and Electronics Engineering is around 600. He has worked as Professor and Head Computer Science and Engineering Department, PSG College of Technology, Coimbatore. He has been identified as an outstanding person in the field of Computer Science and Engineering in MARQUIS 'Who's Who', October 2003 issue, USA. He has also been identified as an outstanding person in the field of Computational Science and Engineering in 'Who's Who', December 2005 issue, Saxe-Coburg Publications, UK. He has been placed as a VIP member in the continental WHO's WHO Registry of National Business Leaders, Inc., NY, August 24, 2006.

A widely published author, he has guided and cogenerated 30 PhD research works and at present 9 PhD research scholars are working under him. The total number of technical publications in International/National Journals/Conferences is around 700. He has also received Certificate of Merit 2005–2006 for his paper from The Institution of Engineers (India). He has chaired 7 International Conferences and 30 National Conferences. He is a member of various professional bodies like IE (India), ISTE, CSI, ACS and SSI. He is a technical advisor for various reputed industries and engineering institutions. His research areas include Modeling and Simulation, Neural Networks, Fuzzy Systems and Genetic Algorithm, Pattern Recognition, Multidimensional system analysis, Linear and Nonlinear control system, Signal and Image processing, Control System, Power system, Numerical methods, Parallel Computing, Data Mining and Database Security.

Dr. S. N. Deepa has completed her BE Degree from Government College of Technology, Coimbatore, 1999, ME Degree from PSG College of Technology, Coimbatore, 2004, and Ph.D. degree in Electrical Engineering from Anna University, Chennai, in the year 2008. She is currently Assistant Professor, Dept. of Electrical and Electronics Engineering, Anna University of Technology, Coimbatore. She was a gold medalist in her BE Degree Program. She has received G.D. Memorial Award in the year 1997 and Best Outgoing Student Award from PSG College of Technology, 2004. Her ME Thesis won National Award from the Indian Society of Technical Education and L&T, 2004. She has published 7 books and 32 papers in International and National Journals. Her research areas include Neural Network, Fuzzy Logic, Genetic Algorithm, Linear and Nonlinear Control Systems, Digital Control, Adaptive and Optimal Control.

Contents

Preface	v
About the Authors	viii
1. Introduction	1
Learning Objectives	1
1.1 Neural Networks	1
1.1.1 Artificial Neural Network: Definition	2
1.1.2 Advantages of Neural Networks	2
1.2 Application Scope of Neural Networks	3
1.3 Fuzzy Logic	5
1.4 Genetic Algorithm	6
1.5 Hybrid Systems	6
1.5.1 Neuro Fuzzy Hybrid Systems	6
1.5.2 Neuro Genetic Hybrid Systems	7
1.5.3 Fuzzy Genetic Hybrid Systems	7
1.6 Soft Computing	8
1.7 Summary	9
2. Artificial Neural Network: An Introduction	11
Learning Objectives	11
2.1 Fundamental Concept	11
2.1.1 Artificial Neural Network	11
2.1.2 Biological Neural Network	12
2.1.3 Brain vs. Computer – Comparison Between Biological Neuron and Artificial Neuron (Brain vs. Computer)	14
2.2 Evolution of Neural Networks	16
2.3 Basic Models of Artificial Neural Network	17
2.3.1 Connections	17
2.3.2 Learning	20
2.3.2.1 Supervised Learning	20
2.3.2.2 Unsupervised Learning	21
2.3.2.3 Reinforcement Learning	21
2.3.3 Activation Functions	22
2.4 Important Terminologies of ANNs	24
2.4.1 Weights	24
2.4.2 Bias	24
2.4.3 Threshold	26
2.4.4 Learning Rate	27
2.4.5 Momentum Factor	27
2.4.6 Vigilance Parameter	27
2.4.7 Notations	27
2.5 McCulloch–Pitts Neuron	27

2.5.1	Theory	27
2.5.2	Architecture	28
2.6	Linear Separability	29
2.7	Hebb Network	31
2.7.1	Theory	31
2.7.2	Flowchart of Training Algorithm	31
2.7.3	Training Algorithm	31
2.8	Summary	33
2.9	Solved Problems	33
2.10	Review Questions	46
2.11	Exercise Problems	46
2.12	Projects	47
3.	Supervised Learning Network	49
	Learning Objectives	49
3.1	Introduction	49
3.2	Perceptron Networks	49
3.2.1	Theory	49
3.2.2	Perceptron Learning Rule	51
3.2.3	Architecture	52
3.2.4	Flowchart for Training Process	52
3.2.5	Perceptron Training Algorithm for Single Output Classes	52
3.2.6	Perceptron Training Algorithm for Multiple Output Classes	54
3.2.7	Perceptron Network Testing Algorithm	55
3.3	Adaptive Linear Neuron (Adaline)	57
3.3.1	Theory	57
3.3.2	Delta Rule for Single Output Unit	57
3.3.3	Architecture	57
3.3.4	Flowchart for Training Process	57
3.3.5	Training Algorithm	58
3.3.6	Testing Algorithm	60
3.4	Multiple Adaptive Linear Neurons	60
3.4.1	Theory	60
3.4.2	Architecture	60
3.4.3	Flowchart of Training Process	60
3.4.4	Training Algorithm	61
3.5	Back-Propagation Network	64
3.5.1	Theory	64
3.5.2	Architecture	65
3.5.3	Flowchart for Training Process	66
3.5.4	Training Algorithm	66
3.5.5	Learning Factors of Back-Propagation Network	70
3.5.5.1	Initial Weights	70
3.5.5.2	Learning Rate α	71
3.5.5.3	Momentum Factor	71
3.5.5.4	Generalization	72

3.5.5.5	Number of Training Data	72
3.5.5.6	Number of Hidden Layer Nodes	72
3.5.6	Testing Algorithm of Back-Propagation Network	72
3.6	Radial Basis Function Network	73
3.6.1	Theory	73
3.6.2	Architecture	74
3.6.3	Flowchart for Training Process	74
3.6.4	Training Algorithm	74
3.7	Time Delay Neural Network	76
3.8	Functional Link Networks	77
3.9	Tree Neural Networks	78
3.10	Wavelet Neural Networks	79
3.11	Summary	80
3.12	Solved Problems	81
3.13	Review Questions	94
3.14	Exercise Problems	95
3.15	Projects	96
4.	Associative Memory Networks	97
	Learning Objectives	97
4.1	Introduction	97
4.2	Training Algorithms for Pattern Association	98
4.2.1	Hebb Rule	98
4.2.2	Outer Products Rule	100
4.3	Autoassociative Memory Network	101
4.3.1	Theory	101
4.3.2	Architecture	101
4.3.3	Flowchart for Training Process	101
4.3.4	Training Algorithm	103
4.3.5	Testing Algorithm	103
4.4	Heteroassociative Memory Network	104
4.4.1	Theory	104
4.4.2	Architecture	104
4.4.3	Testing Algorithm	104
4.5	Bidirectional Associative Memory (BAM)	105
4.5.1	Theory	105
4.5.2	Architecture	105
4.5.3	Discrete Bidirectional Associative Memory	105
4.5.3.1	Determination of Weights	106
4.5.3.2	Activation Functions for BAM	107
4.5.3.3	Testing Algorithm for Discrete BAM	107
4.5.4	Continuous BAM	108
4.5.5	Analysis of Hamming Distance, Energy Function and Storage Capacity	109
4.6	Hopfield Networks	110
4.6.1	Discrete Hopfield Network	110
4.6.1.1	Architecture of Discrete Hopfield Net	111

4.6.1.2	Training Algorithm of Discrete Hopfield Net	111
4.6.1.3	Testing Algorithm of Discrete Hopfield Net	112
4.6.1.4	Analysis of Energy Function and Storage Capacity on Discrete Hopfield Net	113
4.6.2	Continuous Hopfield Network	114
4.6.2.1	Hardware Model of Continuous Hopfield Network	114
4.6.2.2	Analysis of Energy Function of Continuous Hopfield Network	116
4.7	Iterative Autoassociative Memory Networks	118
4.7.1	Linear Autoassociative Memory (LAM)	118
4.7.2	Brain-in-the-Box Network	118
4.7.2.1	Training Algorithm for Brain-in-the-Box Model	119
4.7.3	Autoassociator with Threshold Unit	119
4.7.3.1	Testing Algorithm	120
4.8	Temporal Associative Memory Network	120
4.9	Summary	121
4.10	Solved Problems	121
4.11	Review Questions	143
4.12	Exercise Problems	144
4.13	Projects	146
5.	Unsupervised Learning Networks	147
	Learning Objectives	147
5.1	Introduction	147
5.2	Fixed Weight Competitive Nets	148
5.2.1	Maxnet	148
5.2.1.1	Architecture of Maxnet	148
5.2.1.2	Testing/Application Algorithm of Maxnet	149
5.2.2	Mexican Hat Net	150
5.2.2.1	Architecture	150
5.2.2.2	Flowchart	150
5.2.2.3	Algorithm	152
5.2.3	Hamming Network	153
5.2.3.1	Architecture	154
5.2.3.2	Testing Algorithm	154
5.3	Kohonen Self-Organizing Feature Maps	155
5.3.1	Theory	155
5.3.2	Architecture	156
5.3.3	Flowchart	158
5.3.4	Training Algorithm	160
5.3.5	Kohonen Self-Organizing Motor Map	160
5.4	Learning Vector Quantization	161
5.4.1	Theory	161
5.4.2	Architecture	161
5.4.3	Flowchart	162
5.4.4	Training Algorithm	162
5.4.5	Variants	164

5.4.5.1	LVQ 2	164
5.4.5.2	LVQ 2.1	165
5.4.5.3	LVQ 3	165
5.5	Counterpropagation Networks	165
5.5.1	Theory	165
5.5.2	Full Counterpropagation Net	166
5.5.2.1	Architecture	167
5.5.2.2	Flowchart	169
5.5.2.3	Training Algorithm	172
5.5.2.4	Testing (Application) Algorithm	173
5.5.3	Forward-Only Counterpropagation Net	174
5.5.3.1	Architecture	174
5.5.3.2	Flowchart	175
5.5.3.3	Training Algorithm	175
5.5.3.4	Testing Algorithm	178
5.6	Adaptive Resonance Theory Network	179
5.6.1	Theory	179
5.6.1.1	Fundamental Architecture	179
5.6.1.2	Fundamental Operating Principle	180
5.6.1.3	Fundamental Algorithm	181
5.6.2	Adaptive Resonance Theory 1	181
5.6.2.1	Architecture	182
5.6.2.2	Flowchart of Training Process	184
5.6.2.3	Training Algorithm	184
5.6.3	Adaptive Resonance Theory 2	188
5.6.3.1	Architecture	188
5.6.3.2	Algorithm	188
5.6.3.3	Flowchart	192
5.6.3.4	Training Algorithm	195
5.6.3.5	Sample Values of Parameter	196
5.7	Summary	197
5.8	Solved Problems	197
5.9	Review Questions	226
5.10	Exercise Problems	227
5.11	Projects	229
6.	Special Networks	231
	Learning Objectives	231
6.1	Introduction	231
6.2	Simulated Annealing Network	231
6.3	Boltzmann Machine	233
6.3.1	Architecture	234
6.3.2	Algorithm	234
6.3.2.1	Setting the Weights of the Network	234
6.3.2.2	Testing Algorithm	235
6.4	Gaussian Machine	236

6.5	Cauchy Machine	237
6.6	Probabilistic Neural Net	237
6.7	Cascade Correlation Network	238
6.8	Cognitron Network	240
6.9	Neocognitron Network	241
6.10	Cellular Neural Network	242
6.11	Logicon Projection Network Model	243
6.12	Spatio-Temporal Connectionist Neural Network	243
6.13	Optical Neural Networks	245
6.13.1	Electro-Optical Multipliers	245
6.13.2	Holographic Correlators	246
6.14	Neuroprocessor Chips	247
6.15	Summary	249
6.16	Review Questions	249
7.	Introduction to Fuzzy Logic, Classical Sets and Fuzzy Sets	251
	Learning Objectives	251
7.1	Introduction to Fuzzy Logic	251
7.2	Classical Sets (Crisp Sets)	255
7.2.1	Operations on Classical Sets	256
7.2.1.1	Union	257
7.2.1.2	Intersection	257
7.2.1.3	Complement	257
7.2.1.4	Difference (Subtraction)	258
7.2.2	Properties of Classical Sets	258
7.2.3	Function Mapping of Classical Sets	259
7.3	Fuzzy Sets	260
7.3.1	Fuzzy Set Operations	261
7.3.1.1	Union	261
7.3.1.2	Intersection	261
7.3.1.3	Complement	262
7.3.1.4	More operations on Fuzzy Sets	262
7.3.2	Properties of Fuzzy Sets	263
7.4	Summary	264
7.5	Solved Problems	264
7.6	Review Questions	270
7.7	Exercise Problems	271
8.	Classical Relations and Fuzzy Relations	273
	Learning Objectives	273
8.1	Introduction	273
8.2	Cartesian Product of Relation	273
8.3	Classical Relation	274
8.3.1	Cardinality of Classical Relation	276
8.3.2	Operations on Classical Relations	276
8.3.3	Properties of Crisp Relations	277
8.3.4	Composition of Classical Relations	277

8.4	Fuzzy Relations	279
8.4.1	Cardinality of Fuzzy Relations	281
8.4.2	Operations on Fuzzy Relations	281
8.4.3	Properties of Fuzzy Relations	282
8.4.4	Fuzzy Composition	282
8.5	Tolerance and Equivalence Relations	283
8.5.1	Classical Equivalence Relation	284
8.5.2	Classical Tolerance Relation	285
8.5.3	Fuzzy Equivalence Relation	285
8.5.4	Fuzzy Tolerance Relation	286
8.6	Noninteractive Fuzzy Sets	286
8.7	Summary	286
8.8	Solved Problems	286
8.9	Review Questions	292
8.10	Exercise Problems	293
9.	Membership Functions	295
	Learning Objectives	295
9.1	Introduction	295
9.2	Features of the Membership Functions	295
9.3	Fuzzification	298
9.4	Methods of Membership Value Assignments	298
9.4.1	Intuition	299
9.4.2	Inference	299
9.4.3	Rank Ordering	301
9.4.4	Angular Fuzzy Sets	301
9.4.5	Neural Networks	302
9.4.6	Genetic Algorithms	304
9.4.7	Induction Reasoning	304
9.5	Summary	305
9.6	Solved Problems	305
9.7	Review Questions	309
9.8	Exercise Problems	309
10.	Defuzzification	311
	Learning Objectives	311
10.1	Introduction	311
10.2	Lambda-Cuts for Fuzzy Sets (Alpha-Cuts)	311
10.3	Lambda-Cuts for Fuzzy Relations	313
10.4	Defuzzification Methods	313
10.4.1	Max-Membership Principle	315
10.4.2	Centroid Method	315
10.4.3	Weighted Average Method	316
10.4.4	Mean-Max Membership	317
10.4.5	Center of Sums	317
10.4.6	Center of Largest Area	317
10.4.7	First of Maxima (Last of Maxima)	318

10.5 Summary	320
10.6 Solved Problems	320
10.7 Review Questions	327
10.8 Exercise Problems	327
11. Fuzzy Arithmetic and Fuzzy Measures	329
Learning Objectives	329
11.1 Introduction	329
11.2 Fuzzy Arithmetic	329
11.2.1 Interval Analysis of Uncertain Values	329
11.2.2 Fuzzy Numbers	332
11.2.3 Fuzzy Ordering	333
11.2.4 Fuzzy Vectors	335
11.3 Extension Principle	336
11.4 Fuzzy Measures	337
11.4.1 Belief and Plausibility Measures	338
11.4.2 Probability Measures	340
11.4.3 Possibility and Necessity Measures	340
11.5 Measures of Fuzziness	342
11.6 Fuzzy Integrals	342
11.7 Summary	343
11.8 Solved Problems	343
11.9 Review Questions	345
11.10 Exercise Problems	346
12. Fuzzy Rule Base and Approximate Reasoning	347
Learning Objectives	347
12.1 Introduction	347
12.2 Truth Values and Tables in Fuzzy Logic	347
12.3 Fuzzy Propositions	348
12.4 Formation of Rules	349
12.5 Decomposition of Rules (Compound Rules)	350
12.6 Aggregation of Fuzzy Rules	352
12.7 Fuzzy Reasoning (Approximate Reasoning)	352
12.7.1 Categorical Reasoning	353
12.7.2 Qualitative Reasoning	354
12.7.3 Syllogistic Reasoning	354
12.7.4 Dispositional Reasoning	354
12.8 Fuzzy Inference Systems (FIS)	355
12.8.1 Construction and Working Principle of FIS	355
12.8.2 Methods of FIS	355
12.8.2.1 Mamdani FIS	356
12.8.2.2 Takagi–Sugeno Fuzzy Model (TS Method)	357
12.8.2.3 Comparison between Mamdani and Sugeno Method	358
12.9 Overview of Fuzzy Expert System	359
12.10 Summary	360
12.11 Review Questions	360

12.12 Exercise Problems	361
13. Fuzzy Decision Making	363
Learning Objectives	363
13.1 Introduction	363
13.2 Individual Decision Making	364
13.3 Multiperson Decision Making	364
13.4 Multiobjective Decision Making	365
13.5 Multiattribute Decision Making	366
13.6 Fuzzy Bayesian Decision Making	368
13.7 Summary	371
13.8 Review Questions	371
13.9 Exercise Problems	371
14. Fuzzy Logic Control Systems	373
Learning Objectives	373
14.1 Introduction	373
14.2 Control System Design	374
14.3 Architecture and Operation of FLC System	375
14.4 FLC System Models	377
14.5 Application of FLC Systems	377
14.6 Summary	383
14.7 Review Questions	383
14.8 Exercise Problems	383
15. Genetic Algorithm	385
Learning Objectives	385
15.1 Introduction	385
15.1.1 What are Genetic Algorithms?	386
15.1.2 Why Genetic Algorithms?	386
15.2 Biological Background	386
15.2.1 The Cell	386
15.2.2 Chromosomes	386
15.2.3 Genetics	387
15.2.4 Reproduction	388
15.2.5 Natural Selection	390
15.3 Traditional Optimization and Search Techniques	390
15.3.1 Gradient-Based Local Optimization Method	390
15.3.2 Random Search	392
15.3.3 Stochastic Hill Climbing	392
15.3.4 Simulated Annealing	392
15.3.5 Symbolic Artificial Intelligence	393
15.4 Genetic Algorithm and Search Space	394
15.4.1 Search Space	394
15.4.2 Genetic Algorithms World	395
15.4.3 Evolution and Optimization	395
15.4.4 Evolution and Genetic Algorithms	396

15.5 Genetic Algorithm vs. Traditional Algorithms	397
15.6 Basic Terminologies in Genetic Algorithm	398
15.6.1 Individuals	398
15.6.2 Genes	399
15.6.3 Fitness	399
15.6.4 Populations	400
15.7 Simple GA	401
15.8 General Genetic Algorithm	402
15.9 Operators in Genetic Algorithm	404
15.9.1 Encoding	405
15.9.1.1 Binary Encoding	405
15.9.1.2 Octal Encoding	405
15.9.1.3 Hexadecimal Encoding	406
15.9.1.4 Permutation Encoding (Real Number Coding)	406
15.9.1.5 Value Encoding	406
15.9.1.6 Tree Encoding	407
15.9.2 Selection	407
15.9.2.1 Roulette Wheel Selection	408
15.9.2.2 Random Selection	408
15.9.2.3 Rank Selection	408
15.9.2.4 Tournament Selection	409
15.9.2.5 Boltzmann Selection	409
15.9.2.6 Stochastic Universal Sampling	410
15.9.3 Crossover (Recombination)	410
15.9.3.1 Single-Point Crossover	411
15.9.3.2 Two-Point Crossover	411
15.9.3.3 Multipoint Crossover (<i>N</i> -Point Crossover)	412
15.9.3.4 Uniform Crossover	412
15.9.3.5 Three-Parent Crossover	412
15.9.3.6 Crossover with Reduced Surrogate	413
15.9.3.7 Shuffle Crossover	413
15.9.3.8 Precedence Preservative Crossover	413
15.9.3.9 Ordered Crossover	413
15.9.3.10 Partially Matched Crossover	414
15.9.3.11 Crossover Probability	415
15.9.4 Mutation	415
15.9.4.1 Flipping	415
15.9.4.2 Interchanging	415
15.9.4.3 Reversing	416
15.9.4.4 Mutation Probability	416
15.10 Stopping Condition for Genetic Algorithm Flow	416
15.10.1 Best Individual	417
15.10.2 Worst individual	417
15.10.3 Sum of Fitness	417
15.10.4 Median Fitness	417
15.11 Constraints in Genetic Algorithm	417

15.12 Problem Solving Using Genetic Algorithm	418
15.12.1 Maximizing a Function	418
15.13 The Schema Theorem	422
15.13.1 The Optimal Allocation of Trials	424
15.13.2 Implicit Parallelism	425
15.14 Classification of Genetic Algorithm	426
15.14.1 Messy Genetic Algorithms	426
15.14.2 Adaptive Genetic Algorithms	427
15.14.2.1 Adaptive Probabilities of Crossover and Mutation	427
15.14.2.2 Design of Adaptive p_c and p_m	428
15.14.2.3 Practical Considerations and Choice of Values for k_1 , k_2 , k_3 and k_4	429
15.14.3 Hybrid Genetic Algorithms	430
15.14.4 Parallel Genetic Algorithm	432
15.14.4.1 Global Parallelization	433
15.14.4.2 Classification of Parallel GAs	434
15.14.4.3 Coarse-Grained PGAs – The Island Model	440
15.14.5 Independent Sampling Genetic Algorithm (ISGA)	441
15.14.5.1 Comparison of ISGA with PGA	442
15.14.5.2 Components of ISGAs	442
15.14.6 Real-Coded Genetic Algorithms	444
15.14.6.1 Crossover Operators for Real-Coded GAs	444
15.14.6.2 Mutation Operators for Real-Coded GAs	445
15.15 Holland Classifier Systems	445
15.15.1 The Production System	445
15.15.2 The Bucket Brigade Algorithm	446
15.15.3 Rule Generation	448
15.16 Genetic Programming	449
15.16.1 Working of Genetic Programming	450
15.16.2 Characteristics of Genetic Programming	453
15.16.2.1 Human-Competitive	453
15.16.2.2 High-Return	453
15.16.2.3 Routine	454
15.16.2.4 Machine Intelligence	454
15.16.3 Data Representation	455
15.16.3.1 Crossing Programs	458
15.16.3.2 Mutating Programs	460
15.16.3.3 The Fitness Function	460
15.17 Advantages and Limitations of Genetic Algorithm	461
15.18 Applications of Genetic Algorithm	462
15.19 Summary	463
15.20 Review Questions	464
15.21 Exercise Problems	464
16. Hybrid Soft Computing Techniques	465
Learning Objectives	465
16.1 Introduction	465

16.2	Neuro-Fuzzy Hybrid Systems	466
16.2.1	Comparison of Fuzzy Systems with Neural Networks	466
16.2.2	Characteristics of Neuro-Fuzzy Hybrids	467
16.2.3	Classifications of Neuro-Fuzzy Hybrid Systems	468
16.2.3.1	Cooperative Neural Fuzzy Systems	468
16.2.3.2	General Neuro-Fuzzy Hybrid Systems (General NFHS)	468
16.2.4	Adaptive Neuro-Fuzzy Inference System (ANFIS) in MATLAB	470
16.2.4.1	FIS Structure and Parameter Adjustment	470
16.2.4.2	Constraints of ANFIS	471
16.2.4.3	The ANFIS Editor GUI	471
16.2.4.4	Data Formalities and the ANFIS Editor GUI	472
16.2.4.5	More on ANFIS Editor GUI	472
16.3	Genetic Neuro-Hybrid Systems	476
16.3.1	Properties of Genetic Neuro-Hybrid Systems	476
16.3.2	Genetic Algorithm Based Back-Propagation Network (BPN)	476
16.3.2.1	Coding	477
16.3.2.2	Weight Extraction	477
16.3.2.3	Fitness Function	478
16.3.2.4	Reproduction of Offspring	479
16.3.2.5	Convergence	479
16.3.3	Advantages of Neuro-Genetic Hybrids	479
16.4	Genetic Fuzzy Hybrid and Fuzzy Genetic Hybrid Systems	479
16.4.1	Genetic Fuzzy Rule Based Systems (GFRBSs)	480
16.4.1.1	Genetic Tuning Process	481
16.4.1.2	Genetic Learning of Rule Bases	482
16.4.1.3	Genetic Learning of Knowledge Base	483
16.4.2	Advantages of Genetic Fuzzy Hybrids	483
16.5	Simplified Fuzzy ARTMAP	483
16.5.1	Supervised ARTMAP System	484
16.5.2	Comparison of ARTMAP with BPN	484
16.6	Summary	485
16.7	Solved Problems using MATLAB	485
16.8	Review Questions	509
16.9	Exercise Problems	509
17.	Applications of Soft Computing	511
	Learning Objectives	511
17.1	Introduction	511
17.2	A Fusion Approach of Multispectral Images with SAR (Synthetic Aperture Radar)	
	Image for Flood Area Analysis	511
17.2.1	Image Fusion	513
17.2.2	Neural Network Classification	513
17.2.3	Methodology and Results	514
17.2.3.1	Method	514
17.2.3.2	Results	514
17.3	Optimization of Traveling Salesman Problem using Genetic Algorithm Approach	515

17.3.1	Genetic Algorithms	516
17.3.2	Schemata	517
17.3.3	Problem Representation	517
17.3.4	Reproductive Algorithms	517
17.3.5	Mutation Methods	518
17.3.6	Results	518
17.4	Genetic Algorithm-Based Internet Search Technique	519
17.4.1	Genetic Algorithms and Internet	521
17.4.2	First Issue: Representation of Genomes	521
17.4.2.1	String Representation	521
17.4.2.2	Array of String Representation	522
17.4.2.3	Numerical Representation	522
17.4.3	Second Issue: Definition of the Crossover Operator	523
17.4.3.1	Classical Crossover	523
17.4.3.2	Parent Crossover	523
17.4.3.3	Link Crossover	523
17.4.3.4	Overlapping Links	523
17.4.3.5	Link Pre-Evaluation	524
17.4.4	Third Issue: Selection of the Degree of Crossover	524
17.4.4.1	Limited Crossover	524
17.4.4.2	Unlimited Crossover	524
17.4.5	Fourth Issue: Definition of the Mutation Operator	525
17.4.5.1	Generational Mutation	526
17.4.5.2	Selective Mutation	526
17.4.6	Fifth Issue: Definition of the Fitness Function	527
17.4.6.1	Simple Keyword Evaluation	527
17.4.6.2	Jaccard's Score	527
17.4.6.3	Link Evaluation	529
17.4.7	Sixth Issue: Generation of the Output Set	529
17.4.7.1	Interactive Generation	529
17.4.7.2	Post-Generation	529
17.5	Soft Computing Based Hybrid Fuzzy Controllers	529
17.5.1	Neuro-Fuzzy System	530
17.5.2	Real-Time Adaptive Control of a Direct Drive Motor	530
17.5.3	GA-Fuzzy Systems for Control of Flexible Robots	531
17.5.3.1	Application to Flexible Robot Control	532
17.5.4	GP-Fuzzy Hierarchical Behavior Control	533
17.5.5	GP-Fuzzy Approach	533
17.6	Soft Computing Based Rocker Engine Control	534
17.6.1	Bayesian Belief Networks	535
17.6.2	Fuzzy Logic Control	536
17.6.3	Software Engineering in Marshall's Flight Software Group	537
17.6.4	Experimental Apparatus and Facility Turbine Technologies SR-30 Engine	537
17.6.5	System Modifications	538
17.6.6	Fuel-Flow Rate Measurement System	538
17.6.7	Exit Conditions Monitoring	538

17.7	Summary	539
17.8	Review Questions	539
17.9	Exercise Problems	539
18.	Soft Computing Techniques Using C and C++	541
	Learning Objectives	541
18.1	Introduction	541
18.2	Neural Network Implementation	541
18.2.1	Perceptron Network	541
18.2.2	Adaline Network	543
18.2.3	Madaline Network for XOR Function	545
18.2.4	Back Propagation Network for XOR Function using Bipolar Inputs and Binary Targets	548
18.2.5	Kohonen Self-Organizing Feature Map	551
18.2.6	ART 1 Network with Nine Input Units and Two Cluster Units	552
18.2.7	ART 1 Network to Cluster Four Vectors	554
18.2.8	Full Counterpropagation Network	556
18.3	Fuzzy Logic Implementation	559
18.3.1	Implement the Various Primitive Operations of Classical Sets	559
18.3.2	To Verify Various Laws Associated with Classical Sets	561
18.3.3	To Perform Various Primitive Operations on Fuzzy Sets with Dynamic Components	566
18.3.4	To Verify the Various Laws Associated with Fuzzy Set	570
18.3.5	To Perform Cartesian Product Over Two Given Fuzzy Sets	574
18.3.6	To Perform Max-Min Composition of Two Matrices Obtained from Cartesian Product	575
18.3.7	To Perform Max-Product Composition of Two Matrices Obtained from Cartesian Product	579
18.4	Genetic Algorithm Implementation	582
18.4.1	To Maximize $F(x_1, x_2) = 4x_1 + 3x_2$	583
18.4.2	To Minimize a Function $F(x) = x^2$	587
18.4.3	Traveling Salesman Problem (TSP)	593
18.4.4	Prisoner's Dilemma	598
18.4.5	Quadratic Equation Solving	602
18.5	Summary	606
18.6	Exercise Problems	606
19.	MATLAB Environment for Soft Computing Techniques	607
	Learning Objectives	607
19.1	Introduction	607
19.2	Getting Started with MATLAB	608
19.2.1	Matrices and Vectors	608
19.3	Introduction to Simulink	610
19.4	MATLAB Neural Network Toolbox	612
19.4.1	Creating a Custom Neural Network	612
19.4.2	Commands in Neural Network Toolbox	614
19.4.3	Neural Network Graphical User Interface Toolbox	619

Contents

xxlii

19.5	Fuzzy Logic MATLAB Toolbox	624
19.5.1	Commands in Fuzzy Logic Toolbox	625
19.5.2	Simulink Blocks in Fuzzy Logic Toolbox	626
19.5.3	Fuzzy Logic GUI Toolbox	628
19.6	Genetic Algorithm MATLAB Toolbox	631
19.6.1	MATLAB Genetic Algorithm Commands	632
19.6.2	Genetic Algorithm Graphical User Interface	635
19.7	Neural Network MATLAB Source Codes	639
19.8	Fuzzy Logic MATLAB Source Codes	669
19.9	Genetic Algorithm MATLAB Source Codes	681
19.10	Summary	690
19.11	Exercise Problems	690
	Bibliography	691
	Sample Question Paper 1	707
	Sample Question Paper 2	709
	Sample Question Paper 3	712
	Sample Question Paper 4	714
	Sample Question Paper 5	716
	Index	718

Introduction

Learning Objectives

- Scope of soft computing.
- Various components under soft computing.
- Description on artificial neural networks with its advantages and applications.
- An overview of fuzzy logic.
- A note on genetic algorithm.
- The theory of hybrid systems.

1.1 Neural Networks

A neural network is a processing device, either an algorithm or an actual hardware, whose design was inspired by the design and functioning of animal brains and components thereof. The computing world has a lot to gain from neural networks, also known as artificial neural networks or neural net. The neural networks have the ability to learn by example, which makes them very flexible and powerful. For neural networks, there is no need to devise an algorithm to perform a specific task; that is, there is no need to understand the internal mechanisms of that task. These networks are also well suited for real-time systems because of their fast response and computational times which are because of their parallel architecture.

Before discussing artificial neural networks, let us understand how the human brain works. The human brain is an amazing processor. Its exact workings are still a mystery. The most basic element of the human brain is a specific type of cell, known as neuron, which doesn't regenerate. Because neurons aren't slowly replaced, it is assumed that they provide us with our abilities to remember, think and apply previous experiences to our every action. The human brain comprises about 100 billion neurons. Each neuron can connect with up to 200,000 other neurons, although 1,000–10,000 interconnections are typical.

The power of the human mind comes from the sheer numbers of neurons and their multiple interconnections. It also comes from genetic programming and learning. There are over 100 different classes of neurons. The individual neurons are complicated. They have a myriad of parts, subsystems and control mechanisms. They convey information via a host of electrochemical pathways. Together these neurons and their connections form a process which is not binary, not stable, and not synchronous. In short, it is nothing like the currently available electronic computers, or even artificial neural networks.

1.1.1 Artificial Neural Network: Definition

An artificial neural network (ANN) may be defined as an information-processing model that is inspired by the way biological nervous systems, such as the brain, process information. This model tries to replicate only the most basic functions of the brain. The key element of ANN is the novel structure of its information processing system. An ANN is composed of a large number of highly interconnected processing elements (neurons) working in unison to solve specific problems.

Artificial neural networks, like people, learn by example. An ANN is configured for a specific application, such as pattern recognition or data classification through a learning process. In biological systems, learning involves adjustments to the synaptic connections that exist between the neurons. ANNs undergo a similar change that occurs when the concept on which they are built leaves the academic environment and is thrown into the harsher world of users who simply want to get a job done on computers accurately all the time. Many neural networks now being designed are statistically quite accurate, but they still leave their users with a bad taste as they falter when it comes to solving problems accurately. They might be 85–90% accurate. Unfortunately, few applications tolerate that level of error.

1.1.2 Advantages of Neural Networks

Neural networks, with their remarkable ability to derive meaning from complicated or imprecise data, could be used to extract patterns and detect trends that are too complex to be noticed by either humans or other computer techniques. A trained neural network could be thought of as an “expert” in a particular category of information it has been given to analyze. This expert could be used to provide projections in new situations of interest and answer “what if” questions. Other advantages of working with an ANN include:

1. **Adaptive learning:** An ANN is endowed with the ability to learn how to do tasks based on the data given for training or initial experience.
2. **Self-organization:** An ANN can create its own organization or representation of the information it receives during learning time.
3. **Real-time operation:** ANN computations may be carried out in parallel. Special hardware devices are being designed and manufactured to take advantage of this capability of ANNs.
4. **Fault tolerance via redundant information coding:** Partial destruction of a neural network leads to the corresponding degradation of performance. However, some network capabilities may be retained even after major network damage.

Currently, neural networks can't function as a user interface which translates spoken words into instructions for a machine, but someday they would have this skill. Then VCRs, home security systems, CD players, and word processors would simply be activated by voice. Touch screen and voice editing would replace the word processors of today. Besides, spreadsheets and databases would be imparted such level of usability that would be pleasing to everyone. But for now, neural networks are only entering the marketplace in niche areas where their statistical accuracy is valuable.

Many of these niches indeed involve applications where answers provided by the software programs are not accurate but vague. Loan approval is one such area. Financial institutions make more money if they succeed in having the lowest bad loan rate. For these institutions, installing systems that are “90% accurate” in selecting the genuine loan applicants might be an improvement over their current selection process. Indeed, some banks have proved that the failure rate on loans approved by neural networks is lower than those approved by their

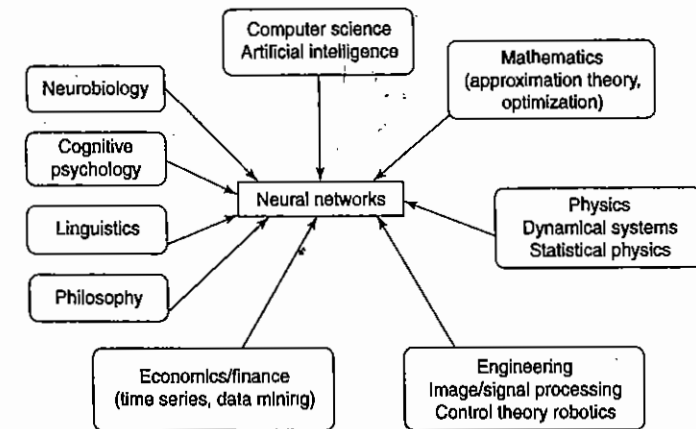


Figure 1-1 The multi-disciplinary point of view of neural networks.

best traditional methods. Also, some credit card companies are using neural networks in their application screening process.

This newest method of looking into the future by analyzing past experiences has generated its own unique set of problems. One such problem is to provide a reason behind a computer-generated answer, say, as to why a particular loan application was denied. To explain how a network learned and why it recommends a particular decision has been difficult. The inner workings of neural networks are “black boxes.” Some people have even called the use of neural networks “voodoo engineering.” To justify the decision-making process, several neural network tool makers have provided programs that explain which input through which node dominates the decision-making process. From this information, experts in the application may be able to infer which data plays a major role in decision-making and its importance.

Apart from filling the niche areas, neural network's work is also progressing in other more promising application areas. The next section of this chapter goes through some of these areas and briefly details the current work. The objective is to make the reader aware of various possibilities where neural networks might offer solutions, such as language processing, character recognition, image compression, pattern recognition, etc.

Neural networks can be viewed from a multi-disciplinary point of view as shown in Figure 1-1.

1.2 Application Scope of Neural Networks

The neural networks have good scope of being used in the following areas:

1. **Air traffic control** could be automated with the location, altitude, direction and speed of each radar blip taken as input to the network. The output would be the air traffic controller's instruction in response to each blip.
2. **Animal behavior, predator/prey relationships and population cycles** may be suitable for analysis by neural networks.
3. **Appraisal and valuation** of property, buildings, automobiles, machinery, etc. should be an easy task for a neural network.

4. *Betting* on horse races, stock markets, sporting events, etc. could be based on neural network predictions.
5. *Criminal sentencing* could be predicted using a large sample of crime details as input and the resulting sentences as output.
6. *Complex physical and chemical processes* that may involve the interaction of numerous (possibly unknown) mathematical formulas could be modeled heuristically using a neural network.
7. *Data mining, cleaning and validation* could be achieved by determining which records suspiciously diverge from the pattern of their peers.
8. *Direct mail advertisers* could use neural network analysis of their databases to decide which customers should be targeted, and avoid wasting money on unlikely targets.
9. *Echo patterns* from sonar, radar, seismic and magnetic instruments could be used to predict their targets.
10. *Econometric modeling* based on neural networks should be more realistic than older models based on classical statistics.
11. *Employee hiring* could be optimized if the neural networks were able to predict which job applicant would show the best job performance.
12. *Expert consultants* could package their intuitive expertise into a neural network to automate their services.
13. *Fraud detection* regarding credit cards, insurance or taxes could be automated using a neural network analysis of past incidents.
14. *Handwriting and typewriting* could be recognized by imposing a grid over the writing, then each square of the grid becomes an input to the neural network. This is called "Optical Character Recognition."
15. *Lake water levels* could be predicted based upon precipitation patterns and river/dam flows.
16. *Machinery control* could be automated by capturing the actions of experienced machine operators into a neural network.
17. *Medical diagnosis* is an ideal application for neural networks.
18. *Medical research* relies heavily on classical statistics to analyze research data. Perhaps a neural network should be included in the researcher's tool kit.
19. *Music composition* has been tried using neural networks. The network is trained to recognize patterns in the pitch and tempo of certain music, and then the network writes its own music.
20. *Photos and fingerprints* could be recognized by imposing a fine grid over the photo. Each square of the grid becomes an input to the neural network.
21. *Recipes and chemical formulations* could be optimized based on the predicted outcome of a formula change.
22. *Retail inventories* could be optimized by predicting demand based on past patterns.
23. *River water levels* could be predicted based on upstream reports, and time and location of each report.
24. *Scheduling of buses, airplanes and elevators* could be optimized by predicting demand.
25. *Staff scheduling* requirements for restaurants, retail stores, police stations, banks, etc., could be predicted based on the customer flow, day of week, paydays, holidays, weather, season, etc.
26. *Strategies* for games, business and war can be captured by analyzing the expert player's response to given stimuli. For example, a football coach must decide whether to kick, pass or run on the last down. The inputs for this decision include score, time, field location, yards to first down, etc.

27. *Traffic flows* could be predicted so that signal timing could be optimized. The neural network could recognize "a weekday morning rush hour during a school holiday" or "a typical winter Sunday morning."
28. *Voice recognition* could be obtained by analyzing the audio oscilloscope pattern, much like a stock market graph.
29. *Weather prediction* may be possible. Inputs would include weather reports from surrounding areas. Output(s) would be the future weather in specific areas based on the input information. Effects such as ocean currents and jet streams could be included.

Today, ANN represents a major extension to computation. Different types of neural networks are available for various applications. They perform operations akin to the human brain though to a limited extent. A rapid increase is expected in our understanding of the ANNs leading to the improved network paradigms and a host of application opportunities.

1.3 Fuzzy Logic

The concept of fuzzy logic (FL) was conceived by Lotfi Zadeh, a Professor at the University of California at Berkeley. An organized method for dealing with imprecise data is called fuzzy logic. The data are considered as fuzzy sets.

Professor Zadeh presented FL not as a control methodology but as a way of processing data by allowing partial set membership rather than crisp set membership or nonmembership. This approach to set theory was not applied to control systems until the 1970s due to insufficient computer capability. Also, earlier the systems were designed only to accept precise and accurate data. However, in certain systems it is not possible to get the accurate data. Therefore, Professor Zadeh reasoned that for processing need not always require precise and numerical information input; processing can be performed even with imprecise inputs. Suitable feedback controllers may be designed to accept noisy, imprecise input, and they would be much more effective and perhaps easier to implement. The processing with imprecise inputs led to the growth of Zadeh's FL. Unfortunately, US manufacturers have not been so quick to embrace this technology while the Europeans and Japanese have been aggressively building real products around it.

Fuzzy logic is a superset of conventional (or Boolean) logic and contains similarities and differences with Boolean logic. FL is similar to Boolean logic in that Boolean logic results are returned by FL operations when all fuzzy memberships are restricted to 0 and 1. FL differs from Boolean logic in that it is permissive of natural language queries and is more like human thinking; it is based on degrees of truth. For example, traditional sets include or do not include an individual element; there is no other case than true or false. However, fuzzy sets allow partial membership. FL is basically a multivalued logic that allows intermediate values to be defined between conventional evaluations such as *yes/no*, *true/false*, *black/white*, etc. Notions like rather warm or pretty cold can be formulated mathematically and processed with the computer. In this way, an attempt is made to apply a more human-like way of thinking in the programming of computers.

Fuzzy logic is a problem-solving control system methodology that lends itself to implementation in systems ranging from simple, small, embedded microcontrollers to large, networked, multichannel PC or workstation-based data acquisition and control systems. It can be implemented in hardware, software or a combination of both. FL provides a simple way to arrive at a definite conclusion based upon vague, ambiguous, imprecise, noisy, or missing input information. FL's approach to control problems mimics how a person would make decisions, only much faster.

1.4 Genetic Algorithm

Genetic algorithm (GA) is reminiscent of sexual reproduction in which the genes of two parents combine to form those of their children. When it is applied to problem solving, the basic premise is that we can create an initial population of individuals representing possible solutions to a problem we are trying to solve. Each of these individuals has certain characteristics that make them more or less fit as members of the population. The more fit members will have a higher probability of mating and producing offspring that have a significant chance of retaining the desirable characteristics of their parents than the less fit members. This method is very effective at finding optimal or near-optimal solutions to a wide variety of problems because it does not impose many limitations required by traditional methods. It is an elegant generate-and-test strategy that can identify and exploit regularities in the environment, and results in solutions that are globally optimal or nearly so.

Genetic algorithms are adaptive computational procedures modeled on the mechanics of natural genetic systems. They express their ability by efficiently exploiting the historical information to speculate on new offspring with expected improved performance. GAs are executed iteratively on a set of coded solutions, called population, with three basic operators: selection/reproduction, crossover and mutation. They use only the payoff (objective function) information and probabilistic transition rules for moving to the next iteration. They are different from most of the normal optimization and search procedures in the following four ways:

1. GAs work with the coding of the parameter set, not with the parameter themselves;
2. GAs work simultaneously with multiple points, not a single point;
3. GAs search via sampling (a blind search) using only the payoff information;
4. GAs search using stochastic operators, not deterministic rules.

Since a GA works simultaneously on a set of coded solutions, it has very little chance to get stuck at local optima when used as optimization technique. Again, it does not need any sort of auxiliary information, like derivative of the optimizing function. Moreover, the resolution of the possible search space is increased by operating on coded (possible) solutions and not on the solutions themselves. Further, this search space need not be continuous. Recently, GAs are finding widespread applications in solving problems requiring efficient and effective search, in business, scientific and engineering circles like synthesis of neural network architectures, traveling salesman problem, graph coloring, scheduling, numerical optimization, and pattern recognition and image processing.

1.5 Hybrid Systems

Hybrid systems can be classified into three different systems: Neuro fuzzy hybrid system; neuron genetic hybrid system; fuzzy genetic hybrid systems. These are discussed in detail in the following sections.

1.5.1 Neuro Fuzzy Hybrid Systems

A neuro fuzzy hybrid system is a fuzzy system that uses a learning algorithm derived from or inspired by neural network theory to determine its parameters (fuzzy sets and fuzzy rules) by processing data samples.

In other words, a neuro fuzzy hybrid system refers to the combination of fuzzy set theory and neural networks having advantages of both which are listed below.

1. It can handle any kind of information (numeric, linguistic, logical, etc.).

2. It can manage imprecise, partial, vague or imperfect information.
3. It can resolve conflicts by collaboration and aggregation.
4. It has self-learning, self-organizing and self-tuning capabilities.
5. It doesn't need prior knowledge of relationships of data.
6. It can mimic human decision-making process.
7. It makes computation fast by using fuzzy number operations.

Neuro fuzzy hybrid systems combine the advantages of fuzzy systems, which deal with explicit knowledge that can be explained and understood, and neural networks, which deal with implicit knowledge that can be acquired by learning. Neural network learning provides a good way to adjust the knowledge of the expert (i.e., artificial intelligence system) and automatically generate additional fuzzy rules and membership functions to meet certain specifications. It helps reduce design time and costs. On the other hand, FL enhances the generalization capability of a neural network system by providing more reliable output when extrapolation is needed beyond the limits of the training data.

1.5.2 Neuro Genetic Hybrid Systems

Genetic algorithms (GAs) have been increasingly applied in ANN design in several ways: topology optimization, genetic training algorithms and control parameter optimization. In topology optimization, GA is used to select a topology (number of hidden layers, number of hidden nodes, interconnection pattern) for the ANN which in turn is trained using some training scheme, most commonly back propagation. In genetic training algorithms, the learning of an ANN is formulated as a weight optimization problem, usually using the inverse mean squared error as a fitness measure. Many of the control parameters such as learning rate, momentum rate, tolerance level, etc., can also be optimized using GAs. In addition, GAs have been used in many other innovative ways, to create new indicators based on existing ones, select good indicators, evolve optimal trading systems and complement other techniques such as fuzzy logic.

1.5.3 Fuzzy Genetic Hybrid Systems

The optimization abilities of GAs are used to develop the best set of rules to be used by a fuzzy inference engine, and to optimize the choice of membership functions. A particular use of GAs is in fuzzy classification systems, where an object is classified on the basis of the linguistic values of the object attributes. The most difficult part of building a system like this is to find the appropriate set of fuzzy rules. The most obvious approach is to obtain knowledge from experts and translate this into a set of fuzzy rules. But this approach is time consuming. Besides, experts may not be able to put their knowledge into an appropriate form of words. A second approach is to obtain the fuzzy rules through machine learning, whereby the knowledge is automatically extracted or deduced from sample cases. A fuzzy GA is a directed random search over all (discrete) fuzzy subsets of an interval and has features which make it applicable for solving this problem. It is capable of creating the classification rules for a fuzzy system where objects are classified by linguistic terms. Coding the rules genetically enables the system to deal with multivalued FL and is more efficient as it is consistent with numeric coding of fuzzy examples. The training data and randomly generated rules are combined to create the initial population, giving a better starting point for reproduction. Finally, a fitness function measures the strength of the rules, balancing the quality and diversity of the population.

1.6 Soft Computing

The two major problem-solving technologies include:

1. hard computing;
2. soft computing.

Hard computing deals with precise models where accurate solutions are achieved quickly. On the other hand, soft computing deals with approximate models and gives solution to complex problems. The two problem-solving technologies are shown in Figure 1-2.

Soft computing is a relatively new concept, the term really entering general circulation in 1994. The term "soft computing" was introduced by Professor Lotfi Zadeh with the objective of exploiting the tolerance for imprecision, uncertainty and partial truth to achieve tractability, robustness, low solution cost and better rapport with reality. The ultimate goal is to be able to emulate the human mind as closely as possible. Soft computing involves partnership of several fields, the most important being neural networks, GA, and FL. Also included is the field of probabilistic reasoning, employed for its uncertainty control techniques. However, this field is not examined here.

Soft computing uses a combination of GAs, neural networks and FL. A hybrid technique, in fact, would inherit all the advantages, but won't have the less desirable features of single soft computing components. It has to possess a good learning capacity, a better learning time than that of pure GAs and less sensitivity to the problem of local extremes than neural networks. In addition, it has to generate a fuzzy knowledge base, which has a linguistic representation and a very low degree of computational complexity.

An important thing about the constituents of soft computing is that they are complementary, not competitive, offering their own advantages and techniques to partnerships to allow solutions to otherwise unsolvable problems. The constituents of soft computing are examined in turn, following which existing applications of partnerships are described.

"Negotiation is the communication process of a group of agents in order to reach a mutually accepted agreement on some matter." This definition is typical of the research being done into negotiation and co-ordination in relation to software agents. It is an obvious necessity that when multiple agents interact, they will be required to co-ordinate their efforts and attempt to sort out any conflicts of resources or interest.

It is important to appreciate that agents are owned and controlled by people in order to complete tasks on their behalf. An example of a possible multiple-agent-based negotiation scenario is the competition between

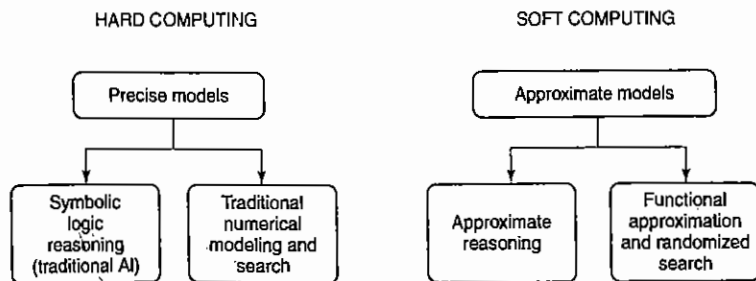


Figure 1-2 Problem-solving technologies.

long-distance phone call providers. When the consumer picks up the phone and dials, an agent will communicate on the consumer's behalf with all the available network providers. Each provider will make an offer that the consumer agent can accept or reject. A realistic goal would be to select the lowest available price for the call. However, given the first round of offers, network providers may wish to modify their offer to make it more competitive. The new offer is then submitted to the consumer agent and the process continues until a conclusion is reached. One advantage of this process is that the provider can dynamically alter its pricing strategy to account for changes in demand and competition, therefore maximizing revenue. The consumer will obviously benefit from the constant competition between providers. Best of all, the process is entirely autonomous as the agents embody and act on the beliefs and constraints of the parties they represent. Further changes can be made to the protocol so that providers can bid low without being in danger of making a loss. For example, if the consumer chooses to go with the lowest bid but pays the second lowest price, this will take away the incentive to underbid or overbid.

Much of the negotiation theory is based around human behavior models and, as a result, it is often translated using Distributed Artificial Intelligence techniques. The problems associated with machine negotiation are as difficult to solve as they are with human negotiation and involve issues such as privacy, security and deception.

1.7 Summary

The computing world has a lot to gain from neural networks whose ability to learn by example makes them very flexible and powerful. In case of neural networks, there is no need to devise an algorithm to perform a specific task, i.e., there is no need to understand the internal mechanisms of that task. Neural networks are also well suited for real-time systems because of their fast response and computational times, which are due to their parallel architecture.

Neural networks also contribute to other areas of research such as neurology and psychology. They are regularly used to model parts of living organisms and to investigate the internal mechanisms of the brain. Perhaps the most exciting aspect of neural networks is the possibility that someday "conscious" networks might be produced. Today, many scientists believe that consciousness is a "mechanical" property and that "conscious" neural networks are a realistic possibility.

Fuzzy logic was conceived as a better method for sorting and handling data but has proven to be an excellent choice for many control system applications since it mimics human control logic. It can be built into anything from small, hand-held products to large, computerized process control systems. It uses an imprecise but very descriptive language to deal with input data more like a human operator. It is robust and often works when first implemented with little or no tuning.

When applied to optimize ANNs for forecasting and classification problems, GAs can be used to search for the right combination of input data, the most suitable forecast horizon, the optimal or near-optimal network interconnection patterns and weights among the neurons, and the control parameters (learning rate, momentum rate, tolerance level, etc.) based on the training data used and the pre-set criteria. Like ANNs, GAs do not always guarantee you a perfect solution, but in many cases, you can arrive at an acceptable solution without the time and expense of an exhaustive search.

Soft computing is a relatively new concept, the term really entering general circulation in 1994, coined by Professor Lotfi Zadeh of the University of California, Berkeley, USA, it encompasses several fields of computing. The three that have been examined in this chapter are neural networks, FL and GAs. Neural networks are important for their ability to adapt and learn, FL for its exploitation of partial truth and imprecision, and GAs

for their application to optimization. The field of probabilistic reasoning is also sometimes included under the soft computing umbrella for its control of randomness and uncertainty. The importance of soft computing lies in using these methodologies in partnership — they all offer their own benefits which are generally not competitive and can therefore, work together. As a result, several hybrid systems were looked at — systems in which such partnerships exist.

Artificial Neural Network: An Introduction

2

Learning Objectives

- The fundamentals of artificial neural network.
- The evolution of neural networks.
- Comparison between biological neuron and artificial neuron.
- Basic models of artificial neural networks.
- The different types of connections of neural networks, learning and activation functions are included.
- Various terminologies and notations used throughout the text.
- The basic fundamental neuron model — McCulloch–Pitts neuron and Hebb network.
- The concept of linear separability to form decision boundary regions.

2.1 Fundamental Concept

Neural networks are those information processing systems, which are constructed and implemented to model the human brain. The main objective of the neural network research is to develop a computational device for modeling the brain to perform various computational tasks at a faster rate than the traditional systems. Artificial neural networks perform various tasks such as pattern-matching and classification, optimization function, approximation, vector quantization, and data clustering. These tasks are very difficult for traditional computers, which are faster in algorithmic computational tasks and precise arithmetic operations. Therefore, for implementation of artificial neural networks, high-speed digital computers are used, which makes the simulation of neural processes feasible.

2.1.1 Artificial Neural Network

As already stated in Chapter 1, an artificial neural network (ANN) is an efficient information processing system which resembles in characteristics with a biological neural network. ANNs possess large number of highly interconnected processing elements called *nodes* or *units* or *neurons*, which usually operate in parallel and are configured in regular architectures. Each neuron is connected with the other by a connection link. Each connection link is associated with weights which contain information about the input signal. This information is used by the neuron net to solve a particular problem. ANNs' collective behavior is characterized by their ability to learn, recall and generalize training patterns or data similar to that of a human brain. They have the capability to model networks of original neurons as found in the brain. Thus, the ANN processing elements are called *neurons* or *artificial neurons*.

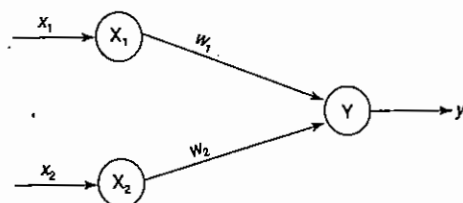


Figure 2-1 Architecture of a simple artificial neuron net.

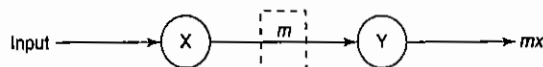


Figure 2-2 Neural net of pure linear equation.

It should be noted that each neuron has an internal state of its own. This internal state is called the activation or activity level of neuron, which is the function of the inputs the neuron receives. The activation signal of a neuron is transmitted to other neurons. Remember, a neuron can send only one signal at a time, which can be transmitted to several other neurons.

To depict the basic operation of a neural net, consider a set of neurons, say X_1 and X_2 , transmitting signals to another neuron, Y . Here X_1 and X_2 are input neurons, which transmit signals, and Y is the output neuron, which receives signals. Input neurons X_1 and X_2 are connected to the output neuron Y , over a weighted interconnection links (w_1 and w_2) as shown in Figure 2-1.

For the above simple neuron net architecture, the net input has to be calculated in the following way:

$$y_{in} = +x_1w_1 + x_2w_2$$

where x_1 and x_2 are the activations of the input neurons X_1 and X_2 , i.e., the output of input signals. The output y of the output neuron Y can be obtained by applying activations over the net input, i.e., the function of the net input:

$$y = f(y_{in})$$

Output = Function (net input calculated)

The function to be applied over the net input is called activation function. There are various activation functions, which will be discussed in the forthcoming sections. The above calculation of the net input is similar to the calculation of output of a pure linear straight line equation ($y = mx$). The neural net of a pure linear equation is as shown in Figure 2-2.

Here, to obtain the output y , the slope m is directly multiplied with the input signal. This is a linear equation. Thus, when slope and input are linearly varied, the output is also linearly varied, as shown in Figure 2-3. This shows that the weight involved in the ANN is equivalent to the slope of the linear straight line.

2.1.2 Biological Neural Network

It is well-known that the human brain consists of a huge number of neurons, approximately 10^{11} , with numerous interconnections. A schematic diagram of a biological neuron is shown in Figure 2-4.

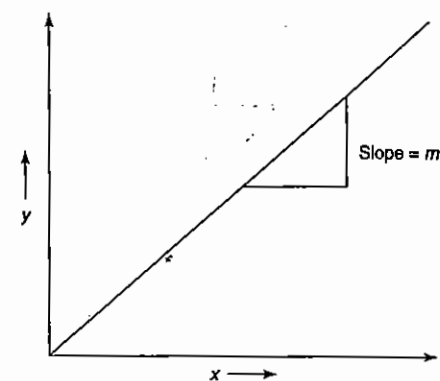
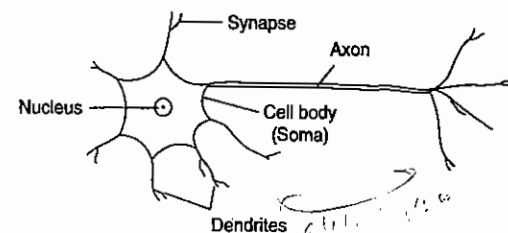
Figure 2-3 Graph for $y = mx$.

Figure 2-4 Schematic diagram of a biological neuron.

The biological neuron depicted in Figure 2-4 consists of three main parts:

1. Soma or cell body – where the cell nucleus is located.
2. Dendrites – where the nerve is connected to the cell body.
3. Axon – which carries the impulses of the neuron.

Dendrites are tree-like networks made of nerve fiber connected to the cell body. An axon is a single, long connection extending from the cell body and carrying signals from the neuron. The end of the axon splits into fine strands. It is found that each strand terminates into a small bulb-like organ called synapse. It is through synapse that the neuron introduces its signals to other nearby neurons. The receiving ends of these synapses on the nearby neurons can be found both on the dendrites and on the cell body. There are approximately 10^4 synapses per neuron in the human brain.

Electric impulses are passed between the synapse and the dendrites. This type of signal transmission involves a chemical process in which specific transmitter substances are released from the sending side of the junction. This results in increase or decrease in the electric potential inside the body of the receiving cell. If the electric potential reaches a threshold then the receiving cell fires and a pulse or action potential of fixed strength and duration is sent out through the axon to the synaptic junctions of the other cells. After firing, a cell has to wait for a period of time called the refractory period before it can fire again. The synapses are said to be inhibitory if they let passing impulses hinder the firing of the receiving cell or excitatory if they let passing impulses cause the firing of the receiving cell.

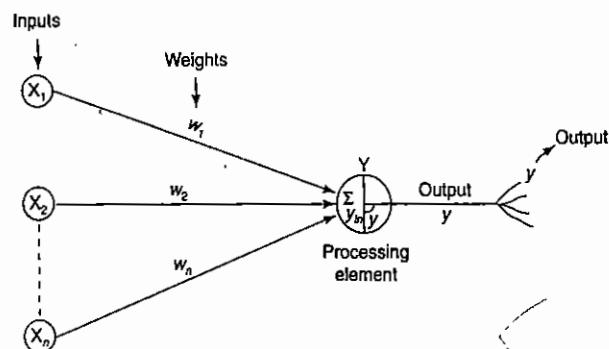


Figure 2-5 Mathematical model of artificial neuron.

Table 2-1 Terminology relationships between biological and artificial neurons

Biological neuron	Artificial neuron
Cell	Neuron
Dendrites	Weights or interconnections
Soma	Net input
Axon	Output

Figure 2-5 shows a mathematical representation of the above-discussed chemical processing taking place in an artificial neuron.

In this model, the net input is elucidated as

$$y_{in} = x_1w_1 + x_2w_2 + \dots + x_nw_n = \sum_{i=1}^n x_iw_i$$

where i represents the i th processing element. The activation function is applied over it to calculate the output. The weight represents the strength of synapse connecting the input and the output neurons. A positive weight corresponds to an excitatory synapse, and a negative weight corresponds to an inhibitory synapse.

The terms associated with the biological neuron and their counterparts in artificial neuron are presented in Table 2-1.

2.1.3 Brain vs. Computer – Comparison Between Biological Neuron and Artificial Neuron (Brain vs. Computer)

A comparison could be made between biological and artificial neurons on the basis of the following criteria:

1. **Speed:** The cycle time of execution in the ANN is of few nanoseconds whereas in the case of biological neuron it is of a few milliseconds. Hence, the artificial neuron modeled using a computer is more faster.

2. **Processing:** Basically, the biological neuron can perform massive parallel operations simultaneously. The artificial neuron can also perform several parallel operations simultaneously, but, in general, the artificial neuron network process is faster than that of the brain.
3. **Size and complexity:** The total number of neurons in the brain is about 10^{11} and the total number of interconnections is about 10^{15} . Hence, it can be noted that the complexity of the brain is comparatively higher, i.e. the computational work takes place not only in the brain cell body, but also in axon, synapse, etc. On the other hand, the size and complexity of an ANN is based on the chosen application and the network designer. The size and complexity of a biological neuron is more than that of an artificial neuron.
4. **Storage capacity (memory):** The biological neuron stores the information in its interconnections or in synapse strength but in an artificial neuron it is stored in its contiguous memory locations. In an artificial neuron, the continuous loading of new information may sometimes overload the memory locations. As a result, some of the addresses containing older memory locations may be destroyed. But in case of the brain, new information can be added in the interconnections by adjusting the strength without destroying the older information. A disadvantage related to brain is that sometimes its memory may fail to recollect the stored information whereas in an artificial neuron, once the information is stored in its memory locations, it can be retrieved. Owing to these facts, the adaptability is more toward an artificial neuron.
5. **Tolerance:** The biological neuron possesses fault tolerant capability whereas the artificial neuron has no fault tolerance. The distributed nature of the biological neurons enables to store and retrieve information even when the interconnections in them get disconnected. Thus biological neurons are fault tolerant. But in case of artificial neurons, the information gets corrupted if the network interconnections are disconnected. Biological neurons can accept redundancies, which is not possible in artificial neurons. Even when some cells die, the human nervous system appears to be performing with the same efficiency.
6. **Control mechanism:** In an artificial neuron modeled using a computer, there is a control unit present in Central Processing Unit, which can transfer and control precise scalar values from unit to unit, but there is no such control unit for monitoring in the brain. The strength of a neuron in the brain depends on the active chemicals present and whether neuron connections are strong or weak as a result of structure layer rather than individual synapses. However, the ANN possesses simpler interconnections and is free from chemical actions similar to those taking place in brain (biological neuron). Thus, the control mechanism of an artificial neuron is very simple compared to that of a biological neuron.

So, we have gone through a comparison between ANNs and biological neural networks. In short, we can say that an ANN possesses the following characteristics:

1. It is a neurally implemented mathematical model.
2. There exist a large number of highly interconnected processing elements called *neurons* in an ANN.
3. The interconnections with their weighted linkages hold the informative knowledge.
4. The input signals arrive at the processing elements through connections and connecting weights.
5. The processing elements of the ANN have the ability to learn, recall and generalize from the given data by suitable assignment or adjustment of weights.
6. The computational power can be demonstrated only by the collective behavior of neurons, and it should be noted that no single neuron carries specific information.

The above-mentioned characteristics make the ANNs as connectionist models, parallel distributed processing models, self-organizing systems, neuro-computing systems and neuro-morphic systems.

2.2 Evolution of Neural Networks

The evolution of neural networks has been facilitated by the rapid development of architectures and algorithms that are currently being used. The history of the development of neural networks along with the names of their designers is outlined Table 2-2.

In the later years, the discovery of the neural net resulted in the implementation of optical neural nets, Boltzmann machine, spatiotemporal nets, pulsed neural networks and support vector machines.

Table 2-2 Evolution of neural networks

Year	Neural network	Designer	Description
1943	McCulloch and Pitts neuron	McCulloch and Pitts	The arrangement of neurons in this case is a combination of logic functions. Unique feature of this neuron is the concept of threshold.
1949	Hebb network	Hebb	It is based upon the fact that if two neurons are found to be active simultaneously then the strength of the connection between them should be increased.
1958, 1959, 1962, 1988	Perceptron	Frank Rosenblatt, Block, Minsky and Papert	Here the weights on the connection path can be adjusted.
1960	Adaline	Widrow and Hoff	Here the weights are adjusted to reduce the difference between the net input to the output unit and the desired output. The result here is very negligible. Mean squared error is obtained.
1972	Kohonen self-organizing feature map	Kohonen	The concept behind this network is that the inputs are clustered together to obtain a fired output neuron. The clustering is performed by winner-take all policy.
1982, 1984, 1985, 1986, 1987	Hopfield network	John Hopfield and Tank	This neural network is based on fixed weights. These nets can also act as associative memory nets.
1986	Back-propagation network	Rumelhart, Hinton and Williams	This network is multi-layer with error being propagated backwards from the output units to the hidden units.
1988	Counter-propagation network	Grossberg	This network is similar to the Kohonen network; here the learning occurs for all units in a particular layer, and there exists no competition among these units.
1987-1990	Adaptive Resonance Theory (ART)	Carpenter and Grossberg	The ART network is designed for both binary inputs and analog valued inputs. Here the input patterns can be presented in any order.
1988	Radial basis function network	Broomhead and Lowe	This resembles a back propagation network but the activation function used is a Gaussian function.
1988	Nco cognitron	Fukushima	This network is essential for character recognition. The deficiency occurred in cognitron network (1975) was corrected by this network.

2.3 Basic Models of Artificial Neural Network

The models of ANN are specified by the three basic entities namely:

1. the model's synaptic interconnections;
2. the training or learning rules adopted for updating and adjusting the connection weights;
3. their activation functions.

2.3.1 Connections

The neurons should be visualized for their arrangements in layers. An ANN consists of a set of highly interconnected processing elements (neurons) such that each processing element output is found to be connected through weights to the other processing elements or to itself; delay lead and lag-free connections are allowed. Hence, the arrangements of these processing elements and the geometry of their interconnections are essential for an ANN. The point where the connection originates and terminates should be noted, and the function of each processing element in an ANN should be specified.

Besides the simple neuron shown in Figure ??, there exist several other types of neural network connections. The arrangement of neurons to form layers and the connection pattern formed within and between layers is called the *network architecture*. There exist five basic types of neuron connection architectures. They are:

1. single-layer feed-forward network;
2. multilayer feed-forward network;
3. single node with its own feedback;
4. single-layer recurrent network;
5. multilayer recurrent network.

Figures 2-6–2-10 depict the five types of neural network architectures. Basically, neural nets are classified into single-layer or multilayer neural nets. A layer is formed by taking a processing element and combining it with other processing elements. Practically, a layer implies a stage, going stage by stage, i.e., the input stage and the output stage are linked with each other. These linked interconnections lead to the formation of various network architectures. When a layer of the processing nodes is formed, the inputs can be connected to these

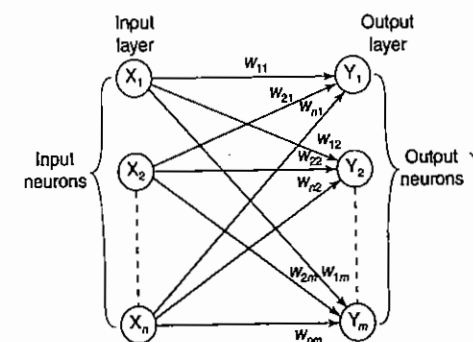


Figure 2-6 Single-layer feed-forward network.

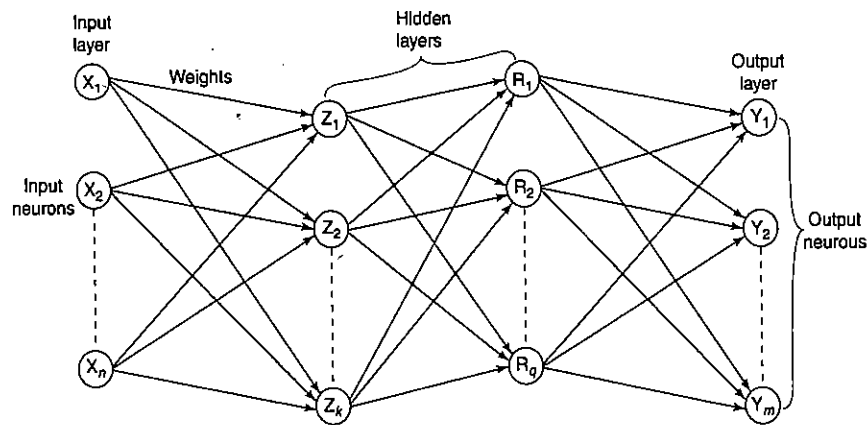


Figure 2-7 Multilayer feed-forward network.

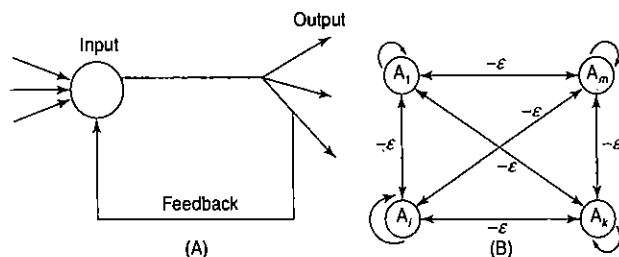


Figure 2-8 (A) Single node with own feedback. (B) Competitive nets.

nodes with various weights, resulting in a series of outputs, one per node. Thus, a single-layer feed-forward network is formed.

A multilayer feed-forward network (Figure 2-7) is formed by the interconnection of several layers. The input layer is that which receives the input and this layer has no function except buffering the input signal. The output layer generates the output of the network. Any layer that is formed between the input and output layers is called *hidden layer*. This hidden layer is internal to the network and has no direct contact with the external environment. It should be noted that there may be zero to several hidden layers in an ANN. More the number of the hidden layers, more is the complexity of the network. This may, however, provide an efficient output response. In case of a fully connected network every output from one layer is connected to each and every node in the next layer.

A network is said to be a feed-forward network if no neuron in the output layer is an input to a node in the same layer or in the preceding layer. On the other hand, when outputs can be directed back as inputs to same or preceding layer nodes then it results in the formation of *feedback networks*.

If the feedback of the output of the processing elements is directed back as input to the processing elements in the same layer then it is called *lateral feedback*. Recurrent networks are feedback networks with closed loop. Figure 2-8(A) shows a simple recurrent neural network having a single neuron with

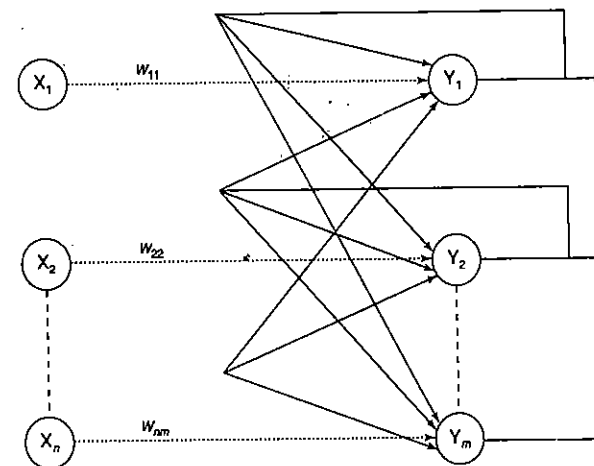


Figure 2-9 Single-layer recurrent network.

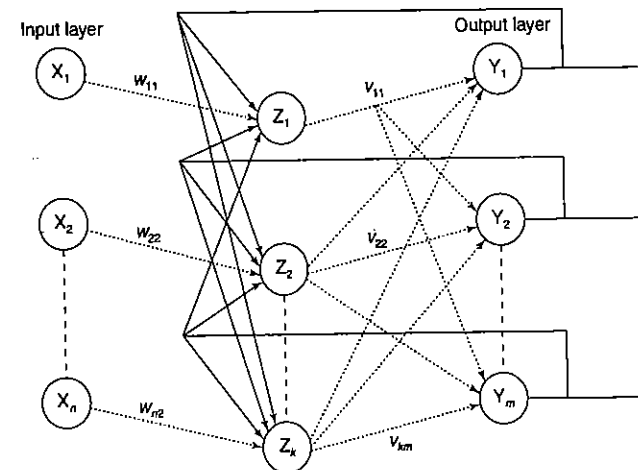


Figure 2-10 Multilayer recurrent network.

feedback to itself. Figure 2-9 shows a single-layer network with a feedback connection in which a processing element's output can be directed back to the processing element itself or to the other processing element or to both.

The architecture of a competitive layer is shown in Figure 2-8(B), the competitive interconnections having fixed weights of $-\epsilon$. This net is called *Maxnet*, and will be discussed in the unsupervised learning network category. Apart from the network architectures discussed so far, there also exists another type of architecture with lateral feedback, which is called the *on-center-off-surround* or *lateral inhibition structure*. In this

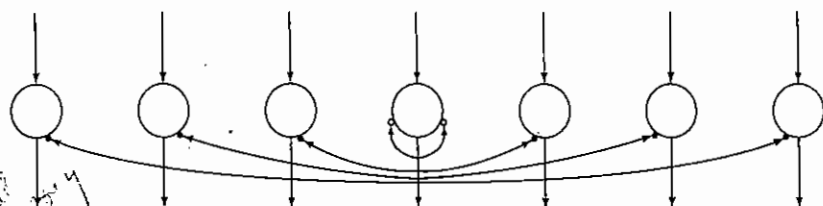


Figure 2-11 Lateral inhibition structure.

structure, each processing neuron receives two different classes of inputs – “excitatory” input from nearby processing elements and “inhibitory” inputs from more distantly located processing elements. This type of interconnection is shown in Figure 2-11.

In Figure 2-11, the connections with open circles are excitatory connections and the links with solid connective circles are inhibitory connections. From Figure 2-10, it can be noted that a processing element output can be directed back to the nodes in a preceding layer, forming a *multilayer recurrent network*. Also, in these networks, a processing element output can be directed back to the processing element itself and to other processing elements in the same layer. Thus, the various network architectures as discussed from Figures 2-6–2-11 can be suitably used for giving effective solution to a problem by using ANN.

2.3.2 Learning

The main property of an ANN is its capability to learn. Learning or training is a process by means of which a neural network adapts itself to a stimulus by making proper parameter adjustments, resulting in the production of desired response. Broadly, there are two kinds of learning in ANNs:

1. **Parameter learning:** It updates the connecting weights in a neural net.
2. **Structure learning:** It focuses on the change in network structure (which includes the number of processing elements as well as their connection types).

The above two types of learning can be performed simultaneously or separately. Apart from these two categories of learning, the learning in an ANN can be generally classified into three categories as: supervised learning; unsupervised learning; reinforcement learning. Let us discuss these learning types in detail.

2.3.2.1 Supervised Learning

The learning here is performed with the help of a teacher. Let us take the example of the learning process of a small child. The child doesn't know how to read/write. He/she is being taught by the parents at home and by the teacher in school. The children are trained and molded to recognize the alphabets, numerals, etc. Their each and every action is supervised by a teacher. Actually, a child works on the basis of the output that he/she has to produce. All these real-time events involve supervised learning methodology. Similarly, in ANNs following the supervised learning, each input vector requires a corresponding target vector, which represents the desired output. The input vector along with the target vector is called training pair. The network here is informed precisely about what should be emitted as output. The block diagram of Figure 2-12 depicts the working of a supervised learning network.

During training, the input vector is presented to the network, which results in an output vector. This output vector is the actual output vector. Then the actual output vector is compared with the desired (target) output vector. If there exists a difference between the two output vectors then an error signal is generated by

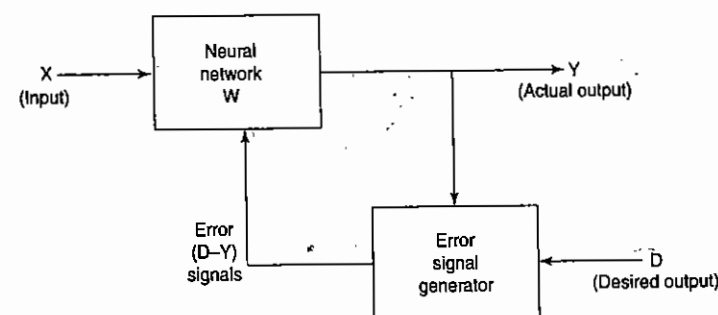


Figure 2-12 Supervised learning.

the network. This error signal is used for adjustment of weights until the actual output matches the desired (target) output. In this type of training, a supervisor or teacher is required for error minimization. Hence, the network trained by this method is said to be using supervised training methodology. In supervised learning, it is assumed that the correct “target” output values are known for each input pattern.

2.3.2.2 Unsupervised Learning

The learning here is performed without the help of a teacher. Consider the learning process of a tadpole, it learns by itself, that is, a child fish learns to swim by itself, it is not taught by its mother. Thus, its learning process is independent and is not supervised by a teacher. In ANNs following unsupervised learning, the input vectors of similar type are grouped without the use of training data to specify how a member of each group looks or to which group a number belongs. In the training process, the network receives the input patterns and organizes these patterns to form clusters. When a new input pattern is applied, the neural network gives an output response indicating the class to which the input pattern belongs. If for an input, a pattern class cannot be found then a new class is generated. The block diagram of unsupervised learning is shown in Figure 2-13.

From Figure 2-13 it is clear that there is no feedback from the environment to inform what the outputs should be or whether the outputs are correct. In this case, the network must itself discover patterns, regularities, features or categories from the input data and relations for the input data over the output. While discovering all these features, the network undergoes change in its parameters. This process is called self-organizing in which exact clusters will be formed by discovering similarities and dissimilarities among the objects.

2.3.2.3 Reinforcement Learning

This learning process is similar to supervised learning. In the case of supervised learning, the correct target output values are known for each input pattern. But, in some cases, less information might be available.

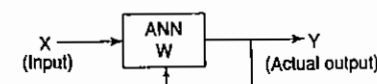


Figure 2-13 Unsupervised learning.

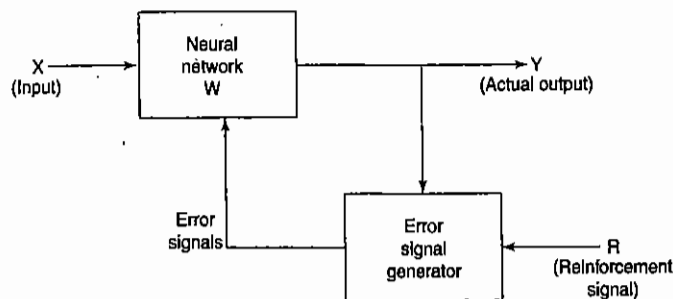


Figure 2-14 Reinforcement learning.

For example, the network might be told that its actual output is only "50% correct" or so. Thus, here only critic information is available, not the exact information. The learning based on this critic information is called reinforcement learning and the feedback sent is called reinforcement signal.

The block diagram of reinforcement learning is shown in Figure 2-14. The reinforcement learning is a form of supervised learning because the network receives some feedback from its environment. However, the feedback obtained here is only evaluative and not instructive. The external reinforcement signals are processed in the critic signal generator, and the obtained critic signals are sent to the ANN for adjustment of weights properly so as to get better critic feedback in future. The reinforcement learning is also called learning with a critic as opposed to learning with a teacher, which indicates supervised learning.

So, now you've a fair understanding of the three generalized learning rules used in the training process of ANNs.

2.3.3 Activation Functions

To better understand the role of the activation function, let us assume a person is performing some work. To make the work more efficient and to obtain exact output, some force or activation may be given. This activation helps in achieving the exact output. In a similar way, the activation function is applied over the net input to calculate the output of an ANN.

The information processing of a processing element can be viewed as consisting of two major parts: input and output. An integration function (say f) is associated with the input of a processing element. This function serves to combine activation, information or evidence from an external source or other processing elements into a net input to the processing element. The nonlinear activation function is used to ensure that a neuron's response is bounded – that is, the actual response of the neuron is conditioned or dampened as a result of large or small activating stimuli and is thus controllable.

Certain nonlinear functions are used to achieve the advantages of a multilayer network from a single-layer network. When a signal is fed through a multilayer network with linear activation functions, the output obtained remains same as that could be obtained using a single-layer network. Due to this reason, nonlinear functions are widely used in multilayer networks compared to linear functions.

There are several activation functions. Let us discuss a few in this section:

1. **Identity function:** It is a linear function and can be defined as

$$f(x) = x \quad \text{for all } x$$

The output here remains the same as input. The input layer uses the identity activation function.

2. **Binary step function:** This function can be defined as

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$

where θ represents the threshold value. This function is most widely used in single-layer nets to convert the net input to an output that is a binary (1 or 0).

3. **Bipolar step function:** This function can be defined as

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ -1 & \text{if } x < \theta \end{cases}$$

where θ represents the threshold value. This function is also used in single-layer nets to convert the net input to an output that is bipolar (+1 or -1).

4. **Sigmoidal functions:** The sigmoidal functions are widely used in back-propagation nets because of the relationship between the value of the functions at a point and the value of the derivative at that point which reduces the computational burden during training. Sigmoidal functions are of two types:

- **Binary sigmoid function:** It is also termed as logistic sigmoid function or unipolar sigmoid function. It can be defined as

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

where λ is the steepness parameter. The derivative of this function is

$$f'(x) = \lambda f(x)[1 - f(x)]$$

Here the range of the sigmoid function is from 0 to 1.

- **Bipolar sigmoid function:** This function is defined as

$$f(x) = \frac{2}{1 + e^{-\lambda x}} - 1 = \frac{1 - e^{-\lambda x}}{1 + e^{-\lambda x}}$$

where λ is the steepness parameter and the sigmoid function range is between -1 and +1. The derivative of this function can be

$$f'(x) = \frac{\lambda}{2} [1 + f(x)][1 - f(x)]$$

The bipolar sigmoidal function is closely related to hyperbolic tangent function, which is written as

$$h(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

The derivative of the hyperbolic tangent function is

$$h'(x) = [1 + h(x)][1 - h(x)]$$

If the network uses a binary data, it is better to convert it to bipolar form and use the bipolar sigmoidal activation function or hyperbolic tangent function.

5. **Ramp function:** The ramp function is defined as

$$f(x) = \begin{cases} 1 & \text{if } x > 1 \\ x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{if } x < 0 \end{cases}$$

The graphical representations of all the activation functions are shown in Figure 2-15(A)–(F).

2.4 Important Terminologies of ANNs

This section introduces you to the various terminologies related with ANNs.

2.4.1 Weights

In the architecture of an ANN, each neuron is connected to other neurons by means of directed communication links, and each communication link is associated with weights. The weights contain information about the input signal. This information is used by the net to solve a problem. The weight can be represented in terms of matrix. The weight matrix can also be called connection matrix. To form a mathematical notation, it is assumed that there are “ n ” processing elements in an ANN and each processing element has exactly “ m ” adaptive weights. Thus, the weight matrix W is defined by

$$W = \begin{bmatrix} w_1^T \\ w_2^T \\ \vdots \\ w_n^T \end{bmatrix} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1m} \\ w_{21} & w_{22} & \dots & w_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n1} & w_{n2} & \dots & w_{nm} \end{bmatrix}$$

where $w_i = [w_{i1}, w_{i2}, \dots, w_{im}]^T$, $i = 1, 2, \dots, n$, is the weight vector of processing element and w_{ij} is the weight from processing element “ i ” (source node) to processing element “ j ” (destination node).

If the weight matrix W contains all the adaptive elements of an ANN, then the set of all W matrices will determine the set of all possible information processing configurations for this ANN. The ANN can be realized by finding an appropriate matrix W . Hence, the weights encode long-term memory (LTM) and the activation states of neurons encode short-term memory (STM) in a neural network.

2.4.2 Bias

The bias included in the network has its impact in calculating the net input. The bias is included by adding a component $x_0 = 1$ to the input vector X . Thus, the input vector becomes

$$X = (1, X_1, \dots, X_i, \dots, X_n)$$

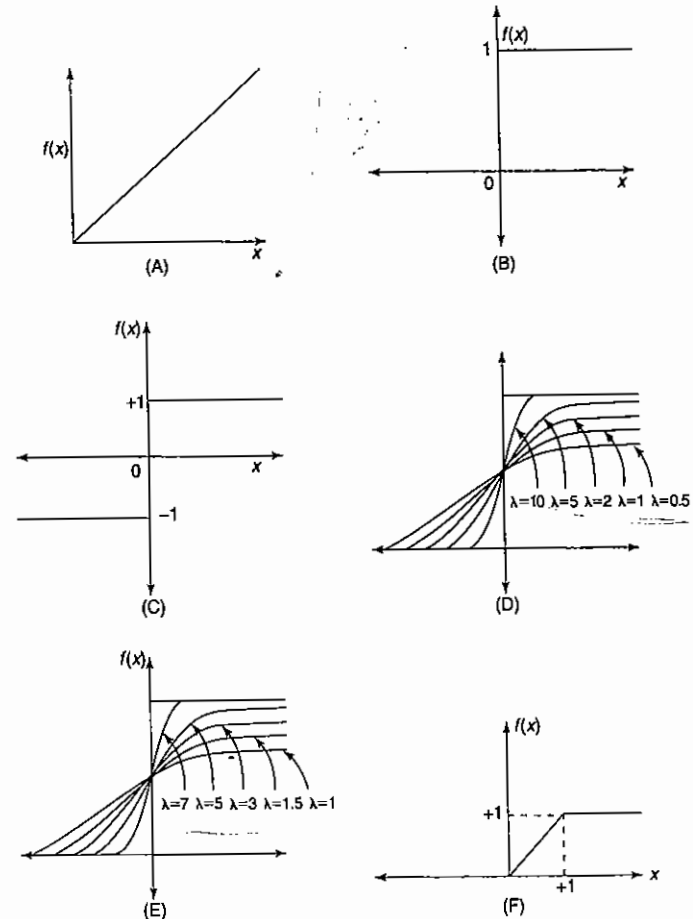


Figure 2-15 Depiction of activation functions: (A) identity function; (B) binary step function; (C) bipolar step function; (D) binary sigmoidal function; (E) bipolar sigmoidal function; (F) ramp function.

The bias is considered like another weight, that is, $w_{0j} = b_j$. Consider a simple network shown in Figure 2-16 with bias. From Figure 2-16, the net input to the output neuron Y_j is calculated as

$$\begin{aligned} y_{inj} &= \sum_{i=0}^n x_i w_{ij} = x_0 w_{0j} + x_1 w_{1j} + x_2 w_{2j} + \dots + x_n w_{nj} \\ &= w_{0j} + \sum_{i=1}^n x_i w_{ij} \\ y_{inj} &= b_j + \sum_{i=1}^n x_i w_{ij} \end{aligned}$$

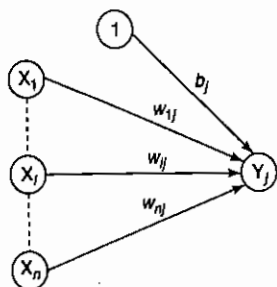


Figure 2-16 Simple net with bias.

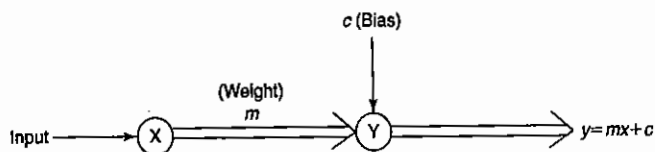


Figure 2-17 Block diagram for straight line.

The activation function discussed in Section 2.3.3 is applied over this net input to calculate the output. The bias can also be explained as follows: Consider an equation of straight line,

$$y = mx + c$$

where x is the input, m is the weight, c is the bias and y is the output. The equation of the straight line can also be represented as a block diagram shown in Figure 2-17. Thus, bias plays a major role in determining the output of the network.

The bias can be of two types: positive bias and negative bias. The positive bias helps in increasing the net input of the network and the negative bias helps in decreasing the net input of the network. Thus, as a result of the bias effect, the output of the network can be varied.

2.4.3 Threshold

Threshold is a set value based upon which the final output of the network may be calculated. The threshold value is used in the activation function. A comparison is made between the calculated net input and the threshold to obtain the network output. For each and every application, there is a threshold limit. Consider a direct current (DC) motor. If its maximum speed is 1500 rpm then the threshold based on the speed is 1500 rpm. If the motor is run on a speed higher than its set threshold, it may damage motor coils. Similarly, in neural networks, based on the threshold value, the activation functions are defined and the output is calculated. The activation function using threshold can be defined as

$$f(\text{net}) = \begin{cases} 1 & \text{if net} \geq \theta \\ -1 & \text{if net} < \theta \end{cases}$$

where θ is the fixed threshold value.

2.4.4 Learning Rate

The learning rate is denoted by " α ." It is used to control the amount of weight adjustment at each step of training. The learning rate, ranging from 0 to 1, determines the rate of learning at each time step.

2.4.5 Momentum Factor

Convergence is made faster if a momentum factor is added to the weight updation process. This is generally done in the back propagation network. If momentum has to be used, the weights from one or more previous training patterns must be saved. Momentum helps the net in reasonably large weight adjustments until the corrections are in the same general direction for several patterns.

2.4.6 Vigilance Parameter

The vigilance parameter is denoted by " ρ ." It is generally used in adaptive resonance theory (ART) network. The vigilance parameter is used to control the degree of similarity required for patterns to be assigned to the same cluster unit. The choice of vigilance parameter ranges approximately from 0.7 to 1 to perform useful work in controlling the number of clusters.

2.4.7 Notations

The notations mentioned in this section have been used in this textbook for explaining each network.

- x_i : Activation of unit X_i , input signal.
- y_j : Activation of unit Y_j , $y_j = f(y_{inj})$
- w_{ij} : Weight on connection from unit X_i to unit Y_j .
- b_j : Bias acting on unit j . Bias has a constant activation of 1.
- W : Weight matrix, $W = \{w_{ij}\}$
- y_{inj} : Net input to unit Y_j given by $y_{inj} = b_j + \sum_i x_i w_{ij}$
- $\|x\|$: Norm of magnitude vector X .
- θ_j : Threshold for activation of neuron Y_j .
- S : Training input vector, $S = (s_1, \dots, s_i, \dots, s_n)$
- T : Training output vector, $T = (t_1, \dots, t_j, \dots, t_n)$
- X : Input vector, $X = (x_1, \dots, x_i, \dots, x_n)$
- Δw_{ij} : Change in weights given by $\Delta w_{ij} = w_{ij}(\text{new}) - w_{ij}(\text{old})$
- α : Learning rate; it controls the amount of weight adjustment at each step of training.

2.5 McCulloch–Pitts Neuron

2.5.1 Theory

The McCulloch–Pitts neuron was the earliest neural network discovered in 1943. It is usually called as *M–P neuron*. The M–P neurons are connected by directed weighted paths. It should be noted that the activation of a M–P neuron is binary, that is, at any time step the neuron may fire or may not fire. The weights associated with the communication links may be excitatory (weight is positive) or inhibitory (weight is negative). All the

excitatory connected weights entering into a particular neuron will have same weights. The threshold plays a major role in M-P neuron: There is a fixed threshold for each neuron, and if the net input to the neuron is greater than the threshold then the neuron fires. Also, it should be noted that any nonzero inhibitory input would prevent the neuron from firing. The M-P neurons are most widely used in the case of logic functions.

2.5.2 Architecture

A simple M-P neuron is shown in Figure 2-18. As already discussed, the M-P neuron has both excitatory and inhibitory connections. It is excitatory with weight ($w > 0$) or inhibitory with weight $-p$ ($p < 0$). In Figure 2-18, inputs from x_1 to x_n possess excitatory weighted connections and inputs from x_{n+1} to x_{n+m} possess inhibitory weighted interconnections. Since the firing of the output neuron is based upon the threshold, the activation function here is defined as

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$

For inhibition to be absolute, the threshold with the activation function should satisfy the following condition:

$$\theta > nw - p$$

The output will fire if it receives say " k " or more excitatory inputs but no inhibitory inputs, where

$$kw \geq \theta > (k-1)w$$

The M-P neuron has no particular training algorithm. An analysis has to be performed to determine the values of the weights and the threshold. Here the weights of the neuron are set along with the threshold to make the neuron perform a simple logic function. The M-P neurons are used as building blocks on which we can model any function or phenomenon, which can be represented as a logic function.

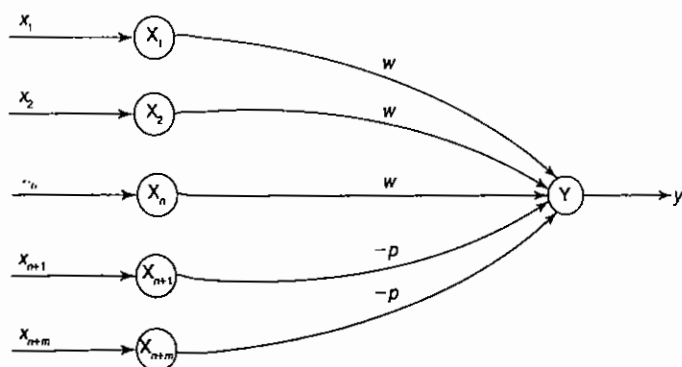


Figure 2-18 McCulloch-Pitts neuron model.

2.6 Linear Separability

An ANN does not give an exact solution for a nonlinear problem. However, it provides possible approximate solutions to nonlinear problems. Linear separability is the concept wherein the separation of the input space into regions is based on whether the network response is positive or negative.

A decision line is drawn to separate positive and negative responses. The decision line may also be called as the *decision-making line* or *decision-support line* or *linear-separable line*. The necessity of the linear separability concept was felt to classify the patterns based upon their output responses. Generally the net input calculated to the output unit is given as

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

For example, if a bipolar step activation function is used over the calculated net input (y_{in}) then the value of the function is 1 for a positive net input and -1 for a negative net input. Also, it is clear that there exists a boundary between the regions where $y_{in} > 0$ and $y_{in} < 0$. This region may be called as *decision boundary* and can be determined by the relation

$$b + \sum_{i=1}^n x_i w_i = 0$$

On the basis of the number of input units in the network, the above equation may represent a line, a plane or a hyperplane. The linear separability of the network is based on the decision-boundary line. If there exist weights (with bias) for which the training input vectors having positive (correct) response, $+1$, lie on one side of the decision boundary and all the other vectors having negative (incorrect) response, -1 , lie on the other side of the decision boundary then we can conclude the problem is "linearly separable."

Consider a single-layer network as shown in Figure 2-19 with bias included. The net input for the network shown in Figure 2-19 is given as

$$y_{in} = b + x_1 w_1 + x_2 w_2$$

The separating line for which the boundary lies between the values x_1 and x_2 , so that the net gives a positive response on one side and negative response on other side, is given as

$$b + x_1 w_1 + x_2 w_2 = 0$$

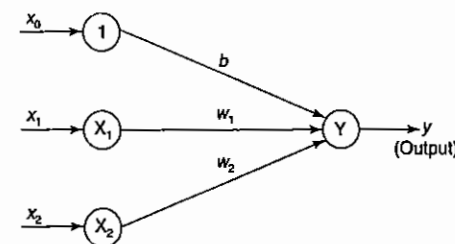


Figure 2-19 A single-layer neural net.

If weight w_2 is not equal to 0 then we get

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$$

Thus, the requirement for the positive response of the net is

$$b + x_1 w_1 + x_2 w_2 > 0$$

During training process, the values of w_1 , w_2 and b are determined so that the net will produce a positive (correct) response for the training data. If on the other hand, threshold value is being used, then the condition for obtaining the positive response from output unit is

$$\text{Net input received} > \theta \text{ (threshold)}$$

$$y_{in} > \theta$$

$$x_1 w_1 + x_2 w_2 > \theta$$

The separating line equation will then be

$$x_1 w_1 + x_2 w_2 = \theta$$

$$x_2 = -\frac{w_1}{w_2}x_1 + \frac{\theta}{w_2} \text{ (with } w_2 \neq 0 \text{)}$$

During training process, the values of w_1 and w_2 have to be determined, so that the net will have a correct response to the training data. For this correct response, the line passes close through the origin. In certain situations, even for correct response, the separating line does not pass through the origin.

Consider a network having positive response in the first quadrant and negative response in all other quadrants (AND function) with either binary or bipolar data, then the decision line is drawn separating the positive response region from the negative response region. This is depicted in Figure 2-20.

Thus, based on the conditions discussed above, the equation of this decision line may be obtained. Also, in all the networks that we would be discussing, the representation of data plays a major role.

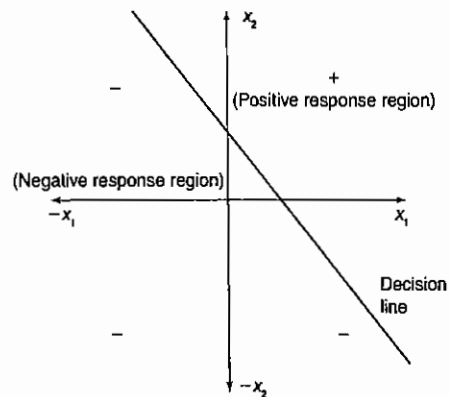


Figure 2-20 Decision boundary line.

However, the data representation mode has to be decided – whether it would be in binary form or in bipolar form. It may be noted that the bipolar representation is better than the binary representation. Using bipolar data representation, the missing data can be distinguished from mistaken data. Missing values are represented by 0 and mistakes can be represented by reversing the input value from +1 to -1 or vice-versa.

2.7 Hebb Network

(only one output unit)

2.7.1 Theory

For a neural net, the Hebb learning rule is a simple one. Let us understand it. Donald Hebb stated in 1949 that in the brain, the learning is performed by the change in the synaptic gap. Hebb explained it: "When an axon of cell A is near enough to excite cell B, and repeatedly or permanently takes place in firing it, some growth process or metabolic change takes place in one or both the cells such that A's efficiency, as one of the cells firing B, is increased."

According to the Hebb rule, the weight vector is found to increase proportionately to the product of the input and the learning signal. Here the learning signal is equal to the neuron's output. In Hebb learning, if two interconnected neurons are 'on' simultaneously then the weights associated with these neurons can be increased by the modification made in their synaptic gap (strength). The weight update in Hebb rule is given by

$$w_i(\text{new}) = w_i(\text{old}) + x_i y$$

The Hebb rule is more suited for bipolar data than binary data. If binary data is used, the above weight updation formula cannot distinguish two conditions namely:

1. A training pair in which an input unit is "on" and target value is "off."
2. A training pair in which both the input unit and the target value are "off."

Thus, there are limitations in Hebb rule application over binary data. Hence, the representation using bipolar data is advantageous.

2.7.2 Flowchart of Training Algorithm

The training algorithm is used for the calculation and adjustment of weights. The flowchart for the training algorithm of Hebb network is given in Figure 2-21. The notations used in the flowchart have already been discussed in Section 2.4.7.

In Figure 2-21, s : t refers to each training input and target output pair. Till there exists a pair of training input and target output, the training process takes place; else, it is stopped.

2.7.3 Training Algorithm

The training algorithm of Hebb network is given below:

Step 0: First initialize the weights. Basically in this network they may be set to zero, i.e., $w_i = 0$ for $i = 1$ to n where " n " may be the total number of input neurons.

Step 1: Steps 2–4 have to be performed for each input training vector and target output pair, s : t .

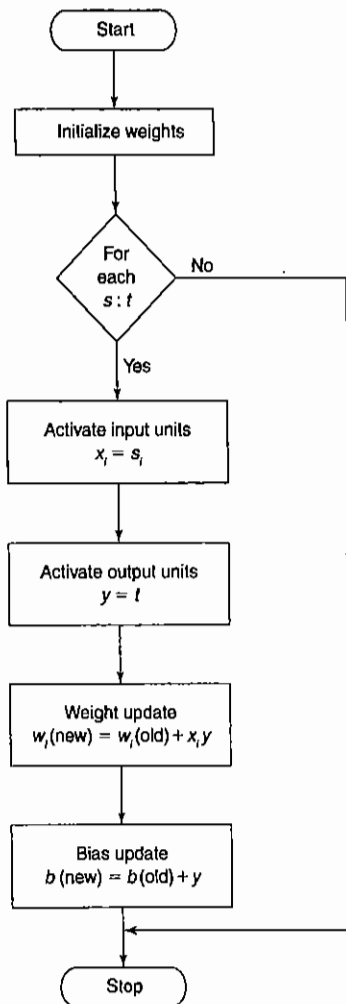


Figure 2-21 Flowchart of Hebb training algorithm.

Step 2: Input units activations are set. Generally, the activation function of input layer is identity function:

$$x_i = s_i \text{ for } i = 1 \text{ to } n$$

Step 3: Output units activations are set: $y = t$

Step 4: Weight adjustments and bias adjustments are performed:

$$w_i(\text{new}) = w_i(\text{old}) + x_i y$$

$$b(\text{new}) = b(\text{old}) + y$$

The above five steps complete the algorithmic process. In Step 4, the weight updation formula can also be given in vector form as

$$w(\text{new}) = w(\text{old}) + xy$$

Here the change in weight can be expressed as

$$\Delta w = xy$$

As a result,

$$w(\text{new}) = w(\text{old}) + \Delta w$$

The Hebb rule can be used for pattern association, pattern categorization, pattern classification and over a range of other areas.

2.8 Summary

In this chapter we have discussed the basics of an ANN and its growth. A detailed comparison between biological neuron and artificial neuron has been included to enable the reader understand the basic difference between them. An ANN is constructed with few basic building blocks. The building blocks are based on the models of artificial neurons and the topology of few basic structures. Concepts of supervised learning, unsupervised learning and reinforcement learning are briefly included in this chapter. Various activation functions and different types of layered connections are also considered here. The basic terminologies of ANN are discussed with their typical values. A brief description on McCulloch-Pitts neuron model is provided. The concept of linear separability is discussed and illustrated with suitable examples. Details are provided for the effective training of a Hebb network.

2.9 Solved Problems

1. For the network shown in Figure 1, calculate the weights are net input to the output neuron.

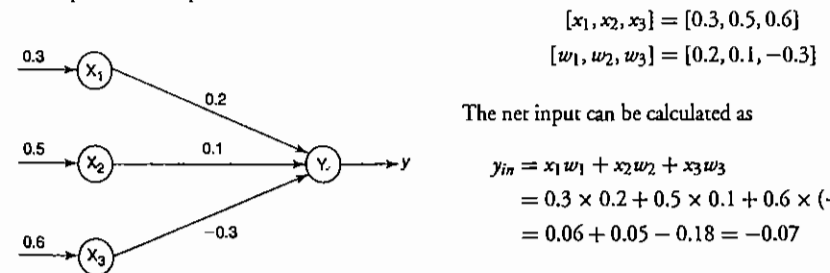


Figure 1 Neural net.

2. Calculate the net input for the network shown in Figure 2 with bias included in the network.

Solution: The given neural net consists of three input neurons and one output neuron. The inputs and

Solution: The given net consists of two input neurons, a bias and an output neuron. The inputs are

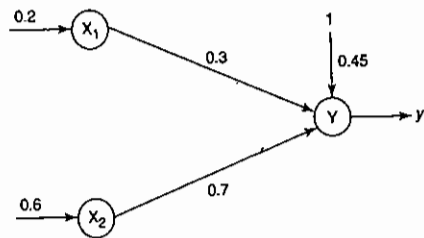


Figure 2 Simple neural net.

$[x_1, x_2] = [0.2, 0.6]$ and the weights are $[w_1, w_2] = [0.3, 0.7]$. Since the bias is included $b = 0.45$ and bias input x_0 is equal to 1, the net input is calculated as

$$\begin{aligned} y_{in} &= b + x_1 w_1 + x_2 w_2 \\ &= 0.45 + 0.2 \times 0.3 + 0.6 \times 0.7 \\ &= 0.45 + 0.06 + 0.42 = 0.93 \end{aligned}$$

Therefore $y_{in} = 0.93$ is the net input.

3. Obtain the output of the neuron Y for the network shown in Figure 3 using activation functions as: (i) binary sigmoidal and (ii) bipolar sigmoidal.

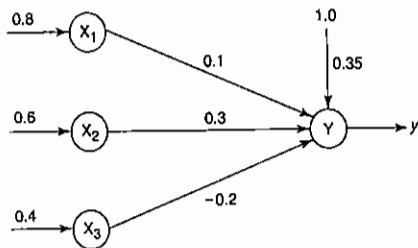


Figure 3 Neural net.

Solution: The given network has three input neurons with bias and one output neuron. These form a single-layer network. The inputs are given as $[x_1, x_2, x_3] = [0.8, 0.6, 0.4]$ and the weights are $[w_1, w_2, w_3] = [0.1, 0.3, -0.2]$ with bias $b = 0.35$ (its input is always 1).

The net input to the output neuron is

$$\begin{aligned} y_{in} &= b + \sum_{i=1}^n x_i w_i \\ [n = 3, \text{ because only } 3 \text{ input neurons are given}] \\ &= b + x_1 w_1 + x_2 w_2 + x_3 w_3 \\ &= 0.35 + 0.8 \times 0.1 + 0.6 \times 0.3 \\ &\quad + 0.4 \times (-0.2) \\ &= 0.35 + 0.08 + 0.18 - 0.08 = 0.53 \end{aligned}$$

- (i) For binary sigmoidal activation function,

$$y = f(y_{in}) = \frac{1}{1 + e^{-y_{in}}} = \frac{1}{1 + e^{-0.53}} = 0.625$$

- (ii) For bipolar sigmoidal activation function,

$$y = f(y_{in}) = \frac{2}{1 + e^{-y_{in}}} - 1 = \frac{2}{1 + e^{-0.53}} - 1 = 0.259$$

4. Implement AND function using McCulloch-Pitts neuron (take binary data).

Solution: Consider the truth table for AND function (Table 1).

Table 1

x_1	x_2	y
1	1	1
1	0	0
0	1	0
0	0	0

In McCulloch-Pitts neuron, only analysis is being performed. Hence, assume the weights be $w_1 = 1$ and $w_2 = 1$. The network architecture is shown in Figure 4. With these assumed weights, the net input is calculated for four inputs: For inputs

$$\begin{aligned} (1, 1), \quad y_{in} &= x_1 w_1 + x_2 w_2 = 1 \times 1 + 1 \times 1 = 2 \\ (1, 0), \quad y_{in} &= x_1 w_1 + x_2 w_2 = 1 \times 1 + 0 \times 1 = 1 \\ (0, 1), \quad y_{in} &= x_1 w_1 + x_2 w_2 = 0 \times 1 + 1 \times 1 = 1 \\ (0, 0), \quad y_{in} &= x_1 w_1 + x_2 w_2 = 0 \times 1 + 0 \times 1 = 0 \end{aligned}$$

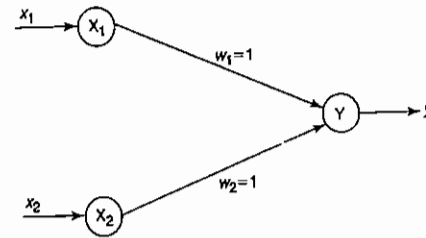


Figure 4 Neural net.

For an AND function, the output is high if both the inputs are high. For this condition, the net input is calculated as 2. Hence, based on this net input, the threshold is set, i.e. if the threshold value is greater than or equal to 2 then the neuron fires, else it does not fire. So the threshold value is set equal to 2 ($\theta = 2$). This can also be obtained by

$$\theta \geq nw - p$$

Here, $n = 2$, $w = 1$ (excitatory weights) and $p = 0$ (no inhibitory weights). Substituting these values in the above-mentioned equation we get

$$\theta \geq 2 \times 1 - 0 \Rightarrow \theta \geq 2$$

Thus, the output of neuron Y can be written as

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq 2 \\ 0 & \text{if } y_{in} < 2 \end{cases}$$

where "2" represents the threshold value.

5. Implement ANDNOT function using McCulloch-Pitts neuron (use binary data representation).

Solution: In the case of ANDNOT function, the response is true if the first input is true and the second input is false. For all other input variations, the response is false. The truth table for ANDNOT function is given in Table 2.

Table 2

x_1	x_2	y
0	0	0
0	1	0
1	0	1
1	1	0

The given function gives an output only when $x_1 = 1$ and $x_2 = 0$. The weights have to be decided only after the analysis. The net can be represented as shown in Figure 5.

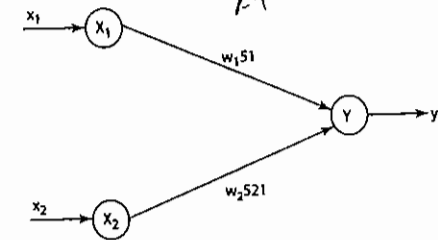


Figure 5 Neural net (weights fixed after analysis).

Case 1: Assume that both weights w_1 and w_2 are excitatory, i.e.,

$$w_1 = w_2 = 1$$

Then for the four inputs calculate the net input using

$$y_{in} = x_1 w_1 + x_2 w_2$$

For inputs

$$(1, 1), \quad y_{in} = 1 \times 1 + 1 \times 1 = 2$$

$$(1, 0), \quad y_{in} = 1 \times 1 + 0 \times 1 = 1$$

$$(0, 1), \quad y_{in} = 0 \times 1 + 1 \times 1 = 1$$

$$(0, 0), \quad y_{in} = 0 \times 1 + 0 \times 1 = 0$$

From the calculated net inputs, it is not possible to fire the neuron for input (1, 0) only. Hence, these weights are not suitable.

Assume one weight as excitatory and the other as inhibitory, i.e.,

$$w_1 = 1, \quad w_2 = -1$$

Now calculate the net input. For the inputs

$$(1, 1), y_{in} = 1 \times 1 + 1 \times -1 = 0$$

$$(1, 0), y_{in} = 1 \times 1 + 0 \times -1 = 1$$

$$(0, 1), y_{in} = 0 \times 1 + 1 \times -1 = -1$$

$$(0, 0), y_{in} = 0 \times 1 + 0 \times -1 = 0$$

From the calculated net inputs, now it is possible to fire the neuron for input (1, 0) only by fixing a threshold of 1, i.e., $\theta \geq 1$ for Y unit. Thus,

$$w_1 = 1; w_2 = -1; \theta \geq 1$$

Note: The value of θ is calculated using the following:

$$\theta \geq nw - p$$

$$\theta \geq 2 \times 1 - 1 \quad [\text{for "p" inhibitory only magnitude considered}]$$

$$\theta \geq 1$$

Thus, the output of neuron Y can be written as

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq 1 \\ 0 & \text{if } y_{in} < 1 \end{cases}$$

6. Implement XOR function using McCulloch-Pitts neuron (consider binary data).

Solution: The truth table for XOR function is given in Table 3.

Table 3

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

In this case, the output is "ON" for only odd number of 1's. For the rest it is "OFF". XOR function cannot be represented by simple and single logic function; it is represented as

$$y = x_1 \bar{x}_2 + \bar{x}_1 x_2$$

$$y = z_1 + z_2$$

where

$$\begin{aligned} z_1 &= x_1 \bar{x}_2 & (\text{function 1}) \\ z_2 &= \bar{x}_1 x_2 & (\text{function 2}) \\ y &= z_1 \text{ OR } z_2 & (\text{function 3}) \end{aligned}$$

A single-layer net is not sufficient to represent the function. An intermediate layer is necessary.

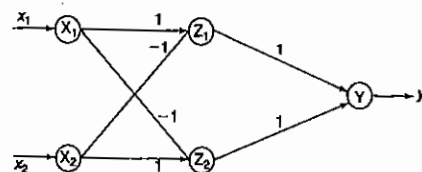


Figure 6 Neural net for XOR function (the weights shown are obtained after analysis).

• First function ($z_1 = x_1 \bar{x}_2$): The truth table for function z_1 is shown in Table 4.

Table 4

x_1	x_2	z_1
0	0	0
0	1	0
1	0	1
1	1	0

The net representation is given as

Case 1: Assume both weights as excitatory, i.e.,

$$w_{11} = w_{21} = 1$$

Calculate the net inputs. For inputs,

$$(0, 0), z_{1in} = 0 \times 1 + 0 \times 1 = 0$$

$$(0, 1), z_{1in} = 0 \times 1 + 1 \times 1 = 1$$

$$(1, 0), z_{1in} = 1 \times 1 + 0 \times 1 = 1$$

$$(1, 1), z_{1in} = 1 \times 1 + 1 \times 1 = 2$$

Hence, it is not possible to obtain function z_1 using these weights.

Case 2: Assume one weight as excitatory and the other as inhibitory, i.e.,

$$w_{11} = 1; w_{21} = -1$$

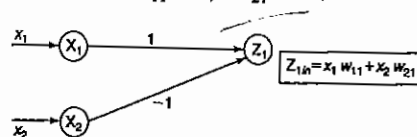


Figure 7 Neural net for Z_1 .

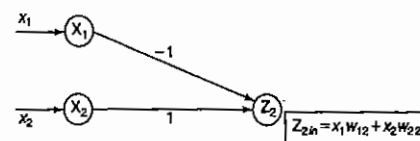


Figure 8 Neural net for Z_2 .

Calculate the net inputs. For inputs

$$(0, 0), z_{2in} = 0 \times 1 + 0 \times -1 = 0$$

$$(0, 1), z_{2in} = 0 \times 1 + 1 \times -1 = -1$$

$$(1, 0), z_{2in} = 1 \times 1 + 0 \times -1 = 1$$

$$(1, 1), z_{2in} = 1 \times 1 + 1 \times -1 = 0$$

On the basis of this calculated net input, it is possible to get the required output. Hence,

$$w_{11} = 1$$

$$w_{21} = -1$$

$$\theta \geq 1 \text{ for the } Z_1 \text{ neuron}$$

• Second function ($z_2 = \bar{x}_1 x_2$): The truth table for function z_2 is shown in Table 5.

Table 5

x_1	x_2	z_2
0	0	0
0	1	1
1	0	0
1	1	0

The net representation is given as follows:

Case 1: Assume both weights as excitatory, i.e.,

$$w_{12} = w_{22} = 1$$

Now calculate the net inputs. For the inputs

$$(0, 0), z_{2in} = 0 \times 1 + 0 \times 1 = 0$$

$$(0, 1), z_{2in} = 0 \times 1 + 1 \times 1 = 1$$

$$(1, 0), z_{2in} = 1 \times 1 + 0 \times 1 = 1$$

$$(1, 1), z_{2in} = 1 \times 1 + 1 \times 1 = 2$$

Hence, it is not possible to obtain function z_2 using these weights.

Case 2: Assume one weight as excitatory and the other as inhibitory, i.e.,

$$w_{12} = -1; w_{22} = 1$$

Now calculate the net inputs. For the inputs

$$(0, 0), z_{2in} = 0 \times -1 + 0 \times 1 = 0$$

$$(0, 1), z_{2in} = 0 \times -1 + 1 \times 1 = 1$$

$$(1, 0), z_{2in} = 1 \times -1 + 0 \times 1 = -1$$

$$(1, 1), z_{2in} = 1 \times -1 + 1 \times 1 = 0$$

Thus, based on this calculated net input, it is possible to get the required output.

$$w_{12} = -1$$

$$w_{22} = 1$$

$$\theta \geq 1 \text{ for the } Z_2 \text{ neuron}$$

• Third function ($y = z_1 \text{ OR } z_2$): The truth table for this function is shown in Table 6.

Table 6

x_1	x_2	y	z_1	z_2
0	0	0	0	0
0	1	1	0	1
1	0	1	1	0
1	1	0	0	0

Here the net input is calculated using

$$y_{in} = z_1 v_1 + z_2 v_2$$

Case 1: Assume both weights as excitatory, i.e.,

$$v_1 = v_2 = 1$$

Now calculate the net input. For inputs

$$(0, 0), y_{in} = 0 \times 1 + 0 \times 1 = 0$$

$$(0, 1), y_{in} = 0 \times 1 + 1 \times 1 = 1$$

$$(1, 0), y_{in} = 1 \times 1 + 0 \times 1 = 1$$

$$(1, 1), y_{in} = 0 \times 1 + 0 \times 1 = 0$$

(because for $x_1 = 1$ and $x_2 = 1, z_1 = 0$ and $z_2 = 0$)

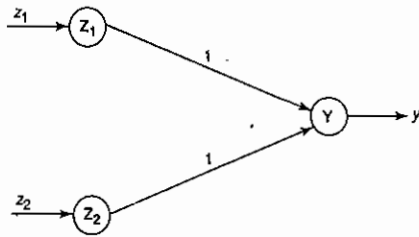


Figure 9 Neural net for $Y(Z_1 \text{ OR } Z_2)$.

Setting a threshold of $\theta \geq 1$, $v_1 = v_2 = 1$, which implies that the net is recognized. Therefore, the analysis is made for XOR function using M-P neurons. Thus for XOR function, the weights are obtained as

$$\begin{aligned} w_{11} &= w_{22} = 1 \quad (\text{excitatory}) \\ w_{12} &= w_{21} = -1 \quad (\text{inhibitory}) \\ v_1 &= v_2 = 1 \quad (\text{excitatory}) \end{aligned}$$

7. Using the linear separability concept, obtain the response for OR function (take bipolar inputs and bipolar targets).

Solution: Table 7 is the truth table for OR function with bipolar inputs and targets.

Table 7

x_1	x_2	y
1	1	1
1	-1	1
-1	1	1
-1	-1	-1

The truth table inputs and corresponding outputs have been plotted in Figure 10. If output is 1, it is denoted as "+" else "-". Assuming the coordinates as $(-1, 0)$ and $(0, -1)$; (x_1, y_1) and (x_2, y_2) , the slope "m" of the straight line can be obtained as

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{-1 - 0}{0 - (-1)} = \frac{-1}{1} = -1$$

We now calculate c:

$$c = y_1 - mx_1 = 0 - (-1)(-1) = -1$$

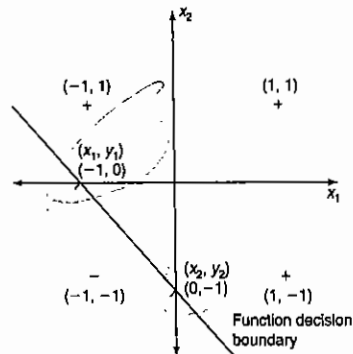


Figure 10 Graph for 'OR' function.

Using this value the equation for the line is given as

$$y = mx + c = (-1)x - 1 = -x - 1$$

Here the quadrants are not x and y but x_1 and x_2 , so the above equation becomes

$$x_2 = -x_1 - 1 \quad (2.1)$$

This can be written as

$$x_2 = \frac{-w_1}{w_2}x_1 - \frac{b}{w_2} \quad (2.2)$$

Comparing Eqs. (2.1) and (2.2), we get

$$\frac{w_1}{w_2} = 1; \quad \frac{b}{w_2} = 1$$

Therefore, $w_1 = 1$, $w_2 = 1$ and $b = 1$. Calculating the net input and output of OR function on the basis of these weights and bias, we get entries in Table 8.

Table 8

x_1	x_2	b	$y_{in} = b + x_1w_1 + x_2w_2$	y
1	1	1	3	1
1	-1	1	1	1
-1	1	1	1	1
-1	-1	1	-1	-1

Thus, the output of neuron Y can be written as

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq 1 \\ 0 & \text{if } y_{in} < 1 \end{cases}$$

where the threshold is taken as "1" ($\theta = 1$) based on the calculated net input. Hence, using the linear separability concept, the response is obtained for "OR" function.

8. Design a Hebb net to implement logical AND function (use bipolar inputs and targets).

Solution: The training data for the AND function is given in Table 9.

Table 9

Inputs			Target
x_1	x_2	b	y
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1

The network is trained using the Hebb network training algorithm discussed in Section 2.7.3. Initially the weights and bias are set to zero, i.e.,

$$w_1 = w_2 = b = 0 \quad (\text{initial})$$

- First input $[x_1 \ x_2 \ b] = [1 \ 1 \ 1]$ and target = 1 [i.e., $y = 1$]: Setting the initial weights as old weights and applying the Hebb rule, we get

$$w_i(\text{new}) = w_i(\text{old}) + x_i y$$

$$w_1(\text{new}) = w_1(\text{old}) + x_1 y = 0 + 1 \times 1 = 1$$

$$w_2(\text{new}) = w_2(\text{old}) + x_2 y = 0 + 1 \times 1 = 1$$

$$b(\text{new}) = b(\text{old}) + y = 0 + 1 = 1$$

The weights calculated above are the final weights that are obtained after presenting the first input. These weights are used as the initial weights when the second input pattern is presented. The weight change here is $\Delta w_i = x_i y$. Hence weight changes relating to the first input are

$$\Delta w_1 = x_1 y = 1 \times 1 = 1$$

$$\Delta w_2 = x_2 y = 1 \times 1 = 1$$

$$\Delta b = y = 1$$

- Second input $[x_1 \ x_2 \ b] = [1 \ -1 \ 1]$ and $y = -1$: The initial or old weights here are the

final (new) weights obtained by presenting the first input pattern, i.e.,

$$[w_1 \ w_2 \ b] = [1 \ 1 \ 1]$$

The weight change here is

$$\Delta w_1 = x_1 y = 1 \times -1 = -1$$

$$\Delta w_2 = x_2 y = -1 \times -1 = 1$$

$$\Delta b = y = -1$$

The new weights here are

$$w_1(\text{new}) = w_1(\text{old}) + \Delta w_1 = 1 - 1 = 0$$

$$w_2(\text{new}) = w_2(\text{old}) + \Delta w_2 = 1 + 1 = 2$$

$$b(\text{new}) = b(\text{old}) + \Delta b = 1 - 1 = 0$$

Similarly, by presenting the third and fourth input patterns, the new weights can be calculated. Table 10 shows the values of weights for all inputs.

Table 10

Inputs			Weight changes				Weights		
x_1	x_2	b	y	Δw_1	Δw_2	Δb	w_1	w_2	b
							(0	0	0)
1	1	1	1	1	1	1	1	1	1
1	-1	1	-1	-1	1	-1	0	2	0
-1	1	1	-1	1	-1	-1	1	1	-1
-1	-1	1	-1	1	1	-1	2	2	-2

The separating line equation is given by

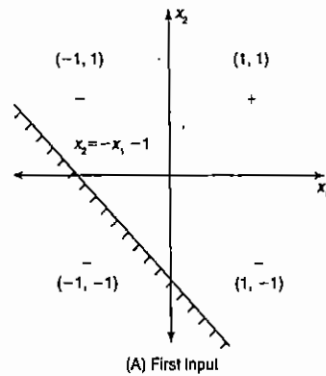
$$x_2 = \frac{-w_1}{w_2}x_1 - \frac{b}{w_2}$$

For all inputs, use the final weights obtained for each input to obtain the separating line. For the first input $[1 \ 1 \ 1]$, the separating line is given by

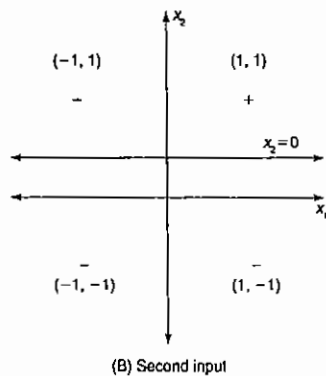
$$x_2 = \frac{-1}{1}x_1 - \frac{1}{1} \Rightarrow x_2 = -x_1 - 1$$

Similarly, for the second input $[1 \ -1 \ 1]$, the separating line is

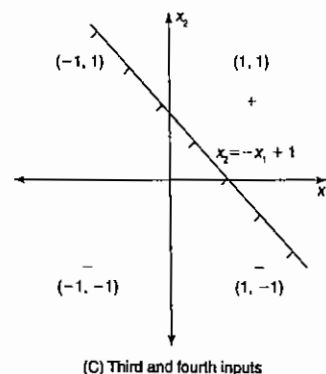
$$x_2 = \frac{-0}{2}x_1 - \frac{0}{2} \Rightarrow x_2 = 0$$



(A) First Input



(B) Second input



(C) Third and fourth inputs

Figure 11 Decision boundary for AND function using Hebb rule for each training pair.

For the third input $[-1 \ 1 \ 1]$, it is

$$x_2 = \frac{-1}{1}x_1 + \frac{1}{1} \Rightarrow x_2 = -x_1 + 1$$

Finally, for the fourth input $[-1 \ -1 \ 1]$, the separating line is

$$x_2 = \frac{-2}{2}x_1 + \frac{2}{2} \Rightarrow x_2 = -x_1 + 1$$

The graphs for each of these separating lines obtained are shown in Figure 11. In this figure "+" mark is used for output "1" and "-" mark is used for output "-1". From Figure 11, it can be noticed that for the first input, the decision boundary differentiates only the first and fourth inputs, and not all negative responses are separated from positive responses. When the second input pattern is presented, the decision boundary separates (1, 1) from (1, -1) and (-1, -1) and not (-1, 1). But the boundary line is same for the both third and fourth training pairs. And, the decision boundary line obtained from these input training pairs separates the positive response region from the negative response region. Hence, the weights obtained from this are the final weights and are given as

$$w_1 = 2; \quad w_2 = 2; \quad b = -2$$

The network can be represented as shown in Figure 12.

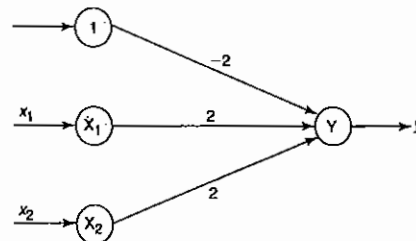


Figure 12 Hebb net for AND function.

9. Design a Hebb net to implement OR function (consider bipolar inputs and targets).

Solution: The training pair for the OR function is given in Table 11.

Table 11

Inputs			Target
x_1	x_2	b	y
1	1	1	1
1	-1	1	1
-1	1	1	1
-1	-1	1	-1

Initially the weights and bias are set to zero, i.e.,

$$w_1 = w_2 = b = 0$$

The network is trained and the final weights are outlined using the Hebb training algorithm discussed in Section 2.7.3. The weights are considered as final weights if the boundary line obtained from these weights separates the positive response region and negative response region.

By presenting all the input patterns, the weights are calculated. Table 12 shows the weights calculated for all the inputs.

Table 12

Inputs				Weight changes			Weights		
x_1	x_2	b	y	Δw_1	Δw_2	Δb	w_1	w_2	b
1	1	1	1	1	1	1	1	1	1
1	-1	1	1	1	-1	1	2	0	2
-1	1	1	1	-1	1	1	1	1	3
-1	-1	1	-1	1	1	-1	2	2	2

Using the final weights, the boundary line equation can be obtained. The separating line equation is

$$x_2 = \frac{-w_1}{w_2}x_1 - \frac{b}{w_2} = \frac{-2}{2}x_1 - \frac{2}{2} = -x_1 - 1$$

The decision region for this net is shown in Figure 13. It is observed in Figure 13 that straight line $x_2 = -x_1 - 1$ separates the pattern space into two regions. The input patterns $\{(1, 1), (1, -1), (-1, 1)\}$ for which the output response is "1" lie on one side of the boundary, and the input pattern $(-1, -1)$ for which

the output response is "-1" lies on the other side of the boundary. Thus, the final weights are

$$w_1 = 2; \quad w_2 = 2; \quad b = 2$$

The network can be represented as shown in Figure 14.

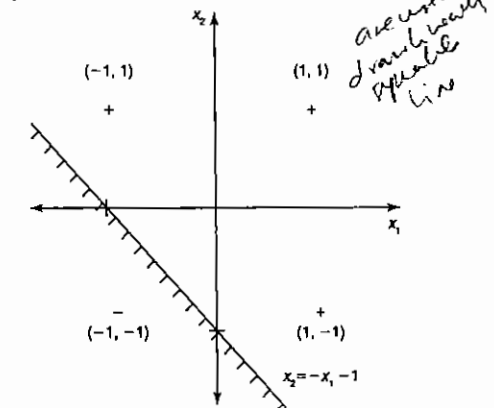


Figure 13 Decision boundary for OR function.

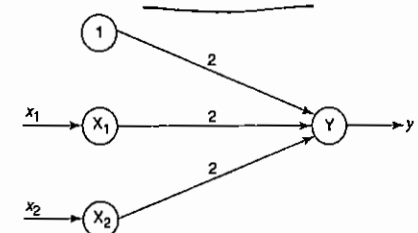


Figure 14 Hebb net for OR function.

10. Use the Hebb rule method to implement XOR function (take bipolar inputs and targets).

Solution: The training patterns for an XOR function are shown in Table 13.

Table 13

Inputs			Target
x_1	x_2	b	y
1	1	1	-1
1	-1	1	1
-1	1	1	1
-1	-1	1	-1

Here, a single-layer network with two input neurons, one bias and one output neuron is considered. In this case also, the initial weights are assumed to be zero:

$$w_1 = w_2 = b = 0$$

By using the Hebb training algorithm, the network is trained and the final weights are calculated as shown in the following Table 14.

Table 14

Inputs				Weight changes			Weights		
x_1	x_2	b	y	Δw_1	Δw_2	Δb	w_1	w_2	b
							0	0	0
1	1	-1	-1	-1	-1	-1	-1	-1	-1
1	-1	1	1	1	-1	1	0	-2	0
-1	1	1	1	-1	1	1	-1	-1	1
-1	-1	1	-1	1	1	-1	0	0	0

The final weights obtained after presenting all the input patterns do not give correct output for all patterns. Figure 15 shows that the input patterns are linearly non-separable. The graph shown in Figure 15 indicates that the four input pairs that are present cannot be divided by a single line to separate them into two regions. Thus XOR function is a case of a pattern classification problem, which is not linearly separable.

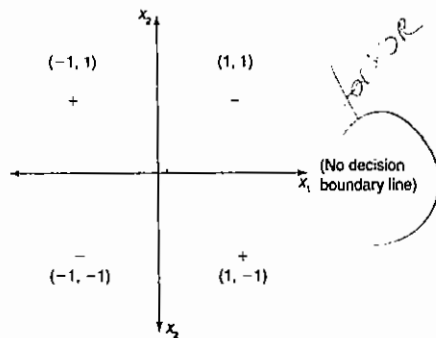


Figure 15 Decision boundary for XOR function.

The XOR function can be made linearly separable by solving it in a manner as discussed in Problem 6. This method of solving will result in two decision boundary lines for separating positive and negative regions of XOR function.

11. Using the Hebb rule, find the weights required to perform the following classifications of the given input patterns shown in Figure 16. The pattern is shown as 3×3 matrix form in the squares. The "+" symbols represent the value "1" and empty squares indicate "-1." Consider "I" belongs to the members of class (so has target value 1) and "O" does not belong to the members of class (so has target value -1).

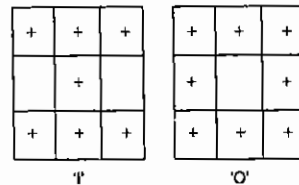


Figure 16 Data for input patterns.

Solution: The training input patterns for the given net (Figure 16) are indicated in Table 15.

Table 15

Pattern	Inputs									Target
	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	y
I	1	1	1	-1	1	-1	1	1	-1	1
O	1	1	1	1	-1	1	1	1	1	-1

Here a single-layer network with nine input neurons, one bias and one output neuron is formed. Set the initial weights and bias to zero, i.e.,

$$w_1 = w_2 = w_3 = w_4 = w_5 \\ = w_6 = w_7 = w_8 = w_9 = b = 0$$

Case 1: Presenting first input pattern (I), we calculate change in weights:

$$\Delta w_i = x_i y, \quad i = 1 \text{ to } 9 \\ \Delta w_1 = x_1 y = 1 \times 1 = 1$$

$$\Delta w_2 = x_2 y = 1 \times 1 = 1$$

$$\Delta w_3 = x_3 y = 1 \times 1 = 1$$

$$\Delta w_4 = x_4 y = -1 \times 1 = -1$$

$$\Delta w_5 = x_5 y = 1 \times 1 = 1$$

$$\Delta w_6 = x_6 y = -1 \times 1 = -1$$

$$\Delta w_7 = x_7 y = 1 \times 1 = 1$$

$$\Delta w_8 = x_8 y = 1 \times 1 = 1$$

$$\Delta w_9 = x_9 y = 1 \times 1 = 1$$

$$\Delta b = y = 1$$

$$w_3(\text{new}) = w_3(\text{old}) + x_3 y = 1 + 1 \times 1 = 0$$

$$w_4(\text{new}) = w_4(\text{old}) + x_4 y = -1 + 1 \times -1 = -2$$

$$w_5(\text{new}) = w_5(\text{old}) + x_5 y = 1 + -1 \times -1 = 2$$

$$w_6(\text{new}) = w_6(\text{old}) + x_6 y = -1 + 1 \times -1 = -2$$

$$w_7(\text{new}) = w_7(\text{old}) + x_7 y = 1 + 1 \times -1 = 0$$

$$w_8(\text{new}) = w_8(\text{old}) + x_8 y = 1 + 1 \times -1 = 0$$

$$w_9(\text{new}) = w_9(\text{old}) + x_9 y = 1 + 1 \times -1 = 0$$

$$b(\text{new}) = b(\text{old}) + y = 1 + 1 \times -1 = 0$$

The final weights after presenting the second input pattern are given as

$$W_{(\text{new})} = [0 \ 0 \ 0 \ -2 \ 2 \ -2 \ 0 \ 0 \ 0]$$

We now calculate the new weights using the formula

$$w_i(\text{new}) = w_i(\text{old}) + \Delta w_i$$

Setting the old weights as the initial weights here, we obtain

$$w_1(\text{new}) = w_1(\text{old}) + \Delta w_1 = 0 + 1 = 1$$

$$w_2(\text{new}) = w_2(\text{old}) + \Delta w_2 = 0 + 1 = 1$$

$$w_3(\text{new}) = w_3(\text{old}) + \Delta w_3 = 0 + 1 = 1$$

Similarly, calculating for other weights we get

$$w_4(\text{new}) = -1, \quad w_5(\text{new}) = 1, \quad w_6(\text{new}) = -1,$$

$$w_7(\text{new}) = 1, \quad w_8(\text{new}) = 1, \quad w_9(\text{new}) = 1,$$

$$b(\text{new}) = 1$$

The weights after presenting first input pattern are

$$W_{(\text{new})} = [1 \ 1 \ 1 \ -1 \ 1 \ -1 \ 1 \ 1 \ 1]$$

Case 2: Now we present the second input pattern (O). The initial weights used here are the final weights obtained after presenting the first input pattern. Here, the weights are calculated as shown below ($y = -1$ with the initial weights being $[1 \ 1 \ 1 \ -1 \ 1 \ -1 \ 1 \ 1 \ 1]$).

$$w_i(\text{new}) = w_i(\text{old}) + \Delta x_i \quad [\Delta w_i = x_i y]$$

$$w_1(\text{new}) = w_1(\text{old}) + x_1 y = 1 + 1 \times -1 = 0$$

$$w_2(\text{new}) = w_2(\text{old}) + x_2 y = 1 + 1 \times -1 = 0$$

A single-layer network with nine input neurons, one bias and one output neuron is formed. Set the initial weights and bias to zero, i.e.,

$$w_1 = w_2 = w_3 = w_4 = w_5 \\ = w_6 = w_7 = w_8 = w_9 = b = 0$$

The weights are calculated using

$$w_i(\text{new}) = w_i(\text{old}) + x_i y$$

The weights obtained are indicated in the Hebb net shown in Figure 17.

12. Find the weights required to perform the following classifications of given input patterns using the Hebb rule. The inputs are "1" where "+" symbol is present and "-1" where "-" is present. "L" pattern belongs to the class (target value +1) and "U" pattern does not belong to the class (target value -1).

Solution: The training input patterns for Figure 18 are given in Table 16.

Table 16

Pattern	Inputs									Target
	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	y
L	1	-1	-1	1	-1	-1	1	1	1	1
U	1	-1	1	1	-1	1	1	1	1	-1

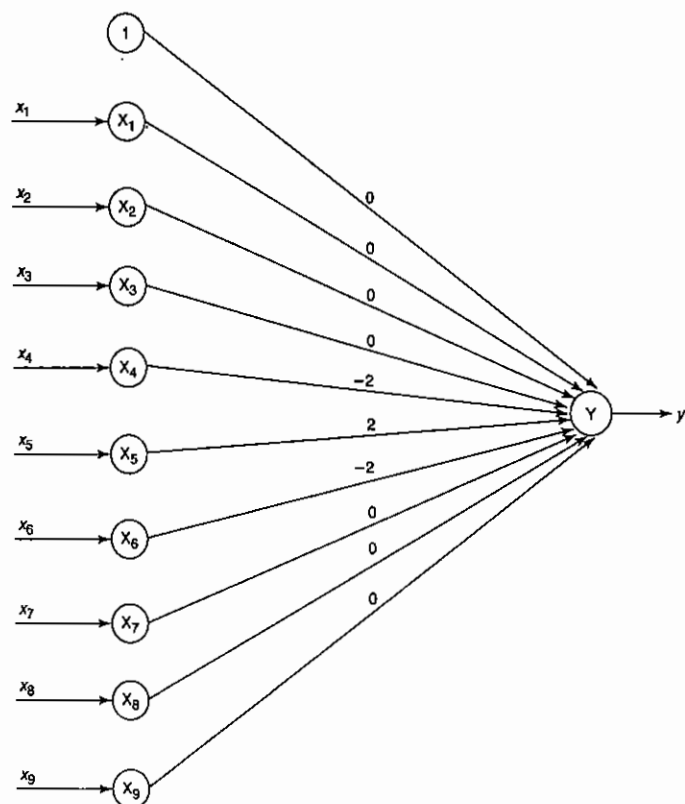


Figure 17 Hebb net for the data matrix shown in Figure 16.

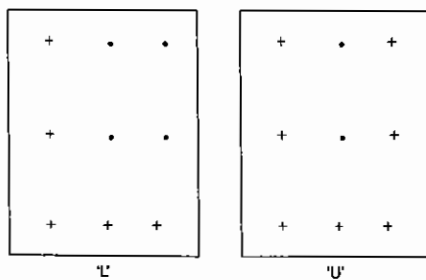


Figure 18 Input data for given patterns.

The calculated weights are given in Table 17.

Table 17

Inputs										Target	Weights									
x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	b	y	w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8	w_9	b
											(0	0	0	0	0	0	0	0	0	0)
1	-1	-1	1	-1	-1	1	1	1	1	1	1	-1	-1	1	-1	-1	1	1	1	1
1	-1	1	1	-1	1	1	1	1	1	-1	0	0	-2	0	0	-2	0	0	0	0

The final weights after presenting the two input patterns are

The obtained weights are indicated in the Hebb net shown in Figure 19.

$$W_{\text{(new)}} = [0 \ 0 \ -2 \ 0 \ 0 \ -2 \ 0 \ 0 \ 0]$$

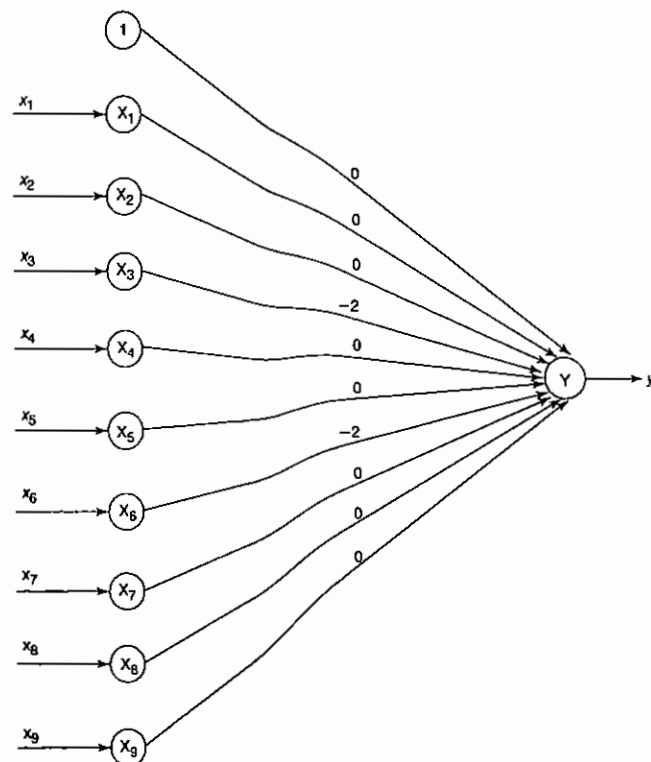


Figure 19 Hebb net of Figure 18.

2.10 Review Questions

1. Define an artificial neural network.
2. State the properties of the processing element of an artificial neural network.
3. How many signals can be sent by a neuron at a particular time instant?
4. Draw a simple artificial neuron and discuss the calculation of net input.
5. What is the influence of a linear equation over the net input calculation?
6. List the main components of the biological neuron.
7. Compare and contrast biological neuron and artificial neuron.
8. State the characteristics of an artificial neural network.
9. Discuss in detail the historical development of artificial neural networks.
10. What are the basic models of an artificial neural network?
11. Define net architecture and give its classifications.
12. Define learning.
13. Differentiate between supervised and unsupervised learning.
14. How is the critic information used in the learning process?
15. What is the necessity of activation function?
16. List the commonly used activation functions.
17. What is the impact of weight in an artificial neural network?
18. What is the other name for weight?
19. Define bias and threshold.
20. What is a learning rate parameter?
21. How does a momentum factor make faster convergence of a network?
22. State the role of vigilance parameter in ART network.
23. Why is the McCulloch-Pitts neuron widely used in logic functions?
24. Indicate the difference between excitatory and inhibitory weighted interconnections.
25. Define linear separability.
26. Justify - XOR function is non-linearly separable by a single decision boundary line.
27. How can the equation of a straight line be formed using linear separability?
28. In what ways is bipolar representation better than binary representation?
29. State the training algorithm used for the Hebb network.
30. Compare feed-forward and feedback network.

2.11 Exercise Problems

1. For the network shown in Figure 20, calculate the net input to the output neuron.

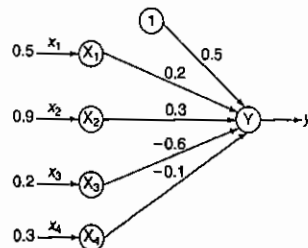


Figure 20 Neural net.

2. Calculate the output of neuron Y for the net shown in Figure 21. Use binary and bipolar sigmoidal activation functions.

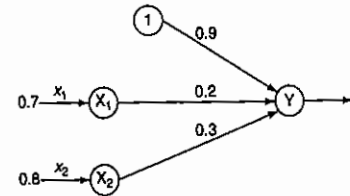


Figure 21 Neural net.

3. Design neural networks with only one M-P neuron that implements the three basic logic operations:

- (i) NOT (x_1);
- (ii) OR (x_1, x_2);
- (iii) NAND (x_1, x_2), where x_1 and $x_2 \in \{0, 1\}$.

4. (a) Show that the derivative of unipolar sigmoidal function is

$$f'(x) = \lambda f(x)[1 - f(x)]$$

- (b) Show that the derivative of bipolar sigmoidal function is

$$f'(x) = \frac{\lambda}{2} [1 + f(x)][1 - f(x)]$$

5. (a) Construct a feed-forward network with five input nodes, three hidden nodes and four output nodes that has lateral inhibition structure in the output layer.

- (b) Construct a recurrent network with four input nodes, three hidden nodes and two output nodes that has feedback links from the hidden layer to the input layer.

6. Using linear separability concept, obtain the response for NAND function.

7. Design a Hebb net to implement logical AND function with

- (a) binary inputs and targets and
- (b) binary inputs and bipolar targets.

8. Implement NOR function using Hebb net with

- (a) bipolar inputs and targets and
- (b) bipolar inputs and binary targets.

9. Classify the input patterns shown in Figure 22 using Hebb training algorithm.

+	+	+	+	+	+	+
+	.	+	+	+	.	.
+	+	+	+	+	+	+
+	.	+	+	+	.	.
+	.	+	+	+	+	+
Target value +1				Target value -1		

Figure 22 Input pattern.

10. Using Hebb rule, find the weights required to perform following classifications. The vectors (1 -1 1 -1) and (1 1 1 -1) belong to class (target value +1); vectors (-1 -1 1 1) and (1 1 -1 -1) do not belong to class (target value -1). Also using each of training x vectors as input, test the response of net.

2.12 Projects

1. Write a program to classify the letters and numerals using Hebb learning rule. Take a pair of letters or numerals of your own. Also, after training the network, test the response of the net using suitable activation function. Perform the classification using bipolar data as well as binary data.
2. Write suitable programs for implementing logic functions using McCulloch-Pitts neuron.
3. Write a computer program to train a Madaline to perform AND function, using MRI algorithm.
4. Write a program for implementing BPN for training a single-hidden-layer back-propagation

network with bipolar sigmoidal units ($\lambda = 1$) to achieve the following two-to-one mappings:

- $y = 6 \sin(\pi x_1) + \cos(\pi x_2)$
- $y = \sin(\pi x_1) \cos(0.2\pi x_2)$

Set up two sets of data, each consisting of 10 input-output pairs, one for training and other for

testing. The input-output data are obtained by varying input variables (x_1, x_2) within $[-1, +1]$ randomly. Also the output data are normalized within $[-1, 1]$. Apply training to find proper weights in the network.

Omsa's

3

Supervised Learning Network

Learning Objectives

- The basic networks in supervised learning. Adaline, Madaline, back-propagation and radial basis function network.
- How the perceptron learning rule is better than the Hebb rule.
- Original perceptron layer description.
- Delta rule with single output unit.
- Architecture, flowchart, training algorithm and testing algorithm for perceptron.
- The various learning factors used in BPN.
- An overview of Time Delay, Function Link, Wavelet and Tree Neural Networks.
- Difference between back-propagation and RBF networks.

3.1 Introduction

The chapter covers major topics involving supervised learning networks and their associated single-layer and multilayer feed-forward networks. The following topics have been discussed in detail – the perceptron learning rule for simple perceptrons, the delta rule (Widrow-Hoff rule) for Adaline and single-layer feed-forward networks with continuous activation functions, and the back-propagation algorithm for multilayer feed-forward networks with continuous activation functions. In short, all the feed-forward networks have been explored.

3.2 Perceptron Networks

3.2.1 Theory

Perceptron networks come under single-layer feed-forward networks and are also called *simple perceptrons*. As described in Table 2-2 (Evolution of Neural Networks) in Chapter 2, various types of perceptrons were designed by Rosenblatt (1962) and Minsky-Papert (1969, 1988). However, a simple perceptron network was discovered by Block in 1962.

The key points to be noted in a perceptron network are:

1. The perceptron network consists of three units, namely, sensory unit (input unit), associator unit (hidden unit), response unit (output unit).

- The sensory units are connected to associator units with fixed weights having values 1, 0 or -1, which are assigned at random.
- The binary activation function is used in sensory unit and associator unit.
- The response unit has an activation of 1, 0 or -1. The binary step with fixed threshold θ is used as activation for associator. The output signals that are sent from the associator unit to the response unit are only binary.
- The output of the perceptron network is given by

$$y = f(y_{in})$$

where $f(y_{in})$ is activation function and is defined as

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

- The perceptron learning rule is used in the weight updation between the associator unit and the response unit. For each training input, the net will calculate the response and it will determine whether or not an error has occurred.
- The error calculation is based on the comparison of the values of targets with those of the calculated outputs.
- The weights on the connections from the units that send the nonzero signal will get adjusted suitably.
- The weights will be adjusted on the basis of the learning rule if an error has occurred for a particular training pattern, i.e.,

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i$$

$$b(\text{new}) = b(\text{old}) + \alpha t$$

If no error occurs, there is no weight updation and hence the training process may be stopped. In the above equations, the target value " t " is +1 or -1 and α is the learning rate. In general, these learning rules begin with an initial guess at the weight values and then successive adjustments are made on the basis of the evaluation of an objective function. Eventually, the learning rules reach a near-optimal or optimal solution in a finite number of steps.

A perceptron network with its three units is shown in Figure 3-1. As shown in Figure 3-1, a sensory unit can be a two-dimensional matrix of 400 photodetectors upon which a lighted picture with geometric black and white pattern impinges. These detectors provide a binary (0) electrical signal if the input signal is found to exceed a certain value of threshold. Also, these detectors are connected randomly with the associator unit. The associator unit is found to consist of a set of subcircuits called *feature predicates*. The feature predicates are hard-wired to detect the specific feature of a pattern and are equivalent to the *feature detectors*. For a particular feature, each predicate is examined with a few or all of the responses of the sensory unit. It can be found that the results from the predicate units are also binary (0 or 1). The last unit, i.e. response unit, contains the pattern-recognizers or perceptrons. The weights present in the input layers are all fixed, while the weights on the response unit are trainable.

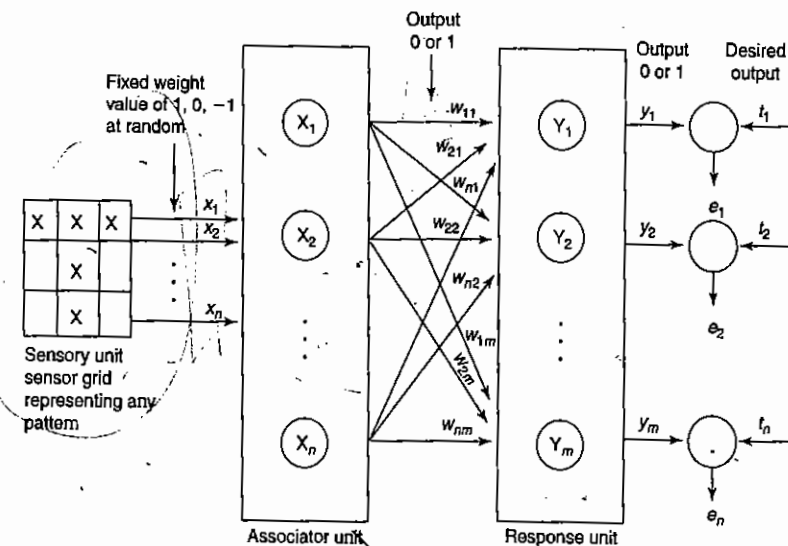


Figure 3-1 Original perceptron network.

binary step with θ is used as activation.

3.2.2 Perceptron Learning Rule

In case of the perceptron learning rule, the learning signal is the difference between the desired and actual response of a neuron. The perceptron learning rule is explained as follows:

Consider a finite " n " number of input training vectors, with their associated target (desired) values $x(n)$ and $t(n)$, where " n " ranges from 1 to N . The target is either +1 or -1. The output " y " is obtained on the basis of the net input calculated and activation function being applied over the net input.

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

The weight updation in case of perceptron learning is as shown.

If $y \neq t$, then

$$w(\text{new}) = w(\text{old}) + \alpha t x \quad (\alpha - \text{learning rate})$$

else, we have

$$w(\text{new}) = w(\text{old})$$

The weights can be initialized at any values in this method. The perceptron rule convergence theorem states that "If there is a weight vector W such that $f(x(n)W) = t(n)$, for all n , then for any starting vector w_1 , the perceptron learning rule will converge to a weight vector that gives the correct response for all

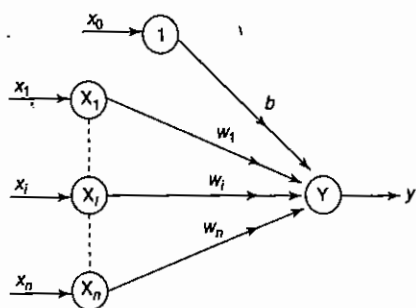


Figure 3-2 Single classification perceptron network.

training patterns, and this learning takes place within a finite number of steps provided that the solution exists."

3.2.3 Architecture

In the original perceptron network, the output obtained from the associator unit is a binary vector, and hence that output can be taken as input signal to the response unit, and classification can be performed. Here only the weights between the associator unit and the output unit can be adjusted, and the weights between the sensory and associator units are fixed. As a result, the discussion of the network is limited to a single portion. Thus, the associator unit behaves like the input unit. A simple perceptron network architecture is shown in Figure 3-2.

In Figure 3-2, there are n input neurons, 1 output neuron and a bias. The input-layer and output-layer neurons are connected through a directed communication link, which is associated with weights. The goal of the perceptron net is to classify the input pattern as a member or not a member to a particular class.

Goal: classify input pattern as a member or not

3.2.4 Flowchart for Training Process

The flowchart for the perceptron network training is shown in Figure 3-3. The network has to be suitably trained to obtain the response. The flowchart depicted here presents the flow of the training process.

As depicted in the flowchart, first the basic initialization required for the training process is performed. The entire loop of the training process continues until the training input pair is presented to the network. The training (weight updation) is done on the basis of the comparison between the calculated and desired output. The loop is terminated if there is no change in weight.

3.2.5 Perceptron Training Algorithm for Single Output Classes

The perceptron algorithm can be used for either binary or bipolar input vectors, having bipolar targets, threshold being fixed and variable bias. The algorithm discussed in this section is not particularly sensitive to the initial values of the weights or the value of the learning rate. In the algorithm discussed below, initially the inputs are assigned. Then the net input is calculated. The output of the network is obtained by applying the activation function over the calculated net input. On performing comparison over the calculated and

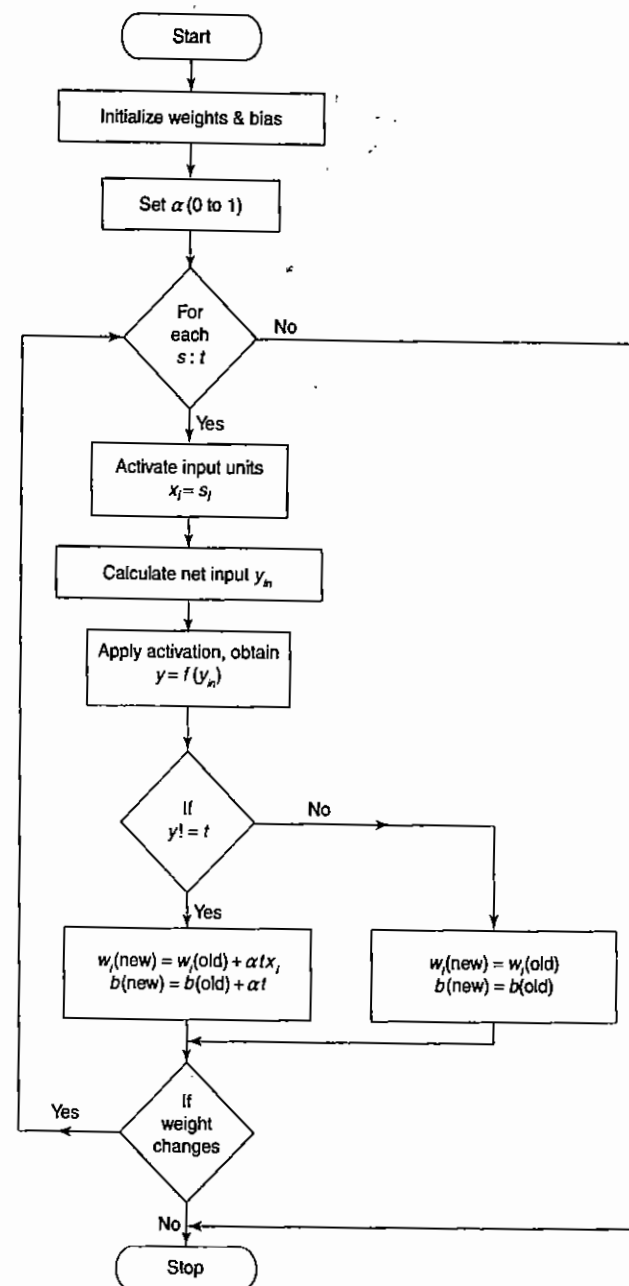


Figure 3-3 Flowchart for perceptron network with single output.

the desired output, the weight updation process is carried out. The entire network is trained based on the mentioned stopping criterion. The algorithm of a perceptron network is as follows:

Step 0: Initialize the weights and the bias (for easy calculation they can be set to zero). Also initialize the learning rate $\alpha (0 < \alpha \leq 1)$. For simplicity α is set to 1.

Step 1: Perform Steps 2–6 until the final stopping condition is false.

Step 2: Perform Steps 3–5 for each training pair indicated by $s:t$.

Step 3: The input layer containing input units is applied with identity activation functions:

$$x_i = s_i$$

Step 4: Calculate the output of the network. To do so, first obtain the net input:

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

where “ n ” is the number of input neurons in the input layer. Then apply activations over the net input calculated to obtain the output:

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

Step 5: Weight and bias adjustment: Compare the value of the actual (calculated) output and desired (target) output.

If $y \neq t$, then

$$w_i(\text{new}) = w_i(\text{old}) + \alpha t x_i$$

$$b(\text{new}) = b(\text{old}) + \alpha t$$

else, we have

$$w_i(\text{new}) = w_i(\text{old})$$

$$b(\text{new}) = b(\text{old})$$

Step 6: Train the network until there is no weight change. This is the stopping condition for the network. If this condition is not met, then start again from Step 2.

The algorithm discussed above is not sensitive to the initial values of the weights or the value of the learning rate.

3.2.6 Perceptron Training Algorithm for Multiple Output Classes

For multiple output classes, the perceptron training algorithm is as follows:

Step 0: Initialize the weights, biases and learning rate suitably.

Step 1: Check for stopping condition; if it is false, perform Steps 2–6.

Step 2: Perform Steps 3–5 for each bipolar or binary training vector pair $s:t$.

Step 3: Set activation (identity) of each input unit $i = 1$ to n :

Step 4: Calculate output response of each output unit $j = 1$ to m . First, the net input is calculated as

$$y_{inj} = b_j + \sum_{i=1}^n x_i w_{ij}$$

each input pattern response is calculated (in turn).

Then activations are applied over the net input to calculate the output response:

$$y_j = f(y_{inj}) = \begin{cases} 1 & \text{if } y_{inj} > \theta \\ 0 & \text{if } -\theta \leq y_{inj} \leq \theta \\ -1 & \text{if } y_{inj} < -\theta \end{cases}$$

Step 5: Make adjustment in weights and bias for $j = 1$ to m and $i = 1$ to n .

If $t_j \neq y_j$, then

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha t_j x_i$$

$$b_j(\text{new}) = b_j(\text{old}) + \alpha t_j$$

else, we have

$$w_{ij}(\text{new}) = w_{ij}(\text{old})$$

$$b_j(\text{new}) = b_j(\text{old})$$

Step 6: Test for the stopping condition, i.e., if there is no change in weights then stop the training process, else start again from Step 2.

It can be noticed that after training, the net classifies each of the training vectors. The above algorithm is suited for the architecture shown in Figure 3-4.

3.2.7 Perceptron Network Testing Algorithm

It is best to test the network performance once the training process is complete. For efficient performance of the network, it should be trained with more data. The testing algorithm (application procedure) is as follows:

Step 0: The initial weights to be used here are taken from the training algorithms (the final weights obtained during training).

Step 1: For each input vector X to be classified, perform Steps 2–3.

Step 2: Set activations of the input unit.

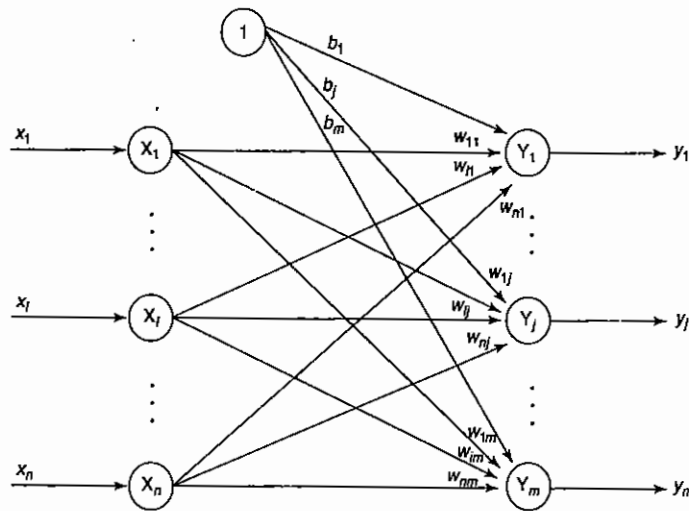


Figure 3-4 Network architecture for perceptron network for several output classes.

Step 3: Obtain the response of output unit.

$$y_{in} = \sum_{i=1}^n x_i w_i$$

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > \theta \\ 0 & \text{if } -\theta \leq y_{in} \leq \theta \\ -1 & \text{if } y_{in} < -\theta \end{cases}$$

Thus, the testing algorithm tests the performance of network.

Note: In the case of perceptron network, it can be used for linear separability concept. Here the separating line may be based on the value of threshold, i.e., the threshold used in activation function must be a non-negative value.

The condition for separating the response from region of positive to region of zero is

$$w_1 x_1 + w_2 x_2 + b > \theta$$

The condition for separating the response from region of zero to region of negative is

$$w_1 x_1 + w_2 x_2 + b < -\theta$$

The conditions above are stated for a single-layer perceptron network with two input neurons and one output neuron and one bias.

3.3 Adaptive Linear Neuron (Adaline)

3.3.1 Theory

The units with linear activation function are called linear units. A network with a single linear unit is called an *Adaline* (adaptive linear neuron). That is, in an Adaline, the input-output relationship is linear. Adaline uses bipolar activation for its input signals and its target output. The weights between the input and the output are adjustable. The bias in Adaline acts like an adjustable weight, whose connection is from a unit with activations being always 1. Adaline is a net which has only one output unit. The Adaline network may be trained using delta rule. The delta rule may also be called as *least mean square* (LMS) rule or Widrow-Hoff rule. This learning rule is found to minimize the mean-squared error between the activation and the target value.

3.3.2 Delta Rule for Single Output Unit

The Widrow-Hoff rule is very similar to perceptron learning rule. However, their origins are different. The perceptron learning rule originates from the Hebbian assumption while the delta rule is derived from the gradient-descent method (it can be generalized to more than one layer). Also, the perceptron learning rule stops after a finite number of learning steps, but the gradient-descent approach continues forever, converging only asymptotically to the solution. The delta rule updates the weights between the connections so as to minimize the difference between the net input to the output unit and the target value. The major aim is to minimize the error over all training patterns. This is done by reducing the error for each pattern, one at a time.

The delta rule for adjusting the weight of i th pattern ($i = 1$ to n) is

$$\Delta w_i = \alpha (t - y_{in}) x_i$$

where Δw_i is the weight change; α the learning rate; x the vector of activation of input unit; y_{in} the net input to output unit, i.e., $Y = \sum_{i=1}^n x_i w_i$; t the target output. The delta rule in case of several output units for adjusting the weight from i th input unit to the j th output unit (for each pattern) is

$$\Delta w_{ij} = \alpha (t_j - y_{mj}) x_i$$

3.3.3 Architecture

As already stated, Adaline is a single-unit neuron, which receives input from several units and also from one unit called bias. An Adaline model is shown in Figure 3-5. The basic Adaline model consists of trainable weights. Inputs are either of the two values (+1 or -1) and the weights have signs (positive or negative). Initially, random weights are assigned. The net input calculated is applied to a quantizer transfer function (possibly activation function) that restores the output to +1 or -1. The Adaline model compares the actual output with the target output and on the basis of the training algorithm, the weights are adjusted.

3.3.4 Flowchart for Training Process

The flowchart for the training process is shown in Figure 3-6. This gives a pictorial representation of the network training. The conditions necessary for weight adjustments have to be checked carefully. The weights and other required parameters are initialized. Then the net input is calculated, output is obtained and compared with the desired output for calculation of error. On the basis of the error factor, weights are adjusted.

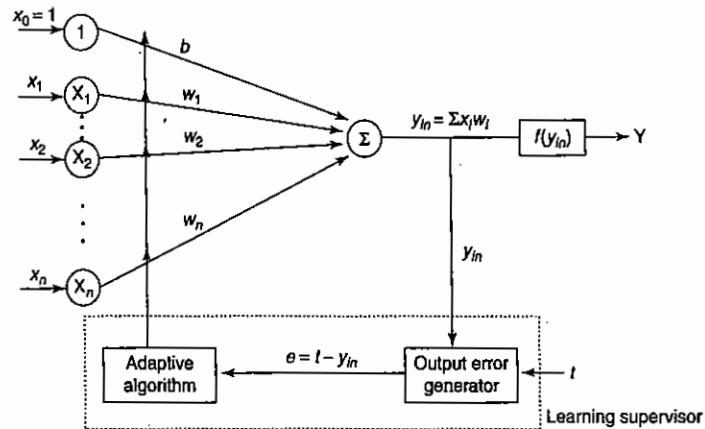


Figure 3-5 Adaline model.

3.3.5 Training Algorithm

The Adaline network training algorithm is as follows:

Step 0: Weights and bias are set to some random values but not zero. Set the learning rate parameter α .

Step 1: Perform Steps 2–6 when stopping condition is false.

Step 2: Perform Steps 3–5 for each bipolar training pair s, t .

Step 3: Set activations for input units $i = 1$ to n .

$$x_i = s_i$$

Step 4: Calculate the net input to the output unit.

$$y_{in} = b + \sum_{i=1}^n x_i w_i$$

Step 5: Update the weights and bias for $i = 1$ to n :

$$\begin{aligned} w_i(\text{new}) &= w_i(\text{old}) + \alpha(t - y_{in})x_i \\ b(\text{new}) &= b(\text{old}) + \alpha(t - y_{in}) \end{aligned}$$

Step 6: If the highest weight change that occurred during training is smaller than a specified tolerance then stop the training process, else continue. This is the test for stopping condition of a network.

The range of learning rate can be between 0.1 and 1.0.

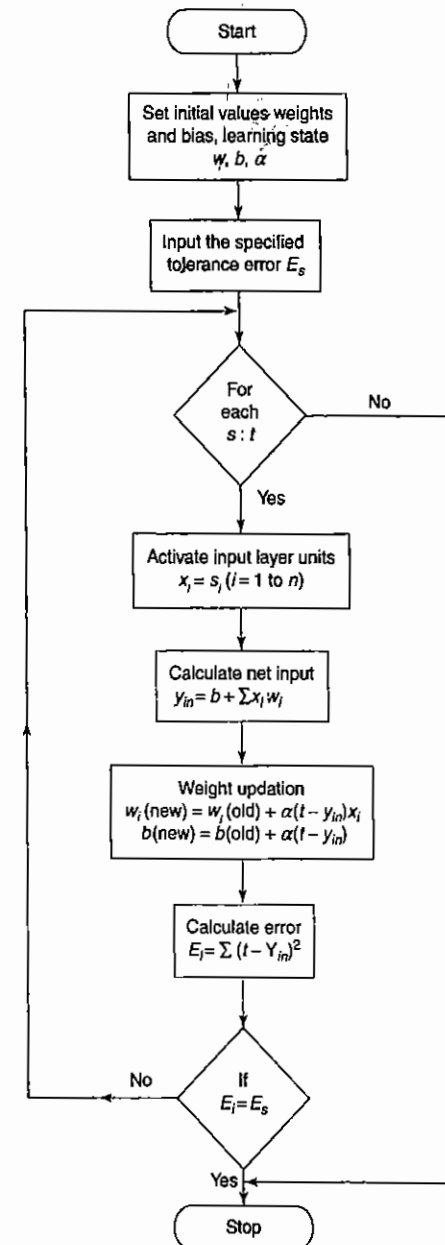


Figure 3-6 Flowchart for Adaline training process.

3.3.6 Testing Algorithm

It is essential to perform the testing of a network that has been trained. When training is completed, the Adaline can be used to classify input patterns. A step function is used to test the performance of the network. The testing procedure for the Adaline network is as follows:

Step 0: Initialize the weights. (The weights are obtained from the training algorithm.)

Step 1: Perform Steps 2–4 for each bipolar input vector x .

Step 2: Set the activations of the input units to x .

Step 3: Calculate the net input to the output unit:

$$y_{in} = b + \sum x_i w_i$$

Step 4: Apply the activation function over the net input calculated:

$$y = \begin{cases} 1 & \text{if } y_{in} \geq 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

3.4 Multiple Adaptive Linear Neurons

3.4.1 Theory

The multiple adaptive linear neurons (Madaline) model consists of many Adalines in parallel with a single output unit whose value is based on certain selection rules. It may use majority vote rule. On using this rule, the output would have as answer either true or false. On the other hand, if AND rule is used, the output is true if and only if both the inputs are true, and so on. The weights that are connected from the Adaline layer to the Madaline layer are fixed, positive and possess equal values. The weights between the input layer and the Adaline layer are adjusted during the training process. The Adaline and Madaline layer neurons have a bias of excitation "1" connected to them. The training process for a Madaline system is similar to that of an Adaline.

3.4.2 Architecture

A simple Madaline architecture is shown in Figure 3-7, which consists of " n " units of input layer, " m " units of Adaline layer and "1" unit of the Madaline layer. Each neuron in the Adaline and Madaline layers has a bias of excitation 1. The Adaline layer is present between the input layer and the Madaline (output) layer; hence, the Adaline layer can be considered a hidden layer. The use of the hidden layer gives the net computational capability which is not found in single-layer nets, but this complicates the training process to some extent.

The Adaline and Madaline models can be applied effectively in communication systems of adaptive equalizers and adaptive noise cancellation and other cancellation circuits.

3.4.3 Flowchart of Training Process

The flowchart of the training process of the Madaline network is shown in Figure 3-8. In case of training, the weights between the input layer and the hidden layer are adjusted, and the weights between the hidden layer

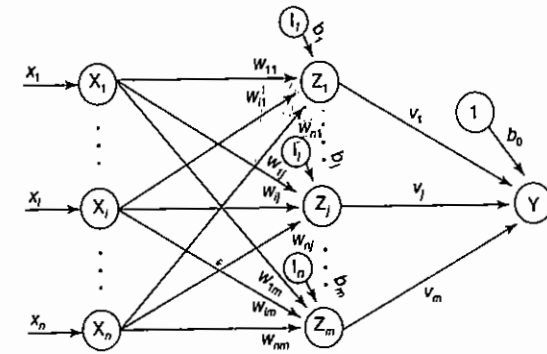


Figure 3-7 Architecture of Madaline layer.

and the output layer are fixed. The time taken for the training process in the Madaline network is very high compared to that of the Adaline network.

3.4.4 Training Algorithm

In this training algorithm, only the weights between the hidden layer and the input layer are adjusted, and the weights for the output units are fixed. The weights $v_1, v_2, \dots, v_m$ and the bias b_0 that enter into output unit Y are determined so that the response of unit Y is 1. Thus, the weights entering Y unit may be taken as

$$v_1 = v_2 = \dots = v_m = \frac{1}{2}$$

and the bias can be taken as

$$b_0 = \frac{1}{2}$$

The activation for the Adaline (hidden) and Madaline (output) units is given by

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

Step 0: Initialize the weights. The weights entering the output unit are set as above. Set initial small random values for Adaline weights. Also set initial learning rate α .

Step 1: When stopping condition is false, perform Steps 2–3.

Step 2: For each bipolar training pair s, t , perform Steps 3–7.

Step 3: Activate input layer units. For $i = 1$ to n ,

$$x_i = s_i$$

Step 4: Calculate net input to each hidden Adaline unit:

$$z_{inj} = b_j + \sum_{i=1}^n x_i w_{ij}, \quad j = 1 \text{ to } m$$

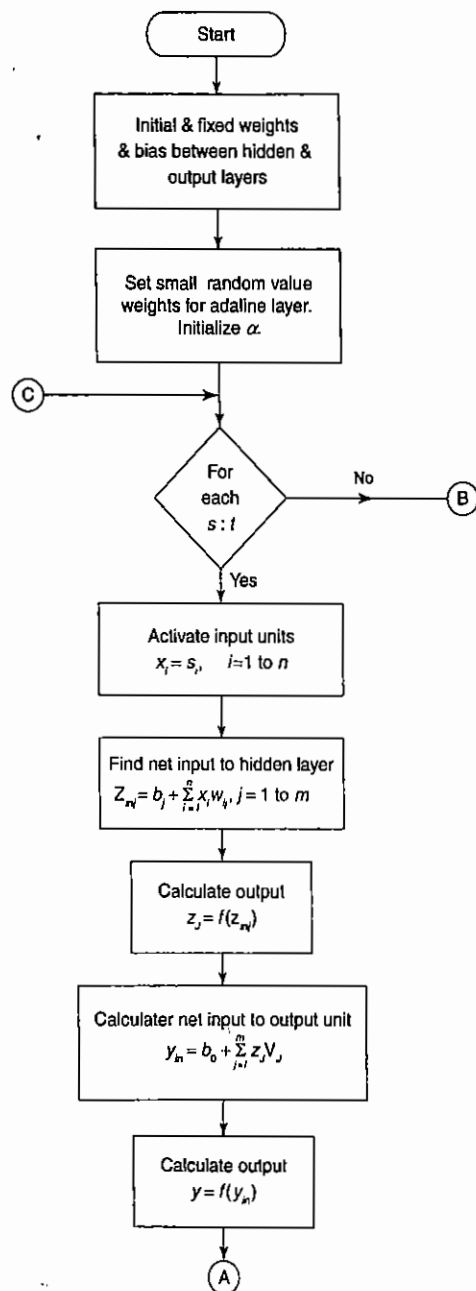


Figure 3-8 Flowchart for training of Madaline.

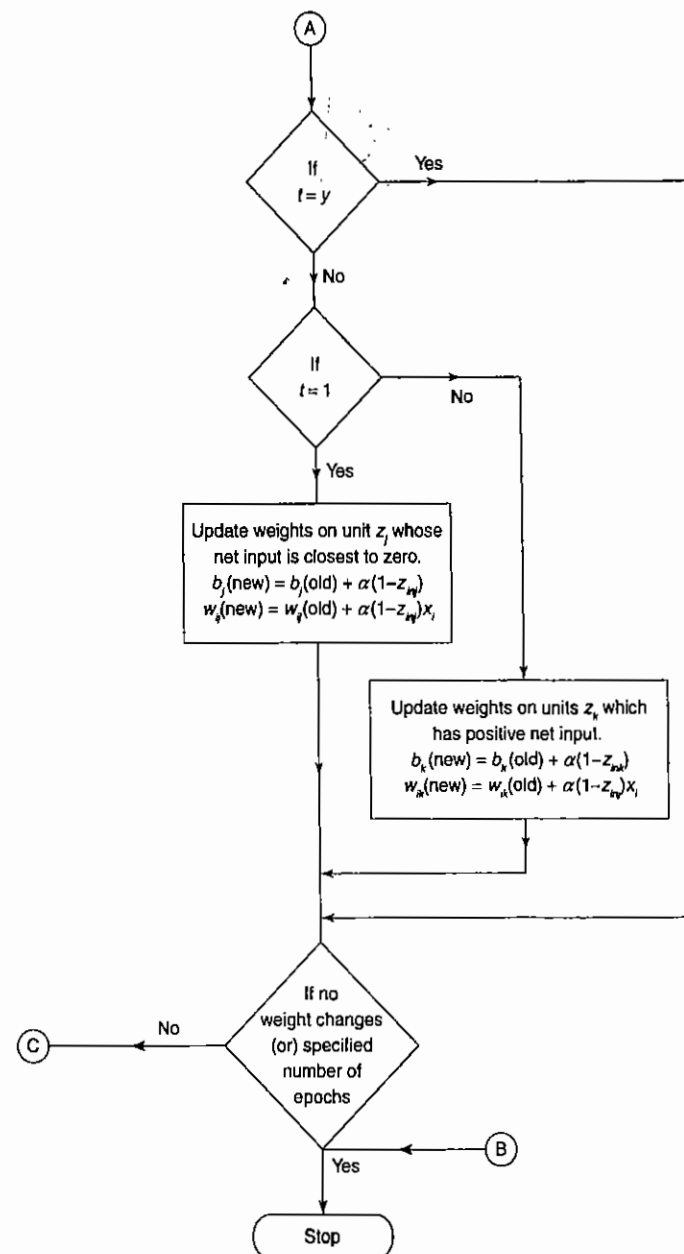


Figure 3-8 (Continued).

Step 5: Calculate output of each hidden unit:

$$z_j = f(z_{inj})$$

Step 6: Find the output of the net:

$$y_{in} = b_0 + \sum_{j=1}^m z_j v_j$$

$$y = f(y_{in})$$

Step 7: Calculate the error and update the weights.

1. If $t = y$, no weight updation is required.
2. If $t \neq y$ and $t = +1$, update weights on z_j , where net input is closest to 0 (zero):

$$b_j(\text{new}) = b_j(\text{old}) + \alpha (1 - z_{inj})$$

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha (1 - z_{inj}) x_i$$

3. If $t \neq y$ and $t = -1$, update weights on units z_k whose net input is positive:

$$w_{ik}(\text{new}) = w_{ik}(\text{old}) + \alpha (-1 - z_{ink}) x_i$$

$$b_k(\text{new}) = b_k(\text{old}) + \alpha (-1 - z_{ink})$$

Step 8: Test for the stopping condition. (If there is no weight change or weight reaches a satisfactory level, or if a specified maximum number of iterations of weight updation have been performed then stop, or else continue).

Madalines can be formed with the weights on the output unit set to perform some logic functions. If there are only two hidden units present, or if there are more than two hidden units, then the "majority vote rule" function may be used.

3.5 Back-Propagation Network

3.5.1 Theory

The back-propagation learning algorithm is one of the most important developments in neural networks (Bryson and Ho, 1969; Werbos, 1974; Lecun, 1985; Parker, 1985; Rumelhart, 1986). This network has re-awakened the scientific and engineering community to the modeling and processing of numerous quantitative phenomena using neural networks. This learning algorithm is applied to multilayer feed-forward networks consisting of processing elements with continuous differentiable activation functions. The networks associated with back-propagation learning algorithm are also called *back-propagation networks* (BPNs). For a given set of training input-output pair, this algorithm provides a procedure for changing the weights in a BPN to classify the given input patterns correctly. The basic concept for this weight update algorithm is simply the gradient-descent method as used in the case of simple perceptron networks with differentiable units. This is a method where the error is propagated back to the hidden unit. The aim of the neural network is to train the net to achieve a balance between the net's ability to respond (memorization) and its ability to give reasonable responses to the input that is similar but not identical to the one that is used in training (generalization).

The back-propagation algorithm is different from other networks in respect to the process by which the weights are calculated during the learning period of the network. The general difficulty with the multilayer perceptrons is calculating the weights of the hidden layers in an efficient way that would result in a very small or zero output error. When the hidden layers are increased the network training becomes more complex. To update weights, the error must be calculated. The error, which is the difference between the actual (calculated) and the desired (target) output, is easily measured at the output layer. It should be noted that at the hidden layers, there is no direct information of the error. Therefore, other techniques should be used to calculate an error at the hidden layer, which will cause minimization of the output error, and this is the ultimate goal.

The training of the BPN is done in three stages – the feed-forward of the input training pattern, the calculation and back-propagation of the error, and updation of weights. The testing of the BPN involves the computation of feed-forward phase only. There can be more than one hidden layer (more beneficial) but one hidden layer is sufficient. Even though the training is very slow, once the network is trained it can produce its outputs very rapidly.

3.5.2 Architecture

A back-propagation neural network is a multilayer, feed-forward neural network consisting of an input layer, a hidden layer and an output layer. The neurons present in the hidden and output layers have biases, which are the connections from the units whose activation is always 1. The bias terms also acts as weights. Figure 3-9 shows the architecture of a BPN, depicting only the direction of information flow for the feed-forward phase. During the back-propagation phase of learning, signals are sent in the reverse direction.

The inputs are sent to the BPN and the output obtained from the net could be either binary (0, 1) or bipolar (-1, +1). The activation function could be any function which increases monotonically and is also differentiable.

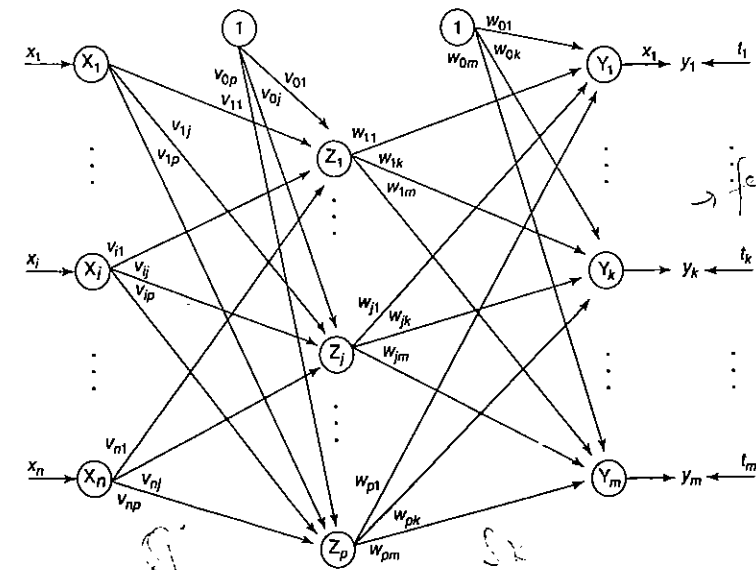


Figure 3-9 Architecture of a back-propagation network.

3.5.3 Flowchart for Training Process

The flowchart for the training process using a BPN is shown in Figure 3-10. The terminologies used in the flowchart and in the training algorithm are as follows:

x = input training vector ($x_1, \dots, x_i, \dots, x_n$)

t = target output vector ($t_1, \dots, t_k, \dots, t_m$)

α = learning rate parameter

x_i = input unit i . (Since the input layer uses identity activation function, the input and output signals here are same.)

v_{0j} = bias on j th hidden unit

w_{0k} = bias on k th output unit

z_j = hidden unit j . The net input to z_j is

$$z_{inj} = v_{0j} + \sum_{i=1}^n x_i v_{ij}$$

and the output is

$$z_j = f(z_{inj})$$

y_k = output unit k . The net input to y_k is

$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

and the output is

$$y_k = f(y_{ink})$$

δ_k = error correction weight adjustment for w_{jk} that is due to an error at output unit y_k , which is back-propagated to the hidden units that feed into unit y_k .

δ_j = error correction weight adjustment for v_{ij} that is due to the back-propagation of error to the hidden unit z_j . *to input unit that feed into z_j*

Also, it should be noted that the commonly used activation functions are binary sigmoidal and bipolar sigmoidal activation functions (discussed in Section 2.3.3). These functions are used in the BPN because of the following characteristics: (i) continuity; (ii) differentiability; (iii) nondecreasing monotony.

The range of binary sigmoid is from 0 to 1, and for bipolar sigmoid it is from -1 to +1.

3.5.4 Training Algorithm

The error back-propagation learning algorithm can be outlined in the following algorithm:

Step 0: Initialize weights and learning rate (take some small random values).

Step 1: Perform Steps 2–9 when stopping condition is false.

Step 2: Perform Steps 3–8 for each training pair.

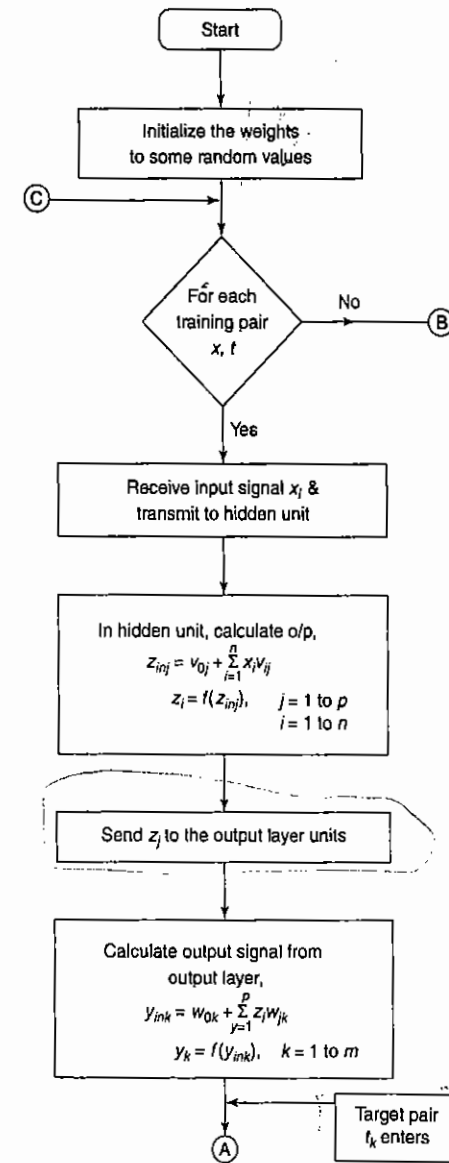


Figure 3-10 Flowchart for back-propagation network training.

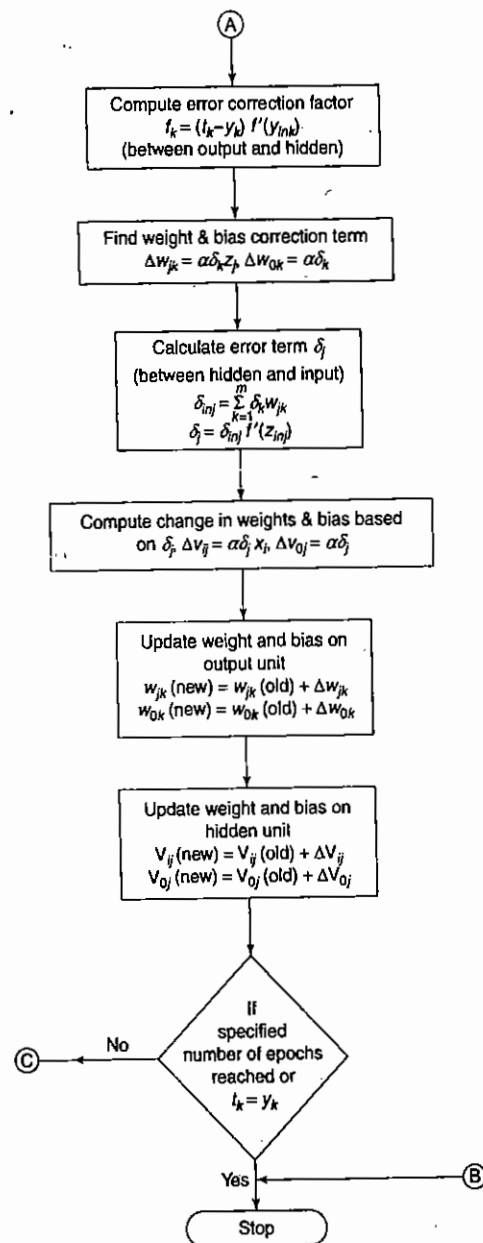


Figure 3-10 (Continued).

Feed-forward phase (Phase I)

Step 3: Each input unit receives input signal x_i and sends it to the hidden unit ($i = 1$ to n).

Step 4: Each hidden unit z_j ($j = 1$ to p) sums its weighted input signals to calculate net input:

$$z_{inj} = v_{0j} + \sum_{i=1}^n x_i v_{ij}$$

Calculate output of the hidden unit by applying its activation functions over z_{inj} (binary or bipolar sigmoidal activation function):

$$z_j = f(z_{inj})$$

and send the output signal from the hidden unit to the input of output layer units.

Step 5: For each output unit y_k ($k = 1$ to m), calculate the net input:

$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

and apply the activation function to compute output signal

$$y_k = f(y_{ink})$$

Back-propagation of error (Phase II)

Step 6: Each output unit y_k ($k = 1$ to m) receives a target pattern corresponding to the input training pattern and computes the error correction term:

$$\delta_k = (t_k - y_k) f'(y_{ink})$$

The derivative $f'(y_{ink})$ can be calculated as in Section 2.3.3. On the basis of the calculated error correction term, update the change in weights and bias:

$$\Delta w_{jk} = \alpha \delta_k z_j, \quad \Delta w_{0k} = \alpha \delta_k$$

Also, send δ_k to the hidden layer backwards.

Step 7: Each hidden unit (z_j , $j = 1$ to p) sums its delta inputs from the output units:

$$\delta_{inj} = \sum_{k=1}^m \delta_k w_{jk}$$

The term δ_{inj} gets multiplied with the derivative of $f(z_{inj})$ to calculate the error term:

$$\delta_j = \delta_{inj} f'(z_{inj})$$

The derivative $f'(z_{inj})$ can be calculated as discussed in Section 2.3.3 depending on whether binary or bipolar sigmoidal function is used. On the basis of the calculated δ_j , update the change in weights and bias:

$$\Delta v_{ij} = \alpha \delta_j x_i, \quad \Delta v_{0j} = \alpha \delta_j$$

Weight and bias updation (Phase III):

Step 8: Each output unit ($y_k, k = 1$ to m) updates the bias and weights:

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \Delta w_{jk}$$

$$w_{0k}(\text{new}) = w_{0k}(\text{old}) + \Delta w_{0k}$$

Each hidden unit ($z_j, j = 1$ to p) updates its bias and weights:

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \Delta v_{ij}$$

$$v_{0j}(\text{new}) = v_{0j}(\text{old}) + \Delta v_{0j}$$

Step 9: Check for the stopping condition. The stopping condition may be certain number of epochs reached or when the actual output equals the target output.

The above algorithm uses the incremental approach for updation of weights, i.e., the weights are being changed immediately after a training pattern is presented. There is another way of training called *batch-mode training*, where the weights are changed only after all the training patterns are presented. The effectiveness of two approaches depends on the problem, but batch-mode training requires additional local storage for each connection to maintain the immediate weight changes. When a BPN is used as a classifier, it is equivalent to the optimal Bayesian discriminant function for asymptotically large sets of statistically independent training patterns.

The problem in this case is whether the back-propagation learning algorithm can always converge and find proper weights for network even after enough learning. It will converge since it implements a gradient-descent on the error surface in the weight space, and this will roll down the error surface to the nearest minimum error and will stop. This becomes true only when the relation existing between the input and the output training patterns is deterministic and the error surface is deterministic. This is not the case in real world because the produced square-error surfaces are always at random. This is the stochastic nature of the back-propagation algorithm, which is purely based on the stochastic gradient-descent method. The BPN is a special case of stochastic approximation.

If the BPN algorithm converges at all, then it may get stuck with local minima and may be unable to find satisfactory solutions. The randomness of the algorithm helps it to get out of local minima. The error functions may have large number of global minima because of permutations of weights that keep the network input-output function unchanged. This causes the error surfaces to have numerous troughs.

3.5.5 Learning Factors of Back-Propagation Network

The training of a BPN is based on the choice of various parameters. Also, the convergence of the BPN is based on some important learning factors such as the initial weights, the learning rate, the updation rule, the size and nature of the training set, and the architecture (number of layers and number of neurons per layer).

3.5.5.1 Initial Weights

The ultimate solution may be affected by the initial weights of a multilayer feed-forward network. They are initialized at small random values. The choice of the initial weight determines how fast the network converges. The initial weights cannot be very high because the sigmoidal activation functions used here may get saturated

from the beginning itself and the system may be stuck at a local minima or at a very flat plateau at the starting point itself. One method of choosing the weight w_{ij} is choosing it in the range

$$w_{ij} = \left[\frac{-3}{\sqrt{o_i}}, \frac{3}{\sqrt{o_i}} \right]$$

where o_i is the number of processing elements j that feed-forward to processing element i . The initialization can also be done by a method called Nyugen-Widrow initialization. This type of initialization leads to faster convergence of network. The concept here is based on the geometric analysis of the response of hidden neurons to a single input. The method is used for improving the learning ability of the hidden units. The random initialization of weights connecting input neurons to the hidden neurons is obtained by the equation

$$v_{ij}(\text{new}) = \gamma \frac{v_{ij}(\text{old})}{\|v_j(\text{old})\|}$$

where $\bar{v}_j$ is the average weight calculated for all values of i , and the scale factor $\gamma = 0.7(P)^{1/n}$ ("n" is the number of input neurons and "P" is the number of hidden neurons).

3.5.5.2 Learning Rate α

The learning rate (α) affects the convergence of the BPN. A larger value of α may speed up the convergence but might result in overshooting, while a smaller value of α has vice-versa effect. The range of α from 10^{-3} to 10 has been used successfully for several back-propagation algorithmic experiments. Thus, a large learning rate leads to rapid learning but there is oscillation of weights, while the lower learning rate leads to slower learning.

3.5.5.3 Momentum Factor

The gradient descent is very slow if the learning rate α is small and oscillates widely if α is too large. One very efficient and commonly used method that allows a larger learning rate without oscillations is by adding a momentum factor to the normal gradient-descent method.

The momentum factor is denoted by $\eta \in [0, 1]$ and the value of 0.9 is often used for the momentum factor. Also, this approach is more useful when some training data are very different from the majority of data. A momentum factor can be used with either pattern by pattern updating or batch-mode updating. In case of batch mode, it has the effect of complete averaging over the patterns. Even though the averaging is only partial in the pattern-by-pattern mode, it leaves some useful information for weight updation.

The weight updation formulas used here are

$$w_{jk}(t+1) = w_{jk}(t) + \underbrace{\alpha \delta_k z_j + \eta [w_{jk}(t) - w_{jk}(t-1)]}_{\Delta w_{jk}(t+1)}$$

and

$$v_{ij}(t+1) = v_{ij}(t) + \underbrace{\alpha \delta_j x_i + \eta [v_{ij}(t) - v_{ij}(t-1)]}_{\Delta v_{ij}(t+1)}$$

The momentum factor also helps in faster convergence.

3.5.5.4 Generalization

The best network for generalization is BPN. A network is said to be generalized when it sensibly interpolates with input networks that are new to the network. When there are many trainable parameters for the given amount of training data, the network learns well but does not generalize well. This is usually called *overfitting* or *overtraining*. One solution to this problem is to monitor the error on the test set and terminate the training when the error increases. With small number of trainable parameters, the network fails to learn the training data and performs very poorly on the test data. For improving the ability of the network to generalize from a training data set to a test data set, it is desirable to make small changes in the input space of a pattern, without changing the output components. This is achieved by introducing variations in the input space of training patterns as part of the training set. However, computationally, this method is very expensive. Also, a net with large number of nodes is capable of memorizing the training set at the cost of generalization. As a result, smaller nets are preferred than larger ones.

3.5.5.5 Number of Training Data

The training data should be sufficient and proper. There exists a rule of thumb, which states that the training data should cover the entire expected input space, and while training, training-vector pairs should be selected randomly from the set. Assume that the input space as being linearly separable into "L" disjoint regions with their boundaries being part of hyper planes. Let "T" be the lower bound on the number of training patterns. Then, choosing T such that $T/L \gg 1$ will allow the network to discriminate pattern classes using fine piecewise hyperplane partitioning. Also in some cases, scaling or normalization has to be done to help learning.

3.5.5.6 Number of Hidden Layer Nodes

If there exists more than one hidden layer in a BPN, then the calculations performed for a single layer are repeated for all the layers and are summed up at the end. In case of all multilayer feed-forward networks, the size of a hidden layer is very important. The number of hidden units required for an application needs to be determined separately. The size of a hidden layer is usually determined experimentally. For a network of a reasonable size, the size of hidden nodes has to be only a relatively small fraction of the input layer. For example, if the network does not converge to a solution, it may need more hidden nodes. On the other hand, if the network converges, the user may try a very few hidden nodes and then settle finally on a size based on overall system performance.

3.5.6 Testing Algorithm of Back-Propagation Network

The testing procedure of the BPN is as follows:

- Step 0: Initialize the weights. The weights are taken from the training algorithm.
- Step 1: Perform Steps 2-4 for each input vector.
- Step 2: Set the activation of input unit for x_i ($i = 1$ to n).
- Step 3: Calculate the net input to hidden unit x and its output. For $j = 1$ to p ,

$$z_{inj} = v_{0j} + \sum_{i=1}^n x_i v_{ij}$$

$$x_j = f(z_{inj})$$

Step 4: Now compute the output of the output layer unit. For $k = 1$ to m ,

$$y_{ink} = w_{0k} + \sum_{j=1}^p z_j w_{jk}$$

$$y_k = f(y_{ink})$$

Use sigmoidal activation functions for calculating the output.

3.6 Radial Basis Function Network

3.6.1 Theory

The radial basis function (RBF) is a classification and functional approximation neural network developed by M.J.D. Powell. The network uses the most common nonlinearities such as sigmoidal and Gaussian kernel functions. The Gaussian functions are also used in regularization networks. The response of such a function is positive for all values of y ; the response decreases to 0 as $|y| \rightarrow \infty$. The Gaussian function is generally defined as

$$f(y) = e^{-y^2}$$

The derivative of this function is given by

$$f'(y) = -2ye^{-y^2} = -2yf(y)$$

The graphical representation of this Gaussian function is shown in Figure 3-11 below.

When the Gaussian potential functions are being used, each node is found to produce an identical output for inputs existing within the fixed radial distance from the center of the kernel, they are found to be radially symmetric, and hence the name radial basis function network. The entire network forms a linear combination of the nonlinear basis function.

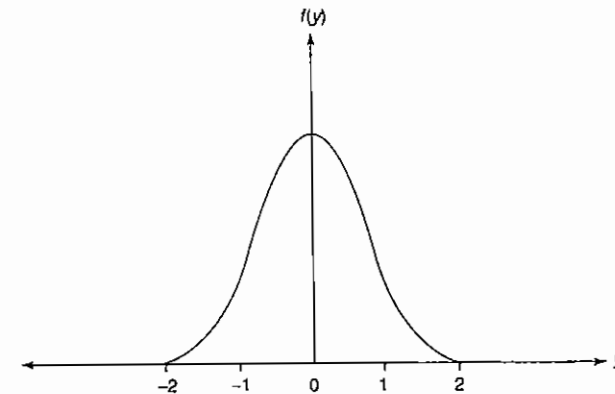


Figure 3-11 Gaussian kernel function.

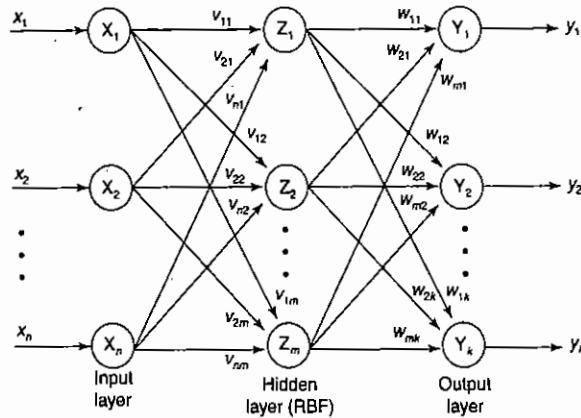


Figure 3-12 Architecture of RBF.

3.6.2 Architecture

The architecture for the radial basis function network (RBFN) is shown in Figure 3-12. The architecture consists of two layers whose output nodes form a linear combination of the kernel (or basis) functions computed by means of the RBF nodes or hidden layer nodes. The basis function (nonlinearity) in the hidden layer produces a significant nonzero response to the input stimulus it has received only when the input of it falls within a small localized region of the input space. This network can also be called as *localized receptive field network*.

3.6.3 Flowchart for Training Process

The flowchart for the training process of the RBF is shown in Figure 3-13 below. In this case, the center of the RBF functions has to be chosen and hence, based on all parameters, the output of network is calculated.

3.6.4 Training Algorithm

The training algorithm describes in detail all the calculations involved in the training process depicted in the flowchart. The training is started in the hidden layer with an unsupervised learning algorithm. The training is continued in the output layer with a supervised learning algorithm. Simultaneously, we can apply supervised learning algorithm to the hidden and output layers for fine-tuning of the network. The training algorithm is given as follows.

- Step 0: Set the weights to small random values.
- Step 1: Perform Steps 2–8 when the stopping condition is false.
- Step 2: Perform Steps 3–7 for each input.
- Step 3: Each input unit (x_i for all $i = 1$ to n) receives input signals and transmits to the next hidden layer unit.

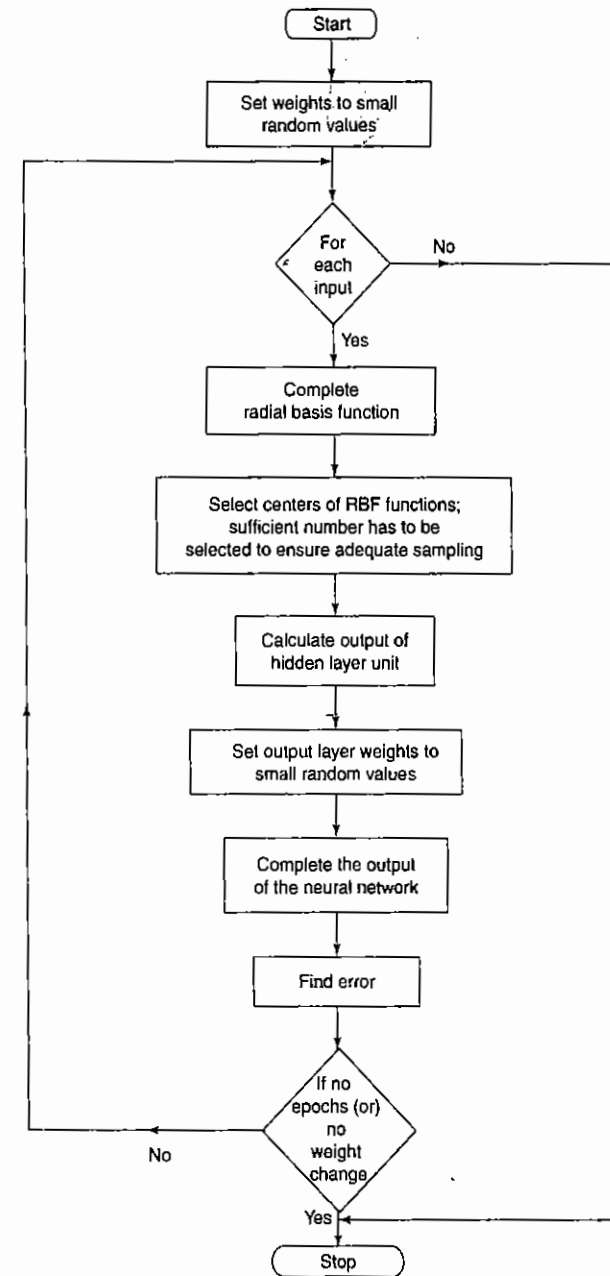


Figure 3-13 Flowchart for the training process of RBF.

Step 4: Calculate the radial basis function.

Step 5: Select the centers for the radial basis function. The centers are selected from the set of input vectors. It should be noted that a sufficient number of centers have to be selected to ensure adequate sampling of the input vector space.

Step 6: Calculate the output from the hidden layer unit:

$$v_i(x_i) = \frac{\exp \left[- \sum_{j=1}^r (x_{ji} - \hat{x}_{ji})^2 \right]}{\sigma_i^2}$$

where $\hat{x}_{ji}$ is the center of the RBF unit for input variables; σ_i the width of i th RBF unit; x_{ji} the j th variable of input pattern.

Step 7: Calculate the output of the neural network:

$$y_{net} = \sum_{i=1}^k w_{im} v_i(x_i) + w_0$$

where k is the number of hidden layer nodes (RBF function); y_{net} the output value of m th node in output layer for the n th incoming pattern; w_{im} the weight between i th RBF unit and m th output node; w_0 the biasing term at n th output node.

Step 8: Calculate the error and test for the stopping condition. The stopping condition may be number of epochs or to a certain extent weight change.

Thus, a network can be trained using RBFN.

3.7 Time Delay Neural Network

The neural network has to respond to a sequence of patterns. Here the network is required to produce a particular output sequence in response to a particular sequence of inputs. A shift register can be considered as a tapped delay line. Consider a case of a multilayer perceptron where the tapped outputs of the delay line are applied to its inputs. This type of network constitutes a *time delay neural network* (TDNN). The output consists of a finite temporal dependence on its inputs, given as

$$U(t) = F[x(t), x(t-1), \dots, x(t-n)]$$

where F is any nonlinearity function. The multilayer perceptron with delay line is shown in Figure 3-14.

When the function $U(t)$ is a weighted sum, then the TDNN is equivalent to a finite impulse response filter (FIR). In TDNN, when the output is being fed back through a unit delay into the input layer, then the net computed here is equivalent to an infinite impulse response (IIR) filter. Figure 3-15 shows TDNN with output feedback.

Thus, a neuron with a tapped delay line is called a TDNN unit, and a network which consists of TDNN units is called a TDNN. A specific application of TDNNs is speech recognition. The TDNN can be trained using the back-propagation-learning rule with a momentum factor.

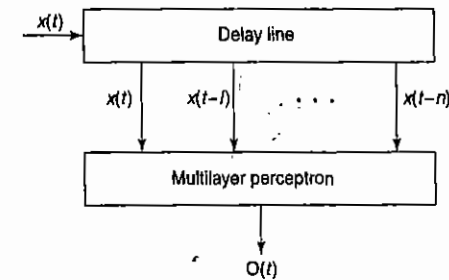


Figure 3-14 Time delay neural network (FIR filter).

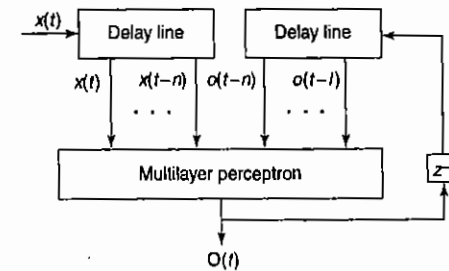


Figure 3-15 TDNN with output feedback (IIR filter).

3.8 Functional Link Networks

These networks are specifically designed for handling linearly non-separable problems using appropriate input representation. Thus, suitable enhanced representation of the input data has to be found out. This can be achieved by increasing the dimensions of the input space. The input data which is expanded is used for training instead of the actual input data. In this case, higher order input terms are chosen so that they are linearly independent of the original pattern components. Thus, the input representation has been enhanced and linear separability can be achieved in the extended space. One of the functional link model networks is shown in Figure 3-16. This model is helpful for learning continuous functions. For this model, the higher-order input terms are obtained using the orthogonal basis functions such as $\sin \pi x$, $\cos \pi x$, $\sin 2\pi x$, $\cos 2\pi x$, etc.

The most common example of linear nonseparability is XOR problem. The functional link networks help in solving this problem. The inputs now are

x_1	x_2	$x_1 x_2$	t
-1	-1	1	1
-1	1	-1	-1
1	-1	-1	-1
1	1	1	1

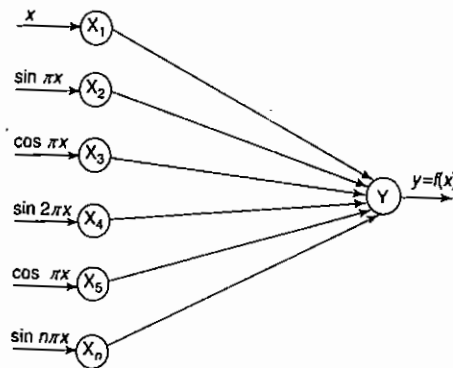


Figure 3-16 Functional line network model.

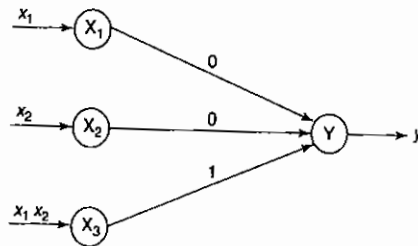


Figure 3-17 The XOR problem.

Thus, it can be easily seen that the functional link network in Figure 3-17 is used for solving this problem. The functional link network consists of only one layer, therefore, it can be trained using delta learning rule instead of the generalized delta learning rule used in BPN. As a result, the learning speed of the functional link network is faster than that of the BPN.

3.9 Tree Neural Networks

The tree neural networks (TNNs) are used for the pattern recognition problem. The main concept of this network is to use a small multilayer neural network at each decision-making node of a binary classification tree for extracting the non-linear features. TNNs completely extract the power of tree classifiers for using appropriate local features at the different levels and nodes of the tree. A binary classification tree is shown in Figure 3-18.

The decision nodes are present as circular nodes and the terminal nodes are present as square nodes. The terminal node has class label denoted by $\hat{c}$ associated with it. The rule base is formed in the decision node (splitting rule in the form of $f(x) < \theta$). The rule determines whether the pattern moves to the right or to the left. Here, $f(x)$ indicates the associated feature of pattern and " θ " is the threshold. The pattern will be given the class label of the terminal node on which it has landed. The classification here is based on the fact that the appropriate features can be selected at different nodes and levels in the tree. The output feature $y = f(x)$

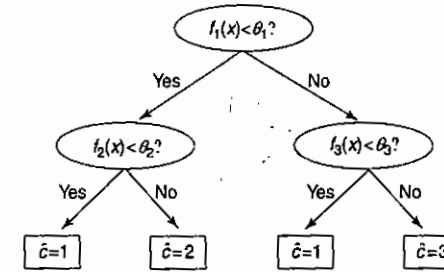


Figure 3-18 Binary classification tree.

obtained by a multilayer network at a particular decision node is used in the following way:

x directed to left child node t_L , if $y < 0$

x directed to right child node t_R , if $y \geq 0$

The algorithm for a TNN consists of two phases:

1. *Tree growing phase:* In this phase, a large tree is grown by recursively finding the rules for splitting until all the terminal nodes have pure or nearly pure class membership, else it cannot split further.
2. *Tree pruning phase:* Here a smaller tree is being selected from the pruned subtree to avoid the overfilling of data.

The training of TNN involves two nested optimization problems. In the inner optimization problem, the BPN algorithm can be used to train the network for a given pair of classes. On the other hand, in outer optimization problem, a heuristic search method is used to find a good pair of classes. The TNN when tested on a character recognition problem decreases the error rate and size of the tree relative to that of the standard classification tree design methods. The TNN can be implemented for waveform recognition problem. It obtains comparable error rates and the training here is faster than the large BPN for the same application. Also, TNN provides a structured approach to neural network classifier design problems.

3.10 Wavelet Neural Networks

The wavelet neural network (WNN) is based on the wavelet transform theory. This network helps in approximating arbitrary nonlinear functions. The powerful tool for function approximation is wavelet decomposition.

Let $f(x)$ be a piecewise continuous function. This function can be decomposed into a family of functions, which is obtained by dilating and translating a single wavelet function $\phi: R^n \rightarrow R$ as

$$f(x) = \sum_{i=1}^n w_i \det [D_i]^{1/2} \phi [D_i(x - t_i)]$$

where D_i is the $\text{diag}(d_i)$, $d_i \in R_n^+$ are dilation vectors; D_i and t_i are the translational vectors; $\det []$ is the determinant operator. The wavelet function ϕ selected should satisfy some properties. For selecting $\phi: R^n \rightarrow R$, the condition may be

$$\phi(x) = \phi_1(x_1) \cdots \phi_n(x_n) \quad \text{for } x = (x_1, x_2, \dots, x_n)$$

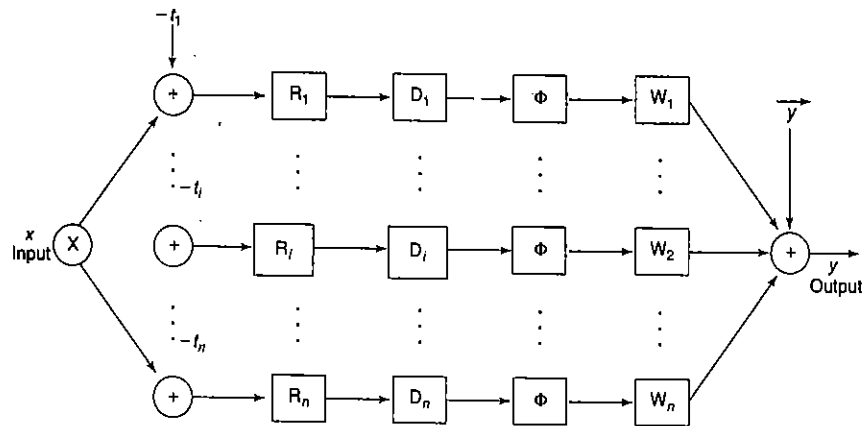


Figure 3-19 Wavelet neural network.

where

$$\phi_i(x) = -x \exp\left(\frac{-x^2}{2}\right)$$

is called scalar wavelet. The network structure can be formed based on the wavelet decomposition as

$$y(x) = \sum_{i=1}^n w_i \phi[D_i(x - t_i)] + \bar{y}$$

where $\bar{y}$ helps to deal with nonzero mean functions on finite domains. For proper dilation, a rotation can be made for better network operation:

$$y(x) = \sum_{i=1}^n w_i \phi[D_i R_i(x - t_i)] + \bar{y}$$

where R_i are the rotation matrices. The network which performs according to the above equation is called wavelet neural network. This is a combination of translation, rotation and dilation; and if a wavelet is lying on the same line, then it is called *wavelon* in comparison to the neurons in neural networks. The wavelet neural network is shown in Figure 3-19.

3.11 Summary

In this chapter we have discussed the supervised learning networks. In most of the classification and recognition problems, the widely used networks are the supervised learning networks. The architecture, the learning rule, flowchart for training process and training algorithm are discussed in detail for perceptron network, Adaline, Madaline, back-propagation network and radial basis function network. The perceptron network can be trained for single output classes as well as multioutput classes. Also, many Adaline networks combine together

to form a Madaline network. These networks are trained using delta learning rule. Back-propagation network is the most commonly used network in the real time applications. The error is back-propagated here and is fine tuned for achieving better performance. The basic difference between the back-propagation network and radial basis function network is the activation function used. The radial basis function network mostly uses Gaussian activation function. Apart from these networks, some special supervised learning networks such as time delay neural networks, functional link networks, tree neural networks and wavelet neural networks have also been discussed.

3.12 Solved Problems

1. Implement AND function using perceptron networks for bipolar inputs and targets.

Solution: Table 1 shows the truth table for AND function with bipolar inputs and targets:

Table 1

x_1	x_2	t
1	1	1
1	-1	-1
-1	1	-1
-1	-1	-1

The perceptron network, which uses perceptron learning rule, is used to train the AND function. The network architecture is as shown in Figure 1. The input patterns are presented to the network one by one. When all the four input patterns are presented, then one epoch is said to be completed. The initial weights and threshold are set to zero, i.e., $w_1 = w_2 = b = 0$ and $\theta = 0$. The learning rate α is set equal to 1.

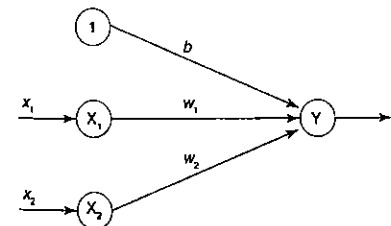


Figure 1 Perceptron network for AND function.

For the first input pattern, $x_1 = 1, x_2 = 1$ and $t = 1$, with weights and bias, $w_1 = 0, w_2 = 0$ and $b = 0$:

- Calculate the net input

$$\begin{aligned} y_{in} &= b + x_1 w_1 + x_2 w_2 \\ &= 0 + 1 \times 0 + 1 \times 0 = 0 \end{aligned}$$

- The output y is computed by applying activations over the net input calculated:

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } y_{in} = 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

Here we have taken $\theta = 0$. Hence, when, $y_{in} = 0$, $y = 0$.

- Check whether $t = y$. Here, $t = 1$ and $y = 0$, so $t \neq y$, hence weight updation takes place:

$$w_i(\text{new}) = w_i(\text{old}) + \alpha x_i$$

$$w_1(\text{new}) = w_1(\text{old}) + \alpha x_1 = 0 + 1 \times 1 \times 1 = 1$$

$$w_2(\text{new}) = w_2(\text{old}) + \alpha x_2 = 0 + 1 \times 1 \times 1 = 1$$

$$b(\text{new}) = b(\text{old}) + \alpha t = 0 + 1 \times 1 = 1$$

Here, the change in weights are

$$\Delta w_1 = \alpha x_1;$$

$$\Delta w_2 = \alpha x_2;$$

$$\Delta b = \alpha t$$

The weights $w_1 = 1, w_2 = 1, b = 1$ are the final weights after first input pattern is presented. The same process is repeated for all the input patterns. The process can be stopped when all the targets become equal to the calculated output or when a separating line is obtained using the final weights for separating the positive responses from negative responses. Table 2 shows the training of perceptron network until its

Table 2

Input			Target (t)	Net input (y_{in})	Calculated output (y)	Weight changes			Weights		
x_1	x_2	1				Δw_1	Δw_2	Δb	w_1 (0)	w_2 (0)	b (0)
EPOCH-1											
1	1	1	1	0	0	1	1	1	1	1	1
1	-1	1	-1	1	1	-1	1	-1	0	2	0
-1	1	1	-1	2	1	+1	-1	-1	1	1	-1
-1	-1	1	-1	-3	-1	0	0	0	1	1	-1
EPOCH-2											
1	1	1	1	1	1	0	0	0	1	1	-1
1	-1	1	-1	-1	-1	0	0	0	1	1	-1
-1	1	1	-1	-1	-1	0	0	0	1	1	-1
-1	-1	1	-1	-3	-1	0	0	0	1	1	-1

target and calculated output converge for all the patterns.

The final weights and bias after second epoch are

$$w_1 = 1, w_2 = 1, b = -1$$

Since the threshold for the problem is zero, the equation of the separating line is

$$x_2 = -\frac{w_1}{w_2}x_1 - \frac{b}{w_2}$$

Here

$$w_1x_1 + w_2x_2 + b > \theta$$

$$w_1x_1 + w_2x_2 + b > 0$$

Thus, using the final weights we obtain

$$x_2 = -\frac{1}{1}x_1 - \frac{(-1)}{1}$$

$$x_2 = -x_1 + 1$$

It can be easily found that the above straight line separates the positive response and negative response region, as shown in Figure 2.

The same methodology can be applied for implementing other logic functions such as OR, AND-NOT, NAND, etc. If there exists a threshold value $\theta \neq 0$, then two separating lines have to be obtained, i.e., one to separate positive response from zero and the other for separating zero from the negative response.

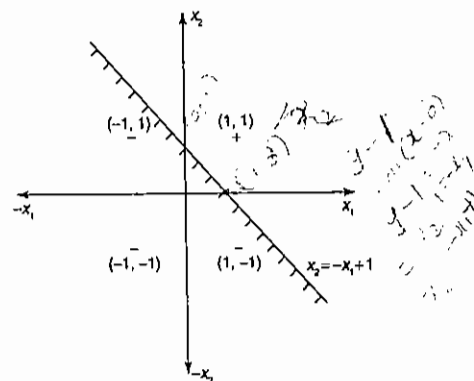


Figure 2 Decision boundary for AND function in perceptron training ($\theta = 0$).

2. Implement OR function with binary inputs and bipolar targets using perceptron training algorithm upto 3 epochs.

Solution: The truth table for OR function with binary inputs and bipolar targets is shown in Table 3.

Table 3

x_1	x_2	t
1	1	1
1	0	1
0	1	1
0	0	-1

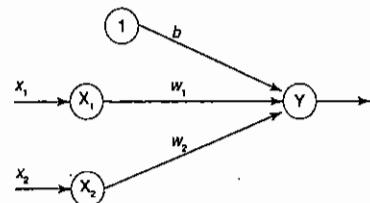


Figure 3 Perceptron network for OR function.

The perceptron network, which uses perceptron learning rule, is used to train the OR function. The network architecture is shown in Figure 3. The initial values of the weights and bias are taken as zero, i.e.,

$$w_1 = w_2 = b = 0$$

Also the learning rate is 1 and threshold is 0.2. So, the activation function becomes

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0.2 \\ 0 & \text{if } -0.2 \leq y_{in} \leq 0.2 \end{cases}$$

The network is trained as per the perceptron training algorithm and the steps are as in problem 1 (given for first pattern). Table 4 gives the network training for 3 epochs.

Table 4

Input			Target (x)	Net input (y_{in})	Calculated output (y)	Weight changes			Weights		
x_1	x_2	1				Δw_1	Δw_2	Δb	w_1 (0)	w_2 (0)	b (0)
EPOCH-1											
1	1	1	1	0	0	1	1	1	1	1	1
1	0	1	1	2	1	0	0	0	1	1	1
0	1	1	1	2	1	0	0	0	1	1	0
0	0	1	-1	1	1	0	0	-1	1	1	0
EPOCH-2											
1	1	1	1	2	1	0	0	0	1	1	0
1	0	1	1	1	1	0	0	0	1	1	0
0	1	1	1	1	1	0	0	0	1	1	0
0	0	1	-1	0	0	0	0	0	1	1	-1
EPOCH-3											
1	1	1	1	1	1	0	0	0	1	1	-1
1	0	1	1	0	0	1	0	1	2	1	0
0	1	1	1	1	1	0	0	0	2	1	0
0	0	1	-1	0	0	0	0	-1	2	1	-1

The final weights at the end of third epoch are

$$w_1 = 2, w_2 = 1, b = -1$$

Further epochs have to be done for the convergence of the network.

3. Find the weights using perceptron network for ANDNOT function when all the inputs are presented only one time. Use bipolar inputs and targets.

Solution: The truth table for ANDNOT function is shown in Table 5.

Table 5

x_1	x_2	t
1	1	-1
1	-1	1
-1	1	-1
-1	-1	-1

The network architecture of ANDNOT function is shown as in Figure 4. Let the initial weights be zero and $\alpha = 1, \theta = 0$. For the first input sample, we compute the net input as

$$y_{in} = b + \sum_{i=1}^n x_i w_i = b + x_1 w_1 + x_2 w_2$$

$$= 0 + 1 \times 0 + 1 \times 0 = 0$$

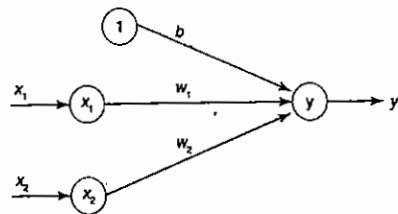


Figure 4 Network for ANDNOT function.

Applying the activation function over the net input, we obtain

$$y = f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } -0.2 \leq y_{in} \leq 0 \\ -1 & \text{if } y_{in} < -0.2 \end{cases}$$

Hence, the output $y = f(y_{in}) = 0$. Since $t \neq y$, the new weights are computed as

$$w_1(\text{new}) = w_1(\text{old}) + \alpha t x_1 = 0 + 1 \times -1 \times 1 = -1$$

$$w_2(\text{new}) = w_2(\text{old}) + \alpha t x_2 = 0 + 1 \times -1 \times 1 = -1$$

$$b(\text{new}) = b(\text{old}) + \alpha t = 0 + 1 \times -1 = -1$$

The weights after presenting the first sample are

$$w = [-1 \ -1 \ -1]$$

For the second input sample, we calculate the net input as

$$y_{in} = b + \sum_{i=1}^n x_i w_i = b + x_1 w_1 + x_2 w_2$$

$$= -1 + 1 \times -1 + (-1 \times -1)$$

$$= -1 - 1 + 1 = -1$$

The output $y = f(y_{in})$ is obtained by applying activation function, hence $y = -1$.

Since $t \neq y$, the new weights are calculated as

$$w_1(\text{new}) = w_1(\text{old}) + \alpha t x_1 = -1 + 1 \times 1 \times 1 = 0$$

$$w_2(\text{new}) = w_2(\text{old}) + \alpha t x_2 = -1 + 1 \times 1 \times -1 = -2$$

$$b(\text{new}) = b(\text{old}) + \alpha t = -1 + 1 \times 1 = 0$$

The weights after presenting the second sample are

$$w = [0 \ -2 \ 0]$$

For the third input sample, $x_1 = -1$, $x_2 = 1$, $t = -1$, the net input is calculated as,

$$\begin{aligned} y_{in} &= b + \sum_{i=1}^n x_i w_i = b + x_1 w_1 + x_2 w_2 \\ &= 0 + -1 \times 0 + 1 \times -2 = 0 + 0 - 2 = -2 \end{aligned}$$

The output is obtained as $y = f(y_{in}) = -1$. Since $t = y$, no weight changes. Thus, even after presenting the third input sample, the weights are

$$w = [0 \ -2 \ 0]$$

For the fourth input sample, $x_1 = -1$, $x_2 = -1$, $t = -1$, the net input is calculated as

$$\begin{aligned} y_{in} &= b + \sum_{i=1}^n x_i w_i = b + x_1 w_1 + x_2 w_2 \\ &= 0 + -1 \times 0 + (-1 \times -2) \\ &= 0 + 0 + 2 = 2 \end{aligned}$$

The output is obtained as $y = f(y_{in}) = 1$. Since $t \neq y$, the new weights on updating are given as

$$w_1(\text{new}) = w_1(\text{old}) + \alpha t x_1 = 0 + 1 \times -1 \times -1 = 1$$

$$w_2(\text{new}) = w_2(\text{old}) + \alpha t x_2 = -2 + 1 \times -1 \times -1 = -1$$

$$b(\text{new}) = b(\text{old}) + \alpha t = 0 + 1 \times -1 = -1$$

The weights after presenting fourth input sample are

$$w = [1 \ -1 \ -1]$$

One epoch of training for ANDNOT function using perceptron network is tabulated in Table 6.

Table 6

Input		Target	Net input	Calculated output	Weights		
x_1	x_2				w_1	w_2	b
1	1	1	(t)	(y)	0	0	0
1	1	1	0	0	-1	-1	-1
1	-1	1	-1	-1	0	-2	0
-1	1	1	-2	-1	0	-2	0
-1	-1	1	2	1	1	-1	-1

4. Find the weights required to perform the following classification using perceptron network. The vectors (1, 1, 1, 1) and (-1, 1, -1, -1) are belonging to the class (so have target value 1), vectors (1, 1, 1, -1) and (1, -1, -1, 1) are not belonging to the class (so have target value -1). Assume learning rate as 1 and initial weights as 0.

Solution: The truth table for the given vectors is given in Table 7.

Let $w_1 = w_2 = w_3 = w_4 = b = 0$ and the learning rate $\alpha = 1$. Since the threshold $\theta = 0.2$, so the activation function is

$$y = \begin{cases} 1 & \text{if } y_{in} > 0.2 \\ 0 & \text{if } -0.2 \leq y_{in} \leq 0.2 \\ -1 & \text{if } y_{in} < -0.2 \end{cases}$$

The net input is given by

$$y_{in} = b + x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4$$

The training is performed and the weights are tabulated in Table 8.

Table 7

Input					Target (t)
x_1	x_2	x_3	x_4	b	
1	1	1	1	1	1
-1	1	-1	-1	1	1
1	1	1	-1	1	-1
1	-1	-1	1	1	-1

Thus, in the third epoch, all the calculated outputs become equal to targets and the network has converged. The network convergence can also be checked by forming separating line equations for separating positive response regions from zero and zero from negative response region.

The network architecture is shown in Figure 5.

5. Classify the two-dimensional input pattern shown in Figure 6 using perceptron network. The symbol "*" indicates the data representation to be +1 and "-" indicates data to be -1. The patterns are I-F. For pattern I, the target is +1, and for F, the target is -1.

Table 8

Inputs					Target (t)	Net input (y_{in})	Output (y)	Weight changes					Weights					
(x_1	x_2	x_3	x_4	b)				(Δw_1	Δw_2	Δw_3	Δw_4	Δb)	(w_1	w_2	w_3	w_4	b)	
EPOCH-1																		
(1	1	1	1	1	1)	1	0	0	1	1	1	1	1	1	1	1	1	
(-1	1	-1	-1	1	1)	1	-1	-1	-1	1	-1	-1	1	0	2	0	0	2
(1	1	1	-1	1	1)	-1	4	1	-1	-1	-1	1	-1	-1	1	-1	1	1
(1	-1	-1	1	1	1)	-1	1	1	-1	1	1	-1	-1	-2	2	0	0	0
EPOCH-2																		
(1	1	1	1	1	1)	1	0	0	1	1	1	1	1	-1	3	1	1	1
(-1	1	-1	-1	1	1)	1	3	1	0	0	0	0	0	-1	3	1	1	1
(1	1	1	-1	1	1)	-1	4	1	-1	-1	-1	1	-1	-2	2	0	2	0
(1	-1	-1	1	1	1)	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0
EPOCH-3																		
(1	1	1	1	1	1)	1	2	1	0	0	0	0	0	-2	2	0	2	0
(-1	1	-1	-1	1	1)	1	2	1	0	0	0	0	0	-2	2	0	2	0
(1	1	1	-1	1	1)	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0
(1	-1	-1	1	1	1)	-1	-2	-1	0	0	0	0	0	-2	2	0	2	0

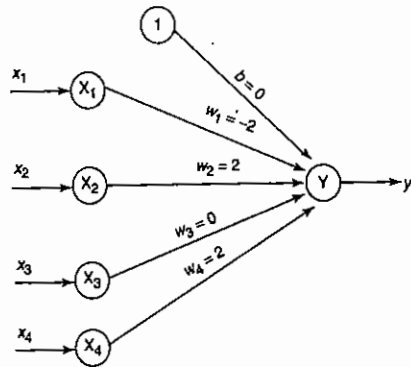


Figure 5 Network architecture.

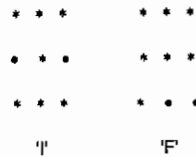


Figure 6 I-F data representation.

Solution: The training patterns for this problem are tabulated in Table 9.

Table 9

Pattern	Input									Target (t)
	x ₁	x ₂	x ₃	x ₄	x ₅	x ₆	x ₇	x ₈	x ₉	
I	1	1	1	-1	1	-1	1	1	1	1
F	1	1	1	1	1	1	-1	-1	-1	-1

The initial weights are all assumed to be zero, i.e., $\theta = 0$ and $\alpha = 1$. The activation function is given by

$$y = \begin{cases} 1 & \text{if } y_{in} > 0 \\ 0 & \text{if } -0 \leq y_{in} \leq 0 \\ -1 & \text{if } y_{in} < -0 \end{cases}$$

For the first input sample, $x_1 = [1 \ 1 \ 1 \ -1 \ 1 \ -1 \ 1 \ 1 \ 1]$, $t = 1$, the net input is calculated as

$$y_{in} = b + \sum_{i=1}^9 x_i w_i$$

$$\begin{aligned} &= b + x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4 + x_5 w_5 \\ &\quad + x_6 w_6 + x_7 w_7 + x_8 w_8 + x_9 w_9 \\ &= 0 + 1 \times 0 + 1 \times 0 + 1 \times 0 + (-1) \times 0 \\ &\quad + 1 \times 0 + (-1) \times 0 + 1 \times 0 + 1 \times 0 + 1 \times 0 \end{aligned}$$

$$y_{in} = 0$$

Therefore, by applying the activation function the output is given by $y = f(y_{in}) = 0$. Now since $t \neq y$, the new weights are computed as

$$\begin{aligned} w_1(\text{new}) &= w_1(\text{old}) + \alpha x_1 = 0 + 1 \times 1 \times 1 = 1 \\ w_2(\text{new}) &= w_2(\text{old}) + \alpha x_2 = 0 + 1 \times 1 \times 1 = 1 \\ w_3(\text{new}) &= w_3(\text{old}) + \alpha x_3 = 0 + 1 \times 1 \times 1 = 1 \\ w_4(\text{new}) &= w_4(\text{old}) + \alpha x_4 = 0 + 1 \times 1 \times (-1) = -1 \\ w_5(\text{new}) &= w_5(\text{old}) + \alpha x_5 = 0 + 1 \times 1 \times 1 = 1 \\ w_6(\text{new}) &= w_6(\text{old}) + \alpha x_6 = 0 + 1 \times 1 \times (-1) = -1 \\ w_7(\text{new}) &= w_7(\text{old}) + \alpha x_7 = 0 + 1 \times 1 \times 1 = 1 \\ w_8(\text{new}) &= w_8(\text{old}) + \alpha x_8 = 0 + 1 \times 1 \times 1 = 1 \\ w_9(\text{new}) &= w_9(\text{old}) + \alpha x_9 = 0 + 1 \times 1 \times 1 = 1 \\ b(\text{new}) &= b(\text{old}) + \alpha t = 0 + 1 \times 1 = 1 \end{aligned}$$

The weights after presenting first input sample are

$$w = [1 \ 1 \ 1 \ -1 \ 1 \ -1 \ 1 \ 1 \ 1]$$

For the second input sample, $x_2 = [1 \ 1 \ 1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1]$, $t = -1$, the net input is calculated as

$$\begin{aligned} y_{in} &= b + \sum_{i=1}^9 x_i w_i \\ &= b + x_1 w_1 + x_2 w_2 + x_3 w_3 + x_4 w_4 + x_5 w_5 \\ &\quad + x_6 w_6 + x_7 w_7 + x_8 w_8 + x_9 w_9 \\ &= 1 + 1 \times 1 + 1 \times 1 + 1 \times 1 + 1 \times (-1) + 1 \times 1 \\ &\quad + 1 \times (-1) + 1 \times 1 + (-1) \times 1 + (-1) \times 1 \end{aligned}$$

$$y_{in} = 2$$

Therefore the output is given by $y = f(y_{in}) = 1$. Since $t \neq y$, the new weights are

$$\begin{aligned} w_1(\text{new}) &= w_1(\text{old}) + \alpha x_1 = 1 + 1 \times 1 \times 1 = 0 \\ w_2(\text{new}) &= w_2(\text{old}) + \alpha x_2 = 1 + 1 \times 1 \times 1 = 0 \\ w_3(\text{new}) &= w_3(\text{old}) + \alpha x_3 = 1 + 1 \times 1 \times 1 = 0 \\ w_4(\text{new}) &= w_4(\text{old}) + \alpha x_4 = -1 + 1 \times 1 \times 1 = -2 \end{aligned}$$

$$\begin{aligned} w_5(\text{new}) &= w_5(\text{old}) + \alpha x_5 = 1 + 1 \times 1 \times 1 = 0 \\ w_6(\text{new}) &= w_6(\text{old}) + \alpha x_6 = -1 + 1 \times 1 \times 1 = -2 \\ w_7(\text{new}) &= w_7(\text{old}) + \alpha x_7 = 1 + 1 \times 1 \times 1 = 0 \\ w_8(\text{new}) &= w_8(\text{old}) + \alpha x_8 = 1 + 1 \times 1 \times (-1) = 2 \\ w_9(\text{new}) &= w_9(\text{old}) + \alpha x_9 = 1 + 1 \times 1 \times (-1) = 2 \\ b(\text{new}) &= b(\text{old}) + \alpha t = 1 + 1 \times (-1) = 0 \end{aligned}$$

The weights after presenting the second input sample are

$$w = [0 \ 0 \ 0 \ -2 \ 0 \ -2 \ 0 \ 2 \ 2]$$

The network architecture is as shown in Figure 7. The network can be further trained for its convergence.

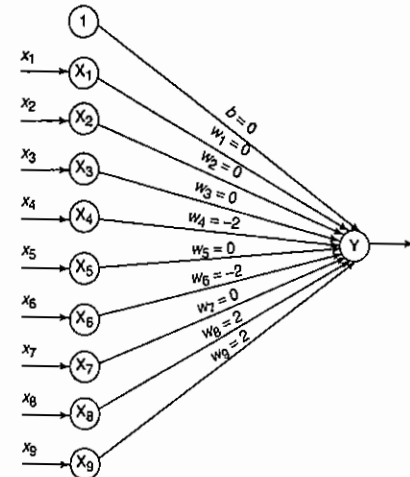


Figure 7 Network architecture.

6. Implement OR function with bipolar inputs and targets using Adaline network.

Solution: The truth table for OR function with bipolar inputs and targets is shown in Table 10.

Table 10

x ₁	x ₂	1	t
1	1	1	1
1	-1	1	1
-1	1	1	1
-1	-1	1	-1

Initially all the weights and links are assumed to be small random values, say 0.1, and the learning rate is also set to 0.1. Also here the least mean square error may be set. The weights are calculated until the least mean square error is obtained.

The initial weights are taken to be $w_1 = w_2 = b = 0.1$ and the learning rate $\alpha = 0.1$. For the first input sample, $x_1 = 1, x_2 = 1, t = 1$, we calculate the net input as

$$\begin{aligned} y_{in} &= b + \sum_{i=1}^n x_i w_i = b + \sum_{i=1}^2 x_i w_i \\ &= b + x_1 w_1 + x_2 w_2 \\ &= 0.1 + 1 \times 0.1 + 1 \times 0.1 = 0.3 \end{aligned}$$

Now compute $(t - y_{in}) = (1 - 0.3) = 0.7$. Updating the weights we obtain,

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_{in})x_i$$

where $\alpha(t - y_{in})x_i$ is called as weight change Δw_i . The new weights are obtained as

$$\begin{aligned} w_1(\text{new}) &= w_1(\text{old}) + \Delta w_1 = 0.1 + 0.1 \times 0.7 \times 1 \\ &= 0.1 + 0.07 = 0.17 \\ w_2(\text{new}) &= w_2(\text{old}) + \Delta w_2 = 0.1 \\ &\quad + 0.1 \times 0.7 \times 1 = 0.17 \\ b(\text{new}) &= b(\text{old}) + \Delta b = 0.1 + 0.1 \times 0.7 = 0.17 \end{aligned}$$

where

$$\Delta w_1 = \alpha(t - y_{in})x_1$$

$$\Delta w_2 = \alpha(t - y_{in})x_2$$

$$\Delta b = \alpha(t - y_{in})$$

Now we calculate the error:

$$E = (t - y_{in})^2 = (0.7)^2 = 0.49$$

The final weights after presenting first input sample are

$$w = [0.17 \ 0.17 \ 0.17]$$

and error $E = 0.49$.

These calculations are performed for all the input samples and the error is calculated. One epoch is completed when all the input patterns are presented. Summing up all the errors obtained for each input sample during one epoch will give the total mean square error of that epoch. The network training is continued until this error is minimized to a very small value.

Adopting the method above, the network training is done for OR function using Adaline network and is tabulated below in Table 11 for $\alpha = 0.1$.

The total mean square error after each epoch is given as in Table 12.

Thus from Table 12, it can be noticed that as training goes on, the error value gets minimized. Hence, further training can be continued for further minimization of error. The network architecture of Adaline network for OR function is shown in Figure 8.

Table 11

Inputs			Target	Net input	Weight changes				Weights			Error
x_1	x_2	1	t	y_{in}	$(t - y_{in})$	Δw_1	Δw_2	Δb	w_1	w_2	b	$(t - y_{in})^2$
									(0.1)	(0.1)	(0.1)	
EPOCH-1												
1	1	1	1	0.3	0.7	0.07	0.07	0.07	0.17	0.17	0.17	0.49
1	-1	1	1	0.17	0.83	0.083	-0.083	0.083	0.253	0.087	0.253	0.69
-1	1	1	1	0.087	0.913	-0.0913	0.0913	0.0913	0.1617	0.1783	0.3443	0.83
-1	-1	1	-1	0.0043	-1.0043	0.1004	0.1004	-0.1004	0.2621	0.2787	0.2439	1.01
EPOCH-2												
1	1	1	1	0.7847	0.2153	0.0215	0.0215	0.0215	0.2837	0.3003	0.2654	0.046
1	-1	1	1	0.2488	0.7512	0.7512	-0.0751	0.0751	0.3588	0.2251	0.3405	0.564
-1	1	1	1	0.2069	0.7931	-0.7931	0.0793	0.0793	0.2795	0.3044	0.4198	0.629
-1	-1	1	-1	-0.1641	-0.8359	0.0836	0.0836	-0.0836	0.3631	0.388	0.336	0.699
EPOCH-3												
1	1	1	1	1.0873	-0.0873	-0.087	-0.087	-0.087	0.3543	0.3793	0.3275	0.0076
1	-1	1	1	0.3025	+0.6975	0.0697	-0.0697	0.0697	0.4241	0.3096	0.3973	0.487
-1	1	1	1	0.2827	0.7173	-0.0717	0.0717	0.0717	0.3523	0.3813	0.469	0.515
-1	-1	1	-1	-0.2647	-0.7353	0.0735	0.0735	-0.0735	0.4259	0.4548	0.3954	0.541
EPOCH-4												
1	1	1	1	1.2761	-0.2761	-0.0276	-0.0276	-0.0276	0.3983	0.4272	0.3678	0.076
1	-1	1	1	0.3389	0.6611	0.0661	-0.0661	0.0661	0.4644	0.3611	0.4339	0.437
-1	1	1	1	0.3307	0.6693	-0.0669	0.0669	0.0699	0.3974	0.428	0.5009	0.448
-1	-1	1	-1	-0.3246	-0.6754	0.0675	0.0675	-0.0675	0.465	0.4956	0.4333	0.456
EPOCH-5												
1	1	1	1	1.3939	-0.3939	-0.0394	-0.0394	-0.0394	0.4256	0.4562	0.393	0.155
1	-1	1	1	0.3634	0.6366	0.0637	-0.0637	0.0637	0.4893	0.3925	0.457	0.405
-1	1	1	1	0.3609	0.6391	-0.0639	0.0639	0.0639	0.4253	0.4654	0.5215	0.408
-1	-1	1	-1	-0.3603	-0.6397	0.064	0.064	-0.064	0.4893	0.5204	0.4575	0.409

Table 12

Epoch	Total mean square error
Epoch 1	3.02
Epoch 2	1.938
Epoch 3	1.5506
Epoch 4	1.417
Epoch 5	1.377

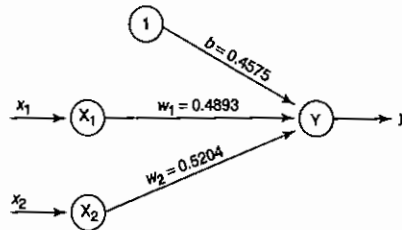


Figure 8 Network architecture of Adaline.

7. Use Adaline network to train ANDNOT function with bipolar inputs and targets. Perform 2 epochs of training.

Solution: The truth table for ANDNOT function with bipolar inputs and targets is shown in Table 13.

Table 13

x_1	x_2	1	t
1	1	1	-1
1	-1	1	1
-1	1	1	-1
-1	-1	1	-1

Initially the weights and bias have assumed a random value say 0.2. The learning rate is also set to 0.2. The weights are calculated until the least mean square error is obtained. The initial weights are $w_1 = w_2 = b = 0.2$, and $\alpha = 0.2$. For the first input sample $x_1 = 1$, $x_2 = 1$, $t = -1$, we calculate the net input as

$$y_{in} = b + x_1 w_1 + x_2 w_2 \\ = 0.2 + 1 \times 0.2 + 1 \times 0.2 = 0.6$$

Now compute $(t - y_{in}) = (-1 - 0.6) = -1.6$.
Updating the weights we obtain

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_{in})x_i$$

The new weights are obtained as

$$w_1(\text{new}) = w_1(\text{old}) + \alpha(t - y_{in})x_1 \\ = 0.2 + 0.2 \times (-1.6) \times 1 = -0.12$$

$$w_2(\text{new}) = w_2(\text{old}) + \alpha(t - y_{in})x_2 \\ = 0.2 + 0.2 \times (-1.6) \times 1 = -0.12$$

$$b(\text{new}) = b(\text{old}) + \alpha(t - y_{in}) \\ = 0.2 + 0.2 \times (-1.6) = -0.12$$

Now we compute the error,

$$E = (t - y_{in})^2 = (-1.6)^2 = 2.56$$

The final weights after presenting first input sample are $w = [-0.12 - 0.12 - 0.12]$ and error $E = 2.56$.

The operational steps are carried for 2 epochs of training and network performance is noted. It is tabulated as shown in Table 14.

The total mean square error at the end of two epochs is summation of the errors of all input samples as shown in Table 15.

Table 15

Epoch	Total mean square error
Epoch 1	5.71
Epoch 2	2.43

Hence from Table 15, it is clearly understood that the mean square error decreases as training progresses. Also, it can be noted that at the end of the sixth epoch, the error becomes approximately equal to 1. The network architecture for ANDNOT function using Adaline network is shown in Figure 9.

Table 14

Inputs			Target t	Net input y_{in}	$(t - y_{in})$	Weight changes			Weights			Error $(t - y_{in})^2$
x_1	x_2					Δw_1	Δw_2	Δb	w_1 (0.2)	w_2 (0.2)	b (0.2)	
EPOCH-1												
1	1	1	-1	0.6	-1.6	-0.32	-0.32	-0.32	-0.12	-0.12	-0.12	2.56
1	-1	1	1	-0.12	1.12	0.22	-0.22	0.22	0.10	-0.34	0.10	1.25
-1	1	1	-1	-0.34	-0.66	0.13	-0.13	-0.13	0.24	-0.48	-0.03	0.43
-1	-1	1	-1	0.21	-1.2	0.24	0.24	-0.24	0.48	-0.23	-0.27	1.47
EPOCH-2												
1	1		-1	-0.02	-0.98	-0.195	-0.195	-0.195	0.28	-0.43	-0.46	0.95
1	-1	1	1	0.25	0.76	0.15	-0.15	0.15	0.43	-0.58	-0.31	0.57
-1	1	1	-1	-1.33	0.33	-0.065	0.065	0.065	0.37	-0.51	-0.25	0.106
-1	-1	1	-1	-0.11	-0.90	0.18	0.18	-0.18	0.55	-0.38	0.43	0.8

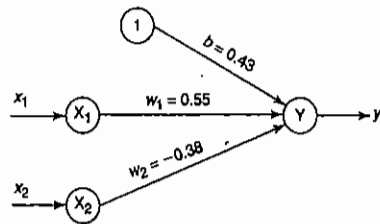


Figure 9 Network architecture for ANDNOT function using Adaline network.

- 8 Using Madaline network, implement XOR function with bipolar inputs and targets. Assume the required parameters for training of the network.

Solution: The training pattern for XOR function is given in Table 16.

Table 16

x_1	x_2	1	t
1	1	1	-1
1	-1	1	1
-1	1	1	1
-1	-1	1	-1

The Madaline Rule I (MRI) algorithm in which the weights between the hidden layer and output layer remain fixed is used for training the network. Initializing the weights to small random values, the network architecture is as shown in Figure 10, with initial weights. From Figure 10, the initial weights and bias are $[w_{11} \ w_{21} \ b_1] = [0.05 \ 0.2 \ 0.3]$, $[w_{12} \ w_{22} \ b_2] = [0.1 \ 0.2 \ 0.15]$ and $[v_1 \ v_2 \ b_3] = [0.5 \ 0.5 \ 0.5]$. For first

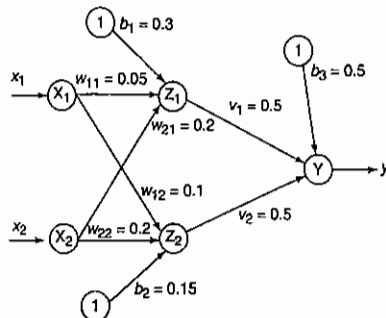


Figure 10 Network architecture of Madaline for XOR functions (initial weights given).

input sample, $x_1 = 1, x_2 = 1$, target $t = -1$, and learning rate α equal to 0.5:

- Calculate net input to the hidden units:

$$\begin{aligned} z_{in1} &= b_1 + x_1 w_{11} + x_2 w_{21} \\ &= 0.3 + 1 \times 0.05 + 1 \times 0.2 = 0.55 \\ z_{in2} &= b_2 + x_1 w_{12} + x_2 w_{22} \\ &= 0.15 + 1 \times 0.1 + 1 \times 0.2 = 0.45 \end{aligned}$$

- Calculate the output z_1, z_2 by applying the activations over the net input computed. The activation function is given by

$$f(z_{in}) = \begin{cases} 1 & \text{if } z_{in} \geq 0 \\ -1 & \text{if } z_{in} < 0 \end{cases}$$

Hence,

$$\begin{aligned} z_1 &= f(z_{in1}) = f(0.55) = 1 \\ z_2 &= f(z_{in2}) = f(0.45) = 1 \end{aligned}$$

- After computing the output of the hidden units, then find the net input entering into the output unit:

$$\begin{aligned} y_{in} &= b_3 + z_1 v_1 + z_2 v_2 \\ &= 0.5 + 1 \times 0.5 + 1 \times 0.5 = 1.5 \end{aligned}$$

- Apply the activation function over the net input y_{in} to calculate the output y :

$$y = f(y_{in}) = f(1.5) = 1$$

- Since $t \neq y$, weight update has to be performed. Also since $t = -1$, the weights are updated on z_1 and z_2 that have positive net input. Since here both net inputs z_{in1} and z_{in2} are positive, updating the weights and bias on both hidden units, we obtain

$$\begin{aligned} w_{ij}(\text{new}) &= w_{ij}(\text{old}) + \alpha(t - z_{inj})x_i \\ b_j(\text{new}) &= b_j(\text{old}) + \alpha(t - z_{inj}) \end{aligned}$$

This implies:

$$\begin{aligned} w_{11}(\text{new}) &= w_{11}(\text{old}) + \alpha(t - z_{in1})x_1 \\ &= 0.05 + 0.5(-1 - 0.55) \times 1 = -0.725 \end{aligned}$$

$$\begin{aligned} w_{12}(\text{new}) &= w_{12}(\text{old}) + \alpha(t - z_{in2})x_1 \\ &= 0.1 + 0.5(-1 - 0.45) \times 1 = -0.625 \end{aligned}$$

$$\begin{aligned} b_1(\text{new}) &= b_1(\text{old}) + \alpha(t - z_{in1}) \\ &= 0.3 + 0.5(-1 - 0.55) = -0.475 \end{aligned}$$

$$\begin{aligned} w_{21}(\text{new}) &= w_{21}(\text{old}) + \alpha(t - z_{in1})x_2 \\ &= 0.2 + 0.5(-1 - 0.55) \times 1 = -0.575 \\ w_{22}(\text{new}) &= w_{22}(\text{old}) + \alpha(t - z_{in2})x_2 \\ &= 0.2 + 0.5(-1 - 0.45) \times 1 = -0.525 \\ b_2(\text{new}) &= b_2(\text{old}) + \alpha(t - z_{in2}) \\ &= 0.15 + 0.5(-1 - 0.45) = -0.575 \end{aligned}$$

All the weights and bias between the input layer and hidden layer are adjusted. This completes the training for the first epoch. The same process is repeated until the weight converges. It is found that the weight converges at the end of 3 epochs. Table 17 shows the training performance of Madaline network for XOR function.

The network architecture for Madaline network with final weights for XOR function is shown in Figure 11.

9. Using back-propagation network, find the new weights for the net shown in Figure 12. It is presented with the input pattern $[0, 1]$ and the target output is 1. Use a learning rate $\alpha = 0.25$ and binary sigmoidal activation function.

Solution: The new weights are calculated based on the training algorithm in Section 3.5.4. The initial weights are $[v_{11} \ v_{21} \ v_{01}] = [0.6 \ -0.1 \ 0.3]$.

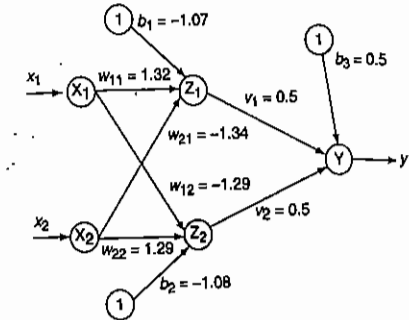


Figure 11 Madaline network for XOR function (final weights given).

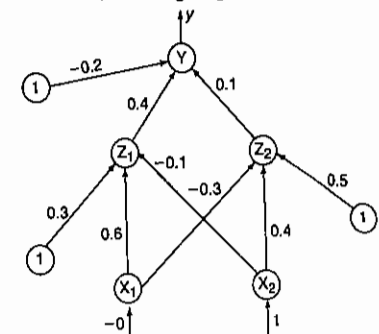


Figure 12 Network.

Table 17

Inputs		Target													
x_1	x_2	1	(t)	z_{in1}	z_{in2}	z_1	z_2	y_{in}	y	w_{11}	w_{21}	b_1	w_{12}	w_{22}	b_2
EPOCH-1															
1	1	1	-1	0.55	0.45	1	1	1.5	1	-0.725	-0.58	-0.475	-0.625	-0.525	-0.575
1	-1	1	1	-0.625	-0.675	-1	-1	-0.5	-1	0.0875	-1.39	0.34	-0.625	-0.525	-0.575
-1	1	1	1	-1.1375	-0.475	-1	-1	-0.5	-1	0.0875	-1.39	0.34	-1.3625	0.2125	0.1625
-1	-1	1	-1	1.6375	1.3125	1	1	1.5	1	1.4065	-0.069	-0.98	-0.207	1.369	-0.994
EPOCH-2															
1	1	1	-1	0.3565	0.168	1	1	1.5	1	0.7285	-0.75	-1.66	-0.791	-0.207	-1.58
1	-1	1	1	-0.1845	-3.154	-1	-1	-0.5	-1	1.3205	-1.34	-1.068	-0.791	0.785	-1.58
-1	1	1	1	-3.728	-0.002	-1	-1	-0.5	-1	1.3205	-1.34	-1.068	-1.29	0.785	-1.08
-1	-1	1	-1	-1.0495	-1.071	-1	-1	-0.5	-1	1.3205	-1.34	-1.068	-1.29	1.29	-1.08
EPOCH-3															
1	1	1	-1	-1.0865	-1.083	-1	-1	-0.5	-1	1.32	-1.34	-1.07	-1.29	1.29	-1.08
1	-1	1	1	1.5915	-3.655	1	-1	0.5	1	1.32	-1.34	-1.07	-1.29	1.29	-1.08
-1	1	1	1	-3.728	1.501	-1	1	0.5	1	1.32	-1.34	-1.07	-1.29	1.29	-1.08
-1	-1	1	-1	-1.0495	-1.701	-1	-1	-0.5	-1	1.32	-1.34	-1.07	-1.29	1.29	-1.08

$[v_{12} \ v_{22} \ v_{02}] = [-0.3 \ 0.4 \ 0.5]$ and $[w_1 \ w_2 \ w_0] = [0.4 \ 0.1 \ -0.2]$, and the learning rate is $\alpha = 0.25$. Activation function used is binary sigmoidal activation function and is given by

$$f(x) = \frac{1}{1 + e^{-x}}$$

Given the output sample $[x_1, x_2] = [0, 1]$ and target $t = 1$,

- Calculate the net input: For z_1 layer

$$z_{in1} = v_{01} + x_1 v_{11} + x_2 v_{21} \\ = 0.3 + 0 \times 0.6 + 1 \times -0.1 = 0.2$$

For z_2 layer

$$z_{in2} = v_{02} + x_1 v_{12} + x_2 v_{22} \\ = 0.5 + 0 \times -0.3 + 1 \times 0.4 = 0.9$$

Applying activation to calculate the output, we obtain

$$z_1 = f(z_{in1}) = \frac{1}{1 + e^{-z_{in1}}} = \frac{1}{1 + e^{-0.2}} = 0.5498 \\ z_2 = f(z_{in2}) = \frac{1}{1 + e^{-z_{in2}}} = \frac{1}{1 + e^{-0.9}} = 0.7109$$

- Calculate the net input entering the output layer. For y layer

$$y_{in} = w_0 + z_1 w_1 + z_2 w_2 \\ = -0.2 + 0.5498 \times 0.4 + 0.7109 \times 0.1 \\ = 0.09101$$

Applying activations to calculate the output, we obtain

$$y = f(y_{in}) = \frac{1}{1 + e^{-y_{in}}} = \frac{1}{1 + e^{-0.09101}} = 0.5227$$

- Compute the error portion δ_k :

$$\delta_k = (t_k - y_k) f'(y_{ink})$$

Now

$$f'(y_{in}) = f(y_{in})[1 - f(y_{in})] = 0.5227[1 - 0.5227] \\ f'(y_{in}) = 0.2495$$

This implies

$$\delta_1 = (1 - 0.5227)(0.2495) = 0.1191$$

Find the changes in weights between hidden and output layer:

$$\Delta w_1 = \alpha \delta_1 z_1 = 0.25 \times 0.1191 \times 0.5498 \\ = 0.0164$$

$$\Delta w_2 = \alpha \delta_1 z_2 = 0.25 \times 0.1191 \times 0.7109 \\ = 0.02117$$

$$\Delta w_0 = \alpha \delta_1 = 0.25 \times 0.1191 = 0.02978$$

- Compute the error portion δ_j between input and hidden layer ($j = 1$ to 2):

$$\delta_j = \delta_{inj} f'(z_{inj})$$

$$\delta_{inj} = \sum_{k=1}^m \delta_k w_{jk}$$

$$\delta_{inj} = \delta_1 w_{j1} \quad [\because \text{only one output neuron}]$$

$$\Rightarrow \delta_{in1} = \delta_1 w_{11} = 0.1191 \times 0.4 = 0.04764$$

$$\Rightarrow \delta_{in2} = \delta_1 w_{21} = 0.1191 \times 0.1 = 0.01191$$

$$\text{Error, } \delta_1 = \delta_{in1} f'(z_{in1})$$

$$f'(z_{in1}) = f(z_{in1})[1 - f(z_{in1})] \\ = 0.5498[1 - 0.5498] = 0.2475$$

$$\delta_1 = \delta_{in1} f'(z_{in1}) \\ = 0.04764 \times 0.2475 = 0.0118$$

$$\text{Error, } \delta_2 = \delta_{in2} f'(z_{in2})$$

$$f'(z_{in2}) = f(z_{in2})[1 - f(z_{in2})] \\ = 0.7109[1 - 0.7109] = 0.2055$$

$$\delta_2 = \delta_{in2} f'(z_{in2}) \\ = 0.01191 \times 0.2055 = 0.00245$$

Now find the changes in weights between input and hidden layer:

$$\Delta v_{11} = \alpha \delta_1 x_1 = 0.25 \times 0.0118 \times 0 = 0$$

$$\Delta v_{21} = \alpha \delta_1 x_2 = 0.25 \times 0.0118 \times 1 = 0.00295$$

$$\Delta v_{01} = \alpha \delta_1 = 0.25 \times 0.0118 = 0.00295$$

$$\Delta v_{12} = \alpha \delta_2 x_1 = 0.25 \times 0.00245 \times 0 = 0$$

$$\Delta v_{22} = \alpha \delta_2 x_2 = 0.25 \times 0.00245 \times 1 = 0.0006125$$

$$\Delta v_{02} = \alpha \delta_2 = 0.25 \times 0.00245 = 0.0006125$$

- Compute the final weights of the network:

$$v_{11}(\text{new}) = v_{11}(\text{old}) + \Delta v_{11} = 0.6 + 0 = 0.6$$

$$v_{12}(\text{new}) = v_{12}(\text{old}) + \Delta v_{12} = -0.3 + 0 = -0.3$$

$$v_{21}(\text{new}) = v_{21}(\text{old}) + \Delta v_{21} \\ = -0.1 + 0.00295 = -0.09705$$

$$v_{22}(\text{new}) = v_{22}(\text{old}) + \Delta v_{22} \\ = 0.4 + 0.0006125 = 0.4006125$$

$$w_1(\text{new}) = w_1(\text{old}) + \Delta w_1 = 0.4 + 0.0164 \\ = 0.4164$$

$$w_2(\text{new}) = w_2(\text{old}) + \Delta w_2 = 0.1 + 0.02117 \\ = 0.12117$$

$$v_{01}(\text{new}) = v_{01}(\text{old}) + \Delta v_{01} = 0.3 + 0.00295 \\ = 0.30295$$

$$v_{02}(\text{new}) = v_{02}(\text{old}) + \Delta v_{02} \\ = 0.5 + 0.0006125 = 0.5006125$$

$$w_0(\text{new}) = w_0(\text{old}) + \Delta w_0 = -0.2 + 0.02978 \\ = -0.17022$$

Thus, the final weights have been computed for the network shown in Figure 12.

- Find the new weights, using back-propagation network for the network shown in Figure 13. The network is presented with the input pattern $[-1, 1]$ and the target output is $+1$. Use a learning rate of $\alpha = 0.25$ and bipolar sigmoidal activation function.

Solution: The initial weights are $[v_{11} \ v_{21} \ v_{01}] = [0.6 \ 0.1 \ 0.3]$, $[v_{12} \ v_{22} \ v_{02}] = [-0.3 \ 0.4 \ 0.5]$ and $[w_1 \ w_2 \ w_0] = [0.4 \ 0.1 \ -0.2]$, and the learning rate is $\alpha = 0.25$.

Activation function used is binary sigmoidal activation function and is given by

$$f(x) = \frac{2}{1 + e^{-x}} - 1 = \frac{1 - e^{-x}}{1 + e^{-x}}$$

Given the input sample $[x_1, x_2] = [-1, 1]$ and target $t = 1$:

- Calculate the net input: For z_1 layer

$$z_{in1} = v_{01} + x_1 v_{11} + x_2 v_{21} \\ = 0.3 + (-1) \times 0.6 + 1 \times -0.1 = -0.4$$

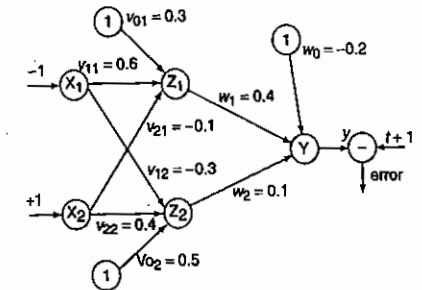


Figure 13 Network.

For z_2 layer

$$z_{in2} = v_{02} + x_1 v_{12} + x_2 v_{22} \\ = 0.5 + (-1) \times -0.3 + 1 \times 0.4 = 1.2$$

Applying activation to calculate the output, we obtain

$$z_1 = f(z_{in1}) = \frac{1 - e^{-z_{in1}}}{1 + e^{-z_{in1}}} = \frac{1 - e^{-0.4}}{1 + e^{-0.4}} = -0.1974 \\ z_2 = f(z_{in2}) = \frac{1 - e^{-z_{in2}}}{1 + e^{-z_{in2}}} = \frac{1 - e^{-1.2}}{1 + e^{-1.2}} = 0.537$$

- Calculate the net input entering the output layer. For y layer

$$y_{in} = w_0 + z_1 w_1 + z_2 w_2 \\ = -0.2 + (-0.1974) \times 0.4 + 0.537 \times 0.1 \\ = -0.22526$$

Applying activations to calculate the output, we obtain

$$y = f(y_{in}) = \frac{1 - e^{-y_{in}}}{1 + e^{-y_{in}}} = \frac{1 - e^{-0.22526}}{1 + e^{-0.22526}} = -0.1122$$

- Compute the error portion δ_k :

$$\delta_k = (t_k - y_k) f'(y_{ink})$$

Now

$$f'(y_{in}) = 0.5[1 + f(y_{in})][1 - f(y_{in})] \\ = 0.5[1 - 0.1122][1 + 0.1122] = 0.4937$$

This implies

$$\delta_1 = (1 + 0.1122)(0.4937) = 0.5491$$

Find the changes in weights between hidden and output layer:

$$\Delta w_1 = \alpha \delta_1 z_1 = 0.25 \times 0.5491 \times -0.1974 = -0.0271$$

$$\Delta w_2 = \alpha \delta_1 z_2 = 0.25 \times 0.5491 \times 0.537 = 0.0737$$

$$\Delta w_0 = \alpha \delta_1 = 0.25 \times 0.5491 = 0.1373$$

- Compute the error portion δ_j between input and hidden layer ($j = 1$ to 2):

$$\delta_j = \delta_{inj} f'(z_{inj})$$

$$\delta_{inj} = \sum_{k=1}^m \delta_k w_{jk}$$

$$\delta_{inj} = \delta_1 w_{j1} \quad [\because \text{only one output neuron}]$$

$$\Rightarrow \delta_{in1} = \delta_1 w_{11} = 0.5491 \times 0.4 = 0.21964$$

$$\Rightarrow \delta_{in2} = \delta_1 w_{21} = 0.5491 \times 0.1 = 0.05491$$

$$\text{Error, } \delta_1 = \delta_{in1} f'(z_{in1}) = 0.21964 \times 0.5 \times (1 + 0.1974)(1 - 0.1974) = 0.1056$$

$$\text{Error, } \delta_2 = \delta_{in2} f'(z_{in2}) = 0.05491 \times 0.5 \times (1 - 0.537)(1 + 0.537) = 0.0195$$

Now find the changes in weights between input and hidden layer:

$$\Delta v_{11} = \alpha \delta_1 x_1 = 0.25 \times 0.1056 \times -1 = -0.0264$$

$$\Delta v_{21} = \alpha \delta_1 x_2 = 0.25 \times 0.1056 \times 1 = 0.0264$$

$$\Delta v_{01} = \alpha \delta_1 = 0.25 \times 0.1056 = 0.0264$$

$$\Delta v_{12} = \alpha \delta_2 x_1 = 0.25 \times 0.0195 \times -1 = -0.0049$$

$$\Delta v_{22} = \alpha \delta_2 x_2 = 0.25 \times 0.0195 \times 1 = 0.0049$$

$$\Delta v_{02} = \alpha \delta_2 = 0.25 \times 0.0195 = 0.0049$$

- Compute the final weights of the network:

$$v_{11}(\text{new}) = v_{11}(\text{old}) + \Delta v_{11} = 0.6 - 0.0264 = 0.5736$$

$$v_{12}(\text{new}) = v_{12}(\text{old}) + \Delta v_{12} = -0.3 - 0.0049 = -0.3049$$

$$v_{21}(\text{new}) = v_{21}(\text{old}) + \Delta v_{21} = -0.1 + 0.0264 = -0.0736$$

$$v_{22}(\text{new}) = v_{22}(\text{old}) + \Delta v_{22} = 0.4 + 0.0049 = 0.4049$$

$$w_1(\text{new}) = w_1(\text{old}) + \Delta w_1 = 0.4 - 0.0271 = 0.3729$$

$$w_2(\text{new}) = w_2(\text{old}) + \Delta w_2 = 0.1 + 0.0737 = 0.1737$$

$$v_{01}(\text{new}) = v_{01}(\text{old}) + \Delta v_{01} = 0.3 + 0.0264 = 0.3264$$

$$v_{02}(\text{new}) = v_{02}(\text{old}) + \Delta v_{02} = 0.5 + 0.0049 = 0.5049$$

$$w_0(\text{new}) = w_0(\text{old}) + \Delta w_0 = -0.2 + 0.1373 = -0.0627$$

Thus, the final weight has been computed for the network shown in Figure 13.

3.13 Review Questions

1. What is supervised learning and how is it different from unsupervised learning?
2. How does learning take place in supervised learning?
3. From a mathematical point of view, what is the process of learning in supervised learning?
4. What is the building block of the perceptron?
5. Does perceptron require supervised learning? If no, what does it require?
6. List the limitations of perceptron.
7. State the activation function used in perceptron network.
8. What is the importance of threshold in perceptron network?
9. Mention the applications of perceptron network.
10. What are feature detectors?
11. With a neat flowchart, explain the training process of perceptron network.
12. What is the significance of error signal in perceptron network?

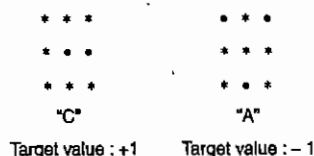
13. State the testing algorithm used in perceptron algorithm.
14. How is the linear separability concept implemented using perceptron network training?
15. Define perceptron learning rule.
16. Define delta rule.
17. State the error function for delta rule.
18. What is the drawback of using optimization algorithm?
19. What is Adaline?
20. Draw the model of an Adaline network.
21. Explain the training algorithm used in Adaline network.
22. How is a Madaline network formed?
23. Is it true that Madaline network consists of many perceptrons?
24. State the characteristics of weighted interconnections between Adaline and Madaline.
25. How is training adopted in Madaline network using majority vote rule?
26. State few applications of Adaline and Madaline.
27. What is meant by epoch in training process?
28. What is meant by gradient descent method?
29. State the importance of back-propagation algorithm.
30. What is called as memorization and generalization?
31. List the stages involved in training of back-propagation network.
32. Draw the architecture of back-propagation algorithm.
33. State the significance of error portions δ_k and δ_j in BPN algorithm.
34. What are the activations used in back-propagation network algorithm?
35. What is meant by local minima and global minima?
36. Derive the generalized delta learning rule.
37. Derive the derivations of the binary and bipolar sigmoidal activation function.
38. What are the factors that improve the convergence of learning in BPN network?
39. What is meant by incremental learning?
40. Why is gradient descent method adopted to minimize error?
41. What are the methods of initialization of weights?
42. What is the necessity of momentum factor in weight updation process?
43. Define "over fitting" or "over training."
44. State the techniques for proper choice of learning rate.
45. What are the limitations of using momentum factor?
46. How many hidden layers can there be in a neural network?
47. What is the activation function used in radial basis function network?
48. Explain the training algorithm of radial basis function network.
49. By what means can an IIR and an FIR filter be formed in neural network?
50. What is the importance of functional link network?
51. Write a short note on binary classification tree neural network.
52. Explain in detail about wavelet neural network.

3.14 Exercise Problems

1. Implement NOR function using perceptron network for bipolar inputs and targets.
2. Find the weights required to perform the following classifications using perceptron network. The vectors $(1, 1, -1, -1)$ and $(1, -1, 1, -1)$

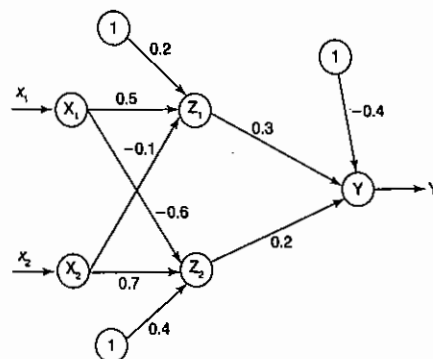
are belonging to the class (so have target value 1), vector $(-1, -1, -1, 1)$ and $(-1, -1, 1, 1)$ are not belonging to the class (so have target value -1). Assume learning rate 1 and initial weights as 0.

3. Classify the two-dimensional pattern shown in figure below using perceptron network.



4. Implement AND function using Adaline network.
5. Using the delta rule, find the weights required to perform following classifications: Vectors (1, 1, -1, -1) and (-1, -1, -1, -1) are belonging to the class having target value 1; vectors (1, 1, 1, 1) and (-1, -1, 1, -1) are not belonging to the class having target value -1. Use a learning rate of 0.5 and assume random value of weights. Also, using each of the training vectors as input, test the response of the net.
6. Implement AND function using Madaline network.
7. With suitable example, discuss the perceptron network training with and without bias.
8. Using back-propagation network, find the new weights for the network shown in the following figure. The network is presented with the input

pattern [1, 0] and target output 1. Use learning rate of $\alpha = 0.3$ and binary sigmoidal activation function.



9. Find the new weights for the network given in the above problem using back-propagation network. The network is presented the input pattern [1, -1] and target output +1. Use learning rate of $\alpha = 0.3$ and bipolar sigmoidal activation function.
10. Find the new weights for the activation function with the network shown in problem 8 using BPN. The network is presented with the input pattern [-1, 1] and target output -1. Use learning rate of $\alpha = 0.45$ and suitable activation function.

3.15 Projects

- Classify upper case letters and lower case letters using perceptron network. Use as many output units based on training set as possible. Test the network with noisy pattern as well.
- Write a suitable computer program to classify the numbers between 0-9 using Adaline network.
- Write a computer program to train a Madaline to perform AND function using MRI algorithm.
- Write a program for implementing BPN for training a single hidden layer back-propagation network with bipolar sigmoidal units ($x = 1$) to

achieve the following two-to-one mappings.

$$\begin{aligned} y &= 6 \sin(\pi x_1) + \cos(\pi x_2) \\ y &= \sin(\pi x_1) + \cos(0.2\pi x_2) \end{aligned}$$

Set up two sets of data, each consisting of 10 input-output pairs, one for training and other for testing. The input-output data are obtained by varying input variables (x_1, x_2) within $[-1, +1]$ randomly. Also the output data is normalized within $[-1, 1]$. Apply training to find proper weights in the network.

Associative Memory Networks

4

Learning Objectives

- Gives details on associative memories.
- Discusses the training algorithm used for pattern association networks - Hebb rule and outer products rule.
- The architecture, flowchart for training process, training algorithm and testing algorithm of autoassociative, heteroassociative and bidirectional associative memory are discussed in detail.
- Variants of BAM - continuous BAM and discrete BAM are included.
- Hopfield network with its electrical model is described with training algorithm.
- Analysis of energy function was performed for BAM, discrete and continuous Hopfield networks.
- An overview is given on the iterative autoassociative network - linear autoassociator memory brain-in-the-box network and autoassociator with threshold unit.
- Also temporal associative memory is discussed in brief.

4.1 Introduction

An associative memory network can store a set of patterns as memories. When the associative memory is being presented with a key pattern, it responds by producing one of the stored patterns, which closely resembles or relates to the key pattern. Thus, the recall is through association of the key pattern, with the help of information memorized. These types of memories are also called as content-addressable memories (CAM) in contrast to that of traditional address-addressable memories in digital computers where stored pattern (in bytes) is recalled by its address. It is also a matrix memory as in RAM/ROM. The CAM can also be viewed as associating data to address, i.e., for every data in the memory there is a corresponding unique address. Also, it can be viewed as data correlator. Here input data is correlated with that of the stored data in the CAM. It should be noted that the stored patterns must be unique, i.e., different patterns in each location. If the same pattern exists in more than one location in the CAM, then, even though the correlation is correct, the address is noted to be ambiguous. The basic structure of CAM is given in Figure 4-1.

Associative memory makes a parallel search within a stored data file. The concept behind this search is to output any one or all stored items which match the given search argument and to retrieve the stored data either completely or partially.

Two types of associative memories can be differentiated. They are autoassociative memory and heteroassociative memory. Both these nets are single-layer nets in which the weights are determined in a manner that the net stores a set of pattern associations. Each of this association is an input-output vector pair, say, x_i . If each of the output vectors is same as the input vectors with which it is associated, then the net is said to

single data set
data set
data set
data set
data set

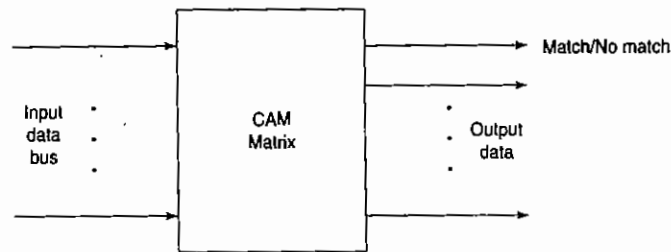


Figure 4-1 CAM architecture.

be autoassociative memory net. On the other hand, if the output vectors are different from the input vectors then the net is said to be heteroassociative memory net.

If there exist vectors, say, $x = (x_1, x_2, \dots, x_n)^T$ and $x' = (x'_1, x'_2, \dots, x'_n)^T$, then the hamming distance (HD) is defined as the number of mismatched components of x and x' vectors, i.e.,

$$HD(x, x') = \begin{cases} \sum_{i=1}^n |x_i - x'_i| & \text{if } x_i, x'_i \in [0, 1] \\ \frac{1}{2} \sum_{i=1}^n |x_i - x'_i| & \text{if } x_i, x'_i \in [-1, 1] \end{cases}$$

The architecture of an associative net may be either feed-forward or iterative (recurrent). As is already known, in a feed-forward net the information flows from the input units to the output units; on the other hand, in a recurrent neural net, there are connections among the units to form a closed-loop structure. In the forthcoming sections, we will discuss the training algorithms used for pattern association and various types of association nets in detail.

4.2 Training Algorithms for Pattern Association

There are two algorithms developed for training of pattern association nets. These are discussed below.

4.2.1 Hebb Rule

The Hebb rule is widely used for finding the weights of an associative memory neural net. The training vector pairs here are denoted as s, t . The flowchart for the training algorithm of pattern association is as shown in Figure 4-2. The weights are updated until there is no weight change. The algorithmic steps followed are given below:

Step 0: Set all the initial weights to zero, i.e.,

$$w_{ij} = 0 \quad (i = 1 \text{ to } n, j = 1 \text{ to } m)$$

Step 1: For each training target input output vector pairs s, t , perform Steps 2-4.

Step 2: Activate the input layer units to current training input,

$$x_i = s_i \quad (\text{for } i = 1 \text{ to } n)$$

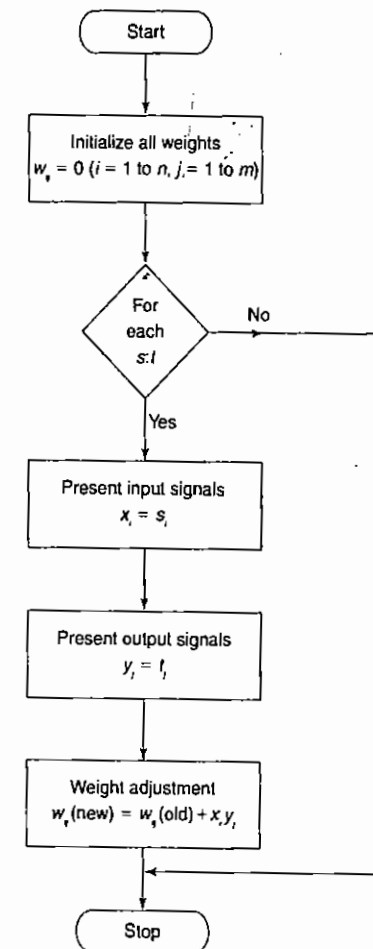


Figure 4-2 Flowchart for Hebb rule.

Step 3: Activate the output layer units to current target output,

$$y_j = t_j \quad (\text{for } j = 1 \text{ to } m)$$

Step 4: Start the weight adjustment

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + x_i y_j \quad (\text{for } i = 1 \text{ to } n, j = 1 \text{ to } m)$$

This algorithm is used for the calculation of the weights of the associative nets. Also, it can be used with patterns that are being represented as either binary or bipolar vectors.

4.2.2 Outer Products Rule

Outer products rule is an alternative method for finding weights of an associative net. This is depicted as follows:

$$\text{Input} \Rightarrow s = (s_1, \dots, s_i, \dots, s_n)$$

$$\text{Output} \Rightarrow t = (t_1, \dots, t_j, \dots, t_m)$$

The outer product of the two vectors is the product of the matrices $S = s^T$ and $T = t$, i.e., between $[n \times 1]$ matrix and $[1 \times m]$ matrix. The transpose is to be taken for the input matrix given.

The matrix multiplication is done as follows:

$$ST = s^T t$$

$$= \begin{bmatrix} s_1 \\ \vdots \\ s_i \\ \vdots \\ s_n \end{bmatrix}_{n \times 1} \begin{bmatrix} t_1 & \dots & t_j & \dots & t_m \end{bmatrix}_{1 \times m}$$

$$W = \begin{bmatrix} s_1 t_1 & \dots & s_1 t_j & \dots & s_1 t_m \\ \vdots & & \vdots & & \vdots \\ s_i t_1 & \dots & s_i t_j & \dots & s_i t_m \\ \vdots & & \vdots & & \vdots \\ s_n t_1 & \dots & s_n t_j & \dots & s_n t_m \end{bmatrix}_{n \times m}$$

This weight matrix is same as the weight matrix obtained by Hebb rule to store the pattern association $s:t$. For storing a set of associations, $s(p):t(p)$, $p = 1$ to P , wherein,

$$s(p) = (s_1(p), \dots, s_i(p), \dots, s_n(p))$$

$$t(p) = (t_1(p), \dots, t_j(p), \dots, t_m(p))$$

the weight matrix $W = \{w_{ij}\}$ can be given as

$$w_{ij} = \sum_{p=1}^P s_i^T(p) t_j(p)$$

This can also be rewritten as

$$W = \sum_{p=1}^P s^T(p) t(p)$$

for finding the weights of the net using Hebbian learning. Similar to the Hebb rule, even the delta rule discussed in Chapter 2 can be used for storing weights of pattern association nets.

4.3 Autoassociative Memory Network

4.3.1 Theory

In the case of an autoassociative neural net, the training input and the target output vectors are the same. The determination of weights of the association net is called storing of vectors. This type of memory net needs suppression of the output noise at the memory output. The vectors that have been stored can be retrieved from distorted (noisy) input if the input is sufficiently similar to it. The net's performance is based on its ability to reproduce a stored pattern from a noisy input. It should be noted, that in the case of autoassociative net, the weights on the diagonal can be set to zero. This can be called as auto associative net with no self-connection. The main reason behind setting the weights to zero is that it improves the net's ability to generalize or increase the biological plausibility of the net. This may be more suited for iterative nets and when delta rule is being used.

4.3.2 Architecture

The architecture of an autoassociative neural net is shown in Figure 4-3. It shows that for an autoassociative net, the training input and target output vectors are the same. The input layer consists of n input units and the output layer also consists of n output units. The input and output layers are connected through weighted interconnections. The input and output vectors are perfectly correlated with each other component by component.

4.3.3 Flowchart for Training Process

The flowchart here is the same as discussed in Section 4.2.1, but it may be noted that the number of input units and output units are the same. The flowchart is shown in Figure 4-4.

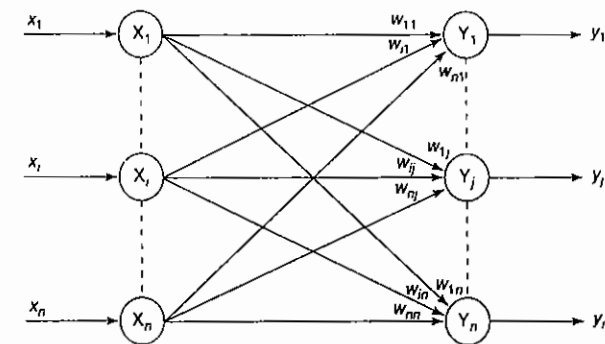


Figure 4-3 Architecture of autoassociative net.

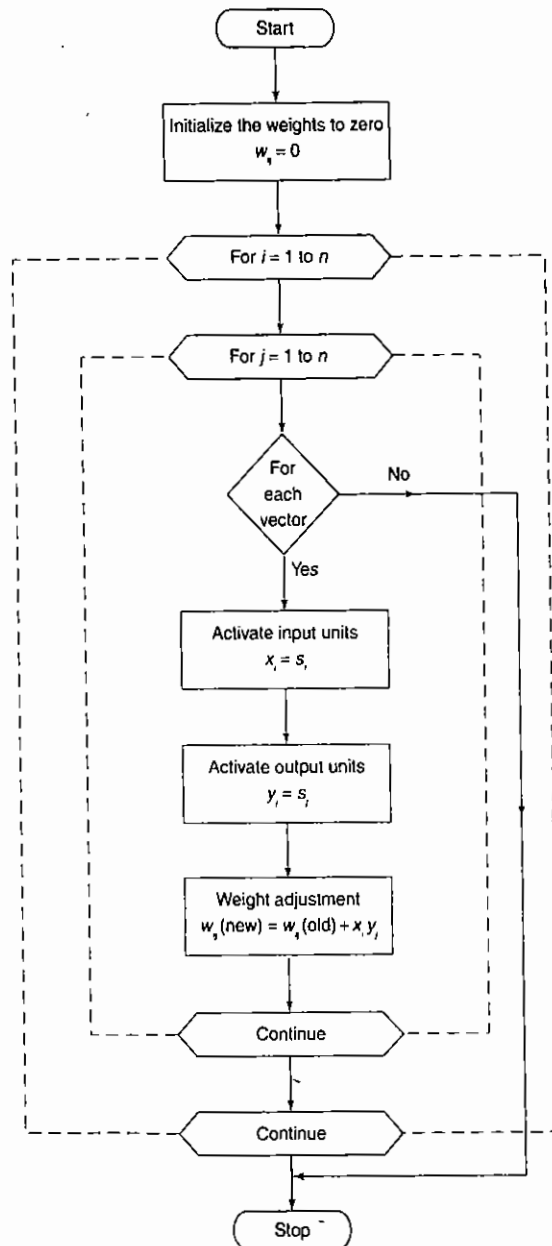


Figure 4-4 Flowchart for training of autoassociative net.

4.3.4 Training Algorithm

The training algorithm discussed here is similar to that discussed in Section 4.2.1 but there are same numbers of output units as that of the input units.

Step 0: Initialize all the weights to zero,

$$w_{ij} = 0 \quad (i = 1 \text{ to } n, j = 1 \text{ to } n)$$

Step 1: For each of the vector that has to be stored perform Steps 2–4.

Step 2: Activate each of the input unit,

$$x_i = s_i \quad (i = 1 \text{ to } n)$$

Step 3: Activate each of the output unit,

$$y_j = s_j \quad (j = 1 \text{ to } n)$$

Step 4: Adjust the weights,

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + x_i y_j$$

The weights can also be determined by the formula

$$W = \sum_{p=1}^P s^T(p) s(p)$$

→ the matched pattern is associated with the output.

4.3.5 Testing Algorithm

An autoassociative memory neural network can be used to determine whether the given input vector is a "known" vector or an "unknown" vector. The net is said to recognize a "known" vector if the net produces a pattern of activation on the output units which is same as one of the vectors stored in it. The testing procedure of an autoassociative neural net is as follows:

Step 0: Set the weights obtained for Hebb's rule or outer products.

Step 1: For each of the testing input vector presented perform Steps 2–4.

Step 2: Set the activations of the input units equal to that of input vector.

Step 3: Calculate the net input to each output unit $j = 1$ to n :

$$y_{inj} = \sum_{i=1}^n x_i w_{ij}$$

Step 4: Calculate the output by applying the activation over the net input:

$$y_j = f(y_{inj}) = \begin{cases} +1 & \text{if } y_{inj} > 0 \\ -1 & \text{if } y_{inj} \leq 0 \end{cases}$$

This type of network can be used in speech processing, image processing, pattern classification, etc.

4.4 Heteroassociative Memory Network

4.4.1 Theory

In case of a heteroassociative neural net, the training input and the target output vectors are different. The weights are determined in a way that the net can store a set of pattern associations. The association here is a pair of training input target output vector pairs $(s(p), t(p))$, with $p = 1, \dots, P$. Each vector $s(p)$ has n components and each vector $t(p)$ has m components. The determination of weights is done either by using Hebb rule or delta rule. The net finds an appropriate output vector, which corresponds to an input vector x , that may be either one of the stored patterns or a new pattern.

4.4.2 Architecture

The architecture of a heteroassociative net is shown in Figure 4-5. From the figure, it can be noticed that for a heteroassociative net, the training input and target output vectors are different. The input layer consists of n number of input units and the output layer consists of m number of output units. There exist weighted interconnections between the input and output layers. The input and output layer units are not correlated with each other. The flowchart of the training process and the training algorithm are the same as discussed in Section 4.2.1.

4.4.3 Testing Algorithm

The testing algorithm used for testing the heteroassociative net with either noisy input or with known input is as follows:

Step 0: Initialize the weights from the training algorithm.

Step 1: Perform Steps 2–4 for each input vector presented.

Step 2: Set the activation for input layer units equal to that of the current input vector given, x_i .

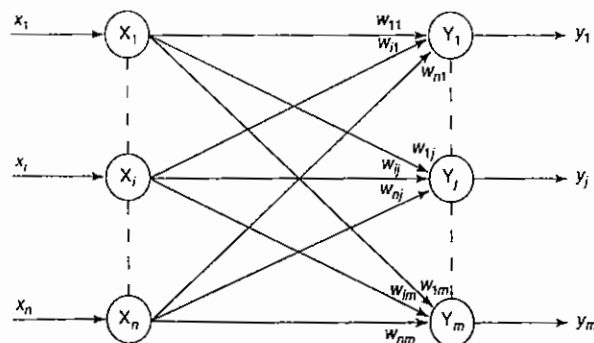


Figure 4-5 Architecture of heteroassociative net.

Step 3: Calculate the net input to the output units:

$$y_{inj} = \sum_{i=1}^n x_i w_{ij} \quad (j = 1 \text{ to } m)$$

Step 4: Determine the activations of the output units over the calculated net input:

$$y_j = \begin{cases} 1 & \text{if } y_{inj} > 0 \\ 0 & \text{if } y_{inj} = 0 \\ -1 & \text{if } y_{inj} < 0 \end{cases}$$

Thus, the output vector y obtained gives the pattern associated with the input vector x .

Note: Heteroassociative memory is not an iterative memory network. If the responses of the net are binary, then the activation function to be used is

$$y_j = \begin{cases} 1 & \text{if } y_{inj} \geq 0 \\ 0 & \text{if } y_{inj} < 0 \end{cases}$$

4.5 Bidirectional Associative Memory (BAM)

4.5.1 Theory

The BAM was developed by Kosko in the year 1988. The BAM network performs forward and backward associative searches for stored stimulus responses. The BAM is a recurrent heteroassociative pattern-matching network that encodes binary or bipolar patterns using Hebbian learning rule. It associates patterns, say from set A to patterns from set B and vice versa is also performed. BAM neural nets can respond to input from either layers (input layer and output layer). There exist two types of BAM, called *discrete* and *continuous* BAM. These two types of BAM are discussed in the following sections.

4.5.2 Architecture

The architecture of BAM network is shown in Figure 4-6. It consists of two layers of neurons which are connected by directed weighted path interconnections. The network dynamics involve two layers of interaction. The BAM network iterates by sending the signals back and forth between the two layers until all the neurons reach equilibrium. The weights associated with the network are bidirectional. Thus, BAM can respond to the inputs in either layer. Figure 4-6 shows a single layer BAM network consisting of n units in X layer and m units in Y layer. The layers can be connected in both directions (bidirectional) with the result the weight matrix sent from the X layer to the Y layer is W and the weight matrix for signals sent from the Y layer to the X layer is W^T . Thus, the weight matrix is calculated in both directions.

4.5.3 Discrete Bidirectional Associative Memory

The structure of discrete BAM is same as shown in Figure 4-6. When the memory neurons are being activated by putting an initial vector at the input of a layer, the network evolves a two-pattern stable state with each pattern at the output of one layer. Thus, the network involves two layers of interaction between each other

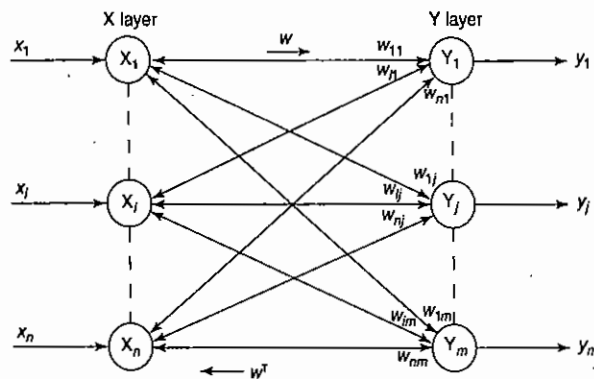


Figure 4-6 Bidirectional associative memory net.

The two bivalent forms of BAM are found to be related with each other, i.e., binary and bipolar. The weights in both the cases are found as the sum of the outer products of the bipolar form of the given training vector pairs. In case of BAM, a definite nonzero threshold is assigned. Thus, the activation function is a step function, with the defined nonzero threshold. When compared, to the binary vectors, bipolar vectors improve the performance of the net to a large extent.

4.5.3.1 Determination of Weights

Let the input vectors be denoted by $s(p)$ and target vectors by $t(p)$, $p = 1, \dots, P$. Then the weight matrix to store a set of input and target vectors, where

$$s(p) = (s_1(p), \dots, s_i(p), \dots, s_n(p))$$

$$t(p) = (t_1(p), \dots, t_j(p), \dots, t_m(p))$$

can be determined by Hebb rule training algorithm discussed in Section 4.2.1. In case of input vectors being binary, the weight matrix $W = \{w_{ij}\}$ is given by

$$w_{ij} = \sum_{p=1}^P [2s_i(p) - 1][2t_j(p) - 1]$$

On the other hand, when the input vectors are bipolar, the weight matrix $W = \{w_{ij}\}$ can be defined as

$$w_{ij} = \sum_{p=1}^P s_i(p)t_j(p)$$

The weights matrix in both the cases is going to be in bipolar form neither the input vectors are in binary or not. The formulas mentioned above can be directly applied to the determination of weights of a BAM.

4.5.3.2 Activation Functions for BAM

The step activation function with a nonzero threshold is used as the activation function for discrete BAM network. The activation function is based on whether the input target vector pairs used are binary or bipolar. The activation function for the Y layer

1. with binary input vectors is

$$y_j = \begin{cases} 1 & \text{if } y_{inj} > 0 \\ y_j & \text{if } y_{inj} = 0 \\ 0 & \text{if } y_{inj} < 0 \end{cases}$$

2. with bipolar input vectors is

$$y_j = \begin{cases} 1 & \text{if } y_{inj} > \theta_j \\ y_j & \text{if } y_{inj} = \theta_j \\ -1 & \text{if } y_{inj} < \theta_j \end{cases}$$

The activation function for the X layer

1. with binary input vectors is

$$x_i = \begin{cases} 1 & \text{if } x_{ini} > 0 \\ x_i & \text{if } x_{ini} = 0 \\ 0 & \text{if } x_{ini} < 0 \end{cases}$$

2. with bipolar input vectors is

$$x_i = \begin{cases} 1 & \text{if } x_{ini} > \theta_i \\ x_i & \text{if } x_{ini} = \theta_i \\ -1 & \text{if } x_{ini} < \theta_i \end{cases}$$

It may be noted that if the threshold value is equal to that of the net input calculated, then the previous output value calculated is left as the activation of that unit. At a particular time instant, signals are sent only from one layer to the other and not in both the directions.

4.5.3.3 Testing Algorithm for Discrete BAM

The testing algorithm is used to test the noisy patterns entering into the network. Based on the training algorithm, weights are determined, by means of which net input is calculated for the given test pattern and activations is applied over it, to recognize the test patterns. The testing algorithm for the net is as follows:

Step 0: Initialize the weights to store p vectors. Also initialize all the activations to zero.

Step 1: Perform Steps 2–6 for each testing input.

Step 2: Set the activations of X layer to current input pattern, i.e., presenting the input pattern x to X layer and similarly presenting the input pattern y to Y layer. Even though, it is bidirectional memory, at one time step, signals can be sent from only one layer. So, either of the input patterns may be the zero vector.

Step 3: Perform Steps 4–6 when the activations are not converged.

Step 4: Update the activations of units in Y layer. Calculate the net input,

$$y_{inj} = \sum_{i=1}^n x_i w_{ij}$$

Applying the activations (as in Section 4.5.3.2), we obtain

$$y_j = f(y_{inj})$$

Send this signal to the X layer.

Step 5: Update the activations of units in X layer. Calculate the net input,

$$x_{ini} = \sum_{j=1}^m y_j w_{ij}$$

Apply the activations over the net input,

$$x_i = f(x_{ini})$$

Send this signal to the Y layer.

Step 6: Test for convergence of the net. The convergence occurs if the activation vectors x and y reach equilibrium. If this occurs then stop, otherwise, continue.

4.5.4 Continuous BAM

A continuous BAM transforms the input smoothly and continuously in the range 0–1 using logistic sigmoid functions as the activation functions for all units. The logistic sigmoidal function may be either binary sigmoidal function or bipolar sigmoidal function. When a bipolar sigmoidal function with a high gain is chosen, then the continuous BAM might converge to a state of vectors which will approach vertices of the cube. When that state of the vector approaches it acts like a discrete BAM.

If the input vectors are binary, $(s(p), t(p))$, $p = 1$ to P , the weights are determined using the formula

$$w_{ij} = \sum_{p=1}^P [2s_i(p) - 1][2t_j(p) - 1]$$

i.e., even though the input vectors are binary, the weight matrix is bipolar. The activation function used here is the logistic sigmoidal function. If it is binary logistic function, then the activation function is

$$f(y_{inj}) = \frac{1}{1 + e^{-y_{inj}}}$$

If the activation function used is a bipolar logistic function, then the function is defined as

$$f(y_{inj}) = \frac{2}{1 + e^{-y_{inj}}} - 1 = \frac{1 - e^{-y_{inj}}}{1 + e^{-y_{inj}}}$$

These activation functions are applied over the net input to calculate the output. The net input can be calculated with a bias included, i.e.,

$$y_{inj} = b_j + \sum_i x_i w_{ij}$$

and all these formulas apply for the units in X layer also.

4.5.5 Analysis of Hamming Distance, Energy Function and Storage Capacity

The hamming distance is defined as the number of mismatched components of two given bipolar or binary vectors. It can also be defined as the number of different bits in two binary or bipolar vectors X and X' . It is denoted as $H[X, X']$. The average hamming distance between the vectors is $(1/n)H[X, X']$, where " n " is the number of components in each vector. Consider the vectors,

$$X = [1 \ 0 \ 1 \ 0 \ 1 \ 1 \ 0] \text{ and } X' = [1 \ 1 \ 1 \ 1 \ 0 \ 0 \ 1]$$

The hamming distance between these two given vectors is equal to 5. The average hamming distance between the corresponding vectors is $5/7$.

The stability analysis of a BAM is based on the definition of Lyapunov function (energy function). Consider that there are p vector association pairs to be stored in a BAM:

$$\{(x^1, y^1), (x^2, y^2), \dots, (x^p, y^p)\}$$

where $x^k = (x_1^k, x_2^k, \dots, x_m^k)^T$ and $y^k = (y_1^k, y_2^k, \dots, y_n^k)^T$ are either binary or bipolar vectors. A Lyapunov function must be always bounded and decreasing. A BAM can be said to be bidirectionally stable if the state converges to a stable point, i.e., $y^k \rightarrow y^{k+1} \rightarrow y^{k+2}$ and $y^{k+2} = y^k$. This gives the minimum of the energy function. The energy function or Lyapunov function of a BAM is defined as

$$E_f(x, y) = -\frac{1}{2} x^T W^T y - \frac{1}{2} y^T W x = -y^T W x$$

The change in energy due to the single bit changes in both vectors y and x given as Δy_i and Δx_j can be found as

$$\Delta E_f(y_i) = \nabla_y E \Delta y_i = -W x \Delta y_i = -\left(\sum_{j=1}^m x_j w_{ij}\right) \times \Delta y_i, \quad i = 1 \text{ to } n$$

$$\Delta E_f(x_j) = \nabla_x E \Delta x_j = -W^T y \Delta x_j = -\left(\sum_{i=1}^n y_i w_{ij}\right) \times \Delta x_j, \quad j = 1 \text{ to } m$$

where Δy_i and Δx_j are given as

$$\Delta x_j = \begin{cases} 2 & \text{if } \sum_{i=1}^n y_i w_{ij} > 0 \\ 0 & \text{if } \sum_{i=1}^n y_i w_{ij} = 0 \\ -2 & \text{if } \sum_{i=1}^n y_i w_{ij} < 0 \end{cases} \quad \text{and} \quad \Delta y_i = \begin{cases} 2 & \text{if } \sum_{j=1}^m x_j w_{ij} > 0 \\ 0 & \text{if } \sum_{j=1}^m x_j w_{ij} = 0 \\ -2 & \text{if } \sum_{j=1}^m x_j w_{ij} < 0 \end{cases}$$

Here the energy function is bounded below by

$$E_f(x, y) \geq - \sum_{i=1}^n \sum_{j=1}^m |w_{ij}|$$

so the discrete BAM will converge to a stable state.

The memory capacity or the storage capacity of BAM may be given as

$$\min(m, n)$$

where “ n ” is the number of units in X layer and “ m ” is the number of units in Y layer. Also a more conservative capacity is estimated as follows:

$$\sqrt{\min(m, n)}$$

4.6 Hopfield Networks

John J. Hopfield developed a model in the year 1982 conforming to the asynchronous nature of biological neurons. The networks proposed by Hopfield are known as Hopfield networks and it is his work that promoted construction of the first analog VLSI neural chip. This network has found many useful applications in associative memory and various optimization problems. In this section, two types of network are discussed: *discrete* and *continuous Hopfield networks*.

4.6.1 Discrete Hopfield Network

The Hopfield network is an autoassociative fully interconnected single-layer feedback network. It is also a symmetrically weighted network. When this is operated in discrete line fashion it is called as *discrete Hopfield network* and its architecture as a single-layer feedback network can be called as *recurrent*. The network takes two-valued inputs: binary (0, 1) or bipolar (+1, -1); the use of bipolar inputs makes the analysis easier. The network has symmetrical weights with no self-connections, i.e.,

$$w_{ij} = w_{ji}; \quad w_{ii} = 0$$

The key points to be noted in Hopfield net are: only one unit updates its activation at a time; also each unit is found to continuously receive an external signal along with the signals it receives from the other units in the net. When a single-layer recurrent network is performing a sequential updating process, an input pattern is first applied to the network and the network's output is found to be initialized accordingly. Afterwards, the initializing pattern is removed, and the output that is initialized becomes the new updated input through the feedback connections. The first updated input forces the first updated output, which in turn acts as the second updated input through the feedback interconnections and results in second updated output. This transition process continues until no new, updated responses are produced and the network reaches its equilibrium.

The asynchronous updation of the units allows a function, called as energy functions or Lyapunov function, for the net. The existence of this function enables us to prove that the net will converge to a stable set of activations. The usefulness of content addressable memory is realized by the discrete Hopfield net.

4.6 Hopfield Networks

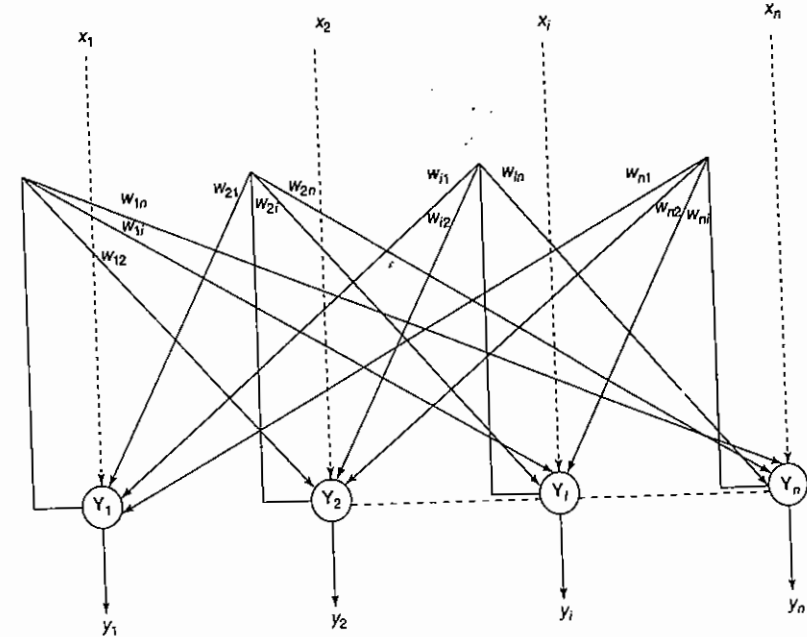


Figure 4-7 Architecture of discrete Hopfield net.

4.6.1.1 Architecture of Discrete Hopfield Net

The architecture of discrete Hopfield net is shown in Figure 4-7. The Hopfield's model consists of processing elements with two outputs, one inverting and the other non-inverting. The outputs from each processing element are fed back to the input of other processing elements but not to itself. The connections are found to be resistive and the connection strength over it is represented as w_{ij} . Here, as such there are no negative resistors, hence excitatory connections use positive outputs and inhibitory connections use inverted outputs. Connections are excitatory if the output of a processing element is found to be same as the input, and they are inhibitory if the inputs differ from the output of the processing element. A connection between the processing elements i and j is found to be associated with a connection strength w_{ij} . This weight is positive if units i and j are both on. On the other hand, if the connection strength is negative, it represents the situation of unit i being on and j being off. Also, the weights are symmetric, i.e., the weights w_{ij} are same as w_{ji} .

4.6.1.2 Training Algorithm of Discrete Hopfield Net

There exist several versions of the discrete Hopfield net. It should be noted that Hopfield's first description used binary input vectors and only later on bipolar input vectors used.

For storing a set of binary patterns $s(p)$, $p = 1$ to P , where $s(p) = (s_1(p), \dots, s_i(p), \dots, s_n(p))$, the weight matrix W is given as

$$w_{ij} = \sum_{p=1}^P [2s_i(p) - 1][2s_j(p) - 1], \quad \text{for } i \neq j$$

For storing a set of bipolar input patterns, $s(p)$ (as defined above), the weight matrix W is given as

$$w_{ij} = \sum_{p=1}^P s_i(p)s_j(p), \quad \text{for } i \neq j$$

and the weights here have no self-connection, i.e., $w_{ij} = 0$.

4.6.1.3 Testing Algorithm of Discrete Hopfield Net

In the case of testing, the update rule is formed and the initial weights are those obtained from the training algorithm. The testing algorithm for the discrete Hopfield net is as follows:

Step 0: Initialize the weights to store patterns, i.e., weights obtained from training algorithm using Hebb rule.

Step 1: When the activations of the net are not converged, then perform Steps 2–8.

Step 2: Perform Steps 3–7 for each input vector X .

Step 3: Make the initial activations of the net equal to the external input vector X :

$$y_i = x_i \quad (i = 1 \text{ to } n)$$

Step 4: Perform Steps 5–7 for each unit Y_i . (Here, the units are updated in random order.)

Step 5: Calculate the net input of the network:

$$y_{ini} = x_i + \sum_j y_j w_{ji}$$

Step 6: Apply the activations over the net input to calculate the output:

$$y_i = \begin{cases} 1 & \text{if } y_{ini} > \theta_i \\ y_i & \text{if } y_{ini} = \theta_i \\ 0 & \text{if } y_{ini} < \theta_i \end{cases}$$

where θ_i is the threshold and is normally taken as zero.

Step 7: Now feed back (transmit) the obtained output y_j to all other units. Thus, the activation vectors are updated.

Step 8: Finally, test the network for convergence.

The updation here is carried out at random, but it should be noted that each unit may be updated at the same average rate. The asynchronous fashion of updation is carried out here. This means that for a given time only a single neural unit is allowed to update its output. The next update can be carried out on a randomly chosen node which uses the already updated output. It can also be said that under asynchronous operation of the network, each output node unit is updated separately by taking into account the most recent values that have already been updated. This type of updation is referred to as an *asynchronous stochastic recursion* of the discrete Hopfield network. By performing the analysis of the Lyapunov function, i.e., the energy function for the Hopfield net, it can be shown that the main feature for the convergence of this net is the asynchronous updation of weights and the weights with no self-connection, i.e., the zeros exist on the diagonals of the weight matrix.

A Hopfield network with binary input vectors is used to determine whether an input vector is a "known" vector or an "unknown" vector. The net has the capacity to recognize a known vector by producing a pattern of activations on the units of the net that is same as the vector stored in the net. For example, if the input vector is an unknown vector, the activation vectors resulted during iteration will converge to an activation vector which is not one of the stored patterns; such a pattern is called as *spurious stable state*.

4.6.1.4 Analysis of Energy Function and Storage Capacity on Discrete Hopfield Net

An energy function generally is defined as a function that is bounded and is a nonincreasing function of the state of the system. The energy function, also called as Lyapunov function, determines the stability property of a discrete Hopfield network. The state of a system for a neural network is the vector of activations of the units. Hence, if it is possible to find an energy function for an iterative neural net, the net will converge to a stable set of activations. An energy function E_f of a discrete Hopfield network is characterized as

$$E_f = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i y_j w_{ij} - \sum_{i=1}^n x_i y_i + \sum_{i=1}^n \theta_i y_i$$

If the network is stable, then the above energy function decreases whenever the state of any node changes. Assuming that node i has changed its state from $y_i^{(k)}$ to $y_i^{(k+1)}$, i.e., the output has changed from +1 to -1 or from -1 to +1, the energy change ΔE_f is then given by

$$\begin{aligned} \Delta E_f &= E_f(y_j^{(k+1)}) - E_f(y_i^{(k)}) \\ &= - \left(\sum_{j=1}^n y_j^{(k)} w_{ij} + x_i - \theta_i \right) (y_i^{(k+1)} - y_i^{(k)}) \\ &= -(\text{net}_i) \Delta y_i \end{aligned}$$

where $\Delta y_i = y_i^{(k+1)} - y_i^{(k)}$. The change in energy is dependent on the fact that only one unit can update its activation at a time. The change in energy equation ΔE_f exploits the fact that $y_j^{(k+1)} = y_j^{(k)}$ for $j \neq i$ and $w_{ij} = w_{ji}$ and $w_{ii} = 0$ (symmetric weight property).

There exist two cases in which a change Δy_i will occur in the activation of neuron Y_i . If y_i is positive, then it will change to zero if

$$\left[x_i + \sum_{j=1}^n y_j w_{ji} \right] < \theta_i$$

This results in a negative change for y_i and $\Delta E_f < 0$. On the other hand, if y_i is zero, then it will change to positive if

$$\left[x_i + \sum_{j=1}^n y_j w_{ji} \right] > \theta_i$$

This results in a positive change for y_i and $\Delta E_f < 0$. Hence Δy_i is positive only if net input is positive and Δy_i is negative only if net input is negative. Therefore, the energy cannot increase in any manner. As a result,

because the energy is bounded, the net must reach a stable state equilibrium, such that the energy does not change with further iteration. From this it can be concluded that the energy change depends mainly on the change in activation of one unit and on the symmetry of weight matrix with zeros existing on the diagonal.

A Hopfield network always converges to a stable state in a finite number of node-updating steps, where every stable state is found to be at the local minima of the energy function E_f . Also, the proving process uses the well-known Lyapunov stability theorem, which is generally used to prove the stability of dynamic system defined with arbitrarily many interlocked differential equations. A positive-definite (energy) function $E_f(y)$ can be found such that:

1. $E_f(y)$ is continuous with respect to all the components y_i for $i = 1$ to n ;
2. $dE_f(y(t))/dt < 0$, which indicates that the energy function is decreasing with time and hence the origin of the state space is asymptotically stable.

Hence, a positive-definite (energy) function $E_f(y)$ satisfying the above requirements can be Lyapunov function for any given system; this function is not unique. If, at least one such function can be found for a system, then the system is asymptotically stable. According to the Lyapunov theorem, the energy function that is associated with a Hopfield network is a Lyapunov function and thus the discrete Hopfield network is asymptotically stable.

The storage capacity is another important factor. It can be found that the number of binary patterns that can be stored and recalled in a network with a reasonable accuracy is given approximately as

$$\text{Storage capacity } C \simeq 0.15n$$

where n is the number of neurons in the net. It can also be given as

$$C \cong \frac{n}{2 \log_2 n}$$

4.6.2 Continuous Hopfield Network

A discrete Hopfield net can be modified to a continuous model, in which time is assumed to be a continuous variable, and can be used for associative memory problems or optimization problems like traveling salesman problem. The nodes of this network have a continuous, graded output rather than a two-state binary output. Thus, the energy of the network decreases continuously with time. The continuous Hopfield networks can be realized as an electronic circuit, which uses non-linear amplifiers and resistors. This helps building the Hopfield network using analog VLSI technology.

4.6.2.1 Hardware Model of Continuous Hopfield Network

The continuous network build up of electrical components is shown in Figure 4-8.

The model consists of n amplifiers, mapping its input voltage u_i into an output voltage y_i over an activation function $a(u_i)$. The activation function used can be a sigmoid function, say,

$$a(\lambda u_i) = \frac{1}{1 + e^{-\lambda u_i}}$$

where λ is called the gain parameter.

The continuous model becomes a discrete one when $\lambda \rightarrow \alpha$. Each of the amplifiers consists of an input capacitance c_i and an input conductance g_i . The external signals entering into the circuit are x_i . The external

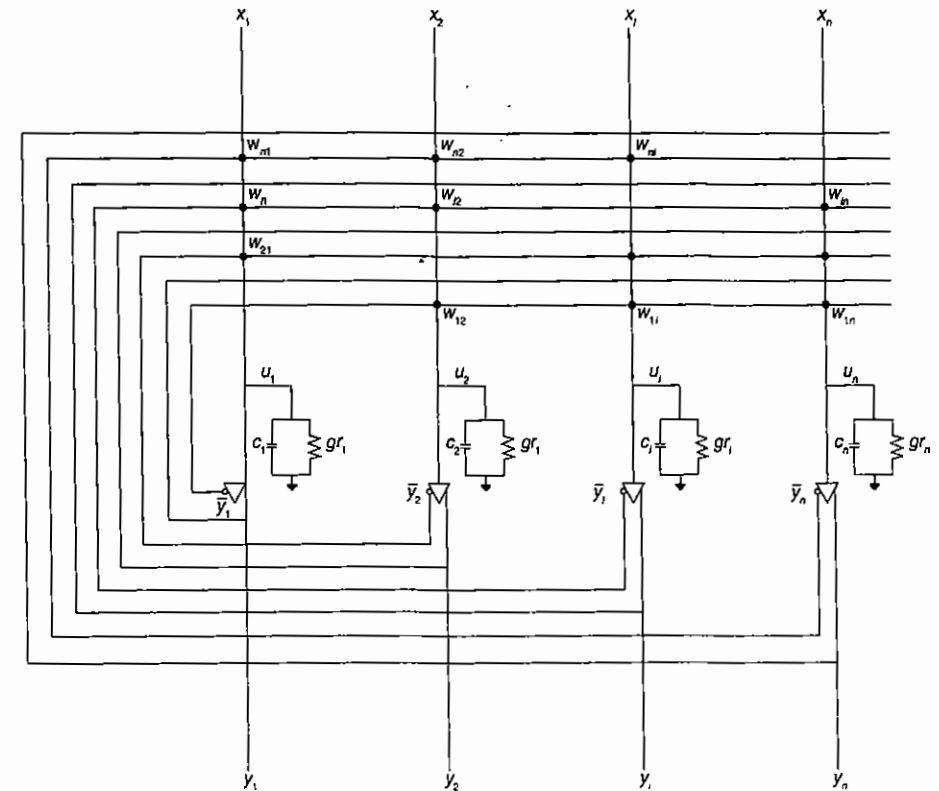


Figure 4-8 Model of Hopfield network using electrical components.

signals supply constant current to each amplifier for an actual circuit. The output of the j th node is connected to the input of the i th node through conductance w_{ij} . Since all real resistor values are positive, the inverted node outputs $\bar{y}_i$ are used to simulate the inhibitory signals. The connection is made with the signal from the noninverted output if the output of a particular node excites some other node. If the connection is inhibitory, then the connection is made with the signal from the inverted output. Here also, the important symmetric weight requirement for Hopfield network is imposed, i.e., $w_{ij} = w_{ji}$ and $w_{ii} = 0$.

The rule of each node in a continuous Hopfield network can be derived as shown in Figure 4-9. Consider the input of a single node as in Figure 4-9. Applying Kirchhoff's current law (KCL), which states that the total current entering a junction is equal to that leaving the same function, we get

$$C_i \frac{du_i}{dt} = \sum_{j=1, j \neq i}^n w_{ij} (y_j - u_i) - g_i u_i + x_i = \sum_{j=1, j \neq i}^n w_{ij} y_j - G_i u_i + x_i$$

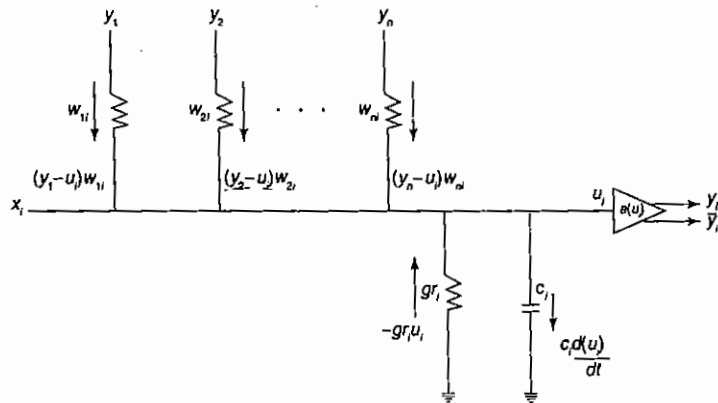


Figure 4-9 Input of a single node of continuous Hopfield network.

where

$$G_i = \sum_{\substack{j=1 \\ j \neq i}}^n w_{ij} + g_i$$

The equation obtained using KCL describes the time evolution of the system completely. If each single node is given an initial value say, $u_i(0)$, then the value $u_i(t)$ and thus the amplifier output, $y_i(t) = a(u_i(t))$ at time t , can be known by solving the differential equation obtained using KCL.

4.6.2.2 Analysis of Energy Function of Continuous Hopfield Network

For evaluating the stability property of continuous Hopfield network, a continuous energy function is defined such that the evolution of the system is in the negative gradient of the energy function and finally converges to one of the stable minima in the state space. The corresponding Lyapunov energy function for the model shown in Figure 4-8 is

$$E_f = -\frac{1}{2} \sum_{i=1}^n \sum_{\substack{j=1 \\ j \neq i}}^n w_{ij} y_i y_j - \sum_{i=1}^n x_i y_i + \frac{1}{\lambda} \sum_{i=1}^n G_i \int_0^{y_i} a^{-1}(y) dy$$

where $a^{-1}(y) = \lambda u$ is the inverse of the function $y = a(\lambda u)$. The inverse of the function $a^{-1}(y)$ is shown in Figure 4-10(A) and the integral of it in Figure 4-10(B).

To prove that E_f obtained is the Lyapunov function for the network, its time derivative is taken with weights w_{ij} symmetric:

$$\frac{dE_f}{dt} = \sum_{i=1}^n \frac{dE}{dy_i} \frac{dy_i}{dt} = \sum_{i=1}^n \left(-\sum_{\substack{j=1 \\ j \neq i}}^n y_j w_{ij} + G_i u_i - x_i \right) \frac{dy_i}{dt} = -\sum_{i=1}^n c_i \frac{dy_i}{dt} \frac{du_i}{dt}$$

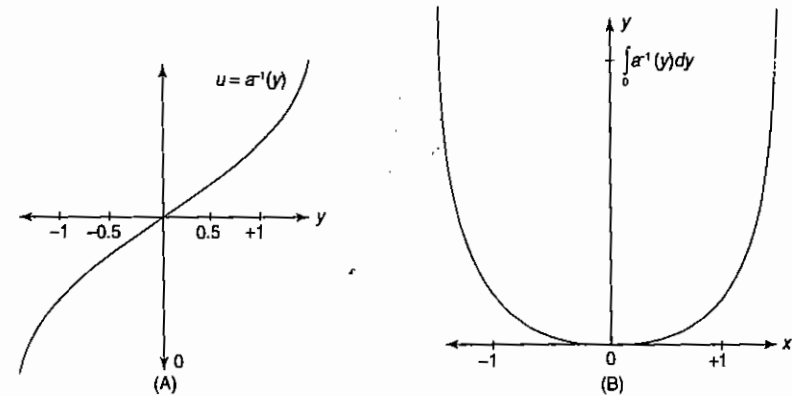


Figure 4-10 (A) Inverse and (B) integral of nonlinear activation function $a^{-1}(y)$.

As

$$u_i = \left(\frac{1}{\lambda} \right) a^{-1}(y_i)$$

we get

$$\frac{du_i}{dt} = \frac{1}{\lambda} \frac{da^{-1}(y_i)}{dy_i} \frac{dy_i}{dt} = \frac{1}{\lambda} a^{-1'}(y_i) \frac{dy_i}{dt}$$

where the derivative of $a^{-1}(y)$ is $a^{-1'}(y)$. So, the derivative of energy function equation becomes

$$\frac{dE_f}{dt} = -\sum_{i=1}^n \frac{1}{\lambda} c_i a^{-1'}(y_i) \left(\frac{dy_i}{dt} \right)^2$$

From Figure 4-10(A), we know that $a^{-1}(y_i)$ is a monotonically increasing function of y_i and hence its derivative is positive, all over. This shows that dE_f/dt is negative, and thus the energy function E_f must decrease as the system evolves. Therefore, if E_f is bounded, the system will eventually reach a stable state, where

$$\frac{dE_f}{dt} = \frac{dy_i}{dt} = 0$$

When the values of threshold are zero, the continuous energy function becomes equal to the discrete energy function, except for the term,

$$\frac{1}{\lambda} \sum_{i=1}^n G_i \int_0^{y_i} a^{-1}(y) dy$$

From Figure 4-10(B), the integral of $a^{-1}(y)$ is zero when y_i is zero and positive for all other values of y_i . The integral becomes very large as y approaches $+1$ or -1 . Hence, the energy function E_f is bounded from below and is a Lyapunov function. The continuous Hopfield nets are best suited for the constrained optimization problems.

4.7 Iterative Autoassociative Memory Networks

There exists a situation where the net does not respond to the input signal immediately with a stored target pattern but the response may be more like the stored pattern, which suggests using the first response as input to the net again. The iterative autoassociative net should be able to recover an original stored vector when presented with a test vector close to it. These types of networks can also be called as *recurrent autoassociative networks* and Hopfield networks discussed in Section 4.6 come under this category.

4.7.1 Linear Autoassociative Memory (LAM)

In 1977, James Anderson focused on the development of the LAM. This was based on Hebbian rule, which states that connections between neuron like elements are strengthened every time when they are activated. Linear algebra is used to analyze the performance of the net.

Consider an $m \times m$ non singular symmetric matrix having " m " mutually orthogonal eigen vectors. The eigen vectors satisfy the property of orthogonality. A recurrent linear autoassociator network is trained using a set of P orthogonal unit vector $u_1, \dots, u_P$, where the number of times each vector going to be presented is not the same.

The weight matrix can be determined using Hebb learning rule, but this allows the repetition of some of the stored vectors. Each of these stored vectors is an eigen vector of the weight matrix. Here, eigen values represent the number of times the vector was presented.

When the input vector X is presented, the output response of the net is XW , where W is the weight matrix. From the concepts of linear algebra, we know that we obtain the largest value of $\|XW\|$ when X is the eigen vector for the largest eigen value; the next largest value of $\|XW\|$ occurs when X is the eigen vector for the next largest eigen value, and so on. Thus, a recurrent linear autoassociator produces its response as the stored vector for which the input vector is most similar. This may perhaps take several iterations. The linear combination of vectors may be used to represent an input pattern. When an input vector is presented, the response of the net is the linear combination of its corresponding eigen values. The eigen vector with largest value in this linear expansion is the one which is most similar to that of the input vectors. Although, the net increases its response corresponding to components of the input pattern over which it is trained most extensively, the overall output response of the system may grow without bound.

The main conditions of linearity between the associative memories is that the set of input vector pairs and output vector pairs (since, autoassociative, both are same) should be mutually orthogonal with each other, i.e., if " A_p " is the input pattern pair, for $p = 1$ to P , then

$$A_i A_j^T = 0, \quad \text{for all } i \neq j$$

Also if all the vectors A_p are normalized to unit length, i.e.,

$$\sum_{i=1}^n (a_i)_p^2 = 1, \quad \text{for all } p = 1 \text{ to } P$$

then the output $Y_j = A_p$, i.e., the desired output has been recalled.

4.7.2 Brain-in-the-Box Network

An extension to the linear associator is the brain-in-the-box model. This model was described by Anderson, 1972, as follows: an activity pattern inside the box receives positive feedback on certain components, which

has the effect of forcing it outward. When its element start to limit (when it hits the wall of the box), it moves to corner of the box where it remains as such. The box resides in the state-space (each neuron occupies one axis) of the network and represents the saturation limits for each state. Each component here is being restricted between -1 and $+1$. The updation of activations of the units in brain-in-the-box model is done simultaneously.

The brain-in-the-box model consists of n units, each being connected to every other unit. Also, there is a trained weight on the self-connection, i.e., the diagonal elements are set to zero. There also exists a self-connection with weight 1. The algorithm for brain-in-the-box model is given in Section 4.7.2.1.

4.7.2.1 Training Algorithm for Brain-in-the-Box Model

Step 0: Initialize the weights to very small random values. Initialize the learning rates α and β .

Step 1: Perform Steps 2–6 for each training input vector.

Step 2: The initial activations of the net are made equal to the external input vector X :

$$y_i = x_i$$

Step 3: Perform Steps 4 and 5 when the activations continue to change.

Step 4: Calculate the net input:

$$y_{inj} = y_i + \alpha \sum_{j=1}^n y_j w_{ji}$$

Step 5: Calculate the output of each unit by applying its activations:

$$y_j = \begin{cases} 1 & \text{if } y_{inj} > 1 \\ y_{inj} & \text{if } -1 \leq y_{inj} \leq 1 \\ -1 & \text{if } y_{inj} < -1 \end{cases}$$

The vertex of the box will be a stable state for the activation vector.

Step 6: Update the weights:

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \beta y_i y_j$$

4.7.3 Autoassociator with Threshold Unit

If a threshold unit is set, then a threshold function can be used as the activation function for an iterative autoassociator net. The testing algorithm of autoassociator with specified threshold for bipolar vectors and activations with symmetric weights and no self-connections, i.e., $w_{ij} = w_{ji}$ and $w_{ii} = 0$ is given in the following section.

4.7.3.1 Testing Algorithm

Step 0: The weights are initialized from the training algorithm to store patterns (use Hebbian learning).

Step 1: Perform Steps 2-5 for each testing input vector.

Step 2: Set the activations of X .

Step 3: Perform Steps 4 and 5 when the stopping condition is false.

Step 4: Update the activations of all units:

$$x_i = \begin{cases} 1 & \text{if } \sum_{j=1}^n x_j w_{ij} > \theta_i \\ x_i & \text{if } \sum_{j=1}^n x_j w_{ij} = \theta_i \\ -1 & \text{if } \sum_{j=1}^n x_j w_{ij} < \theta_i \end{cases}$$

The threshold θ_i may be taken as zero.

Step 5: Test for the stopping condition.

The network performs iteration until the correct vector X matches a stored vector or the testing input matches a previous vector or the maximum number of iterations allowed is reached.

4.8 Temporal Associative Memory Network

The associative memories discussed so far evolve a stable state and stay there. All are acting as content addressable memories for a set of static patterns. But there is also a possibility of storing the sequences of patterns in the form of dynamic transitions. These types of patterns are called as *temporal patterns* and an associative memory with this capability is called as a *temporal associative memory*. In this section, we shall learn how the BAM act as temporal associative memories. Assume all temporal patterns as bipolar or binary vectors given by an ordered set S with p vectors:

$$S = \{s_1, s_2, \dots, s_i, \dots, s_p\} \quad (p = 1 \text{ to } P)$$

where column vectors are n -dimensional. The neural network can memorize the sequence S in its dynamic state transitions such that the recalled sequence is $s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_i \rightarrow \dots \rightarrow s_p \rightarrow s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_i \rightarrow \dots$ or in reverse order.

A BAM can be used to generate the sequence $S = \{s_1, s_2, \dots, s_i, \dots, s_p\}$. The pair of consecutive vectors s_k and s_{k+1} are taken as heteroassociative. From this point of view, s_1 is associated with s_2 , s_2 is associated with s_3 , ... and s_p is again associated with s_1 . The weight matrix is then given as

$$W = \sum_{k=1}^p (s_{k+1})(s_k)^T$$

A BAM for temporal patterns can be modified so that both layers X and Y are described by identical weight matrices W . Hence, the recalling is based on

$$x = f(Wy); \quad y = f(Wx)$$

where $f(\cdot)$ is the activation function of the network. Also a reverse order recall can be implemented using the transposed weight matrices in both layers X and Y . In case of temporal BAM, layers X and Y update nonsimultaneously and in an alternate circular fashion.

The energy function for a temporal BAM can be defined as

$$E_f = - \sum_{k=1}^p s_{k+1} W s_k$$

The energy function E_f decreases during the temporal sequence retrieval $s_1 \rightarrow s_2 \rightarrow \dots \rightarrow s_p$. The energy is found to increase stepwise at the transition $s_p \rightarrow s_1$ and then it continues to decrease in the following cycle of $(p-1)$ retrievals. The storage capacity of the BAM is estimated using $p \leq \min(m, n)$. Hence, the maximum length sequence is bounded by $p < n$, where n is number of components in input vector and m is number of components in output vector.

4.9 Summary

Pattern association is carried out efficiently by associative memory networks. The two main algorithms used for training a pattern association network are the Hebb rule and the outer products rule. The basic architecture, flowchart for training process and the training algorithm are discussed in detail for autoassociative net, heteroassociative memory net, BAM, Hopfield net and iterative nets. Also, in all cases suitable testing algorithm is included. The variations in BAM, discrete BAM and continuous BAM, are discussed in this chapter. The analysis of hamming distance, energy function and storage capacity is done for few networks such as BAM, discrete Hopfield network and continuous Hopfield network. In case of iterative autoassociative memory network, the linear autoassociative memory, brain-in-the-box model and an autoassociator with a threshold unit are discussed. Also temporal associative memory network is discussed briefly.

4.10 Solved Problems

1. Train a heteroassociative memory network using Hebb rule to store input row vector $s = (s_1, s_2, s_3, s_4)$ to the output row vector $t = (t_1, t_2)$.

The vector pairs are given in Table 1.

Table 1

Input targets	s_1	s_2	s_3	s_4	t_1	t_2
1 st	1	0	1	0	1	0
2 nd	1	0	0	1	1	0
3 rd	1	1	0	0	0	1
4 th	0	0	1	1	0	1

Solution: The network for the given problem is as shown in Figure 1. The training algorithm based on Hebb rule is used to determine the weights.

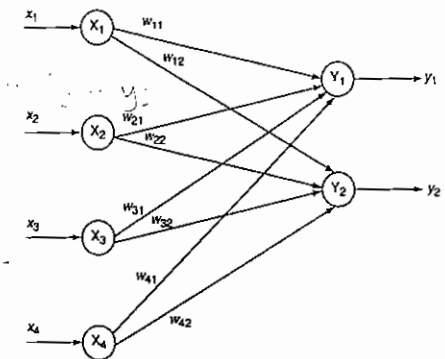


Figure 1 Neural net.

For 1st input vector:

Step 0: Initialize the weights, the initial weights are taken as zero.

Step 1: For first pair (1, 0, 1, 0):(1, 0)

Step 2: Set the activations of input units:

$$x_1 = 1, x_2 = 0, x_3 = 1, x_4 = 0$$

Step 3: Set the activations of output unit:

$$y_1 = 1, y_2 = 0$$

Step 4: Update the weights,

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + x_i y_j$$

$$w_{11}(\text{new}) = w_{11}(\text{old}) + x_1 y_1 = 0 + 1 \times 1 = 1$$

$$w_{21}(\text{new}) = w_{21}(\text{old}) + x_2 y_1 = 0 + 0 \times 1 = 0$$

$$w_{31}(\text{new}) = w_{31}(\text{old}) + x_3 y_1 = 0 + 1 \times 1 = 1$$

$$w_{41}(\text{new}) = w_{41}(\text{old}) + x_4 y_1 = 0 + 0 \times 1 = 0$$

$$w_{12}(\text{new}) = w_{12}(\text{old}) + x_1 y_2 = 0 + 1 \times 0 = 0$$

$$w_{22}(\text{new}) = w_{22}(\text{old}) + x_2 y_2 = 0 + 0 \times 0 = 0$$

$$w_{32}(\text{new}) = w_{32}(\text{old}) + x_3 y_2 = 0 + 1 \times 0 = 0$$

$$w_{42}(\text{new}) = w_{42}(\text{old}) + x_4 y_2 = 0 + 0 \times 0 = 0$$

For 2nd input vector:

The input-output vector pair is (1, 0, 0, 1):(1, 0)

$$x_1 = 1, x_2 = 0, x_3 = 0, x_4 = 1,$$

$$y_1 = 1, y_2 = 0$$

The final weights obtained for the input vector pair is used as initial weight here:

$$w_{11}(\text{new}) = w_{11}(\text{old}) + x_1 y_1 = 1 + 1 \times 1 = 2$$

$$w_{41}(\text{new}) = w_{41}(\text{old}) + x_4 y_1 = 0 + 1 \times 1 = 1$$

Since $x_2 = x_3 = y_2 = 0$, the other weights remains the same.

The final weights after second input vector is presented are

$$w_{11} = 2, w_{21} = 0, w_{31} = 1, w_{41} = 1$$

$$w_{12} = 0, w_{22} = 0, w_{32} = 0, w_{42} = 0$$

For 3rd input vector:

The input-output vector pair is (1, 1, 0, 0):(0, 1)

$$x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 0, y_1 = 0, y_2 = 1$$

Training, using Hebb rule, evolves the final weights as follows:

Since $y_1 = 0$, the weights of y_1 are going to the same.

Computing the weights of y_2 unit, we obtain

$$w_{12}(\text{new}) = w_{12}(\text{old}) + x_1 y_2 = 0 + 1 \times 1 = 1$$

$$w_{22}(\text{new}) = w_{22}(\text{old}) + x_2 y_2 = 0 + 1 \times 1 = 1$$

$$w_{32}(\text{new}) = w_{32}(\text{old}) + x_3 y_2 = 0 + 0 \times 1 = 0$$

$$w_{42}(\text{new}) = w_{42}(\text{old}) + x_4 y_2 = 0 + 0 \times 1 = 0$$

The final weights after presenting third input vector are

$$w_{11} = 2, w_{21} = 0, w_{31} = 1, w_{41} = 1$$

$$w_{12} = 1, w_{22} = 1, w_{32} = 0, w_{42} = 0$$

For 4th input vector:

The input-output vector pair is (0, 0, 1, 1):(0, 1)

$$x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1, y_1 = 0, y_2 = 1$$

The weights are given by

$$w_{32}(\text{new}) = w_{32}(\text{old}) + x_3 y_2 = 0 + 1 \times 1 = 1$$

$$w_{42}(\text{new}) = w_{42}(\text{old}) + x_4 y_2 = 0 + 1 \times 1 = 1$$

Since, $x_1 = x_2 = y_1 = 0$, the other weights remains the same. The final weights after presenting the fourth input vector are

$$w_{11} = 2, w_{21} = 0, w_{31} = 1, w_{41} = 1$$

$$w_{12} = 1, w_{22} = 1, w_{32} = 1, w_{42} = 1$$

Thus, the weight matrix in matrix form is

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \\ w_{41} & w_{42} \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 0 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}$$

2. Train the heteroassociative memory network using outer products rule to store input row vectors $s = (s_1, s_2, s_3, s_4)$ to the output row vectors $t = (t_1, t_2)$. Use the vector pairs as given in Table 2.

Table 2

Input and targets	s_1	s_2	s_3	s_4	t_1	t_2
1 st	1	0	1	0	1	0
2 nd	1	0	0	1	1	0
3 rd	1	1	0	0	0	1
4 th	0	0	1	1	0	1

Solution: Use outer products rule to determine the weight matrix:

$$W = \sum_{p=1}^P s^T(p) t(p)$$

For 1st pair: The input and output vectors are $s = (1 \ 0 \ 1 \ 0)$, $t = (1 \ 0)$. For $p = 1$,

$$s^T(p) t(p) = s^T(1) t(1)$$

$$= \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}_{4 \times 1} [1 \ 0]_{1 \times 2} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 0 \end{bmatrix}_{4 \times 2}$$

For 2nd pair: The input and output vectors are $s = (1 \ 0 \ 0 \ 1)$, $t = (1 \ 0)$. For $p = 2$,

$$s^T(p) t(p) = s^T(2) t(2)$$

$$= \begin{bmatrix} 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}_{4 \times 1} [1 \ 0]_{1 \times 2} = \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix}_{4 \times 2}$$

For 3rd pair: The input and output vectors are $s = (1 \ 1 \ 0 \ 0)$, $t = (0 \ 1)$. For $p = 3$,

$$s^T(p) t(p) = s^T(3) t(3)$$

$$= \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}_{4 \times 1} [0 \ 1]_{1 \times 2} = \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}_{4 \times 2}$$

For 4th pair: The input and output vectors are $s = (0 \ 0 \ 1 \ 1)$, $t = (0 \ 1)$. For $p = 4$,

$$s^T(p) t(p) = s^T(4) t(4)$$

$$= \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}_{4 \times 1} [0 \ 1]_{1 \times 2} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}_{4 \times 2}$$

The final weight matrix is the summation of all the individual weight matrices obtained for each pair.

$$W = \sum_{p=1}^4 s^T(p) t(p) = s^T(1) t(1) + s^T(2) t(2) + s^T(3) t(3) + s^T(4) t(4)$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}$$

$$W = \begin{bmatrix} 2 & 1 \\ 0 & 1 \\ 1 & 1 \\ 1 & 1 \end{bmatrix}$$

3. Train a heteroassociative memory network to store the input vectors $s = (s_1, s_2, s_3, s_4)$ to the output vectors $t = (t_1, t_2)$. The vector pairs are given in Table 3. Also test the performance of the network using its training input as testing input.

Table 3

Input and targets	s_1	s_2	s_3	s_4	t_1	t_2
1 st	1	0	0	0	0	1
2 nd	1	1	0	0	0	1
3 rd	0	0	0	1	1	0
4 th	0	0	1	1	1	0

Solution: The network architecture for the given input-target vector pair is shown in Figure 2. Training the network means the determination of weights of the network. Here outer products rule is used to determine the weight.

The weight matrix W using outer products rule is given by

$$W = \sum_{p=1}^P s^T(p) t(p)$$

For $p = 1$ to 4,

$$\begin{aligned}
 W &= \sum_{p=1}^4 s^T(p) t(p) \\
 &= s^T(1)t(1) + s^T(2)t(2) + s^T(3)t(3) + s^T(4)t(4) \\
 &= \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 & 1 \end{bmatrix} \\
 &\quad + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} \\
 &= \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 1 & 0 \end{bmatrix} \\
 W &= \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}
 \end{aligned}$$

This is the final weight of the matrix.

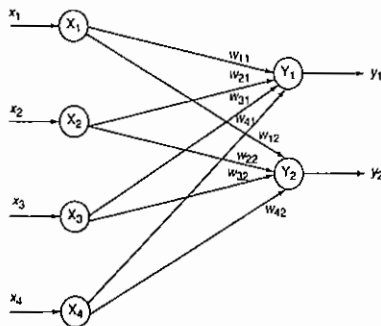


Figure 2 Network architecture.

Testing the Network

Method I

The testing algorithm for a heteroassociative memory network is used to test the performance of the net. The weight obtained from training algorithm is the initial weight in testing algorithm.

For 1st testing input

Step 0: Initialize the weights:

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \\ w_{41} & w_{42} \end{bmatrix} = \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}$$

Step 1: Performs Steps 2-4 for each testing input-output vector.

Step 2: Set the activations, $x = [1 \ 0 \ 0 \ 0]$.

Step 3: Compute the net input, $n = 4$, $m = 2$.

For $i = 1$ to 4 and $j = 1$ to 2:

$$\begin{aligned}
 y_{in1} &= \sum_{i=1}^n x_i w_{ij} \\
 y_{in1} &= \sum_{i=1}^4 x_i w_{i1} \\
 &= x_1 w_{11} + x_2 w_{21} + x_3 w_{31} + x_4 w_{41} \\
 &= 1 \times 0 + 0 \times 0 + 0 \times 1 + 0 \times 2 = 0 \\
 y_{in2} &= \sum_{i=1}^n x_i w_{i2} \\
 &= x_1 w_{12} + x_2 w_{22} + x_3 w_{32} + x_4 w_{42} \\
 &= 1 \times 2 + 0 \times 1 + 0 \times 0 + 0 \times 0 = 2
 \end{aligned}$$

Step 4: Applying activation over the net input to calculate the output.

$$\begin{aligned}
 y_1 &= f(y_{in1}) = f(0) = 0 \\
 y_2 &= f(y_{in2}) = f(2) = 1
 \end{aligned}$$

The output is $[0, 1]$ which is correct response for first input pattern.

For 2nd testing input

Set the activation $x = [1 \ 1 \ 0 \ 0]$. Computing the net input, we obtain

$$\begin{aligned}
 y_{in1} &= x_1 w_{11} + x_2 w_{21} + x_3 w_{31} + x_4 w_{41} \\
 &= 0 + 0 + 0 + 0 = 0 \\
 y_{in2} &= x_1 w_{12} + x_2 w_{22} + x_3 w_{32} + x_4 w_{42} \\
 &= 2 + 1 + 0 + 0 = 3
 \end{aligned}$$

Compute the output by applying activations over net test performance of network. The initial weights for the network are

$$W = \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}$$

The output is $[0, 1]$ which is correct response for second input pattern.

For 3rd testing input

Set the activation $x = [0 \ 0 \ 0 \ 1]$. Computing net input, we obtain

$$\begin{aligned}
 y_{in1} &= x_1 w_{11} + x_2 w_{21} + x_3 w_{31} + x_4 w_{41} \\
 &= 0 + 0 + 0 + 2 = 2 \\
 y_{in2} &= x_1 w_{12} + x_2 w_{22} + x_3 w_{32} + x_4 w_{42} \\
 &= 0 + 0 + 0 + 0 = 0
 \end{aligned}$$

Calculate output of the network,

$$\begin{aligned}
 y_1 &= f(y_{in1}) = f(2) = 1 \\
 y_2 &= f(y_{in2}) = f(0) = 0
 \end{aligned}$$

The output is $[1 \ 0]$ which is correct response for third testing input pattern.

For 4th testing input

Set the activation $x = [0 \ 0 \ 1 \ 1]$. Calculating the net input, we obtain

$$\begin{aligned}
 y_{in1} &= x_1 w_{11} + x_2 w_{21} + x_3 w_{31} + x_4 w_{41} \\
 &= 0 + 0 + 1 + 2 = 3 \\
 y_{in2} &= x_1 w_{12} + x_2 w_{22} + x_3 w_{32} + x_4 w_{42} \\
 &= 0 + 0 + 0 + 0 = 0
 \end{aligned}$$

Calculate the output of the network,

$$\begin{aligned}
 y_1 &= f(y_{in1}) = f(3) = 1 \\
 y_2 &= f(y_{in2}) = f(0) = 0
 \end{aligned}$$

The output is $[1 \ 0]$ which is correct response for fourth testing input pattern.

Method II

Since net input is the dot product of the input row vector with the column of weight matrix, hence a method using matrix multiplication can be used to

The binary activations are used, i.e.,

$$f(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

For 1st testing input

Set the activation $x = [1 \ 0 \ 0 \ 0]$. The net input is given by $y_{in} = xW$ (in vector form):

$$\begin{aligned}
 [y_{in1} \ y_{in2}] &= [1 \ 0 \ 0 \ 0]_{1 \times 4} \times \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}_{4 \times 2} \\
 &= [0 + 0 + 0 + 0 \quad 2 + 0 + 0 + 0] \\
 &= [0 \ 2]
 \end{aligned}$$

Applying activations over the net input, we get

$$[y_1 \ y_2] = [0 \ 1]$$

The correct response is obtained for first testing input pattern.

For 2nd testing input

Set the activation $x = [1 \ 1 \ 0 \ 0]$. The net input is obtained by

$$\begin{aligned}
 [y_{in1} \ y_{in2}] &= [1 \ 1 \ 0 \ 0] \times \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix} \\
 &= [0 + 0 + 0 + 0 \quad 2 + 1 + 0 + 0] \\
 &= [0 \ 3]
 \end{aligned}$$

Apply activations over the net input to obtain output, we get

$$[y_{in1} \ y_{in2}] = [0 \ 1]$$

The correct response is obtained for second testing input.

For 3rd testing input

Set the activation $x = [0 \ 0 \ 0 \ 1]$. The net input is obtained by

$$[y_{in1} \ y_{in2}] = [0 \ 0 \ 0 \ 1] \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}$$

$$= [0 + 0 + 0 + 2 \quad 0 + 0 + 0 + 0]$$

$$= [2 \quad 0]$$

Applying activations to calculate the output, we get

$$[y_1 \ y_2] = [1 \ 0]$$

Thus, correct response is obtained for third testing input.

For 4th testing input

Set the activation $x = [0 \ 0 \ 1 \ 1]$. The net input is calculated as

$$[y_{in1} \ y_{in2}] = [0 \ 0 \ 1 \ 1] \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}$$

$$= [0 + 0 + 1 + 2 \quad 0 + 0 + 0 + 0]$$

$$= [3 \ 0]$$

The output is obtained by applying activations over the net input:

$$[y_1 \ y_2] = [1 \ 0]$$

The correct response is obtained for fourth testing input. Thus, training and testing of a hetero associative network is done here.

4. For Problem 3, test a heteroassociative network with a similar test vector and unsimilar test vector.

Solution: The heteroassociative network has to be tested with similar and unsimilar test vector.

With similar test vector. From Problem 3, the second input vector is $x = [1 \ 1 \ 0 \ 0]$ with target $y = [0 \ 1]$. To test the network with a similar vector, making a change in one component of the input vector, we get

$$x = [0 \ 1 \ 0 \ 0]$$

The weight matrix is

$$W = \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}$$

The net input is calculated for the similar vector,

$$[y_{in1} \ y_{in2}] = [0 \ 1 \ 0 \ 0] \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}$$

$$= [0 + 0 + 0 + 0 \quad 0 + 1 + 0 + 0]$$

$$= [0 \ 1]$$

The output is obtained by applying activations over the net input

$$[y_1 \ y_2] = [0 \ 1]$$

The correct response same as the target is found, hence the vector similar to the input vector is recognized by the network.

With unsimilar-input vector: The second input vector is $x = [1 \ 1 \ 0 \ 0]$ with target $y = [0 \ 1]$. To test the network with unsimilar vectors by making a change in two components of the input vector, we get

$$x = [0 \ 1 \ 1 \ 0]$$

The weight matrix is

$$W = \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}$$

The net input is calculated for unsimilar vector,

$$[y_{in1} \ y_{in2}] = [0 \ 1 \ 1 \ 0] \begin{bmatrix} 0 & 2 \\ 0 & 1 \\ 1 & 0 \\ 2 & 0 \end{bmatrix}$$

$$= [0 + 0 + 1 + 0 \quad 0 + 1 + 0 + 0]$$

$$= [1 \ 1]$$

The output is obtained by applying activations over the net input

$$[y_1 \ y_2] = [1 \ 1]$$

The correct response is not obtained when the vector unsimilar to the input network is presented to the network.

5. Train a heteroassociative network to store the input vectors $s = (s_1 \ s_2 \ s_3 \ s_4)$ to the output vector $t = (t_1 \ t_2)$. The training input-target output vector pairs are in binary form. Obtain the weight vector in bipolar form. The binary vector pairs are as given in Table 4.

Table 4

	s_1	s_2	s_3	s_4	t_1	t_2
1 st	1	0	0	0	0	1
2 nd	1	1	0	0	0	1
3 rd	0	0	0	1	1	0
4 th	0	0	1	1	1	0

Solution: In this case, the hybrid representation of the network is adopted to find the weight matrix in bipolar form. The weight matrix can be formed using

$$w_{11} = (2 \times 1 - 1)(2 \times 0 - 1)$$

$$+ (2 \times 1 - 1)(2 \times 0 - 1)$$

$$+ (2 \times 0 - 1)(2 \times 1 - 1)$$

$$+ (2 \times 0 - 1)(2 \times 1 - 1)$$

$$= -1 - 1 - 1 - 1 = -4$$

$$w_{12} = (2 \times 1 - 1)(2 \times 1 - 1)$$

$$+ (2 \times 1 - 1)(2 \times 1 - 1)$$

$$+ (2 \times 0 - 1)(2 \times 0 - 1)$$

$$+ (2 \times 0 - 1)(2 \times 0 - 1)$$

$$= 1 + 1 + 1 + 1 = 4$$

$$w_{21} = -1 \times -1 + 1 \times -1 + -1 \times 1 + -1 \times 1$$

$$= 1 - 1 - 1 - 1 = -2$$

$$w_{22} = -1 \times 1 + 1 \times 1 + -1 \times -1 + -1 \times -1$$

$$= -1 + 1 + 1 + 1 = 2$$

$$w_{31} = -1 \times -1 + -1 \times -1 + -1 \times 1 + 1 \times 1$$

$$= 1 + 1 - 1 + 1 = 2$$

$$w_{32} = -1 \times 1 + -1 \times 1 + -1 \times -1 + 1 \times -1$$

$$= -1 - 1 + 1 - 1 = -2$$

$$w_{41} = -1 \times -1 + -1 \times -1 + 1 \times 1 + 1 \times 1$$

$$= 1 + 1 + 1 + 1 = 4$$

$$w_{42} = -1 \times 1 + -1 \times 1 + 1 \times -1 + 1 \times -1$$

$$= -1 - 1 - 1 - 1 = -4$$

The weight matrix W is given by

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \\ w_{41} & w_{42} \end{bmatrix} = \begin{bmatrix} -4 & 4 \\ -2 & 2 \\ 2 & -2 \\ 4 & -4 \end{bmatrix}$$

6. Train a heteroassociative network to store the given bipolar input vectors $s = (s_1 \ s_2 \ s_3 \ s_4)$ to the output vector $t = (t_1 \ t_2)$. The bipolar vector pairs are as given in Table 5.

Table 5

	s_1	s_2	s_3	s_4	t_1	t_2
1 st	1	-1	-1	-1	-1	1
2 nd	1	1	-1	-1	-1	1
3 rd	-1	-1	-1	1	1	-1
4 th	-1	-1	1	1	1	-1

Solution: To store a bipolar vector pair, the weight matrix is

$$w_{ij} = \sum_{p=1}^P s_i(p)t_j(p)$$

If the outer products rule is used, then

$$W = \sum_p s^T(p)t(p)$$

For 1st pair

$$s = [1 \ -1 \ -1 \ -1], \quad t = [-1 \ 1]$$

$$s^T(1)t(1) = \begin{bmatrix} 1 \\ -1 \\ -1 \\ -1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix}$$

For 2nd pair

$$s = [1 \ 1 \ -1 \ -1], \quad t = [-1 \ 1]$$

$$s^T(2)t(2) = \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix}$$

For 3rd pair

$$s = [-1 \ -1 \ -1 \ 1], \quad t = [1 \ -1]$$

$$s^T(3)t(3) = \begin{bmatrix} -1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [1 \ -1] = \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$$

For 4th pair

$$s = [-1 \ -1 \ 1 \ 1], \quad t = [1 \ -1]$$

$$s^T(4)t(4) = \begin{bmatrix} -1 \\ -1 \\ 1 \\ 1 \end{bmatrix} [1 \ -1] = \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix}$$

The final weight matrix is

$$\begin{aligned} W &= \sum_{p=1}^4 s^T(p)t(p) = \begin{bmatrix} -1 & 1 \\ 1 & -1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix} + \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix} \\ &+ \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix} + \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix} \\ &= \begin{bmatrix} -4 & 4 \\ -2 & 2 \\ 2 & -2 \\ 4 & -4 \end{bmatrix} \end{aligned}$$

7. For Problem 6, test the performance of the network with missing and mistaken data in the test vector.

Solution: With missing data

Let the test vector be $x = [0 \ 1 \ 0 \ -1]$ with changes made in two components of second input vector $[1 \ 1 \ -1 \ -1]$. Computing the net input, we get

$$\begin{aligned} [y_{in1} \ y_{in2}] &= [0 \ 1 \ 0 \ -1] \begin{bmatrix} -4 & 4 \\ -2 & 2 \\ 2 & -2 \\ 4 & -4 \end{bmatrix} \\ &= [0 - 2 + 0 - 4 \quad 0 + 2 + 0 + 4] \\ &= [-6 \ 6] \end{aligned}$$

Applying activations to compute the output, we get

$$[y_1 \ y_2] = [-1 \ 1]$$

Thus, the net has recognized the missing data.

With mistaken data: Let the test vector be $x = [-1 \ 1 \ 1 \ -1]$ with changes made in two components of second input vector $[1 \ 1 \ -1 \ -1]$. Computing the net input for the test vector, using the final weights obtained in Problem 6, as initial weight to test the test vector, we get

$$\begin{aligned} [y_{in1} \ y_{in2}] &= [-1 \ 1 \ 1 \ -1] \begin{bmatrix} -4 & 4 \\ -2 & 2 \\ 2 & -2 \\ 4 & -4 \end{bmatrix} \\ &= [0 \ 0] \end{aligned}$$

Applying the activations over the net input to calculate the output, we obtain

$$[y_1 \ y_2] = [0 \ 0]$$

Thus, the net does not recognize the mistaken data because the output obtained $[0, 0]$ has a mismatch with the target vector $[-1 \ 1]$.

8. Train the autoassociative network for input vector $[-1 \ 1 \ 1 \ 1]$ and also test the network for the same input vector. Test the autoassociative network with one missing, one mistake, two missing and two mistake entries in test vector.

Solution: The input vector is $x = [-1 \ 1 \ 1 \ 1]$. The weight vector is

$$\begin{aligned} W &= \sum s^T(p)s(p) = \begin{bmatrix} -1 \\ 1 \\ 1 \\ 1 \end{bmatrix} [-1 \ 1 \ 1 \ 1]_{1 \times 4} \\ &= \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix}_{4 \times 4} \end{aligned}$$

Testing the network with same input vector: The test input is $[-1 \ 1 \ 1 \ 1]$. The weight obtained above

is used as the initial weight here. Computing the input, we get

$$\begin{aligned} y_{inj} &= x \cdot W = [-1 \ 1 \ 1 \ 1] \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix} \\ &= [-4 \ 4 \ 4 \ 4] \end{aligned}$$

Applying activations over the net input to calculate the output, we have

$$y_j = f(y_{inj}) = \begin{cases} 1 & \text{if } y_{inj} > 0 \\ -1 & \text{if } y_{inj} \leq 0 \end{cases}$$

$$= [-1 \ 1 \ 1 \ 1]$$

Hence the correct response is obtained.

Testing the network with one missing entry

• Test input $x = [0 \ 1 \ 1 \ 1]$. Computing the input, we get

$$\begin{aligned} y_{inj} &= x \cdot W = [0 \ 1 \ 1 \ 1] \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix} \\ &= [-3 \ 3 \ 3 \ 3] \end{aligned}$$

Applying the activations, we get $y_j = [-1 \ 1 \ 1 \ 1]$ which is the correct response.

Test input $x = [-1 \ 1 \ 0 \ 1]$. Computing net input, we obtain

$$\begin{aligned} y_{inj} &= x \cdot W \\ &= [-1 \ 1 \ 0 \ 1] \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix} \\ &= [-3 \ 3 \ 3 \ 3] \end{aligned}$$

Applying the activations, we get $y_j = [-1 \ 1 \ 1 \ 1]$ which is the correct response.

Testing the network with one mistake entry

• Test input $x = [-1 \ -1 \ 1 \ 1]$. Computing net input, we get

$$y_{inj} = x \cdot W$$

$$\begin{aligned} &= [-1 \ -1 \ 1 \ 1] \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix} \\ &= [-2 \ 2 \ 2 \ 2] \end{aligned}$$

Applying the activations, we get $y_j = [-1 \ 1 \ 1 \ 1]$ which is the correct response.

• Test input $x = [1 \ 1 \ 1 \ 1]$. Computing net input, we get

$$\begin{aligned} y_{inj} &= x \cdot W = [1 \ 1 \ 1 \ 1] \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix} \\ &= [-2 \ 2 \ 2 \ 2] \end{aligned}$$

Applying the activations, we get $y_j = [-1 \ 1 \ 1 \ 1]$ which is the correct response.

Testing the network with two missing entry

• Test input $x = [0 \ 0 \ 1 \ 1]$. Computing net input, we get

$$\begin{aligned} y_{inj} &= x \cdot W = [0 \ 0 \ 1 \ 1] \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix} \\ &= [-2 \ 2 \ 2 \ 2] \end{aligned}$$

Applying the activations, we get $y_j = [-1 \ 1 \ 1 \ 1]$ which is the correct response.

• Test input $x = [-1 \ 0 \ 0 \ 1]$. Computing net input, we obtain

$$\begin{aligned} y_{inj} &= x \cdot W \\ &= [-1 \ 0 \ 0 \ 1] \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix} \\ &= [-2 \ 2 \ 2 \ 2] \end{aligned}$$

Applying the activations, we get $y_j = [-1 \ 1 \ 1 \ 1]$ which is the correct response.

Testing the network with two mistaken entry

Test input $x = [-1 \ -1 \ -1 \ 1]$. Computing net input, we obtain

$$y_{inj} = x \cdot W$$

$$= [-1 \ -1 \ -1 \ 1] \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix}$$

$$= [0 \ 0 \ 0 \ 0]$$

Applying the activations over the net input, we get $y_j = [0 \ 0 \ 0 \ 0]$ which is the incorrect response. Thus, the network with two mistakes is not recognized.

9. Check the autoassociative network for input vector $[1 \ 1 \ -1]$. Form the weight vector with no self-connection. Test whether the net is able to recognize with one missing entry.

Solution: Input vector $x = [1 \ 1 \ -1]$. The weight vector is

$$W = \sum s^T(p)s(p) = \begin{bmatrix} 1 \\ 1 \\ -1 \end{bmatrix} [1 \ 1 \ -1]$$

$$= \begin{bmatrix} 1 & 1 & -1 \\ 1 & 1 & -1 \\ -1 & -1 & 1 \end{bmatrix}$$

The weight vector with no self-connection (make the diagonal elements in the weight vector zero) is given by

$$W = \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \end{bmatrix}$$

Testing the network with one missing entry

• Test input $x = [1 \ 0 \ -1]$

$$y_{inj} = x \cdot W = [1 \ 0 \ -1] \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \end{bmatrix}$$

$$= [1 \ 2 \ -1]$$

Applying the activations, we get $y_j = [1 \ 1 \ -1]$, hence a correct response is obtained.

• Test input $x = [1 \ 1 \ 0]$

$$y_{inj} = x \cdot W = [1 \ 1 \ 0] \begin{bmatrix} 0 & 1 & -1 \\ 1 & 0 & -1 \\ -1 & -1 & 0 \end{bmatrix}$$

$$= [1 \ 1 \ -2]$$

Applying the activations, we get $y_j = [1 \ 1 \ -1]$, hence a correct response is obtained.

10. Use outer products rule to store the vectors $[1 \ 1 \ 1 \ 1]$ and $[-1 \ 1 \ 1 \ -1]$ in an auto-associative network. (a) Find the weight matrix (do not set diagonal term to zero). (b) Test the vector using $[1 \ 1 \ 1 \ 1]$ as input. (c) Test the vector $[-1 \ 1 \ 1 \ -1]$ as input. (d) Test the net using $[1 \ 1 \ 1 \ 0]$ as input. (e) Repeat (a)–(d) with the diagonal terms in the weight matrix to be zero.

Solution:

• Weight matrix for $[1 \ 1 \ 1 \ 1]$ is

$$W_1 = \sum s^T(p)s(p)$$

$$= \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} [1 \ 1 \ 1 \ 1] = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Weight matrix for $[-1 \ 1 \ 1 \ -1]$ is

$$W_2 = \sum s^T(p)s(p) = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [-1 \ 1 \ 1 \ -1]$$

$$= \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

The weight matrix to store two vectors is

$$W = W_1 + W_2$$

$$= \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

• Test the vector using $[1 \ 1 \ 1 \ 1]$ as input

Test vector $x = [1 \ 1 \ 1 \ 1]$. Computing net input, we obtain

$$y_{inj} = x \cdot W = [1 \ 1 \ 1 \ 1] \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

$$= [4 \ 4 \ 4 \ 4]$$

Applying the activations to calculate output, we get $y_j = [1 \ 1 \ 1 \ 1]$, hence correct response is obtained.

• Test the vector using $[-1 \ 1 \ 1 \ -1]$ as input

Test vector $x = [-1 \ 1 \ 1 \ -1]$. Computing net input, we obtain

$$y_{inj} = x \cdot W = [-1 \ 1 \ 1 \ -1] \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

$$= [-4 \ 4 \ 4 \ -4]$$

Applying the activations to calculate output, we get $y_j = [-1 \ 1 \ 1 \ -1]$, hence correct response is obtained.

• Test the net using $[1 \ 1 \ 1 \ 0]$ as input

Test vector $x = [1 \ 1 \ 1 \ 0]$. Computing net input, we obtain

$$y_{inj} = x \cdot W = [1 \ 1 \ 1 \ 0] \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

$$= [2 \ 4 \ 4 \ 2]$$

Applying the activations, we get $y_j = [1 \ 1 \ 1 \ 1]$, hence the known response is obtained.

• Repeat parts a–d with diagonal element in weight matrix set to zero

(i) The weight matrix is

$$W = \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

(ii) Test the vector using $x = [1 \ 1 \ 1 \ 1]$ as input. Computing net input, we obtain

$$y_{inj} = x \cdot W = [1 \ 1 \ 1 \ 1] \begin{bmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}$$

$$= [2 \ 2 \ 2 \ 2]$$

Applying the activations, we get $y_j = [1 \ 1 \ 1 \ 1]$, hence correct response is obtained.

(iii) Test the vector using $x = [-1 \ 1 \ 1 \ -1]$ as input.

$$y_{inj} = x \cdot W = [-1 \ 1 \ 1 \ -1] \begin{bmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}$$

$$= [-2 \ 2 \ 2 \ -2]$$

Applying the activations to calculate output, we get $y_j = [-1 \ 1 \ 1 \ -1]$, hence an unknown response is obtained.

(iv) Test the vector using $x = [1 \ 1 \ 1 \ 0]$ as input.

$$y_{inj} = x \cdot W = [1 \ 1 \ 1 \ 0] \begin{bmatrix} 0 & 0 & 0 & 2 \\ 0 & 0 & 2 & 0 \\ 0 & 2 & 0 & 0 \\ 2 & 0 & 0 & 0 \end{bmatrix}$$

$$= [0 \ 2 \ 2 \ 2]$$

Applying the activations to calculate output, we get $y_j = [-1 \ 1 \ 1 \ 1]$, hence an unknown response is obtained.

11. Find the weight matrix required to store the vectors $[1 \ 1 \ -1 \ -1]$, $[-1 \ 1 \ 1 \ -1]$ and $[-1 \ 1 \ -1 \ 1]$ into W_1 , W_2 , W_3 respectively. Calculate the total weight matrix to store all the

vector and check whether it is capable of recognizing the same vectors presented. Let the weight matrix be with no self-connection.

Solution: For the first vector $[1 \ 1 \ -1 \ -1]$

$$W_1 = \sum s^T(p)s(p) = \begin{bmatrix} 1 \\ 1 \\ -1 \\ -1 \end{bmatrix} [1 \ 1 \ -1 \ -1]$$

$$= \begin{bmatrix} 1 & 1 & -1 & -1 \\ 1 & 1 & -1 & -1 \\ -1 & -1 & 1 & 1 \\ -1 & -1 & 1 & 1 \end{bmatrix}$$

With no self-connection,

$$W_{10} = \begin{bmatrix} 0 & 1 & -1 & -1 \\ 1 & 0 & -1 & -1 \\ -1 & -1 & 0 & 1 \\ -1 & -1 & 1 & 0 \end{bmatrix}$$

For the second vector $[-1 \ 1 \ 1 \ -1]$

$$W_2 = \sum s^T(p)s(p) = \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [-1 \ 1 \ 1 \ -1]$$

$$= \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}$$

With no self-connection,

$$W_{20} = \begin{bmatrix} 0 & -1 & -1 & 1 \\ -1 & 0 & 1 & -1 \\ -1 & 1 & 0 & -1 \\ 1 & -1 & -1 & 0 \end{bmatrix}$$

For the third vector $[-1 \ 1 \ -1 \ 1]$

$$W_3 = \sum s^T(p)s(p) = \begin{bmatrix} -1 \\ 1 \\ -1 \\ 1 \end{bmatrix} [-1 \ 1 \ -1 \ 1]$$

$$= \begin{bmatrix} 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \\ 1 & -1 & 1 & -1 \\ -1 & 1 & -1 & 1 \end{bmatrix}$$

With no self-connection,

$$W_{30} = \begin{bmatrix} 0 & -1 & 1 & -1 \\ -1 & 0 & -1 & 1 \\ 1 & -1 & 0 & -1 \\ -1 & 1 & -1 & 0 \end{bmatrix}$$

The total weight matrix required to store all this is

$$W = W_{10} + W_{20} + W_{30}$$

$$= \begin{bmatrix} 0 & 1 & -1 & -1 \\ 1 & 0 & -1 & -1 \\ -1 & -1 & 0 & 1 \\ -1 & -1 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & -1 & -1 & 1 \\ -1 & 0 & 1 & -1 \\ 1 & -1 & 0 & -1 \\ -1 & 1 & -1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Testing the network

- With first vector $x = [1 \ 1 \ -1 \ -1]$. Net input is given by

$$y_{inj} = x \cdot W$$

$$= [1 \ 1 \ -1 \ -1] \begin{bmatrix} 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$= [1 \ 1 \ -1 \ -1]$$

Applying activations, we get $y_j = [1 \ 1 \ -1 \ -1]$ which is the correct response.

- With second vector $x = [-1 \ 1 \ 1 \ -1]$. Net input is given by

$$y_{inj} = x \cdot W$$

$$= [-1 \ 1 \ 1 \ -1] \begin{bmatrix} 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$= [-1 \ 1 \ 1 \ -1]$$

Applying activations, we get $y_j = [-1 \ 1 \ 1 \ -1]$ which is the correct response.

- With third vector $x = [-1 \ 1 \ -1 \ 1]$. Computing net input, we get

$$y_{inj} = x \cdot W$$

$$= [-1 \ 1 \ -1 \ 1] \begin{bmatrix} 0 & 0 & -2 & 0 \\ 0 & 0 & 0 & -2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$= [-1 \ 1 \ -1 \ 1]$$

Applying activations, we get $y_j = [-1 \ 1 \ -1 \ 1]$ which is the correct response.

Thus, the network is capable of recognizing the vectors.

- Construct an autoassociative network to store vectors $[-1 \ 1 \ 1 \ 1]$. Use iterative autoassociative network to test the vector with three missing elements.

Solution: The input vector is $x = [-1 \ 1 \ 1 \ 1]$. The weight matrix is obtained as

$$W = \sum_p s^T(p)s(p) = \begin{bmatrix} -1 \\ 1 \\ 1 \\ 1 \end{bmatrix} [-1 \ 1 \ 1 \ 1]$$

$$= \begin{bmatrix} 1 & -1 & -1 & -1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \\ -1 & 1 & 1 & 1 \end{bmatrix}$$

The weight matrix with no self-connection is

$$W_0 = \begin{bmatrix} 0 & -1 & -1 & -1 \\ -1 & 0 & 1 & 1 \\ -1 & 1 & 0 & 1 \\ -1 & 1 & 1 & 0 \end{bmatrix}$$

Test vector with three missing elements

- For test input vector $x = [-1 \ 0 \ 0 \ 0]$, the net input is calculated as

$$y_{inj} = x \cdot W$$

$$= [-1 \ 0 \ 0 \ 0] \begin{bmatrix} 0 & -1 & -1 & -1 \\ -1 & 0 & 1 & 1 \\ -1 & 1 & 0 & 1 \\ -1 & 1 & 1 & 0 \end{bmatrix}$$

$$= [0 \ 1 \ 1 \ 1]$$

Applying activations, we get $y_j = [-1 \ 1 \ 1 \ 1]$, i.e., known response is obtained.

- For test input vector $x = [0 \ 0 \ 0 \ 1]$. Computing net input, we obtain

$$y_{inj} = x \cdot W$$

$$= [0 \ 0 \ 0 \ 1] \begin{bmatrix} 0 & -1 & -1 & -1 \\ -1 & 0 & 1 & 1 \\ -1 & 1 & 0 & 1 \\ -1 & 1 & 1 & 0 \end{bmatrix}$$

$$= [-1 \ 1 \ 1 \ 0] \Rightarrow \text{winning}$$

Applying activations, we get $y_j = [-1 \ 1 \ 1 \ -1]$, i.e., unknown response is obtained. Iterate the network again using the net input calculated as input vector:

$$y_{inj} = [-1 \ 1 \ 1 \ 0] \begin{bmatrix} 0 & -1 & -1 & -1 \\ -1 & 0 & 1 & 1 \\ -1 & 1 & 0 & 1 \\ -1 & 1 & 1 & 0 \end{bmatrix}$$

$$= [-2 \ 2 \ 2 \ 3]$$

Applying activations, we get $y_j = [-1 \ 1 \ 1 \ 1]$, i.e., known response is obtained after iteration. Thus, iterative autoassociative network recognizes the test pattern. Similarly, the network can be tested for the test input vectors $[0 \ 1 \ 0 \ 0]$ and $[0 \ 0 \ 1 \ 0]$.

- Construct an autoassociative discrete Hopfield network with input vector $[1 \ 1 \ 1 \ -1]$. Test the discrete Hopfield network with missing entries in first and second components of the stored vector.

Solution: The input vector is $x = [1 \ 1 \ 1 \ -1]$. The weight matrix is given by

$$W = \sum s^T(p)r(p) = \begin{bmatrix} 1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [1 \ 1 \ 1 \ -1]$$

$$= \begin{bmatrix} 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ 1 & 1 & 1 & -1 \\ -1 & -1 & -1 & 1 \end{bmatrix}$$

The weight matrix with no self-connection is

$$W = \begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 \\ -1 & -1 & -1 & 0 \end{bmatrix}$$

The binary representation for the given input vector is $[1 \ 1 \ 1 \ 0]$. We carry out asynchronous updation of weights here. Let it be Y_1, Y_4, Y_3, Y_2 .

For the test input vector with two missing entries in first and second components of the stored vector.

Iteration 1

Step 0: Weights are initialized to store patterns:

$$W = \begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 \\ -1 & -1 & -1 & 0 \end{bmatrix}$$

Step 1: The input vector is $x = [0 \ 0 \ 1 \ 0]$.

Step 2: For this vector $y = [0 \ 0 \ 1 \ 0]$.

Step 3: Choose unit Y_1 for updating its activations:

$$y_{in1} = x_1 + \sum_{j=1}^4 y_j w_{j1} \\ = 0 + [0 \ 0 \ 1 \ 0] \begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \end{bmatrix} = 0 + 1 = 1$$

Applying activations we get $y_{in1} > 0 \Rightarrow y_1 = 1$. Broadcasting y_1 to all other units, we get

$y = [1 \ 0 \ 1 \ 0] \rightarrow$ No convergence

Step 4: Choosing unit Y_4 for updating its activations:

$$y_{in4} = x_4 + \sum_{j=1}^4 y_j w_{j4} \\ = 0 + [1 \ 0 \ 1 \ 0] \begin{bmatrix} -1 \\ -1 \\ -1 \\ 0 \end{bmatrix} \\ = 0 - 1 - 1 = -2$$

Applying activations we get $y_{in4} < 0 \Rightarrow y_4 = 0$. Therefore, $y = [1 \ 0 \ 1 \ 0] \rightarrow$ No convergence.

Step 5: Choosing unit Y_3 for updating its activations:

$$y_{in3} = x_3 + \sum_{j=1}^4 y_j w_{j3} \\ = 1 + [1 \ 0 \ 1 \ 0] \begin{bmatrix} 1 \\ 1 \\ 0 \\ -1 \end{bmatrix} \\ = 1 + 1 = 2$$

Applying activations we get $y_{in3} > 0 \Rightarrow y_3 = 1$. Therefore, $y = [1 \ 0 \ 1 \ 0] \rightarrow$ No convergence.

Step 6: Choosing unit Y_2 for updating its activations:

$$y_{in2} = x_2 + \sum_{j=1}^4 y_j w_{j2} \\ = 0 + [1 \ 0 \ 1 \ 0] \begin{bmatrix} 1 \\ 0 \\ 1 \\ -1 \end{bmatrix} \\ = 0 + 2 = 2$$

Applying activations we get $y_{in2} > 0 \Rightarrow y_2 = 1$. Therefore, $y = [1 \ 1 \ 1 \ 0] \rightarrow$ Converges with vector x .

Thus, the output y has converged with vector x in this iteration itself. But, one more iteration can be done to check whether further activations are there or not.

Iteration 2

Step 0: Weights are initialized to store patterns.

$$W = \begin{bmatrix} 0 & 1 & 1 & -1 \\ 1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 \\ -1 & -1 & -1 & 0 \end{bmatrix}$$

Step 1: The input vector is $x = [1 \ 1 \ 1 \ 0]$.

Step 2: For this vector $y = [1 \ 1 \ 1 \ 0]$.

Step 3: Choosing unit Y_1 for updating its activations:

$$y_{in1} = x_1 + \sum_{j=1}^4 y_j w_{j1} \\ = 1 + [1 \ 1 \ 1 \ 0] \begin{bmatrix} 0 \\ 1 \\ 1 \\ -1 \end{bmatrix} = 3$$

Apply activations we get $y_{in1} > 0 \Rightarrow y_1 = 1$. Now $y = [1 \ 1 \ 1 \ 0]$.

Step 4: Choose unit Y_4 for updation.

$$y_{in4} = x_4 + \sum_{j=1}^4 y_j w_{j4} \\ = 0 + [1 \ 1 \ 1 \ 0] \begin{bmatrix} -1 \\ -1 \\ -1 \\ 0 \end{bmatrix} = -3$$

Applying activations we get $y_{in4} < 0 \Rightarrow y_4 = 0$. Therefore, $y = [1 \ 1 \ 1 \ 0]$.

Step 5: Choose unit Y_3 for updation.

$$y_{in3} = x_3 + \sum_{j=1}^4 y_j w_{j3} \\ = 0 + [1 \ 1 \ 1 \ 0] \begin{bmatrix} 1 \\ 1 \\ 0 \\ -1 \end{bmatrix} = 3$$

Applying activations we get $y_{in3} > 0 \Rightarrow y_3 = 1$. Therefore, $y = [1 \ 1 \ 1 \ 0]$.

Step 6: Choose unit Y_2 for updation.

$$y_{in2} = x_2 + \sum_{j=1}^4 y_j w_{j2} \\ = 0 + [1 \ 1 \ 1 \ 0] \begin{bmatrix} 1 \\ 0 \\ 1 \\ -1 \end{bmatrix} = 3$$

Applying activations we get $y_{in2} > 3 \Rightarrow y_2 = 1$. Therefore, $y = [1 \ 1 \ 1 \ 0]$.

Thus, further iterations do not change the activation of any unit.

14. Construct an autoassociative network to store the vectors $x_1 = [1 \ 1 \ 1 \ 1]$, $x_2 = [1 \ -1 \ -1 \ 1]$, $x_3 = [-1 \ 1 \ -1 \ -1]$. Find weight matrix with no self-connection. Calculate the energy of the stored patterns. Using discrete Hopfield network stored patterns if the test pattern are given as $x_1 = [1 \ 1 \ 1 \ 1]$, $x_2 = [1 \ -1 \ -1 \ -1]$ and $x_3 = [1 \ 1 \ -1 \ -1]$. Compare the test patterns energy with the stored patterns energy.

Solution: The weights matrix for the three given vectors is

$$W = \sum_i x_i^T(p) x_i(p) \\ = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} [1 \ 1 \ 1 \ 1] + \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [1 \ -1 \ -1 \ 1] + \begin{bmatrix} -1 \\ 1 \\ -1 \\ -1 \end{bmatrix} [-1 \ 1 \ -1 \ -1] \\ = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \\ -1 & 1 & 1 & -1 \end{bmatrix} \\ = \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

$$W = \begin{bmatrix} 1 & -1 & 1 & 1 & 1 \\ -1 & 1 & -1 & -1 & -1 \\ 1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 & 1 \\ 1 & -1 & 1 & 1 & 1 \end{bmatrix} + \begin{bmatrix} 3 & -1 & 1 & 3 & 1 \\ -1 & 3 & 1 & -1 & 1 \\ 1 & 1 & 3 & 1 & 3 \\ 3 & -1 & 1 & 3 & 1 \\ 1 & 1 & 3 & 1 & 3 \end{bmatrix}$$

The weight matrix with no self-connection is

$$W_0 = \begin{bmatrix} 0 & -1 & 1 & 3 & 1 \\ -1 & 0 & 1 & -1 & 1 \\ 1 & 1 & 0 & 1 & 3 \\ 3 & -1 & 1 & 0 & 1 \\ 1 & 1 & 3 & 1 & 0 \end{bmatrix}$$

The energy function is defined as

$$E = -0.5[x W^T x^T]$$

Therefore the energy for the i th pattern is given by

$$E_i = -0.5[x_i W^T x_i^T]$$

Energy for first pattern

$$\begin{aligned} E_1 &= -0.5[x_1 W^T x_1^T] \\ &= -0.5[1 \ 1 \ 1 \ 1 \ 1]_{1 \times 5} \begin{bmatrix} 0 & -1 & 1 & 3 & 1 \\ -1 & 0 & 1 & -1 & 1 \\ 1 & 1 & 0 & 1 & 3 \\ 3 & -1 & 1 & 0 & 1 \\ 1 & 1 & 3 & 1 & 0 \end{bmatrix}_{5 \times 5} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}_{5 \times 1} \\ &= -0.5[1 \ 1 \ 1 \ 1 \ 1]_{1 \times 5} \begin{bmatrix} 4 \\ 0 \\ 6 \\ 4 \\ 6 \end{bmatrix}_{5 \times 1} \\ &= -0.5[4 + 0 + 6 + 4 + 6]_{1 \times 1} \\ &= -0.5[20] = -10 \end{aligned}$$

Energy for second pattern

$$\begin{aligned} E_2 &= -0.5[x_2 W^T x_2^T] \\ &= -0.5[1 \ -1 \ -1 \ 1 \ -1] \begin{bmatrix} 0 & -1 & 1 & 3 & 1 \\ -1 & 0 & 1 & -1 & 1 \\ 1 & 1 & 0 & 1 & 3 \\ 3 & -1 & 1 & 0 & 1 \\ 1 & 1 & 3 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \\ -1 \end{bmatrix} \\ &= -0.5[1 \ -1 \ -1 \ 1 \ -1] \begin{bmatrix} 2 \\ -4 \\ -2 \\ 2 \\ -2 \end{bmatrix} \\ &= -0.5[2 + 4 + 2 + 2 + 2] = -0.5[12] = -6 \end{aligned}$$

Energy for third pattern

$$\begin{aligned} E_3 &= -0.5[x_3 W^T x_3^T] \\ &= -0.5[-1 \ 1 \ -1 \ -1 \ -1] \begin{bmatrix} 0 & -1 & 1 & 3 & 1 \\ -1 & 0 & 1 & -1 & 1 \\ 1 & 1 & 0 & 1 & 3 \\ 3 & -1 & 1 & 0 & 1 \\ 1 & 1 & 3 & 1 & 0 \end{bmatrix} \begin{bmatrix} -1 \\ 1 \\ -1 \\ -1 \\ -1 \end{bmatrix} \\ &= -0.5[-1 \ 1 \ -1 \ -1 \ -1] \begin{bmatrix} -6 \\ 0 \\ -4 \\ -6 \\ -4 \end{bmatrix} \\ &= -0.5[6 + 0 + 4 + 6 + 4] = -0.5[20] = -10 \end{aligned}$$

Applying test patterns

For first test pattern $x'_1 = [1 \ 1 \ 1 \ -1 \ 1]$ and $y = [1 \ 1 \ 1 \ -1 \ 1]$. Choosing unit 4 for updation, we get

$$\begin{aligned} y_{in4} &= x_4 + \sum_{j=1}^4 y_j w_{j4} \\ &= -1 + [1 \ 1 \ 1 \ -1 \ 1] \begin{bmatrix} 3 \\ -1 \\ 1 \\ 0 \\ 1 \end{bmatrix} \end{aligned}$$

$$= -1 + 3 - 1 + 1 - 0 + 1 = 3 > 0$$

Applying activations, we get $y_4 = 1$. Therefore, $x'_1 = [1 \ 1 \ 1 \ 1 \ 1] \rightarrow$ convergence. The energy function is given by

$$E'_1 = -0.5[x'_1 W^T x'^T_1]$$

On substituting the corresponding values, we get

$$E'_1 = -10$$

• For second test pattern $x'_2 = [1 \ -1 \ -1 \ -1 \ -1]$ and $y = [1 \ -1 \ -1 \ -1 \ -1]$. Choosing unit 4 for updation, we get

$$\begin{aligned} y_{in4} &= x_4 + \sum_{j=1}^4 y_j w_{j4} \\ &= -1 + [1 \ -1 \ -1 \ -1 \ -1] \begin{bmatrix} 3 \\ -1 \\ 1 \\ 0 \\ 1 \end{bmatrix} \\ &= -1 + 3 + 1 - 1 - 0 - 1 = 1 > 0 \end{aligned}$$

Applying activations, we get $y_4 = 1$. Therefore, $x'_2 = [1 \ -1 \ 1 \ -1 \ 1] \rightarrow$ convergence. The energy function is given by

$$\begin{aligned} E'_2 &= -0.5[x'_2 W^T x'^T_2] \\ &= -0.5[1 \ -1 \ -1 \ 1 \ -1] [W^T] \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \\ -1 \end{bmatrix} \\ &= -0.5[12] = -6 \end{aligned}$$

• For third test pattern $x'_3 = [1 \ 1 \ -1 \ -1 \ -1]$ and $y = [1 \ 1 \ -1 \ -1 \ -1]$. Choosing unit 1

for updation, we get

$$\begin{aligned} y_{in1} &= x_1 + \sum_{j=1}^4 y_j w_{j1} \\ &= 1 + [1 \ 1 \ -1 \ -1 \ -1] \begin{bmatrix} 0 \\ -1 \\ 1 \\ 3 \\ 1 \end{bmatrix} \\ &= 1 + 1 - 1 - 3 - 1 = -5 < 0 \end{aligned}$$

Applying activations, we get $y = -1$. Therefore, modified $x'_3 = [-1 \ 1 \ -1 \ -1 \ -1] \rightarrow$ convergence. The energy function is given by

$$\begin{aligned} E'_3 &= -0.5[x'_3 W^T x'^T_3] \\ &= -0.5[-1 \ 1 \ -1 \ -1 \ -1] [W^T] \begin{bmatrix} -1 \\ 1 \\ -1 \\ -1 \\ -1 \end{bmatrix} \\ &= -0.5[20] = -10 \end{aligned}$$

Thus, the energy of the stored pattern is same as that of the test pattern.

15. Construct and test a BAM network to associate letters E and F with simple bipolar input-output vectors. The target output for E is $(-1, 1)$ and for F is $(1, 1)$. The display matrix size is 5×3 . The input patterns are

$$\begin{array}{ccc} * & * & * \\ * & * & * \\ * & * & * \\ * & * & * \\ * & * & * \end{array} \quad \begin{array}{ccc} * & * & * \\ * & * & * \\ * & * & * \\ * & * & * \\ * & * & * \end{array}$$

"E" "F"

Target output $(-1, 1)$ $(1, 1)$

Solution: The inputs are

Input pattern	Inputs	Targets	Weights
E	[1 1 1 1 -1 -1 1 1 1 1 -1 -1 1 1 1]	[-1, 1]	W_1
F	[1 1 1 1 1 1 1 1 -1 -1 1 -1 1 -1 -1]	[1 1]	W_2

(i) X vectors as input: The weight matrix is obtained by

$$W = \sum x^T(p) x(p)$$

$$W_1 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 1 \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \\ -1 & 1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \\ 1 & -1 \\ 1 & -1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ 1 \\ -1 \\ -1 \\ 1 \\ -1 \\ -1 \\ 1 \\ -1 \\ -1 \end{bmatrix} \begin{bmatrix} 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ -1 & -1 \\ -1 & -1 \\ 1 & 1 \\ -1 & -1 \\ -1 & -1 \\ 1 & 1 \\ -1 & -1 \\ -1 & -1 \end{bmatrix}$$

The total weight matrix is

$$W = W_1 + W_2 = \begin{bmatrix} -1 & 1 \\ -1 & 1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \\ -1 & 1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ 1 & 1 \\ -1 & -1 \\ -1 & -1 \\ -1 & -1 \\ 1 & 1 \\ -1 & -1 \\ -1 & -1 \\ 1 & 1 \\ -1 & -1 \\ -1 & -1 \end{bmatrix} = \begin{bmatrix} 0 & 2 \\ 0 & 2 \\ 0 & 2 \\ 0 & 2 \\ 2 & 0 \\ 2 & 0 \\ 0 & 2 \\ -2 & 0 \\ -2 & 0 \\ 0 & 2 \\ -2 & 0 \\ -2 & 0 \\ 0 & -2 \\ 0 & -2 \\ -2 & 0 \end{bmatrix}$$

Testing the network with test vectors "E" and "F."

- For test pattern E , computing net input we get

$$y_{in} = [1 \ 1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1 \ 1]_{1 \times 15}$$

$$\begin{bmatrix} 0 & 2 \\ 0 & 2 \\ 0 & 2 \\ 0 & 2 \\ 2 & 0 \\ -2 & 0 \\ 0 & 2 \\ -2 & 0 \\ -2 & 0 \\ 0 & 2 \\ 0 & -2 \\ 0 & -2 \\ 0 & 2 \\ -2 & 0 \\ -2 & 0 \end{bmatrix}_{15 \times 2}$$

$$= [-12 \ 18]_{1 \times 2}$$

Applying activations, we get $y = [-1 \ 1]$, hence correct response is obtained.

- For test pattern E. Computing net input, we get

$$y_{in} = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1 \ -1 \ 1 \ -1 \ -1 \ 1 \ -1 \ -1]$$

$$= [12 \ 18]$$

Applying activations over the net input, to calculate output, we get $y = [1 \ 1]$, hence correct response is obtained.

- (ii) Y vectors as input: The weight matrix when Y vectors are used as input is obtained as the transpose of the weight matrix when X vectors were presented as input, i.e.,

$$W^T = \begin{bmatrix} 0 & 0 & 0 & 0 & 2 & 2 & 0 & -2 & -2 & 0 & 0 & 0 & 0 & -2 & -2 \\ 2 & 2 & 2 & 2 & 0 & 0 & 2 & 0 & 0 & 2 & -2 & -2 & 2 & 0 & 0 \end{bmatrix}$$

Testing the network

- (a) For test pattern E, now the input is $[-1 \ 1]$. Computing net input, we have

$$y_{in} = x \cdot W^T = [-1 \ 1] \cdot \begin{bmatrix} 0 & 0 & 0 & 0 & 2 & 2 & 0 & -2 & -2 & 0 & 0 & 0 & 0 & -2 & -2 \\ 2 & 2 & 2 & 2 & 0 & 0 & 2 & 0 & 0 & 2 & -2 & -2 & 2 & 0 & 0 \end{bmatrix}$$

$$= [2 \ 2 \ 2 \ 2 \ -2 \ -2 \ 2 \ 2 \ 2 \ -2 \ -2 \ 2 \ 2 \ 2]$$

Applying the activation functions, we get

$$y = [1 \ 1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1 \ 1]$$

which is the correct response.

- (b) For test pattern F, now the input is $[1 \ 1]$. Computing net input, we have

$$y_{in} = x \cdot W^T = [1 \ 1] \cdot \begin{bmatrix} 0 & 0 & 0 & 0 & 2 & 2 & 0 & -2 & -2 & 0 & 0 & 0 & 0 & -2 & -2 \\ 2 & 2 & 2 & 2 & 0 & 0 & 2 & 0 & 0 & 2 & -2 & -2 & 2 & 0 & 0 \end{bmatrix}$$

$$= [2 \ 2 \ 2 \ 2 \ 2 \ 2 \ 2 \ -2 \ -2 \ 2 \ -2 \ -2 \ 2 \ -2 \ -2]$$

Applying the activation functions, we get

$$y = [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ -1 \ -1 \ 1 \ -1 \ -1 \ 1 \ -1 \ -1]$$

which is the correct response. Thus, a BAM network has been constructed and tested in both the directions from X to Y and Y to X.

16. (a) Find the weight matrix in bipolar form for the bidirectional associative memory using outer products rule for the following binary input-output vector pairs:

$$\begin{aligned} s(1) &= (1 \ 0 \ 0 \ 0), & t(1) &= (1 \ 0) \\ s(2) &= (1 \ 0 \ 0 \ 1), & t(2) &= (1 \ 0) \\ s(3) &= (0 \ 1 \ 0 \ 0), & t(3) &= (0 \ 1) \\ s(4) &= (0 \ 1 \ 1 \ 0), & t(4) &= (0 \ 1) \end{aligned}$$

- (b) Using the unit step function (with threshold 0) as the output units activation function, test the response of the network on each of the input patterns.

- (c) Test the response of the network on various combinations of input patterns with "mistakes" or "missing" data.

- (i) $[1 \ 0 \ -1 \ -1]$; (ii) $[-1 \ 0 \ 0 \ -1]$; (iii) $[-1 \ 1 \ 0 \ -1]$; (iv) $[1 \ 1 \ -1 \ -1]$; (v) $[1 \ 1]$

Solution:

- (a) The weight matrix for storing the four input vectors in bipolar form is

$$W = \sum_{p=1}^4 s_p^T t_p$$

$$= \begin{bmatrix} 1 \\ -1 \\ -1 \\ -1 \end{bmatrix} [1 \ -1] + \begin{bmatrix} 1 \\ -1 \\ -1 \\ 1 \end{bmatrix} [1 \ -1]$$

$$+ \begin{bmatrix} -1 \\ 1 \\ -1 \\ -1 \end{bmatrix} [-1 \ 1] + \begin{bmatrix} -1 \\ 1 \\ 1 \\ -1 \end{bmatrix} [-1 \ 1]$$

$$= \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ -1 & 1 \\ -1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$+ \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ 1 & -1 \\ 1 & -1 \end{bmatrix} + \begin{bmatrix} 1 & -1 \\ -1 & 1 \\ -1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$W = \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix}$$

- (b) The unit step function for binary with threshold 0 is used.

$$\text{For Y layer} \Rightarrow y_j = \begin{cases} 1 & \text{if } y_{in,j} > 0 \\ y_j & \text{if } y_{in,j} = 0 \\ 0 & \text{if } y_{in,j} < 0 \end{cases}$$

$$\text{For X layer} \Rightarrow x_i = \begin{cases} 1 & \text{if } x_{in,i} > 0 \\ x_i & \text{if } x_{in,i} = 0 \\ 0 & \text{if } x_{in,i} < 0 \end{cases}$$

Presenting s-input pattern

- $s(1) = [1 \ 0 \ 0 \ 0]$. Computing net input, we have

$$t_{in,j} = [1 \ 0 \ 0 \ 0] \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix}$$

$$= [4 \ -4]$$

Applying activations we get $t_j = [1 \ 0]$ which is the correct response.

- $s(2) = [1 \ 0 \ 0 \ 1]$. Computing net input,

$$t_{in,j} = [1 \ 0 \ 0 \ 1] \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix}$$

$$= [6 \ -6]$$

Applying activations we get $t_j = [1 \ 0]$ which is the correct response.

- $s(3) = [0 \ 1 \ 0 \ 0]$. Computing the net input, we have

$$t_{inj} = [0 \ 1 \ 0 \ 0] \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix} \\ = [-4 \ 4]$$

Applying activations we get $t_j = [0 \ 1]$ which is the correct response.

- $s(4) = [0 \ 1 \ 1 \ 0]$. Computing the net input, we get

$$t_{inj} = [0 \ 1 \ 1 \ 0] \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix} \\ = [-6 \ 6]$$

Applying activations we get $t_j = [0 \ 1]$ which is the correct response.

Presenting t -input pattern

- $t(1) = [1 \ 0]$. Computing the net input, we obtain

$$s_{inj} = [1 \ 0] \begin{bmatrix} 4 & -4 & -2 & 2 \\ -4 & 4 & 2 & -2 \end{bmatrix} \\ = [4 \ -4 \ -2 \ 2]$$

Applying activations we get $s_j = [1 \ 0 \ 0 \ 1]$ which is the correct response.

- $t(3) = [0 \ 1]$. Computing the net input, we obtain

$$s_{inj} = [0 \ 1] \begin{bmatrix} 4 & -4 & -2 & 2 \\ -4 & 4 & 2 & -2 \end{bmatrix} \\ = [-4 \ 4 \ 2 \ -2]$$

Applying activations we get $s_j = [0 \ 1 \ 1 \ 0]$ which is the correct response.

On presenting the pattern $[1 \ 0]$ we obtain only $[1 \ 0 \ 0 \ 1]$ and not $[1 \ 0 \ 0 \ 0]$. Similarly, on presenting the pattern $[0 \ 1]$ we obtain only $[0 \ 1 \ 1 \ 0]$ and not $[0 \ 1 \ 0 \ 0]$. This depends upon the missing data entries.

(c) Test response of network

- (i) Here $x = [1 \ 0 \ -1 \ -1]$. Calculating net input, we get

$$y_{inj} = x \cdot W \\ = [1 \ 0 \ -1 \ -1] \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix} \\ = [4 \ -4]$$

Applying activations we get $y_j = [1 \ 0]$ which is the correct response.

- (ii) Here $x = [-1 \ 0 \ 0 \ -1]$. Calculating the net input, we get

$$y_{inj} = [-1 \ 0 \ 0 \ -1] \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix} \\ = [-6 \ 6]$$

Applying activations we get $y_j = [0 \ 1]$ which is the correct response.

- (iii) Here $x = [-1 \ 1 \ 0 \ -1]$. Calculating the net input, we get

$$y_{inj} = [-1 \ 1 \ 0 \ -1] \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix} \\ = [-1 \ 0 \ 1 \ 0]$$

Applying activations we get $y_j = [0 \ 1]$ which is the correct response.

- (iv) Here $x = [1 \ 1 \ -1 \ -1]$. Calculating the net input, we get

$$y_{inj} = [1 \ 1 \ -1 \ -1] \begin{bmatrix} 4 & -4 \\ -4 & 4 \\ -2 & 2 \\ 2 & -2 \end{bmatrix} \\ = [0 \ 0]$$

Applying the previous activation and taking closely related pattern activation we get $y_j = [0 \ 1]$.

- (v) $Y = [1 \ 1]$. Computing the net input, we get

$$x_{inj} = [1 \ 1] \begin{bmatrix} 4 & -4 & -2 & 2 \\ -4 & 4 & 2 & -2 \end{bmatrix} \\ = [0 \ 0 \ 0 \ 0]$$

Thus, in this case since all the x_{inj} values are zero, to apply the activation function it may take the previous x_j values for $x_{inj} = 0$. Hence the closely related pattern can be taken to obtain the correct response.

- 17. Find the hamming distance and average hamming distance for the two given input vectors below.

$$X_1 = [1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1]$$

$$X_2 = [-1 \ 1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1 \ -1 \ 1 \ 1]$$

Solution: The hamming distance is number of different bits in two binary or bipolar vectors. Here

$$H[X_1, X_2] = 8$$

Average hamming distance equals

$$\frac{1}{n} H[X_1, X_2] = \frac{1}{12} \times 8 = \frac{8}{12}$$

("n" is the no. of components in given vector.)

4.11 Review Questions

1. What is content addressable memory?
2. Specify the functional difference between a RAM and a CAM.
3. Indicate the two main types of associative memory.
4. State the advantages of associative memory.
5. Discuss the limitations of associative memory network.
6. Explain the Hebb rule training algorithm used in pattern association.
7. State the outer products rule used for training pattern association networks.
8. Draw the architecture of an autoassociative network.
9. Explain the testing algorithm adopted to test an autoassociative network.
10. What is a heteroassociative memory network?
11. With a neat architecture, explain the training algorithm of a heteroassociative network.
12. What is a bidirectional associative memory network?
13. Is it true that input patterns may be applied at the outputs of a BAM?
14. List the activation functions used in BAM net.
15. What are the two types of BAM?
16. How are the weights determined in a discrete BAM?
17. State the testing algorithm of a discrete BAM.
18. What is the activation function used in continuous BAM?
19. Define hamming distance and storage capacity.
20. What is an energy function of a discrete BAM?
21. What is a Hopfield net?
22. Compare and contrast BAM and Hopfield networks.
23. Mention the applications of Hopfield network.
24. What is the necessity of weights with no self-connection?
25. Why are symmetrical weights and weights with no self-connection important in discrete Hopfield net?

26. What is a recurrent neural network?
27. What are the two types of Hopfield net?
28. Draw the architecture of discrete Hopfield net.
29. State the testing algorithm used in discrete Hopfield network.
30. What is the energy function of a discrete Hopfield network?
31. Mention the formula used for determining the storage capacity of a discrete Hopfield net.
32. Discuss in detail on continuous Hopfield network.
33. Make an analysis of energy function of a continuous Hopfield network.
34. What are iterative autoassociative memory nets?
35. Explain in detail on linear autoassociative memory. State the conditions of linearity.
36. Write short note on brain-in-the box model.
37. What is the functional equivalent of a temporal associative memory network?

4.12 Exercise Problems

1. Train a heteroassociative memory network using Hebb rule to store input row vector $s = (s_1 s_2 s_3 s_4)$ to the output row vector $t = (t_1 t_2)$. The vector pairs are given as below:

$$\begin{aligned} s(1) &= (1 \ 0 \ 0 \ 1), & t(1) &= (1 \ 0) \\ s(2) &= (1 \ 1 \ 1 \ 1), & t(2) &= (1 \ 0) \\ s(3) &= (1 \ 1 \ 0 \ 0), & t(3) &= (0 \ 1) \\ s(4) &= (0 \ 0 \ 1 \ 1), & t(4) &= (0 \ 1) \end{aligned}$$

2. Construct and test a heteroassociative memory network using outer products rule to store the given input-target vector pairs:

$$\begin{aligned} s(1) &= (1 \ 0 \ 1), & t(1) &= (1 \ 0) \\ s(2) &= (0 \ 1 \ 1), & t(2) &= (0 \ 1) \end{aligned}$$

3. Construct and test a heteroassociative memory net to store the given vector pairs:

$$\begin{aligned} s(1) &= (0 \ 0 \ 0 \ 1), & t(1) &= (0 \ 1) \\ s(2) &= (0 \ 0 \ 1 \ 1), & t(2) &= (0 \ 1) \\ s(3) &= (0 \ 1 \ 0 \ 0), & t(3) &= (1 \ 0) \\ s(4) &= (1 \ 1 \ 0 \ 0), & t(4) &= (1 \ 0) \end{aligned}$$

Also test the network with "noisy" input patterns included.

4. Construct and train a heteroassociative network to store the following input-output vector pair. The training input-target output vector pairs are in binary form. Obtain the weight vector in

bipolar form. The binary vector pairs are:

$$\begin{aligned} s(1) &= (1 \ 0), & t(1) &= (0 \ 1) \\ s(2) &= (1 \ 1), & t(2) &= (1 \ 0). \end{aligned}$$

Also test the performance of the network with missing and mistaken data.

5. Construct a heteroassociative network for the pattern given below:

$$\begin{array}{ccc} * & * & * \\ \bullet & * & \bullet \\ * & * & * \end{array} \quad \begin{array}{ccc} * & * & * \\ * & \bullet & \bullet \\ * & * & * \end{array}$$

"I" "C"

The target of "I" and "C" are (1, -1) and (-1, 1) respectively. Store the pattern and as well recognize the pattern.

6. Train an autoassociative network for input vector $[-1 \ 1 \ 1 \ -1]$ and also test the network with same input vector. Test the autoassociative network with one missing, one mistake, two missing and two mistake entries in test vector.
7. Check the autoassociative network for input vector $[-1 \ -1 \ 1 \ 1]$. Form the weight vector with no self-connection. Test whether the net is able to recognize with one missing and two missing data. Comment on network performance.
8. Use outer products rule to store vectors $[-1 \ -1 \ -1 \ 1]$ and $[1 \ 1 \ 1 \ -1]$ in an auto-associative network.

- Find the weight without setting diagonal terms to zero.
- Test vector using $[-1 \ -1 \ -1 \ -1]$ as input.
- Test network using $[1 \ 1 \ 1 \ 1]$ as input.
- Test the net using $[0 \ 1 \ 1 \ 0]$ as input.
- Repeat (a)-(d) with diagonal elements set to zero.

9. Find the weight matrix required to store the vectors $[1 \ 1 \ -1 \ 1 \ -1]$, $[1 \ 1 \ 1 \ 1 \ -1]$, $[-1 \ -1 \ 1 \ 1 \ -1]$ and $[1 \ 1 \ -1 \ -1 \ 1]$ in w_1, w_2, w_3, w_4 , respectively. Calculate the total weight matrix to store all the vectors and check whether it is capable of recognizing the same vectors presented. Perform the association for weight matrix with no self-connection.

10. Construct an autoassociative network to store vector $[1 \ 1 \ -1 \ +1]$. Use iterative autoassociative network to test the vector with three missing elements.

11. Construct and test an associative discrete Hopfield network with input vector $[1 \ -1 \ 1 \ 1]$. Test the network with missing entries in first and fourth components of the stored vector.

12. Construct an autoassociative network to store the vectors $x_1 = [1 \ 1 \ 1 \ 1 \ -1]$, $x_2 = [-1 \ -1 \ -1 \ 1 \ 1]$, $x_3 = [1 \ 1 \ 1 \ -1 \ -1]$. Find weight matrix with no self-connection. Calculate the energy of the stored patterns.

13. Consider a two node continuous Hopfield network. Assume the conductance is $g_{r1} = g_{r2} = 3$ mho. The gain parameter is $\lambda = 1.2$ and the external inputs are zero. Calculate the accurate energy value of the state $y = [0.1 \ 0.1]^T$.

14. Design a linear heteroassociate network that associates the following pairs of vectors.

$$\begin{aligned} x_1 &= [1, 3, -5, 1]^T, & y_1 &= [0 \ 0 \ 0]^T \\ x_2 &= [2, 2, 0, -4]^T, & y_2 &= [1 \ 0 \ 1]^T \\ x_3 &= [1, 0, -3, 4]^T, & y_3 &= [0 \ 1 \ 1]^T \end{aligned}$$

Verify that vectors x_1, x_2 and x_3 are linearly independent. Compute weight matrix of linear associates.

15. Consider a discrete Hopfield network with a synchronous update.

- Show that if all given pattern vectors are orthogonal, then every original pattern is an global minimum.
- Show that in general other global minima exist.

16. Construct and test a BAM network to associate letters T and O with simple bipolar input-output vectors. The target output for T is (1, -1) and for O is (1, 1). The display matrix size is 4×3 . The input patterns are

$$\begin{array}{ccc} * & * & * \\ \bullet & * & \bullet \\ * & * & * \\ \bullet & * & \bullet \end{array} \quad \begin{array}{ccc} * & * & * \\ * & \bullet & * \\ * & \bullet & * \\ * & * & * \end{array}$$

"T" "O"

17. Find the weight matrix in bipolar form for the BAM using outer products rule for the following binary input-output vector pairs.

$$\begin{aligned} s(1) &= (1 \ 0 \ 0 \ 0), & t(1) &= (0 \ 1) \\ s(2) &= (0 \ 1 \ 1 \ 0), & t(2) &= (1 \ 0) \end{aligned}$$

Using the unit step function as the output unit's activation function, test the response of the network on each of the input patterns. Also test the response of the network on various combinations of input pattern with "mistakes" or "missing" data.

18. Find the hamming distance and average hamming distance for the two given input vectors below:

$$\begin{aligned} X_1 &= [1 \ 1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ -1 \ -1 \ -1 \ 1 \ 1 \ -1] \\ X_2 &= [1 \ 1 \ -1 \ 1 \ 1 \ -1 \ 1 \ -1 \ -1 \ 1 \ 1 \ -1 \ 1 \ -1] \end{aligned}$$

19. Prove the stability of the continuous BAM using (a) Kohonen Grossberg theorem and (b) the Lyapunov theorem.

20. Design a BAM-based temporal associative memory with a threshold activation function to recall the following sequence:

$$s = \{[1 \ 1 \ 1 \ 1 - 1 \ 1 \ 1], [1 \ 1 \ 1 \ 1 - 1 - 1 - 1], [-1 \ 1 \ 1 \ 1 \ 1 - 1 - 1]\}$$

4.13 Projects

1. Write a computer program to implement a heteroassociative memory network using Hebb rule to set the weights. Develop the input patterns and target output of your own.
2. Write a program to construct and test an autoassociative network to store numerical values from 0–9. Also create the patterns for 0–9 using a 5×3 array matrix. Add “noise” to the input signals and test the network.
3. Write a “C” program to implement a discrete Hopfield net to store the letters A–E. Form the input patterns for the letters in a 4×3 array matrix.
4. Write a computer program to implement a bipolar BAM. Allow 15 units in X layer and 3 units in Y layer use the program to store the following patterns (the X layer vectors are the letters given in the 5×3 arrays and the associated Y layer vectors are given below in each

Compute the weight matrix W and check the recall of patterns in forward and backward directions.

x pattern):

“A”	“B”	“C”
* * *	* * *	* * *
* * *	* * *	* * *
* * *	* * *	* * *
* * *	* * *	* * *
(1, 1, 1)	(-1, -1, 1)	(1, -1, 1)
“D”	“E”	“F”
* * *	* * *	* * *
* * *	* * *	* * *
* * *	* * *	* * *
* * *	* * *	* * *
(-1, 1, 1)	(1, 1, -1)	(-1, -1, -1)

Is it possible to store all six patterns at once? If not, how many can be stored at the same time? Perform some experiments with noisy data.

5

Unsupervised Learning Networks

Learning Objectives

- Definition of unsupervised networks.
- Gives details on fixed weight competitive nets like Maxnet, Mexican hat and Hamming net.
- Discusses the neighborhood topology of Kohonen self-organizing feature maps.
- Provides architecture, training algorithm, flowchart depicting training process and testing algorithm of different unsupervised networks like KSOFM, counterpropagation
- network, adaptive resonance theory and LVQ.
- Enhance the features and star topology of CPN network.
- Details the variants of LVQ (LVQ2, LVQ3) and ART (ART 1 and ART 2).
- Variety of solved problems using unsupervised learning network.

5.1 Introduction

In this chapter, the study is made on the second major learning paradigm—unsupervised learning. In this learning, there exists no feedback from the system (environment) to indicate the desired outputs of a network. The network by itself should discover any relationships of interest, such as features, patterns, contours, correlations or categories, classifications in the input data, and thereby translate the discovered relationships into outputs. Such networks are also called self-organizing networks. An unsupervised learning can judge how similar a new input pattern is to typical patterns already seen, and the network gradually learns what similarity is; the network may construct a set of axes along which to measure similarity to previous patterns, i.e., it performs principal component analysis, clustering, adaptive vector quantization and feature mapping. For example, when net has been trained to classify the input patterns into any one of the output classes, say, P, Q, R, S or T, the net may respond to both the classes, P and Q or R and S. In the case mentioned, only one of several neurons should fire, i.e., respond. Hence the network has an added structure by means of which the net is forced to make a decision, so that only one unit will respond. The process for achieving this is called competition. Practically, considering a set of students, if we want to classify them on the basis of evaluation performance, their score may be calculated, and the one whose score is higher than the others should be the winner. The same principle adopted here is followed in the neural networks for pattern classification. In this case, there may exist a tie; a suitable solution is presented even when a tie occurs. Hence these nets may also be called competitive nets. The extreme form of these competitive nets is called winner-take-all. The name itself implies that only one neuron in the competing group will possess a nonzero output signal at the end of competition.

There exist several neural networks that come under this category. To list out a few: Maxnet, Mexican hat, Hamming net, Kohonen self-organizing feature map, counterpropagation net, learning vector quantization (LVQ) and adaptive resonance theory (ART). These networks are dealt in detail in forthcoming sections. In the case of unsupervised learning, the net seeks to find patterns or regularity in the input data by forming clusters. ART networks are called clustering nets. In these types of clustering nets, there are as many input units as an input vector possessing components. Since each output unit represents a cluster, the number of output units will limit the number of clusters that can be formed.

The learning algorithm used in most of these nets is known as Kohonen learning. In this learning, the units update their weights by forming a new weight vector, which is a linear combination of the old weight vector and the new input vector. Also, the learning continues for the unit whose weight vector is closest to the input vector. The weight updation formula used in Kohonen learning for output cluster unit j is given as

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha [x - w_{ij}(\text{old})]$$

where x is the input vector; w_{ij} the weight vector for unit j ; α the learning rate whose value decreases monotonically as training continues. There exist two methods to determine the winner of the network during competition. One of the methods for determining the winner uses the square of the Euclidean distance between the input vector and weight vector, and the unit whose weight vector is at the smallest Euclidean distance from the input vector is chosen as the winner. The next method uses the dot product of the input vector and weight vector. The dot product between the input vector and weight vector is nothing but the net inputs calculated for the corresponding cluster units. The unit with the largest dot product is chosen as the winner and the weight updation is performed over it because the one with largest dot product corresponds to the smallest angle between the input and weight vectors, if both are of unit length. Both the methods can be applied for vectors of unit length. But generally, to avoid normalization of the input and weight vectors, the square of the Euclidean distance may be used.

5.2 Fixed Weight Competitive Nets

These competitive nets are those where the weights remain fixed, even during training process. The idea of competition is used among neurons for enhancement of contrast in their activation functions. In this section, three nets – Maxnet, Mexican hat and Hamming net – are discussed in detail.

5.2.1 Maxnet

In 1987, Lippmann developed the Maxnet which is an example for a neural net based on competition. The Maxnet serves as a subnet for picking the node whose input is larger. All the nodes present in this subnet are fully interconnected and there exist symmetrical weights in all these weighted interconnections. As such, there is no specific algorithm to train Maxnet; the weights are fixed in this case.

5.2.1.1 Architecture of Maxnet

The architecture of Maxnet is shown in Figure 5-1, where fixed symmetrical weights are present over the weighted interconnections. The weights between the neurons are inhibitory and fixed. The Maxnet with this structure can be used as a subnet to select a particular node whose net input is the largest.

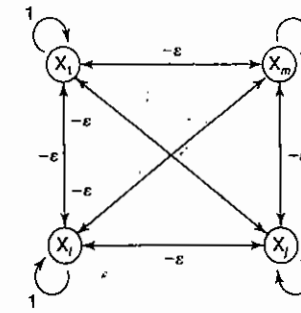


Figure 5-1 Maxnet structure.

5.2.1.2 Testing/Application Algorithm of Maxnet

The Maxnet uses the following activation function:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

The testing algorithm is as follows:

Step 0: Initial weights and initial activations are set. The weight is set as $[0 < \varepsilon < 1/m]$, where “ m ” is the total number of nodes. Let

$$x_j(0) = \text{input to the node } X_j$$

and

$$w_{ij} = \begin{cases} 1 & \text{if } i = j \\ -\varepsilon & \text{if } i \neq j \end{cases}$$

Step 1: Perform Steps 2–4, when stopping condition is false.

Step 2: Update the activations of each node. For $j = 1$ to m ,

$$x_j(\text{new}) = f\left[x_j(\text{old}) - \varepsilon \sum_{i \neq j} x_i(\text{old})\right]$$

Step 3: Save the activations obtained for use in the next iteration. For $j = 1$ to m ,

$$x_j(\text{old}) = x_j(\text{new})$$

Step 4: Finally, test the stopping condition for convergence of the network. The following is the stopping condition: If more than one node has a nonzero activation, continue; else stop.

In this algorithm, the input given to the function $f(\cdot)$ is simply the total input to node X_j from all others, including its own input.

5.2.2 Mexican Hat Net

In 1989, Kohonen developed the Mexican hat network which is a more generalized contrast enhancement network compared to the earlier Maxnet. There exist several "cooperative neighbors" (neurons in close proximity) to which every other neuron is connected by excitatory links. Also each neuron is connected over inhibitory weights to a number of "competitive neighbors" (neurons present farther away). There are several other farther neurons to which the connections between the neurons are not established. Here, in addition to the connections within a particular layer of neural net, the neurons also receive some other external signals. This interconnection pattern is repeated for several other neurons in the layer.

5.2.2.1 Architecture

The architecture of Mexican hat is shown in Figure 5-2, with the interconnection pattern for node X_i . The neurons here are arranged in linear order; having positive connections between X_i and near neighboring units, and negative connections between X_i and farther away neighboring units. The positive connection region is called region of cooperation and the negative connection region is called region of competition. The size of these regions depends on the relative magnitudes existing between the positive and negative weights and also on the topology of regions such as linear, rectangular, hexagonal grids, etc. In Mexican Hat, there exist two symmetric regions around each individual neuron.

The individual neuron in Figure 5-2 is denoted by X_i . This neuron is surrounded by other neurons X_{i+1} , X_{i-1} , X_{i+2} , X_{i-2} , The nearest neighbors to the individual neuron X_i are X_{i+1} , X_{i-1} , X_{i+2} , and X_{i-2} . Hence, the weights associated with these are considered to be positive and are denoted by w_1 and w_2 . The farthest neighbors to the individual neuron X_i are taken as X_{i+3} and X_{i-3} , the weights associated with these are negative and are denoted by w_3 . It can be seen that X_{i+4} and X_{i-4} are not connected to the individual neuron X_i , and therefore no weighted interconnections exist between these connections. To make it easier, the units present within a radius of 2 [query for unit] to the unit X_i are connected with positive weights, the units within radius 3 are connected with negative weights and the units present farther from radius 3 are not connected in any manner to the neuron X_i .

5.2.2.2 Flowchart

The flowchart for Mexican hat is shown in Figure 5-3. This clearly depicts the flow of the process performed in Mexican hat network.

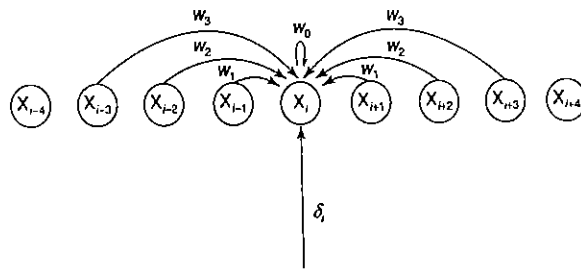


Figure 5-2 Structure of Mexican hat.

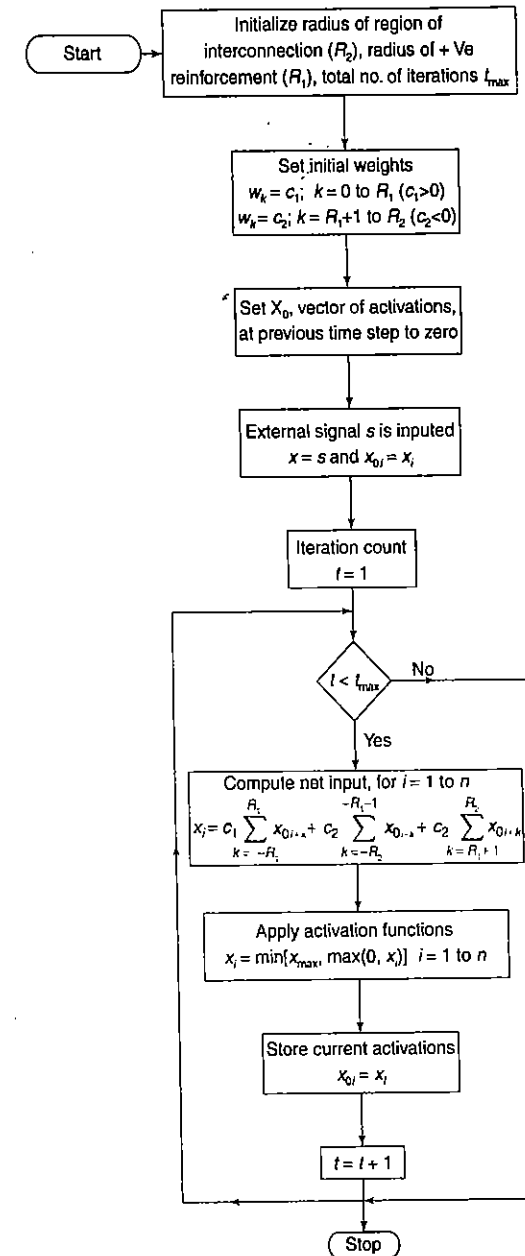


Figure 5-3 Flowchart of Mexican hat.

5.2.2.3 Algorithm

The various parameters used in the training algorithm are as shown below.

R_2 = radius of regions of interconnections

X_{i+k} and X_{i-k} are connected to the individual units X_i for $k = 1$ to R_2 .

R_1 = radius of region with positive reinforcement ($R_1 < R_2$)

w_k = weight between X_i and the units X_{i+k} and X_{i-k}

$$0 \leq k \leq R_1, \quad w_k = \text{positive}$$

$$R_1 \leq k \leq R_2, \quad w_k = \text{negative}$$

s = external input signal

x = vector of activation

x_0 = vector of activations at previous time step

$t_{\max}$ = total number of iterations of contrast enhancement.

Here the iteration is started only with the incoming of the external signal presented to the network.

Step 0: The parameters R_1 , R_2 , $t_{\max}$ are initialized accordingly. Initialize weights as

$$\begin{aligned} w_k &= c_1 \quad \text{for } k = 0, \dots, R_1 \quad (\text{where } c_1 > 0) \\ w_k &= c_2 \quad \text{for } k = R_1 + 1, \dots, R_2 \quad (\text{where } c_2 < 0) \end{aligned}$$

Initialize $x_0 = 0$.

Step 1: Input the external signal s :

$$x = s$$

The activations occurring are saved in array x_0 . For $i = 1$ to n ,

$$x_{0i} = x_i$$

Once activations are stored, set iteration counter $t = 1$.

Step 2: When t is less than $t_{\max}$, perform Steps 3–7.

Step 3: Calculate net input. For $i = 1$ to n ,

$$x_i = c_1 \sum_{k=-R_1}^{R_1} x_{0i+k} + c_2 \sum_{k=-R_2}^{-R_1-1} x_{0i+k} + c_2 \sum_{k=R_1+1}^{R_2} x_{0i+k}$$

Step 4: Apply the activation function. For $i = 1$ to n ,

$$x_i = \min[x_{\max}, \max(0, x_i)]$$

Step 5: Save the current activations in x_0 , i.e., for $i = 1$ to n ,

$$x_{0i} = x_i$$

Step 6: Increment the iteration counter:

$$t = t + 1$$

Step 7: Test for stopping condition. The following is the stopping condition:

If $t < t_{\max}$, then continue

Else stop

The positive reinforcement here has the capacity to increase the activation of units with larger initial activations and the negative reinforcement has the capacity to reduce the activation of units with smaller initial activations. The activation function used here for unit X_i at a particular time instant " t " is given by

$$x_i(t) = f \left[s_i(t) + \sum_k w_k x_{i+k} + k(t-1) \right]$$

The terms present within the summation symbol are the weighted signals that arrived from other units at the previous time step.

5.2.3 Hamming Network

The Hamming network selects stored classes, which are at a maximum Hamming distance (H) from the noisy vector presented at the input (Lippmann, 1987). The vectors involved in this case are all binary and bipolar. Hamming network is a maximum likelihood classifier that determines which of several exemplar vectors (the weight vector for an output unit in a clustering net is exemplar vector or code book vector for the pattern of inputs, which the net has placed on that cluster unit) is most similar to an input vector (represented as an n -tuple). The weights of the net are determined by the exemplar vectors. The difference between the total number of components and the Hamming distance between the vectors gives the measure of similarity between the input vector and stored exemplar vectors. It is already discussed in Chapter 4 that the Hamming distance between the two vectors is the number of components in which the vectors differ.

Consider two bipolar vectors x and y ; we use a relation

$$x \cdot y = a - d$$

where a is the number of components in which the vectors agree, d the number of components in which the vectors disagree. The value " $a - d$ " is the Hamming distance existing between two vectors. Since, the total number of components is n , we have,

$$\begin{aligned} n &= a + d \\ \text{i.e., } d &= n - a \end{aligned}$$

On simplification, we get

$$\begin{aligned} x \cdot y &= a - d \\ x \cdot y &= a - (n - a) \\ x \cdot y &= 2a - n \\ 2a &= x \cdot y + n \\ a &= \frac{1}{2}(x \cdot y) + \frac{1}{2}(n) \end{aligned}$$

From the above equation, it is clearly understood that the weights can be set to one-half the exemplar vector and bias can be set initially to $n/2$. By calculating the unit with the largest net input, the net is able to locate a particular unit that is closest to the exemplar. The unit with the largest net input is obtained by the Hamming net using Maxnet as its subnet.

5.2.3.1 Architecture

The architecture of Hamming network is shown in Figure 5-4. The Hamming network consists of two layers. The first layer computes the difference between the total number of components and Hamming distance between the input vector x and the stored pattern of vectors in the feed-forward path. The efficient response in this layer of a neuron is the indication of the minimum Hamming distance value between the input and the category, which this neuron represents. The second layer of the Hamming network is composed of Maxnet (used as a subnet) or a winner-take-all network which is a recurrent network. The Maxnet is found to suppress the values at Maxnet output nodes except the initially maximum output node of the first layer.

The function of Maxnet is to enhance the initial dominant response of the node and suppress others. Since Maxnet possesses recurrent processing, the j th node is found to respond positively while the response of all the remaining nodes decays to zero. This result needs a positive self-feedback connection with itself and a negative lateral inhibition connection.

5.2.3.2 Testing Algorithm

The given bipolar input vector is x and for a given set of " m " bipolar exemplar vectors say $e(1), \dots, e(j), \dots, e(m)$, the Hamming network is used to determine the exemplar vector that is closest to the input

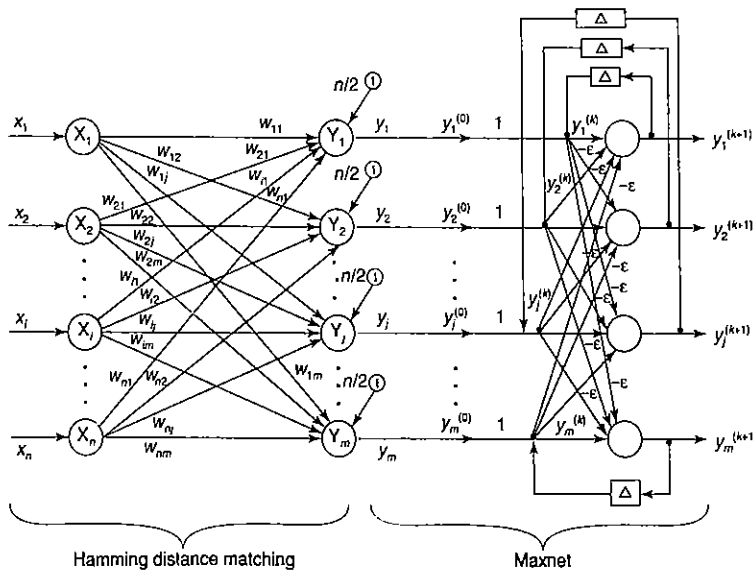


Figure 5-4 Structure of Hamming network.

vector x . The net input entering unit Y_j gives the measure of the similarity between the input vector and exemplar vector. The parameters used here are the following:

n = number of input units (number of components of input-output vector)

m = number of output units (number of components of exemplar vector)

$e(j)$ = j th exemplar vector, i.e.,

$$e(j) = [e_1(j), \dots, e_i(j), \dots, e_n(j)]$$

The testing algorithm for the Hamming Net is as follows:

Step 0: Initialize the weights. For $i = 1$ to n and $j = 1$ to m ,

$$w_{ij} = \frac{e_i(j)}{2}$$

Initialize the bias for storing the " m " exemplar vectors. For $j = 1$ to m ,

$$b_j = \frac{n}{2}$$

Step 1: Perform Steps 2–4 for each input vector x .

Step 2: Calculate the net input to each unit Y_j , i.e.,

$$y_{inj} = b_j + \sum_{i=1}^n x_i w_{ij}, \quad j = 1 \text{ to } m$$

Step 3: Initialize the activations for Maxnet, i.e.,

$$y_j(0) = y_{inj}, \quad j = 1 \text{ to } m$$

Step 4: Maxnet is found to iterate for finding the exemplar that best matches the input patterns.

The Hamming network is found to retrieve only the closest class index and not the entire vector. Hence, the Hamming network is a classifier, rather than being an associative memory. The Hamming network can be modified to be an associative memory by just adding an extra layer over the Maxnet, such that the winner unit, $y_j(k+1)$, present in the Maxnet may trigger a corresponding stored weight vector. Such an associative memory network can be called a Hamming memory network.

5.3 Kohonen Self-Organizing Feature Maps

5.3.1 Theory

Feature mapping is a process which converts the patterns of arbitrary dimensionality into a response of one- or two-dimensional arrays of neurons, i.e., it converts a wide pattern space into a typical feature space. The network performing such a mapping is called feature map. Apart from its capability to reduce the higher dimensionality, it has to preserve the neighborhood relations of the input patterns, i.e., it has to obtain a

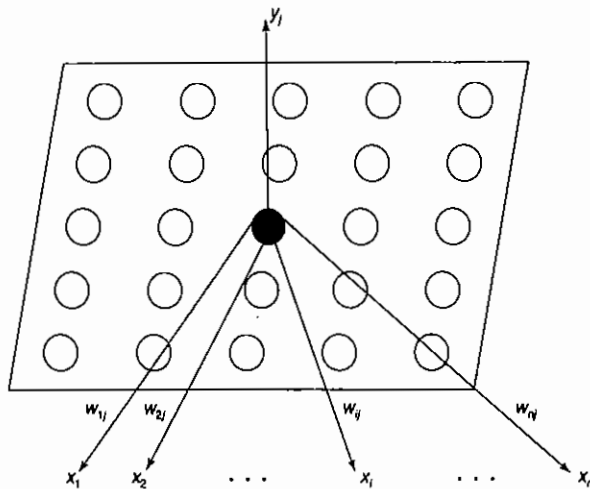


Figure 5-5 One-dimensional feature mapping network.

topology preserving map. For obtaining such feature maps, it is required to find a self-organizing neural array which consists of neurons arranged in a one-dimensional array or a two-dimensional array. To depict this, a typical network structure where each component of the input vector x is connected to each of the nodes is shown in Figure 5-5.

On the other hand, if the input vector is two-dimensional, the inputs, say $x(a, b)$, can arrange themselves in a two-dimensional array defining the input space (a, b) as in Figure 5-6. Here, the two layers are fully connected.

The topological preserving property is observed in the brain, but not found in any other artificial neural network. Here, there are m output cluster units arranged in a one- or two-dimensional array and the input signals are n -tuples. The cluster (output) units' weight vector serves as an exemplar of the input pattern that is associated with that cluster. At the time of self-organization, the weight vector of the cluster unit which matches the input pattern very closely is chosen as the winner unit. The closeness of weight vector of cluster unit to the input pattern may be based on the square of the minimum Euclidean distance. The weights are updated for the winning unit and its neighboring units. It should be noted that the weight vectors of the neighboring units are not close to the input pattern and the connective weights do not multiply the signal sent from the input units to the cluster units until dot product measure of similarity is being used.

5.3.2 Architecture

Consider a linear array of cluster units as in Figure 5-7. The neighborhoods of the units designated by "o" of radii $N_i(k_1)$, $N_i(k_2)$ and $N_i(k_3)$, $k_1 > k_2 > k_3$, where $k_1 = 2, k_2 = 1, k_3 = 0$.

For a rectangular grid, a neighborhood (N_i) of radii k_1, k_2 and k_3 is shown in Figure 5-8 and for a hexagonal grid the neighborhood is shown in Figure 5-9. In all the three cases (Figures 5-7–5-9), the unit with

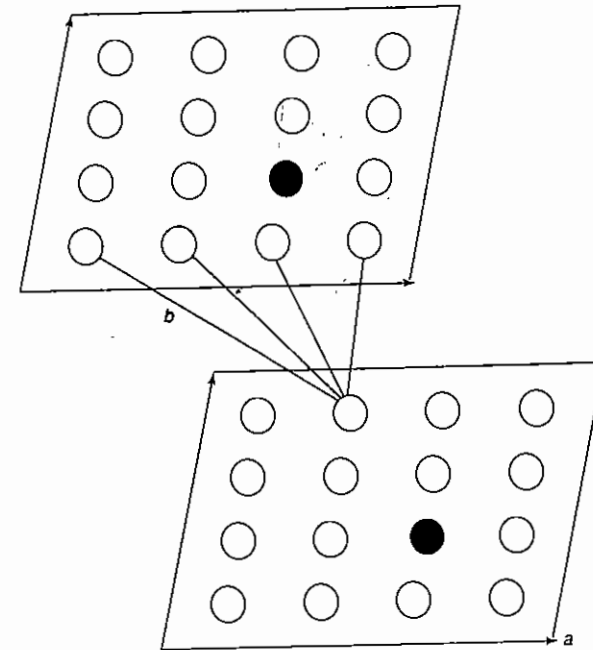


Figure 5-6 Two-dimensional feature mapping network.

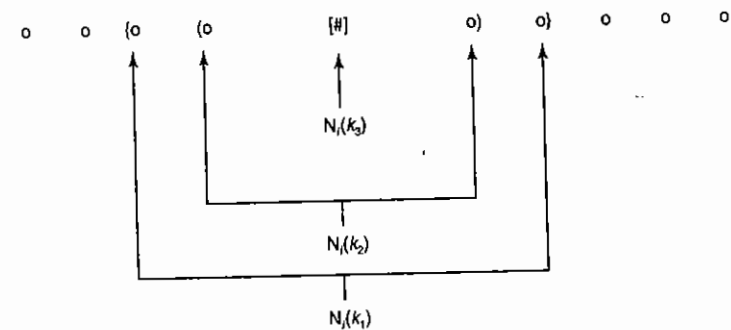


Figure 5-7 Linear array of cluster units.

"#" symbol is the winning unit and the other units are indicated by "o." In both rectangular and hexagonal grids, $k_1 > k_2 > k_3$, where $k_1 = 2, k_2 = 1, k_3 = 0$.

For rectangular grid, each unit has eight nearest neighbors but there are only six neighbors for each unit in the case of a hexagonal grid. Missing neighborhoods may just be ignored. A typical architecture of Kohonen self-organizing feature map (KSOFM) is shown in Figure 5-10.

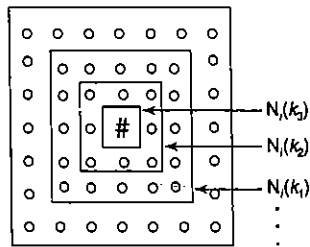


Figure 5-8 Rectangular grid.

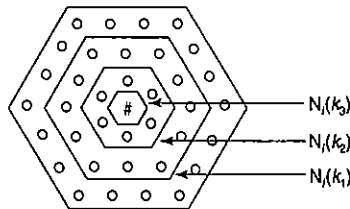


Figure 5-9 Hexagonal grid.

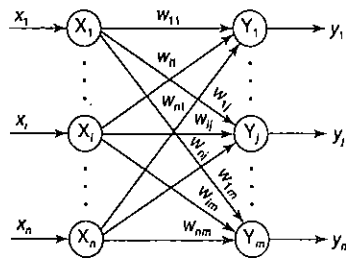


Figure 5-10 Kohonen self-organizing feature map architecture.

5.3.3 Flowchart

The flowchart for KSOFM is shown in Figure 5-11, which indicates the flow of training process. The process is continued for particular number of epochs or till the learning rate reduces to a very small rate.

The architecture consists of two layers: input layer and output layer (cluster). There are " n " units in the input layer and " m " units in the output layer. Basically, here the winner unit is identified by using either dot product or Euclidean distance method and the weight updation using Kohonen learning rules is performed over the winning cluster unit.

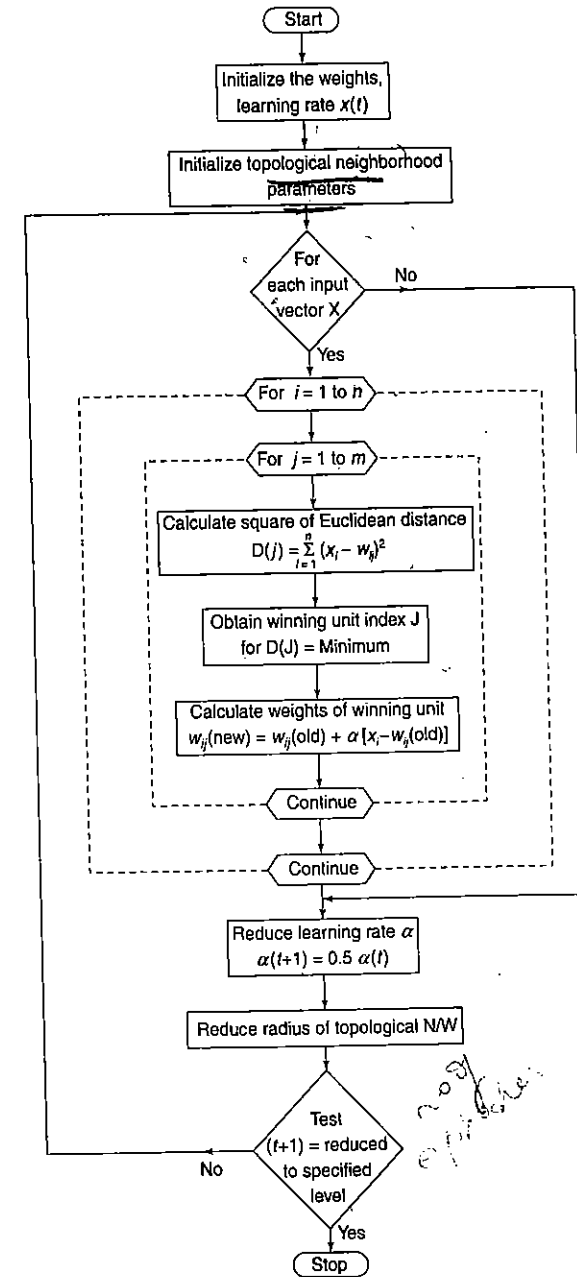


Figure 5-11 Flowchart for training process of KSOFM.

5.3.4 Training Algorithm

The steps involved in the training algorithm are as shown below.

- Step 0:
- Initialize the weights w_{ij} : Random values may be assumed. They can be chosen as the same range of values as the components of the input vector. If information related to distribution of clusters is known, the initial weights can be taken to reflect that prior knowledge.
 - Set topological neighborhood parameters: As clustering progresses, the radius of the neighborhood decreases.
 - Initialize the learning rate α : It should be a slowly decreasing function of time.

Step 1: Perform Steps 2-8 when stopping condition is false.

Step 2: Perform Steps 3-5 for each input vector x .

Step 3: Compute the square of the Euclidean distance, i.e., for each $j = 1$ to m ,

$$D(j) = \sum_{i=1}^n \sum_{j=1}^m (x_i - w_{ij})^2$$

Step 4: Find the winning unit index J , so that $D(J)$ is minimum. (In Steps 3 and 4, dot product method can also be used to find the winner, which is basically the calculation of net input, and the winner will be the one with the largest dot product.)

Step 5: For all units j within a specific neighborhood of J and for all i , calculate the new weights:

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha (x_i - w_{ij}(\text{old}))$$

or

$$w_{ij}(\text{new}) = (1 - \alpha) w_{ij}(\text{old}) + \alpha x_i$$

Step 6: Update the learning rate α using the formula $\alpha(t+1) = 0.5\alpha(t)$.

Step 7: Reduce radius of topological neighborhood at specified time intervals.

Step 8: Test for stopping condition of the network.

Thus using this training algorithm, an efficient training can be performed for an unsupervised learning network.

5.3.5 Kohonen Self-Organizing Motor Map

The extension of Kohonen feature map for a multilayer network involves the addition of an association layer to the output of the self-organizing feature map layer. The output node is found to associate the desired output values with certain input vectors. This type of architecture is called as Kohonen self-organizing motor map (KSOMM; Ritter, 1992) and layer that is added is called a motor map in which the movement commands are being mapped into two-dimensional locations of excitation. The architecture of KSOMM is shown in Figure 5-12. Here, the feature map is a hidden layer and this acts as a competitive network which classifies the input vectors. The feature map is trained as discussed in Section 5.3.3. The motor map formation is based on the learning of a control task. The motor map learning may be either supervised or unsupervised learning and can be performed by delta learning rule or outstar learning rule (to be discussed later). The motor map learning is an extension of Kohonen's original learning algorithm.

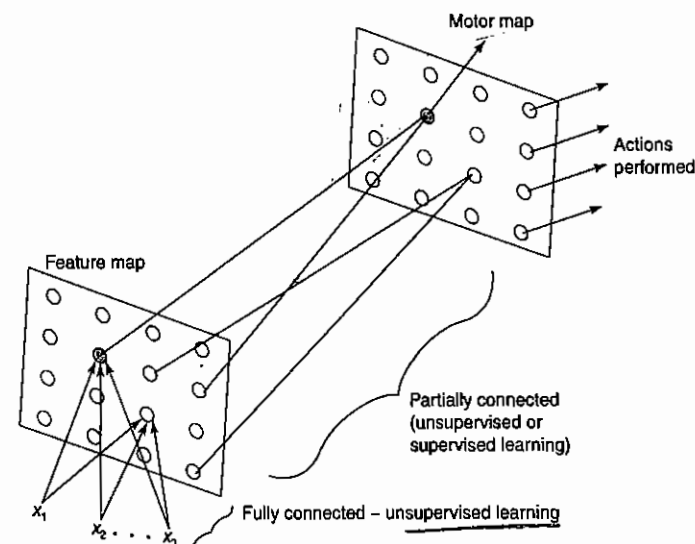


Figure 5-12 Architecture of Kohonen self-organizing motor map.

5.4 Learning Vector Quantization

5.4.1 Theory

Learning vector quantization (LVQ) is a process of classifying the patterns, wherein each output unit represents a particular class. Here, for each class several units should be used. The output unit weight vector is called the reference vector or code book vector for the class which the unit represents. This is a special case of competitive net, which uses supervised learning methodology. During training, the output units are found to be positioned to approximate the decision surfaces of the existing Bayesian classifier. Here, the set of training patterns with known classifications is given to the network, along with an initial distribution of the reference vectors. When the training process is complete, an LVQ net is found to classify an input vector by assigning it to the same class as that of the output unit, which has its weight vector very close to the input vector. Thus, LVQ is a classifier paradigm that adjusts the boundaries between categories to minimize existing misclassification. LVQ is used for optical character recognition, converting speech into phonemes and other applications as well. LVQ net may resemble KSOFM net. Unlike LVQ, KSOFM output nodes do not correspond to the known classes but rather correspond to unknown clusters that the KSOFM finds in the data autonomously.

5.4.2 Architecture

Figure 5-13 shows the architecture of LVQ, which is almost the same as that of KSOFM, with the difference being that in the case of LVQ, the topological structure at the output unit is not being considered. Here, each output unit has knowledge about what a known class represents.

From Figure 5-13 it can be noticed that there exists input layer with " n " units and output layer with " m " units. The layers are found to be fully interconnected with weighted linkage acting over the links.

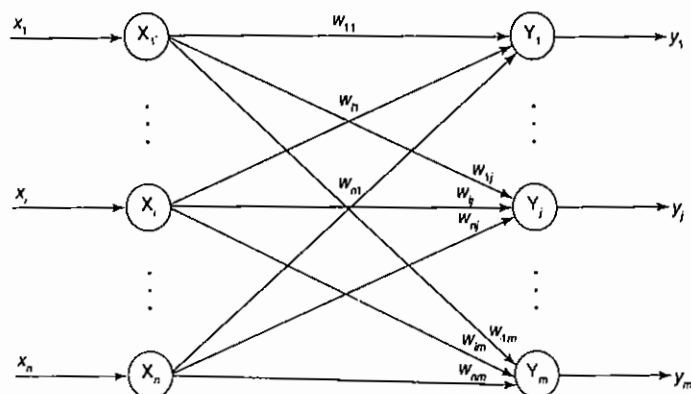


Figure 5-13 Architecture of LVQ.

5.4.3 Flowchart

The parameters used for the training process of a LVQ include the following:

x = training vector $(x_1, \dots, x_i, \dots, x_n)$

T = category or class for the training vector x

w_j = weight vector for j th output unit $(w_{1j}, \dots, w_{ij}, \dots, w_{nj})$

c_j = cluster or class or category associated with j th output unit.

The Euclidean distance of j th output unit is $D(j) = \sum (x_i - w_{ij})^2$. The flowchart indicating the flow of training process is shown in Figure 5-14.

5.4.4 Training Algorithm

In case of training, a set of training input vectors with a known classification is provided with some initial distribution of reference vector. Here, each output unit will have a known class. The objective of the algorithm is to find the output unit that is closest to the input vector.

Step 0: Initialize the reference vectors. This can be done using the following steps.

- From the given set of training vectors, take the first " m " (number of clusters) training vectors and use them as weight vectors, the remaining vectors can be used for training.
- Assign the initial weights and classifications randomly.
- K-means clustering method.

Set initial learning rate α .

Step 1: Perform Steps 2–6 if the stopping condition is false.

Step 2: Perform Steps 3–4 for each training input vector x .

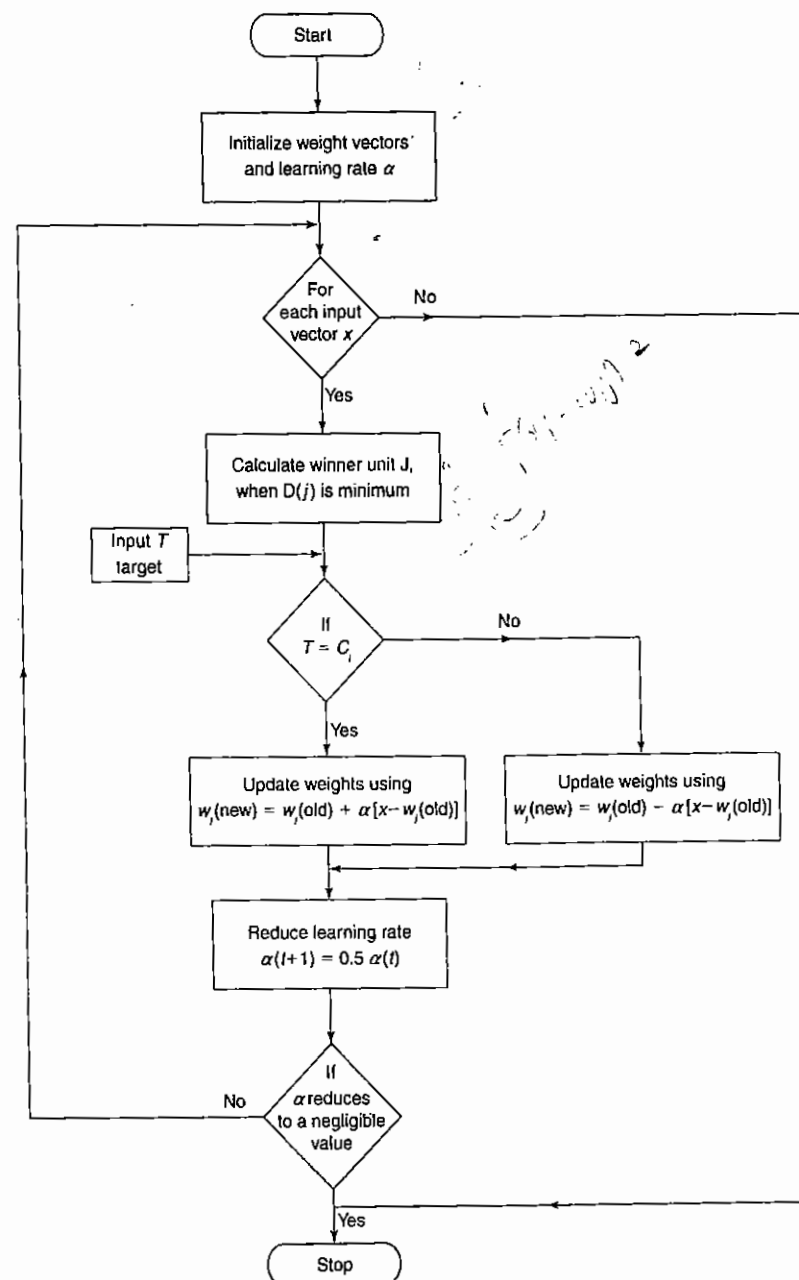


Figure 5-14 Flowchart for LVQ.

Step 3: Calculate the Euclidean distance; for $i = 1$ to n , $j = 1$ to m ,

$$D(j) = \sum_{i=1}^n \sum_{j=1}^m (x_i - w_{ij})^2$$

Find the winning unit index J , when $D(J)$ is minimum.

Step 4: Update the weights on the winning unit, w_j using the following conditions.

$$\text{If } T = q, \text{ then } w_j(\text{new}) = w_j(\text{old}) + \alpha [x - w_j(\text{old})]$$

$$\text{If } T \neq q, \text{ then } w_j(\text{new}) = w_j(\text{old}) - \alpha [x - w_j(\text{old})]$$

Step 5: Reduce the learning rate α .

Step 6: Test for the stopping condition of the training process. (The stopping conditions may be fixed number of epochs or if learning rate has reduced to a negligible value.)

5.4.5 Variants

There exists several variants of LVQ net proposed by Kohonen. These include LVQ2, LVQ2.1 and LVQ3. In the LVQ algorithm, only the reference vector that is closest to the input vector is updated. The movement it moves is based on whether or not the winning vector belongs to the same class as the input vector. In the developed versions of LVQ, two vectors called winner vector and runner-up vector learn if several conditions are satisfied. Here two distances have to be calculated. Learning takes place only if the input is approximately the same distance from winner and runner. One distance is from winner to input layer and the other is from runner to input layer.

5.4.5.1 LVQ 2

The conditions over which both vectors are modified in the case of LVQ 2 are the following.

1. The winner and the runner-up unit belong to different classes.
2. The runner-up vector is of the same class as the input vector.
3. The distances between the input vector and winner and between the input vector and runner-up are almost equal to each other.

If x is the current input vector, y_1 the reference vector closer to x (winner), y_2 the reference vector next closer to x (runner-up), d_1 the distance from x to y_1 , d_2 the distance from x to y_2 , then the conditions for the updation of the reference vector can be defined as follows:

$$\frac{d_1}{d_2} > (1 - \varepsilon)$$

and

$$\frac{d_2}{d_1} < (1 + \varepsilon)$$

where the value of ε is based on the number of training samples. The weight updation formulas in this case are given by

$$y_1(t+1) = y_1(t) - \alpha(t)[x(t) - y_1(t)] \text{ (belongs to different class)}$$

$$y_2(t+1) = y_2(t) + \alpha(t)[x(t) - y_2(t)] \text{ (belongs to same class)}$$

5.4.5.2 LVQ 2.1

In LVQ 2.1, the two closest reference vectors y_{1c} and y_{2c} are taken. Here updating is done on the basis of the requirements that (a) y_{1c} belongs to the correct class for the given input vector x and (b) y_{2c} does not belong to the same class as x . LVQ 2.1 does not distinguish whether the closest vector is that representing the correct class or incorrect class for the given input. The conditions for this case are given by

$$\min \left[\frac{d_{1c}}{d_{2c}}, \frac{d_{2c}}{d_{1c}} \right] > (1 - \varepsilon)$$

and

$$\max \left[\frac{d_{1c}}{d_{2c}}, \frac{d_{2c}}{d_{1c}} \right] < (1 + \varepsilon)$$

Here, it is not sure whether x is closer to y_{1c} or to y_{2c} . When the above conditions are met, the following weight updation formulas are used. If the reference vector belongs to the same class as input vector, then

$$y_{1c}(t+1) = y_{1c}(t) + \alpha(t)[x(t) - y_{1c}(t)]$$

else

$$y_{2c}(t+1) = y_{2c}(t) - \alpha(t)[x(t) - y_{2c}(t)]$$

5.4.5.3 LVQ 3

In LVQ 3, the two closest vectors are allowed to learn as long as the input vector satisfies the condition (take $\varepsilon = 0.2$)

$$\min \left[\frac{d_{1c}}{d_{2c}}, \frac{d_{2c}}{d_{1c}} \right] > (1 - \varepsilon)(1 + \varepsilon)$$

The weight updations are done in a similar manner as in LVQ 2.1 if one of the two closest vectors, y_{1c} , belongs to the same class as the input vector x and the other vector, y_{2c} , belongs to a different class. LVQ 3 extends this training algorithm to provide training if x , y_{1c} and y_{2c} belong to the same class. The weight updates, here, are given by the equation

$$y_c(t+1) = y_c(t) + \beta(t)[x(t) - y_c(t)]$$

Replace y_c with y_{1c} or y_{2c} , as the case may be. The learning rate $\beta(t)$ is a multiple of the learning rate $\alpha(t)$ that is used if y_{1c} and y_{2c} belong to different classes, i.e.,

$$\beta(t) = q \alpha(t)$$

where q is between 0.1 and 0.5

5.5 Counterpropagation Networks

5.5.1 Theory

Counterpropagation networks were proposed by Hecht Nielsen in 1987. They are multilayer networks based on the combinations of the input, output and clustering layers. The applications of counterpropagation nets are data compression, function approximation and pattern association. The counterpropagation network is basically constructed from an instar-outstar model. This model is a three-layer neural network that performs input-output data mapping, producing an output vector y in response to an input vector x , on the basis of competitive learning. The three layers in an instar-outstar model are the input layer, the hidden (competitive)

layer and the output layer. The connections between the input layer and the competitive layer are the instar structure, and the connections existing between the competitive layer and the output layer are the outstar structure. The competitive layer is going to be a winner-take-all network or a Maxnet with lateral feedback connections. There exists no lateral connection within the input layer and the output layer. The connections between the layers are fully connected.

A counterpropagation net is an approximation of its training input vector pairs by adaptively constructing a look-up-table. By this method, several data points can be compressed to a more manageable number of look-up-table entries. The accuracy of the function approximation and data compression is based on the number of entries in the look-up-table, which equals the number of units in the cluster layer of the net.

There are two stages involved in the training process of a counterpropagation net. The input vectors are clustered in the first stage. Originally, it is assumed that there is no topology included in the counterpropagation network. However, on the inclusion of a linear topology, the performance of the net can be improved. The clusters are formed using Euclidean distance method or dot product method. In the second stage of training, the weights from the cluster layer units to the output units are tuned to obtain the desired response. There are two types of counterpropagation nets: (i) Full counterpropagation net and (ii) forward-only counterpropagation net.

5.5.2 Full Counterpropagation Net

Full counterpropagation net (full CPN) efficiently represents a large number of vector pairs $x:y$ by adaptively constructing a look-up-table. The approximation here is $x^*:y^*$, which is based on the vector pairs $x:y$, possibly with some distorted or missing elements in either vector or both vectors. The network is defined to approximate a continuous function f , defined on a compact set A . The full CPN works best if the inverse function f^{-1} exists and is continuous. The vectors x and y propagate through the network in a counterflow manner to yield output vectors x^* and y^* , which are the approximations of x and y , respectively. During competition, the winner can be determined either by Euclidean distance or by dot product method. In case of dot product method, the one with the largest net input is the winner. Whenever vectors are to be compared using the dot product metric, they should be normalized. Even though the normalization can be performed without loss of information by adding an extra component, yet to avoid the complexity Euclidean distance method can be used. On the basis of this, direct comparison can be made between the full CPN and forward-only CPN.

For continuous function, the CPN is as efficient as the back-propagation net; it is a universal continuous function approximator. In case of CPN, the number of hidden nodes required to achieve a particular level of accuracy is greater than the number required by the back-propagation network. The greatest appeal of CPN is its speed of learning. Compared to various mapping networks, it requires only fewer steps of training to achieve best performance. This is common for any hybrid learning method that combines unsupervised learning (e.g., instar learning) and supervised learning (e.g., outstar learning).

As already discussed, the training of CPN occurs in two phases. In the input phase, the units in the cluster layer and input layer are found to be active. In CPN, no topology is assumed for the cluster layer units; only the winning units are allowed to learn. The weight updation learning rule on the winning cluster units is

$$\begin{aligned} v_{ij}(\text{new}) &= v_{ij}(\text{old}) + \alpha [x_i - v_{ij}(\text{old})], & i &= 1 \text{ to } n \\ w_{kj}(\text{new}) &= w_{kj}(\text{old}) + \beta [y_k - w_{kj}(\text{old})], & k &= 1 \text{ to } m \end{aligned}$$

The above is standard Kohonen learning which consists of competition among the units and selection of winner unit. The weight updation is performed for the winning unit.

In the second phase of training, only the winner unit j remains active in the cluster layer. The weights between the winning cluster unit j and the output units are adjusted so that the vector of activations of the units in the Y -output layer is Y^* which is an approximation to the input vector y and X^* which is an approximation to the input vector x . The weight updations for the units in the Y -output and X -output layers are

$$\begin{aligned} u_{jk}(\text{new}) &= u_{jk}(\text{old}) + a[y_k - u_{jk}(\text{old})], & k &= 1 \text{ to } m \\ t_{ji}(\text{new}) &= t_{ji}(\text{old}) + b[x_i - t_{ji}(\text{old})], & i &= 1 \text{ to } n \end{aligned}$$

This is Grossberg learning, a more general case of outstar learning. Outstar learning is found to occur for all units in a particular layer; there exists no competition among those units. The form of weight updation is similar for Kohonen learning and Grossberg learning. The learning rule for the output layers can also be viewed as delta learning rule. The weight change in all these cases is the product of the learning rate and the error. When tie occurs in the selection of winning unit, the unit with smallest index is chosen as the winner.

5.5.2.1 Architecture

The general structure of full CPN is shown in Figure 5-15. The complete architecture of full CPN is shown in Figure 5-16.

The four major components of the instar-outstar model are the input layer, the instar, the competitive layer and the outstar. For each node i in the input layer, there is an input value x_i . An instar responds maximally to the input vectors from a particular cluster. All the instars are grouped into a layer called the competitive layer. Each of the instar responds maximally to a group of input vectors in a different region of space. This layer of instars classifies any input vector because, for a given input, the winning instar with the strongest response identifies the region of space in which the input vector lies. Hence, it is necessary that the competitive layer single outs the winning instar by setting its output to a nonzero value and also suppressing the other outputs to zero. That is, it is a winner-take-all or a Maxnet-type network. An outstar model is found to have all the nodes in the output layer and a single node in the competitive layer. The outstar looks like the fan-out of a node. Figures 5-17 and 5-18 indicate the units that are active during each of the two phases of training a full CPN.

In the instar-outstar network model, the competitive layer participates in both the instar and outstar structures of the network. The function of these competitive instars is to recognize an input pattern through a winner-take-all competition. The winner activates a corresponding outstar which associates some desired output pattern with input pattern.

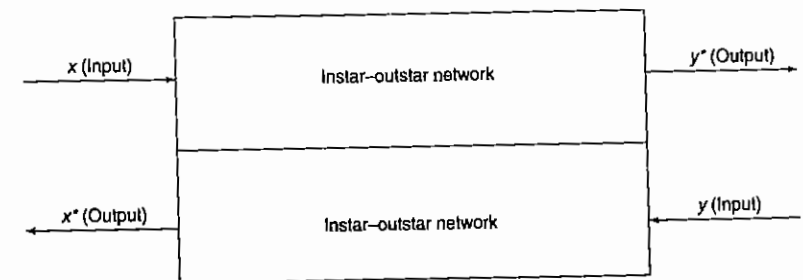


Figure 5-15 General structure of full CPN.

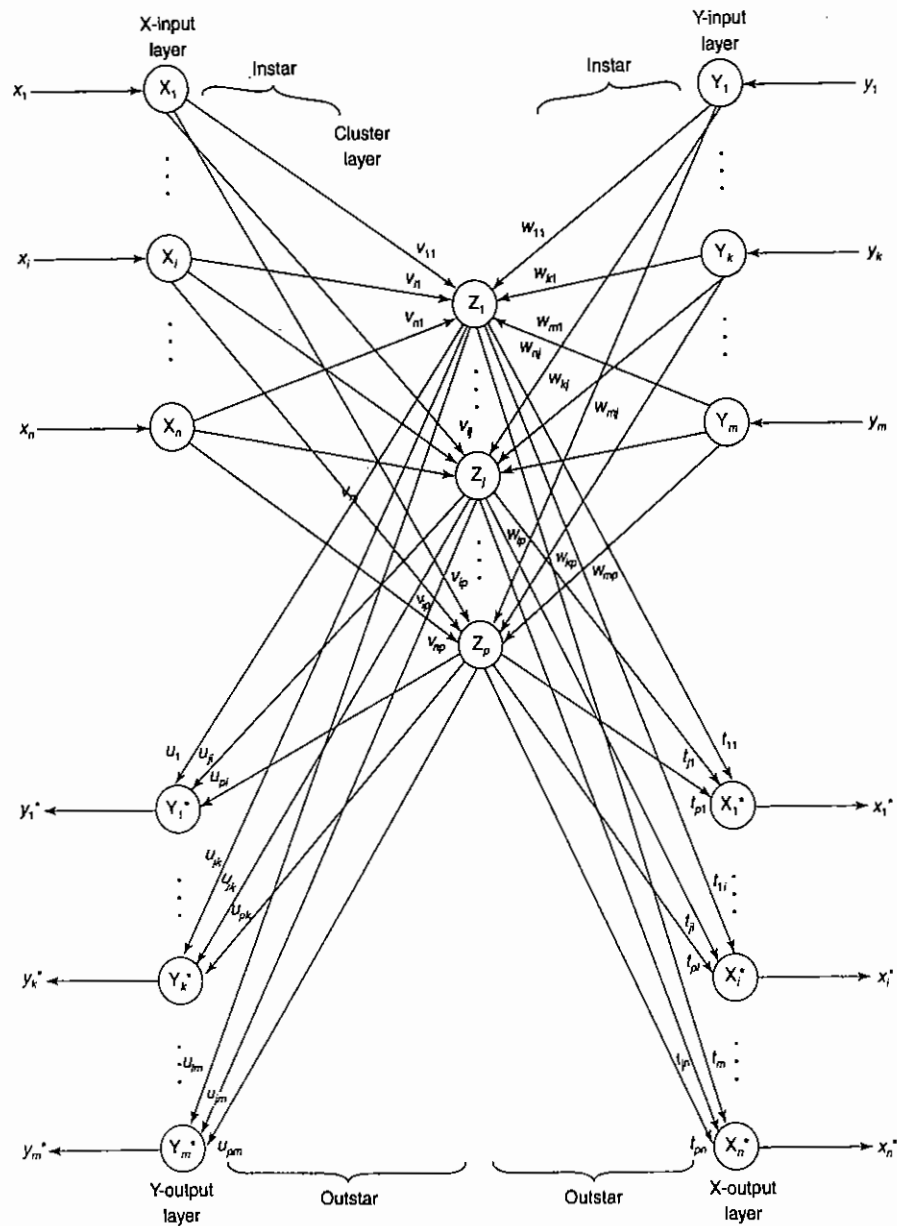


Figure 5-16 Architecture of full CPN.

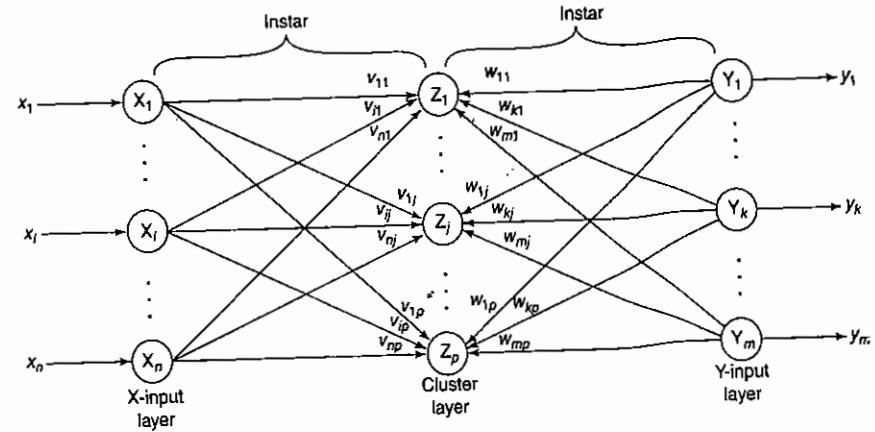


Figure 5-17 First phase of training of full CPN.

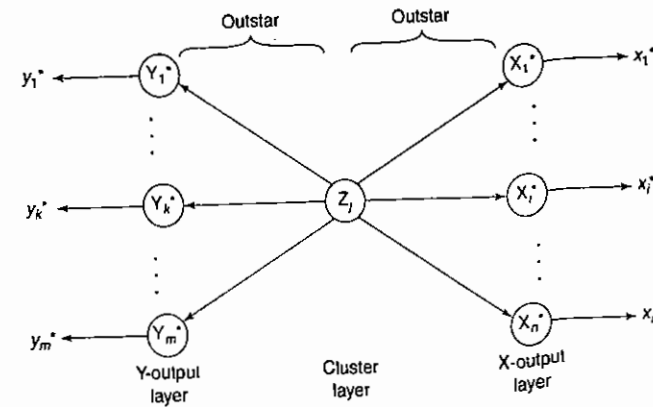


Figure 5-18 Second phase of training of full CPN.

5.5.2.2 Flowchart

The flowchart for the training process of full CPN is shown in Figure 5-19. The parameters used in the CPN are as follows:

x = input training vector $x = (x_1, \dots, x_i, \dots, x_n)$

y = target output corresponding to input x , $y = (y_1, \dots, y_k, \dots, y_m)$

z_j = the output of cluster layer unit z_j

v_{ij} = weight from X-input layer unit X_i to cluster layer unit z_j

w_{kj} = weight from Y-input layer unit Y_k to cluster layer unit z_j

u_{jk} = weight from cluster layer unit z_j to Y-output layer unit Y_k^*

t_{ji} = weight from cluster layer unit z_j to X-output layer unit X_i^*

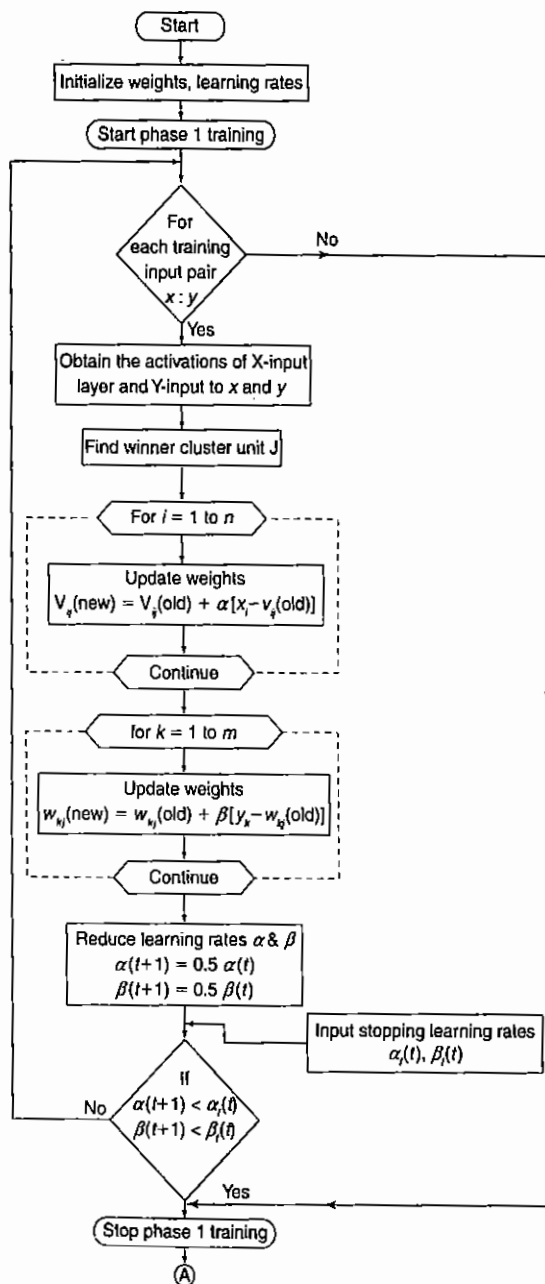


Figure 5-19 Flowchart for training of full CPN.

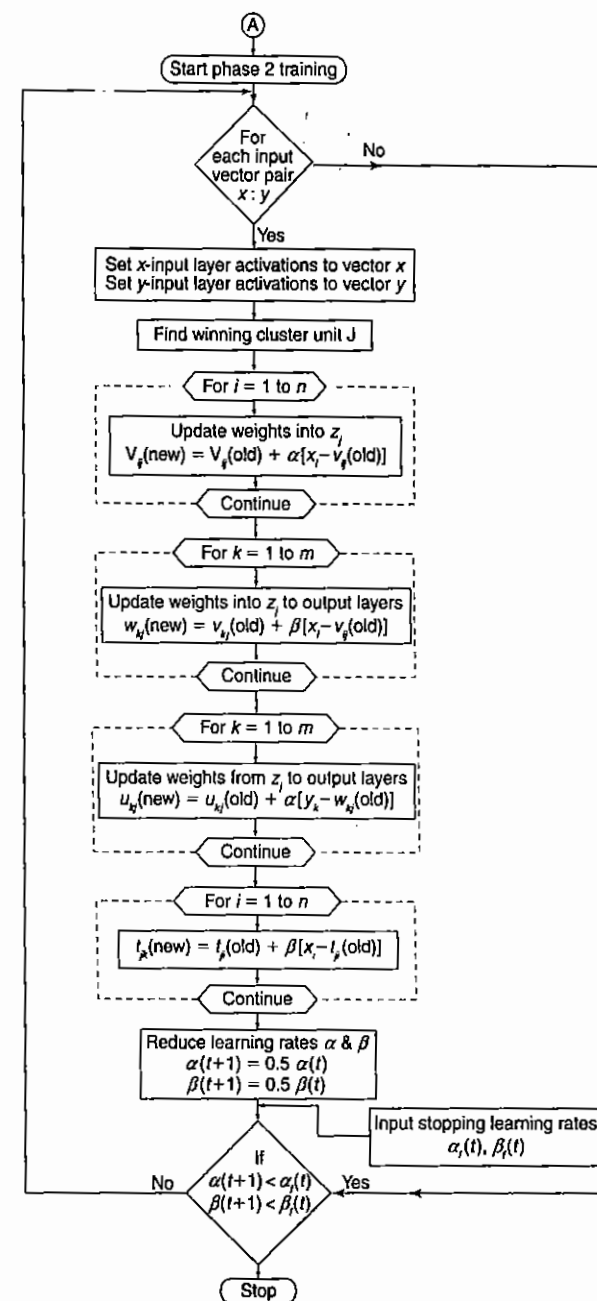


Figure 5-19 (continued).

X^* = calculated approximation to vector x

Y^* = calculated approximation to vector y

a, b = learning rates for weights out from cluster layer

α, β = learning rates for weights into cluster layer

The training phase is performed here in two stages. The stopping conditions here may be number of epochs to be reached. So the training process is performed until the number of epochs specified is completed. The reduction in learning rate can also be a stopping condition. The formula for reduction of learning rate is $\alpha(t+1) = 0.5 \alpha(t)$, where $\alpha(t)$ is learning rate at time instant " t " and $\alpha(t+1)$ is learning rate of next epoch for a time instant " $t+1$ ".

5.5.2.3 Training Algorithm

The steps involved in the training process of a full CPN are given below.

Step 0: Set the initial weights and the initial learning rate.

Step 1: Perform Steps 2–7 if stopping condition is false for phase I training.

Step 2: For each of the training input vector pair $x : y$ presented, perform Steps 3–5.

Step 3: Make the X-input layer activations to vector X .

Make the Y-input layer activations to vector Y .

Step 4: Find the winning cluster unit.

If dot product method is used, find the cluster unit z_j with target net input: for $j = 1$ to p ,

$$z_{inj} = \sum_{i=1}^n x_i v_{ij} + \sum_{k=1}^m y_k w_{kj}$$

If Euclidean distance method is used, find the cluster unit z_j whose squared distance from input vectors is the smallest:

$$D_j = \sum_{i=1}^n (x_i - v_{ij})^2 + \sum_{k=1}^m (y_k - w_{kj})^2$$

If there occurs a tie in case of selection of winner unit, the unit with the smallest index is the winner. Take the winner unit index as J .

Step 5: Update the weights over the calculated winner unit z_j .

$$\text{For } i = 1 \text{ to } n, \quad v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha [x_i - v_{ij}(\text{old})]$$

$$\text{For } k = 1 \text{ to } m, \quad w_{kj}(\text{new}) = w_{kj}(\text{old}) + \beta [y_k - w_{kj}(\text{old})]$$

Step 6: Reduce the learning rates.

$$\alpha(t+1) = 0.5 \alpha(t); \quad \beta(t+1) = 0.5 \beta(t)$$

Step 7: Test stopping condition for phase I training.

Step 8: Perform Steps 9–15 when stopping condition is false for phase II training.

Step 9: Perform Steps 10–13 for each training input pair $x : y$. Here α and β are small constant values.

Step 10: Make the X-input layer activations to vector x . Make the Y-input layer activations to vector y .

Step 11: Find the winning cluster unit (use formulas from Step 4). Take the winner unit index as J .

Step 12: Update the weights entering into unit z_j .

$$\text{For } i = 1 \text{ to } n, \quad v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha [x_i - v_{ij}(\text{old})]$$

$$\text{For } k = 1 \text{ to } m, \quad w_{kj}(\text{new}) = w_{kj}(\text{old}) + \beta [y_k - w_{kj}(\text{old})]$$

Step 13: Update the weights from unit z_j to the output layers.

$$\text{For } i = 1 \text{ to } n, \quad t_{ji}(\text{new}) = t_{ji}(\text{old}) + b [x_i - t_{ji}(\text{old})]$$

$$\text{For } k = 1 \text{ to } m, \quad u_{jk}(\text{new}) = u_{jk}(\text{old}) + a [y_k - u_{jk}(\text{old})]$$

Step 14: Reduce the learning rates a and b .

$$a(t+1) = 0.5 a(t); \quad b(t+1) = 0.5 b(t)$$

Step 15: Test stopping condition for phase II training.

If during training process initial weights are chosen appropriately, then after the completion of phase I of training, the cluster units will be uniformly distributed. When phase II of training is completed, the weights to the output units will be approximately the same as the weights into the cluster unit.

5.5.2.4 Testing (Application) Algorithm

A CPN once trained can be used for finding approximations X^* and Y^* to the input-output vector pair X and Y . The application algorithm for full CPN is as follows:

Step 0: Initialize the weights (from training algorithm).

Step 1: Perform Steps 2–4 for each input pair $X : Y$.

Step 2: Set X-input layer activations to vector X .

Set Y-input layer activations to vector Y .

Step 3: Find the cluster unit z_j that is closest to the input pair.

Step 4: Calculate approximations to x and y :

$$x_i^* = t_{ji}; \quad y_k^* = u_{jk}$$

One important variation of the CPN is operating it in an interpolation mode after the training has been completed. Here, more than one hidden node is allowed to win the competition, i.e., we have first winner, second winner, third winner, fourth winner and so on, with nonzero output values. On making the total strength of these multiple winners normalized to 1, the total output will interpolate linearly among the individual vectors. To select which nodes to fire, we can choose all those with weight vectors within a certain radius of the input x . The interpolated approximations to x and y are then

$$x_i^* = \sum_j z_j t_{ji}; \quad y_k^* = \sum_j z_j u_{jk}$$

By using interpolation, the approximation accuracy is highly increased.

5.5.3 Forward-Only Counterpropagation Net

A simplified version of full CPN is the forward-only CPN. The approximation of the function $y = f(x)$ but not of $x = f(y)$ can be performed using forward-only CPN, i.e., it may be used if the mapping from x to y is well defined but mapping from y to x is not defined. In forward-only CPN only the x -vectors are used to form the clusters on the Kohonen units. Forward-only CPN uses only the x vectors to form the clusters on the Kohonen units during first phase of training.

In case of forward-only CPN, first input vectors are presented to the input units. The cluster layer units compete with each other using winner-take-all policy to learn the input vector. Once entire set of training vectors has been presented, there exist reduction in learning rate and the vectors are presented again, performing several iterations. First, the weights between the input layer and cluster layer are trained. This is a specific competitive network, with target known. Hence, when each input vector is presented to the input vector, its associated target vectors are presented to the output layer. The winning cluster unit sends its signal to the output layer. Thus each of the output unit has a computed signal (w_{jk}) and the target value (y_k). The difference between these values is calculated; based on this, the weights between the winning layer and output layer are updated.

The weight updation from input units to cluster units is done using the learning rule given below: For $i = 1$ to n ,

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha[x_i - v_{ij}(\text{old})] = (1 - \alpha)v_{ij}(\text{old}) + \alpha x_i$$

The weight updation from cluster units to output units is done using following the learning rule: For $k = 1$ to m ,

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \alpha[y_k - w_{jk}(\text{old})] = (1 - \alpha)w_{jk}(\text{old}) + \alpha y_k$$

The learning rule for weight updation from the cluster units to output units can be written in the form of delta rule when the activations of the cluster units (z_j) are included, and is given as

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \alpha z_j [y_k - w_{jk}(\text{old})]$$

where

$$z_j = \begin{cases} 1 & \text{if } j = J \\ 0 & \text{if } j \neq J \end{cases}$$

This occurs when w_{jk} is interpreted as the computed output (i.e., $y_k = w_{jk}$). In the formulation of forward-only CPN also, no topological structure was assumed.

5.5.3.1 Architecture

Figure 5-20 shows the architecture of forward-only CPN. It consists of three layers: input layer, cluster (competitive) layer and output layer. The architecture of forward-only CPN resembles the back-propagation network, but in CPN there exists interconnections between the units in the cluster layer (which are not connected in Figure 5-20). Once competition is completed in a forward-only CPN, only one unit will be active in that layer and it sends signal to the output layer. As inputs are presented to the network, the desired outputs will also be presented simultaneously.

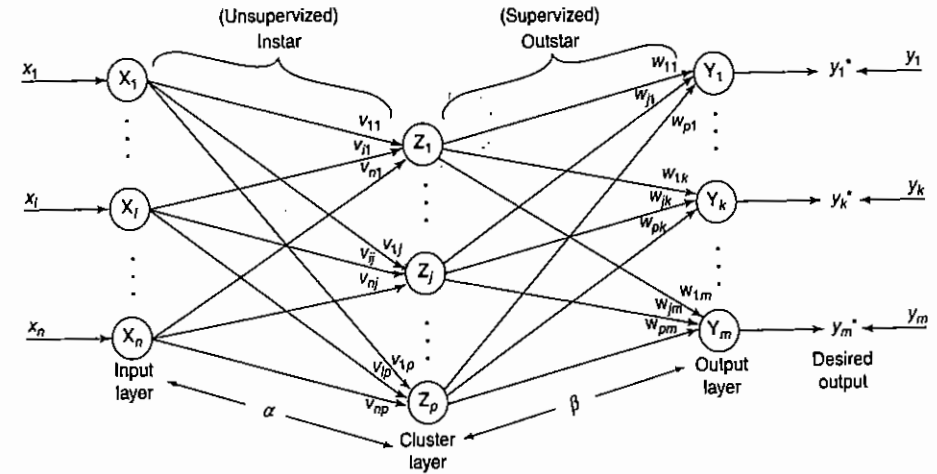


Figure 5-20 Architecture of forward-only CPN.

5.5.3.2 Flowchart

The flowchart helps in depicting the training process of forward-only CPN and the manner in which the weights are updated. The training is performed in two phases. The parameters used in flowchart and training algorithm are as follows:

α, β = learning rate parameters where $\alpha = 0.5$ to 0.8 and $\beta = 0$ to 1 . The typical values of learning rates may be $\alpha = 0.6$ and $\beta = 1$

X = activation vector for input layer units, i.e.,

$$X = (x_1, \dots, x_i, \dots, x_n)$$

$\|x - v\|$ = Euclidean distance between vectors X and V

Figure 5-21 shows the flowchart for training process of forward-only CPN.

5.5.3.3 Training Algorithm

The steps involved in the training algorithm of forward-only CPN are as follows:

- Step 0: Initialize the weights and learning rates.
- Step 1: Perform Steps 2–7 when stopping condition for phase I training is false.
- Step 2: Perform Steps 3–5 for each of training input X .
- Step 3: Set the X -input layer activations to vector X .
- Step 4: Compute the winning cluster unit (J). If dot product method is used, find the cluster unit z_j with the largest net input:

$$z_{inj} = \sum_{i=1}^n x_i v_{ij}$$

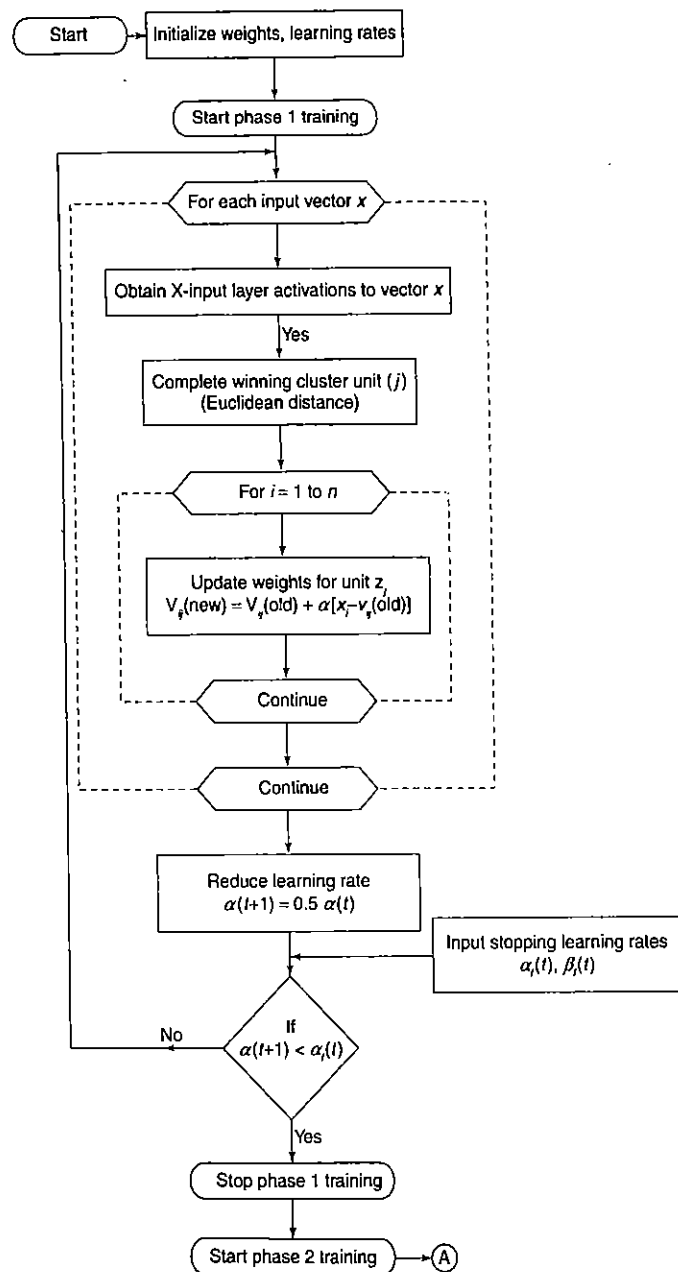


Figure 5-21 Flowchart for training of forward-only CPN.

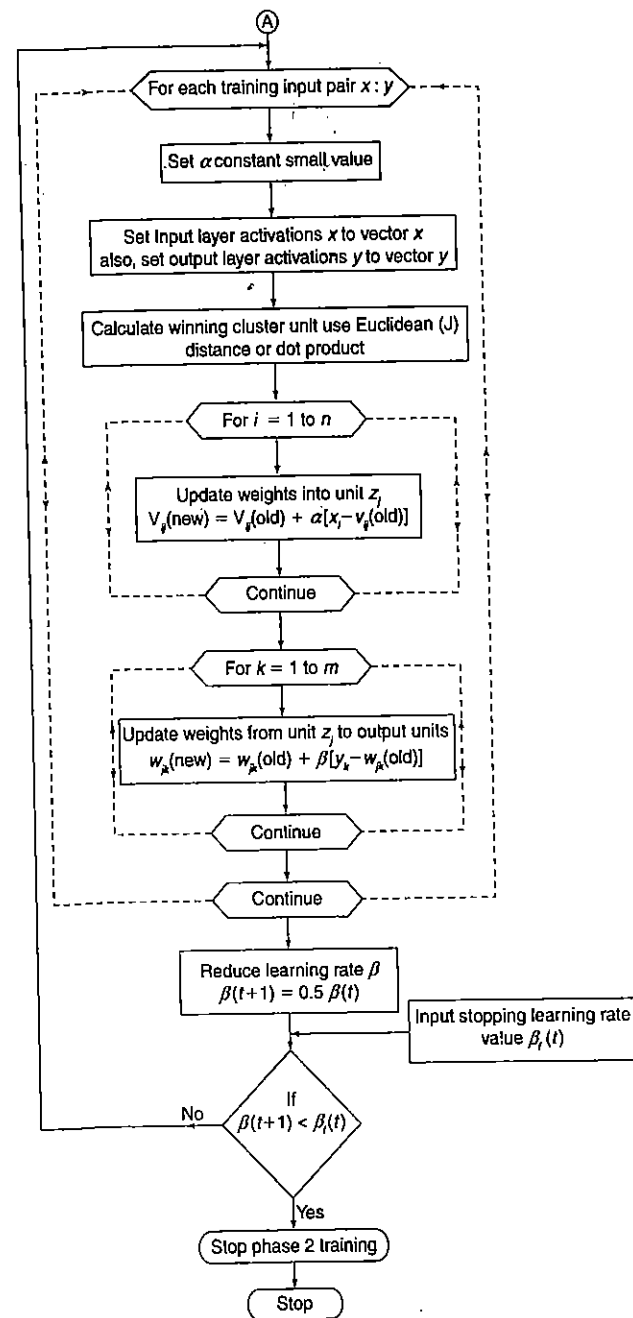


Figure 5-21 (continued).

If Euclidean distance is used, find the cluster unit z_j square of whose distance from the input pattern is smallest:

$$D_j = \sum_{i=1}^n (x_i - v_{ij})^2$$

If there exists a tie in the selection of winner unit, the unit with the smallest index is chosen as the winner.

Step 5: Perform weight updation for unit z_j . For $i = 1$ to n ,

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha[x_i - v_{ij}(\text{old})]$$

Step 6: Reduce learning rate α

$$\alpha(t+1) = 0.5\alpha(t)$$

Step 7: Test the stopping condition for phase I training.

Step 8: Perform Steps 9–15 when stopping condition for phase II training is false. (Set α a small constant value for phase II training.)

Step 9: Perform Steps 10–13 for each training input pair x, y .

Step 10: Set X-input layer activations to vector X . Set Y-output layer activations to vector Y .

Step 11: Find the winning cluster unit (j) [use formulas as in Step 4].

Step 12: Update the weights into unit z_j . For $i = 1$ to n ,

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha[x_i - v_{ij}(\text{old})]$$

Step 13: Update the weights from unit z_j to the output units. For $k = 1$ to m ,

$$w_{jk}(\text{new}) = w_{jk}(\text{old}) + \beta[y_k - w_{jk}(\text{old})]$$

Step 14: Reduce learning rate β , i.e.,

$$\beta(t+1) = 0.5\beta(t)$$

Step 15: Test the stopping condition for phase II training.

The stopping condition for both phase I and phase II training may be the reduction in learning rate or number of iterations to be performed.

5.5.3.4 Testing Algorithm

The testing algorithm used for forward-only CPN is given as follows:

Step 0: Set initial weights. (The initial weights here are the weights obtained during training.)

Step 1: Present input vector X .

Step 2: Find unit J that is closest to vector X .

Step 3: Set activations of output units:

$$y_k = w_{jk}$$

As in the case of full CPN, the forward-only CPN can also be used in the interpolation mode. Here, if more than one unit is the winner, with nonzero activation value, then

$$\sum_{j=1}^p Z_j = 1$$

Hence the activation of the output unit is given by

$$y_k = \sum_j Z_j w_{jk}$$

Use of interpolation mode results in increase of accuracy.

5.6 Adaptive Resonance Theory Network

5.6.1 Theory

The adaptive resonance theory (ART) network, developed by Steven Grossberg and Gail Carpenter (1987), is consistent with behavioral models. This is an unsupervised learning, based on competition, that finds categories autonomously and learns new categories if needed. The adaptive resonance model was developed to solve the problem of instability occurring in feed-forward systems. There are two types of ART: ART 1 and ART 2. ART 1 is designed for clustering binary vectors and ART 2 is designed to accept continuous-valued vectors. In both the nets, input patterns can be presented in any order. For each pattern, presented to the network, an appropriate cluster unit is chosen and the weights of the cluster unit are adjusted to let the cluster unit learn the pattern. This network controls the degree of similarity of the patterns placed on the same cluster units. During training, each training pattern may be presented several times. It should be noted that the input patterns should not be presented on the same cluster unit, when it is presented each time. On the basis of this, the stability of the net is defined as that wherein a pattern is not presented to previous cluster units.

The stability may be achieved by reducing the learning rates. The ability of the network to respond to a new pattern equally at any stage of learning is called as plasticity. ART nets are designed to possess the properties, stability and plasticity. The key concept of ART is that the stability plasticity can be resolved by a system in which the network includes bottom-up (input-output) competitive learning combined with top-down (output-input) learning. The instability of instar-outstar networks could be solved by reducing the learning rate gradually to zero by freezing the learned categories. But, at this point, the net may lose its plasticity or the ability to react to new data. Thus it is difficult to possess both stability and plasticity. ART networks are designed particularly to resolve the stability-plasticity dilemma, that is, they are stable to preserve significant past learning but nevertheless remain adaptable to incorporate new information whenever it appears.

5.6.1.1 Fundamental Architecture

Three groups of neurons are used to build an ART network. These include:

1. Input processing neurons (F_1 layer).

2. Clustering units (F_2 layer).
3. Control mechanism (controls degree of similarity of patterns placed on the same cluster).

The input processing neuron (F_1) layer consists of two portions: Input portion and interface portion. The input portion may perform some processing based on the inputs it receives. This is especially performed in the case of ART 2 compared to ART 1. The interface portion of the F_1 layer combines the input from input portion of F_1 and F_2 layers for comparing the similarity of the input signal with the weight vector for the cluster unit that has been selected as a unit for learning. F_1 layer input portion may be denoted as $F_1(a)$ and interface portion as $F_1(b)$.

There exist two sets of weighted interconnections for controlling the degree of similarity between the units in the interface portion and the cluster layer. The bottom-up weights are used for the connection from $F_1(b)$ layer to F_2 layer and are represented by b_{ji} (i th F_1 unit to j th F_2 unit). The top-down weights are used for the connection from F_2 layer to $F_1(b)$ layer and are represented by t_{ji} (j th F_2 unit to i th F_1 unit). The competitive layer in this case is the cluster layer and the cluster unit with largest net input is the victim to learn the input pattern, and the activations of all other F_2 units are made zero. The interface units combine the data from input and cluster layer units. On the basis of the similarity between the top-down weight vector and input vector, the cluster unit may be allowed to learn the input pattern. This decision is done by reset mechanism unit on the basis of the signals it receives from interface portion and input portion of the F_1 layer. When cluster unit is not allowed to learn, it is inhibited and a new cluster unit is selected as the victim.

5.6.1.2 Fundamental Operating Principle

In ART network, presentation of one input pattern forms a learning trial. The activations of all the units in the net are set to zero before an input pattern is presented. All units in the F_2 layer are inactive. On presentation of a pattern, the input signals are sent continuously until the learning trial is completed. There exists a user-defined parameter, called vigilance parameter, which controls the degree of similarity of the patterns assigned to the same cluster unit. The function of the reset mechanism is to control the state of each node in F_2 layer. Each unit in F_2 layer, at any time instant, can be in any one of the three states mentioned below.

1. Active: Unit is ON. The activation in this case is equal to 1. For ART 1, $d = 1$ and for ART 2, $0 < d < 1$.
2. Inactive: Unit is OFF. The activation here is zero and the unit may be available to participate in competition.
3. Inhibited: Unit is OFF. The activation here is also zero but the unit here is prevented from participating in any further competition during the presentation of current input vector.

The ART nets can perform their learning in two ways: Fast learning and slow learning. The weight updation takes place rapidly in fast learning, relative to the length of time a pattern is being presented on any particular learning trial. In fast learning, the weights reach equilibrium in each trial. On the contrary, in slow learning the weight change occurs slowly relative to the time taken for a learning trial and the weights do not reach equilibrium in each trial. More patterns have to be presented for slow learning compared to that for fast learning. For each learning trial, there occurs only minimum number of calculations in slow learning. In case of fast learning, the net is considered to be stabilized when each pattern closes its correct cluster unit.

The patterns are binary in ART 1 network, hence the weights associated with each cluster unit stabilize in the fast learning mode. The weight vectors obtained are appropriate for the type of input patterns used in ART 1. In case of ART 2 network, the weights produced by fast learning continue to change each time a pattern is presented. The net is found to stabilize only after few presentations of each training pattern. It is not easy to find equilibrium weights immediately for ART 2 as it is for ART 1. In slow learning

process, the weight changes do not reach equilibrium during any particular learning trial and more trials are required before the net stabilizes. Slow learning is generally not adopted for ART 1. For ART 2, the weights produced by slow learning are far better than those produced by fast learning for particular types of data.

5.6.1.3 Fundamental Algorithm

This algorithm discovers clusters of a set of pattern vectors. The steps involved in various stages of training algorithm are as follows:

- Step 0: Initialize the necessary parameters.
- Step 1: Perform Steps 2–9 when stopping condition is false.
- Step 2: Perform Steps 3–8 for each input vector.
- Step 3: F_1 layer processing is done.
- Step 4: Perform Steps 5–7 when reset condition is true.
- Step 5: Find the victim unit to learn the current input pattern. The victim unit is going to be the F_2 unit (that is not inhibited) with the largest input.
- Step 6: $F_1(b)$ units combine their inputs from $F_1(a)$ and F_2 .
- Step 7: Test for reset condition.
If reset is true, then the current victim unit is rejected (inhibited); go to Step 4. If reset is false, then the current victim unit is accepted for learning; go to next step (Step 8).
- Step 8: Weight updation is performed.
- Step 9: Test for stopping condition.

The ART network does not require all training patterns to be presented in the same order, it also accepts if all patterns are presented in the same order; we refer to this as an epoch. The flowchart showing the flow of training process is depicted separately for ART 1 and ART 2.

5.6.2 Adaptive Resonance Theory 1

Adaptive resonance theory 1 (ART 1) network is designed for binary input vectors. As discussed generally, the ART 1 net consists of two fields of units—input unit (F_1 unit) and output unit (F_2 unit)—along with the reset control unit for controlling the degree of similarity of patterns placed on the same cluster unit. There exist two sets of weighted interconnection path between F_1 and F_2 layers. The supplemental unit present in the net provides the efficient neural control of the learning process. Carpenter and Grossberg have designed ART 1 network as a real-time system. In ART 1 network, it is not necessary to present an input pattern in a particular order; it can be presented in any order. ART 1 network can be practically implemented by analog circuits governing the differential equations, i.e., the bottom-up and top-down weights are controlled by differential equations. ART 1 network runs throughout autonomously. It does not require any external control signals and can run stably with infinite patterns of input data.

ART 1 network is trained using fast learning method, in which the weights reach equilibrium during each learning trial. During this resonance phase, the activations of F_1 units do not change; hence the equilibrium weights can be determined exactly. The ART 1 network performs well with perfect binary input patterns, but it is sensitive to noise in the input data. Hence care should be taken to handle the noise.

Suppose input unit is inhibited
→ new unit

5.6.2.1 Architecture

The ART 1 network is made up of two units:

1. Computational units.
2. Supplemental units.

In this section we will discuss in detail about these two units.

Computational units

The computational unit for ART 1 consists of the following:

1. Input units (F_1 unit – both input portion and interface portion).
2. Cluster units (F_2 unit – output unit).
3. Reset control unit (controls degree of similarity of patterns placed on same cluster).

The basic architecture of ART 1 (computational unit) is shown in Figure 5-22. Here each unit present in the input portion of F_1 layer (i.e., $F_1(a)$ layer unit) is connected to the respective unit in the interface portion of F_1 layer (i.e., $F_1(b)$ layer unit). Reset control unit has connections from each of $F_1(a)$ and $F_1(b)$ units. Also, each unit in $F_1(b)$ layer is connected through two weighted interconnection paths to each unit in F_2 layer and the reset control unit is connected to every F_2 unit. The X_j unit of $F_1(b)$ layer is connected to Y_j unit of F_2 layer through bottom-up weights (b_{ij}) and the Y_j unit of F_2 is connected to X_j unit of F_1 through top-down weights (t_{ji}). Thus ART 1 includes a bottom-up competitive learning system combined with a top-down outstar learning system. In Figure 5-22 for simplicity only the weighted interconnections b_{ij} and t_{ji} are shown, the other units' weighted interconnections are in a similar way. The cluster layer (F_2 layer) unit is a competitive layer, where only the uninhibited node with the largest net input has nonzero activation.

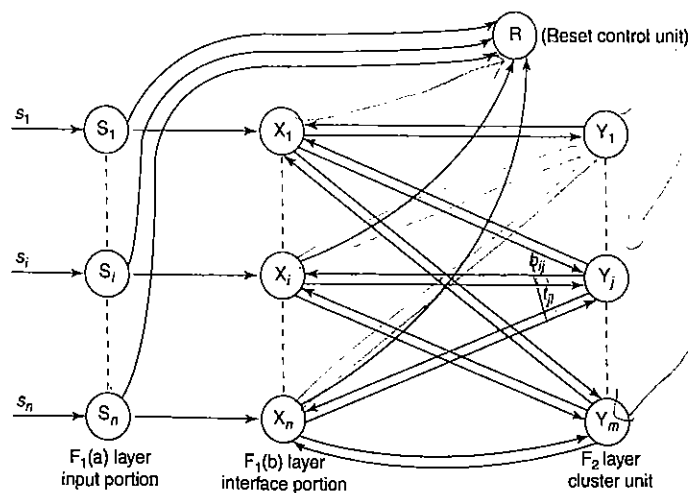


Figure 5-22 Basic architecture of ART 1.

Supplemental units

Figure 5-23 shows the supplemental unit interconnection involving two gain control units along with one reset unit. The discussion on supplemental units is important based on theoretical point of view.

Difficulty faced by computational units: It is necessary for these units to respond differently at different stages of the process, and these are not supported by any of the biological neuron to decide what to do when. The other difficulty is that the operation of the reset mechanism is not well defined for its implementation in neural systems.

The above difficulties are rectified by the introduction of two supplemental units (called as gain control units) G_1 and G_2 , along with the reset control unit R . These three units receive signals from and send signals to all of the units in input layer and cluster layer. In Figure 5-23, the excitatory weighted signals are denoted by "+" and inhibitory signals are indicated by "-". Whenever any unit in designated layer is "on," a signal is sent. $F_1(b)$ unit and F_2 unit receive signal from three sources. $F_1(b)$ unit can receive signal from either $F_1(a)$ unit or F_2 units or G_1 unit. In the similar way, F_2 unit receives signal from either $F_1(b)$ unit or reset control unit R or gain control unit G_2 . An $F_1(b)$ unit or F_2 unit should receive two excitatory signals for them to be on. Both $F_1(b)$ unit and F_2 unit can receive signals through three possible ways; this is called as two-thirds rule. The $F_1(b)$ unit should send a signal whenever it receives input from $F_1(a)$ and no F_2 node is active. After an F_2 node has been chosen in competition, it is necessary that only $F_1(b)$ units whose input signal and top-down signal match remain constant. This is performed by the two gain control units G_1 and G_2 , in addition with two-thirds rule. Whenever F_2 unit is on, G_1 unit is inhibited. When no F_2 unit is on, each F_1 interface unit receives a signal from G_1 unit; here, all of the units that receive a positive input signal from the input vector presented fire. In the same way, G_2 unit controls the firing of F_2 units, obeying the two thirds rule. The choice of parameters and initial weights may also be based on two-thirds rule. On the other hand, the vigilance matching is controlled by the reset control unit R . An excitatory signal is always sent to R when any unit in $F_1(a)$ layer is on. The strength of the signal depends on how many F_1 (input) units are on. It should be noted that the reset control unit R also receives inhibitory signals from the F_1 interface units that are on. If sufficient number of interface units is on, then unit " R " may be prevented from firing. When unit " R " fires, it will inhibit any F_2 unit that is on. This may force the F_2 layer to choose a new winning node.

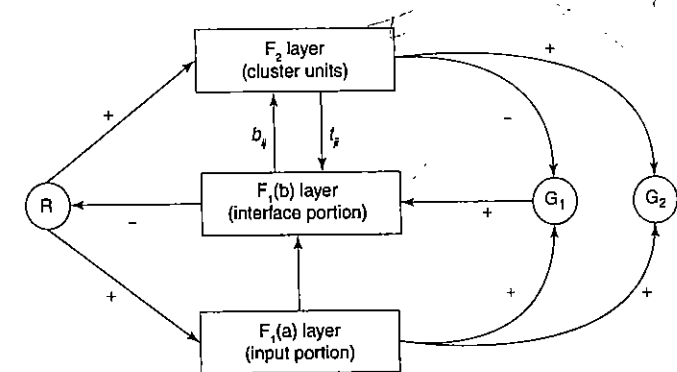


Figure 5-23 Supplemental unit of ART 1.

5.6.2.2 Flowchart of Training Process

The flowchart for the training process of ART 1 network is shown in Figure ?? . The parameters used in flowchart and training algorithm are as follows:

n = number of components in training input vector

m = maximum number of cluster units that can be formed

ρ = vigilance parameter (0 to 1)

b_{ij} = bottom-up weights (weights from X_i unit of F_1 (b) layer to Y_j unit of F_2 layer)

t_{ji} = top-down weights (weights from Y_j units of F_2 layer to X_i unit of F_1 (b) layer)

s = binary input vector

x = activation vector for F_1 (b) layer

$\|x\|$ = norm of vector x that is defined as the sum of components of x_i ($i = 1$ to n)

Initially, binary input vector " s " is presented in the F_1 (a) layer. Then the signals are sent to the corresponding X layer, i.e., F_1 (b) layer. Each F_1 (b) layer sends the activation to the F_2 layer over the weighted interconnection paths. Each F_2 layer unit then calculates the net input. The unit with the largest net input is selected as the winner and will have activation "1," the other units' activation will be 0. The winning unit is specified by its index " j ." Only this winner unit can learn the current input pattern. Then the signal is sent from F_2 layer to F_1 (b) layer over the top-down weights (i.e., signals get multiplied with top-down weights). The X units present in the interface portion F_1 (b) layer remain on, only if they receive a nonzero signal from both F_1 (a) and F_2 layer units.

Now we calculate the factor $\|x\|$. The norm of vector x gives the number of components in which the top-down weight vector for the winning F_2 unit t_{ji} and input vector s are both 1. This is called Match. The ratio of norm of x , $\|x\|$, to norm of s , $\|s\|$, is called Match Ratio, which if greater than or equal to vigilance parameter, then both the top-down and bottom-up weights have to be adjusted. This is called reset condition. That is

- If $\|x\|/\|s\| \geq \rho$, then weight updation is done. This testing condition is called reset condition.
- If $\|x\|/\|s\| < \rho$, then current unit is rejected and another unit should be chosen. The current winning cluster unit becomes inhibited, so this unit again cannot be chosen as a unit, on this particular learning trial, and the activations of the F_1 units are reset to zero.

This process is repeated until a satisfactory match is found (units get accepted) or until all the units are inhibited.

5.6.2.3 Training Algorithm

The training algorithm for ART 1 network is shown below.

Step 0: Initialize the parameters:

$$\alpha > 1 \text{ and } 0 < \rho \leq 1$$

Initialize the weights:

$$0 < b_{ij}(0) < \frac{\alpha}{\alpha - 1 + n} \text{ and } t_{ji}(0) = 1$$

Step 1: Perform Steps 2–13 when stopping condition is false.

Step 2: Perform Steps 3–12 for each of the training input.

Step 3: Set activations of all F_2 units to zero. Set the activations of F_1 (a) units to input vectors.

Step 4: Calculate the norm of s :

$$\|s\| = \sum_i s_i$$

Step 5: Send input signal from F_1 (a) layer to F_1 (b) layer:

$$x_i = s_i$$

Step 6: For each F_2 node that is not inhibited, the following rule should hold: If $y_j \neq -1$, then

$$y_j = \sum_i b_{ij} x_i$$

Step 7: Perform Steps 8–11 when reset is true.

Step 8: Find J for $y_j \geq y_i$ for all nodes j . If $y_j = -1$, then all the nodes are inhibited and note that this pattern cannot be clustered.

Step 9: Recalculate activation X of F_1 (b):

$$x_i = s_i t_{ji}$$

Step 10: Calculate the norm of vector x :

$$\|x\| = \sum_i x_i$$

Step 11: Test for reset condition.

If $\|x\|/\|s\| < \rho$, then inhibit node J , $y_j = -1$. Go back to step 7 again.

Else if $\|x\|/\|s\| \geq \rho$, then proceed to the next step (Step 12).

Step 12: Perform weight updation for node J (fast learning):

$$b_{ij}(\text{new}) = \frac{\alpha x_i}{\alpha - 1 + \|x\|}$$

$$t_{ji}(\text{new}) = x_i$$

Step 13: Test for stopping condition. The following may be the stopping conditions:

- No change in weights.
- No reset of units.
- Maximum number of epochs reached.

When calculating the winner unit, if there occurs a tie, the unit with smallest index is chosen as winner. Note that in Step 3 all the inhibitions obtained from the previous learning trial are removed. When $y_j = -1$, the node is inhibited and it will be prevented from becoming the winner. The unit x_i in Step 9 will be ON only if it receives both an external signal s_i and the other signal from F_2 unit to F_1 (b) unit, t_{ji} . Note that t_{ji} is either 0 or 1, and once it is set to 0, during learning, it can never be set back to 1 (provides stable learning method).

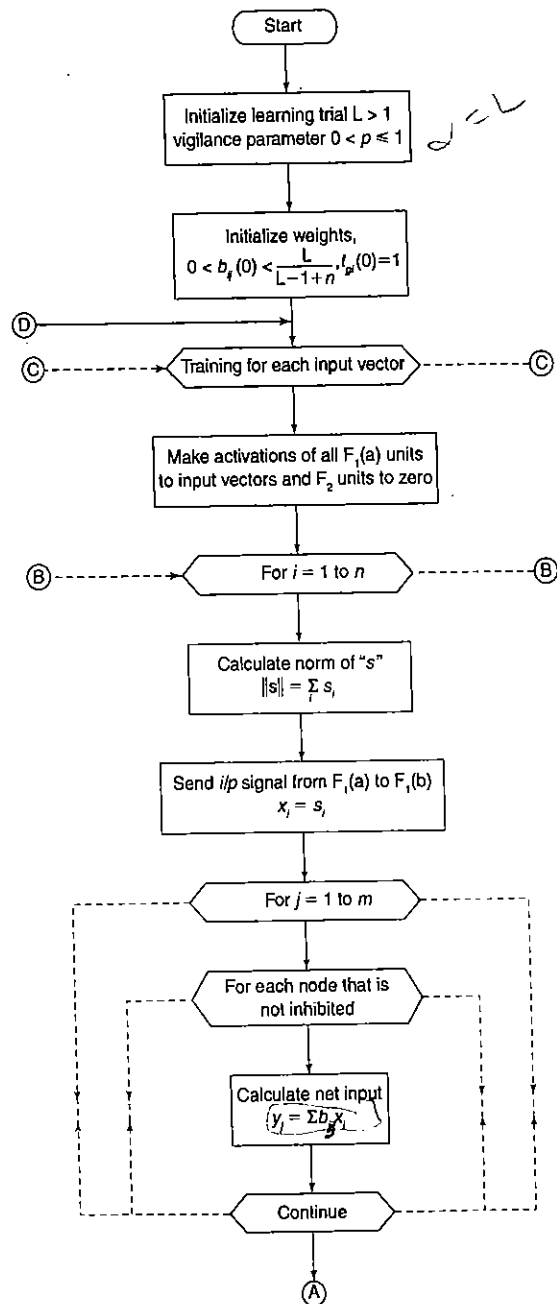


Figure 5-24 Flowchart for training of ART 1 network.

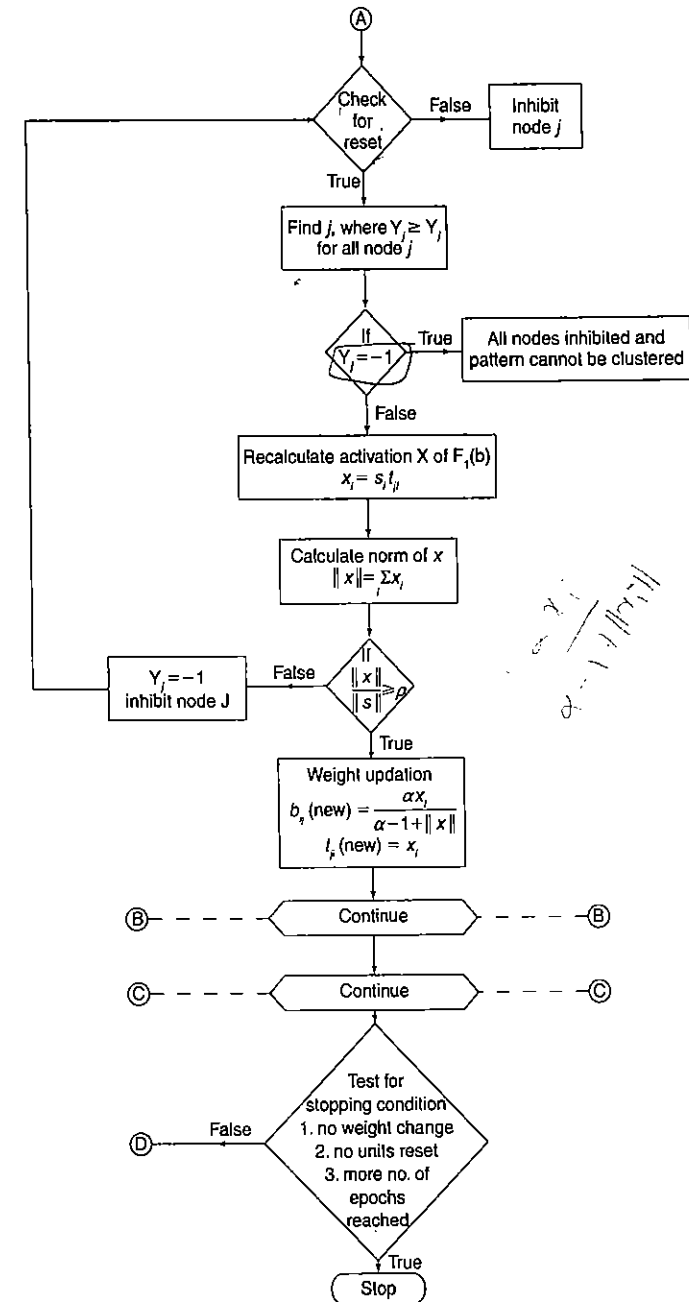


Figure 5-24 (continued).

Supplemental connection between W_i and X_i units

As discussed in Section 5.6.3.1, there exist supplemental connections between W and X , U and V , and P and Q . Each of the x_i receives signal from w_i units. After receiving, it will calculate the norm of w , $\|w\|$ and then sends that signal to each of the X units. Normalization is done in the F_1 units from W to X , V to U and P to Q . Each of the X_i units are connected to V_i and Q_i units are also connected to V_i . The weights a , b , c shown in Figure 5-25 are fixed. The weights on the connection path indicate the transformation taking place from one unit to other (no multiplication takes place here), i.e., u_i is transformed to au_i but not multiplied. When signals are transferred from F_1 units to F_2 units, i.e., from P_i to Y_j , the multiplication of weights is done. The activation of the F_2 unit is " d " which ranges between 0 and 1 ($0 < d < 1$). It should be noted that these activations are continuously changing.

Processing of F_1 layer and F_2 layer

For understanding the training algorithm of ART 2 network, it is important to know the processing of F_1 and F_2 layers. In F_1 layer, the output activation from P_i is p and output activation from Q_j is q . The activation vector q , which is the activation of Q_j units, should be equal to vector p , activation of P_i units that is normalized approximately for unit length. U_j unit performs similar process of $F_1(a)$ layer of ART 1 and P_i unit performs similar process of $F_1(b)$ layer of ART 1 network. The activation function used here is the functional representation of noise suppression parameter " Q " and is given by

$$f(x) = \begin{cases} x & x \geq \theta \\ 0 & x < \theta \end{cases}$$

The noise suppression parameter Q is defined by the user and is used to achieve stability. Stability occurs where there is no reset, i.e., the same winner unit is chosen in the next trial also. Units x_i and Q_j apply activation to V_i , which suppresses the components to achieve stability. Hence Q is used here.

In ART 2 network continuous processing of the input units is done. The continuous-valued input signals $s = (s_1, \dots, s_n)$ are sent continuously. For each learning trial, one input pattern is presented. At the beginning of training, the activations are set to zero, i.e., inactive not inhibit. The computation cycle for a particular learning trial within F_1 layers starts with u_i which is equal to activation of V_i approximated to unit length. Unit u_i is given by

$$u_i = \frac{v_i}{\epsilon + \|v\|}$$

where " ϵ " is a small parameter for preventing the division by zero when $\|v\|$ becomes zero. Also q_i and x_i are given by

$$q_i = \frac{p_i}{\epsilon + \|p\|}; \quad x_i = \frac{w_i}{\epsilon + \|w\|}$$

The noise suppression parameter is applied only to x_i and q_i .

The signal will be sent from each unit of u_i to w_i and p_i . The activations of units w_i and p_i have to be done. The activation of w_i is the sum of input signal received (s_i) and au_i :

$$w_i = s_i + au_i$$

P_i is found to receive signals from u_i and top-down weights, i.e., sums u_i (activation of u_i) and top-down weight (t_{ji}), and is given by

$$P_i = u_i + dt_{ji}$$

where d is the activation of winning F_2 unit. Before entering into V_i , activation function is applied to each of x_i and Q_j units. Unit V_i sums the signals from x_i and Q_j which receive signal concurrently:

$$v_i = f(x_i) + bf(q_i)$$

Activation function is designed to select the noise suppression parameter (user specified " Q "). According to Carpenter and Grossberg, the activations of P_i and Q_j (i.e., the outputs) will reach equilibrium (stable set of weights) only after two updates of weights. This completes the phase I or one-cycle process of F_1 layer. Only after F_1 units reach equilibrium, the processing of F_2 layer starts (i.e., after three updates).

F_2 layer being a competitive layer uses winner-take-all policy to determine its winner. Dot product method may be used for the selection of the winner. When the top-down and weight vector remain similar, then that unit is the winner (active). If for a unit, the top-down and input vectors are not similar, then that unit becomes inhibit. This layer receives signals from P_i units via bottom-up weights and P_i units in turn send signals to F_2 unit. Here only the winner unit is allowed to learn the input pattern S_i .

The reset mechanism controls the degree of similarity of the input patterns. The checking for reset condition in ART 2 differs from ART 1 network. The reset is checked every time it receives signal from u_i and P_i .

In fast learning mode, the updation of weights is continued until the weights reach equilibrium on each trial. It requires only less number of epochs, but a large number of iterations through the weight update- F_1 portion must be performed on each learning trial. Here, the placement of patterns on clusters stabilizes, but the weight will change for each pattern presented.

In slow learning mode, only one iteration of weight updates will be performed on each learning trial. Large number of learning trials is required for each pattern, but only little computation is done on each trial. There is no requirement that the patterns should be presented in the same order or that exactly the same set of patterns is presented on each cycle through them. Thus it is preferable to have slow learning than fast learning.

Computations for algorithm

The following computations have to be performed in several steps of the algorithm and are referred as "updation of F_1 activations." Unit J is the winning F_2 unit after competition is completed. If no winning unit is chosen, then " d " is zero for all units. The calculations for P_i and w_i , and x_i and q_i can be done in parallel.

F_1 layer consists of six units; the update F_1 activations are given by

$$u_i = \frac{v_i}{\epsilon + \|v\|}; \quad P_i = u_i + dt_{ji}$$

$$w_i = s_i + au_i; \quad x_i = \frac{w_i}{\epsilon + \|w\|}$$

$$q_i = \frac{p_i}{\epsilon + \|p\|}; \quad v_i = f(x_i) + bf(q_i)$$

The activation function is given by

$$f(x) = \begin{cases} x & f(x) \geq \theta \\ 0 & f(x) < \theta \end{cases}$$

5.6.3.3 Flowchart

The flowchart for the training process of ART 2 network is shown in Figure 5-27. The flowchart clearly depicts the flow of the training process of the network. The check for reset in the flowchart differs for ART 1 and ART 2 networks.

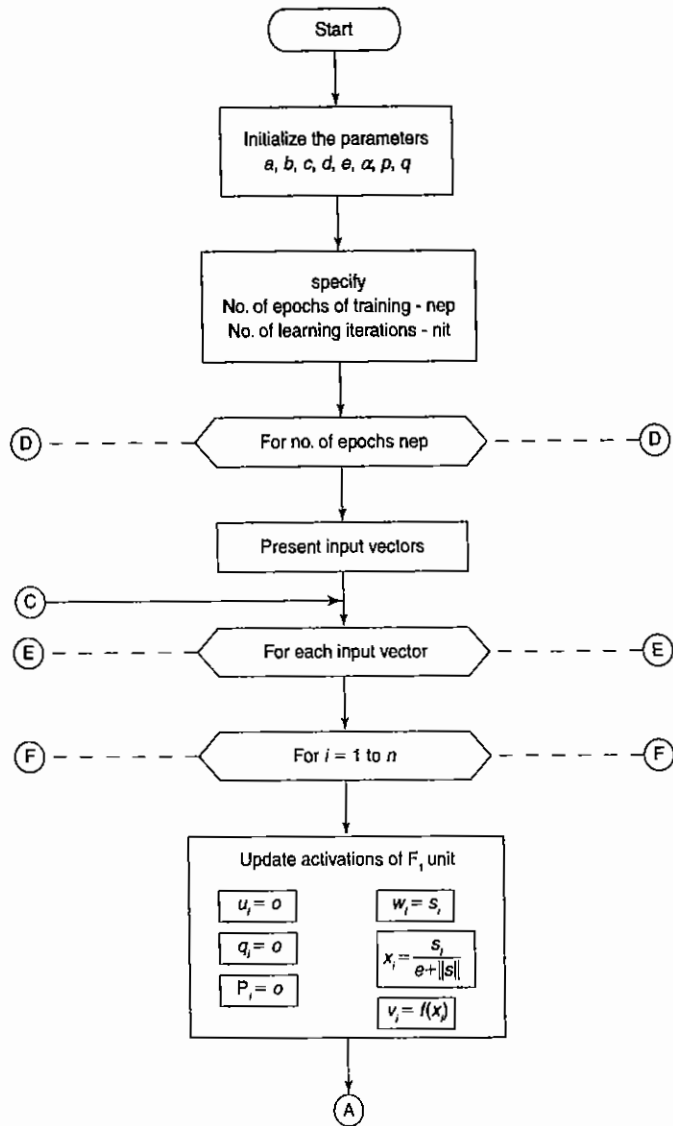


Figure 5-27 Flowchart for training of ART 2 network.

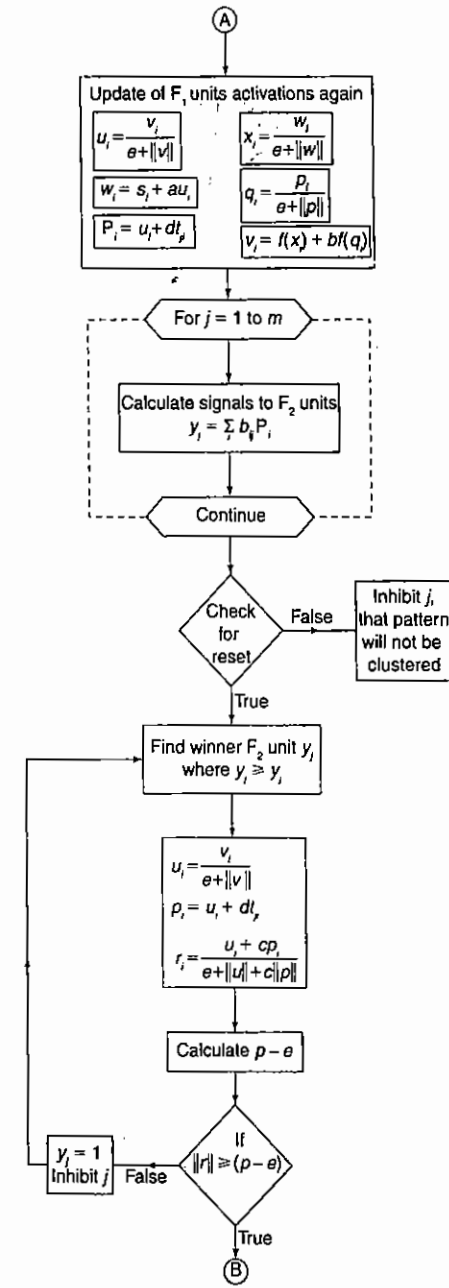


Figure 5-27 (continued).

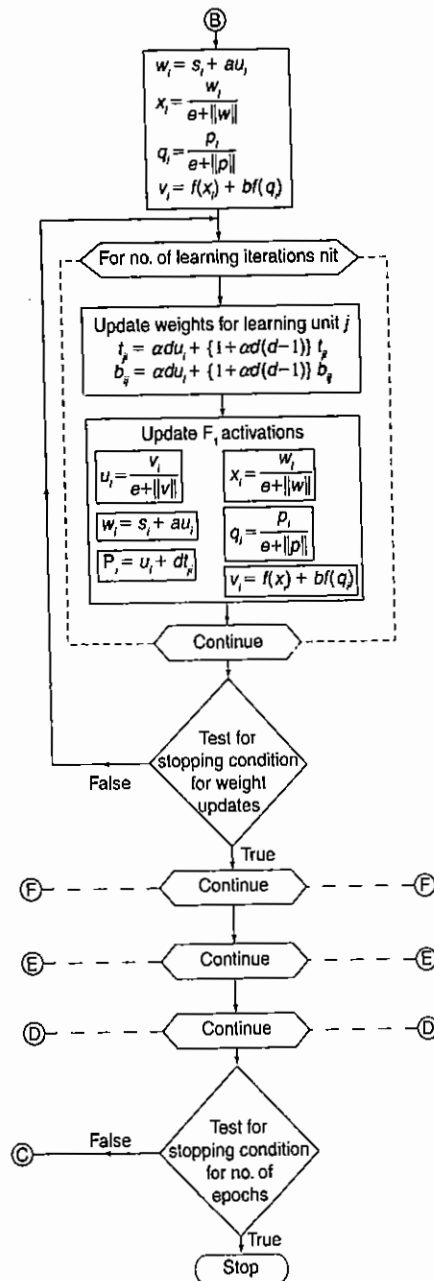


Figure 5-27 (continued).

5.6.3.4 Training Algorithm

The training algorithm of ART 2 network is shown below.

Step 0: Initialize the following parameters: $a, b, c, d, e, \alpha, \rho, \theta$. Also, specify the number of epochs of training (nep) and number of learning iterations (nit).

Step 1: Perform Steps 2–12 (nep) times.

Step 2: Perform Steps 3–11 for each input vector s .

Step 3: Update F_1 unit activations:

$$u_i = 0; \quad w_i = s_i; \quad P_i = 0; \quad q_i = 0; \quad v_i = f(x_i);$$

$$x_i = \frac{s_i}{e + \|s\|}$$

Update F_1 unit activations again:

$$u_i = \frac{v_i}{e + \|v\|}; \quad w_i = s_i + a u_i;$$

$$P_i = u_i; \quad x_i = \frac{w_i}{e + \|w\|};$$

$$q_i = \frac{P_i}{e + \|P\|}; \quad v_i = f(x_i) + b f(q_i)$$

In ART 2 networks, norms are calculated as the square root of the sum of the squares of the respective values.

Step 4: Calculate signals to F_2 units:

$$y_j = \sum_{i=1}^n b_{ij} p_i$$

Step 5: Perform Steps 6 and 7 when reset is true.

Step 6: Find F_2 unit Y_j with largest signal (J is defined such that $y_j \geq y_j, j = 1$ to m).

Step 7: Check for reset:

$$u_i = \frac{v_i}{e + \|v\|}; \quad P_i = u_i + d t_{ji}; \quad r_i = \frac{u_i + c P_i}{e + \|u\| + c \|P\|}$$

If $\|r\| < (\rho - e)$, then $y_j = -1$ (inhibit J). Reset is true; perform Step 5.

If $\|r\| \geq (\rho - e)$, then

$$w_i = s_i + a u_i; \quad x_i = \frac{w_i}{e + \|w\|};$$

$$q_i = \frac{P_i}{e + \|P\|}; \quad v_i = f(x_i) + b f(q_i)$$

Reset is false. Proceed to Step 8.

Step 8: Perform Steps 9–11 for specified number of learning iterations.

Step 9: Update the weights for winning unit j :

$$t_{ji} = \alpha d u_i + \{[1 + \alpha d(d-1)]\} t_{ji}$$

$$b_{ij} = \alpha d u_i + \{[1 + \alpha d(d-1)]\} b_{ij}$$

Step 10: Update F_1 activations:

$$u_i = \frac{v_i}{e + \|v\|}; \quad w_i = s_i + \alpha u_i;$$

$$p_i = u_i + d t_{ji}; \quad x_i = \frac{w_i}{e + \|w\|};$$

$$q_i = \frac{p_i}{e + \|p\|}; \quad v_i = f(x_i) + b f(q_i)$$

Step 11: Check for the stopping condition of weight updation.

Step 12: Check for the stopping condition for number of epochs.

In the above algorithm, at resonance period, reset will not occur and new winning unit cannot be chosen. Since in slow learning number of learning iterations is 1, Step 10 in training algorithm need not be processed. Perform Step 8 until the weight changes are below some specified tolerance. If slow learning is performed, then repeat Step 1 until the weight changes are below some specified tolerance. If fast learning is adopted, then repeat Step 1 until the patterns placement on the cluster units do not change from one epoch to the next.

5.6.3.5 Sample Values of Parameter

The sample values of the parameters used in ART 2 network and their role in effective training process are mentioned below.

- n = number of F_1 layer input units
- m = number of F_2 layer cluster units
- a, b = fixed weights present in the F_1 layer. The sample values are $a = 10$ and $b = 10$; when $a = 0$ and $b = 0$, the net becomes instable
- c = fixed weight for testing of reset. The sample value is $c = 0.1$. For small c , larger effective range of the vigilance parameter is achieved
- d = activation of winning F_2 unit. Sample value is $d = 0.9$. The values of c and d should be selected satisfying the inequality $cd/1 - d \leq 1$. The value of $cd/1 - d$ should be closer to 1, so that effective vigilance could be achieved
- e = a small parameter included to prevent division by zero error when the norm of vector is zero.
- θ = noise suppression parameter. A sample value of noise suppression parameter is $\theta = 1/\sqrt{n}$. The components of the normalized input vector, which are less than this value, are set to zero.
- α = learning rate parameter. In both slow and fast learning methods, a small value of α slows down the learning process.
- ρ = vigilance parameter used in reset condition. Vigilance parameter can range from 0 to 1. For effectively controlling the number of clusters, a sample value of 0.7–1 may be allowed. The range of ρ may also be affected by the values of c and d .
- $t_{ji}(0)$ = initial top-down weights. The initial weights of this weight vectors are given by $t_{ji}(0) = 0$.

$b_{ij}(0)$ = initial bottom-up weights. These should be chosen to satisfy the inequality $b_{ij}(0) \leq 1/(1-d)\sqrt{n}$. High values of b_{ij} allow the net to form more clusters.

On the basis of all these roles of parameters and their sample values, care should be taken in selecting their values for effective training of the network.

5.7 Summary

Unsupervised learning networks are widely used when clustering of units is performed. In case of unsupervised learning networks, the information about the output is not known; when the weights of a net remain fixed for the entire operation then it results in fixed weight competitive nets. Fixed weight competitive nets include Maxnet, Mexican hat and Hamming net. In case of Hamming net, Maxnet is used as a subnet. The most important unsupervised learning network is the Kohonen self-organizing feature map, where clustering is performed over the training vectors and the network training is achieved. An extension of KSOFM, Kohonen self-organizing motor map is also included. An unsupervised learning network with targets known is the learning vector quantization (LVQ) network. A study is made on LVQ net with its architecture, flowchart for training process and training algorithm. The variants of LVQ net are also included. The compression network discussed in this chapter is the counterpropagation network (CPN). The two types of counterpropagation networks – full CPN and forward-only CPN – are discussed. The testing algorithms for these networks are also given. Another important unsupervised learning network is the adaptive resonance theory (ART) network. In this chapter, ART 1 and ART 2 networks with all relevant information are discussed in detail.

5.8 Solved Problems

- Construct a Maxnet with four neurons and inhibitory weight $\epsilon = 0.2$, given the initial activations (input signals) as follows:

$$a_1(0) = 0.3; \quad a_2(0) = 0.5; \quad a_3(0) = 0.7;$$

$$a_4(0) = 0.9$$

Solution: Update the activations for each node, i.e.,

$$a_j(\text{new}) = f \left[a_j(\text{old}) - \epsilon \sum_{k \neq j} a_k(\text{old}) \right]$$

The activation function is given by

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

First iteration:

$$a_1(1) = f \left[a_1(0) - \epsilon \sum_{k \neq 1} a_k(0) \right]$$

$$= f[0.3 - 0.2(0.5 + 0.7 + 0.9)]$$

$$= f(0.3 - 0.42) = f(-0.12) = 0$$

$$a_2(1) = f \left[a_2(0) - \epsilon \sum_{k \neq 2} a_k(0) \right]$$

$$= f[0.5 - 0.2(0.3 + 0.7 + 0.9)]$$

$$= f(0.12) = 0.12$$

$$a_3(1) = f \left[a_3(0) - \epsilon \sum_{k \neq 3} a_k(0) \right]$$

$$= f[0.7 - 0.2(0.3 + 0.5 + 0.9)]$$

$$= f(0.36) = 0.36$$

$$a_4(1) = f \left[a_4(0) - \epsilon \sum_{k \neq 4} a_k(0) \right]$$

$$= f[0.9 - 0.2(0.3 + 0.5 + 0.7)]$$

$$= f(0.9 - 0.3) = f(0.6) = 0.6$$

Second iteration:

$$\begin{aligned}
 a_1(2) &= f \left[a_1(1) - \varepsilon \sum_{k \neq j} a_k(1) \right] \\
 &= f[0 - 0.2(0.12 + 0.36 + 0.6)] \\
 &= f(0 - 0.216) = 0 \\
 a_2(2) &= f \left[a_2(1) - \varepsilon \sum_{k \neq j} a_k(1) \right] \\
 &= f[0.12 - 0.2(0 + 0.36 + 0.6)] \\
 &= f(-0.072) = 0 \\
 a_3(2) &= f \left[a_3(1) - \varepsilon \sum_{k \neq j} a_k(1) \right] \\
 &= f[0.36 - 0.2(0 + 0.12 + 0.6)] \\
 &= f(0.216) = 0.216 \\
 a_4(2) &= f \left[a_4(1) - \varepsilon \sum_{k \neq j} a_k(1) \right] \\
 &= f[0.6 - 0.2(0 + 0.12 + 0.36)] \\
 &= f(0.504) = 0.504
 \end{aligned}$$

Third iteration:

$$\begin{aligned}
 a_1(3) &= f \left[a_1(2) - \varepsilon \sum_{k \neq j} a_k(2) \right] \\
 &= f[0 - 0.2(0 + 0.216 + 0.504)] \\
 &= f(0 - 0.144) = 0 \\
 a_2(3) &= f \left[a_2(2) - \varepsilon \sum_{k \neq j} a_k(2) \right] \\
 &= f[0 - 0.2(0 + 0.216 + 0.504)] \\
 &= f(0 - 0.144) = 0 \\
 a_3(3) &= f \left[a_3(2) - \varepsilon \sum_{k \neq j} a_k(2) \right] \\
 &= f[0.216 - 0.2(0 + 0 + 0.504)] \\
 &= f(0.1152) = 0.1152 \\
 a_4(3) &= f \left[a_4(2) - \varepsilon \sum_{k \neq j} a_k(2) \right] \\
 &= f[0.504 - 0.2(0 + 0 + 0.216)] \\
 &= f(0.4608) = 0.4608
 \end{aligned}$$

Fourth iteration:

$$\begin{aligned}
 a_1(4) &= f \left[a_1(3) - \varepsilon \sum_{k \neq j} a_k(3) \right] \\
 &= f[0 - 0.2(0 + 0.1152 + 0.4608)] = 0 \\
 a_2(4) &= f \left[a_2(3) - \varepsilon \sum_{k \neq j} a_k(3) \right] \\
 &= f[0 - 0.2(0 + 0.1152 + 0.4608)] = 0 \\
 a_3(4) &= f \left[a_3(3) - \varepsilon \sum_{k \neq j} a_k(3) \right] \\
 &= f[0.1152 - 0.2(0 + 0 + 0.4608)] \\
 &= f(0.02304) = 0.02304 \\
 a_4(4) &= f \left[a_4(3) - \varepsilon \sum_{k \neq j} a_k(3) \right] \\
 &= f[0.4608 - 0.2(0 + 0 + 0.1152)] \\
 &= f(0.43776) = 0.43776
 \end{aligned}$$

Fifth iteration:

$$\begin{aligned}
 a_1(5) &= f \left[a_1(4) - \varepsilon \sum_{k \neq j} a_k(4) \right] \\
 &= f[0 - 0.2(0 + 0.02304 + 0.43776)] = 0 \\
 a_2(5) &= f \left[a_2(4) - \varepsilon \sum_{k \neq j} a_k(4) \right] \\
 &= f[0 - 0.2(0 + 0.02304 + 0.43776)] = 0 \\
 a_3(5) &= f \left[a_3(4) - \varepsilon \sum_{k \neq j} a_k(4) \right] \\
 &= f[0.02304 - 0.2(0 + 0 + 0.43776)] \\
 &= f(-0.0645) = 0 \\
 a_4(5) &= f \left[a_4(4) - \varepsilon \sum_{k \neq j} a_k(4) \right] \\
 &= f[0.43776 - 0.2(0 + 0 + 0.02304)] \\
 &= f(0.433152) = 0.433152
 \end{aligned}$$

Even though if further iterations are made, the value of $a_4(5)$ remains the same, since all the other values $a_1(5)$, $a_2(5)$ and $a_3(5)$ are equal to zero. Thus the convergence has occurred.

2. Construct and test the Hamming network to cluster four vectors. Given the exemplar vectors,

$$e(1) = [1 \ -1 \ -1 \ -1]; \quad e(2) = [-1 \ -1 \ -1 \ 1]$$

the bipolar input vectors are $x_1 = [-1 \ -1 \ 1 \ -1]$; $x_2 = [-1 \ -1 \ 1 \ 1]$; $x_3 = [-1 \ -1 \ -1 \ 1]$; $x_4 = [1 \ 1 \ -1 \ -1]$.

Solution: Number of components in input vector $n = 4$.

Number of exemplar vectors $m = 2$.

Setting the initial weights to $1/2$ of the exemplar vectors, we get

$$w_{ij} = \frac{e_{ij}}{2} = \begin{pmatrix} \frac{e(1)}{2} & \frac{e(2)}{2} \end{pmatrix}$$

where

$$e(1) = [1 \ -1 \ -1 \ -1]; \quad e(2) = [-1 \ -1 \ -1 \ 1]$$

Step 0: The weights are given by

$$w_{ij} = \begin{bmatrix} 0.5 & -0.5 \\ -0.5 & -0.5 \\ -0.5 & -0.5 \\ -0.5 & 0.5 \end{bmatrix}$$

Setting the bias to $n/2$, we obtain

$$b_1 = b_2 = \frac{n}{2} = \frac{4}{2} = 2$$

First input vector:

Step 1: For $x_1 = [-1 \ -1 \ 1 \ -1]$, perform Steps 2-4.

Step 2:

$$y_{in1} = b_1 + \sum_i x_i w_{i1}$$

$$\begin{aligned}
 &= 2 + [-1 \ -1 \ 1 \ -1] \begin{bmatrix} 0.5 \\ -0.5 \\ -0.5 \\ -0.5 \end{bmatrix} \\
 &= 2 - 0.5 + 0.5 - 0.5 + 0.5 = 2
 \end{aligned}$$

The value $y_{in1} = 2$ is the number of components at which the input vector x_1 and $e(1)$ agree. Now

$$\begin{aligned}
 y_{in2} &= b_2 + \sum_i x_i w_{i2} \\
 &= 2 + [-1 \ -1 \ 1 \ -1] \begin{bmatrix} -0.5 \\ -0.5 \\ -0.5 \\ +0.5 \end{bmatrix} \\
 &= 2 + 0.5 + 0.5 - 0.5 - 0.5 = 2
 \end{aligned}$$

The value $y_{in2} = 2$ is the number of components at which the input vector x_1 and $e(2)$ agree.

Step 3: Initialize the activations of Maxnet as

$$y_1(0) = 2; \quad y_2(0) = 2$$

Step 4: Since $y_1(0) = y_2(0)$, Maxnet will find the unit with the smallest index as the best match exemplar for input $x_1 = [-1 \ -1 \ 1 \ -1]$ or in some cases both may be chosen as best match exemplars.

Second input vector:

Step 1: For $x_2 = [-1 \ -1 \ 1 \ 1]$, perform Steps 2-4.

Step 2:

$$\begin{aligned}
 y_{in1} &= b_1 + \sum_i x_i w_{i1} \\
 &= 2 + [-1 \ -1 \ 1 \ 1] \begin{bmatrix} +0.5 \\ -0.5 \\ -0.5 \\ -0.5 \end{bmatrix} \\
 &= 2 - 0.5 + 0.5 - 0.5 - 0.5 = 1
 \end{aligned}$$

$$y_{m2} = b_2 + \sum_i x_i w_{i2}$$

$$= 2 + [-1 \ -1 \ 1 \ 1] \begin{bmatrix} -0.5 \\ -0.5 \\ -0.5 \\ 0.5 \end{bmatrix}$$

$$= (2 + 0.5 - 0.5 + 0.5 + 0.5) = 3$$

Step 3: Initialize the activations of Maxnet as

$$y_1(0) = 1; \quad y_2(0) = 3$$

Step 4: Since $y_2(0) > y_1(0)$, Maxnet will find that the unit y_2 has the best match exemplar for input vector $x_2 = [-1 \ -1 \ 1 \ 1]$.

Third input vector:

Step 1: For $x_3 = [-1 \ -1 \ -1 \ 1]$, perform Steps 2-4.

Step 2:

$$y_{m1} = b_1 + \sum_i x_i w_{i1}$$

$$= 2 + [-1 \ -1 \ -1 \ 1] \begin{bmatrix} 0.5 \\ -0.5 \\ -0.5 \\ -0.5 \end{bmatrix}$$

$$= 2 - 0.5 + 0.5 + 0.5 - 0.5 = 2$$

$$y_{m2} = b_2 + \sum_i x_i w_{i2}$$

$$= 2 + [-1 \ -1 \ -1 \ 1] \begin{bmatrix} -0.5 \\ -0.5 \\ -0.5 \\ 0.5 \end{bmatrix}$$

$$= 2 + 0.5 + 0.5 + 0.5 + 0.5 = 4$$

Step 3: Initialize the activations of Maxnet as

$$y_1(0) = 2; \quad y_2(0) = 4$$

Step 4: Since $y_2(0) > y_1(0)$, Maxnet will find the unit y_2 as the best match exemplar for input vector $x_3 = [-1 \ -1 \ -1 \ 1]$.

Fourth input vector:

Step 1: For $x_4 = [1 \ 1 \ -1 \ -1]$, perform Steps 2-4.

Step 2:

$$y_{m1} = b_1 + \sum_i x_i w_{i1}$$

$$= 2 + [1 \ 1 \ -1 \ -1] \begin{bmatrix} 0.5 \\ -0.5 \\ -0.5 \\ -0.5 \end{bmatrix}$$

$$= 2 + 0.5 - 0.5 + 0.5 + 0.5 = 3$$

$$y_{m2} = b_2 + \sum_i x_i w_{i2}$$

$$= 2 + [1 \ 1 \ -1 \ -1] \begin{bmatrix} -0.5 \\ -0.5 \\ -0.5 \\ 0.5 \end{bmatrix}$$

$$= 2 - 0.5 - 0.5 + 0.5 - 0.5 = 1$$

Step 3: Initialize the activations of Maxnet as

$$y_1(0) = 3; \quad y_2(0) = 1$$

Step 4: Since $y_1(0) > y_2(0)$, Maxnet will find that the unit y_1 is the best match exemplar for the input vector $x_4 = [1 \ 1 \ -1 \ -1]$.

The architecture for the Hamming net for this problem is given by Figure 1.

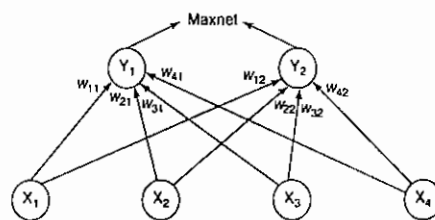


Figure 1 Hamming net architecture.

3. Construct a Kohonen self-organizing map to cluster the four given vectors, $[0 \ 0 \ 1 \ 1]$, $[1 \ 0 \ 0 \ 0]$, $[0 \ 1 \ 1 \ 0]$ and $[0 \ 0 \ 0 \ 1]$. The number of clusters to

be formed is two. Assume an initial learning rate of 0.5.

Solution: The number of input vectors is four and number of clusters to be formed is two. Thus, $n = 4$ and $m = 2$. The architecture of the Kohonen self-organizing feature map is given by Figure 2.

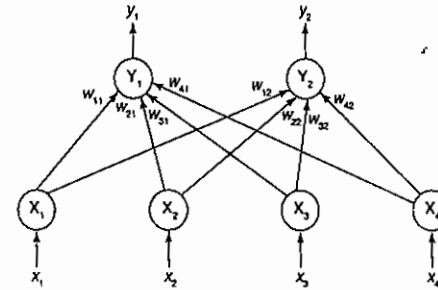


Figure 2 Architecture of KSOFM.

Step 0: Initialize the weights randomly between 0 and 1.

$$w_{ij} = \begin{bmatrix} 0.2 & 0.9 \\ 0.4 & 0.7 \\ 0.6 & 0.5 \\ 0.8 & 0.3 \end{bmatrix}_{4 \times 2}; \quad R = 0; \alpha(0) = 0.5$$

First input vector:

Step 1: For $x = [0 \ 0 \ 1 \ 1]$, perform Steps 2-4.

Step 2: Calculate the Euclidean distance:

$$D(j)' = \sum_i (w_{ij} - x_i)^2$$

$$D(1) = \sum_{i=1}^4 (w_{i1} - x_i)^2$$

$$= (0.2 - 0)^2 + (0.4 - 0)^2$$

$$+ (0.6 - 1)^2 + (0.8 - 1)^2$$

$$= 0.04 + 0.16 + 0.16 + 0.04$$

$$= 0.4$$

$$D(2) = \sum_{i=1}^4 (w_{i2} - x_i)^2$$

$$= (0.9 - 0)^2 + (0.7 - 0)^2$$

$$+ (0.5 - 1)^2 + (0.3 - 1)^2$$

$$= 0.81 + 0.49 + 0.25 + 0.49$$

$$= 2.04$$

Step 3: Since $D(1) < D(2)$, therefore $D(1)$ is minimum. Hence the winning cluster unit is Y_1 , i.e., $J = 1$.

Step 4: Update the weights on the winning cluster unit $J = 1$.

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})]$$

$$w_{i1}(\text{new}) = w_{i1}(\text{old}) + 0.5[x_i - w_{i1}(\text{old})]$$

$$w_{11}(n) = w_{11}(0) + 0.5[x_1 - w_{11}(0)]$$

$$= 0.2 + 0.5(0 - 0.2) = 0.1$$

$$w_{21}(n) = w_{21}(0) + 0.5[x_2 - w_{21}(0)]$$

$$= 0.4 + 0.5(0 - 0.4) = 0.2$$

$$w_{31}(n) = w_{31}(0) + 0.5[x_3 - w_{31}(0)]$$

$$= 0.6 + 0.5(1 - 0.6) = 0.8$$

$$w_{41}(n) = w_{41}(0) + 0.5[x_4 - w_{41}(0)]$$

$$= 0.8 + 0.5(1 - 0.8) = 0.9$$

The updated weight matrix after presentation of first input pattern is

$$w_{ij} = \begin{bmatrix} 0.1 & 0.9 \\ 0.2 & 0.7 \\ 0.8 & 0.5 \\ 0.9 & 0.3 \end{bmatrix}$$

Second input vector:

Step 1: For $x = [1 \ 0 \ 0 \ 0]$, perform Steps 2-4.

Step 2: Calculate the Euclidean distance:

$$D(j) = \sum_i (w_{ij} - x_i)^2$$

$$D(1) = \sum_{i=1}^4 (w_{i1} - x_i)^2$$

$$\begin{aligned}
 &= (0.1 - 1)^2 + (0.2 - 0)^2 \\
 &\quad + (0.8 - 0)^2 + (0.9 - 0)^2 \\
 &= 0.81 + 0.04 + 0.64 + 0.81 \\
 &= 2.3
 \end{aligned}$$

$$\begin{aligned}
 D(2) &= \sum_{i=1}^4 (w_{i2} - x_i)^2 \\
 &= (0.9 - 1)^2 + (0.7 - 0)^2 \\
 &\quad + (0.5 - 0)^2 + (0.3 - 0)^2 \\
 &= 0.01 + 0.49 + 0.25 + 0.09 \\
 &= 0.84
 \end{aligned}$$

Step 3: Since $D(2) < D(1)$, therefore $D(2)$ is minimum. Hence the winning cluster unit is Y_2 , i.e., $j = 2$.

Step 4: Update the weights on the winning cluster unit $j = 2$:

$$\begin{aligned}
 w_{ij}(\text{new}) &= w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})] \\
 w_{12}(\text{new}) &= w_{12}(\text{old}) + 0.5[x_1 - w_{12}(\text{old})] \\
 w_{12}(n) &= w_{12}(0) + 0.5[x_1 - w_{12}(0)] \\
 &= 0.9 + 0.5(1 - 0.9) = 0.95 \\
 w_{22}(n) &= w_{22}(0) + 0.5[x_2 - w_{22}(0)] \\
 &= 0.7 + 0.5(0 - 0.7) = 0.35 \\
 w_{32}(n) &= w_{32}(0) + 0.5[x_3 - w_{32}(0)] \\
 &= 0.5 + 0.5(0 - 0.5) = 0.25 \\
 w_{42}(n) &= w_{42}(0) + 0.5[x_4 - w_{42}(0)] \\
 &= 0.3 + 0.5(0 - 0.3) = 0.15
 \end{aligned}$$

The updated weight matrix after presentation of second input pattern is

$$w_{ij} = \begin{bmatrix} 0.1 & 0.95 \\ 0.2 & 0.35 \\ 0.8 & 0.25 \\ 0.9 & 0.15 \end{bmatrix}$$

Third input vector:

Step 1: For $x = [0 \ 1 \ 1 \ 0]$, perform Steps 2-4.

Step 2: Calculate the Euclidean distance:

$$\begin{aligned}
 D(j) &= \sum_i (w_{ij} - x_i)^2 \\
 D(1) &= \sum_{i=1}^4 (w_{i1} - x_i)^2 \\
 &= (0.1 - 0)^2 + (0.2 - 1)^2 \\
 &\quad + (0.8 - 1)^2 + (0.9 - 0)^2 \\
 &= 0.01 + 0.64 + 0.04 + 0.81 = 1.5 \\
 D(2) &= \sum_{i=1}^4 (w_{i2} - x_i)^2 \\
 &= (0.95 - 0)^2 + (0.35 - 1)^2 \\
 &\quad + (0.25 - 1)^2 + (0.15 - 0)^2 \\
 &= (0.9025) + (0.4225) + (0.5625) \\
 &\quad + (0.0225) = 1.91
 \end{aligned}$$

Step 3: Since $D(1) < D(2)$, therefore $D(1)$ is minimum. Hence the winning cluster unit is Y_1 , i.e., $j = 1$.

Step 4: Update the weights on the winning cluster unit $j = 1$:

$$\begin{aligned}
 w_{ij}(\text{new}) &= w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})] \\
 w_{11}(\text{new}) &= w_{11}(\text{old}) + 0.5[x_1 - w_{11}(\text{old})] \\
 w_{11}(n) &= w_{11}(0) + 0.5[x_1 - w_{11}(0)] \\
 &= 0.1 + 0.5(0 - 0.1) = 0.05 \\
 w_{21}(n) &= w_{21}(0) + 0.5[x_2 - w_{21}(0)] \\
 &= 0.2 + 0.5(1 - 0.2) = 0.6 \\
 w_{31}(n) &= w_{31}(0) + 0.5[x_3 - w_{31}(0)] \\
 &= 0.8 + 0.5(1 - 0.8) = 0.9 \\
 w_{41}(n) &= w_{41}(0) + 0.5[x_4 - w_{41}(0)] \\
 &= 0.9 + 0.5(0 - 0.9) = 0.45
 \end{aligned}$$

The weight update after presentation of third input pattern is

$$w_{ij} = \begin{bmatrix} 0.05 & 0.95 \\ 0.6 & 0.35 \\ 0.9 & 0.25 \\ 0.45 & 0.15 \end{bmatrix}$$

Fourth input vector:

Step 1: For $x = [0 \ 0 \ 1 \ 1]$, perform Steps 2-4.

Step 2: Compute the Euclidean distance:

$$\begin{aligned}
 D(j) &= \sum_{i=1}^4 (w_{ij} - x_i)^2 \\
 D(1) &= \sum_{i=1}^4 (w_{i1} - x_i)^2 \\
 &= (0.05 - 0)^2 + (0.6 - 0)^2 \\
 &\quad + (0.9 - 0)^2 + (0.45 - 1)^2 \\
 &= 0.0025 + 0.36 + 0.81 + 0.3025 \\
 &= 1.475 \\
 D(2) &= \sum_{i=1}^4 (w_{i2} - x_i)^2 \\
 &= (0.95 - 0)^2 + (0.35 - 0)^2 \\
 &\quad + (0.25 - 0)^2 + (0.15 - 1)^2 \\
 &= (0.9025) + (0.1225) + (0.0625) \\
 &\quad + (0.7225) \\
 &= 1.81
 \end{aligned}$$

Step 3: Since $D(1) < D(2)$, therefore $D(1)$ is minimum. Hence the winning cluster unit is Y_1 , i.e., $j = 1$.

Step 4: Update the weights on the winning cluster unit $j = 1$:

$$\begin{aligned}
 w_{ij}(\text{new}) &= w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})] \\
 w_{11}(\text{new}) &= w_{11}(\text{old}) + 0.5[x_1 - w_{11}(\text{old})] \\
 w_{11}(n) &= w_{11}(0) + 0.5[x_1 - w_{11}(0)] \\
 &= 0.05 + 0.5(0 - 0.05) = 0.025 \\
 w_{21}(n) &= w_{21}(0) + 0.5[x_2 - w_{21}(0)] \\
 &= 0.6 + 0.5(0 - 0.6) = 0.3 \\
 w_{31}(n) &= w_{31}(0) + 0.5[x_3 - w_{31}(0)] \\
 &= 0.9 + 0.5(1 - 0.9) = 0.45 \\
 w_{41}(n) &= w_{41}(0) + 0.5[x_4 - w_{41}(0)] \\
 &= 0.45 + 0.5(1 - 0.95) = 0.475
 \end{aligned}$$

The final weight obtained after the presentation of fourth input pattern is

$$w_{ij} = \begin{bmatrix} 0.025 & 0.95 \\ 0.3 & 0.35 \\ 0.45 & 0.25 \\ 0.475 & 0.15 \end{bmatrix}$$

Since all the four given input patterns are presented, this is end of first iteration or 1-epoch. Now the learning rate can be updated as

$$\alpha(t+1) = 0.5 \alpha(t)$$

$$\alpha(1) = 0.5 \alpha(0) = 0.5 \times 0.5 = 0.25$$

With this learning rate you can proceed further up to 100 iterations or till radius becomes zero or the weight matrix reduces to a very negligible value. The net with updated weights is shown by Figure 3.

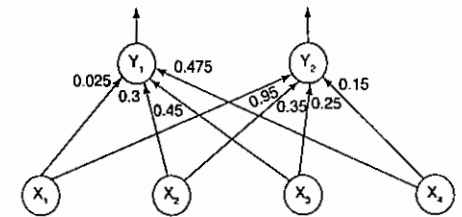


Figure 3 Net for problem 3.

4. For a given Kohonen self-organizing feature map with weights shown in Figure 4: (a) Use the square of the Euclidean distance to find the cluster unit Y_j closest to the input vector $(0.2, 0.4)$. Using a learning rate of 0.2, find the new weights for unit Y_j . (b) For the input vector $(0.6, 0.6)$ with learning rate 0.1, find the winning cluster unit and its new weights.

Solution: (a) For the input vector $(0.2, 0.4) = (x_1, x_2)$ and $\alpha = 0.2$, the weight vector W is given by

$$W = \begin{bmatrix} 0.3 & 0.2 & 0.1 & 0.8 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.9 & 0.2 \end{bmatrix}$$

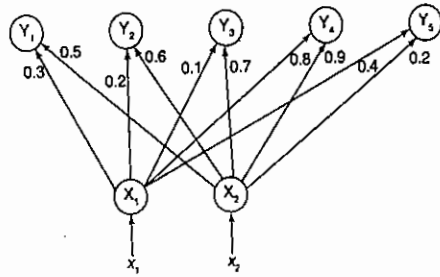


Figure 4 KSOFM net for problem 4.

Now we find the winner unit using square of Euclidean distance, i.e.,

$$D(j) = \sum_{i=1}^2 (w_{ij} - x_i)^2 = (w_{1j} - x_1)^2 + (w_{2j} - x_2)^2$$

For $j = 1$ to 5

$$D(1) = (0.3 - 0.2)^2 + (0.5 - 0.4)^2 = 0.01 + 0.01 = 0.02$$

$$D(2) = (0.2 - 0.2)^2 + (0.6 - 0.4)^2 = 0 + 0.4 = 0.04$$

$$D(3) = (0.1 - 0.2)^2 + (0.7 - 0.4)^2 = 0.01 + 0.09 = 0.10$$

$$D(4) = (0.8 - 0.2)^2 + (0.9 - 0.4)^2 = 0.36 + 0.25 = 0.61$$

$$D(5) = (0.4 - 0.2)^2 + (0.2 - 0.4)^2 = 0.04 + 0.04 = 0.08$$

Since $D(1) = 0.02$ is the minimum value, the winner unit is $j = 1$. We now update the weights on the winner unit $j = 1$. The weight updation formula is given by

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})]$$

Substituting $j = 1$ in the equation above, we obtain

$$w_{11}(\text{new}) = w_{11}(\text{old}) + \alpha[x_1 - w_{11}(\text{old})]$$

For $i = 1$ to 2,

$$w_{11}(n) = w_{11}(0) + \alpha[x_1 - w_{11}(0)] = 0.3 + 0.2(0.2 - 0.3) = 0.28$$

$$w_{21}(n) = w_{21}(0) + \alpha[x_2 - w_{21}(0)] = 0.5 + 0.2(0.4 - 0.5) = 0.48$$

The updated weight matrix is given by

$$W = \begin{bmatrix} 0.28 & 0.2 & 0.1 & 0.8 & 0.4 \\ 0.48 & 0.6 & 0.7 & 0.9 & 0.2 \end{bmatrix}$$

- For the input vector $(x_1, x_2) = (0.6, 0.6)$ and $\alpha = 0.1$, the weight matrix is initialized from Figure 4 as

$$W = \begin{bmatrix} 0.3 & 0.2 & 0.1 & 0.8 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.9 & 0.2 \end{bmatrix}$$

Now we find the winner unit using square of Euclidean distance, i.e.,

$$D(j) = \sum_{i=1}^2 (w_{ij} - x_i)^2 = (w_{1j} - x_1)^2 + (w_{2j} - x_2)^2$$

For $j = 1$ to 5

$$D(1) = (0.3 - 0.6)^2 + (0.5 - 0.6)^2 = 0.09 + 0.01 = 0.10$$

$$D(2) = (0.2 - 0.6)^2 + (0.6 - 0.6)^2 = 0.16 + 0 = 0.16$$

$$D(3) = (0.1 - 0.6)^2 + (0.7 - 0.6)^2 = 0.25 + 0.01 = 0.26$$

$$D(4) = (0.8 - 0.6)^2 + (0.9 - 0.6)^2 = 0.04 + 0.09 = 0.13$$

$$D(5) = (0.4 - 0.6)^2 + (0.2 - 0.6)^2 = 0.04 + 0.16 = 0.20$$

Since $D(2) = 0.16$ is the minimum value, the winner unit is $j = 2$. We now update the weights on the winner unit with $\alpha = 0.1$. The weight updation formula is given by

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})]$$

Substituting $j = 2$ in the equation above, we obtain

$$w_{12}(\text{new}) = w_{12}(\text{old}) + \alpha[x_1 - w_{12}(\text{old})]$$

For $i = 1$ to 2,

$$w_{12}(n) = w_{12}(0) + \alpha[x_1 - w_{12}(0)] = 0.2 + 0.1(0.6 - 0.2) = 0.24$$

$$w_{22}(n) = w_{22}(0) + \alpha[x_2 - w_{22}(0)] = 0.6 + 0.1(0.6 - 0.6) = 0.6$$

The new weight matrix is given by

$$W = \begin{bmatrix} 0.3 & 0.24 & 0.1 & 0.8 & 0.4 \\ 0.5 & 0.6 & 0.7 & 0.9 & 0.2 \end{bmatrix}$$

- Consider a Kohonen self-organizing net with two cluster units and five input units. The weight vectors for the cluster units are given by

$$w_1 = [1.0 \ 0.9 \ 0.7 \ 0.5 \ 0.3]$$

$$w_2 = [0.3 \ 0.5 \ 0.7 \ 0.9 \ 1.0]$$

Use the square of the Euclidean distance to find the winning cluster unit for the input pattern $x = [0.0 \ 0.5 \ 1.0 \ 0.5 \ 0.0]$. Using a learning rate of 0.25, find the new weights for the winning unit.

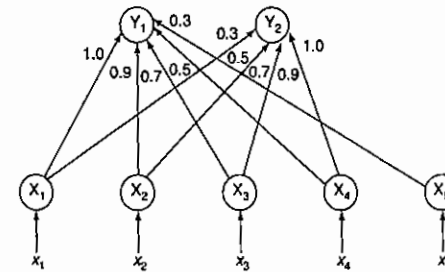


Figure 5 KSOFM net.

Solution: The net can be formed as shown in Figure 5. For the input vector $x = [0.0 \ 0.5 \ 1.0 \ 0.5 \ 0.0]$ and the learning rate $\alpha = 0.25$, the weight vector W is given by

$$W = \begin{bmatrix} 1.0 & 0.3 \\ 0.9 & 0.5 \\ 0.7 & 0.7 \\ 0.5 & 0.9 \\ 0.3 & 1.0 \end{bmatrix}$$

Now we find the winner unit using square of Euclidean distance, i.e.,

$$D(j) = \sum_i (w_{ij} - x_i)^2$$

For $i = 1$ to 5 and $j = 1$ to 2,

$$D(1) = (1.0 - 0.0)^2 + (0.9 - 0.5)^2 + (0.7 - 1.0)^2 + (0.5 - 0.5)^2 + (0.3 - 0.0)^2 = 1 + 0.16 + 0.09 + 0 + 0.09 = 1.34$$

$$D(2) = (0.3 - 0.0)^2 + (0.5 - 0.5)^2 + (0.7 - 1.0)^2 + (0.9 - 0.5)^2 + (1.0 - 0.0)^2 = 0.09 + 0 + 0.09 + 0.16 + 1 = 1.34$$

As we can see, in this case $D(1) = D(2)$, so the winner unit is the one with the smallest index. Thus, winner unit is Y_1 , i.e., $j = 1$. We now update the weights on the winner unit with $\alpha = 0.25$. The weight updation formula is given by

$$w_{ij}(\text{new}) = w_{ij}(\text{old}) + \alpha[x_i - w_{ij}(\text{old})]$$

Substituting $j = 1$ in the equation above, we obtain

$$w_{11}(\text{new}) = w_{11}(\text{old}) + \alpha[x_1 - w_{11}(\text{old})]$$

For $i = 1$ to 5,

$$w_{11}(n) = w_{11}(0) + \alpha[x_1 - w_{11}(0)] = 1 + 0.25(0 - 1) = 0.75$$

$$w_{21}(n) = w_{21}(0) + \alpha[x_2 - w_{21}(0)] = 0.9 + 0.25(0.5 - 0.9) = 0.8$$

$$w_{31}(n) = w_{31}(0) + \alpha[x_3 - w_{31}(0)] = 0.7 + 0.25(1.0 - 0.7) = 0.775$$

$$w_{41}(n) = w_{41}(0) + \alpha[x_4 - w_{41}(0)] = 0.5 + 0.25(0.5 - 0.5) = 0.5$$

$$w_{51}(n) = w_{51}(0) + \alpha[x_5 - w_{51}(0)] = 0.3 + 0.25(0 - 0.3) = 0.225$$

The updated weight matrix for the winning unit is given by

$$W = \begin{bmatrix} 0.75 & 0.3 \\ 0.8 & 0.5 \\ 0.775 & 0.7 \\ 0.5 & 0.9 \\ 0.225 & 1.0 \end{bmatrix}$$

6. Construct and test an LVQ net with five vectors assigned to two classes. The given vectors along with the classes are as shown in Table 1.

Table 1

Vector	Class
[0 0 1 1]	1
[1 0 0 0]	2
[0 0 0 1]	2
[1 1 0 0]	1
[0 1 1 0]	1

Solution: In the given five vectors, first two vectors are used as initial weight vectors and the remaining three vectors are used as input vectors. Based on this, LVQ net is shown in Figure 6, along with initial weights.

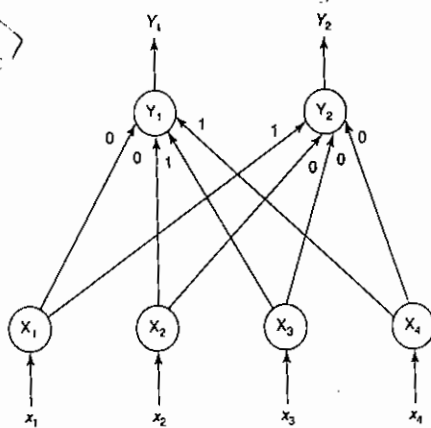


Figure 6 LVQ net.

Initialize the reference weight vectors as

$$w_1 = [0 \ 0 \ 1 \ 1]; \quad w_2 = [1 \ 0 \ 0 \ 0]$$

Let the learning rate be $\alpha = 0.1$.

First input vector

For [0 0 0 1] with $T = 2$, calculate the square of the Euclidean distance, i.e.,

$$D(j) = \sum_{i=1}^4 (w_{ij} - x_i)^2$$

For $j = 1$ to 2,

$$D(1) = (0 - 0)^2 + (0 - 0)^2 + (1 - 0)^2 + (1 - 1)^2 = 1$$

$$D(2) = (1 - 0)^2 + (0 - 0)^2 + (0 - 0)^2 + (0 - 0)^2 = 1$$

Since $D(1) < D(2)$, $D(1)$ is minimum; hence the winner unit index is $J = 1$. Now that $T \neq J$, the weight updation is performed as

$$w_j(\text{new}) = w_j(\text{old}) - \alpha[x - w_j(\text{old})]$$

$$w_{11}(n) = w_{11}(0) - \alpha[x_1 - w_{11}(0)] = 0 - 0.1(0 - 0) = 0$$

$$w_{21}(n) = w_{21}(0) - \alpha[x_2 - w_{21}(0)] = 0 - 0.1(0 - 0) = 0$$

$$w_{31}(n) = w_{31}(0) - \alpha[x_3 - w_{31}(0)] = 1 - 0.1(0 - 1) = 1.1$$

$$w_{41}(n) = w_{41}(0) - \alpha[x_4 - w_{41}(0)] = 1 - 0.1(1 - 1) = 1$$

After the presentation of first input pattern, the weight matrix becomes

$$W = \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 1.1 & 0 \\ 1 & 0 \end{bmatrix}$$

Second input vector

For [1 1 0 0] with $T = 1$, calculate the square of the Euclidean distance, i.e.,

$$D(j) = \sum_{i=1}^4 (w_{ij} - x_i)^2$$

For $j = 1$ to 2,

$$D(1) = (0 - 1)^2 + (0 - 1)^2 + (1.1 - 0)^2 + (1 - 0)^2 = 4.21$$

$$D(2) = (1 - 1)^2 + (0 - 1)^2 + (0 - 0)^2 + (0 - 0)^2 = 1$$

Since $D(2) < D(1)$, $D(2)$ is minimum; hence the winner unit index is $J = 2$. Again since $T \neq J$, the weight updation is performed as

$$w_j(\text{new}) = w_j(\text{old}) - \alpha[x - w_j(\text{old})]$$

$$w_{12}(n) = w_{12}(0) - \alpha[x_1 - w_{12}(0)] = 1 - 0.1(1 - 1) = 1$$

$$w_{22}(n) = w_{22}(0) - \alpha[x_2 - w_{22}(0)] = 0 - 0.1(1 - 0) = -0.1$$

$$w_{32}(n) = w_{32}(0) - \alpha[x_3 - w_{32}(0)] = 0 - 0.1(0 - 0) = 0$$

$$w_{42}(n) = w_{42}(0) - \alpha[x_4 - w_{42}(0)] = 0 - 0.1(0 - 0) = 0$$

After the presentation of second input pattern, the weight matrix becomes

$$W = \begin{bmatrix} 0 & 1 \\ 0 & -0.1 \\ 1.1 & 0 \\ 1 & 0 \end{bmatrix}$$

Third input vector

For [0 1 1 0] with $T = 1$, calculate the square of the Euclidean distance as

$$D(j) = \sum_{i=1}^4 (w_{ij} - x_i)^2$$

For $j = 1$ to 2,

$$D(1) = (0 - 0)^2 + (0 - 1)^2 + (1.1 - 1)^2 + (1 - 0)^2 = 2.01$$

$$D(2) = (1 - 0)^2 + (-0.1 - 1)^2 + (0 - 1)^2 + (0 - 0)^2 = 3.21$$

Since $D(1) < D(2)$, $D(1)$ is minimum; hence the winner unit index is $J = 1$. Now that $T = J$, the weight updation is performed as

$$w_j(\text{new}) = w_j(\text{old}) + \alpha[x - w_j(\text{old})]$$

Updating the weights on the winner unit, we obtain

$$w_{11}(n) = w_{11}(0) + \alpha[x_1 - w_{11}(0)] = 0 + 0.1(0 - 0) = 0$$

$$w_{21}(n) = w_{21}(0) + \alpha[x_2 - w_{21}(0)] = 0 + 0.1(1 - 0) = 0.1$$

$$w_{31}(n) = w_{31}(0) + \alpha[x_3 - w_{31}(0)] = 1.1 + 0.1(1 - 1.1) = 1.09$$

$$w_{41}(n) = w_{41}(0) + \alpha[x_4 - w_{41}(0)] = 1 + 0.1(0 - 1) = 0.9$$

After the presentation of third input pattern, the weight matrix becomes

$$W = \begin{bmatrix} 0 & 1 \\ 0.1 & -0.1 \\ 1.09 & 0 \\ 0.9 & 0 \end{bmatrix}$$

Thus the first epoch of the training has been completed. It is noted that if correct class is obtained for first and second input patterns, further epochs can be performed until all the winner units become equal to all the classes, i.e., all $T = J$.

7. Consider an LVQ net with two input units and four target classes: c_1, c_2, c_3 and c_4 . There exist 16 classification units, with weight vectors indicated by the coordinates on the following chart, read in row-column order. For example, the unit with weight vector (0.2, 0.2), (0.2, 0.6) is assigned to represent class 1 and the classification units for class 2 have initial weight vectors of (0.4, 0.2), (0.4, 0.6), (0.8, 0.4) and (0.8, 0.8). The chart is given in Table 2.

Table 2

x_2				
1.0				
0.8	c_3	c_4	c_1	c_2
0.6	c_1	c_2	c_3	c_4
0.4	c_3	c_4	c_1	c_2
0.2	c_1	c_2	c_3	c_4
0.0				
0.0	0.2	0.4	0.6	0.8
	x_1			

Use square of Euclidean distance to measure the changes occurring.

- Given an input vector of (0.25, 0.25) representing class 1 and using a learning rate

of $\alpha = 0.25$, show which classification unit moves where (i.e., determine its new weight vector).

- Given the input vector of (0.4, 0.35) representing class 1, using initial weight vector and learning rate of $\alpha = 0.25$, note what happens?
- Given the input vector of (0.4, 0.45), determine the performance of the net. The input vector represents class 1.

Solution: The LVQ net for this problem with two input units and four cluster units is shown in Figure 7. The initial weight vectors for the respective classes are shown below.

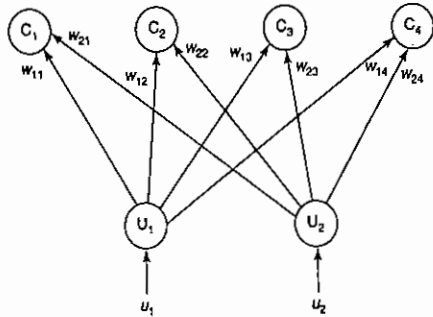


Figure 7 LVQ net (with two input units, four cluster units).

Class 1:

Initial weight vector

$$W_1 = \begin{bmatrix} 0.2 & 0.2 & 0.6 & 0.6 \\ 0.2 & 0.6 & 0.8 & 0.4 \end{bmatrix}$$

with target $t = 1$.

Class 2:

Initial weight vector

$$W_2 = \begin{bmatrix} 0.4 & 0.4 & 0.8 & 0.8 \\ 0.2 & 0.6 & 0.8 & 0.4 \end{bmatrix}$$

with target $t = 2$.

Class 3:

Initial weight vector

$$W_3 = \begin{bmatrix} 0.2 & 0.2 & 0.6 & 0.6 \\ 0.4 & 0.8 & 0.6 & 0.2 \end{bmatrix}$$

with target $t = 3$.

Class 4:

Initial weight vector

$$W_4 = \begin{bmatrix} 0.4 & 0.4 & 0.8 & 0.8 \\ 0.4 & 0.8 & 0.6 & 0.2 \end{bmatrix}$$

with target $t = 4$.

- For the given input vector $(u_1, u_2) = (0.25, 0.25)$ with $\alpha = 0.25$ and $t = 1$, we calculate the square of the Euclidean distance using the formula

$$D(j) = \sum_{i=1}^2 (w_{ij} - x_i)^2 = (w_{1j} - x_1)^2 + (w_{2j} - x_2)^2$$

For $j = 1$ to 4,

$$D(1) = (0.2 - 0.25)^2 + (0.2 - 0.25)^2 = 0.005$$

$$D(2) = (0.2 - 0.25)^2 + (0.6 - 0.25)^2 = 0.125$$

$$D(3) = (0.6 - 0.25)^2 + (0.8 - 0.25)^2 = 0.425$$

$$D(4) = (0.6 - 0.25)^2 + (0.4 - 0.25)^2 = 0.145$$

As $D(1)$ is minimum, therefore the winner unit index is $j = 1$. Now we update the weights on the winner unit, since $t = j = 1$, $\alpha = 0.25$, using the weight updation formula

$$w_j(\text{new}) = w_j(\text{old}) + \alpha[x - w_j(\text{old})]$$

Updating the weights on the winner unit, we obtain

$$w_{11}(\text{new}) = w_{11}(\text{old}) + \alpha[x_1 - w_{11}(\text{old})] \\ = 0.2 + 0.25(0.25 - 0.2) = 0.2125$$

$$w_{21}(\text{new}) = w_{21}(\text{old}) + \alpha[x_2 - w_{21}(\text{old})] \\ = 0.2 + 0.25(0.25 - 0.2) = 0.2125$$

Therefore, the new weight vector is

$$W_1 = \begin{bmatrix} 0.2125 & 0.2 & 0.6 & 0.6 \\ 0.2125 & 0.6 & 0.8 & 0.4 \end{bmatrix}$$

- For the given input vector $(u_1, u_2) = (0.4, 0.35)$ with $\alpha = 0.25$ and $t = 1$, we calculate the square of the Euclidean distance using the formula

$$D(j) = \sum_{i=1}^2 (w_{ij} - x_i)^2 = (w_{1j} - x_1)^2 + (w_{2j} - x_2)^2$$

For $j = 1$ to 4,

$$D(1) = (0.2 - 0.4)^2 + (0.2 - 0.35)^2 = 0.0625$$

$$D(2) = (0.2 - 0.4)^2 + (0.6 - 0.35)^2 = 0.1025$$

$$D(3) = (0.6 - 0.4)^2 + (0.8 - 0.35)^2 = 0.2425$$

$$D(4) = (0.6 - 0.4)^2 + (0.4 - 0.35)^2 = 0.0425$$

As $D(4)$ is minimum, therefore the winner unit index is $j = 4$. Thus, fourth unit is the winner unit that is closest to the input vector. Since $t \neq j$, the weight updation formula used is

$$w_j(\text{new}) = w_j(\text{old}) - \alpha[x - w_j(\text{old})]$$

Updating the weights on the winner unit, we obtain

$$w_{14}(\text{new}) = w_{14}(\text{old}) - \alpha[x_1 - w_{14}(\text{old})] \\ = 0.6 - 0.25(0.4 - 0.6) = 0.65$$

$$w_{24}(\text{new}) = w_{24}(\text{old}) - \alpha[x_2 - w_{24}(\text{old})] \\ = 0.4 - 0.25(0.35 - 0.4) = 0.4125$$

Therefore, the new weight vector is

$$W_1 = \begin{bmatrix} 0.2 & 0.2 & 0.6 & 0.65 \\ 0.2 & 0.6 & 0.8 & 0.4125 \end{bmatrix}$$

- For the given input vector $(u_1, u_2) = (0.4, 0.45)$ with $\alpha = 0.25$ and $t = 1$, we calculate the square of the Euclidean distance using the formula

$$D(j) = \sum_{i=1}^2 (w_{ij} - x_i)^2 = (w_{1j} - x_1)^2 + (w_{2j} - x_2)^2$$

For $j = 1$ to 4,

$$D(1) = (0.2 - 0.4)^2 + (0.2 - 0.45)^2 = 0.1025$$

$$D(2) = (0.2 - 0.4)^2 + (0.6 - 0.45)^2 = 0.0625$$

$$D(3) = (0.6 - 0.4)^2 + (0.8 - 0.45)^2 = 0.1625$$

$$D(4) = (0.6 - 0.4)^2 + (0.4 - 0.45)^2 = 0.0425$$

As $D(4)$ is minimum, therefore in this case also the winner unit index is $j = 4$. Since $t \neq j$, the weight updation formula used is

$$w_j(\text{new}) = w_j(\text{old}) - \alpha[x - w_j(\text{old})]$$

Updating the weights on the winner unit, we obtain

$$w_{14}(\text{new}) = w_{14}(\text{old}) - \alpha[x_1 - w_{14}(\text{old})] \\ = 0.6 - 0.25(0.4 - 0.6) = 0.65$$

$$w_{24}(\text{new}) = w_{24}(\text{old}) - \alpha[x_2 - w_{24}(\text{old})] \\ = 0.4 - 0.25(0.45 - 0.4) = 0.3875$$

Therefore, the new weight vector is

$$W_1 = \begin{bmatrix} 0.2 & 0.2 & 0.6 & 0.65 \\ 0.2 & 0.6 & 0.8 & 0.3875 \end{bmatrix}$$

8. Consider the following full CPN shown in Figure 8. Using the input pair $x = [1 \ 0 \ 0 \ 0]$ and $y = [1 \ 0]$, perform the phase I of training (one step only). Find the activation of the cluster layer units and update the weights using learning rates $\alpha = \beta = 0.2$.

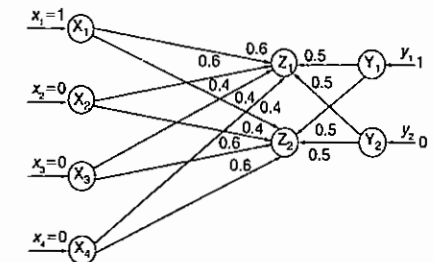


Figure 8 Instar model of CPN net.

Solution: The input pair is $x = [1 \ 0 \ 0 \ 0]$ and $y = [1 \ 0]$ and the learning rates are $\alpha = 0.2$ and $\beta = 0.2$.

Phase I of training: The initial weights are obtained from Figure 8 as

$$V = \begin{bmatrix} 0.6 & 0.4 \\ 0.6 & 0.4 \\ 0.4 & 0.6 \\ 0.4 & 0.6 \end{bmatrix} \quad \text{and} \quad W = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}$$

Now we calculate the square of the Euclidean distance using the formula

$$D(j) = \sum_{i=1}^4 (x_i - v_{ij})^2 + \sum_{k=1}^2 (y_k - w_{kj})^2$$

For $j = 1$ to 2,

$$\begin{aligned} D(1) &= \sum_{i=1}^4 (x_i - v_{i1})^2 + \sum_{k=1}^2 (y_k - w_{k1})^2 \\ &= (x_1 - v_{11})^2 + (x_2 - v_{21})^2 + (x_3 - v_{31})^2 \\ &\quad + (x_4 - v_{41})^2 + (y_1 - w_{11})^2 + (y_2 - w_{21})^2 \\ &= (1 - 0.6)^2 + (0 - 0.6)^2 + (0 - 0.4)^2 \\ &\quad + (0 - 0.4)^2 + (1 - 0.5)^2 + (0 - 0.5)^2 \\ &= 0.16 + 0.36 + 0.16 + 0.16 + 0.25 + 0.25 \\ D(1) &= 1.34 \end{aligned}$$

$$\begin{aligned} D(2) &= \sum_{i=1}^4 (x_i - v_{i2})^2 + \sum_{k=1}^2 (y_k - w_{k2})^2 \\ &= (x_1 - v_{12})^2 + (x_2 - v_{22})^2 + (x_3 - v_{32})^2 \\ &\quad + (x_4 - v_{42})^2 + (y_1 - w_{12})^2 + (y_2 - w_{22})^2 \\ &= (1 - 0.4)^2 + (0 - 0.4)^2 + (0 - 0.6)^2 \\ &\quad + (0 - 0.6)^2 + (1 - 0.5)^2 + (0 - 0.5)^2 \\ &= 0.36 + 0.16 + 0.36 + 0.36 + 0.25 + 0.25 \\ D(2) &= 1.74 \end{aligned}$$

Since, $D(1) < D(2)$, therefore the winner unit index is $j = 1$. We now update the weights on the winner unit.

Weight updation: The weight updation between the x -input and cluster layer is performed as shown below:

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha[x_i - v_{ij}(\text{old})]$$

For $i = 1$ to 4 and $j = 1$, we obtain

$$\begin{aligned} v_{11}(\text{new}) &= v_{11}(\text{old}) + \alpha[x_1 - v_{11}(\text{old})] \\ &= 0.6 + 0.2(1 - 0.6) = 0.68 \end{aligned}$$

$$\begin{aligned} v_{21}(\text{new}) &= v_{21}(\text{old}) + \alpha[x_2 - v_{21}(\text{old})] \\ &= 0.6 + 0.2(0 - 0.6) = 0.48 \end{aligned}$$

$$\begin{aligned} v_{31}(\text{new}) &= v_{31}(\text{old}) + \alpha[x_3 - v_{31}(\text{old})] \\ &= 0.4 + 0.2(0 - 0.4) = 0.32 \end{aligned}$$

$$\begin{aligned} v_{41}(\text{new}) &= v_{41}(\text{old}) + \alpha[x_4 - v_{41}(\text{old})] \\ &= 0.4 + 0.2(0 - 0.4) = 0.32 \end{aligned}$$

The weight updation between the y -input and cluster layer is performed as shown below:

$$w_{kj}(\text{new}) = w_{kj}(\text{old}) + \beta[y_k - w_{kj}(\text{old})]$$

For $k = 1$ to 2 and $j = 1$, we obtain

$$\begin{aligned} w_{11}(\text{new}) &= w_{11}(\text{old}) + \beta[y_1 - w_{11}(\text{old})] \\ &= 0.5 + 0.2(1 - 0.5) = 0.6 \end{aligned}$$

$$\begin{aligned} w_{21}(\text{new}) &= w_{21}(\text{old}) + \beta[y_2 - w_{21}(\text{old})] \\ &= 0.5 + 0.2(0 - 0.5) = 0.4 \end{aligned}$$

Thus the updated weights are

$$V = \begin{bmatrix} 0.68 & 0.4 \\ 0.48 & 0.4 \\ 0.32 & 0.6 \\ 0.32 & 0.6 \end{bmatrix} \quad \text{and} \quad W = \begin{bmatrix} 0.6 & 0.5 \\ 0.4 & 0.5 \end{bmatrix}$$

9. Consider the CPN net shown in Figure 9. Using the input pair $x = [0 \ 1 \ 1 \ 0]$ and $y = [0 \ 1]$, perform phase I of training (one step only). Find the activation of the cluster layer units and update the weights using learning rates $\alpha = 0.2$ and $\beta = 0.3$.

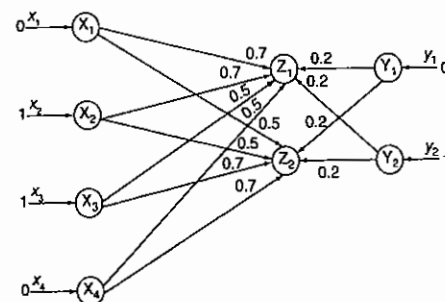


Figure 9 Instar model of CPN net.

Solution: The input pair is $x = [0 \ 1 \ 1 \ 0]$, $y = [0 \ 1]$ and learning rates $\alpha = 0.2$ and $\beta = 0.3$.

Phase I of training: The initial weights obtained from Figure 9 are

$$V = \begin{bmatrix} 0.7 & 0.5 \\ 0.7 & 0.5 \\ 0.5 & 0.7 \\ 0.5 & 0.7 \end{bmatrix} \quad \text{and} \quad W = \begin{bmatrix} 0.2 & 0.2 \\ 0.2 & 0.2 \end{bmatrix}$$

Now we calculate the square of the Euclidean distance using the formula

$$D(j) = \sum_{i=1}^4 (x_i - v_{ij})^2 + \sum_{k=1}^2 (y_k - w_{kj})^2$$

For $j = 1$ to 2,

$$\begin{aligned} D(1) &= \sum_{i=1}^4 (x_i - v_{i1})^2 + \sum_{k=1}^2 (y_k - w_{k1})^2 \\ &= (0 - 0.7)^2 + (1 - 0.7)^2 + (1 - 0.5)^2 \\ &\quad + (0 - 0.5)^2 + (0 - 0.2)^2 + (1 - 0.2)^2 \\ &= 0.49 + 0.09 + 0.25 + 0.25 + 0.04 + 0.64 \\ D(1) &= 1.76 \end{aligned}$$

$$\begin{aligned} D(2) &= \sum_{i=1}^4 (x_i - v_{i2})^2 + \sum_{k=1}^2 (y_k - w_{k2})^2 \\ &= (0 - 0.5)^2 + (1 - 0.5)^2 + (1 - 0.7)^2 \\ &\quad + (0 - 0.7)^2 + (0 - 0.2)^2 + (1 - 0.2)^2 \\ &= 0.25 + 0.25 + 0.09 + 0.49 + 0.04 + 0.64 \\ D(2) &= 1.76 \end{aligned}$$

In this case, $D(1) = D(2) = 1.76$, i.e., both the distances are equal. Hence the unit with the *smallest index* is chosen as the winner and weights are updated, i.e., we take $j = 1$ and update the weights on this winner unit.

Weight updation: The weight updation between the x -input and cluster layer is performed as shown below:

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha[x_i - v_{ij}(\text{old})]$$

For $i = 1$ to 4 and $j = 1$, we obtain

$$\begin{aligned} v_{11}(\text{new}) &= v_{11}(\text{old}) + \alpha[x_1 - v_{11}(\text{old})] \\ &= 0.7 + 0.2(0 - 0.7) = 0.56 \end{aligned}$$

$$\begin{aligned} v_{21}(\text{new}) &= v_{21}(\text{old}) + \alpha[x_2 - v_{21}(\text{old})] \\ &= 0.7 + 0.2(1 - 0.7) = 0.76 \end{aligned}$$

$$\begin{aligned} v_{31}(\text{new}) &= v_{31}(\text{old}) + \alpha[x_3 - v_{31}(\text{old})] \\ &= 0.5 + 0.2(1 - 0.5) = 0.60 \end{aligned}$$

$$\begin{aligned} v_{41}(\text{new}) &= v_{41}(\text{old}) + \alpha[x_4 - v_{41}(\text{old})] \\ &= 0.5 + 0.2(0 - 0.5) = 0.40 \end{aligned}$$

The weight updation between the Y -input and cluster layer is performed as shown below:

$$w_{kj}(\text{new}) = w_{kj}(\text{old}) + \beta[y_k - w_{kj}(\text{old})]$$

For $k = 1$ to 2 and $j = 1$, we obtain

$$\begin{aligned} w_{11}(\text{new}) &= w_{11}(\text{old}) + \beta[y_1 - w_{11}(\text{old})] \\ &= 0.2 + 0.3(0 - 0.2) = 0.14 \end{aligned}$$

$$\begin{aligned} w_{21}(\text{new}) &= w_{21}(\text{old}) + \beta[y_2 - w_{21}(\text{old})] \\ &= 0.2 + 0.3(1 - 0.2) = 0.44 \end{aligned}$$

Thus the updated weights are

$$V = \begin{bmatrix} 0.56 & 0.5 \\ 0.76 & 0.5 \\ 0.6 & 0.7 \\ 0.4 & 0.7 \end{bmatrix} \quad \text{and} \quad W = \begin{bmatrix} 0.14 & 0.2 \\ 0.44 & 0.2 \end{bmatrix}$$

10. Consider the forward-only CPN net shown in Figure 10. Using the input pair $x = [1 \ 0 \ 0 \ 0]$ and $y = [1 \ 0]$, perform phases I and II of training and update the weights using learning rates $\alpha = 0.2$.

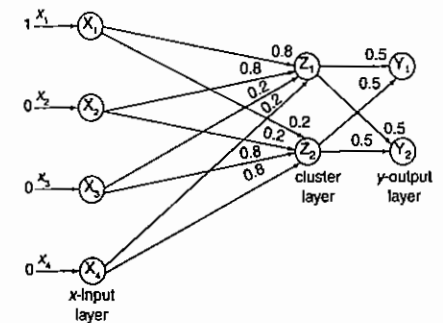


Figure 10 Forward-only CPN network.

Solution: The given input pair is $x = [1 \ 0 \ 0 \ 0]$ and $y = [1 \ 0]$ with learning rates $\alpha = a = 0.2$. The initial weights obtained from Figure 10 are

$$V = \begin{bmatrix} 0.8 & 0.2 \\ 0.8 & 0.2 \\ 0.2 & 0.8 \\ 0.2 & 0.8 \end{bmatrix} \text{ and } W = \begin{bmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{bmatrix}$$

Phase I of training: We calculate the Euclidean distance using the formula

$$D(j) = \sum_{i=1}^4 (x_i - v_{ij})^2$$

For $j = 1$ to 2,

$$\begin{aligned} D(1) &= \sum_{i=1}^4 (x_i - v_{i1})^2 \\ &= (1 - 0.8)^2 + (0 - 0.8)^2 + (0 - 0.2)^2 \\ &\quad + (0 - 0.2)^2 \\ &= 0.04 + 0.64 + 0.04 + 0.04 \\ D(1) &= 0.76 \\ D(2) &= \sum_{i=1}^4 (x_i - v_{i2})^2 \\ &= (1 - 0.2)^2 + (0 - 0.2)^2 + (0 - 0.8)^2 \\ &\quad + (0 - 0.8)^2 \\ &= 0.64 + 0.04 + 0.64 + 0.64 \\ D(2) &= 1.96 \end{aligned}$$

Since $D(1) < D(2)$, the winner unit index is $j = 1$.

Weight updation: The weight updation on the winner unit is given by

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha[x_i - v_{ij}(\text{old})]$$

For $i = 1$ to 4 and $j = 1$, we obtain

$$\begin{aligned} v_{11}(\text{new}) &= v_{11}(\text{old}) + \alpha[x_1 - v_{11}(\text{old})] \\ &= 0.8 + 0.2(1 - 0.8) = 0.84 \\ v_{21}(\text{new}) &= v_{21}(\text{old}) + \alpha[x_2 - v_{21}(\text{old})] \\ &= 0.8 + 0.2(0 - 0.8) = 0.64 \end{aligned}$$

$$\begin{aligned} v_{31}(\text{new}) &= v_{31}(\text{old}) + \alpha[x_3 - v_{31}(\text{old})] \\ &= 0.2 + 0.2(0 - 0.2) = 0.16 \\ v_{41}(\text{new}) &= v_{41}(\text{old}) + \alpha[x_4 - v_{41}(\text{old})] \\ &= 0.2 + 0.2(0 - 0.2) = 0.16 \end{aligned}$$

The updated weight matrix is,

$$V = \begin{bmatrix} 0.84 & 0.2 \\ 0.64 & 0.2 \\ 0.16 & 0.8 \\ 0.16 & 0.8 \end{bmatrix}$$

Phase II of training: We calculate the Euclidean distance using the formula

$$D(j) = \sum_{i=1}^4 (x_i - v_{ij})^2$$

For $j = 1$ to 2,

$$\begin{aligned} D(1) &= \sum_{i=1}^4 (x_i - v_{i1})^2 \\ &= (1 - 0.84)^2 + (0 - 0.64)^2 + (0 - 0.16)^2 \\ &\quad + (0 - 0.16)^2 \\ &= 0.0256 + 0.4096 + 0.0256 + 0.0256 \\ D(1) &= 0.4864 \\ D(2) &= \sum_{i=1}^4 (x_i - v_{i2})^2 \\ &= (1 - 0.2)^2 + (0 - 0.2)^2 + (0 - 0.8)^2 \\ &\quad + (0 - 0.8)^2 \\ &= 0.64 + 0.04 + 0.64 + 0.64 \\ D(2) &= 1.96 \end{aligned}$$

Since $D(1) < D(2)$, the winner unit index is $j = 1$.

Weight updation on winner unit:

• Updating the weights into unit j :

$$v_{ij}(\text{new}) = v_{ij}(\text{old}) + \alpha[x_i - v_{ij}(\text{old})]$$

For $i = 1$ to 4 and $j = 1$, we obtain

$$\begin{aligned} v_{11}(\text{new}) &= v_{11}(\text{old}) + \alpha[x_1 - v_{11}(\text{old})] \\ &= 0.84 + 0.2(1 - 0.84) = 0.872 \end{aligned}$$

$$\begin{aligned} v_{21}(\text{new}) &= v_{21}(\text{old}) + \alpha[x_2 - v_{21}(\text{old})] \\ &= 0.64 + 0.2(0 - 0.64) = 0.512 \\ v_{31}(\text{new}) &= v_{31}(\text{old}) + \alpha[x_3 - v_{31}(\text{old})] \\ &= 0.16 + 0.2(0 - 0.16) = 0.128 \\ v_{41}(\text{new}) &= v_{41}(\text{old}) + \alpha[x_4 - v_{41}(\text{old})] \\ &= 0.16 + 0.2(0 - 0.16) = 0.128 \end{aligned}$$

• Updating the weights from unit j to output layer:

$$w_{kj}(\text{new}) = w_{kj}(\text{old}) + a[y_k - w_{kj}(\text{old})]$$

For $k = 1$ to 2 and $j = 1$, we obtain

$$\begin{aligned} w_{11}(\text{new}) &= w_{11}(\text{old}) + a[y_1 - w_{11}(\text{old})] \\ &= 0.5 + 0.2(1 - 0.5) = 0.6 \\ w_{21}(\text{new}) &= w_{21}(\text{old}) + a[y_2 - w_{21}(\text{old})] \\ &= 0.5 + 0.2(0 - 0.5) = 0.4 \end{aligned}$$

Thus, the updated weights are after phase II of training are

$$V = \begin{bmatrix} 0.872 & 0.2 \\ 0.512 & 0.2 \\ 0.128 & 0.8 \\ 0.128 & 0.8 \end{bmatrix} \text{ and } W = \begin{bmatrix} 0.6 & 0.5 \\ 0.4 & 0.5 \end{bmatrix}$$

11. Construct an ART 1 network for clustering four input vectors with low vigilance parameter of 0.4 into three clusters. The four input vectors are $[0 \ 0 \ 0 \ 1]$, $[0 \ 1 \ 0 \ 1]$, $[0 \ 0 \ 1 \ 1]$ and $[1 \ 0 \ 0 \ 0]$. Assume the necessary parameters needed.

Solution: The values assumed in this case are $\rho = 0.4$, $\alpha = 2$. Also it can be noted that $n = 4$ and $m = 3$. Hence, Bottom-up-weights, $b_{ij}(0) = 1/(1 + n) = 1/1 + 4 = 0.2$.

Top-down-weights $t_{ji}(0) = 1$.

For $i = 1$ to 4 and $j = 1$ to 2,

$$b_{ij} = \begin{bmatrix} 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 \\ 0.2 & 0.2 & 0.2 \end{bmatrix}_{4 \times 3}$$

$$\text{and } t_{ji} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}_{3 \times 4}$$

Step 0: Initialize the parameters:

$$\rho = 0.4; \quad \alpha = 2$$

Initialize weights:

$$b_{ij}(0) = 0.2; \quad t_{ji}(0) = 1$$

Step 1: Start computation.

Step 2: For the first input vector $[0 \ 0 \ 0 \ 1]$, perform Steps 3–12.

Step 3: Set activations of all F_2 units to zero. Set activations of F_1 (a) units to input vector $s = [0 \ 0 \ 0 \ 1]$.

Step 4: Compute norm of s :

$$\|s\| = 0 + 0 + 0 + 1 = 1$$

Step 5: Compute activations for each node in the F_1 layer:

$$x = [0 \ 0 \ 0 \ 1]$$

Step 6: Compute net input to each node in the F_2 layer:

$$y_j = \sum_{i=1}^4 b_{ij}x_i$$

For $j = 1$ to 3,

$$\begin{aligned} y_1 &= 0.2(0) + 0.2(0) + 0.2(0) + 0.2(1) \\ &= 0.2 \\ y_2 &= 0.2(0) + 0.2(0) + 0.2(0) + 0.2(1) \\ &= 0.2 \\ y_3 &= 0.2(0) + 0.2(0) + 0.2(0) + 0.2(1) \\ &= 0.2 \end{aligned}$$

Step 7: When reset is true, perform Steps 8–11.

Step 8: Since all the inputs pose same net input, there exists a tie and the unit with the smallest index is the winner, i.e., $j = 1$.

Step 9: Recompute the F_1 activations (for $j = 1$):

$$\begin{aligned} x_j &= s_j t_{ji} \\ x_1 &= s_1 t_{11} = [0 \ 0 \ 0 \ 1][1 \ 1 \ 1 \ 1] \\ x_1 &= [0 \ 0 \ 0 \ 1] \end{aligned}$$

Step 10: Calculate norm of x :

$$\|x\| = 1$$

Step 11: Test for reset condition:

$$\frac{\|x\|}{\|s\|} = \frac{1}{1} = 1.0 \geq 0.4 (\rho)$$

Hence reset is false. Proceed to Step 12.

Step 12: • Update bottom-up weights for $\alpha = 2$:

$$b_{ij}(\text{new}) = \frac{\alpha x_i}{\alpha - 1 + \|x\|} = \frac{2x_i}{2 - 1 + \|x\|} = \frac{2x_i}{1 + \|x\|}$$

$$b_{11} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{21} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{31} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{41} = \frac{2 \times 1}{1 + 1} = \frac{2}{2} = 1$$

Therefore, the bottom-up weight matrix b_{ij} becomes

$$b_{ij} = \begin{bmatrix} 0 & 0.2 & 0.2 \\ 0 & 0.2 & 0.2 \\ 0 & 0.2 & 0.2 \\ 1 & 0.2 & 0.2 \end{bmatrix}$$

• Update the top-down weights:

$$t_{ji}(\text{new}) = x_i$$

$$t_{ji} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Steps 0 and 1 remain the same.

Step 2: For the second input vector $[0 \ 1 \ 0 \ 1]$, perform Steps 3–12.

Step 3: Set activations of all F_2 units to zero. Set activations of $F_1(a)$ units to input vector $s = [0 \ 1 \ 0 \ 1]$.

Step 4: Compute norm of s :

$$\|s\| = 0 + 1 + 0 + 1 = 2$$

Step 5: Compute activations of each node in the F_1 layer:

$$x = [0 \ 1 \ 0 \ 1]$$

Step 6: Compute net input to each node in the F_2 layer:

$$y_j = \sum_{i=1}^4 b_{ij}x_i$$

For $j = 1$ to 3,

$$y_1 = 0(0) + 0(1) + 0(0) + 1(1) = 1$$

$$y_2 = 0.2(0) + 0.2(1) + 0.2(0) + 0.2(1) = 0.4$$

$$y_3 = 0.2(0) + 0.2(1) + 0.2(0) + 0.2(1) = 0.4$$

Step 7: When reset is true, perform Steps 8–11.

Step 8: The unit with largest net input is the winner, i.e., $j = 1$.

Step 9: Recompute F_1 activations (for $j = 1$):

$$x_i = s_i t_{ji} = [0 \ 1 \ 0 \ 1][0 \ 0 \ 0 \ 1] = [0 \ 0 \ 0 \ 1]$$

Step 10: Calculate norm of x :

$$\|x\| = 0 + 0 + 0 + 1 = 1$$

Step 11: Test for reset condition:

$$\frac{\|x\|}{\|s\|} = \frac{1}{2} = 0.5 \geq 0.4 (\rho)$$

Hence reset is false. Proceed to Step 12.

Step 12: • Update bottom-up weights for $\alpha = 2$:

$$b_{ij}(\text{new}) = \frac{2x_i}{1 + \|x\|}$$

$$b_{11} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{21} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{31} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{41} = \frac{2 \times 1}{1 + 1} = \frac{2}{2} = 1$$

Therefore, the bottom-up weight matrix b_{ij} becomes

$$b_{ij} = \begin{bmatrix} 0 & 0.2 & 0.2 \\ 0 & 0.2 & 0.2 \\ 0 & 0.2 & 0.2 \\ 1 & 0.2 & 0.2 \end{bmatrix}$$

• Update the top-down weights:

$$t_{ji}(\text{new}) = x_i$$

$$t_{ji} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Steps 0 and 1 remain the same.

Step 2: For the third input vector $[0 \ 0 \ 1 \ 1]$, perform Steps 3–12.

Step 3: Set activations of all F_2 units to zero. Set activations of $F_1(a)$ units to input vector $s = [0 \ 0 \ 1 \ 1]$.

Step 4: Compute norm of s :

$$\|s\| = 0 + 0 + 1 + 1 = 2$$

Step 5: Compute activations of each node in the F_1 layer:

$$x = [0 \ 0 \ 1 \ 1]$$

Step 6: Compute net input to each node in the F_2 layer:

$$y_j = \sum_{i=1}^4 b_{ij}x_i$$

For $j = 1$ to 3,

$$y_1 = 0(0) + 0(0) + 0(1) + 1(1) = 1$$

$$y_2 = 0.2(0) + 0.2(0) + 0.2(1) + 0.2(1) = 0.4$$

$$y_3 = 0.2(0) + 0.2(0) + 0.2(1) + 0.2(1) = 0.4$$

Step 7: When reset is true, perform Steps 8–11.

Step 8: The unit with largest net input is the winner, i.e., $j = 1$.

Step 9: Recompute F_1 activations (for $j = 1$):

$$x_i = s_i t_{ji} = [0 \ 0 \ 1 \ 1][0 \ 0 \ 0 \ 1] = [0 \ 0 \ 0 \ 1]$$

Step 10: Calculate norm of x :

$$\|x\| = 0 + 0 + 0 + 1 = 1$$

Step 11: Test for reset condition:

$$\frac{\|x\|}{\|s\|} = \frac{1}{2} = 0.5 \geq 0.4 (\rho)$$

Hence reset is false. Proceed to Step 12.

Step 12: • Update bottom-up weights for $\alpha = 2$:

$$b_{ij}(\text{new}) = \frac{2x_i}{1 + \|x\|}$$

$$b_{11} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{21} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{31} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{41} = \frac{2 \times 1}{1 + 1} = \frac{2}{2} = 1$$

Therefore, the bottom-up weight matrix b_{ij} becomes

$$b_{ij} = \begin{bmatrix} 0 & 0.2 & 0.2 \\ 0 & 0.2 & 0.2 \\ 0 & 0.2 & 0.2 \\ 1 & 0.2 & 0.2 \end{bmatrix}$$

• Update the top-down weights:

$$t_{ji}(\text{new}) = x_i$$

$$t_{ji} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Steps 0 and 1 remain the same.

Step 2: For the fourth input vector [1 0 0 0], perform Steps 3–12.

Step 3: Set activations of all F_2 units to zero. Set activations of $F_1(a)$ units to input vector $s = [1 \ 0 \ 0 \ 0]$.

Step 4: Compute norm of s :

$$\|s\| = 1 + 0 + 0 + 0 = 1$$

Step 5: Compute activations of each node in the F_1 layer:

$$x = [1 \ 0 \ 0 \ 0]$$

Step 6: Compute net input to each node in the F_2 layer:

$$y_j = \sum_{i=1}^4 b_{ij}x_i$$

For $j = 1$ to 3,

$$y_1 = 0(1) + 0(0) + 0(0) + 1(0) = 0$$

$$y_2 = 0.2(1) + 0.2(0) + 0.2(0) + 0.2(0) = 0.2$$

$$y_3 = 0.2(1) + 0.2(0) + 0.2(0) + 0.2(0) = 0.2$$

Step 7: When reset is true, perform Steps 8–11.

Step 8: The unit with largest net input is the winner, i.e., $j = 2$.

Step 9: Recompute F_1 activations:

$$x_i = s_i t_{ji} = [1 \ 0 \ 0 \ 0][1 \ 1 \ 1 \ 1] = [1 \ 0 \ 0 \ 0]$$

Step 10: Calculate norm of x :

$$\|x\| = 1 + 0 + 0 + 0 = 1$$

Step 11: Test for reset condition:

$$\frac{\|x\|}{\|s\|} = \frac{1}{1} = 1 \geq 0.4 (\rho)$$

Hence reset is false. Proceed to Step 12.

Step 12: • Update bottom-up weights for $\alpha = 2$:

$$b_{ij}(\text{new}) = \frac{2x_i}{1 + \|x\|}$$

$$b_{11} = \frac{2 \times 1}{1 + 1} = \frac{2}{2} = 1;$$

$$b_{21} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{31} = \frac{2 \times 0}{1 + 1} = 0;$$

$$b_{41} = \frac{2 \times 0}{1 + 1} = 0$$

Therefore, the bottom-up weight matrix b_{ij} becomes

$$b_{ij} = \begin{bmatrix} 0 & 1 & 0.2 \\ 0 & 0 & 0.2 \\ 0 & 0 & 0.2 \\ 1 & 0 & 0.2 \end{bmatrix}$$

• Update the top-down weights:

$$t_{ji}(\text{new}) = x_i$$

$$t_{ji} = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Step 13: Test for stopping condition. (This completes one epoch of training.)

The network may be trained for a particular number of epochs on the basis of the stopping condition.

12. Consider an ART 1 neural net with four F_1 units and three F_2 units. After some training, the weights are as follows:

Bottom-up weights	Top-down weights
$b_{ij} = \begin{bmatrix} 0.67 & 0 & 0.2 \\ 0 & 0 & 0.2 \\ 0 & 0 & 0.2 \\ 0 & 0.67 & 0.2 \end{bmatrix}_{4 \times 3}$	$t_{ji} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}_{3 \times 4}$

Determine the new weight matrices after the vector [0, 0, 1, 1] is presented if

- The vigilance parameter is 0.3.
- The vigilance parameter is 0.7.

Solution: It can be noted that $n = 4$, $m = 3$ (clusters) and $\alpha = 2$.

- Vigilance parameter $\rho = 0.3$.

Bottom-up weight,

$$b_{ij}(0) = \frac{1}{1 + n} = \frac{1}{1 + 4} = 0.2$$

Top-down weight

$$t_{ji}(0) = 1$$

Note: These are not necessary as b_{ij} and t_{ji} weights are already given.

We now compute norm of $s = [0 \ 0 \ 1 \ 1]$:

$$\|s\| = 0 + 0 + 1 + 1 = 2$$

Then we compute the activations of F_1 layer:

$$x = [0 \ 0 \ 1 \ 1]$$

Now, calculate the net input:

$$y_j = \sum_{i=1}^4 x_i b_{ij}$$

$$y_1 = \sum_{i=1}^4 x_i b_{i1} = 0.67 \times 0 + 0 \times 0 + 1 \times 0 + 1 \times 0 = 0$$

$$y_2 = \sum_{i=1}^4 x_i b_{i2} = 0 \times 0 + 0 \times 0 + 1 \times 0 + 1 \times 0.67 = 0.67$$

$$y_3 = \sum_{i=1}^4 x_i b_{i3} = 0 \times 0.2 + 0 \times 0.2 + 1 \times 0.2 + 1 \times 0.2 = 0.4$$

Since y_2 is the largest, hence the winner unit is $j = 2$.

Compute F_1 activations again,

$$x_i = s_i t_{ji} = [0 \ 0 \ 1 \ 1][0 \ 0 \ 0 \ 1] = [0 \ 0 \ 0 \ 1]$$

Computing the norm of x we obtain

$$\|x\| = 0 + 0 + 0 + 1 = 1$$

We now test for reset. Since

$$\frac{\|x\|}{\|s\|} = \frac{1}{2} = 0.5 \geq 0.3 (\rho)$$

we update the weights. Update bottom-up weights (b_{ij}),

$$b_{ij}(\text{new}) = \frac{\alpha x_i}{\alpha - 1 + \|x\|} \quad (\alpha = 2)$$

$$b_{12} = \frac{2 \times 0}{2 - 1 + 1} = 0;$$

$$b_{22} = \frac{2 \times 0}{2 - 1 + 1} = 0;$$

$$b_{32} = \frac{2 \times 0}{2 - 1 + 1} = 0;$$

$$b_{42} = \frac{2 \times 1}{2 - 1 + 1} = 1$$

Update top-down weights, $t_{ji}(\text{new}) = x_i$.

The new top-down weights are

$$t_{ji} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

The new bottom-up weights are

$$b_{ij} = \begin{bmatrix} 0.67 & 0 & 0.2 \\ 0 & 0 & 0.2 \\ 0 & 0 & 0.2 \\ 0 & 1 & 0.2 \end{bmatrix}$$

- Vigilance parameter $\rho = 0.7$. The input vector is $s = [0 \ 0 \ 1 \ 1]$. Now calculate norm of s ,

$$\|s\| = 0 + 0 + 1 + 1 = 2$$

Set activations of F_1 layer as

$$x = [0 \ 0 \ 1 \ 1]$$

Calculating the net input we obtain

$$y_j = \sum_{i=1}^4 x_i b_{ij}$$

$$y_1 = \sum_{i=1}^4 x_i b_{i1}$$

$$= 0.67 \times 0 + 0 \times 0 + 1 \times 0 + 1 \times 0 = 0$$

$$y_2 = \sum_{i=1}^4 x_i b_{i2} = 0 \times 0 + 0 \times 0 + 1 \times 0 + 1 \times 0.67 = 0.67$$

$$y_3 = \sum_{i=1}^4 x_i b_{i3} = 0 \times 0.2 + 0 \times 0 + 1 \times 0.2 + 1 \times 0.2 = 0.4$$

As y_2 is the largest, therefore the winner unit index is $j = 2$.

Recomputing the activations of F_1 layer we get ($j = 2$)

$$x_i = s_i t_{ji} = [0 \ 0 \ 1 \ 1][0 \ 0 \ 0 \ 1] = [0 \ 0 \ 0 \ 1]$$

The norm of x is

$$\|x\| = 0 + 0 + 0 + 1 = 1$$

We now test for reset condition. Since

$$\frac{\|x\|}{\|s\|} = \frac{1}{2} = 0.5 < 0.7 (\rho)$$

$y_2 = -1$ (inhibit node 2). Therefore, the net input becomes

$$y_1 = 0; \quad y_2 = -1; \quad y_3 = 0.4$$

As the largest is y_3 , the winner unit index is $j = 3$.

Recomputing F_1 layer activations, we get

$$x_i = s_i t_{ji} = [0 \ 0 \ 1 \ 1][1 \ 1 \ 1 \ 1] = [0 \ 0 \ 1 \ 1]$$

From this we get that the norm of $\|x\| = 2$. Testing for reset we obtain

$$\frac{\|x\|}{\|s\|} = \frac{2}{2} = 1 > 0.7 (\rho)$$

Hence we update the weights. The bottom-up weights are $x_i = [0 \ 0 \ 1 \ 1]$, $j = 3$

$$b_{ij}(\text{new}) = \frac{\alpha x_i}{\alpha - 1 + \|x\|} = \frac{2 \times 0}{2 - 1 + 2} = 0;$$

$$b_{23} = \frac{2 \times 0}{2 - 1 + 2} = 0;$$

$$b_{33} = \frac{2 \times 1}{2 - 1 + 2} = 0.67;$$

$$b_{43} = \frac{2 \times 1}{2 - 1 + 2} = 0.67$$

The updated bottom-up weights are

$$b_{ij} = \begin{bmatrix} 0.67 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0.67 \\ 0 & 0.67 & 0.67 \end{bmatrix}$$

The top-down weights are given by $t_{ji}(\text{new}) = x_i$. Hence the updated top-down weights are

$$t_{ji} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

13. Consider an ART 1 network with nine input F_1 units and two cluster F_2 units. After some training, the bottom-up weights b_{ij} and top-down weights t_{ji} have the following values:

The bottom-up weights

$$b_{ij} = \begin{bmatrix} 1/3 & 1/10 \\ 0 & 1/10 \\ 1/3 & 1/10 \\ 0 & 1/10 \\ 1/3 & 1/10 \\ 0 & 1/10 \\ 1/3 & 1/10 \\ 0 & 1/10 \\ 1/3 & 1/10 \end{bmatrix}$$

Top-down weights t_{ji}

$$t_{ji} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

The pattern $(1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1)$ is presented to the network. Compute the action of the network if

- the vigilance parameter is 0.5;
- the vigilance parameter is 0.8.

Solution: Here $n = 9$, $m = 2$.

- Vigilance parameter $\rho = 0.5$. The initial weight matrices are

$$b_{ij} = \begin{bmatrix} 1/3 & 1/10 \\ 0 & 1/10 \\ 1/3 & 1/10 \\ 0 & 1/10 \\ 1/3 & 1/10 \\ 0 & 1/10 \\ 1/3 & 1/10 \\ 0 & 1/10 \\ 1/3 & 1/10 \end{bmatrix}$$

$$t_{ji} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

The input pattern is $s = [1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1]$.

Calculating the norm of s , we obtain

$$\|s\| = \sum_{i=1}^9 s_i = 8$$

We now compute the activations of F_1 layer,

$$x = [1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1]$$

Calculating the net input, we obtain

$$y_j = \sum_{i=1}^9 x_i b_{ij}$$

$$y_1 = \sum_{i=1}^9 x_i b_{i1} = 1(1/3) + 1(0) + 1(1/3) + 1(0) + 0(1/3) + 1(0) + 1(1/3) + 1(0) + 1(1/3) = \frac{1}{3} + \frac{1}{3} + \frac{1}{3} + \frac{1}{3} = \frac{4}{3} = 1.3$$

$$y_2 = \sum_{i=1}^9 x_i b_{i2} = 1(1/10) + 1(1/10) + 1(1/10) + 1(1/10) + 0(1/10) + 1(1/10) + 1(1/10) + 1(1/10) + 1(1/10) = \frac{8}{10} = 0.8$$

It can be seen that $y_1 > y_2$, so the winner unit index is $j = 1$. Recomputing the activations of F_1 layer, we obtain (for $j = 1$)

$$x_i = s_i t_{ji} = [1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1]$$

$$[1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1]$$

$$x = [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1]$$

Calculating the norm of x , we obtain

$$\|x\| = 1 + 0 + 1 + 0 + 0 + 0 + 1 + 0 + 1 = 4$$

Testing for reset implies

$$\frac{\|x\|}{\|s\|} = \frac{4}{8} = \frac{1}{2} = 0.5 \geq 0.5 (\rho)$$

Reset is false. Hence we update the weights.

Updating bottom-up weights for $\alpha = 2$ we get

$$b_{ij}(\text{new}) = \frac{\alpha x_i}{\alpha - 1 + \|x\|} = \frac{2x_i}{2 - 1 + \|x\|} = \frac{2x_i}{1 + \|x\|}$$

$$b_{11} = \frac{2(1)}{1 + 4} = \frac{2}{5}; \quad b_{21} = \frac{2(0)}{1 + 4} = 0;$$

$$b_{31} = \frac{2(1)}{1 + 4} = \frac{2}{5}; \quad b_{41} = \frac{2(0)}{1 + 4} = 0;$$

$$b_{51} = \frac{2(0)}{1 + 4} = 0; \quad b_{61} = \frac{2(0)}{1 + 4} = 0;$$

$$b_{71} = \frac{2(1)}{1 + 4} = \frac{2}{5}; \quad b_{81} = \frac{2(0)}{1 + 4} = 0;$$

$$b_{91} = \frac{2(1)}{1 + 4} = \frac{2}{5}$$

The updated bottom-up weights are

$$b_{ij} = \begin{bmatrix} 2/5 & 1/10 \\ 0 & 1/10 \\ 2/5 & 1/10 \\ 0 & 1/10 \\ 0 & 1/10 \\ 0 & 1/10 \\ 2/5 & 1/10 \\ 0 & 1/10 \\ 2/5 & 1/10 \end{bmatrix}$$

We now update top-down weights using t_{ji} (new) = x_i . The new updated top-down weight are

$$t_{ji} = \begin{bmatrix} 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

- Vigilance parameter $\rho = 0.8$. The input pattern is $s = [1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1]$. Calculating the norm of s using the formula as in (1), we obtain

$$\|s\| = 8$$

We now compute the activations of F_1 layer,

$$x = [1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1]$$

Calculating the net input, we get

$$\begin{aligned} y_j &= \sum_{i=1}^9 x_i b_{ij} \\ y_1 &= \sum_{i=1}^9 x_i b_{i1} \\ &= 1(1/3) + 1(0) + 1(1/3) + 1(0) + 0(1/3) \\ &\quad + 1(0) + 1(1/3) + 1(0) + 1(1/3) \\ &= \frac{1}{3} + \frac{1}{3} + \frac{1}{3} + \frac{1}{3} + \frac{4}{3} = 1.3 \\ y_2 &= \sum_{i=1}^9 x_i b_{i2} \\ &= 1(1/10) + 1(1/10) + 1(1/10) + 1(1/10) \\ &\quad + 1(1/10) + 1(1/10) + 1(1/10) \\ &\quad + 1(1/10) + 1(1/10) + 1(1/10) \\ &= \frac{8}{10} = 0.8 \end{aligned}$$

It can be seen that $y_1 > y_2$. Hence the winner unit index is $J = 1$.

Recomputing the activations of F_1 layer, we obtain (for $J = 1$)

$$\begin{aligned} x_i &= s_i t_{ji} = [1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1] \\ &\quad [1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1 \ 0 \ 1] \\ x &= [1 \ 0 \ 1 \ 0 \ 0 \ 0 \ 1 \ 0 \ 1] \end{aligned}$$

Computing the norm of x , we have

$$\|x\| = 1 + 0 + 1 + 0 + 0 + 0 + 1 + 0 + 1 = 4$$

Testing for reset we obtain

$$\frac{\|x\|}{\|s\|} = \frac{4}{8} = \frac{1}{2} = 0.5 < 0.8$$

Hence $y_1 = -1$ (inhibit node 1). Therefore, the dot products become

$$y_1 = -1; \quad y_2 = 0.8$$

Since $y_2 > y_1$, the winner unit index is $J = 2$. Recomputing activations of F_1 layer (for $J = 2$) gives

$$\begin{aligned} x_i &= s_i t_{ji} = [1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1] \\ &\quad \times [1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1] \\ x &= [1 \ 1 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 1] \end{aligned}$$

Again computing the norm of x , we get

$$\|x\| = 1 + 1 + 1 + 1 + 0 + 1 + 1 + 1 + 1 = 8$$

Testing for reset gives

$$\frac{\|x\|}{\|s\|} = \frac{8}{8} = 1 > 0.8$$

Hence we update the weights.

Update bottom-up weights, for $\alpha = 2$, using the formula

$$\begin{aligned} b_{ij}(\text{new}) &= \frac{\alpha x_i}{\alpha - 1 + \|x\|} \\ &= \frac{2x_i}{2 - 1 + \|x\|} = \frac{2x_i}{1 + \|x\|} \end{aligned}$$

For all $x_i = 1$ (where $i = 1$ to 4 and 6 to 9, and $J = 2$),

$$b_{ij}(\text{new}) = \frac{2 \times 1}{1 + 8} = \frac{2}{9}$$

For $x_i = 0$ (where $i = 5$ and $J = 2$)

$$b_{ij}(\text{new}) = \frac{2 \times 0}{1 + 8} = 0$$

The updated bottom-up weights are

$$b_{ij} = \begin{bmatrix} 1/3 & 2/9 \\ 0 & 2/9 \\ 1/3 & 2/9 \\ 0 & 2/9 \\ 1/3 & 0 \\ 0 & 2/9 \\ 1/3 & 2/9 \\ 0 & 2/9 \\ 1/3 & 2/9 \end{bmatrix}$$

Updated top-down weights can be calculated using the formula $t_{ji}(\text{new}) = x_i$.

The new updated top-down weights are

$$t_{ji} = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}$$

14. Consider an ART 2 network with two input units. ($n = 2$). Show that using $\theta = 0.7$ will force the input patterns (0.71, 0.69) and (0.69, 0.71) to different clusters. What role does the vigilance parameters play in this case? (Do not calculate the weights, stop with checking of reset condition.) Assume the necessary parameters.

Solution: The parameters are assumed to be

$$a = b = 10, \quad c = 0.1, \quad d = 0.9, \quad e = 0, \quad \alpha = 0.6,$$

$$\rho = 0.9, \quad \theta = 0.7, \quad n = 2, \quad t_j = (0, 0),$$

$$b_j = \frac{1}{(1 - \rho)^n} = (7.0, 7.0)$$

1st pattern:

$$s = (0.71, 0.69)$$

$$u = \frac{v}{e + \|v\|} = (0, 0)$$

$$w = s + au = (0.71, 0.69)$$

$$\rho = u + dt_j = (0, 0)$$

$$x = \frac{w}{e + \|w\|} = \frac{(0.71, 0.69)}{0.99} = (0.717, 0.697)$$

$$(\text{where } \|w\| = \sqrt{(0.71)^2 + (0.69)^2} = 0.99)$$

$$q = \frac{\rho}{e + \|\rho\|} = (0, 0)$$

$$\begin{aligned} v &= f(x_i) + bf(q_i) \\ &= f(0.717, 0.697) \\ v &= (0.72, 0) \end{aligned}$$

Here the activation function is

$$f(x) = \begin{cases} x & f(x) \geq \theta \\ 0 & f(x) < \theta \end{cases}$$

Since $\theta = 0.7$, the output $v = (0.72, 0)$. Update F_1 activations again:

$$u = \frac{v}{e + \|v\|} = \frac{(0.72, 0)}{0.72} = (1, 0)$$

$$w = s + au = (0.71, 0.69) + 10(1, 0) = (10.71, 0.69)$$

$$\rho = u + dt_j = (1, 0)$$

$$x = \frac{w}{e + \|w\|} = \frac{(10.71, 0.69)}{10.732} = (0.998, 0.064)$$

$$q = \frac{\rho}{e + \|\rho\|} = \frac{(1, 0)}{1} = (1, 0)$$

$$\begin{aligned} v_i &= f(x_i) + bf(q_i) \\ &= f(0.998, 0.064) + 10f(1, 0) \\ &= (0.998, 0) + 10(1, 0) \\ v_i &= (10.998, 0) \end{aligned}$$

Calculating signals to F_2 units, we get

$$y_j = \sum b_{ij} \rho_i = (7, 7) \times (1, 0) = (7, 0)$$

The winner unit is $J = 1$, since y_1 has a larger value, i.e. 7, than $y_2 = 0$.

Check for reset:

$$u = \frac{v}{e + \|v\|} = \frac{(10.998, 0)}{10.998} = (1, 0)$$

$$\rho = u + dt_j = (1, 0) + 0.9(0, 0) = (1, 0)$$

$$\begin{aligned} r &= \frac{u + c\rho}{e + \|u\| + c\|\rho\|} = \frac{(1, 0) + 0.1(1, 0)}{0 + 1 + 0.1 \times 1} \\ &= \frac{(1.1, 0)}{1.1} = (1, 0) \end{aligned}$$

Computing norm of r , we get

$$\|r\| = 1 > (\rho - e) = 1 > 0.9$$

This implies

$$\begin{aligned}w &= s + au = (0.71, 0.69) + 10(1, 0) \\&= (10.71, 0.69) \\x &= \frac{w}{e + \|w\|} = (0.998, 0.064) \\q &= \frac{\rho}{e + \|\rho\|} = (1, 0) \\v &= f(x) + bf(q) = (10.998, 0)\end{aligned}$$

2nd pattern

$$\begin{aligned}s &= (0.69, 0.71) \\u &= \frac{v}{e + \|v\|} = 0 \\w &= s + au = (0.69, 0.71) \\p &= u + dt_j = (0, 0) \\x &= \frac{w}{e + \|w\|} = \frac{(0.69, 0.71)}{0.99} = (0.697, 0.717) \\q &= \frac{\rho}{e + \|\rho\|} = (0, 0) \\v &= f(x) + bf(q) = f(0.697, 0.717) \\&= (0, 0.717) = (0, 0.71)\end{aligned}$$

Update F₁ activations again:

$$\begin{aligned}u &= \frac{v}{e + \|v\|} = \frac{(0, 0.72)}{0.72} = (0, 1) \\w &= s + au = (0.69, 0.71) + 10(0, 1) \\&= (0.69, 10.71) \\p &= u + dt_j = (0, 1) \\x &= \frac{w}{e + \|w\|} = \frac{(0.69, 10.71)}{10.732} = (0.064, 0.998) \\q &= \frac{\rho}{e + \|\rho\|} = \frac{(0, 1)}{1} = (0, 1) \\v &= f(x) + bf(q) = f(0.064, 0.998) + 10f(0, 1) \\&= (0, 0.998) + 10(0, 1) \\v &= (0, 10.998)\end{aligned}$$

Calculating signals to F₂ units, we get

$$y_j = \sum b_{ij} p_i = (7, 7) \times (0, 1) = (0, 7)$$

The winner unit is $j = 2$, since $y_2 > y_1$ [i.e., $7 > 0$]
Check for reset

$$\begin{aligned}u &= \frac{v}{e + \|v\|} = \frac{(0, 10.998)}{10.998} = (0, 1) \\p &= u + dt_j = (0, 1) \\r &= \frac{u + cp}{e + \|u\| + c\|\rho\|} = \frac{(0, 1) + 0.1(0, 1)}{0 + 1 + 0.1 \times 1} \\&= \frac{(0, 1.1)}{1.1} = (0, 1) \\\|r\| &= 1 > (\rho - e) = 0.9.\end{aligned}$$

Then,

$$\begin{aligned}w &= s + au = (0.69, 0.71) + 10(0, 1) \\&= (0.69, 10.71) \\x &= \frac{w}{e + \|w\|} = \frac{(0.69, 10.71)}{10.732} = (0.064, 0.998) \\q &= \frac{\rho}{e + \|\rho\|} = (0, 1) \\v &= f(x) + bf(q) = (0, 10.998)\end{aligned}$$

Thus the vigilance parameter assumed $\rho = 0.9$ does not affect the solutions for first and second pattern. It activates the same for both the input patterns.

15. Consider an ART 2 network to cluster the input vectors (0.6, 0.8, 0.0) and (0.8, 0.6, 0.0) together? When will it place (0.6, 0.2, 0.0) together with (0.0, 1.0, 0.0)? Use the noise suppression parameter value $\theta = 1/\sqrt{3} = 0.577$ and consider different values of the vigilance and different initial weights. Assume necessary parameters.

Solution: Case (i) Taking $\rho = 0.9$, presenting (0.6, 0.8, 0) and (0.8, 0.6, 0)

$$\begin{aligned}a &= 10, b = 10, c = 0.1, d = 0.9, e = 0, \rho = 0.9, \\\theta &= 0.577, b_j = \frac{1}{(1-d)\sqrt{n}} = \frac{1}{(1-0.9)\sqrt{3}} \\&= (5.0, 5.0, 5.0), t_j = (0, 0, 0), \alpha = 0.6\end{aligned}$$

1st pattern:

$$s = (0.6, 0.8, 0.0)$$

$$\begin{aligned}u &= \frac{v}{e + \|v\|} = (0, 0, 0) \\w &= s + au = (0.6, 0.8, 0.0) \\p &= u + dt_j = (0, 0, 0) \\x &= \frac{w}{e + \|w\|} = \frac{(0.6, 0.8, 0.0)}{0 + 1} = (0.6, 0.8, 0.0) \\q &= \frac{\rho}{e + \|\rho\|} = (0, 0, 0) \\v_i &= f(x_i) + bf(q_i) \\&= f(0.6, 0.8, 0) + 10f(0, 0, 0)\end{aligned}$$

$$\left[\begin{array}{l} \theta = 0.55 \\ f(x) = \begin{cases} x, & f(x) \geq 0 \\ 0, & f(x) < \theta \end{cases} \end{array} \right]$$

$$v_i = (0.6, 0.8, 0)$$

Update F₁ activations again:

$$\begin{aligned}u &= \frac{v_i}{e + \|v\|} = \frac{(0.6, 0.8, 0)}{0 + 1} = (0.6, 0.8, 0) \\w &= s + au = (0.6, 0.8, 0) + 10(0.6, 0.8, 0) \\&= (6.6, 8.8, 0) \\p &= u + dt_j = (0.6, 0.8, 0) \\x &= \frac{w}{e + \|w\|} = \frac{(0.6, 8.8, 0)}{0 + 11} = (0.6, 0.8, 0) \\q &= \frac{\rho}{e + \|\rho\|} = \frac{(0.6, 0.8, 0)}{0 + 1} = (0.6, 0.8, 0) \\v_i &= f(x_i) + bf(q_i) \\&= f(0.6, 0.8, 0) + 10f(0.6, 0.8, 0) \\&= (0.6, 0.8, 0) + (6, 8, 0) \\v_i &= (6.6, 8.8, 0)\end{aligned}$$

Calculate the signals to F₂ units:

$$\begin{aligned}y_j &= \sum b_{ij} p_i = (5.0 \times 0.6, 5.0 \times 0.8, 5.0 \times 0) \\&= (3.0, 4.0, 0)\end{aligned}$$

The winner unit index is $j = 2$, since the largest net input is 4.0.

Check for reset:

$$\begin{aligned}u &= \frac{v}{e + \|v\|} = \frac{(6.6, 8.8, 0)}{0 + 11} = (0.6, 0.8, 0) \\p_i &= u_i + dt_j \\&= (0.6, 0.8, 0) + 0.9(0, 0, 0) = (0.6, 0.8, 0) \\r &= \frac{u_i + cp_i}{e + \|u\| + c\|\rho\|} \\&= \frac{(0.6, 0.8, 0) + 0.1(0.6, 0.8, 0)}{0 + 1 + 0.1 \times 1} \\&= \frac{(0.66, 0.88, 0)}{1.1} = (0.6, 0.8, 0)\end{aligned}$$

Computing norm of r , we get

$$\begin{aligned}\|r\| &= 1 > (\rho - e) = 0.9 \\q &= \frac{\rho}{e + \|\rho\|} = \frac{(0.6, 0.8, 0)}{0 + 1} = (0.6, 0.8, 0) \\w &= \frac{w}{e + \|w\|} \\&= (0.6, 0.8, 0) + 10(0.6, 0.8, 0) = (6.6, 8.8, 0) \\x &= \frac{w}{e + \|w\|} = \frac{(6.6, 8.8, 0)}{0 + 11} = (0.6, 0.8, 0) \\v &= f(x) + bf(q) = (6.6, 8.8, 0)\end{aligned}$$

Update of weights: Weights are updated for winning unit $j = 2$.

$$\begin{aligned}t_{ji}(\text{new}) &= \alpha du_i + \{1 + \alpha d(d-1)\} t_{ji}(\text{old}) \\&= 0.6 \times 0.9 \times 0.8 \\&\quad + \{1 + 0.6 \times 0.9(0.9 - 1)\} \times 0 \\&= 0.432 \\b_{ij}(\text{new}) &= \alpha du_i + \{1 + \alpha d(d-1)\} b_{ij}(\text{old}) \\&= 0.6 \times 0.9 \times 0.8 \\&\quad + \{1 + 0.6 \times 0.9(0.9 - 1)\} \times 5.0 \\&= 5.162\end{aligned}$$

2nd pattern:

$$\begin{aligned}s &= (0.8, 0.6, 0) \\u &= \frac{v}{e + \|v\|} = (0, 0) \\w &= s + au = (0.8, 0.6, 0) \\p &= u + dt_j = (0, 0)\end{aligned}$$

$$x = \frac{w}{e + \|w\|} = (0.8, 0.6, 0.0)$$

$$q = \frac{\rho}{e + \|\rho\|} = (0, 0, 0)$$

$$v = f(x_i) + bf(q_i)$$

$$= f(0.8, 0.6, 0.0) + 0 = (0.8, 0.6, 0)$$

Update F_1 activations again:

$$u = \frac{v}{e + \|v\|} = \frac{(0.8, 0.6, 0)}{0 + 1} = (0.8, 0.6, 0)$$

$$w = s + qu = (0.8, 0.6, 0) + 10(0.8, 0.6, 0)$$

$$= (8.8, 6.6, 0)$$

$$\rho = u + dt_j = (0.8, 0.6, 0)$$

$$x = \frac{w}{e + \|w\|} = (0.8, 0.6, 0)$$

$$q = \frac{\rho}{e + \|\rho\|} = (0.8, 0.6, 0)$$

$$v = f(x) + bf(q)$$

$$= f(0.8, 0.6, 0) + 10f(0.8, 0.6, 0)$$

$$= (0.8, 0.6, 0) + (8, 6, 0)$$

$$v = (8.8, 6.6, 0)$$

Calculate signals to F_2 units:

$$y_j = \sum b_{ij} \rho_i = (5, 5, 0) \times (0.8, 0.6, 0) = (4, 3, 0)$$

The winner unit index is $J = 1$, since the largest net input is 4.0.

Check for reset:

$$u = \frac{v}{e + \|v\|} = \frac{(8.8, 6.6, 0)}{0 + 11} = (0.8, 0.6, 0)$$

$$\rho_i = u + dt_j = (0.8, 0.6, 0)$$

$$r_i = \frac{u_i + c\rho_i}{e + \|u\| + c\|\rho\|} = (0.8, 0.6, 0)$$

Computing norm of r , we get

$$\|r\| = 1 > (\rho - e) = 0.9$$

$$q = \frac{\rho}{e + \|\rho\|} = \frac{(0.8, 0.6, 0)}{0 + 1} = (0.8, 0.6, 0)$$

$$w = s + au = (8.8, 6.6, 0)$$

$$x = \frac{w}{e + \|w\|} = (0.8, 0.6, 0)$$

$$v = f(x) + bf(q) = (8.8, 6.6, 0)$$

Update of weights: Weights are updated for winning unit $J = 1$.

$$t_{ij}(\text{new}) = \alpha du_i + \{1 + \alpha d(d - 1)\} t_{ij}(\text{old})$$

$$= 0.6 \times 0.9 \times 0.8$$

$$+ \{1 + 0.6 \times 0.9(0.9 - 1)\} \times 0$$

$$= 0.432$$

$$b_{ij}(\text{new}) = \alpha du_i + \{1 + \alpha d(d - 1)\} b_{ij}(\text{old})$$

$$= 0.6 \times 0.9 \times 0.8$$

$$+ \{1 + 0.6 \times 0.9(0.9 - 1)\} \times 5.0$$

$$= 5.162$$

Thus from the results for presenting the patterns (0.6, 0.8, 0) and (0.6, 0.8, 0) with the assumption of $\rho = 0.9$, even though the winning cluster unit is different, due to the components of input vector, output weights remains same. Thus they are placed together—since both the weights are same, both the patterns will be placed at the same location.

Case (ii): Taking $\rho = 0.7$, presenting (0.6, 0.8, 0) and (0, 1, 0).

$$a = 10, b = 10, c = 0.1, d = 0.9, e = 0, \rho = 0.7,$$

$$\theta = 0.577, \alpha = 0.6, b_j = (6.0, 6.0, 6.0),$$

$$t_j = (0, 0, 0)$$

1st pattern

$$s = (0.6, 0.8, 0.0)$$

$$u = \frac{v}{e + \|v\|} = (0, 0, 0)$$

$$w = s + au = (0.6, 0.8, 0)$$

$$\rho = u + dt_j = (0, 0, 0)$$

$$x = \frac{w}{e + \|w\|} = (0.6, 0.8, 0.0)$$

$$q = \frac{\rho}{e + \|\rho\|} = (0, 0, 0)$$

$$v_i = f(x_i) + bf(q_i) = (0.6, 0.8, 0)$$

Update F_1 activations again:

$$u = \frac{v}{e + \|v\|} = (0.6, 0.8, 0)$$

$$w = s + au = (6.6, 8.8, 0)$$

$$\rho = u + dt_j = (0.6, 0.8, 0)$$

$$x = \frac{w}{e + \|w\|} = (0.6, 0.8, 0)$$

$$q = \frac{\rho}{e + \|\rho\|} = (0.6, 0.8, 0)$$

$$v = f(x_i) + bf(q_i) = (6.6, 8.8, 0)$$

Calculate the signals to F_2 units:

$$y_j = \sum b_{ij} \rho_i = (6 \times 0.6, 6 \times 0.8, 6 \times 0)$$

$$= (3.6, 4.8, 0)$$

The winner unit index is $J = 2$, since the largest net input is 4.8.

Check for reset:

$$u = \frac{v}{e + \|v\|} = \frac{(6.6, 8.8, 0)}{0 + 11} = (0.6, 0.8, 0)$$

$$\rho_i = u + dt_j = (0.6, 0.8, 0)$$

$$r = \frac{u + c\rho}{e + \|u\| + c\|\rho\|} = (0.6, 0.8, 0)$$

$$\|r\| = 1 > (\rho - e) = 0.7$$

$$q = \frac{\rho}{e + \|\rho\|} = \frac{(0.6, 0.8, 0)}{0 + 1} = (0.6, 0.8, 0)$$

$$w = s + au = (6.6, 8.8, 0)$$

$$x = \frac{w}{e + \|w\|} = (0.6, 0.8, 0.0)$$

$$v = f(x) + bf(q) = (6.6, 8.8, 0)$$

Update of weights: Weights are updated for winning unit $J = 2$.

$$t_{ij}(\text{new}) = \alpha du_i + \{1 + \alpha d(d - 1)\} t_{ij}(\text{old})$$

$$= 0.6 \times 0.9 \times 0.8 + 0$$

$$= 0.432$$

$$b_{ij}(\text{new}) = \alpha du_i + \{1 + \alpha d(d - 1)\} b_{ij}(\text{old})$$

$$= 0.6 \times 0.9 \times 0.8$$

$$+ \{1 + 0.6 \times 0.9(0.9 - 1)\} \times 6.0$$

$$= 6.108$$

2nd pattern:

$$s = (0, 1, 0)$$

$$u = \frac{v}{e + \|v\|} = (0, 0, 0)$$

$$w = s + au = (0, 1, 0)$$

$$\rho = u + dt_j = (0, 1, 0)$$

$$x = \frac{w}{e + \|w\|} = (0.0, 1.0, 0.0)$$

$$q = \frac{\rho}{e + \|\rho\|} = (0.0, 1.0, 0.0)$$

$$v = f(x_i) + bf(q_i) = (0.0, 1.0, 0.0)$$

Update F_1 activations again:

$$u = \frac{v}{e + \|v\|} = (0.0, 1.0, 0.0)$$

$$w = s + au = (0, 1, 0) + 10(0, 1, 0) = (0, 11, 0)$$

$$\rho = u + dt_j = (0, 1, 0)$$

$$x = \frac{w}{e + \|w\|} = (0, 1, 0)$$

$$q = \frac{\rho}{e + \|\rho\|} = (0, 1, 0)$$

$$v = f(x) + bf(q) = (0, 1, 0) + (0, 10, 0)$$

$$= (0, 11, 0)$$

Calculating signals to F_2 units, we get

$$y_j = \sum b_{ij} \rho_i = (6.0, 6.0, 6.0)(0.0, 1.0, 0.0)$$

$$= (0.0, 6.0, 0.0)$$

The winner unit index is $J = 2$, since the largest net input is 6.0.

Check for reset:

$$u = \frac{v}{\epsilon + \|u\|} = \frac{(0, 11, 0)}{0 + 11} = (0, 1, 0)$$

$$\rho = u + d t_j = (0, 1, 0)$$

$$r = \frac{u + \epsilon \rho}{\epsilon + \|u\| + \epsilon \|\rho\|} = \frac{(0, 1, 0) + 0.1(0, 1, 0)}{0 + 1 + 0.1}$$

$$r = \frac{(0.0, 1.1, 0.0)}{0 + 1.1} = (0.0, 1.0, 0.0)$$

$$\|r\| = 1 > (\rho - \epsilon) = 0.7$$

$$q = \frac{\rho}{\epsilon + \|\rho\|} = (0.0, 1.0, 0.0)$$

$$w = s + a u = (0.0, 1.0, 0.0) + 10(0.0, 1.0, 0.0) = (0, 11, 0)$$

$$x = \frac{w}{\epsilon + \|w\|} = (0, 1, 0)$$

$$v = f(x) + b f(q) = f(0, 1, 0) + 10 f(0, 1, 0) = (0, 11, 0)$$

Update of weights: Weights are updated for winning unit $J = 2$.

$$t_{ji}(\text{new}) = \alpha d u_i + \{1 + \alpha d(d - 1)\} t_{ji}(\text{old})$$

$$= 0.6 \times 0.9 \times 1 + 0 = 0.54$$

$$b_{ij}(\text{new}) = \alpha d u_i + \{1 + \alpha d(d - 1)\} b_{ij}(\text{old})$$

$$= 0.6 \times 0.9 \times 1$$

$$+ \{1 + 0.6 \times 0.9(0.9 - 1)\} \times 6 = 6.216$$

Thus the two inputs may be clustered together only when their weights become equal, this can be achieved by proper selection of initial weights.

5.9 Review Questions

- What is meant by unsupervised learning?
- Define exemplar vector or code book vector.
- List the fixed weight competitive nets.
- Draw the architecture of Mexican hat and state its activation function.
- What is winner-takes-all or clustering principle or competitive learning?
- Why inhibitory weights are used in Max net?
- What are "topology preserving" maps?
- Define Euclidean distance.
- Briefly discuss about Hamming net.
- How is competition performed for clustering of the vectors?
- State the application of Kohonen self-organizing maps.
- With near architecture, explain the training algorithm of Kohonen self-organizing feature maps.
- Discuss the important features of Kohonen self-organizing maps.
- Write the principle involved in learning vector quantization.
- What is the purpose of LVQ net?
- How are the initial weights determined for LVQ net?
- With architecture, describe how LVQ nets are trained.
- List the variants of LVQ net.
- State Kohonen's learning rule and Grossberg learning rule.
- Discuss the applications of counterpropagation network.
- How many stages are needed for training a CPN network?
- Mention the importance of in-star model and out-star model.
- Sketch the architecture of full Counter Propagation Network.
- How are CPN nets used for function approximation?

- Write the training algorithms and testing algorithms used in full counterpropagation network.
- Compare full counterpropagation network and forward-only counterpropagation network.
- What is the principle strength of competitive learning?
- State the merits and demerits of Kohonen self-organizing feature maps.
- What are called as similarity maps?
- Define stability and plasticity.
- Differentiate between ART networks and CPN networks.
- List the type of input patterns given to ART 1 and ART 2 network.
- Mention the three main components of an ART network.
- Define bottom-up weight and top-down weight.
- What is vigilance parameter and noise suppression parameter?
- Illustrate with neat figure, the two basic units of an ART 1 network.
- Discuss the importance of supplemental units in ART 1 network.
- Differentiate fast learning and slow learning.
- List the advantages and disadvantages of ART network.
- What are the applications of ART networks?
- Sketch the architecture of ART 1 network and discuss its training algorithm.
- State the significance of ART 2 network.
- Why more complexity is involved in the F_1 layer of ART 2 network?
- How slow learning and fast learning is achieved in ART 2 network?
- What is the activation function used in ART 2 networks?
- List the characteristics of ART network.
- Why reset mechanism is essential in ART networks?
- With neat architecture, explain the training algorithm used in ART network.
- State the assumptions made in ART 2 network.
- Mention the limitation of ART 1 network and how is it overcome in ART 2 network.

5.10 Exercise Problems

- Construct a Max net with four neurons and inhibitory weights $E = 0.25$ when given the initial activations (input signals). The initial activations are $a_1(0) = 0.1$, $a_2(0) = 0.3$, $a_3(0) = 0.4$, $a_4(0) = 0.7$.
- Construct a Kohonen self-organizing feature map to cluster four vectors $[0 \ 0 \ 1]$, $[1 \ 0 \ 0]$, $[0 \ 1 \ 0]$, $[1 \ 1 \ 1]$. The maximum number of clusters to be formed is 2 and assume learning rate as 0.5. Assume random initial weights.
- Given a Kohonen self-organizing map with weights as shown in the following figure, use square of euclidean distance to find the cluster unit that is close to the input vector $(0.35,$

$0.05)$. Using learning rate of 0.25, find the new weights.

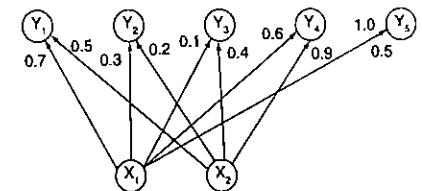


Figure 11 KSOFM net.

- Repeat the preceding exercise problem for input vector $[0.4, 0.4]$ with $\alpha = 0.15$.

5. Consider a Kohonen net with two cluster units and five input units. The weights vectors for the cluster units are

$$w_1 = (1.0 \ 0.9 \ 0.7 \ 0.3 \ 0.2)$$

$$w_2 = (0.6 \ 0.7 \ 0.5 \ 0.4 \ 1.0)$$

Use the square of the Euclidean distance to find the winning cluster unit for the input pattern $x = (0.0 \ 0.2 \ 0.1 \ 0.2 \ 0.0)$. Using a learning rate of 0.2, find the new weights for the winning unit.

6. Construct an LVQ net to cluster five vectors assigned to two classes. The following input vectors represent two classes 1 and 2.

Vectors	Class
(1 0 0 1)	1
(1 1 0 0)	2
(0 1 1 0)	1
(1 0 0 0)	2
(0 0 1 1)	1

Perform only one epoch of training.

7. Consider an LVQ net with two input units and four target classes: c_1, c_2, c_3 and c_4 . There are 16 classification units, with weight vectors indicated by the coordinates on the following chart, read in row-column order.

x_2					
1.0					
0.9	c_4	c_3	c_2	c_1	
0.7	c_2	c_1	c_4	c_3	
0.5	c_4	c_3	c_2	c_1	
0.3	c_2	c_1	c_4	c_3	
0.0					
	0.0	0.3	0.5	0.7	0.9
	x_1				

Using the square of the euclidean distance, perform the following.

- Present an input vector of (0.35, 0.35) representing class 3. Using a learning rate of

$\alpha = 0.4$, show which classification unit moves where.

- Present an input vector of (0.6, 0.75) representing class 1. What happens to the network performance?
- Present the vector (0.4, 0.55) representing class 1. Note what happens.

8. Implement a counterpropagation network for approximating the functions:

$$f(x) = 1/x$$

$$f(x) = 7x^2$$

9. Consider the following full CPN net:

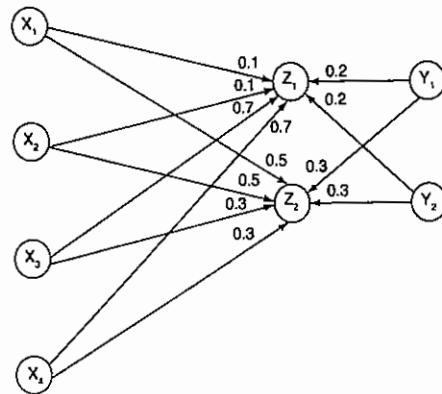


Figure 12 Full CPN Net.

Using the input pair $x = (1, 0, 0, 0), y = (0, 1)$, perform the first phase of training (one step only). Find the activation of the cluster layer units. Update the weights using a learning rate of 0.25.

10. Repeat Problem 9 using $x = (0 \ 1 \ 1 \ 1)$ and $y = (1, 0)$ with a learning rate of 0.3.
11. Modify Problem 9 to implement forward-only CPN.
12. Construct an ART 1 network to cluster four vectors (1, 0, 1, 1), (1, 1, 1, 0), (1, 0, 0, 0) and (0, 1, 0, 1) in at most three clusters using

very low vigilance parameter. Assume necessary parameters.

13. Consider an ART 1 neural net with four F_1 units and three F_2 units. After some training, the weights are as follows:

Bottom-up weights b_{ij}	Top-down weights t_{ji}
$\begin{bmatrix} 0.57 & 0 & 0.2 \\ 0 & 0 & 0.2 \\ 0 & 0.37 & 0.2 \\ 0 & 0.37 & 0.2 \end{bmatrix}$	$\begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$

Determine the new weight matrices after the vector (0, 0, 1, 1) is presented if

- the vigilance parameter is 0.4;
- the vigilance parameter is 0.8.

14. Consider an ART 1 network with eight input (F_1) units and two cluster (F_2) units. After some training, the bottom-up weight (b_{ij}) and top-down weights (t_{ji}) are the following

values:

Bottom-up weights b_{ij}	Top down weights t_{ji}
$\begin{bmatrix} 1/2 & 1/8 \\ 0 & 1/8 \\ 1/2 & 1/8 \\ 0 & 1/8 \\ 1/2 & 1/8 \\ 0 & 1/8 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$

The pattern [1 1 1 0 0 1 1 1] is presented to the network. Compute the action of the network if

- the vigilance parameter is 0.3;
- the vigilance parameter is 0.7.

15. Consider an ART 2 network with two input units ($n = 2$). Show that using $\theta = 0.7$ will force the input patterns (0.61, 0.59) and (0.59, 0.61) to different clusters. What role does vigilance parameter play here? Determine the new weights.

5.11 Projects

- Write a computer program to implement Kohonen self-organizing map. Take suitable application. Use 2 input units and 25 cluster units and a linear topology for the cluster units. Perform 20 epochs of training.
- Write a computer program to implement the LVQ net absorbed in Problem 7. Train the net with several sets of data. Experiment with different learning rates and different numbers of classification units.
- Write a program for counterpropagation network to approximate the function $f(x) = 1/x$.
- Implement counterpropagation network for performing data compression. Take data sets like heart disease data, cancer data and credit card data.
- Write a program to approximate the function $f(x) = 7/x$ using forward-only counterpropagation net.
- Let the digits 0, 1, 2, ..., 7 be represented as

0:	0	0	0	0	0	0	0	1
1:	0	0	0	0	0	0	1	0
2:	0	0	0	0	0	1	0	0
3:	0	0	0	0	1	0	0	0
4:	0	0	0	1	0	0	0	0
5:	0	0	1	0	0	0	0	0
6:	0	1	0	0	0	0	0	0
7:	1	0	0	0	0	0	0	0

Use forward-only and full counterpropagation nets to map digits to their binary representations

```
0: 0 0 0
1: 0 0 1
2: 0 1 0
3: 0 1 1
4: 1 0 0
5: 1 0 1
6: 1 1 0
7: 1 1 1
```

Assume the necessary parameters involved.

7. Write a computer program to implement full counterpropagation network for approximating the function $f(x) = 3x + 2$.
8. Build a computer program to implement ART 1 neural net.
9. Write a computer program to implement the ART 2 neural network, allowing for either fast learning or slow learning, based on the number of epochs of training and the number of weight update iterations performed on each learning trial.
10. Write a compute program to implement ART 2 network for Problem 15.

6

Special Networks

Learning Objectives

- The other special networks apart from supervised learning, unsupervised learning and association networks.
- A simulated annealing network.
- How Boltzmann machine can be used to solve optimization problems.
- An introduction to Cauchy and Gaussian machine.
- A probabilistic neural network.
- The feature of cascade correlation network to fluid its own architecture during training progresses.
- A cognitron and neocognitron nets with their basic features.
- Apart from these, cellular NN, logicon NN, STCNN are also discussed.
- List of neuroprocessor chips that are currently in use.

6.1 Introduction

In this chapter, we will discuss some specialized networks in more detail. Among the networks to be discussed are Boltzmann network, cascade correlation net, probabilistic neural net, Cauchy and Gaussian net, cognitron and neocognitron nets, spatio-temporal network, optical neural net, simulated annealing network, cellular neural net and logicon neural net. Besides, neuroprocessor chip has also been discussed for the benefit of the reader. Boltzmann network is designed for optimization problems, such as traveling salesman problem. In this network, fixed weights are used based on the constraints and quantity to be optimized. Probabilistic neural net is designed using the probability theory to classify the input data (Bayesian method). Cascade correlation net is designed depending on the hierarchical arrangement of the hidden units. Cauchy and Gaussian net is the variation of fixed weight optimization net. Cognitron and neocognitron nets were designed for recognition of handwritten digits.

6.2 Simulated Annealing Network

The concept of simulated annealing has its origin in the physical annealing process performed over metals and other substances. In metallurgical annealing, a metal body is heated almost to its melting point and then cooled back slowly to room temperature. This process eventually makes the metal's global energy function reach an absolute minimum value. If the metal's temperature is reduced quickly, the energy of the metallic lattice will be higher than this minimum value because of the existence of frozen lattice dislocations that would otherwise disappear due to thermal agitation. Analogous to the physical annealing behavior, simulated annealing can make a system change its state to a higher energy state having a chance to jump from local

minima to global minima. There exists a *cooling procedure* in the simulated annealing process such that the system has a higher probability of changing to an increasing energy state in the beginning phase of convergence. Then, as time goes by, the system becomes stable and always moves in the direction of decreasing energy state as in the case of normal minimization produce.

With simulated annealing, a system changes its state from the original state SA^{old} to a new state SA^{new} with a probability P given by

$$P = \frac{1}{1 + \exp(-\Delta E/T)}$$

where $\Delta E = E^{old} - E^{new}$ (energy change = difference in new energy and old energy) and T is the non-negative parameter (acts like temperature of a physical system). The probability P as a function of change in energy (ΔE) obtained for different values of the temperature T is shown in Figure 6-1. From Figure 6-1, it can be noticed that the probability when $\Delta E > 0$ is always higher than the probability when $\Delta E < 0$ for any temperature.

An optimization problem seeks to find some configuration of parameters $\hat{X} = (X_1, \dots, X_n)$ that minimizes some function $f(\hat{X})$ called cost function. In an artificial neural network, configuration parameters are associated with the set of weights and the cost function is associated with the error function.

The simulated annealing concept is used in statistical mechanics and is called Metropolis algorithm. As discussed earlier, this algorithm is based on a material that anneals into a solid as temperature is slowly decreased. To understand this, consider the slope of a hill having local valleys. A stone is moving down the hill. Here, the local valleys are local minima, and the bottom of the hill is going to be the global or universal minimum. It is possible that the stone may stop at a local minimum and never reaches the global minimum. In neural nets, this would correspond to a set of weights that correspond to that of local minimum, but this is not the desired solution. Hence, to overcome this kind of situation, simulated annealing perturbs the stone such that if it is trapped in a local minimum, it escapes from it and continues falling till it reaches its global minimum (optimal solution). At that point, further perturbations cannot move the stone to a lower position. Figure 6-2 shows the simulated annealing between a stone and a hill.

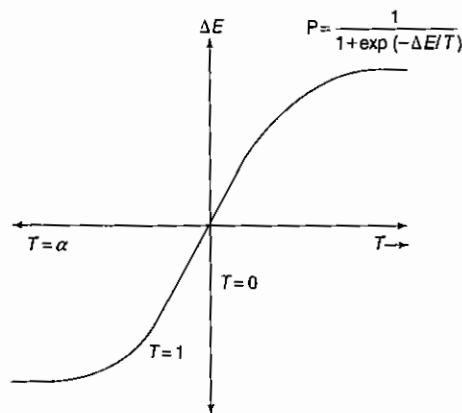


Figure 6-1 Probability "P" as a function of change in energy (ΔE) for different values of temperature T .

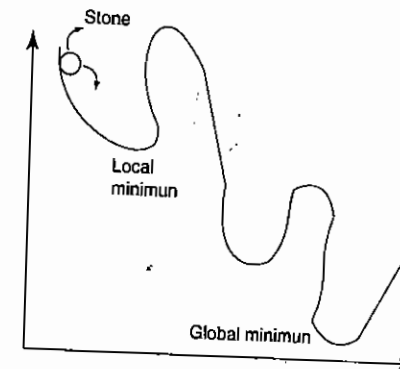


Figure 6-2 Simulated annealing—stone and hill.

The components required for annealing algorithm are the following.

1. **A basic system configuration:** The possible solution of a problem over which we search for a best (optimal) answer. (In a neural net, this is optimum steady-state weight.)
2. **The move set:** A set of allowable moves that permit us to escape from local minima and reach all possible configurations.
3. **A cost function** associated with the error function.
4. **A cooling schedule:** Starting of the cost function and rules to determine when it should be lowered and by how much, and when annealing should be terminated.

Simulated annealing networks can be used to make a network converge to its global minimum.

6.3 Boltzmann Machine

The early optimization technique used in artificial neural networks is based on the Boltzmann machine. When the simulated annealing process is applied to the discrete Hopfield network, it becomes a Boltzmann machine. The network is configured as the vector of the states of the units, and the states of the units are binary valued with probabilistic state transitions. The Boltzmann machine described in this section has fixed weights w_{ij} . On applying the Boltzmann machine to a constrained optimization problem, the weights represent the constraints of the problem and the quantity to be optimized. The discussion here is based on the fact of maximization of a consensus function (CF).

The Boltzmann machine consists of a set of units (X_i and X_j) and a set of bi-directional connections between pairs of units. This machine can be used as an associative memory. If the units X_i and X_j are connected, then $w_{ij} \neq 0$. There exists symmetry in the weighted interconnections based on the directional nature. It can be represented as $w_{ij} = w_{ji}$. There also may exist a self-connection for a unit (w_{ii}). For unit X_j , its state x_j may be either 1 or 0. The objective of the neural net is to maximize the CF given by

$$CF = \sum_i \sum_{j \leq i} w_{ij} x_i x_j$$

The maximum of the CF can be obtained by letting each unit attempt to change its state (alter between "1" and "0" or "0" and "1"). The change of state can be done either in parallel or sequential manner. However, in this case all the description is based on sequential manner. The consensus change when unit X_i changes its state is given by

$$\Delta CF(i) = (1 - 2x_i) \left(w_{ij} + \sum_{j \neq i} w_{ij} x_j \right)$$

where x_j is the current state of unit X_j . The variation in coefficient $(1 - 2x_i)$ is given by

$$(1 - 2x_i) = \begin{cases} +1, & X_i \text{ is currently off} \\ -1, & X_i \text{ is currently on} \end{cases}$$

If unit X_i were to change its activations then the resulting change in the CF can be obtained from the information that is local to unit X_i . Generally, X_i does not change its state, but if the states are changed, then this increases the consensus of the net. The probability of the network that accepts a change in the state for unit X_i is given by

$$AF(i, T) = \frac{1}{1 + \exp[-\Delta CF(i)/T]}$$

where T (temperature) is the controlling parameter and it will gradually decrease as the CF reaches the maximum value. Low values of T are acceptable because they increase the net consensus since the net accepts a change in state. To help the net not to stick with the local maximum, probabilistic functions are used widely.

6.3.1 Architecture

The architecture of a Boltzmann machine is represented through a two-dimensional array of the units in Figure 6-3. The units within each row and column are fully interconnected. The weights on the interconnections are given by $-p$ where $(p > 0)$. Also, there exists a self-connection for each unit, with weight $b > 0$. Unit X_{ij} is the common unit on which our discussion is based. The weights present on the interconnections are inhibitory.

6.3.2 Algorithm

6.3.2.1 Setting the Weights of the Network

The weights of a Boltzmann machine are fixed; hence there is no specific training algorithm for updation of weights. (For a Boltzmann machine with learning, there exists a training procedure.) With the Boltzmann machine weights remaining fixed, the net makes its transition toward maximum of the CF.

As shown in Figure 6-3, each unit is connected to every other unit in the same row and same column by the weights $-p$ ($p > 0$). The weights indicate the penalties obtained due to the violation that occurs when more than one unit is "on" in each row and column. There also exists a self-connection for each unit given by weight $b > 0$. The self-connection is a bonus given to the unit to turn on if it can do so without causing more than one unit to be on in each row and column. The net function will be desired if $p > b$. If unit X_{ij} is "off" and no other unit in its row or column is "on" then changing the status of X_{ij} to on increases the consensus of the net by amount b . This is an acceptable change because this increases the consensus. As a result, the net accepts it instead of rejecting it.

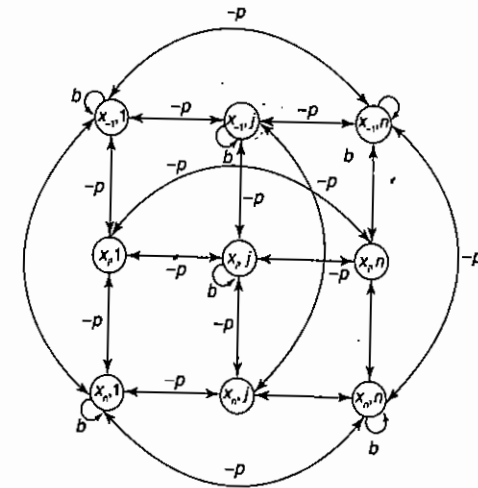


Figure 6-3 Architecture of Boltzmann machine.

On the other hand, if any one unit in each row or column of X_{ij} is on then changing the state of X_{ij} to on effects a change in consensus by $(b - p)$. Hence $(b - p) < 0$ makes $p > b$, i.e., the effect decreases the consensus. The net rejects this situation.

6.3.2.2 Testing Algorithm

It is assumed here that the units are arranged in a two-dimensional array. There are n^2 units. So, the weights between unit $X_{i,j}$ and unit $X_{l,j}$ are denoted by $w(i, j; l, j)$.

$$w(i, j; l, j) = \begin{cases} -p & \text{if } i = l \text{ or } j = j \text{ (but not both)} \\ b & \text{otherwise} \end{cases}$$

The testing algorithm is as follows.

- Step 0: Initialize the weights representing the constraints of the problem. Also initialize control parameter T and activate the units.
- Step 1: When stopping condition is false, perform Steps 2–8.
- Step 2: Perform Steps 3–6 n^2 times. (This forms an epoch.)
- Step 3: Integers I and J are chosen random values between 1 and n . (Unit $U_{I,J}$ is the current victim to change its state.)
- Step 4: Calculate the change in consensus:

$$\Delta CF = (1 - 2X_{I,J}) \left[w(I, J; I, J) + \sum_{i \neq I} \sum_{j \neq J} w(i, j; I, J) X_{i,j} \right]$$

Step 5: Calculate the probability of acceptance of the change in state:

$$AF(T) = 1/1 + \exp[-(\Delta CF/T)]$$

Step 6: Decide whether to accept the change or not. Let R be a random number between 0 and 1. If $R < AF$, accept the change:

$$X_{i,j} = 1 - X_{i,j} \text{ (This changes the state } U_{i,j}\text{)}$$

If $R \geq AF$, reject the change.

Step 7: Reduce the control parameter T .

$$T(\text{new}) = 0.95 T(\text{old})$$

Step 8: Test for stopping condition, which is:

If the temperature reaches a specified value or if there is no change of state for specified number of epochs then stop, else continue.

The initial temperature should be taken large enough for accepting the change of state quickly. Also remember that the Boltzmann machine can be applied for various optimization problems such as traveling salesman problem.

6.4 Gaussian Machine

Gaussian machine is one which includes Boltzmann machine, Hopfield net and other neural networks. The Gaussian machine is based on the following three parameters: (a) a slope parameter of sigmoidal function α , (b) a time step Δt , (c) temperature T .

The steps involved in the operation of the Gaussian net are the following:

Step 1: Compute the net input to unit X_i :

$$\text{net}_i = \sum_{j=1}^N w_{ij} v_j + \theta_i + \epsilon$$

where θ_i is the threshold and ϵ the random noise which depends on temperature T .

Step 2: Change the activity level of unit X_i :

$$\frac{\Delta x_i}{\Delta t} = -\frac{x_i}{\tau} + \text{net}_i$$

Step 3: Apply the activation function:

$$v_i = f(x_i) = 0.5[1 + \tanh(x_i)]$$

where the binary step function corresponds to $\alpha = \infty$ (infinity).

The Gaussian machine with $T = 0$ corresponds the Hopfield net. The Boltzmann machine can be obtained by setting $\Delta t = \tau = 1$ to get

$$\Delta x_i = -x_i + \text{net}_i$$

$$\text{or } x_i(\text{new}) = \text{net}_i = \sum_{j=1}^N w_{ij} v_j + \theta_i + \epsilon$$

The approximate Boltzmann acceptance function is obtained by integrating the Gaussian noise distribution

$$\int_0^{\infty} \frac{1}{\sqrt{2\pi}\sigma^2} \exp\left(-\frac{(x-x_i)^2}{2\sigma^2}\right) dx \approx AF(i, T) = \frac{1}{1 + \exp(-x_i/T)}$$

where $x_i = \Delta CF(i)$. The noise which is found to obey a logistic rather than a Gaussian distribution produces a Gaussian machine that is identical to Boltzmann machine having Metropolis acceptance function, i.e., the output set to 1 with probability,

$$AF(i, T) = \frac{1}{1 + \exp(-x_i/T)}$$

This does not bother about the unit's original state. When noise is added to the net input of a unit then using probabilistic state transition gives a method for extending the Gaussian machine into Cauchy machine.

6.5 Cauchy Machine

Cauchy machine can be called fast simulated annealing, and it is based on including more noise to the net input for increasing the likelihood of a unit escaping from a neighborhood of local minimum. Larger changes in the system's configuration can be obtained due to the unbounded variance of the Cauchy distribution. Noise involved in Cauchy distribution is called "colored noise" and the noise involved in the Gaussian distribution is called "white noise."

By setting $\Delta t = \tau = 1$, the Cauchy machine can be extended into the Gaussian machine, to obtain

$$\Delta x_i = -x_i + \text{net}_i$$

$$\text{or } x_i(\text{new}) = \text{net}_i = \sum_{j=1}^N w_{ij} v_j + \theta_i + \epsilon$$

The Cauchy acceptance function can be obtained by integrating the Cauchy noise distribution:

$$\int_0^{\infty} \frac{1}{\pi} \frac{T dx}{T^2 + (x - x_i)^2} = \frac{1}{2} + \frac{1}{\pi} \arctan\left(\frac{x_i}{T}\right) = AF(i, T)$$

where $x_i = \Delta CF(i)$. The cooling schedule and temperature have to be considered in both Cauchy and Gaussian machines.

6.6 Probabilistic Neural Net

The probabilistic neural net is based on the idea of conventional probability theory, such as Bayesian classification and other estimators for probability density functions, to construct a neural net for classification. This net instantly approximates optimal boundaries between categories. It assumes that the training

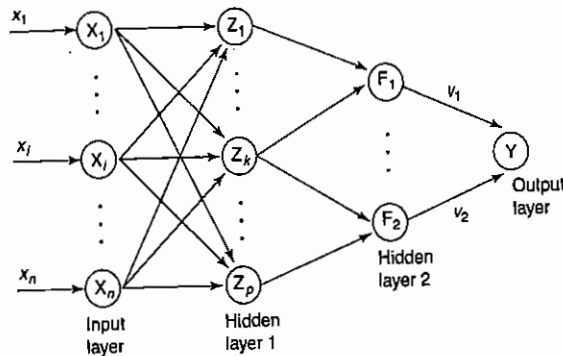


Figure 6-4 Probabilistic neural network.

data are original representative samples. The probabilistic neural net consists of two hidden layers as shown in Figure 6-4. The first hidden layer contains a dedicated node for each training pattern and the second hidden layer contains a dedicated node for each class. The two hidden layers are connected on a class-by-class basis, that is, the several examples of the class in the first hidden layer are connected only to a single matching unit in the second hidden layer.

During training process, the probabilistic neural net uses the training patterns for estimating the class probability distributions; each new input is classified according to the weighted average of the training sample which is very closer. The probabilistic neural net avoids the iterative process by simply storing the training patterns. Owing to this, probabilistic neural net learns very fast, but large networks are needed for large data sets.

The algorithm for the construction of the net is as follows:

Step 0: For each training input pattern $x(p)$, $p = 1$ to P , perform Steps 1 and 2.

Step 1: Create pattern unit z_k (hidden-layer-1 unit). Weight vector for unit z_k is given by

$$w_k = x(p)$$

Unit z_k is either z -class-1 unit or z -class-2 unit.

Step 2: Connect the hidden-layer-1 unit to the hidden-layer-2 unit.

If $x(p)$ belongs to class 1, then connect the hidden layer unit z_k to the hidden layer unit F_1 .

Otherwise, connect pattern hidden layer unit z_k to the hidden layer unit F_2 .

The net can be used for classification when an example of a pattern from each class has been presented to it. The net's ability for generalization improves when it is trained on more examples.

6.7 Cascade Correlation Network

Cascade correlation is a network which builds its own architecture as the training progresses. This algorithm was proposed by Fahlman and Lebiere in 1990. Figure 6-5 shows the cascade correlation architecture. The network begins with some inputs and one or more output nodes, but it has no hidden nodes. Each and every input

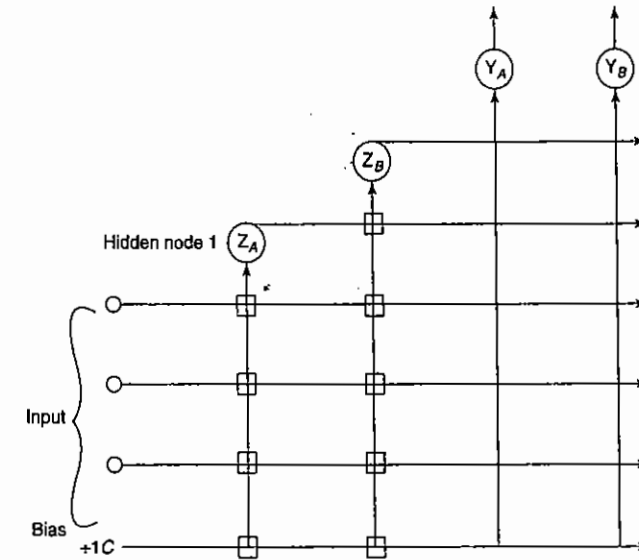


Figure 6-5 Cascade architecture after two hidden nodes have been added.

is connected to every output node. There may be linear units or some nonlinear activation function such as bipolar sigmoidal activation function in the output nodes. During training process, new hidden nodes are added to the network one by one. For each new hidden node, the correlation magnitude between the new node's output and the residual error signal is maximized. The connection is made to each node from each of the network's original inputs and also from every preexisting hidden node. During the time when the node is being added to the network, the input weights of the hidden nodes are frozen, and only the output connections are trained repeatedly. Each new node thus adds a new one-node layer to the network.

In Figure 6-5, the vertical lines sum all incoming activations. The rectangular boxed connections are frozen and "O" connections are trained continuously. In the beginning of the training, there are no hidden nodes, and the network is trained over the complete training set. Since there is no hidden node, a simple learning rule, Widrow-Hoff learning rule, is used for training. After a certain number of training cycles, when there is no significant error reduction and the final error obtained is unsatisfactory, we try to reduce the residual errors further by adding a new hidden node. For performing this task, we begin with a candidate node that receives trainable input connections from the network's external inputs and from all pre-existing hidden nodes. The output of this candidate node is not yet connected to the active network. After this, we run several numbers of epochs for the training set. We adjust the candidate node's input weights after each epoch to maximize C which is defined as

$$C = \sum_i \left| \sum_j (v_j - \bar{v})(E_{j,i} - \bar{E}_o) \right|$$

where i is the network output at which error is measured, j the training pattern, v the candidate node's output value, E_o the residual output error at node o , $\bar{v}$ the value of v averaged over all patterns, $\bar{E}_o$ the value of E_o

averaged over all patterns. The value " C " measures the correlation between the candidate node's output value and the calculated residual output error. For maximizing C , the gradient $\partial C / \partial w_i$ is obtained as

$$\frac{\partial C}{\partial w_i} = \sum_{j,i} \sigma_i (E_{j,i} - \bar{E}_i) a'_j I_{mj}$$

where σ_i is the sign of the correlation between the candidate's value and output i ; a'_j the derivative for pattern j of the candidate node's activation function with respect to sum of its inputs; I_{mj} the input the candidate node receives from node m for pattern j . When gradient $\partial C / \partial w_i$ is calculated, perform gradient ascent to maximize C . As we are training only a single layer of weights, simple delta learning rule can be applied. When C stops improving, again a new candidate can be brought in as a node in the active network and its input weights are frozen. Once again all the output weights are trained by the delta learning rule as done previously, and the whole cycle repeats itself until the error becomes acceptably small.

On the basis of this cascade correlation network, Fahlman (1991) proposed another training method for creating a recurrent network called the recurrent cascade correlation network. Its structure is same as shown in Figure 6-5, but each hidden node is a recurrent node, i.e., each hidden node has a connection to itself. Cascade correlation network is mainly suitable for classification problems. Even if modified, it can be used for approximation of functions.

6.8 Cognitron Network

Cognitron network was proposed by Fukushima in 1975. The learning hypothesis put forth by him is given in the following paragraphs.

The synaptic strength from cell X to cell Y is reinforced if and only if the following two conditions are true:

1. Cell X – presynaptic cell fires.
2. None of the postsynaptic cells present near cell Y fire stronger than Y .

The model developed by Fukushima was called cognitron as a successor to the perceptron which can perform cognizance of symbols from any alphabet after training. Figure 6-6 shows the connection between presynaptic cell and postsynaptic cell.

The cognitron network is a self-organizing multilayer neural network. Its nodes receive input from the defined areas of the previous layer and also from units within its own area. The input and output neural elements can take the form of positive analog values, which are proportional to the pulse density of firing biological neurons. The cells in the cognitron model use a mechanism of shunting inhibition, i.e., a cell is bound in terms of a maximum and minimum activities and is driven toward these extremities. The area from which the cell receives input is called connectable area. The area formed by the inhibitory cluster is called the vicinity area. Figure 6-7 shows the model of a cognitron. Since the connectable areas for cells in the same vicinity are defined to overlap, but are not exactly the same, there will be a slight difference appearing between the cells which is reinforced so that the gap becomes more apparent. Like this, each cell is allowed to develop its own characteristics.

Cognitron network can be used in neurophysiology and psychology. Since this network closely resembles the natural characteristics of a biological neuron, this is best suited for various kinds of visual and auditory information processing systems. However, a major drawback of cognitron net is that it cannot deal with the problems of orientation or distortion. To overcome this drawback, an improved version called neocognitron was developed.

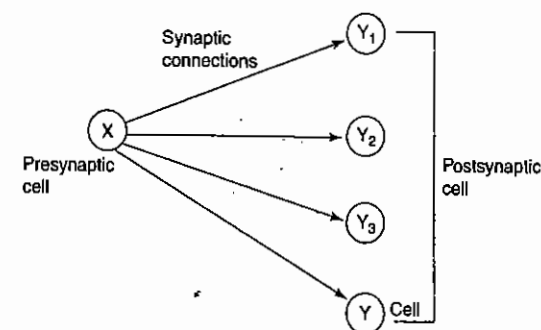


Figure 6-6 Connection between presynaptic cell and postsynaptic cell.

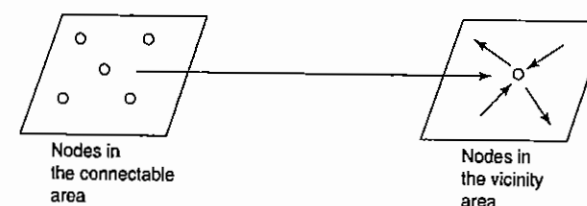


Figure 6-7 Model of a cognitron network.

6.9 Neocognitron Network

Neocognitron is a multilayer feed-forward network model for visual pattern recognition. It is a hierarchical net comprising many layers and there is a localized pattern of connectivity between the layers. It is an extension of cognitron network. Neocognitron net can be used for recognizing hand-written characters. A neocognitron model is shown in Figure 6-8.

The algorithm used in cognitron and neocognitron is same, except that neocognitron model can recognize patterns that are position-shifted or shape-distorted. The cells used in neocognitron are of two types:

1. *S-cell*: Cells that are trained suitably to respond to only certain features in the previous layer.

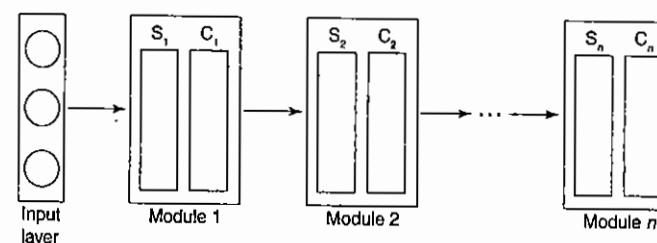


Figure 6-8 Neocognitron models.

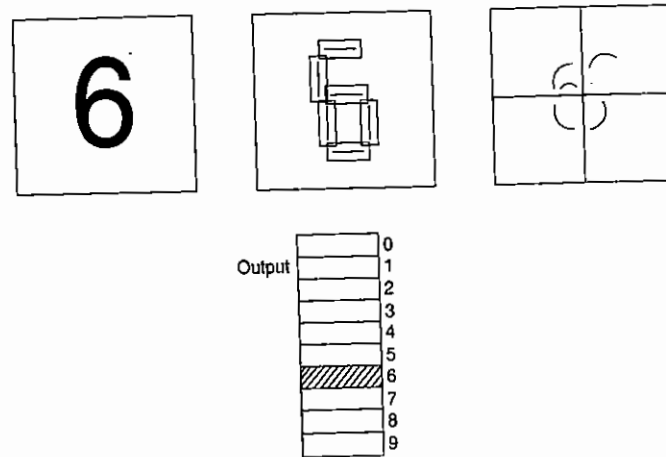


Figure 6-9 Spreading effect in neocognitron.

2. *C-cell*: A C-cell displaces the result of an S-cell in space, i.e., sort of "spreads" the features recognized by the S-cell.

Neocognitron net consists of many modules with the layered arrangement of S-cells and C-cells. The S-cells receive the input from the previous layer, while C-cells receive the input from the S-layer. During training, only the inputs to the S-layer are modified. The S-layer helps in the detection of specific features and their complexities. The feature recognized in the S_1 layer may be a horizontal bar or a vertical bar but the feature in the S_n layer may be more complex. Each unit in the C-layer corresponds to one relative position independent feature. For the independent feature, C-node receives the inputs from a subset of S-layer nodes. For instance, if one node in C-layer detects a vertical line and if four nodes in the preceding S-layer distribute a vertical line, then these four nodes will give the input to the specific node in C-layer to spatially distribute the extracted features. Modules present near the input layer (lower in hierarchy) will be trained before the modules that are higher in hierarchy, i.e., module 1 will be trained before module 2 and so on.

The users have to fix the "receptive field" of each C-node before training starts because the inputs to C-node cannot be modified. The lower level modules have smaller receptive fields while the higher level modules indicate complex independent features present in the hidden layer. The spreading effect used in neocognitron is shown in Figure 6-9.

The S-layers are trained to respond to a particular pattern or group of patterns. The C-arrays then combine the results from related S-arrays and correspondingly thin out the number of units in each array. Training is found to progress layer by layer. The weights from the input units to the first layer are first trained and then frozen. Then the next trainable weights are adjusted and so on. When the net is designed, the weights between some layers are fixed as they are connection patterns.

6.10 Cellular Neural Network

The cellular neural network (CNN), introduced in 1988, is based on cellular automata, i.e., every cell in the network is connected only to its neighbor cells. Figures 6-10(A) and (B) show 2×2 CNN and 3×3 CNN,

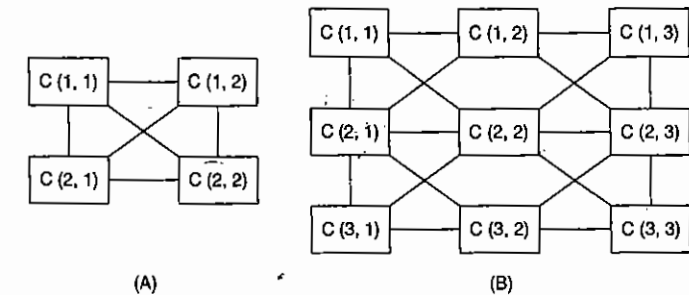


Figure 6-10 (A) A 2×2 CNN; (B) a 3×3 CNN.

respectively. The basic unit of a CNN is a cell. In Figures 6-10(A) and (B), C(1, 1) and C(2, 1) are called as cells.

Even if the cells are not directly connected with each other, they affect each other indirectly due to propagation effects of the network dynamics. The CNN can be implemented by means of a hardware model. This is achieved by replacing each cell with linear capacitors and resistors, linear and nonlinear controlled sources, and independent sources. An electronic circuit model can be constructed for a CNN. The CNNs are used in a wide variety of applications including image processing, pattern recognition and array computers.

6.11 Logicon Projection Network Model

Logicon projection network model (LPNM) is a learning process developed by researchers at Logicon. This model combines supervised and unsupervised training during the learning phases. When the unsupervised learning is used, the network learns quickly but not accurately. On the other hand, with supervised learning, it learns slowly but the error is minimized. The learning phase uses a feed-forward network with a hidden layer in between input and output layers. At the beginning of the learning phase, an unsupervised method such as Kohonen or ART is used to quickly initialize the weights of the network to some gross values, and then a supervised method like BPN may be used to finetune weight values. As the supervised method starts from "almost acceptable" solution, the network is claimed to converge quickly to a global minimum. Logicon claims that LPNM method is best than other methods. This network does not have to be reinitialized if more knowledge is to be added. Also, a network with some knowledge can be added to another network with different knowledge to obtain the sum of both.

6.12 Spatio-Temporal Connectionist Neural Network

Spatio-temporal connectionist neural network (STCNN) characterizes connectionist approaches for learning input-output relationships in which the data is distributed across space and time—spatio-temporal patterns. An STCNN is defined as a parallel distributed information processing structure which is capable of dealing with input data presented across both time and space. In STCNN, input and output patterns vary across time as well as space. For analyzing the network's performance, it is useful to discretize the temporal dimension by sampling at regular intervals. The system considered here produces the response when the time proceeds by intervals of Δt . Symbol " t " may be used to represent a particular point in time. Here Δt can be considered

as the unit measure for quantity t or some small variations in t . In STCNN, even a continuous time system is converted into a set of first-order difference equations, making it to be in the form of discrete time systems.

The time dimension in STCNN differs from the spatial dimensions in conventional connectionist networks. Components of an input pattern distributed across space can be accessed at the same time. However, only the current components of patterns distributed along the time are accessible at any given instant. The input vector for an STCNN at instant t is denoted by input vector $\bar{x}(t)$. This vector is supplied to STCNN at time t by setting the activation values of the input units of the STCNN to the components of the vector. Hence, input vector can be considered as a stimulus.

The conventional and spatio-temporal networks are equipped with memory in the form of connection weights denoted by one or more matrices depending on the number of layers of connections present in the network. These are updated after each training step and constitute a memory of all previous training. Assuming this memory extends back past the current input pattern, all the way to the first training step, we refer to the weights as long-term memory. After a connectionist network has been successfully trained, this long-term memory remains fixed during the operation of the network.

Along with these weight matrices, some networks also use other trainable parameters. These parameters may represent either of the three mentioned below:

1. connectivity scheme of the network;
2. types of transmission delays associated with connections;
3. initial activation values of the internal processing elements.

These parameters also form a part of the long-term memory. We define " W " to denote an n -tuple representing all the adaptable parameters of the network. This n -tuple includes one or more weight matrices, and also may contain connectivity scheme parameters, transmission delay parameters and initial activation values depending on the type of the network.

The STCNNs also include a short-term memory. This memory allows these networks to deal with input and output patterns that vary across time and thus defines them as STCNNs. Conventional connectionist networks compute the activation values of all the nodes at time t based only on the input at time t . On the other hand, in STCNNs the activations of some nodes at time t are computed on the basis of the activations at time $(t-1)$ or earlier. These activations serve as short-term memory. The state vector $\bar{s}(t-1)$ is used here to represent the activations at time $(t-1)$ of those nodes that are used to compute the activations of other nodes at a time t , i.e., state nodes. The long-term memory is stored in connection weights (which are updated only during training) while the short-term memory is represented by node activations (which are computed with each time step even after training).

STCNNs encode their output responses in the activations of a special set of units called output units. The output of STCNN is represented by vector $\bar{y}(t)$. Most connectionist networks learn by computing the difference between their response and desired (ideal) response and adjusting their long-term memory suitably. The desired response is denoted by $\bar{y}_d(t)$. The difference between the desired output vector and the actual output vector is the error vector $\bar{E}(t) = \bar{y}_d(t) - \bar{y}(t)$, and the total network error, ε , is defined as the one-half of the square of the magnitude of this vector, i.e.,

$$\varepsilon = \sum_{t \geq 0} \left\{ \frac{1}{2} \|\bar{E}(t)\|^2 \right\}$$

The total error, given by ε , is the measure of overall performance. It is this quantity that is minimized via gradient descent during training. STCNNs are applicable to dynamical system identification and control, syntactic

pattern recognition and grammatical induction. There are various taxonomies that are being developed for STCNNs.

6.13 Optical Neural Networks

Optical neural networks interconnect neurons with light beams. Owing to this interconnection, no insulation is required between signal paths and the light rays can pass through each other without interacting. The path of the signal travels in three dimensions. The transmission path density is limited by the spacing of light sources, the divergence effect and the spacing of detectors. As a result, all signal paths operate simultaneously, and true data rate results are produced. In holograms with high density, the weighted strengths are stored. These stored weights can be modified during training for producing a fully adaptive system. There are two classes of this optical neural network. They are:

1. electro-optical multipliers;
2. holographic correlators.

6.13.1 Electro-Optical Multipliers

Electro-optical multipliers, also called electro-optical matrix multipliers, perform matrix multiplication in parallel. The network speed is limited only by the available electro-optical components; here the computation time is potentially in the nanosecond range. A model of electro-optical matrix multiplier is shown in Figure 6-11.

Figure 6-11 shows a system which can multiply a nine-element input vector by a 9×7 matrix, which produces a seven-element NET vector. There exists a column of light sources that passes its rays through a lens; each light illuminates a single row of weight shield. The weight shield is a photographic film where transmittance of each square (as shown in Figure 6-11) is proportional to the weight. There is another

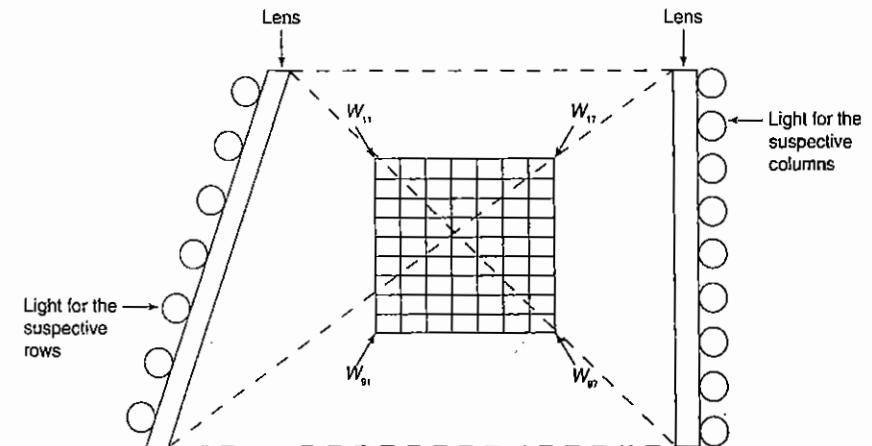


Figure 6-11 Electro-optical multiplier.

lens that focuses the light from each column of the shield to a corresponding photoelector. The NET is calculated as

$$\text{NET}_k = \sum_i w_{ik} x_i$$

where NET_k is the net output of neuron k ; w_{ik} the weight from neuron i to neuron k ; x_i the input vector component i . The output of each photodetector represents the dot product between the input vector and a column of the weight matrix. The output vector set is equal to the product of the input vector with weight matrix. Hence, matrix multiplication is performed parallelly. The speed is independent of the size of the array. So, the network is scaled up without increasing the time required for computation. Variable weights may be designed for use in the adaptive system. A liquid crystal light valve instead of photographic film may be used for weights. This makes the weights to get adjusted electronically. This type of electro-optical multiplier can be used in Hopfield net and bidirectional associative memory.

6.13.2 Holographic Correlators

In holographic correlators, the reference images are stored in a thin hologram and are retrieved in a coherently illuminated feedback loop. The input signal, either noisy or incomplete, may be applied to the system and can simultaneously be correlated optically with all the stored reference images. These correlations can be thresholded and are fed back to the input, where the strongest correlation reinforces the input image. The enhanced image passes around the loop repeatedly, which approaches the stored image more closely on each pass, until the system gets stabilized on the desired image. The best performance of optical correlators is obtained when they are used for image recognition. A generalized optical image recognition system with holograms is shown in Figure 6-12.

The system input is an image from a laser beam. This passes through a beam splitter, which sends it to the threshold device. The image is reflected, then gets reflected from the threshold device, passes back to the beam splitter, then goes to lens 1, which makes it fall on the first hologram. There are several stored images

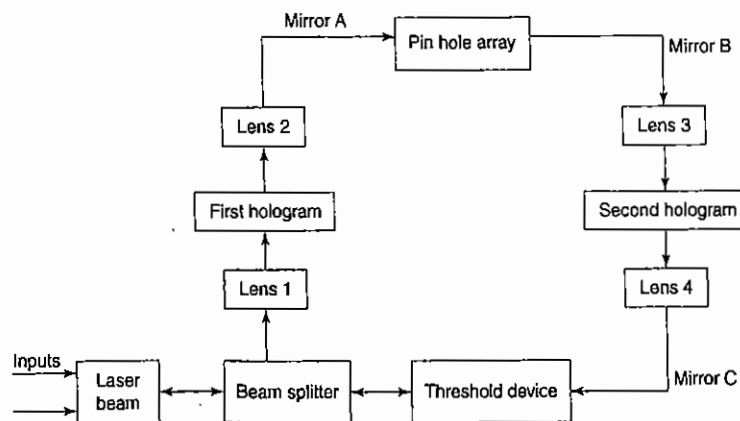


Figure 6-12 Optical image recognition system.

in first hologram. The image then gets correlated with each stored image. This correlation produces light patterns. The brightness of the patterns varies with the degree of correlation. The projected images from lens 2 and mirror A pass through pinhole array, where they are spatially separated. From this array, light patterns go to mirror B through lens 3 and then are applied to the second hologram. Lens 4 and mirror C then produce superposition of the multiple correlated images onto the back side of the threshold device.

The front surface of the threshold device reflects most strongly that pattern which is brightest on its rear surface. Its rear surface has projected on it the set of four correlations of each of the four stored images with the input image. The stored image that is similar to the input image possesses highest correlation. This reflected image again passes through the beam splitter and re-enters the loop for further enhancement. The system gets converged on the stored patterns most like the input pattern.

Here we have discussed the basic operation of the holographic optical image recognition system. Employing hologram correlator, we can design Hopfield network. Optical neural networks are more advantageous in terms of speed and interconnect density. They can virtually construct any network architecture.

6.14 Neuroprocessor Chips

Neural networks implemented in hardware can take advantage of their inherent parallelism and run orders of magnitude faster than software simulations. There exists a wide variety of commercial neural network chips and neurocomputers.

The probabilistic RAM, pRAM-256 Very Large Scale Integrated (VLSI) neural network processor, was developed by the Electrical Engineering Department of King's College, London. pRAM has 256 reconfigurable neurons, each with six inputs. Its on-chip learning unit utilizes reinforced learning where learning can be global, local or competitive. The external static RAM of pRAM stores the synaptic weights. The p-RAM possesses both stochastic and nonlinear aspects of biological neurons in a typical manner, which allows exploitation of hardware.

The Neuro Accelerator Chip (NAC) was developed in 1992 by the Information Defence Division, Australian Defence Science and Technology Organization. It is made up of an array of 16-component 10-bit integer processing elements that can be cascaded in two dimensions with necessary control signals. Each processing element multiplies its input by one of 16 weights preloaded in dual port registers and accumulates the results to 23-bit precision at a rate of 500 million operations per second. The NAC can be hard wired to implement various neural networks.

Neural Network Processor (NNP), developed by Accurate Automation Corporation, uses a multiple instruction multiple data architecture capable of running multiple chips in parallel without performance degradation. Each chip houses high-speed 16-bit processor with on-chip storage for synaptic weights. Only nine assembly language instructions are executed by the processor. Communication among multiple NNPs is performed by interprocessor. NNP can be programmed to implement any particular neural network training algorithms. Its performance is 140 MCPS for a single chip and up to 1.4 GCPS for a 10-processor system.

The CNAPS system, developed by Adaptive Solutions, is mainly based on CNAPS-1064 digital parallel processor chip that has 64 sub-processors operating in SIMD mode. Each sub-processor can emulate one or more neurons, and multiple chips can be ganged together. The CNAPS/PC ISA card uses 1, 2 or 4 of new CNAPS-1016 parallel processor chips or two of the 1064 chips to obtain 16, 32, 64 or 128 CNAPS processors. Learning algorithms can be programmed. It can be noted that back propagation and several other algorithms come in the Build Net package. Here, back propagation feed-forward performs 1.16 billion multiply/accumulates/second and 293 million weight updates/second with 1 chip and

5.80 1.16 billion multiply/accumulates/second / 1.95 million weight updates/second, respectively, with four chips.

The IBM ZISC 036 is a digital chip with 64-component 8-bit inputs and 36 radial basis function neurons. Multiple chips can be easily cascaded to create networks of arbitrary size. Here input vector V is compared to store prototype vector ϕ for each neuron. It takes $3.5 \mu\text{s}$ to load 64 elements and another $0.5 \mu\text{s}$ for the classification signal to appear. Learning processing of a vector takes about another $2 \mu\text{s}$ beyond $4 \mu\text{s}$ for loading and evaluation. Its performance at 16 MHz, $4 \mu\text{s}$ classification of a 64-component 8-bit vector.

The INTEL 80170NX Electrically Trainable Analog Neural Network (ETANN) is one with 64 inputs (0-3v), 16 internal biases, and 64 neurons with sigmoidal transfer functions. Two-layer feed-forward networks can be implemented with 64 inputs, 64 hidden neurons, and 64 output neurons using the two 80×64 weight matrices. Hidden layer outputs are clocked back through second weight matrix to perform output layer processing. Instead of this, a single 64-layer network with 128 inputs can be implemented using both matrices and clocking in two sets of 64 inputs. Weights possess 6-bit precision and are stored in nonvolatile floating gate synapses. There is no on-chip learning. Emulation is performed in software and the weights are downloaded to the chip. In this case, about $8 \mu\text{s}$ propagation time is taken for a two-layer network. This is equivalent to roughly 2 billion multiply/accumulates per second.

MCE MT 19003 Neural Instruction Set Processor is a digital processor chip using signed 12-bit internal neuron values, with 16-bit multiplier and 35-bit accumulator. Network input values, bias values, synapse and neuron values are held in off-chip memory. The network processing is also guided by a given program in off-chip memory using seven-element instruction set. Neuron values can be sealed by a transfer function using four available tables. This processor also has no on-chip learning. Its performance is 1 synapse per clock cycle.

RC Module Neuro Processor NM6403 is a high-performance microprocessor with super scalar architecture. The architecture includes control unit, address calculation and scalar processing units, node to support vector operations with elements of variable bit length. There is no on-chip learning in this processor. Its performance is

1. *Scalar operations:* 50 MIPS, 50 MIPS for 32 bit data.
2. *Vector operations:* 1.2 billion multiplications and additions/second.

Nestor N11000 is a network with Radial Basis function neurons. During its learning, prototype vectors are stored under the assumption that they are picked randomly from original parent distributions. Here up to 1024 prototypes can be stored. Each prototype is then assigned to a given middle layer neuron. This middle layer neuron is assigned to an output neuron that represents the particular class for that vector. All middle layer neurons that correspond to the same class are designed to same output neuron. In recall stage, an input vector is compared to each prototype parallelly, and if the distance between them is above a given threshold, it fires, leading to firing of the corresponding output, or class, neuron. Here two on-chip learning algorithms are available:

1. Probabilistic neural net (PNN);
2. Restricted Coulomb energy (RCE).

Also, microcoding can be modified for user-defined algorithms. Its performance is 40K, 256 element patterns per second. There is National Semiconductors NeuFuz/COP8 Microcontroller processor, which uses a combination of neural network and fuzzy logic software to generate code for National's COP8

microcontrollers. Neural network can be used to learn the fuzzy based rules and membership functions. There exist several packages of it. Some are listed below:

1. *NeuFuz Learning Kit (NF2 - C&A - Kit):* A neural network PC/AT software (2 inputs, 1 output) and fuzzy rule and membership function generator (max 3 membership functions), COP8 code generator and COP8 assembler/linker.
2. *NeuFuz 4 (NF2 - C&A):* Neural network PC/AT software (4 inputs, 1 output) and fuzzy rule and membership function generator (max 3 member functions), COP8 code generator and COP8 assm/linker.
3. *NeuFuz 4 Development System (NF2 - C&A):* Neural network PC/AT software (4 inputs, 1 output) and fuzzy rule membership function generators (max 3 member functions), COP8 code generator, COP8 assembler/linker and COP8 in-circuit emulator with PROM programming.

Learning performed here is only software learning. Apart from the above listed chips, there are several other neuroprocessor chips. Besides, a wide variety of research is going on for further development of neural network hardware.

6.15 Summary

In this chapter, we have discussed certain specific networks based on their special characteristics and performance. The networks are designed for optimization problems and classifications. Certain nets discussed use Bayesian decision making method and hierarchical arrangement of units. The variations of Boltzmann machine, which include Gaussian and Cauchy nets were also discussed. Besides, our discussion focused on the cognitron and neocognitron networks, which are used for recognition of hand written characters. Other networks discussed include spatio-temporal neural network, annealing network, optical neural nets, cellular neural nets, and Logicon neural nets. To give the reader an idea of neural network hardware, a few neuroprocessor chips have also been listed.

6.16 Review Questions

1. List a few special neural networks designed for typical applications.
2. What is the principle behind simulated annealing network?
3. How is Boltzmann machine used in constrained optimization problems?
4. With a neat architectural diagram explain the application procedure used in Boltzmann machine.
5. Write short note on Gaussian and Cauchy machines.
6. What is the importance of probabilistic neural network?
7. With an architectural diagram, explain the probabilistic neural network.
8. Discuss the algorithm used in probabilistic neural network.
9. How does cascade correlation network build its network as the training progress?
10. Justify that cascade correlation network is a hierarchical network.
11. Compare and contrast presynaptic and postsynaptic cells in cognitron model.
12. Explain the working principle of cognitron network.
13. What is the drawback of cognitron net?
14. Write short note on neocognitron model, stating how does it overcome the drawback of cognitron model.

15. Describe the working methodology in cellular neural network.
16. In what way are the supervised and unsupervised learning methods combined to obtain high performance in Logicon projection network?
17. Discuss in detail the spatio-temporal connectionist neural network.
18. State the principle of optical neural networks.
19. Briefly explain the concept involved in electro-multiplier networks and holographic correlators.
20. Mention a few latest neuroprocessor chips.

Handwritten notes:

Learning is more important than reasoning
Reasoning is more important than learning
Learning is more important than reasoning
Reasoning is more important than learning

MATLAB Environment for Soft Computing Techniques

19

Learning Objectives

- Discusses how soft computing techniques are implemented using MATLAB software.
- Gives brief note on the development of MATLAB software.
- Details how basic operations are carried out using MATLAB.
- An introduction SIMULINK which is a branch in MATLAB package is discussed.
- The various soft computing toolboxes in MATLAB – Neural network toolbox, Fuzzy logic toolbox, Genetic algorithm toolbox – are included with commands for ready reference of the user.
- The chapter apart from the command line functions also discusses the implementation of soft computing techniques using SIMULINK blocks and graphical user interface (GUI) toolbox.
- The chapter provides the reader several problems solved using MATLAB software for soft computing techniques.

19.1 Introduction

MATLAB (Matrix Laboratory), a product of Mathworks, is a scientific software package designed to provide integrated numeric computation and graphics visualization in high-level programming language. Cleve Moler, Chief Scientist at MathWorks, Inc., originally wrote MATLAB to provide easy access to matrix software developed in the LINPACK and EISPACK projects. The very first version was written in the late 1970s for use in courses in matrix theory, linear algebra and numerical analysis. MATLAB is therefore built upon a foundation of sophisticated matrix software, in which the basic data element is a matrix that does not require predimensioning.

MATLAB program consists of standard and specialized toolboxes allowing users to take advantage of the matrix-algorithm-based projects. MATLAB offers interactive features allowing the users a great flexibility in the manipulation of data and in the form of matrix arrays for computation and visualization. MATLAB inputs can be entered at the "command line" or from "mfiles," which contains a programming-like set of instructions to be executed by MATLAB. In the aspect of programming, MATLAB works differently from FORTRAN, C, or Basic; for example, no dimensioning required for matrix arrays and no object code file generated. MATLAB offers some standard toolboxes and many optional (at extra charges) toolboxes such as Financial Toolbox and Statistics Toolbox. Users may create their own toolboxes consisting of "mfiles" written for specific applications. MATLAB is a high-performance language for technical computing. It integrates computation, visualization and programming in an easy-to-use environment where problems and solutions are expressed in familiar mathematical notation. Typical use includes:

1. math and computation;
2. algorithm development;

3. modeling, simulation and prototyping;
4. data analysis, exploration and visualization;
5. scientific and engineering graphics;
6. application development, including graphical-user interface building.

MATLAB's two- and three-dimensional graphics are object-oriented. MATLAB is thus both an environment and a matrix/vector-oriented programming language, which enables the use to build own required tools. The main features of MATLAB are:

1. Advance algorithms for high-performance numerical computations, especially in the field of matrix algebra.
2. A large collection of predefined mathematical functions and the ability to define one's own functions.
3. Two- and three-dimensional graphics for plotting and displaying data.
4. A complete help system online.
5. Powerful matrix/vector-oriented high-level programming language for individual applications.
6. Ability to cooperate with programs written in other languages and for importing and exporting formatted data.
7. Toolboxes available for solving advanced problems in several application areas.

19.2 Getting Started with MATLAB

Clicking on the program icon on a Windows/Mac machine can start MATLAB. The command window can be used to interactively issue commands and evaluate expressions. The extensive help files are essential for obtaining information about the various commands and functions available. Typing `help` at the command prompt generates a list of the various function categories and toolboxes with a brief description of each. Typing `help topic` generates help on the specified topic, which is generally a MATLAB command or toolbox name. This can be used to get the syntax for different commands. A more user-friendly graphical help system is also available via the help menu.

MATLAB uses conventional notation for real numbers. Scientific notation is also accepted in the form of a real number followed by the letter `e` and an integer exponent. Imaginary numbers are obtained by using either `i` or `j` as a suffix. Examples of valid numbers: 5, 4.55, 1.945e-20, `i`, 10+15i.

The basic mathematical operators (+, -, *, /, ^) can be used directly at the command prompt to perform calculations, as can various elementary mathematical functions. `help elfun` gives a list of the elementary mathematical functions. Remember that angles are specified in radians to functions like `sin()` and `cos()`. Examples of usage:

```
>> 8 + 3i + 5
ans = 13.0000 + 3.0000i
>> sqrt(-4)
ans = 0 + 1.4142i
```

19.2.1 Matrices and Vectors

Construction: The simplest way to construct a matrix in MATLAB is to enumerate its elements row by row within square brackets, the rows separated by semi-colons, the elements of each row separated by spaces.

These elements can be real/complex numbers or other vectors and matrices, as long as the dimensions match up. For example,

```
>> A = [1 2 3 4; 5 6 7 8]
A =
     1     2     3     4
     5     6     7     8
```

Row vectors are simply matrices with one row of elements:

```
>> x = [4 5 6 7]
x =
     4     5     6     7
```

Vectors with equally spaced elements can be constructed using the colon notation:

$x = \text{starting value} : \text{increment} : \text{maximum value}$

A default increment of 1 is used if the increment is omitted. If the vector has to have a specific last element, we use the `linspace` command:

$x = \text{linspace}(\text{first_element}, \text{last_element}, \text{number_of_elements})$

Column vectors are constructed as matrices with several rows of one element each:

```
>> y = [1; 2; 3]
y =
     1
     2
     3
```

Equally spaced column vectors are obtained by first generating a row vector using the colon notation or the `linspace` command, and then applying the transpose operator. There are other built-in functions to generate specific types of matrices:

1. `eye(m, n)` generates an $m \times n$ matrix with ones on the main diagonal and zeros elsewhere. If $m = n$, `eye(n)` can be used instead.
2. `zeros(m, n)` is an $m \times n$ matrix whose elements are all zeros.
3. `ones(m, n)` is an $m \times n$ matrix whose elements are all ones.
4. `diag(v)` is a square diagonal matrix with vector `v` on the main diagonal.
5. `diag(A)` is a column vector formed from the main diagonal of `A`.

Addressing elements: The element in the i th row and j th column of a matrix A is $A(i, j)$. The subscripts i and j cannot be negative or zero. In the case of a vector x , the i th element of the vector is addressed as $x(i)$:

```
>> A = [1 2 3; 7 8 9]
A(2, 1) = 7
>> x = [4 5 6]
x(2) = 5
```

Matrix operations: Matrix addition, subtraction and multiplication are implemented using the traditional operators $+$, $-$ and $*$. The inverse of a square matrix A is given by $\text{inv}(A)$. There are two matrix division operators, $\backslash$ and $/$. If A is a non-singular matrix, then $A \backslash B$ and B/A correspond to the left and right multiplication of B by $\text{inv}(A)$. The transpose of a matrix A is given by A' . The expression A' produces the conjugate transpose of A , that is it transposes A and replaces its elements by their complex conjugates. If the elements of A are real, then A' and A' are equivalent.

Array operations: It is also possible to work with the matrix as an array, that is to perform uniform elementwise operations. The array operators ($+$, $-$, $*$, $/$, $.$) are used for this purpose. For example, if A and B are of the same dimensions, the expression $A.*B$ would multiply each element of A with the corresponding element of B to produce a matrix of the same dimension as A and B .

Scripts: Instead of executing individual commands at the prompt, it is possible to make a text file with a sequence of commands, that is a MATLAB script, and execute all the commands in sequence. The script must be a plain ASCII file with a ".m" extension. When this file is in the working directory, typing the name of the file without the extension is sufficient to execute the script.

Plotting: The general form of the two-dimensional plot command is $\text{plot}(x, y, S)$ where x and y are vectors of the same type and dimension and S is a string of characters within quotes which specifies plot attributes like color, line style, etc. Use help plot to find out options and related commands.

19.3 Introduction to Simulink

Simulink (Simulation and Link) is an extension of MATLAB by Mathworks. It works with MATLAB to offer modeling, simulating and analyzing of dynamical systems under a graphical user interface (GUI) environment. The construction of a model is simplified with click-and-drag mouse operations. Simulink includes a comprehensive block library of toolboxes for both linear and nonlinear analyses. Models are hierarchical, which allow using both top-down and bottom-up approaches. As Simulink is an integral part of MATLAB; it is easy to switch back and forth during the analysis process, and thus, the user may take full advantage of features offered in both environments. MATLAB is an interactive package for numerical analysis, matrix computation, control system design and linear system analysis and design available on most CAEN (Computer Aided Engineering Network) platforms (Macintosh, PC, Sun and Hewlett-Packard). In addition to the standard functions provided by MATLAB, there exist large set of toolboxes, or collections of functions and procedures, available as part of the MATLAB package. The toolboxes are:

1. **Control system:** It provides several features for advanced control system design and analysis.
2. **Communications:** It provides functions to model the components of a communication system's physical layer.
3. **Signal processing:** It contains functions to design analog and digital filters and apply these filters to data and analyze the results.

4. **System identification:** It provides features to build mathematical models of dynamical systems based on observed system data.
5. **Robust control:** It allows users to create robust multivariable feedback control system designs based on the concept of the singular-value Bode plot.
6. **Simulink:** It allows you to model dynamic systems graphically.
7. **Neural network:** It allows you to simulate neural networks.
8. **Fuzzy logic:** It allows for manipulation of fuzzy systems and membership functions.
9. **Image processing:** It provides access to a wide variety of functions for reading, writing, and filtering images of various kinds in different ways.
10. **Analysis:** It includes a wide variety of system analysis tools for varying matrices.
11. **Optimization:** It contains basic tools for use in constrained and unconstrained optimization problems.
12. **Spline:** It can be used to find approximate functional representations of data sets.
13. **Symbolic:** It allows for symbolic (rather than purely numeric) manipulation of functions.
14. **User interface utilities:** It includes tools for creating dialog boxes, menu utilities and other user interaction for script files.

In MATLAB command window, enter: `>> simulink` and press ENTER to invoke Simulink. A Simulink library browser window would appear as shown in Figure 19-1.

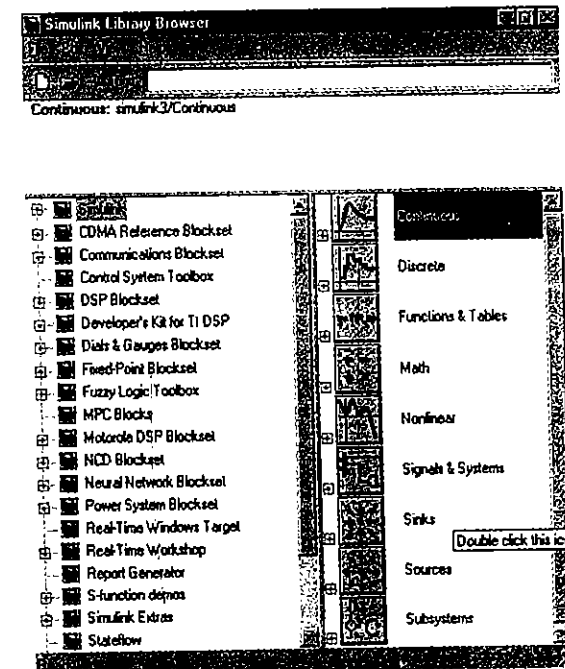


Figure 19-1 Simulink library browser.

19.4 MATLAB Neural Network Toolbox

The MATLAB neural network toolbox provides a complete set of functions and a GUI for the design, implementation, visualization and simulation of neural networks. It supports the most commonly used supervised and unsupervised network architectures and a comprehensive set of training and learning functions. The neural network toolbox extends the MATLAB computing environment to provide tools for the design, implementation, visualization and simulation of neural networks. Neural networks are uniquely powerful tools that are used in applications where formal analysis would be difficult or impossible, such as pattern recognition and nonlinear system identification and control.

Salient Features:

1. GUI for creating, training and simulation of neural networks.
2. Set of training and learning functions.
3. Automatic generation of Simulink models from neural network objects.
4. Pre- and post-processing functions for improving network training and assessing network performance.
5. Routines for improving generalization.
6. Visualization functions for viewing network performance.

19.4.1 Creating a Custom Neural Network

The command `NETWORK` creates a custom neural network.

Synopsis:

```
net = network
net = network(numInputs,numLayers,biasConnect,
             inputConnect, layerConnect,outputConnect,targetConnect)
```

Description:

`NETWORK` creates new custom networks. It is used to create networks that are then customized by functions such as `NEWP`, `NEWLIN`, `NEWFF`, etc.

`NETWORK` takes the following optional arguments (shown with default values):

numInputs: Number of inputs, 0.
 numLayers: Number of layers, 0.
 biasConnect: numLayers-by-1 Boolean vector, zeros.
 inputConnect: numLayers-by-numInputs Boolean matrix, zeros.
 layerConnect: numLayers-by-numLayers Boolean matrix, zeros.
 outputConnect: 1-by-numLayers Boolean vector, zeros.
 targetConnect: 1-by-numLayers Boolean vector, zeros.
 and returns,
 NET: New network with the given property values.

Properties

Architecture properties

net.numInputs: 0 or a positive integer.
 Number of inputs.

net.numLayers: 0 or a positive integer.

Number of layers.

net.biasConnect: numLayer-by-1 Boolean vector.

If net.biasConnect(*i*) is 1 then the layer *i* has a bias and

net.biases{*i*} is a structure describing that bias.

net.inputConnect: numLayer-by-numInputs Boolean vector.

If net.inputConnect(*i,j*) is 1 then layer *i* has a weight coming from

input *j* and net.inputWeights{*i,j*} is a structure describing that weight.

net.layerConnect: numLayer-by-numLayers Boolean vector.

If net.layerConnect(*i,j*) is 1 then layer *i* has a weight coming from

layer *j* and net.layerWeights{*i,j*} is a structure describing that weight.

net.outputConnect: 1-by-numLayers Boolean vector.

If net.outputConnect(*i*) is 1 then the network has an output from layer *i* and net.outputs{*i*} is a structure describing that output.

net.targetConnect: 1-by-numLayers Boolean vector.

If net.outputConnect(*i*) is 1 then the network has a target from layer *i* and net.targets{*i*} is a structure describing that target.

net.numOutputs: 0 or a positive integer. Read only.

Number of network outputs according to net.outputConnect.

net.numTargets: 0 or a positive integer. Read only.

Number of targets according to net.targetConnect.

net.numInputDelays: 0 or a positive integer. Read only.

Maximum input delay according to all

net.inputWeight{*i,j*}.delays.

net.numLayerDelays: 0 or a positive number. Read only.

Maximum layer delay according to all Net.layerWeight{*i,j*}.delays.

Subobject structure properties

net.inputs: numInputs-by-1 cell array.

net.inputs{*i*} is a structure defining input *i*:

net.layers: numLayers-by-1 cell array.

net.layers{*i*} is a structure defining layer *i*:

net.biases: numLayers-by-1 cell array.

if net.biasConnect(*i*) is 1, then net.biases{*i*} is a structure defining the bias for layer *i*.

net.inputWeights: numLayers-by-numInputs cell array.

if net.inputConnect(*i,j*) is 1, then net.inputWeights{*i,j*} is a structure defining the weight to layer *i* from input *j*.

net.layerWeights: numLayers-by-numLayers cell array.

if net.layerConnect(*i,j*) is 1, then net.layerWeights{*i,j*} is a structure defining the weight to layer *i* from layer *j*.

net.outputs: 1-by-numLayers cell array.

if net.outputConnect(*i*) is 1, then net.outputs{*i*} is a structure defining the network output from layer *i*.

net.targets: 1-by-numLayers cell array.

if net.targetConnect(*i*) is 1, then net.targets{*i*} is a structure defining the network target to layer *i*.

Function properties:

net.adaptFcn: name of a network adaption function

net.initFcn: name of a network initialization function

net.performFcn: name of a network performance function

net.trainFcn: name of a network training function or

Parameter properties:

net.adaptParam: network adaption parameters.

net.initParam: network initialization parameters.

net.performParam: network performance parameters.

net.trainParam: network training parameters.

Weight and bias value properties:

net.IW: numLayers-by-numInputs cell array of input weight values.

net.LW: numLayers-by-numLayers cell array of layer weight values.

net.b: numLayers-by-1 cell array of bias values.

Other properties:

net.userdata: structure you can use to store useful values.

19.4.2 Commands in Neural Network Toolbox

The various commands used in the neural network toolbox are as follows:

Graphical user interface functions:

nntool: Neural network toolbox graphical user interface.

Analysis functions:

errsurf: Error surface of single input neuron.

maxlinr: Maximum learning rate for a linear layer.

Distance functions:

boxdist: Box distance function.

dist: Euclidean distance weight function.

mandist: Manhattan distance weight function.

linkdist: Link distance function.

Layer initialization functions:

initnw: Nguyen-Widrow layer initialization function.

initwb: By-weight-and-bias layer initialization function.

Learning functions:

learncon: Conscience bias learning function.

learnkd: Gradient descent weight/bias learning function.

learnkdgm: Gradient descent w/momentum weight/bias learning function.

learnh: Hebb weight learning function.

learnhd: Hebb with decay weight learning function.

learnis: Instar weight learning function.

learnk: Kohonen weight learning function.

learnlv1: LVQ1 weight learning function.

learnlv2: LVQ2 weight learning function.

learnos: Outstar weight learning function.

learnp: Perceptron weight/bias learning function.

learnpn: Normalized perceptron weight/bias learning function.

learnsom: Self-organizing map weight learning function.

learnwh: Widrow-Hoff weight/bias learning rule.

Line search functions:

srchbac: Backtracking search.

srchbre: Brent's combination golden section/quadratic interpolation.

srchcha: Charalambous' cubic interpolation.

srchgol: Golden section search.

srchhyb: Hybrid bisection/cubic search.

New networks:

network: Create a custom neural network.

newc: Create a competitive layer.

newcf: Create a cascade-forward backpropagation network.

newelm: Create an Elman backpropagation network.

newff: Create a feed-forward backpropagation network.

newfftd: Create a feed-forward input-delay backprop network.

newgrnn: Design a generalized regression neural network.

newhop: Create a Hopfield recurrent network.

newlin: Create a linear layer.

newlind: Design a linear layer.

newlvq: Create a learning vector quantization network.

newp: Create a perceptron.

newpnn: Design a probabilistic neural network.

newrb: Design a radial basis network.

newrbe: Design an exact radial basis network.

newsom: Create a self-organizing map.

Net input functions:

netprod: Product net input function.

netsum: Sum net input function.

Net input derivative functions:

dnetprod: Product net input derivative function.

dnetsum: Sum net input derivative function.

Network initialization functions:

initlay: Layer-by-layer network initialization function.

Performance functions:

mae: Mean absolute error performance function.

mse: Mean squared error performance function.

msereg: Mean squared error with regularization performance function.

sse: Sum squared error performance function.

Performance derivative functions:

dmae: Mean absolute error performance derivatives function.

dmse: Mean squared error performance derivatives function.

dmserg: Mean squared error w/reg performance derivative function.

dsse: Sum squared error performance derivative function.

Plotting functions:

hintonw: Hinton graph of weight matrix.

hintonwb: Hinton graph of weight matrix and bias vector.

plotbr: Plot network performance for Bayesian regularization training.

plotes: Plot an error surface of a single input neuron.

plotpc: Plot classification line on perceptron vector plot.

plotpv: Plot perceptron input/target vectors.

plotep: Plot a weight-bias position on an error surface.

plotperf: Plot network performance.

plotsom: Plot self-organizing map.

plotv: Plot vectors as lines from the origin.

plotvec: Plot vectors with different colors.

Pre- and post-processing:

prestd: Normalize data for unity standard deviation and zero mean.

poststd: Unnormalize data which has been normalized by PRESTD.

trastd: Transform data with precalculated mean and standard deviation.

premnmx: Normalize data for maximum of 1 and minimum of -1.

postmnmx: Unnormalize data which has been normalized by PREMNMX.

tramnm: Transform data with precalculated minimum and maximum.

prepca: Principal component analysis on input data.

trapca: Transform data with PCA matrix computed by PREPCA.

postreg: Post-training regression analysis.

Simulink support:

gensim: Generate a Simulink block to simulate a neural network.

Topology functions:

gridtop: Grid layer topology function.

hextop: Hexagonal layer topology function.

randtop: Random layer topology function.

Transfer functions:

compet: Competitive transfer function.

hardlim: Hard limit transfer function.

hardlims: Symmetric hard limit transfer function.

logsig: Log sigmoid transfer function.

poslin: Positive linear transfer function.

purelin: Linear transfer function.

radbas: Radial basis transfer function.

satlin: Saturating linear transfer function.

satlins: Symmetric saturating linear transfer function.

softmax: Soft max transfer function.

tansig: Hyperbolic tangent sigmoid transfer function.

tribas: Triangular basis transfer function.

Training functions:

trainb: Batch training with weight and bias learning rules.

trainbfg: BFGS quasi-Newton backpropagation.

trainbr: Bayesian regularization.

trainc: Cyclical order incremental training w/learning functions.

traincgb: Powell-Beale conjugate gradient backpropagation.

traincgf: Fletcher-Powell conjugate gradient backpropagation.

traincgp: Polak-Ribiere conjugate gradient backpropagation.

traingd: Gradient descent backpropagation.

traingdm: Gradient descent with momentum backpropagation.

traingda: Gradient descent with adaptive lr backpropagation.

traingdx: Gradient descent w/momentum and adaptive lr backpropagation.

trainlm: Levenberg-Marquardt backpropagation.

trainoss: One step secant backpropagation.

trainr: Random order incremental training w/learning functions.

trainrp: Resilient backpropagation (Rprop).

trains: Sequential order incremental training w/learning functions.

trainscg: Scaled conjugate gradient backpropagation.

Transfer derivative functions:

dhardlim: Hard limit transfer derivative function.

dhardlims: Symmetric hard limit transfer derivative function.

dlogsig: Log sigmoid transfer derivative function.

dposlin: Positive linear transfer derivative function.

dpurelin: Hard limit transfer derivative function.

dradbas: Radial basis transfer derivative function.

dsatlin: Saturating linear transfer derivative function.

dsatlins: Symmetric saturating linear transfer derivative function.

dtansig: Hyperbolic tangent sigmoid transfer derivative function.

dtribas: Triangular basis transfer derivative function.

Update networks from previous versions:

nnt2c: Update NNT 2.0 competitive layer.

nnt2elm: Update NNT 2.0 Elman backpropagation network.

nnt2ff: Update NNT 2.0 feed-forward network.

nnt2hop: Update NNT 2.0 Hopfield recurrent network.

nnt2lin: Update NNT 2.0 linear layer.
 nnt2lvq: Update NNT 2.0 learning vector quantization network.
 nnt2p: Update NNT 2.0 perceptron.
 nnt2rb: Update NNT 2.0 radial basis network.
 nnt2som: Update NNT 2.0 self-organizing map.

Using networks:

sim: Simulate a neural network.
 init: Initialize a neural network.
 adapt: Allow a neural network to adapt.
 train: Train a neural network.
 disp: Display a neural network's properties.
 display: Display the name and properties of a neural network variable.

Vectors:

cell2mat: Combine cell array of matrices into one matrix.
 concur: Create concurrent bias vectors.
 con2seq: Convert concurrent vectors to sequential vectors.
 combvec: Create all combinations of vectors.
 ind2vec: Convert indices to vectors.
 mat2cell: Break matrix up into cell array of matrices.
 minmax: Ranges of matrix rows.
 nncopy: Copy matrix or cell array.
 normc: Normalize columns of a matrix.
 normr: Normalize rows of a matrix.
 pnormc: Pseudo-normalize columns of a matrix.
 quant: Discretize values as multiples of a quantity.
 seq2con: Convert sequential vectors to concurrent vectors.
 sumsq: Sum squared elements of matrix.
 vec2ind: Convert vectors to indices.

Weight functions:

dist: Euclidean distance weight function.
 dotprod: Dot product weight function.
 mandist: Manhattan distance weight function.
 negdist: Dot product weight function.
 normprod: Normalized dot product weight function.

Weight and bias initialization functions:

initcon: Conscience bias initialization function.
 initzero: Zero weight/bias initialization function.
 midpoint: Midpoint weight initialization function.
 randnc: Normalized column weight initialization function.
 randnr: Normalized row weight initialization function.
 rands: Symmetric random weight/bias initialization function.

Weight derivative functions:

ddotprod: Dot product weight derivative function.

Train and adapt

There are two types of training that are given below.

1. *Incremental training*: updating the weights after the presentation of each single training sample.
2. *Batch training*: updating the weights after each presenting the complete data set.

When using adapt, both incremental and batch training can be used. When using train, on the other hand, only batch training can be used, regardless of the format of the data. The big plus point of train is that it gives you a lot more choice in training functions (gradient descent, gradient descent w/momentum, Levenberg-Marquardt, etc.) which are implemented very efficiently.

The difference between train and adapt is similar as *the difference between passes and epochs*. When using adapt, the property that determines how many times the complete training data set is used for training the network is called net.adaptParam.passes. But, when using train, the same property is called net.trainParam.epochs.

```
>> net.trainFcn = 'traingdm';
>> net.trainParam.epochs = 1000;
>> net.adaptFcn = 'adaptwb';
>> net.adaptParam.passes = 10;
```

19.4.3 Neural Network Graphical User Interface Toolbox

A GUI can be used to

1. create networks;
2. create data;
3. train the networks;
4. export the networks;
5. export the data to the command line workspace.

The salient features of GUI are the following:

1. It is designed to be simple and user-friendly. This tool lets you import potentially large and complex data sets.
2. It also enables you to create, initialize, train, simulate and manage the networks. It has the GUI Network/Data Manager window.
3. The window has its own work area, separate from the more familiar command line workspace. Thus, when using the GUI, one might "export" the GUI results to the (command line) workspace. Similarly, one might "import" results from the command line workspace to the GUI.
4. Once the Network/Data Manager is up and running, create a network, view it, train it, simulate it and export the final results to the workspace. Similarly, import data from the workspace for use in the GUI.

This tool lets you import potentially large and complex data sets. The GUI also enables you to create, initialize, train, simulate and manage your networks. Simple graphical representations allow you to visualize and understand network architecture (see Figure 19-2).

The following example deals with a perceptron network. It gives a step-by-step procedure of creating a network.

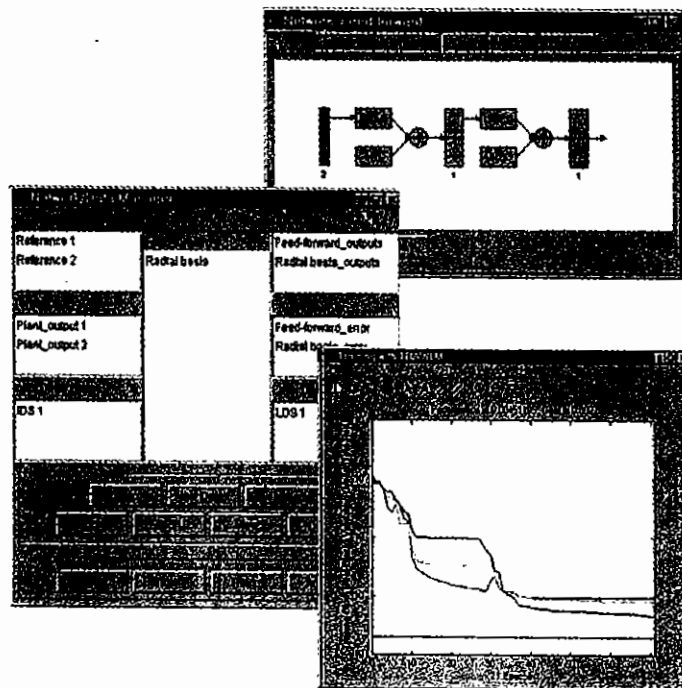


Figure 19-2 This window displays portions of the neural network GUI. Dialogs and panes allow you to visualize your network (top), evaluate training results (bottom), and manage your networks (center).

Create a Perceptron Network (NN Tool)

Create a perceptron network to perform the OR function in this example. It has an input vector data 1 = [0 0 1 1; 0 1 0 1] and a target vector data 2 = [0 1 1 1]. The network can be called OR Net. Once created, the network will be trained. Then save the network, its output, etc. by "exporting" it to the command line.

Input and target:

To start, type NN tool. The window shown in Figure 19-3 appears.

First step is to define the network input, data 1, as having the particular value [0 0 1 1; 0 1 0 1]. Thus, the network had a two-element input and four sets of such two-element vectors are presented to it in training. Create New Data appears. Set the Name to data 1, the Value to [0 0 1 1; 0 1 0 1], and make sure that Data Type is set to Inputs. The Create New Data window will then look like the one shown in Figure 19-4.

Now click Create to actually create an input file data 1. The Network/Data Manager window comes up and data 1 is shown as an input.

Step 2 is to create a network target. Click on New Data again, and this time enter the variable name data 2, specify the value [0 1 1 1] and click on Target under data type. The window will look the one given in Figure 19-5.

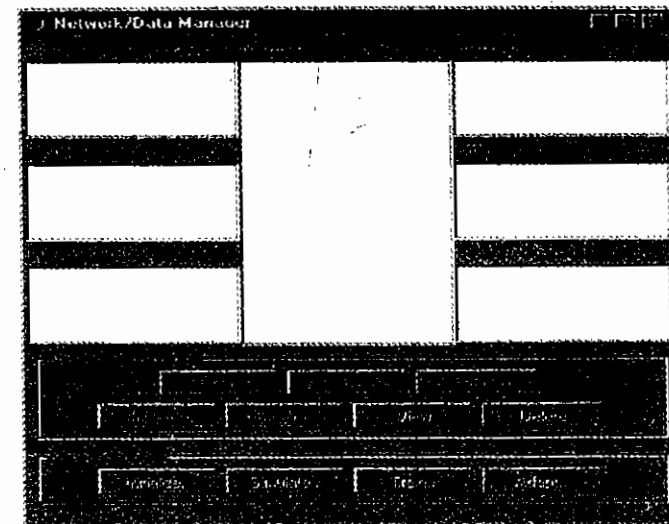


Figure 19-3 Neural network launch pad window network manager.

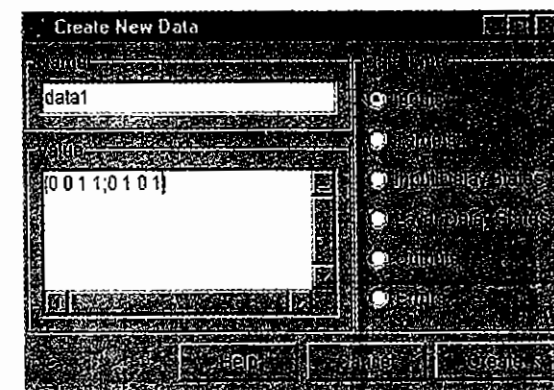


Figure 19-4 Window for creating new data (inputs).

Again click on Create and see the resulting Network/Data Manager window that has data 2 as a target as well as the previous data 1 as an input.

Create network:

Now we want to create a new network, which is OR NET. To do this, click on New Network, and a Create New Network window appears. Enter OR NET under Network Name. Set the Network Type to Perceptron, because that is the kind of network to create. The input ranges can be set by entering numbers in that field, but it is easier to get them from the particular input data that you want to use. To do this, click on the down

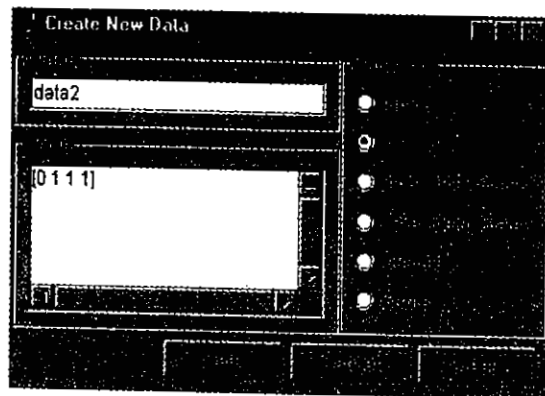


Figure 19-5 Window for creating new data (targets).

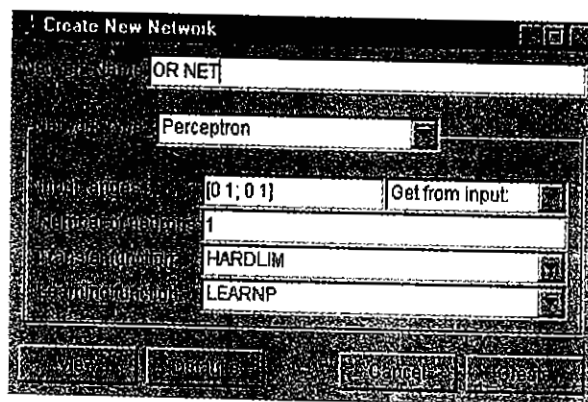


Figure 19-6 Window for creating new network.

arrow at the right side of **Input Range**. This pull-down menu shows that you can get the input ranges from the file *p*. This should lead to input ranges $[0 \ 1; \ 0 \ 1]$. We want to use a *hardlim* transfer function and a *learnp* learning function, so set those values using the arrows for **Transfer function** and **Learning function**, respectively. By now your **Create New Network** window should look like the one given in Figure 19-6.

Next you might look at the network by clicking on **View** (Figure 19-7). Figure 19-7 shows a network with a single input (composed of two elements), a *hardlim* transfer function and a single output which is going to be created. This is the perceptron network. Now click **Create** to generate the network. Now go back to the **Network/Data Manager** window. Note that **OR NET** is now listed as a network.

Train the perceptron:

To train the network, click on **OR NET** to highlight it. Then click on **Train**. This leads to a new window labeled **Network:OR NET** (Figure 19-8). At this point you can view the network again by clicking on the top tab **Train**. You can also check on the initialization by clicking on the top tab **Initialize**. Now click on the

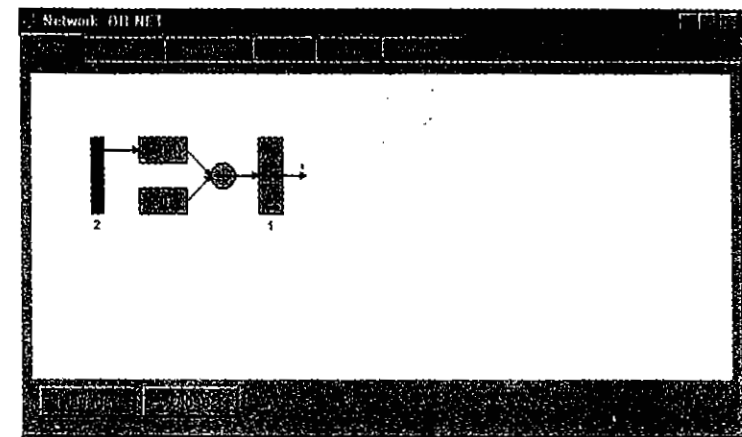


Figure 19-7 Window to view the new architecture created.

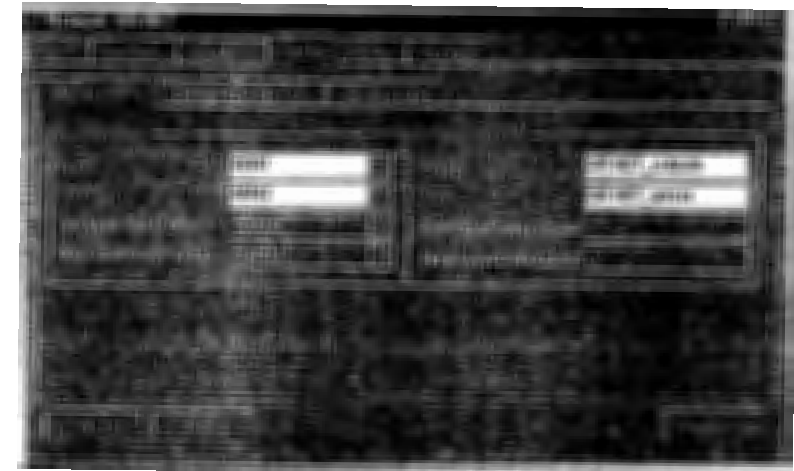


Figure 19-8 Perceptron net for AND function.

top tab **Train**. Specify the inputs and outputs by clicking on the left tab **Training Info** and selecting *data1* from the pop-down list of inputs and *data2* from the pull-down list of targets (Figure 19-8).

On clicking the **Training Parameters** tab, it shows parameters such as the epochs and error goal. These parameters can be changed at this point if desired. Now click **Train Network** to train the perceptron network and see the following training results (Figure 19-9).

Thus, the network was trained to zero error in four epochs. (Note that other kinds of networks commonly do not train to zero error and their errors commonly cover a much larger range. On that account, plot their errors on a log scale rather than on a linear scale such as that used above for perceptrons.)

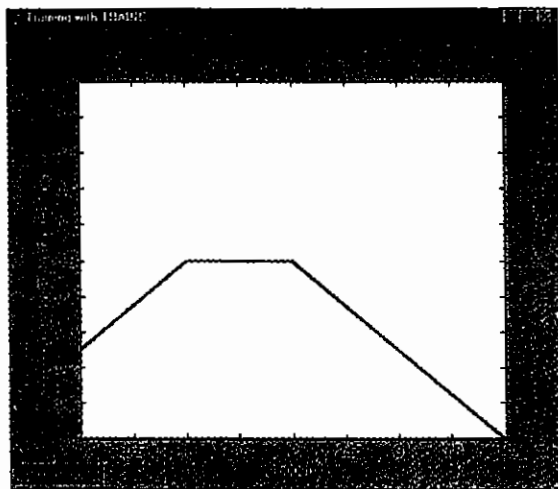


Figure 19-9 Training results of OR function.

To check that the trained network does indeed give zero error by using the input p and simulating the network, go to the **Network/Data Manager** window and click on **Network Only: Simulate**. This will bring up the **Network: OR NET** window. Click there on **Simulate**. Now use the **Input** pull-down menu to specify data 1 as the input, and label the output as **OR NET_outputsSim** to distinguish it from the training output. Now click **Simulate Network** in the lower-right corner. Look at the **Network/Data Manager**. It will show a new variable in the output: **OR NET_outputsSim**. Double-click on it and a small window **Data:OR NET_outputsSim** appears with the value $[0 \ 1 \ 1]$. Thus, the network does perform the OR of the inputs, giving 0 as an output only in this first case, when both inputs are 0.

19.5 Fuzzy Logic MATLAB Toolbox

Fuzzy logic in MATLAB can be dealt very easily because of the existing new Fuzzy Logic Toolbox. This provides a complete set of functions to design and implement various fuzzy logic processes. The major fuzzy logic operations include fuzzification, defuzzification and the fuzzy inference. These all are performed by means of various functions and can be even implemented using the GUI. Many of the applications can be simulated using the "fuzzy logic controller" simulink block present in Matlab Simulink toolbox. The features are the following.

1. It provides tools to create and edit Fuzzy Inference Systems (FIS).
2. It allows integrating fuzzy systems into simulation with SIMULINK.
3. It is possible to create stand-alone C programs that call on fuzzy systems built with MATLAB.

The Toolbox provides three categories of tools:

1. command line functions;
2. graphical or interactive tools;
3. SIMULINK blocks.

19.5.1 Commands in Fuzzy Logic Toolbox

The various commands in Fuzzy Logic Toolbox to be operated in command line are as follows:

GUI editors:

anfisedit:	ANFIS training and testing UI tool
findcluster:	Clustering UI tool.
fuzzy:	Basic FIS editor.
mfedit:	Membership function editor.
ruleedit:	Rule editor and parser.
ruleview:	Rule viewer and fuzzy inference diagram.
surfview:	Output surface viewer.

Membership functions:

dsigmf:	Difference of two sigmoid membership functions.
gauss2mf:	Two-sided Gaussian curve membership function.
gaussmf:	Gaussian curve membership function.
gbellmf:	Generalized bell-shaped curve membership function.
pimf:	Pi-shaped curve membership function.
psigmf:	Product of two sigmoid membership functions.
smf:	S-shaped curve membership function.
sigmf:	Sigmoid curve membership function.
trapmf:	Trapezoidal membership function.
trimf:	Triangular membership function.
zmf:	Z-shaped curve membership function.

Command line FIS functions:

addmf:	Add membership function to FIS.
addrule:	Add rule to FIS.
addvar:	Add variable to FIS.
defuzz:	Defuzzify membership function.
evalfis:	Perform fuzzy inference calculation.
evalmf:	Generic membership function evaluation.
gensurf:	Generate FIS output surface.
getfis:	Get fuzzy system properties.
mf2mf:	Translate parameters between functions.
newfis:	Create new FIS.
parsrule:	Parse fuzzy rules.
plotfis:	Display FIS input-output diagram.
plotmf:	Display all membership functions for one variable.
readfis:	Load FIS from disk.
rmmf:	Remove membership function from FIS.
rmvar:	Remove variable from FIS.
setfis:	Set fuzzy system properties.
showfis:	Display annotated FIS.
showrule:	Display FIS rules.
writfis:	Save FIS to disk.

Advanced techniques:

- anfis: Training routine for Sugeno-type FIS (MEX only).
- fcm: Find clusters with fuzzy c-means clustering.
- genfis1: Generate FIS matrix using generic method.
- genfis2: Generate FIS matrix using subtractive clustering.
- subclust: Estimate cluster centers with subtractive clustering.

Miscellaneous functions:

- convertfis: Convert v1.0 fuzzy matrix to v2.0 fuzzy structure.
- discfis: Discretize a fuzzy inference system.
- evalmmf: For multiple membership functions evaluation.
- fstrcat: Concatenate matrices of varying size.
- fuzarith: Fuzzy arithmetic function.
- findrow: Find the rows of a matrix that match the input string.
- genparam: Generates initial premise parameters for ANFIS learning.
- probor: Probabilistic OR.
- sugmax: Maximum output range for a Sugeno system.

GUI helper files:

- cmfdlg: Add customized membership function dialog.
- cmthdlg: Add customized inference method dialog.
- fisgui: Generic GUI handling for the Fuzzy Logic Toolbox.
- gfimfdlg: Generate FIS using grid partition method dialog.
- mfdlg: Add membership function dialog.
- mfdrag: Drag membership functions using mouse.
- popundo: Pull the last change off the undo stack.
- pushundo: Push the current FIS data onto the undo stack.
- savdlg: Save before closing dialog.
- statmsg: Display messages in a status field.
- updfis: Update Fuzzy Logic Toolbox GUI tools.
- wsdlg: Open from/save to workspace dialog.

19.5.2 Simulink Blocks in Fuzzy Logic Toolbox

Once fuzzy system is created using GUI tools or some other method, it can be directly embedded into SIMULINK using the fuzzy logic controller (FLC) block as shown in Figure 19-10.

Make sure that the FIS matrix corresponding to the fuzzy system is both in the MATLAB workspace and referred to by name in the dialog box associated with this FLC. Although it is possible to use the Fuzzy



Figure 19-10 Fuzzy logic controller Simulink block.

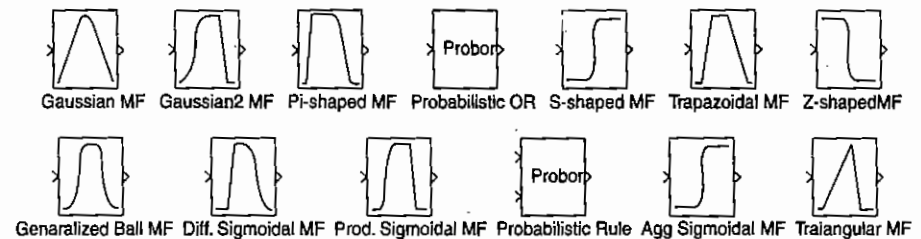


Figure 19-11 Membership functions.

Logic Toolbox by working strictly from the command line, in general it is much easier to build a system up graphically so that GUI tools are commonly used for building, editing and observing FIS.

The process of mapping from a given input to an output using fuzzy logic involves membership functions, fuzzy logic operators and If-Then rules.

Membership functions: This toolbox includes 11 built-in membership function types, built from several basic functions: piecewise linear functions (*triangular and trapezoidal*), the Gaussian distribution function (*Gaussian curves and generalized bell*), the sigmoid curve, and quadratic and cubic polynomial curves (*Z, S, and Pi curves*) (Figure 19-11).

Fuzzy logic operators: According to the fuzzy logical operations, any number of well-defined methods can fill in for the AND operation or the OR operation. In the Fuzzy Logic Toolbox, two built-in AND methods are supported: *min* (minimum) and *prod* (algebraic product). Two built-in OR methods are also supported: *max* (maximum) and the *probor* (probabilistic OR, also known as algebraic sum).

Based on **implication method**, two built-in methods are supported. These are the same functions that are used by the AND method, so that, *min* method truncates the output fuzzy set and *prod* scales the output fuzzy set.

Based on **aggregation method**, three built-in methods are supported: *max* (maximum), *probor* (probabilistic OR) and *sum* (simply the sum of each rule's output set).

Although **centroid calculation** is the most popular **defuzzification method**, there are five built-in methods supported: *centroid*, *bisector*, *middle of maximum*, *largest of maximum* and *smallest of maximum*.

If-Then rules: Since rules can be edited in three different formats (*verbose, symbolic and indexed*), verbose format makes the system easier to interpret. Every rule has a *weight* (a number between 0 and 1) which is applied to the number given by the antecedent. Generally this weight is 1 and so it has no effect at all on the implication process. For example, let us enter a sample rule (rule number one):

Verbose format: 1. if Temperature is warm then Sky is grey (1)

Symbolic format: 1. (Temperature == warm) => Sky = grey (1)

Indexed format: 1, 1 (1): 1

Here the first "1" corresponds to the input variable, the second corresponds to the output variable, the third displays the *weight* applied to each rule and the fourth is shorthand that indicates whether this is an OR (2) rule or an AND (1) rule. So a literal interpretation of rule number one is: "if input1 is MF1 (the first membership function associated with input 1) then output1 should be MF1 (the first membership function associated with output 1) with the weight 1". Note that as long as the aggregation method is commutative, the order in which the rules are executed is not important.

Once an FLC is created, it can be saved on a disk (FIS-file is created, i.e., juggler.fis) as an ASCII text format so that it can be edited and modified. An FLC can also be saved into MATLAB workspace as a matrix

variable (FIS matrix) so that it can be modified; however, its representation is extremely different from FIS-file representation.

19.5.3 Fuzzy Logic GUI Toolbox

The fuzzy logic can be simulated in MATLAB using GUI. On typing "fuzzy" in the command prompt, the fuzzy GUI toolbox opens up. The main windows corresponding to Fuzzy GUI tools are shown in Figures 19-12–19-17.

In Figure 19-12 the FIS editor – Mamdani or Sugeno model – is selected. The inference mechanism can be selected at this step. The various mechanisms to be selected in the FIS editor are AND method, OR method, implication, aggregation and defuzzification. Here, the number of input and output variables can be specified.

The membership function editor is shown in Figure 19-13. In this editor, for the input variable and the corresponding output variable, the membership functions using linguistic variables along with their range are defined. Figure 19-13 shows the membership function editor for the input variable and Figure 19-14 shows the membership function editor for the output variable.

The rules to be formed based on the input variables to get the output are defined in the rule editor. The inference of these rules gives the fuzzified output of the problem under consideration. For rule definition either AND connective or OR connective can be used. Figure 19-15 shows a rule editor with 3 rules formulated. The formulated rules can be viewed in the rule viewer as shown in Figure 19-16. On viewing these rules, information about the output can be obtained.

Figure 19-17 shows the surface view of the defined fuzzy inference editor. The fuzzy logic GUI toolbox helps us in designing a suitable FLC module for any application.

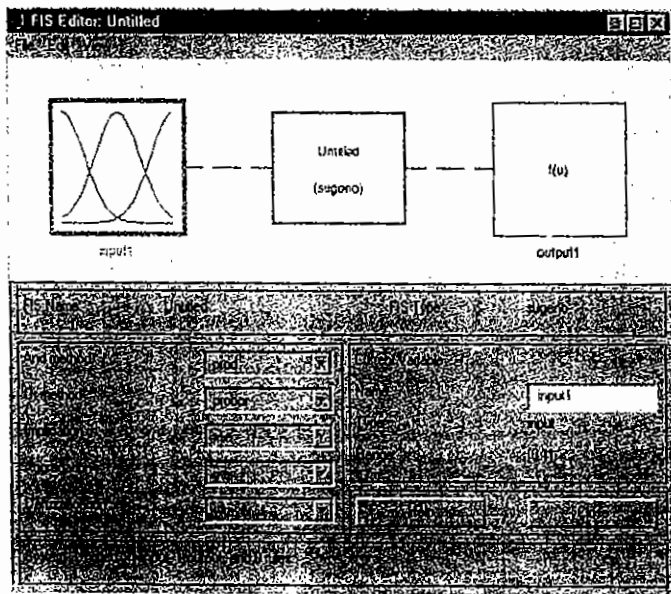


Figure 19-12 Basic FIS editor.

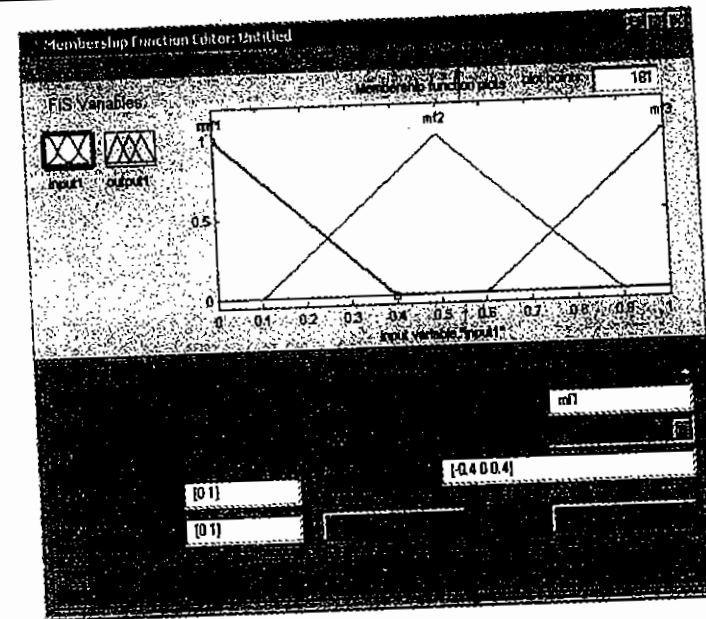


Figure 19-13 Membership function editor (input 1).

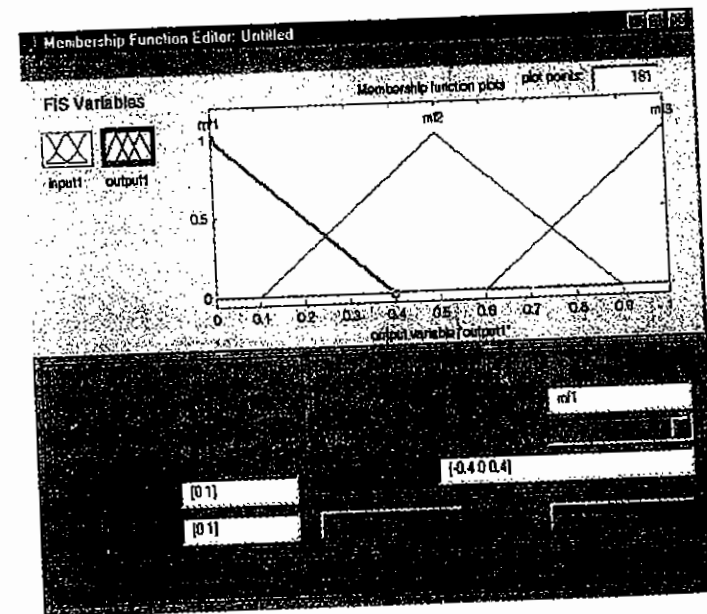


Figure 19-14 Membership function editor (output 1, in Sugeno-style).

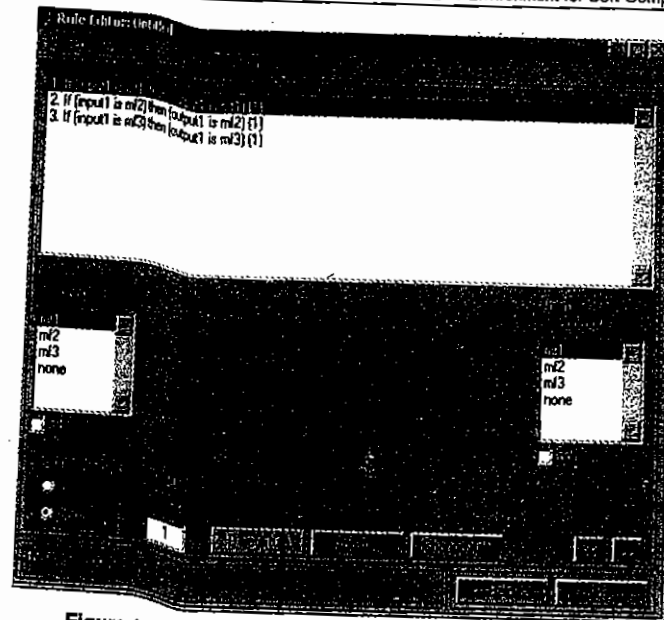


Figure 19-15 Rule editor (sample with 3 rules in verbose format).

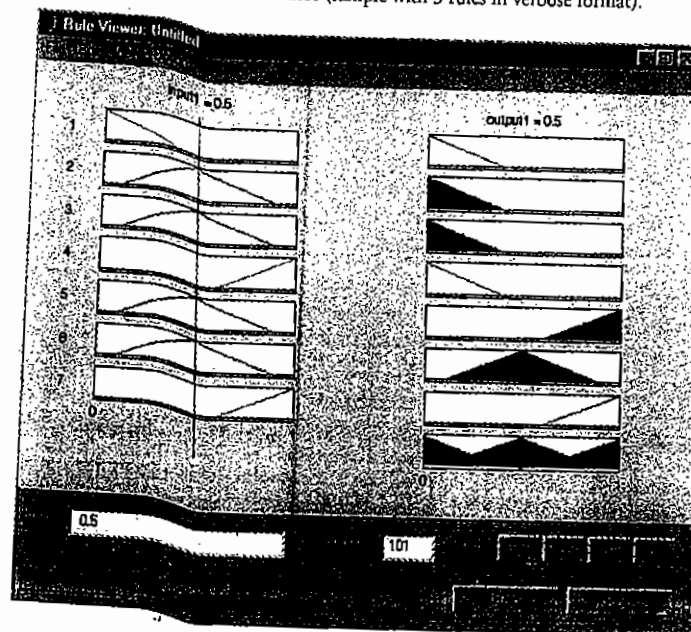


Figure 19-16 Rule viewer (in Sugeno-style).

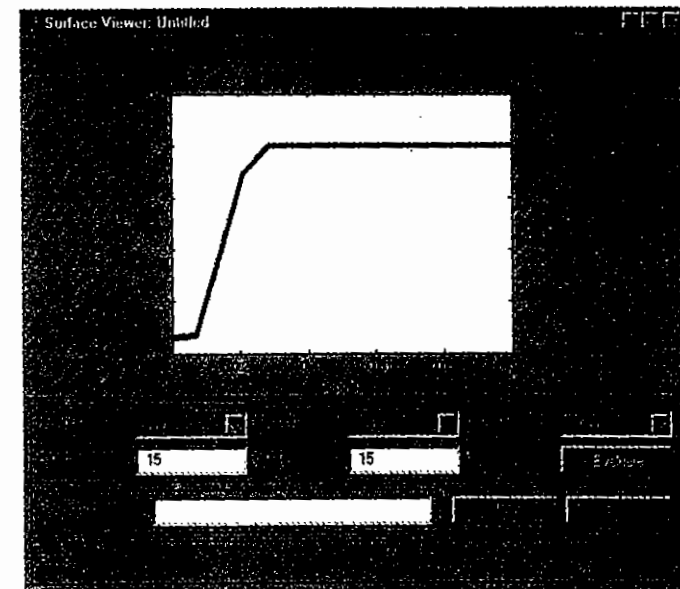


Figure 19-17 Surface viewer (input 1 and input 2 versus output 1).

19.6 Genetic Algorithm MATLAB Toolbox

The genetic algorithm (GA) is a method for solving both constrained and unconstrained optimization problems that are based on natural selection, the process that drives biological evolution. The GA repeatedly modifies a population of individual solutions. At each step, the GA selects individuals at random from the current population to be parents and uses them to produce the children for the next generation. Over successive generations, the population "evolves" toward an optimal solution. One can apply the GA to solve a variety of optimization problems that are not well suited for standard optimization algorithms, including problems in which the objective function is discontinuous, nondifferentiable, stochastic or highly nonlinear.

The GA and direct search toolbox are a collection of functions that extend the capabilities of the optimization toolbox and the MATLAB numeric computing environment. The GA and direct search toolbox include routines for solving optimization problems using

1. genetic algorithm;
2. direct search.

These algorithms enable you to solve a variety of optimization problems that lie outside the scope of the Optimization Toolbox.

The GA uses three main types of rules at each step to create the next generation from the current population:

1. *Selection rules* select the individuals, called *parents*, that contribute to the population at the next generation.
2. *Crossover rules* combine two parents to form children for the next generation.

3. Mutation rules apply random changes to individual parents to form children.

The GA at the command line calls the GA function ga with the syntax

```
[x fval] = ga(@fitnessfun, nvars, options)
```

where @fitnessfun is a handle to the fitness function; nvars is the number of independent variables for the fitness function; options is a structure containing options for the GA. If you do not pass in this argument, ga uses its default options.

The results are given by

x: Point at which the final value is attained.

fval: Final value of the fitness function.

The GA tool is a GUI that enables one to use the GA without working at the command line. To open the GA tool, enter

```
Gatool
```

at the MATLAB command prompt.

The Optimization Toolbox extends the MATLAB technical computing environment with tools and widely used algorithms for standard and large-scale optimization. These algorithms solve constrained and unconstrained continuous and discrete problems. The toolbox includes functions for linear programming, quadratic programming, nonlinear optimization, nonlinear least squares, nonlinear equations, multi-objective optimization and binary integer programming.

19.6.1 MATLAB Genetic Algorithm Commands

The various commands used in GA MATLAB toolbox are as follows.

bin2int: BINary string to INTEger string conversion.
 bin2real: BINary string to REAL vector conversion.
 bindecod: BINary DECODing to binary, integer or real numbers.
 compdiv: COMPUte DIVERse things of GEA Toolbox.
 compdiv2: COMPUte DIVERse things of GEA Toolbox.
 compete: COMPETition between subpopulations.
 Comploc: COMPUte LOCAL model things of toolbox.
 Compplot: COMPUte PLOT things of GEA toolbox.
 geamain2: MAIN function for Genetic and Evolutionary Algorithm toolbox for matlab.
 Initbp: CREAte an initial Binary Population.
 Initip: CREAte an initial (Integer value) Population.
 Initpop: INITIALIZation of POPulation (including inoculation).
 Initpp: Create an INITIAL Permutation Population.
 Initrp: INITIAlize a Real value Population.
 Migrate: MIGRATion of individuals between subpopulations.
 Mutate: high level MUTATion function.
 Mutbin: MUTATion for BINary representation.
 Mutbmd: real value Mutation like Discrete Breeder genetic algorithm.
 mutcomb: MUTATion for combinatorial problems.
 mutes1: MUTATion by Evolutionary Strategies 1, derandomized self adaption.
 mutes2: MUTATion by Evolutionary Strategies 2, derandomized self adaption.

mutexch: MUTATion by EXCHange.
 mutint: MUTATion for INTEger representation.
 Mutinvert: MUTATion by INVERTing variables.
 mutmove: MUTATion by MOVEing variables.
 Mutrand: MUTATion RANDOM.
 Mutrandbin: MUTATion RANDOM of binary variables.
 Mutrandint: MUTATion RANDOM of integer variables.
 Mutrandperm: MUTATion RANDOM of binary variables.
 mutrandreal: MUTATion RANDOM of real variables.
 Mutreal: real value Mutation like Discrete Breeder genetic algorithm.
 Mutswap: MUTATion by SWAPping variables.
 mutswaptyp: MUTATion by SWAPping variables of identical type.
 rankgoal: perform goal preference calculation between multiple objective values.
 Ranking: RANK-based fitness assignment, single and multi objective, linear and nonlinear.
 rankplt: RANK two multi objective values Partially Less Than.
 rankshare: SHARing between individuals.
 recdis: RECOMbination DIScrete.
 recdp: RECOMbination Double Point.
 recdprs: RECOMbination Double Point with Reduced Surrogate.
 recgp: RECOMbination Generalized Position.
 recint: RECOMbination extended INTERmediate.
 reclin: RECOMbination extended LINE.
 reclinex: EXtended LINE RECOMbination.
 recmp: RECOMbination Multi-Point, low level function.
 recombint: high level RECOMBINATIou function.
 recpm: RECOMbination Partial Matching.
 recsh: RECOMbination Shuffle.
 recshrs: RECOMbination SHuffle with Reduced Surrogate.
 recsp: RECOMbination Single Point.
 recsprs: RECOMbination Single Point with Reduced Surrogate.
 reins: high-level RE-INSErtion function.
 reinsloc: RE-INSErtion of offspring in population replacing parents LOCAL.
 reinsreg: REINSErtion of offspring in REGional population model replacing parents.
 selection: high level SELECTION function.
 sellocal: SELECTION in a LOCAL neighborhood.
 selrws: SELECTION by Roulette Wheel Selection.
 selsus: SELECTION by Stochastic Universal Sampling.
 seltour: SELECTION by TOURnament.
 seltrunc: SELECTION by TRUNCation.
 tbx3bin: ToolBoX function to define parameters for optimization of binary variables.
 tbx3comp: ToolBoX function to define parameters for COMPeting subpopulation.
 tbx3es1: ToolBoX function to define parameters for local oriented optimization of real variables.
 tbx3guifun1: ToolBoX function to define parameters for optimization, test of gui.
 tbx3int: ToolBoX function to define parameters for optimization of integer variables.
 tbx3perm: ToolBoX function to define parameters for optimization of permutation variables.

tbx3plot1: ToolBoX function to define parameters for graphical display options.
 tbx3real: ToolBoX function to define parameters for optimization of real variables.
 terminat: TERMINATION function.

Objective functions:

initdopi: INITIALIZATION function for DOuble Integrator objdopi.
 initfun1: INITIALIZATION function for de jong's FUNCTION 1.
 mopfonseca1: MultiObjective Problem: FONSECA's function 1.
 mopfonseca2: MultiObjective Problem: FONSECA's function 1.
 moptest: MultiObjective function TESTING.
 obj4wings: OBJECTIVE function FOUR-WINGS.
 objbtan: OBJECTIVE function for BRANin rcos function.
 objdopi: OBJECTIVE function for DOuble Integrator.
 objeaso: OBJECTIVE function for EASom function.
 objfletwell: OBJECTIVE function after FLETcher and PoWELL.
 objfractal: OBJECTIVE function Fractal Mandelbrot.
 objfun1: OBJECTIVE function for de jong's FUNCTION 1.
 objfun10: OBJECTIVE function for ackley's path FUNCTION 10.
 objfun11: OBJECTIVE function for langermann's FUNCTION 11.
 objfun12: OBJECTIVE function for michalewicz's FUNCTION 12.
 objfun1a: OBJECTIVE function for axis parallel hyper-ellipsoid.
 objfun1b: OBJECTIVE function for rotated hyper-ellipsoid.
 objfun1c: OBJECTIVE function for moved axis parallel hyper ellipsoid 1c.
 objfun2: OBJECTIVE function for rosenbrock's FUNCTION.
 objfun6: OBJECTIVE function for rastrigins FUNCTION 6.
 objfun7: OBJECTIVE function for schwefel's FUNCTION.
 objfun8: OBJECTIVE function for griewangk's FUNCTION.
 objfun9: OBJECTIVE function for sum of different power FUNCTION 9.
 objgold: OBJECTIVE function for GOLDstein-price function.
 objharv: OBJECTIVE function for HARVest problem.
 objint1: OBJECTIVE function for INT function 1.
 objint2: OBJECTIVE function for INT function 2.
 objint3: OBJECTIVE function for INT function 3.
 objint4: OBJECTIVE function for INT function 4.
 objlinq: OBJECTIVE function for discrete LINear Quadratic problem.
 objlinq2: OBJECTIVE function for LINear Quadratic problem 2.
 objone1: OBJECTIVE function for ONEmax function 1.
 objpush: OBJECTIVE function for PUSH-cart problem.
 objridge: OBJECTIVE function RIDGE.
 objsixh: OBJECTIVE function for SIX Hump camelback function.
 objsoland: OBJECTIVE function for SOLAND function.
 objtsp1: OBJECTIVE function for the traveling salesman example.
 objtsp1b: OBJECTIVE function for the traveling salesman library.
 plordopi: PLOTing of DO(Ppel)uble Integration results.
 plotstsp1b: PLOTing of results of TSP optimization (TSPLIB examples).
 simdopi1: M-file description of the SIMULINK system named SIMDOPI1.
 simdopiv: SIMulation Modell of DOPpelIntegrator, s-function, Vectorized.

simlinq1: M-file description of the SIMULINK system named SIMLINQ1.
 simlinq2: Modell of Linear Quadratic Problem, s-function.
 tsp_readlib: TSP utility function, reads TSPLIB data files.
 tsp_uscity: TSP utility function, reads US City definitions.

Plot functions:

Fitdistc: FITness DISTance Correlation computation.
 meshvar: create grafics of objective functions with plotmesh.
 plotmesh: PLOT of objective functions as MESH Plot.
 plotmop: PLOT properties of MultiObjective functions.
 reslook: LOOK at saved RESults.
 resplot: RESult PLOTing of GEA Toolbox optimization.
 samdata: sammon mapping: data examples.
 samgrad: Sammon mapping gradient calculation.
 sammon: Multidimensional scaling (SAMMON mapping).
 samobj: Sammon mapping objective function.
 samplot: Plot function for Multidimensional scaling (SAMMON mapping).

19.6.2 Genetic Algorithm Graphical User Interface

The GA tool is a GUI that enables you to use the GA without working at the command line. To open the GA tool, enter

gatool

at the MATLAB command prompt. This opens the tool as shown in Figure 19-18. To use the GA tool, you must first enter the following information.

1. *Fitness function:* The objective function you want to minimize. Enter the fitness function in the form @fitnessfun, where fitnessfun.m which is an M-file that computes the fitness function.
2. *Number of variables:* The number of variables in the given fitness function should be given.

The plot options

1. best fitness;
2. best individual;
3. distance;
4. expectation;
5. genealogy;
6. range;
7. score diversity;
8. scores;
9. selection;
10. stopping.

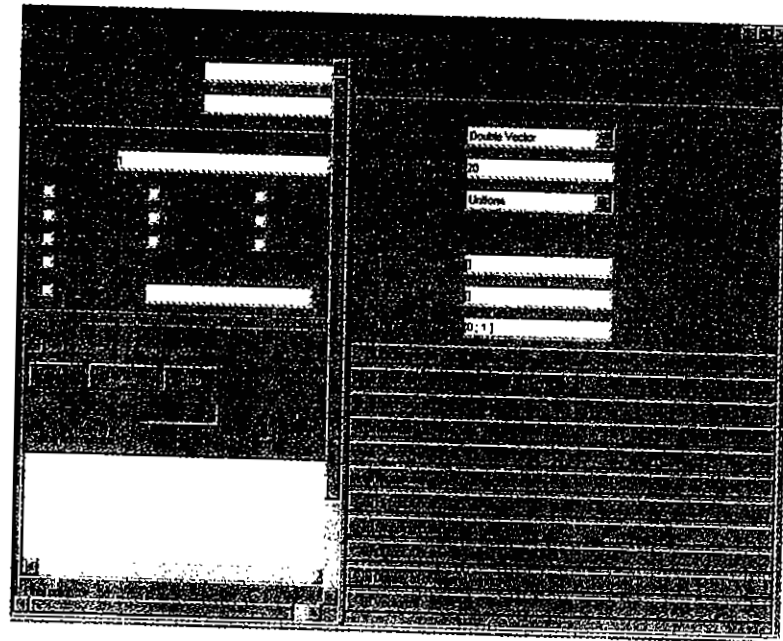


Figure 19-18 Genetic algorithm tool.

On the basis of the problem, custom function may also be built. The various parameters essential for running GA tool should be specified appropriately. The parameters appear on the right-hand side of the GA tool. The description is as follows:

1. **Population:** In this case population type, population size and creation function may be selected. The initial population and initial score may be specified, if not, the "GA tool" creates them. The initial range should be given.
2. **Fitness scaling:** It should be any of the following:
 - rank;
 - proportional;
 - top;
 - shift linear;
 - custom.
3. **Selection:** The selection is made on any one of the methods shown in Figure 19-19.
4. **Reproduction:** In reproduction the elite count and crossover fraction should be given. If elite count is not specified, it is taken as 2 (Figure 19-20).
5. **Mutation:** Generally Gaussian or uniform mutation is carried out. The user may define own customized mutation operation (Figure 19-21).

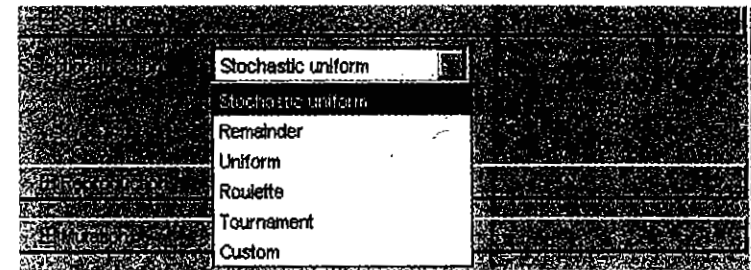


Figure 19-19 Selection.

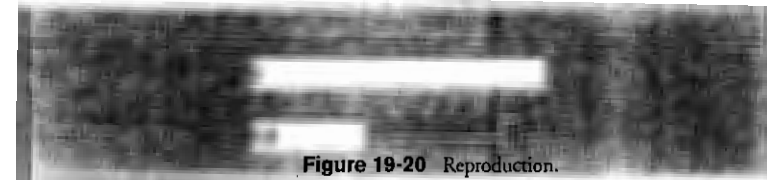


Figure 19-20 Reproduction.

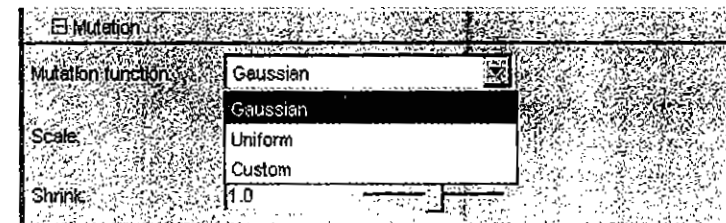


Figure 19-21 Mutation.

6. **Crossover:** The various crossover techniques are shown in Figure 19-22.
7. **Migration:** The parameter for migration should be defined as in Figure 19-23.
8. **Hybrid function:** Any one of the hybrid functions shown in Figure 19-24 may be selected.
9. **Stopping criteria:** The stopping criteria play a major role in simulation. They are shown in Figure 19-25.

The other parameters Output function, Display to command window and Vectorize may be suitably defined by the user.

Running and Simulation

The menu shown in Figure 19-26 helps the user for running the GA tool.

The running process may be temporarily stopped using "Pause" option and permanently stopped using "Stop" option. The "current generation" will be displayed during the iteration. Once the iterations are completed, the status and results will be displayed. Also the "final point" for the fitness function will be displayed.

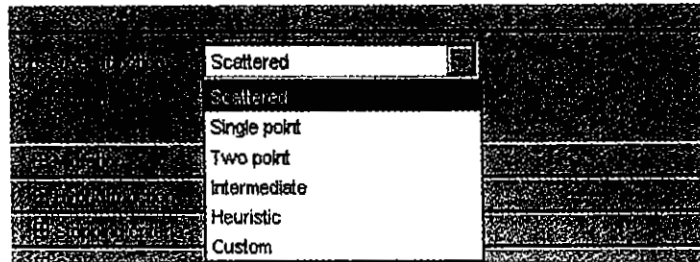


Figure 19-22 Crossover.

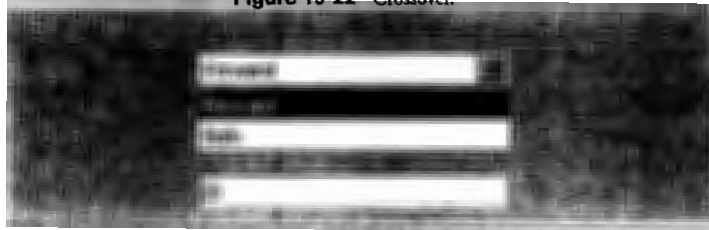


Figure 19-23 Migration.

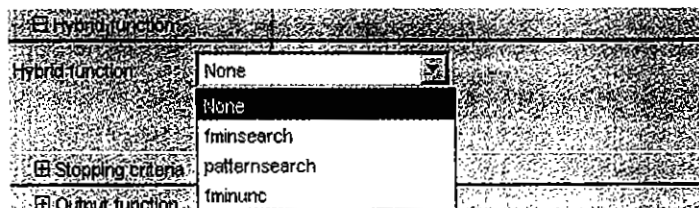


Figure 19-24 Hybrid function.

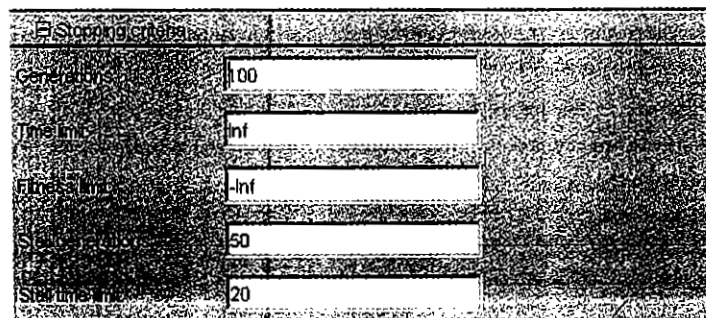


Figure 19-25 Stopping criteria.

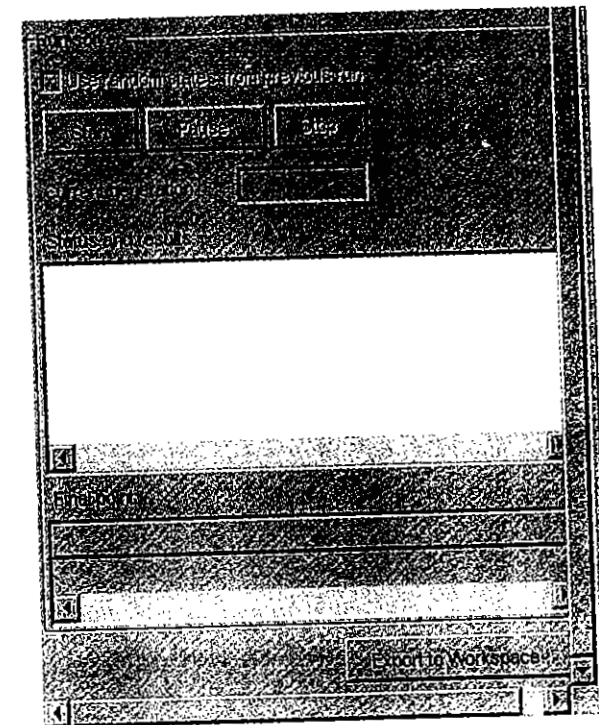


Figure 19-26 Run solver.

19.7 Neural Network MATLAB Source Codes

1. Write a program to implement AND function using ADALINE with bipolar inputs and outputs

Source Code

```
clear all;
clc;
disp('adaline network for and function bipolar inputs, bipolar
targets');
x1=[1 1 -1 -1]; %input pattern
x2=[1 -1 1 -1]; %input pattern
x3=[1 1 1 1]; %x3 for bias
t=[1 -1 -1 -1]; %target
w1=0.1;
w2=0.1;
b=0.1;
alpha=0.1;
e=2;
delw1=0;
delw2=0;
```

```

delb=0;
epoch=0;
while(e>1.018)
    epoch=epoch+1
    e=0;
    for i=1:4
        nety(i)=w1*x1(i)+w2*x2(i)+b;
        nt=[nety(i) t(i)]; %netinput,target
        delw1=alpha*(t(i)-nety(i))*x1(i);
        delw2=alpha*(t(i)-nety(i))*x2(i);
        delb=alpha*(t(i)-nety(i))*x3(i);
        wc=[delw1 delw2 delb]; %weight chances
        w1=w1+delw1; %updating of weights
        w2=w2+delw2;
        b=b+delb;
        w=[w1 w2 b]; %weights
        x=[x1(i) x2(i) x3(i)]; %input pattern
        pr=[x nt wc w] %to print the result
    end
    for i=1:4
        nety(i)=w1*x1(i)+w2*x2(i)+b;
        e=e+(t(i)-nety(i))^2;
    end
end

```

2. Write a program to implement AND function using MADALINE with bipolar inputs and outputs.

Source Code

```

clear all;
clc;
disp('madaline network for and function bipolar inputs, bipolar targets');
x1=[1 1 -1 -1]; %input pattern
x2=[1 -1 1 -1]; %input pattern
x3=[1 1 1 1]; %x3 for bias
t=[1 -1 -1 -1]; %target
w11=0.1;
w12=0.1;
w21=0.1;
w22=0.1;
b1=0.1;
b2=0.1;
b3=0.5;
v1=0.5;
v2=0.5;
alpha=0.5;
e=2;
delw11=0;
delw12=0;
delw21=0;
delw22=0;

```

```

delb1=0;
delb2=0;
delb3=0;
delv1=0;
delv2=0;
epoch=0;
while (e>1.00)
    epoch=epoch+1
    e=0;
    for i=1:4
        zin1=x1(i)*w11+x2(i)*w21+b1;
        zin2=x1(i)*w12+x2(i)*w22+b2;
        z=[zin1 zin2];
        if (zin1>=0)
            z1=1;
        else
            z1=-1;
        end
        if (zin2>=0)
            z2=1;
        else
            z2=-1;
        end
        hid=[z1 z2];
        nety=b3+z1*v1+z2*v2;

        if (nety>=0)
            y=1;
        else
            y=-1;
        end
        nt=[t(i) nety y];

        if (t(i)==1)
            if (zin1<z1n2)
                delb1=alpha*(1-zin1);
                b1=b1+delb1;
                delw11=alpha*(1-zin1)*x1(i);
                w11=w11+delw11;
                delw21=alpha*(1-zin1)*x1(i);
                w21=w21+delw21;
            else
                delb2=alpha*(1-zin2);
                b2=b2+delb2;
                delw12=alpha*(1-zin2)*x2(i);
                w12=w12+delw12;
                delw22=alpha*(1-zin2)*x2(i);
                w22=w22+delw22;
            end
        end
    end
end

```

```

elseif (t(i)==-1)
    if (zin1>0)
        delb1=alpha*(-1-zin1);
        b1=b1+delb1;
        delw11=alpha*(-1-zin1)*x1(i);
        w11=w11+delw11;
        delw21=alpha*(-1-zin1)*x1(i);
        w21=w21+delw21;
    else
        delb2=alpha*(-1-zin2);
        b2=b2+delb2;
        delw12=alpha*(-1-zin2)*x2(i);
        w12=w12+delw12;
        delw22=alpha*(-1-zin2)*x2(i);
        w22=w22+delw22;
    end
end

del=[delw11 delw21 delb1 delw12 delw22 delb2];
in=[x1(i) x2(i) x3(i)];
bi=[v1 v2 b3];

pr=[in z hid nt del bi]
end

for i=1:4
    zin1=b1+x1(i)*w11+x2(i)*w21;
    zin2=b2+x1(i)*w12+x2(i)*w22;
    z=[zin1 zin2];
    if (zin1>=0)
        z1=1;
    else
        z1=-1;
    end
    if (zin2>=0)
        z2=1;
    else
        z2=-1;
    end
    nety=v1*z1+v2*z2+b3;
    e=e+(t(i)-nety)^2;
end
end

```

3. Write a MATLAB program to construct and test auto associative network for input vector using HEBB rule.

Source Code

```

clear all;
clc;
disp(' AUTO ASSOCIATIVE NETWORK-----HEBB RULE');

```

```

w=[0 0 0 0 ; 0 0 0 0 ; 0 0 0 0 ; 0 0 0 0 ];
s=[1 1 1 -1];
t=[1 1 1 -1];
ip=[1 -1 -1 -1];
disp('INPUT VECTOR');
s
for i=1:4
    for j=1:4
        w(i,j)=w(i,j)+(s(i)*t(j));
    end
end
disp('WEIGHTS TO STORE THE GIVEN VECTOR IS');
w
disp('TESTING THE NET WITH VECTOR');
ip
yin=ip*w;
for i=1:4
    if yin(i)>0
        y(i)=1;
    else
        y(i)=-1;
    end
end
if y==s
    disp('PATTERN IS RECOGNIZED')
else
    disp('PATTERN IS NOT RECOGNIZED')
end

```

Output

```

>> AUTO ASSOCIATIVE NETWORK-----HEBB RULE
INPUT VECTOR
s =
    1    1    1   -1
WEIGHTS TO STORE THE GIVEN VECTOR IS
w =
    1    1    1   -1
    1    1    1   -1
    1    1    1   -1
   -1   -1   -1    1
TESTING THE NET WITH VECTOR
ip =
    1   -1   -1   -1
PATTERN IS NOT RECOGNIZED

```

4. Write a MATLAB program to construct and test auto associative network for input vector using outer product rule.

Source Code

```

clear all;
clc;

```


5. Write a MATLAB program to construct and test heteroassociative network for binary inputs and targets.

Source Code

```
%To construct and test Heteroassociative network for binary inputs
and targets
clear all;
clc;
disp('Heteroassociative Network');
x1=[1 0 0 0];
x2=[1 1 0 0];
x3=[0 0 0 1];
x4=[0 0 1 1];
t1=[1 0];
t2=[1 0];
t3=[0 1];
t4=[0 1];
n=0;
for i=1:4
    for j=1:2
        w(i,j)=((2*x1(i))-1)*((2*t1(j))-1)+((2*x2(i))-1)*((2*t2(j))-1)+
            ((2*x3(i))-1)*((2*t3(j))-1)+((2*x4(i))-1)*((2*t4(j))-1);
    end
end
w
yin1=x1*w
yin2=x2*w
yin3=x3*w
yin4=x4*w
t1=[ 1 -1];
t2=[ 1 -1];
t3=[-1 1];
t4=[-1 1];
for i=1:2
    if(yin1(i)>0)
        y1(i)=1;
    elseif (yin1(i)==0)
        y1(i)=0;
    else
        y1(i)=-1;
    end
end
y1
for i=1:4
    if(y1(i)==t1(i))
        n=n+1;
    end
end
if (n==2)
    disp('The pattern is matched');
else
```

```
disp('The pattern is not matched');
end
n=0;
for i=1:2
    if(yin2(i)>0)
        y2(i)=1;
    elseif (yin2(i)==0)
        y2(i)=0;
    else
        y2(i)=-1;
    end
end
y2
for i=1:2
    if(y2(i)==t2(i))
        n=n+1;
    end
end
if (n==2)
    disp('The pattern is matched');
else
    disp('The pattern is not matched');
end
n=0;
for i=1:2
    if(yin3(i)>0)
        y3(i)=1;
    elseif (yin3(i)==0)
        y3(i)=0;
    else
        y3(i)=-1;
    end
end
y3
for i=1:2
    if(y3(i)==t3(i))
        n=n+1;
    end
end
if (n==2)
    disp('The pattern is matched');
else
    disp('The pattern is not matched');
end
n=0;
for i=1:2
    if(yin4(i)>0)
        y4(i)=1;
    elseif (yin4(i)==0)
        y4(i)=0;
```

```

else
    y4(i)=-1;
end
end
y4
for i=1:2
    if(y4(i)==t4(i))
        n=n+1;
    end
end
if (n==2)
    disp('The pattern is matched');
else
    disp('The pattern is not matched');
end
n=0;

```

6. Write a MATLAB program to implement Discrete Hopfield Network and test the input pattern.

Source Code

```

clear all;
clc;
disp('Discrete Hopfield Network');
theta=0;
x=[1 -1 -1 -1;-1 1 1 -1;-1 -1 -1 1]
%Calculating Weight Matrix
w=x'*x
%calculating Energy
k=1;
while(k<=3)
    temp=0;
    for i=1:4
        for j=1:4
            temp=temp+(x(k,i)*w(i,j)*x(k,j));
        end
    end
    E(k)=(-0.5)*temp;
    k=k+1;
end
%Energy Function for 3 samples
E

%Test for given pattern s=[-1 1 -1 -1]
disp('Given input pattern for testing');
x1=[-1 1 -1 -1]
temp=0;
for i=1:4
    for j=1:4
        temp=temp+(x1(i)*w(i,j)*x1(j));
    end
end
end

```

```

SE=(-0.5)*temp
disp('By synchronous updation method');
disp('The net input calculated is');
yin=x1*w
for i=1:4
    if(yin(i)>theta)
        y(i)=1;
    elseif(yin(i)==theta)
        y(i)=yin(i);
    else
        y(i)=-1;
    end
end
disp('The output calculated from net input is');
y
temp=0;
for i=1:4
    for j=1:4
        temp=temp+(y(i)*w(i,j)*y(j));
    end
end
SE=(-0.5)*temp
n=0;
for i=1:3
    if (SE==E(i))
        n=0;
        k=i;
    else
        n=n+1;
    end
end

if(n==3)
    disp('Pattern is not associated with any input pattern');
else
    disp('The test pattern');
    x1
    disp('is associated with');
    x(k,:)
end

```

Output

```

>> Discrete Hopfield Network

x =

     1     -1     -1     -1
    -1      1      1     -1
    -1     -1     -1      1

```

```

w =
    3 -1 -1 -1
   -1  3  3 -1
   -1  3  3 -1
   -1 -1 -1  3
E = -10 -12 -10
Given input pattern for testing
x1 = -1 1 -1 -1
SE = -2
By synchronous updation method
The net input calculated is
yin = -2 2 2 -2
The output calculated from net input is
y = -1 1 1 -1
SE = -12
The test pattern
x1 = -1 1 -1 -1
is associated with
ans = -1 1 1 -1

```

7. Write a program to implement Kohonen self-organizing feature maps for given input pattern using learning rate as 0.6.

Source Code

```

clear all;
clc;
disp('Kohonen self organizing feature maps');
disp('The input patterns are');
x=[1 1 0 0; 0 0 0 1; 1 0 0 0; 0 0 1 1]
t=1;
alpha(t)=0.6;
e=1;
disp('Since we have 4 input pattern and cluster unit to be formed
is 2, the weight matrix is');
w=[0.2 0.8; 0.6 0.4; 0.5 0.7; 0.9 0.3]
disp('The learning rate of this epoch is');
alpha
while(e<=3)
    i=1;
    j=1;
    k=1;
    m=1;
    disp('Epoch =');
    e
    while(i<=4)
        for j=1:2
            temp=0;
            for k=1:4
                temp= temp + ((w(k,j)-x(i,k))^ 2);
            end

```

```

        D(j)=temp
    end
    if(D(1)<D(2))
        J=1;
    else
        J=2;
    end
    disp('The winning unit is ');
    J
    disp('Weight updation');
    for m=1:4
        w(m,J)=w(m,J) + (alpha(e) * (x(i,m)-w(m,J)));
    end
    w
    i=i+1;
end
temp=alpha(e);
e=e+1;
alpha(e)=(0.5*temp);
%disp('First Epoch completed');
%disp('Learning rate updated for second epoch');
alpha(e)
end

```

8. Write a MATLAB program to implement full counter propagation network for a given input pattern.

Source Code

```

clear all;
clc;
disp('FULL COUNTERPROPAGATION NETWORK');
x=[1 0 0 0];
y=[1 0];
alpha=0.4;
beta=0.3;
a=0.2;
b=0.1;
e=1;
v=[0.8 0.2; 0.8 0.2; 0.2 0.8; 0.2 0.8];
w=[0.5 0.5; 0.5 0.5];
t=[0.6 0.4 0.4 0.6];
u=[0.7 0.7];

while(e<=3)
    m=1;
    n=1;
    for j=1:2
        temp=0;
        for k=1:4
            temp= temp + ((v(k,j)-x(k))^ 2);
        end

```

```

    for k=1:2
        temp= temp + ((w(k,j)-y(k))^ 2);
    end
    D(j)=temp
end
if(D(1)<D(2))
    J=1;
else
    J=2;
end
disp('The winning unit is ');
J
disp('Weight updation');
for m=1:4
    v(m,J)=v(m,J) + (alpha(e) * (x(m)-v(m,J)));
end
v
for n=1:2
    w(n,J)=w(n,J) + (beta(e) * (y(n)-w(n,J)));
end
w

temalpha=alpha(e);
tembeta=beta(e);
tema=a(e);
temb=b(e);
oe=e;
te(e)=e;
e=e+1;
te(e)=e;
tel(oe)=oe;
alpha(e)=(0.5*temalpha);
alpha
beta(e)=(0.5*tembeta);
beta

disp('for Weight updation from cluster unit to output unit');
for m=1:4
    v(m,J)=v(m,J) + (alpha(e) * (x(m)-v(m,J)));
end
v
for n=1:2
    w(n,J)=w(n,J) + (beta(e) * (y(n)-w(n,J)));
end
w
for m=1:4
    t(m)=t(m) + (b(oe) * (x(m)-t(m)));
end
t
for n=1:2

```

```

    u(n)=u(n) + (a(oe) * (y(n)-u(n)));
end
u

a(e)=(0.5*tema);
b(e)=(0.5*temb);
a
b
end
tel(e)=e;
x1=te;
x2=tel;
y1=alpha;
y2=beta;
y3=a;
y4=b;
figure(1)
h=plot(x1,y1,x1,y2,x2,y3,x2,y4)
set(h,{'Color'},{'r','g','b','m'})
grid on
xlabel('EPOCH')
ylabel('ERROR RATE')
title('COUNTERPROPAGATION NETWORK')
legend(h,'alpha','beta','a','b')

```

The error rate versus epoch for full counter propagation network is shown in Figure 19-27.

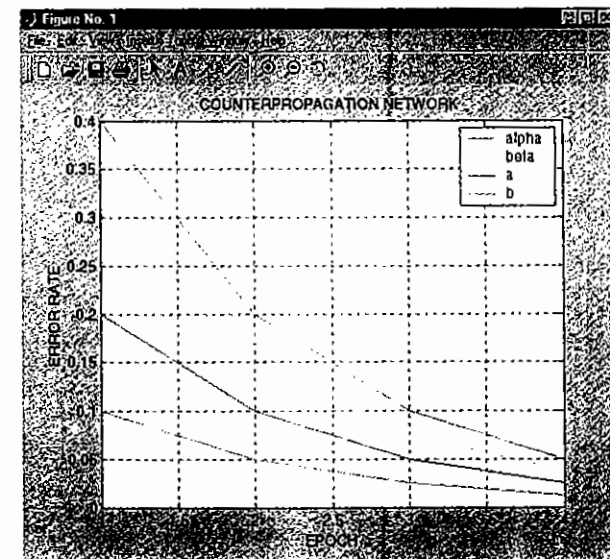


Figure 19-27 Epoch vs error rate for full counter propagation network.

9. Implement a back propagation network for a given input pattern by a suitable MATLAB program. Perform 3 epochs of operation.

Source Code

```
%back propagation network
clear all;
clc;
disp('Back propagation Network');
v=[0.7 -0.4;-0.2 0.3]
x=[0 1]
t=[1]
w=[0.5;0.1]
t1=0;
wb=-0.3
vb=[0.4 0.6]
alpha=0.25
e=1;
temp=0;
while (e<=3)
    e
    for i=1:2
        for j=1:2
            temp=temp+(v(j,i)*x(j));
        end
        zin(i)=temp+vb(i);
        temp1=e^(-zin(i));
        fz(i)=(1/(1+temp1));
        z(i)=fz(i);
        fdz(i)=fz(i)*(1-fz(i));
        temp=0;
    end

    for k=1
        for j=1:2
            temp=temp+z(j)*w(j,k);
        end
        yin(k)=temp+wb(k);
        fy(k)=(1/(1+(e^-yin(k))));
        y(k)=fy(k);
        temp=0;
    end

    for k=1
        fdy(k)=fy(k)*(1-fy(k));
        delk(k)=(t(k)-y(k))*fdy(k);
    end
    for k=1
        for j=1:2
            dw(j,k)=alpha*delk(k)*z(j);
        end
```

```
        dwb(k)=alpha*delk(k);
    end

    for j=1:2
        for k=1
            delin(j)=delk(k)*w(j,k);
        end
        delj(j)=delin(j)*fdz(j);
    end

    for i=1:2
        for j=1:2
            dv(i,j)=alpha*delj(j)*x(i);
        end
        dvb(i)=alpha*delj(i);
    end

    for k=1
        for j=1:2
            w(j,k)= w(j,k)+dw(j,k);
        end
        wb(k)=wb(k)+dwb(k);
    end
    w,wb

    for i=1:2
        for j=1:2
            v(i,j)=v(i,j)+dv(i,j);
        end
        vb(i)=vb(i)+dvb(i);
    end
    v,vb
    te(e)=e;
    e=e+1;
end
```

10. Write a program to implement ART 1 network for clustering input vectors with vigilance parameter.

Source Code

```
clear all;
clc;
disp('Adaptive Resonance Theory Network 1');
L=2;
m=3;
n=4;
rho=0.4;
te=L/(L-1+n);
te=te/2;
b=[te te te;te te te;te te te;te te te]
```

```

t=ones(3,4)
s=[1 1 0 0;0 0 0 1;1 0 0 0;0 0 1 1]
e=1;

while(e<=4)
    temp=0;
    for i=1:4
        temp=temp+s(e,i);
    end
    ns=temp;
    x(e,:)=s(e,:);
    for i=1:3
        temp=0;
        for j=1:4
            temp=temp+(x(e,j)*b(j,i));
        end
        yin(i)=temp;
    end
    j=1;
    if (yin(j)>=yin(j+1)& yin(j)>=yin(j+2))
        J=1;
    elseif (yin(j+1)>=yin(j)&yin(j+1)>=yin(j+2))
        J=2;
    else
        J=3;
    end
    J
    for i=1:4
        x1(i)=x(e,i)*t(J,i);
    end
    x1;
    temp=0;
    for i=1:4
        temp=temp+x1(i);
    end
    nx=temp;
    m=nx/ns;
    if (m<rho)
        yin(J)=-yin(J);
        j=1;
        if (yin(j)>=yin(j+1)&yin(j)>=yin(j+2))
            J=1;
        elseif (yin(j+1)>=yin(j)&yin(j+1)>=yin(j+2))
            J=2;
        else
            J=3;
        end
        J
    end
    for i=1:4

```

```

        temp=0;
        temp=L-1+nx;
        b(i,J)=(L*x1(i))/temp;
    end
    b
    for i=1:4
        t(J,i)=x1(i);
    end
    t
    e=e+1;
end

```

Output

```
>> Adative Resonance Theory Network 1
```

```

b =
    0.2000    0.2000    0.2000
    0.2000    0.2000    0.2000
    0.2000    0.2000    0.2000
    0.2000    0.2000    0.2000

```

```

t =
    1    1    1    1
    1    1    1    1
    1    1    1    1

```

```

s =
    1    1    0    0
    0    0    0    1
    1    0    0    0
    0    0    1    1

```

```

J = 1
b =
    0.6667    0.2000    0.2000
    0.6667    0.2000    0.2000
    0    0.2000    0.2000
    0    0.2000    0.2000

```

```

t =
    1    1    0    0
    1    1    1    1
    1    1    1    1

```

```

J = 2
b =
    0.6667    0    0.2000
    0.6667    0    0.2000
    0    0    0.2000
    0    1.0000    0.2000

```

```

t =
    1    1    0    0
    0    0    0    1
    1    1    1    1

```

```

J = 1
b =
    1.0000    0    0.2000
    0    0    0.2000
    0    0    0.2000
    0    1.0000    0.2000
t =
    1    0    0    0
    0    0    0    1
    1    1    1    1
J = 2
b =
    1.0000    0    0.2000
    0    0    0.2000
    0    0    0.2000
    0    1.0000    0.2000
t =
    1    0    0    0
    0    0    0    1
    1    1    1    1

```

11. Implement adaptive resonance theory network 2 for given inputs by a MATLAB program. Perform 2 iterations only.

Source Code

```

clear all;
clc;
disp('Adaptive Resonance Theory 2');
s=[0.8 0.6]
a=10;
b=10;
c=0.1;
d=0.9;
e=0;
rho=0.9;
theta=0.7;
wb={7.0 7.0};
wt=[0 0];
alpha=0.6;
it=1;
u=[0.0 0.0]
tem=0;
for i=1:2
    tem=s(i) 2+tem;
end
ns=sqrt(tem);
p=[0 0]
for i=1:2
    x(i)=s(i);
    w(i)=s(i);

```

```

    q(i)=p(i);
end
x
w
q
temp=0;
temp1=0;
for i=1:2
    temp=w(i) 2+temp;
    temp1=p(i) 2+temp1;
end
nw=sqrt(temp);
np=sqrt(temp1);

for i=1:2
    if (x(i)>=theta)
        fx=x(i);
    else
        fx=0;
    end
    if (q(i)>=theta)
        fq=q(i);
    else
        fq=0;
    end
    v(i)=fx+(b*fq);
end
v

tem=0;
for i=1:2
    tem=tem+v(i) 2;
end
nv=sqrt(tem);

disp('Updating F1 activation again');
for i=1:2
    u(i)=v(i);
    w(i)=s(i)+(a*u(i));
    p(i)=u(i);
end
u
w
p
tem=0;
temp=0;
for i=1:2
    tem=tem+w(i) 2;
    temp=temp+p(i) 2;
end

```

```

nw=sqrt(temp);
np=sqrt(temp);
for i=1:2
    x(i)=w(i);
    q(i)=p(i);
end
x
q
for i=1:2
    if (x(i)>=theta)
        fx=x(i);
    else
        fx=0;
    end
    if (q(i)>=rho)
        fq=q(i);
    else
        fq=0;
    end
    v(i)=fx+(b*fq);
end
v
disp('Computing signal to F2');
for i=1:2
    temp=0;
    temp=temp+wb(i)*p(i);
    y(i)=temp;
end
y
temp=0;
temp1=0;
for i=1:2
    temp=temp+v(i)^2;
    temp1=temp1+u(i)^2;
end
nv=sqrt(temp);
nu=sqrt(temp1);
for i=1:2
    u(i)=v(i);
    p(i)=u(i)+(d*wt(i));
end
u
p
temp=0;
for i=1:2
    temp=temp+p(i)^2;
end
np=sqrt(temp);
for i=1:2
    r(i)=(u(i)+c*p(i))/(e+nu+(c*np));

```

```

end
temp=0;
for i=1:2
    temp=r(i)^2+temp;
end
nr=sqrt(temp);
i=1;
if (y(i)>=y(i+1))
    J=1;
else
    J=2;
end
%Check for RESET
if (nr>=rho)
    for i=1:2
        w(i)=s(i)+a*u(i);
        v(i)=fx+b*fq;
        temp=0;
        tem=0;
        for i=1:2
            temp=temp+w(i)^2;
            tem=tem+p(i)^2;
        end
        nw=sqrt(temp);
        np=sqrt(tem);
        x(i)=w(i)/(e+nw);
        q(i)=p(i)/(e+np);
    end
end
disp('Update weights for 2 iterations');
while (it<=2)
    wt(J)=(alpha*d*u(J))+1+(alpha*d)*wt(J);
    wb(J)=(alpha*d*u(J))+1+(alpha*d)*wb(J);
    wt
    wb
    for i=1:2
        u(i)=v(i);
        p(i)=u(i)+d*wt(i);
        w(i)=s(i)+a*u(i);
        x(i)=w(i);
        q(i)=p(i);
        v(i)=fx+b*fq;
    end
    it=it+1;
end

```

12. A perceptron neural net uses a hard-limit transfer function. Plot this transfer function.

Source Code

```

%Plot of hard limit transfer function
x = -4:0.1:4;

```



```
y = hardlim(x);
plot(x,y)
```

Output

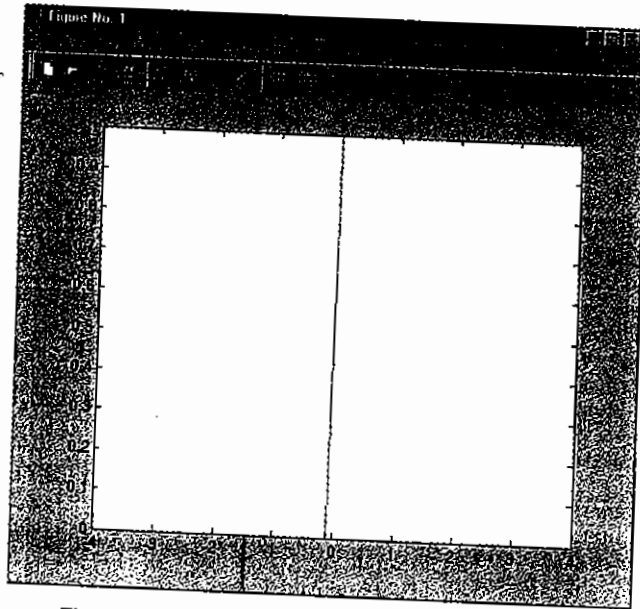


Figure 19-28 Plot of perceptron hard limit transfer function.

13. Create a perceptron network using the command "newp" and obtain its performance.

Source Code

```
%Program to create a perceptron network using command 'newp'
net = newp([-2 2; -2 2], 1);
%No. of epochs is given as 4
net.trainParam.epochs = 4;
%Let define the input vectors and the target vector
p = [ 2 ; 2 ] [1; -2] [-2 ; 2], [-1; 1] ];
t = [0 1 0 1];
%The net can be train with
net = train(net, p, t);
%Finally simulate the trained network for each of the inputs.
a = sim(net,p)
```

Output

```
TRAINC, Epoch 0/4
TRAINC, Epoch 3/4
TRAINC, Performance goal met.
```

```
a =
0 1 0 1
```

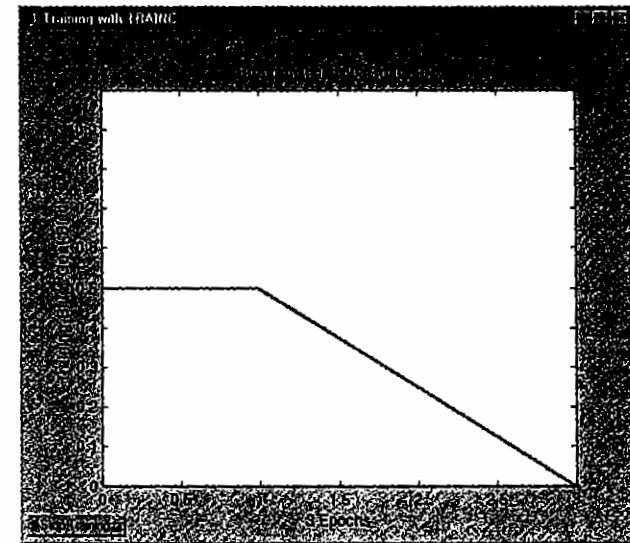


Figure 19-29 Training performance for a perceptron net.

14. Write a MATLAB program to create a feed forward network and perform Batch training

Source Code

```
%Program to create a feed forward network and perform batch training
%Create a training set of inputs p and targets t.
%For batch training, all of the input vectors are placed in one matrix.
p = [-1 -1 2 2; 0 5 0 5];
t = [-1 -1 1 1];
%Create the feedforward network. The function minmax is used to
%determine the range of the inputs to be used in creating the network.
net=newff(minmax(p),[3,1],{'tansig','purelin'},'traingd');
%Set training parameters.
net.trainParam.show = 50;
net.trainParam.lr = 0.05;
net.trainParam.epochs = 300;
net.trainParam.goal = 1e-5;
% Now train the network.
[net,tr]=train(net,p,t);
% The training record tr contains information about the progress
of training.
% Now the trained network can be simulated to obtain its response
to the inputs in the
% training set.
a = sim(net,p)
```

Output

```
>> TRAINGD, Epoch 0/300, MSE 0.69466/1e-005, Gradient 2.29478/1e-010
TRAINGD, Epoch 50/300, MSE 4.17837e-005/1e-005,
Gradient 0.00840093/1e-010
TRAINGD, Epoch 68/300, MSE 9.35073e-006/1e-005,
Gradient 0.0038652/1e-010
TRAINGD, Performance goal met.
a =
-1.0008 -0.9996 1.0053 0.9971
```

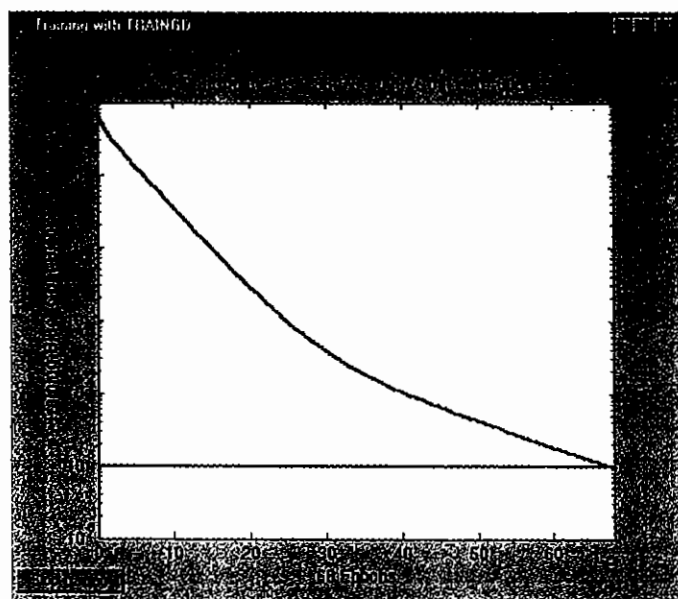


Figure 19-30 Training performance of a feed-forward network.

15. A radial basis network is a network with two layers. It consists of a hidden layer of radial basis neurons and an output layer of linear neurons. Plot a radial basis function.

Source Code

```
%Plot of radial basis function
clear all;
clc;
x = -5:.1:5;
y = radbas(x);
plot(x,y)
```

Output

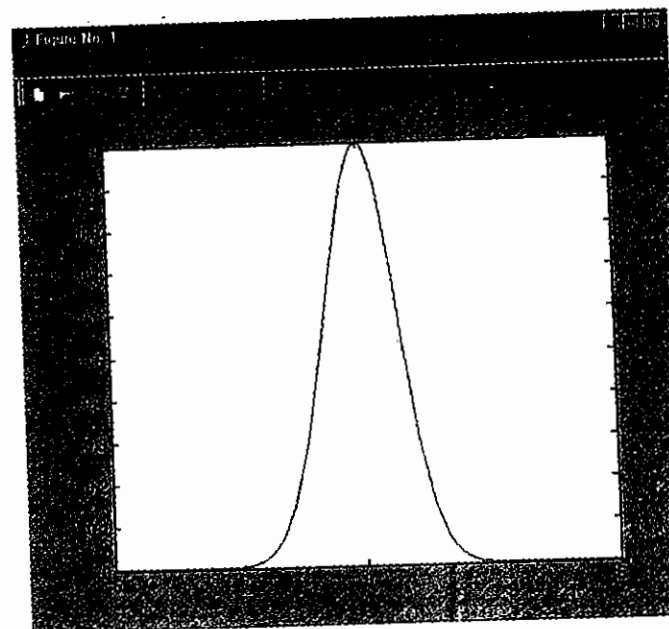


Figure 19-31 Radial basis function.

16. Consider a surface described by $z = \sin(x)\cos(y)$ defined on a square $-3 \leq x \leq 3, -3 \leq y \leq 3$.

- Plot the surface z as a function of x and y .
- Design a neural network which will fit the data. You should study different alternatives and test the final result by studying the fitting error.

Source Code

```
%Generate data
x = -3:0.25:3;
y = -3:0.25:3;
z = sin(x)'*cos(y);
surf(x,y,z);
xlabel('x axis');
ylabel('y axis');
zlabel('z axis');
title('surface z = sin(x)cos(y)');
%Store data in input matrix P and output vector T
P = [x;y];
T = z;
%Set small number of neurons in the first layer, say 25, 25 in
%the output.
%Initialize the network
```

```

net=newff([-3 3; -3 3], [25 25], {'tansig' 'purelin'}, 'trainlm');
%Apply Levenberg-Marquardt algorithm
%Define parameters
net.trainParam.show = 50;
net.trainParam.lr = 0.05;
net.trainParam.epochs = 300;
net.trainParam.goal = 1e-3;
%Train network
net1 = train(net,P,T);
a = sim(net1,P);
surf(x,y,a)

```

Output

TRAINLM, Epoch 0/300, MSE 6.57445/0.001, Gradient 1010.2/1e-010
 TRAINLM, Epoch 4/300, MSE 0.000424834/0.001, Gradient 10.0448/1e-010
 TRAINLM, Performance goal met.

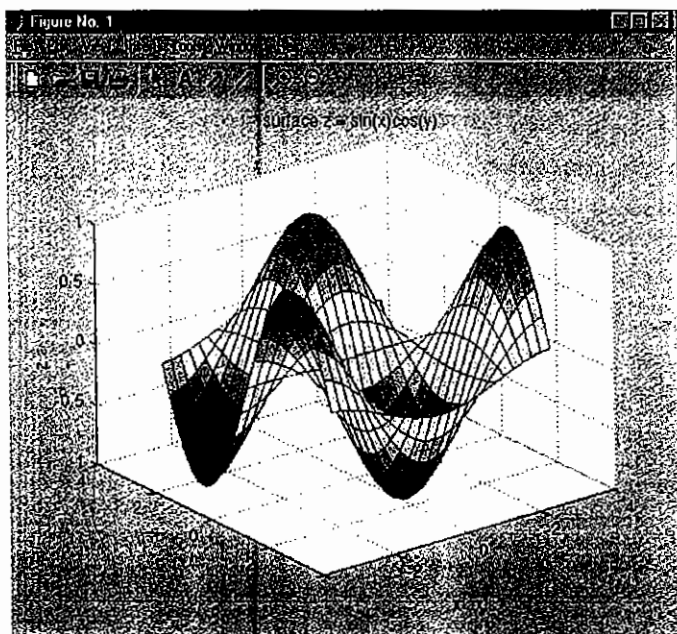


Figure 19-32 Surface for $z = \sin(x)\cos(y)$.

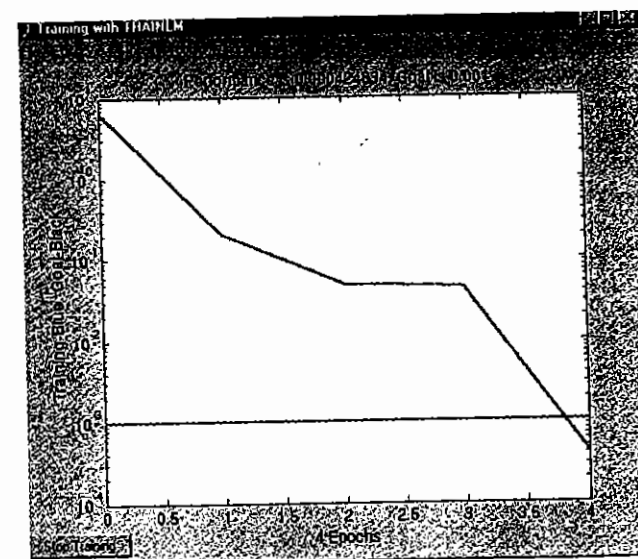


Figure 19-33 Training performance

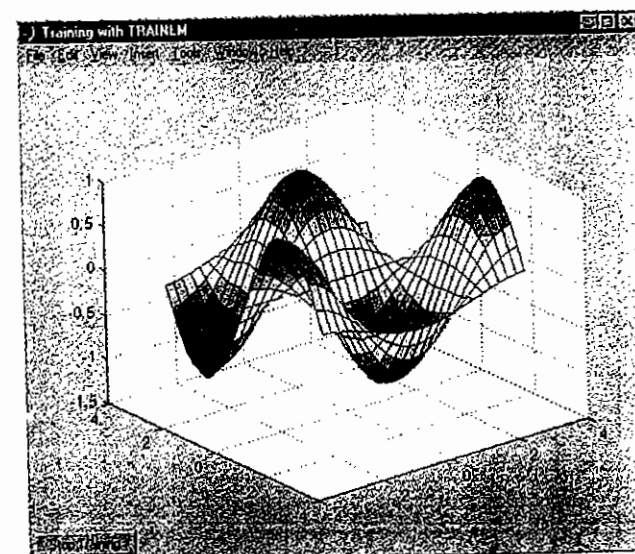


Figure 19-34 Surface of (x, y, a) .

17. Find a neural network model, which produces the same behavior as Van der Pol equation:

$$\dot{x} + (x^2 - 1)\dot{x} + x = 0$$

Solution: The given Van der Pol equation can be represented in state space form as

$$\begin{aligned}\dot{x}_1 &= x_2(1 - x_1^2) - x_1 \\ \dot{x}_2 &= x_1\end{aligned}$$

Here various initial functions can be used. On applying vector notations, the above state space form is given as

$$\dot{x} = f(x)$$

where

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \text{ and } f(x) = \begin{bmatrix} f_1(x_1, x_2) \\ f_2(x_1, x_2) \end{bmatrix} = \begin{bmatrix} x_2(1 - x_1^2) - x_1 \\ x_1 \end{bmatrix}$$

For building this Van der Pol model, Simulink is used.

Source Code

The simulink model for Van der Pol equation is shown in Figure 19-35.

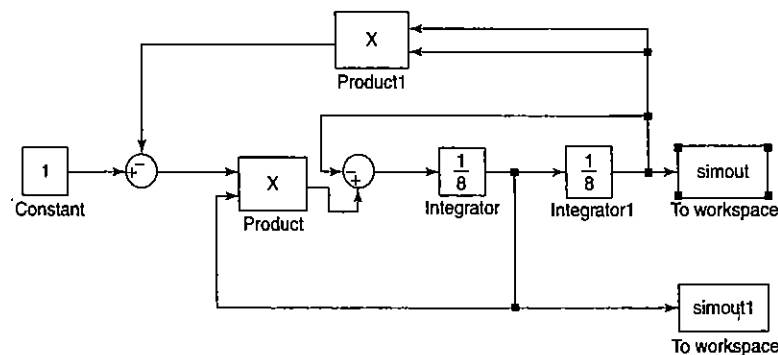


Figure 19-35 Simulink model for Van der Pol equation.

```
% Define the simulation parameters for Van der Pol equation
% The period of simulation: tfinal = 15 seconds;
tfinal = 15;
% Solve Van der Pol differential equation
[t,x]=sim('vandpol',tfinal);
% Plot the states as function of time
plot(t,x)
xlabel('time (secs)');
ylabel('x1 and x2 - states');
title('Van Der Pol Equation');
```

```
grid
%Plot of training vectors
P = t';
T = x';
plot(P,T,'+');
title('Training Vectors');
xlabel('Input Vector P');
ylabel('Target Vector T');
% Define the learning algorithm parameters- a feed forward
network chosen
net=newff([0 20], [10,2], {'tansig','purelin'},'trainlm');
%Define parameters
net.trainParam.show = 100;
net.trainParam.lr = 0.05;
net.trainParam.epochs = 500;
net.trainParam.goal = 1e-3;
%Train network
net1 = train(net, P, T);
```

Output

```
TRAINLM, Epoch 0/500, MSE 6.81368/0.001, Gradient 408.177/1e-010
TRAINLM, Epoch 100/500, MSE 0.0208639/0.001, Gradient 0.112283/1e-010
TRAINLM, Epoch 200/500, MSE 0.0208277/0.001, Gradient 0.00613187/1e-010
TRAINLM, Epoch 300/500, MSE 0.0208226/0.001, Gradient 0.0603704/1e-010
TRAINLM, Epoch 400/500, MSE 0.0208181/0.001, Gradient 1.62252/1e-010
TRAINLM, Epoch 500/500, MSE 0.0208168/0.001, Gradient 0.0403124/1e-010
TRAINLM, Maximum epoch reached, performance goal was not met.
```

The states of the Van der Pol equation are plotted as function of time as shown in Figure 19-36. The training vectors are shown in Figure 19-37.

The convergence has not occurred (performance goal not met), since network structure is simple. As a result, by modifying its structure, perform further iterations to achieve the performance goal. Figure 19-38 shows the training performance.

19.8 Fuzzy Logic MATLAB Source Codes

1. Write a MATLAB program to implement fuzzy set operation and properties

Source Code

```
%Program for fuzzy set with properties and operations
clear all;
clc;
disp('Fuzzy set with properties and operation');
a=[0 1 0.5 0.4 0.6];
b=[0 0.5 0.7 0.8 0.4];
c=[0.3 0.9 0.2 0 1];
phi=[0 0 0 0 0];
disp('Union of a and b');
```

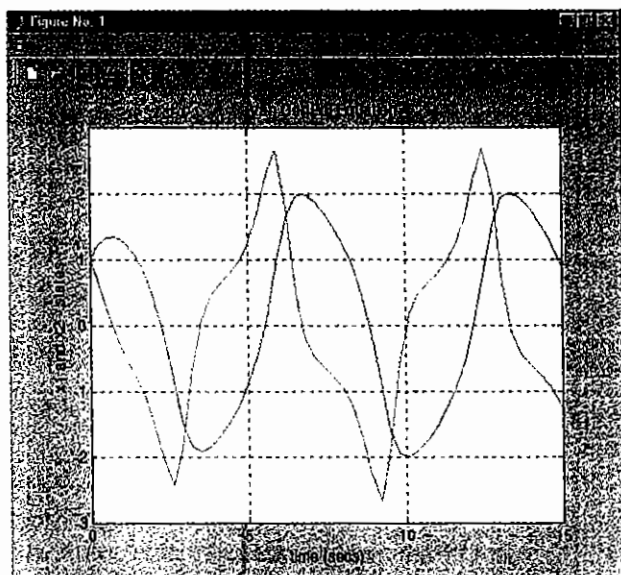


Figure 19-36 Plot of states vs time (Van der Pol equation).

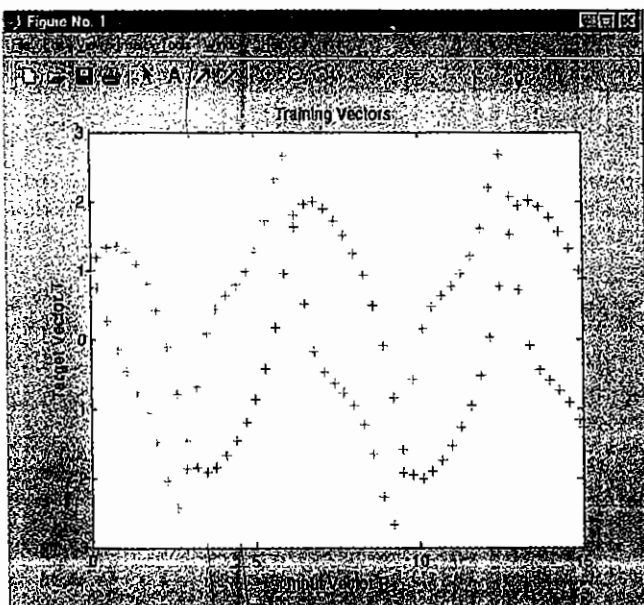


Figure 19-37 Plot of training vectors.

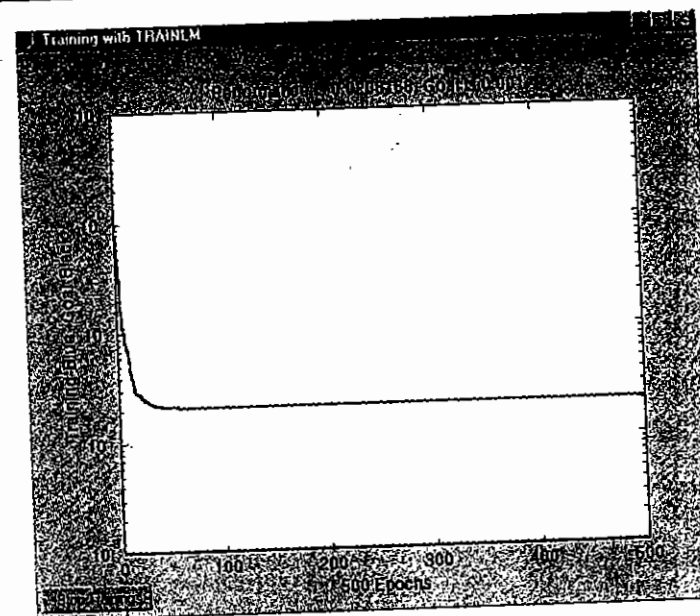


Figure 19-38 Training performance (goal not met).

```

au=max(a,b)
disp('Intersection of a and b');
iab=min(a,b)
disp('Union of b and a');
bu=max(b,a)
if (au==bu)
    disp('Commutative law is satisfied');
else
    disp('Commutative law is not satisfied');
end
disp('Union of b and c');
cu=max(b,c)
disp('a U (b U c)');
acu=max(a,cu)
disp('(a U b) U c');
auc=max(au,c)
if (acu==auc)
    disp('Associative law is satisfied');
else
    disp('Associative law is not satisfied');
end
disp('intersection of b and c');
ibc=min(b,c)
disp('a U (b I c)');

```

```

dls=max(a,ibc);
disp('Union of a and c');
uac=max(a,c);
disp('(a U b) I (a U c)');
drs=min(au,uac);
if (dls==drs)
    disp('Distributive law is satisfied');
else
    disp('distributive law is not satisfied');
end
disp('a U a');
idl=max(a,a);
a
if (idl==a)
    disp('Idempotency law is satisfied');
else
    disp('Idempotency law is not satisfied');
end
disp('a U phi');
idtl=max(a,phi);
a
if (idtl==a)
    disp('Identity law is satisfied');
else
    disp('Identity law is not satisfied');
end
disp('a I phi');
idtl=min(a,phi);
phi
if (idtl==phi)
    disp('Identity law is satisfied');
else
    disp('Identity law is not satisfied');
end
disp('Complement of (a I b)');
for i=1:5
    ciab(i)=1-iab(i);
end
ciab
disp('Complement of a');
for i=1:5
    ca(i)=1-a(i);
end
ca
disp('Complement of b');
for i=1:5
    cb(i)=1-b(i);
end
cb
disp('a Complement U b Complient');

```

```

dml=max(ca,cb);
if (dml==ciab)
    disp('Demorgans law is satisfied');
else
    disp('Demorgans law is not satisfied');
end
disp('Complement of complement of a');
for i=1:5
    cca(i)=1-ca(i);
end
cca
a
if (a==cca)
    disp('Involution law is satisfied');
else
    disp('Involution law is not satisfied');
end

```

2. Write a program to implement composition of Fuzzy and Crisp relations

Source Code

```

%program for composition on Fuzzy and Crisp relations
clear all;
clc;
disp('Composition on Crisp relation');
a=[0.2 0.6]
b=[0.3 0.5]
c=[0.6 0.7]
for i=1:2
    r(i)=a(i)*b(i);
    s(i)=b(i)*c(i);
end
r
s
irs=min(r,s)
disp('Crisp - Composition of r and s using max-min composition');
crs=max(irs)
for i=1:2
    prs(i)=r(i)*s(i);
end
prs
disp('Crisp - Composition of r and s using max-product composition');
mprs=max(prs)
disp('Fuzzy Composition');
firs=min(r,s)
disp('Fuzzy - Composition of r and s using max-min composition');
frs=max(firs)
for i=1:2
    fprs(i)=r(i)*s(i);
end
fprs

```

```
disp('Fuzzy - Composition of r and s using max-product composition');
fmprs=max(fprs)
```

3. Consider the following fuzzy sets

$$A = \left\{ \frac{1}{2} + \frac{0.4}{3} + \frac{0.6}{4} + \frac{0.3}{5} \right\}$$

$$B = \left\{ \frac{0.3}{2} + \frac{0.2}{3} + \frac{0.6}{4} + \frac{0.5}{5} \right\}$$

Calculate, $A \cup B$, $A \cap B$, $\bar{A}$, $\bar{B}$ by a MATLAB program.

Source Code

```
%Program to find union, intersection and complement of fuzzys sets
% Enter the two Fuzzy sets
u=input('enter the first fuzzy set A');
v=input('enter the second fuzzy set B');
disp('Union of A and B');
w=max(u,v)
disp('Intersection of A and B');
p=min(u,v)
[m]=size(u);
disp('Complement of A');
q1=ones(m)-u
[n]=size(v);
disp('Complement of B');
q2=ones(n)-v
```

Output

```
enter the first fuzzy set A[1 0.4 0.6 0.3]
enter the second fuzzy set B[0.3 0.2 0.6 0.5]
Union of A and B
w =
    1.0000    0.4000    0.6000    0.5000
Intersection of A and B
p =
    0.3000    0.2000    0.6000    0.3000
Complement of A
q1 =
    0    0.6000    0.4000    0.7000
Complement of B
q2 =
    0.7000    0.8000    0.4000    0.5000
```

4. Find whether the following relation is a tolerance relation or not by writing a MATLAB file.

$$R = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

Source Code

```
% Program to check whether the given relation is tolerance relation
or not
p=input('enter the relation')
sum=0;
sum1=0;
[m,n]=size(p);
if(m==n)
    for i=1:m
        if(p(1,1)==p(i,i))
            else
                fprintf(' the given relation is irrelexive and ');
                sum1=1;
                break;
            end
        end
    end
    if(sum1 ~= 1)
        fprintf('the given relation is reflexive and ');
    end
    for i=1:m
        for j=1:n
            if(p(i,j)==p(j,i))
                else
                    fprintf('not symmetry hence ');
                    sum=1;
                    break;
                end
            end
        end
    end
    if(sum==1)
        break;
    end
    end
    if(sum~=1)
        fprintf('symmetry hence ');
    end
end
if(sum1~=1)
    if(sum~=1)
        fprintf('the given relation tolerance relation');
    else
        fprintf(' the given relation is not tolerance relation');
    end
else
    fprintf(' the given relation is not tolerance relation');
end
end
```

Output

```
enter the relation[1 1 0 0 0;1 1 0 0 0;0 0 1 0 0;0 0 0 1 1;0 0 0 1 1]
```

```
p =
1 1 0 0 0
1 1 0 0 0
0 0 1 0 0
0 0 0 1 1
0 0 0 1 1
```

The given relation is reflexive and symmetry hence the given relation is a tolerance relation.

5. To find whether the following relation is equivalence or not using a MATLAB program.

$$R = \begin{bmatrix} 1 & 0.87 & 0 & 0.13 & 0.35 \\ 0.87 & 1 & 0.46 & 0 & 0.98 \\ 0 & 0.46 & 1 & 0 & 0 \\ 0.13 & 0 & 0 & 1 & 0.54 \\ 0.24 & 0.98 & 0 & 0.54 & 1 \end{bmatrix}$$

Source Code

```
%Program to check whether the given relation
%is an Equivalence relation or not
p=input('enter the matrix')
sum=0;
sum1=0;
sum2=0;
sum3=0;
[m,n]=size(p);
l=m;
if(m==n)
    for i=1:m
        if(p(l,l)==p(i,i))
            else
                fprintf(' the given relation is irreflexive ');
                sum1=1;
                break;
            end
        end
    end
    if(sum1 ~= 1)
        fprintf(' the given relation is reflexive');
    end
    m;
    n;
    [m,n]=size(p)
    for i=1:m
        for j=1:n
            if(p(i,j)==p(j,i))
                else
                    fprintf(' , not symmetry');
                    sum=1;
                    break;
                end
            end
        end
    end
```

```
if(sum==1)
    break;
end
end
if(sum~=1)
    fprintf(' ,symmetry');
end
for i=1:m
    for j=1:n
        for k=1:-1:1
            lambda1=p(i,j);
            lambda2=p(j,k);
            lambda3=p(i,k);
            q=min(lambda1,lambda2);
            if(lambda3 >= q)
                else
                    sum2=1;
                    break;
                end
            end
        end
    end
end
if(sum2 ~= 1)
    fprintf(' and transitivity hence ');
else
    fprintf(' and not transitivity hence ');
end
if(sum1~=1)
    if(sum~=1)
        if(sum2~=1)
            fprintf(' the given relation is equivalence relation');
        else
            fprintf('the given relation is not equivalence relation');
        end
    else
        fprintf('not equivalence relation');
    end
else
    fprintf('not equivalence relation');
end
end
end
```

Output

```
enter the matrix,1 0.87 0 0.13 0.35;0.87 1 0.46 0 0.98; 0 0.46 1 0 0;
0.13 0 0 1 0.54;0.24 0.98 0 0.54 1]
```

```
p =
1.0000 0.8700 0 0.1300 0.3500
0.8700 1.0000 0.4600 0 0.9800
0 0.4600 1.0000 0 0
0.1300 0 0 1.0000 0.5400
0.2400 0.9800 0 0.5400 1.0000
```


The given relation is reflexive, not symmetry and not transitivity and hence not an equivalence relation.

6. Find the fuzzy relation using fuzzy max-min method for the following using MATLAB program:

$$R = \begin{bmatrix} 0.2 & 0.3 & 0.4 \\ 0.3 & 0.5 & 0.7 \\ 1 & 0.8 & 0.6 \end{bmatrix} \text{ and } S = \begin{bmatrix} 0.1 & 1 \\ 0.4 & 0.2 \\ 0.3 & 0.7 \end{bmatrix}$$

```
%Program to find a relation using Max-Min Composition
%enter the two vectors whose relation is to be find
R=input('enter the first vector')
S=input('enter the second vector')
% find the size of two vectors
[m,n]=size(R)
[x,y]=size(S)
if(n==x)
for i=1:m
for j=1:y
c=R(i,:)
d=S(:,j)
f=d'
%find the minimum of two vectors
q=min(c,f)
%find the maximum of two vectors
h(i,j)=max(q);
end
end
%print the result
display('the fuzzy relation between two vectors is');
display(h)
else
display('The fuzzy relation cannot be find')
end
```

Output

```
enter the first vector[0.2 0.3 0.4;0.3 0.5 0.7;1 0.8 0.6]
R =
    0.2000    0.3000    0.4000
    0.3000    0.5000    0.7000
    1.0000    0.8000    0.6000
enter the second vector[0.1 1;0.4 0.2;0.3 0.7]
S =
    0.1000    1.0000
    0.4000    0.2000
    0.3000    0.7000
ans =
the fuzzy relation between two vectors is
h =
    0.3000    0.4000
    0.4000    0.7000
    0.4000    1.0000
```

7. Use MATLAB commands to display the triangular and Gaussian membership function. Given $x = 0$ to 10 with increment of 0.1. Triangular membership function is defined between [5 6 7] and Gaussian function is defined between 2 and 4.

Source Code

```
%Program to depict membership functions
x=(0:0.1:10)';
y1=gaussmf(x,[2 4]);
%Plot of Gaussian membership function
plot(x,y1)
hold
%Plot of Triangular membership function
y2=trimf(x,[5 6 7]);
plot(x,y2)
```

Output

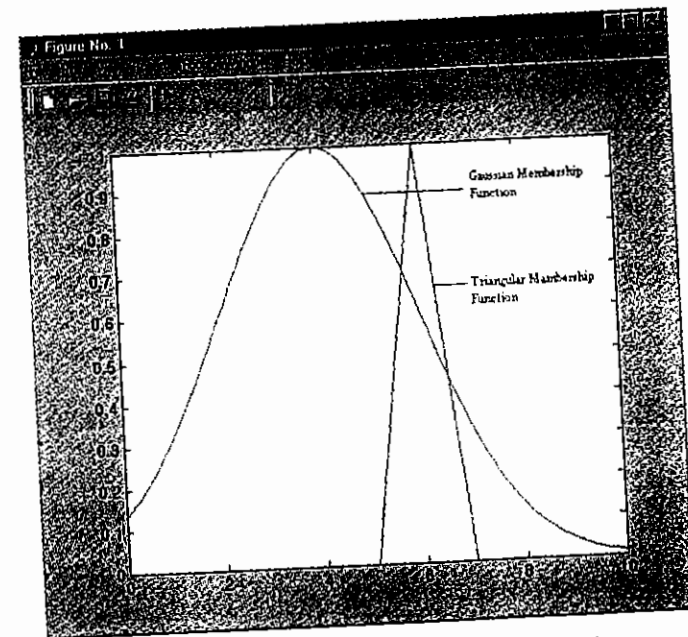


Figure 19-39 Gaussian and triangular membership functions.

8. Find the fuzzy relation between two vectors R and S using max-product method by a MATLAB program.

$$R = \begin{bmatrix} 0.2 & 0.3 & 0.4 \\ 0.3 & 0.5 & 0.7 \\ 1 & 0.8 & 0.6 \end{bmatrix} \text{ and } S = \begin{bmatrix} 0.1 & 1 \\ 0.4 & 0.2 \\ 0.3 & 0.7 \end{bmatrix}$$

Source Code

```
%Program to find a relation using Max-Product Composition
%enter the two input vectors
R=input('enter the first vector')
S=input('enter the second vector')
%find the size of the two vector
[m,n]=size(R);
[x,y]=size(S);
if(n==x)
for i=1:m
for j=1:y
c=R(i,:);
d=S(:,j);
[f,g]=size(c);
[h,q]=size(d);
%finding product
for l=1:g
e(l,l)=c(l,1)*d(1,l);
end
%finding maximum
t(i,j)=max(e);
end
end
disp('Max-product composition relation is');
disp(t)
else
display('Cannot find relation using max product composition');
end
```

9. Using MATLAB program find the crisp lambda cut set relations for $\lambda = 0.6$. The fuzzy matrix is given by

$$R = \begin{bmatrix} 0.1 & 0.6 & 0.8 & 1 \\ 1 & 0.7 & 0.4 & 0.2 \\ 0 & 0.6 & 1 & 0.5 \\ 0.1 & 0.5 & 1 & 0.9 \end{bmatrix}$$

Source code

```
%Lambda Cut method of defuzzification
% Enter the given relational matrix
R=input('Enter the relational matrix')
% Enter the lambda value
lambda=input('enter the lambda value')
[m,n]=size(R);
for i=1:m
for j=1:n
if(R(i,j)<lambda)
b(i,j)=0;
```

```
else
b(i,j)=1;
end
end
end
% output value
display('the crisp value is')
display(b)
```

Output

Enter the relational matrix[0.1 0.6 0.8 1;1 0.7 0.4 0.2;0 0.6 1 0.5;
0.1 0.5 1 0.9]

```
R =
0.1000 0.6000 0.8000 1.0000
1.0000 0.7000 0.4000 0.2000
0 0.6000 1.0000 0.5000
0.1000 0.5000 1.0000 0.9000
```

enter the lambda value 0.6

```
lambda =
0.6000
ans =
the crisp value is
b =
0 1 1 1
1 1 0 0
0 1 1 0
0 0 1 1
```

19.9 Genetic Algorithm MATLAB Source Codes

1. Write a MATLAB program for maximizing $f(x) = x^2$ using GA, where x is ranges from 0 to 31. Perform 5 iterations only.

Steps involved

- Step 1: Generate initial four populations of binary string with 5 bits length.
- Step 2: Calculate corresponding x and fitness value $f(x) = x^2$.
- Step 3: Use the tournament selection method to generate new four populations.
- Step 4: Apply crossover operator to the new four populations and generate new populations.
- Step 5: Apply mutation operator for each population.
- Step 6: Repeat the steps 2-5 for 5 iterations.
- Step 7: Finally print the result.

Source Code

```
%program for Genetic algorithm to maximize the function  $f(x) = x^2$ 
clear all;
```

```

clc;
%x ranges from 0 to 31 2power5 = 32
%five bits are enough to represent x in binary representation
n=input('Enter no. of population in each iteration');
nit=input('Enter no. of iterations');
%Generate the initial population
[oldchrom]=initbp(n,5)
%The population in binary is converted to integer
FieldD=[5;0;31;0;0;1;1]
for i=1:nit
    phen=bindecod(oldchrom,FieldD,3); % phen gives the integer value
    % of the binary population
    %obtain fitness value
    sqx=phen.^ 2;
    sumsqx=sum(sqx);
    avsqx=sumsqx/n;
    hsqx=max(sqx);
    pselect=sqx./sumsqx;
    sumpselect=sum(pselect);
    avpselect=sumpselect/n;
    hpselect=max(pselect);
    %apply roulette wheel selection
    FitnV=sqx;
    Nsel=4;
    newchrix=selrws(FitnV, Nsel);
    newchrom=oldchrom(newchrix,:);
    %Perform Crossover
    crossrate=1;
    newchromc=recsp(newchrom,crossrate); %new population after crossover
    %Perform mutation
    vlub=0:31;
    mutrate=0.001;
    newchromm=mutrandbin(newchromc,vlub,0.001); %new population
    after mutation
    disp('For iteration');
    i
    disp('Population');
    oldchrom
    disp('X');
    phen
    disp('f(X)');
    sqx
    oldchrom=newchromm;
end

```

Output

Enter no. of population in each iteration4
 Enter no. of iterations5
 At the end of fifth iteration, the output is

```

For iteration
i =
    5
Population
oldchrom =
    0  0  0  0  1
    0  0  0  1  0
    0  1  1  0  0
    0  1  0  0  0
X
phen =
    1
    2
   12
    8
f(X)
sqx =
    1
    4
   144
    64

```

2. Use Gatoool and minimize the quadratic equation $f(x) = x^2 + 3x + 2$ within the range $-6 \leq x \leq 0$.

Function Definition

Define the given function $f(x) = x^2 + 3x + 2$ in a separate m-file as shown in Figure 19-40.

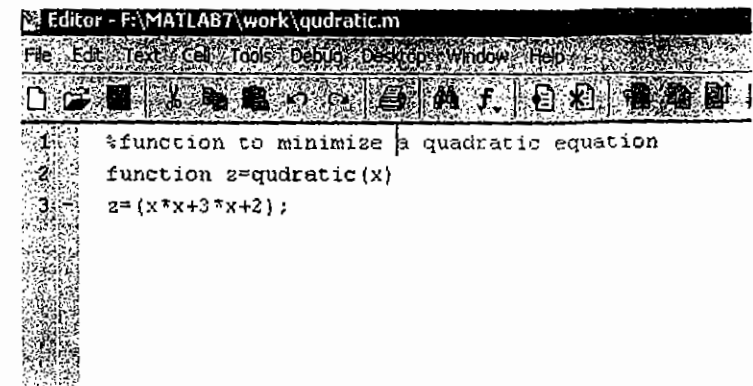


Figure 19-40 M-file showing defined quadratic function.

Creation of Gatoool

On typing "gatoool" in the command prompt, the GA toolbox opens. In tool, for fitness value type @quadratic and mention the number of variables defined in the function. Select best fitness in plot and specify the other parameters as shown in Figure 19-41.

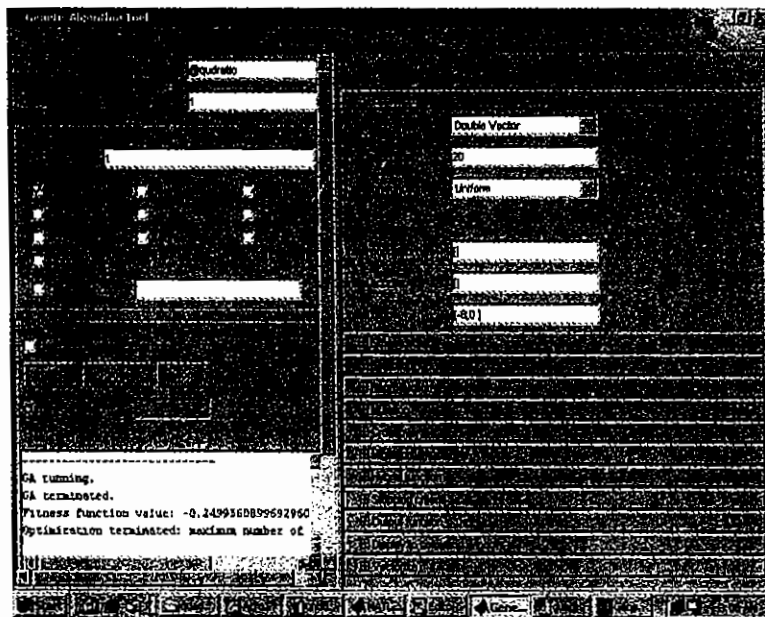


Figure 19-41 Genetic algorithm tool for quadratic equation.

Output

The output showing the best fitness for 50 generations is shown in Figure 19-42.

The status and results for this functions for 50 generations are shown in Figure 19-43.

3. Create a Gatool to maximize the function $f(x_1, x_2) = 4x_1 + 5x_2$ within the range 1–1.1.

Function Definition

Define the given function $f(x_1, x_2) = 4x_1 + 5x_2$ in a separate m-file as shown in Figure 19-44.

Creation of Gatool

On typing "gatool" in the command prompt, the GA toolbox opens. In tool, for fitness value type @twofunc and mention the number of variables defined in the function. Select best fitness and best individual in plot and specify the other parameters as shown in Figure 19-45.

Output

The output for 50 generations is as shown in Figure 19-46. The output also shows the best individual.

The status and result for this function are shown in Figure 19-47.

4. Use Gatool and minimize the function $f(x_1, x_2, x_3) = -5 \sin(x_1) \sin(x_2) \sin(x_3) + [-\sin(5x_1) \sin(5x_2) \sin(5x_3)]$, where $0 \leq x_i \leq \pi$, for $1 \leq i \leq 3$.

Function Definition

Define the given function

$$f(x_1, x_2, x_3) = -5 \sin(x_1) \sin(x_2) \sin(x_3) + [-\sin(5x_1) \sin(5x_2) \sin(5x_3)]$$

in a separate m-file as shown in Figure 19-48.

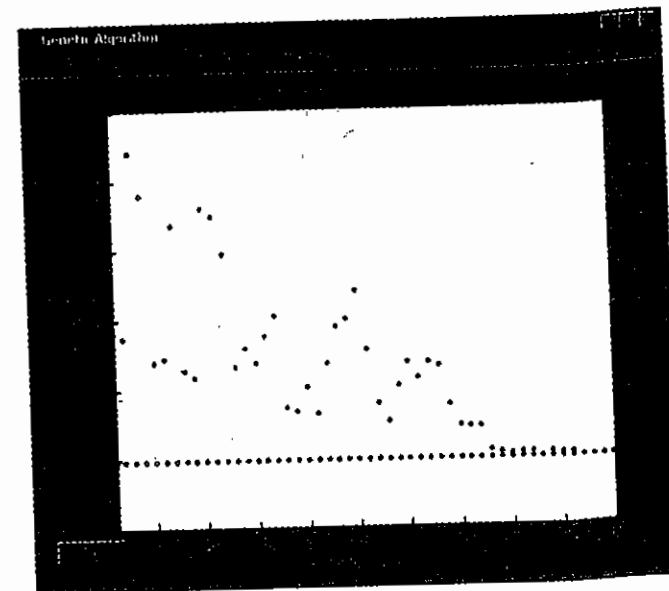


Figure 19-42 Output response (best fitness).

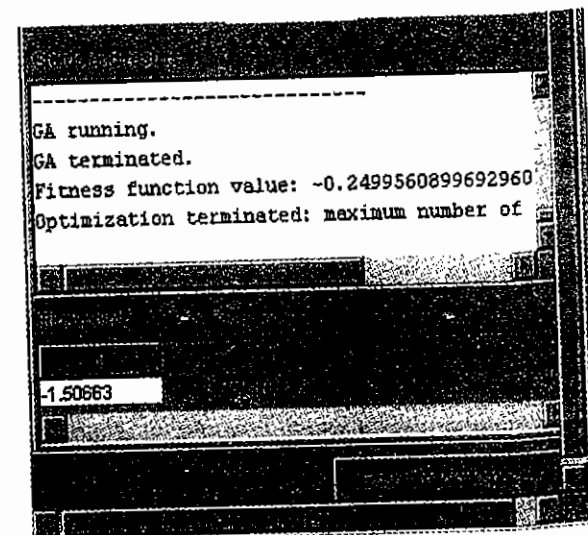


Figure 19-43 Status and results.

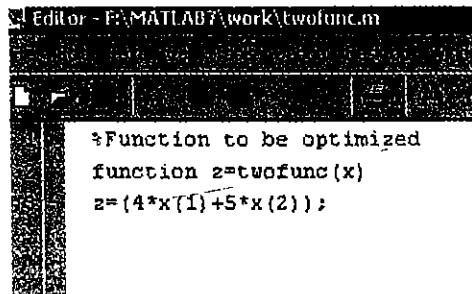


Figure 19-44 M-file showing defined function.

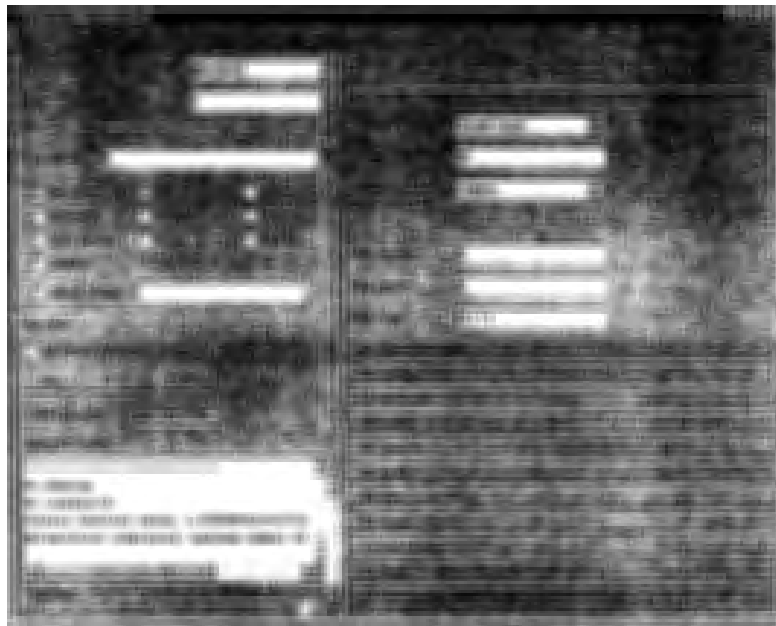


Figure 19-45 Genetic algorithm tool for given function.

Creation of Gatool

On typing "gatool" in the command prompt, the GA toolbox opens. In tool, for fitness value type @sinefn and mention the number of variables defined in the function. Select best fitness in plot and specify the other parameters as shown in Figure 19-49.

Output

The output for 100 generations is as shown in Figure 19-50. The status and result for this function are shown in Figure 19-51.

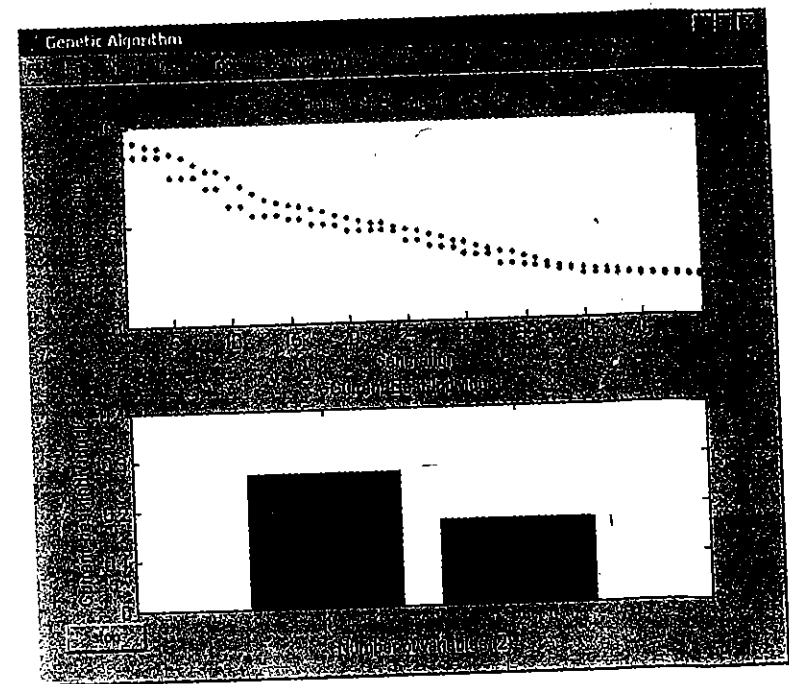


Figure 19-46 Output response (best fitness and best individual).

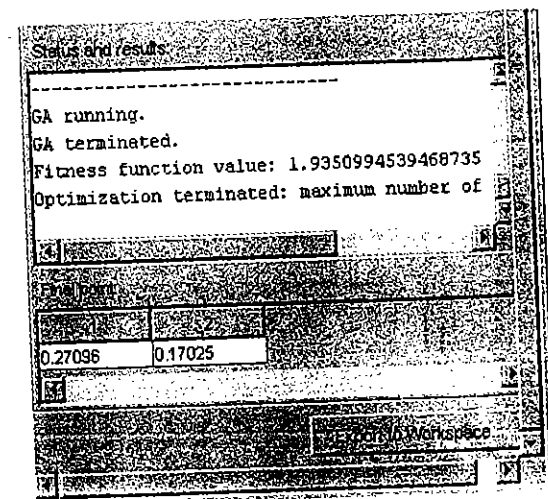


Figure 19-47 Status and results.

```

Editor - E:\MATLAB7\work\sinefn.m

%Function to minimize the given function
function z=sinefn(x)
z= -(5*sin(x(1))*sin(x(2))*sin(x(3)))+
    |-(sin(5*x(1))*sin(5*x(2))*sin(x(3)))|

```

Figure 19-48 M-file showing defined sine function.

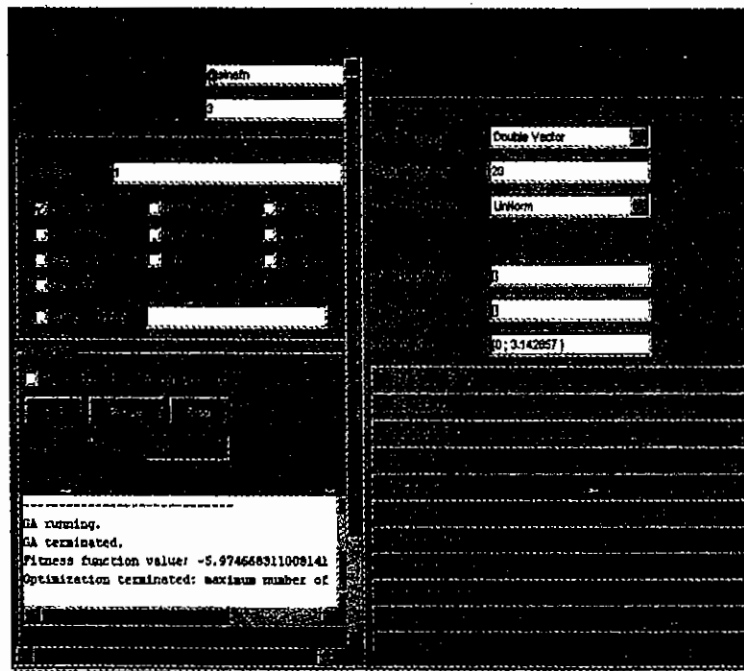


Figure 19-49 Genetic algorithm tool for sine equation.

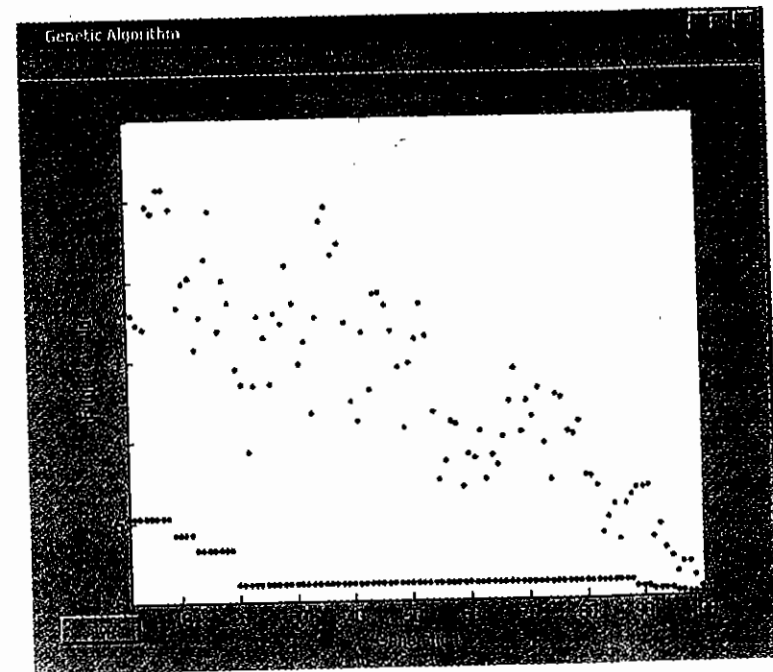


Figure 19-50 Output response (best fitness).

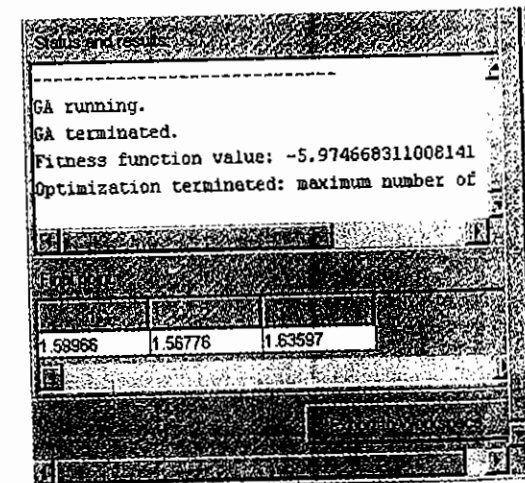


Figure 19-51 Status and results.

19.10 Summary

In this chapter soft computing techniques are implemented using MATLAB software. MATLAB software is very user-friendly and enables the user to simulate the ideas of soft computing for their applications. This chapter provides an overview of the various commands and GUI module involved in MATLAB for neural networks, fuzzy logic and GA approaches. The source codes developed using these commands and GUI toolbox for the soft computing techniques have also been included for the ready reference of the reader.

19.11 Exercise Problems

1. Implement the AND function using perceptron network using a MATLAB program.
2. Write a MATLAB program to apply back propagation network for a pattern recognition problem.
3. Implement OR function with bipolar inputs and targets with a MADALINE neural net.
4. Write a program to create an ART 1 network to cluster 7 input units and 3 cluster units.
5. Develop a Kohonen self-organizing feature map for a image recognition problem.
6. Write a program to implement various operations of fuzzy sets.
7. Implement the properties of fuzzy sets using an m-file.
8. Develop an m-file to perform compositional operations in fuzzy relations.
9. Maximize Rosenbrock's function using a MATLAB program.
10. Minimize Rastrigin's function using MATLAB GUI GA toolbox.
11. Given a polynomial equation of the form $f(x) = 4x^4 + 3x^3 + 2x^2 + x + 7$, find its roots using GA approach.
12. Consider a hyperbolic tangent function. Maximize it within the range $0 < x < 22/7$. Apply two-point crossover and tournament selection process. Construct a GA GUI toolbox.
13. Find the roots of the quadratic equation using genetic algorithm. The quadratic equation is $f(x) = 6x^2 + 5x + 3$.
14. Find the solution of the function $f(x) = \sin(7\pi x) + 10$ with the constraint $-3 < x < 3$ by using genetic algorithm and MATLAB programming.
15. Write a program to minimize "cosine" function.